

Programación Básica para Ingeniería en Sistemas de
Información:
De los Sistemas de Información al Código

Gleiston Guerrero Ulloa

Índice general

I Fundamentos Computacionales para Sistemas de Información	1
1. Sistemas de Información, Tecnología y Programación Aplicada	3
1.1. Ingeniería en Sistemas de Información y Programación Aplicada	5
1.1.1. La programación como competencia profesional en Sistemas de Información	6
1.1.2. Relación entre procesos organizacionales y software	9
1.2. Rol Profesional del Desarrollador en Sistemas de Información	12
1.2.1. Automatización, digitalización y transformación organizacional . . .	13
1.2.2. Responsabilidades y competencias técnicas en IS	15
1.3. Datos, Información y Conocimiento en Sistemas de Información	18
1.3.1. Fundamentos conceptuales	18
1.3.2. Ciclo de vida del dato en entornos organizacionales	21
2. Representación digital de la información	25
2.1. Fundamentos de representación binaria	26
2.1.1. Bits, bytes y almacenamiento	26
2.1.2. Sistemas de numeración	27
2.1.3. Conversión de valores enteros entre distintos sistemas de numeración	30
2.1.4. Conversión y representación de valores reales en distintos sistemas de numeración	46
2.1.5. Operaciones bit a bit, máscaras y estados	55
2.2. Representación de Información en Sistemas Computacionales	60
2.2.1. Representación de texto, números y contenidos multimedia	61
2.2.2. Riesgos de representación y prácticas de programación robusta . . .	73

3. Tecnología Computacional como Soporte del Sistema de Información	93
3.0.1. Componentes Tecnológicos	93
3.0.2. Modelo funcional de entrada–proceso–salida en SI	94
3.1. Formulación del problema informacional	95
3.1.1. Entradas del sistema	96
3.1.2. Salidas esperadas	97
3.1.3. Procesos	98
3.1.4. Restricciones organizacionales y técnicas	101
3.1.5. Del enunciado organizacional al modelo resoluble	102
4. Algoritmos y Modelado de Soluciones Informacionales	103
4.1. Proceso de resolución de problemas en Sistemas de Información	103
4.1.1. Análisis del problema	104
4.1.2. Diseño de la solución	114
4.1.3. Verificación y mejora continua	117
4.2. Algoritmos: fundamento para soluciones organizacionales	119
4.2.1. Definición y propósito en IS	119
4.2.2. Definición formal	119
4.2.3. Propósito de los algoritmos en IS	120
4.2.4. Los algoritmos y los SI	121
4.2.5. Propiedades esenciales	123
4.2.6. Correctitud y eficiencia	124
4.3. Lenguajes formales y lenguajes de programación	125
4.3.1. Sintaxis	125
4.3.2. Semántica	126
4.3.3. Errores comunes en principiantes	127
4.4. Buenas prácticas para aprender programación	128
4.4.1. Métodos de estudio	128
4.4.2. La importancia del error	130
4.4.3. Trabajo colaborativo	130
4.4.4. Herramientas de apoyo	130
4.4.5. Lenguajes de programación en Sistemas de Información	130
4.5. Representación en lenguaje natural estructurado	132
4.5.1. Pasos numerados	133
4.5.2. Precisión y claridad	134

4.5.3.	Evitar ambigüedades	134
4.6.	Pseudocódigo	134
4.6.1.	Elementos del lenguaje algorítmico (UPSAM 2.0)	134
4.6.2.	Convenciones de escritura	139
4.6.3.	Construcción de sentencias	140
4.7.	Diagramas de flujo	148
4.7.1.	Símbolos básicos	149
4.7.2.	Construcción de diagramas	158
4.8.	Resolver Problemas Mediante Algoritmos y/o Diagramas de Flujo	159
4.8.1.	Análisis del problema.	159
4.8.2.	Identificación de Datos de Entrada	160
4.8.3.	Identificación de Datos de Salida	160
4.8.4.	Identificación de Cálculos Intermedios (Procesos Intermedios)	161
4.8.5.	Identificación de Cálculos Finales (Procesos Finales)	161
4.8.6.	Identificación de Condiciones (Decisiones)	161
4.8.7.	Identificación de Repeticiones o Iteraciones	161
4.8.8.	Condiciones para Finalizar el Algoritmo	162
4.9.	Ejercicios Resueltos mediante Diagramas de Flujo	162
4.9.1.	Ejercicio 1: Problema a resolver	162
4.9.2.	Ejercicio 2: Problema a resolver	167
4.9.3.	Ejercicio 3: Problema a resolver	172
4.9.4.	Ejercicio 4: Cálculo de la Edad Exacta (Años, Meses y Días)	177
4.9.5.	Ejercicio 5: Determinación del Valor Mayor entre Tres Números	183
4.9.6.	Ejercicio 6: Cálculo del Máximo Común Divisor (MCD)	188
4.9.7.	Ejercicio 7: Cálculo del Mínimo Común Múltiplo (MCM)	193
4.9.8.	Ejercicio 8: Ordenamiento de un Vector de Valores	199
4.9.9.	Ejercicio 9: Cálculo de Medidas de Tendencia Central	207
4.9.10.	Ejercicio 10: Conversión de un Número Entero en base 10 a Otra Base Menor que 10	214
4.10.	Estrategias para el aprendizaje técnico en IS	221
4.10.1.	Práctica deliberada	221
4.10.2.	Estrategias cognitivas aplicadas	223

II Desarrollo de aplicaciones para Sistemas de Información 227

5. Lenguajes de Programación y Desarrollo de Aplicaciones 229

5.1. Clasificación general de los lenguajes de programación	230
5.1.1. Lenguajes de bajo nivel y lenguajes de alto nivel	230
5.1.2. Lenguajes de alto nivel	232
5.2. Paradigma imperativo	236
5.2.1. Idea central	236
5.2.2. Cómo abordar un problema en el paradigma imperativo	236
5.2.3. Lenguajes representativos del paradigma imperativo	237
5.2.4. Análisis del enfoque imperativo	240
5.3. Paradigma orientado a objetos	240
5.3.1. Idea central	240
5.3.2. Cómo abordar un problema en orientación a objetos	241
5.3.3. Lenguajes representativos del paradigma orientado a objetos	241
5.3.4. Análisis del enfoque orientado a objetos	246
5.4. Paradigma funcional	246
5.4.1. Idea central	247
5.4.2. Cómo abordar un problema en el paradigma funcional	247
5.4.3. Lenguajes representativos del paradigma funcional	247
5.4.4. Análisis del enfoque funcional	250
5.5. Paradigma declarativo	250
5.5.1. Idea central	251
5.5.2. Cómo abordar un problema en el paradigma declarativo	251
5.5.3. Lenguajes representativos del paradigma declarativo	251
5.5.4. Análisis del enfoque declarativo	253
5.6. Paradigmas y estrategias de resolución del mismo problema	253
5.6.1. Diferencia de enfoque	254
5.6.2. Buenas prácticas para aprender a resolver problemas desde diferentes paradigmas	254
5.6.3. Tabla comparativa de enfoques	254
5.7. Conclusión	255

6. Del Algoritmo a la Aplicación Organizacional 257

6.1. Estructura Mínima de un Programa	258
---	-----

6.1.1.	Entradas y Salidas	263
6.1.2.	Bloques y alcance	265
6.1.3.	Ejecución paso a paso	266
6.2.	Errores frecuentes en el desarrollo inicial	268
6.2.1.	Errores de sintaxis	270
6.2.2.	Errores lógicos	273
6.2.3.	Errores de ejecución	274
6.3.	Entorno de desarrollo profesional	275
6.3.1.	IDE y flujo de trabajo	275
6.3.2.	Estándares básicos de codificación	276
6.4.	Gestión de entrada y salida de datos	278
6.4.1.	Lectura y validación	278
6.4.2.	Presentación de resultados	285
7.	Modelado de Datos y Expresiones en Aplicaciones	287
7.1.	Tipos de datos en aplicaciones informacionales	288
7.1.1.	Datos numéricos y lógicos	288
7.1.2.	Texto y estructuras básicas	294
7.2.	Variables y gestión del estado	302
7.2.1.	Declaración e inicialización	303
7.3.	Operadores y construcción de expresiones	311
7.3.1.	Operadores aritméticos, relacionales y lógicos	312
7.3.2.	Conversión y validación de datos	318
8.	Ejercicios de Aplicación	321
8.1.	Implementación directa de los algoritmos en Java	323
8.1.1.	Promedio aritmético de N valores	323
8.1.2.	Raíces de una ecuación cuadrática	326
8.1.3.	Determinar si una palabra es palíndroma	332
8.1.4.	Cálculo de la edad exacta	336
8.1.5.	Cálculo del MCD mediante el Algoritmo de Euclides	344
8.1.6.	Cálculo del mínimo común múltiplo	349

III Control de Flujo y Gestión de Datos 353

9. Estructuras de Control en Aplicaciones Informacionales 355

9.1. Estructuras de selección	355
9.1.1. Selección simple y doble	356
9.1.2. Selección múltiple	360
9.2. Estructuras de repetición	362
9.2.1. Repetición condicional	363
9.2.2. Repetición por contador	365
9.3. Ejercicios integradores de estructuras de control	367
9.3.1. Ejercicio integrador: control de calificaciones en una aplicación in- formacional	367
9.3.2. Ejercicio integrador: análisis de rendimientos en una aplicación financiera	376

10. Estructuración y Organización de Datos 387

10.1. Arreglos unidimensionales	387
10.1.1. Concepto y uso	387
10.2. Arreglos bidimensionales	391
10.2.1. Matrices	391
10.3. Registros y datos compuestos	393
10.3.1. Estructuras simples	394
10.3.2. Ejercicio 1: Análisis de ventas mensuales	397
10.3.3. Ejercicio 2: Matriz de ingresos por sucursal	404
10.3.4. Ejercicio 3: Registro de clientes	412
10.3.5. Ejercicio 4: Implementación en Java del Algoritmo 4.9.9	423

IV Modularidad, Calidad e Integración en Sistemas de Información 429

11. Modularidad y Funciones en Aplicaciones 431

11.1. Abstracción y descomposición funcional	432
11.1.1. Diseño modular	432
11.2. Funciones y procedimientos	432
11.2.1. Definición e invocación	432

11.3. Pruebas y calidad básica	432
11.3.1. Estrategia de pruebas	432
V Material Complementario	433
12. Proyecto Integrador en Sistemas de Información	435
12.1. Planteamiento organizacional	435
12.1.1. Contexto y alcance	435
12.2. Diseño e implementación	435
12.2.1. Estructura del sistema	435
13. Gestión básica de versiones en entornos colaborativos	437
13.1. Fundamentos	437
13.1.1. Repositorio y control de cambios	437
14. Glosario y Recursos Académicos	439
14.1. Términos fundamentales	439
14.2. Bibliografía y fuentes de consulta	439
A. Operaciones bit a bit, máscaras y estados	441
A.1. Operador OR bit a bit 	441
A.1.1. Ejemplo 1: OR a nivel de bit (8 bits)	441
A.1.2. Ejemplo 2: OR a nivel de palabra (32 bits en C)	441
A.2. Operador OR lógico 	442
A.2.1. Ejemplo 1: Evaluación lógica con cortocircuito	442
A.2.2. Ejemplo 2: Diferencia práctica entre y 	443
A.3. Operador AND &	443
A.4. Operador XOR ^	444
A.5. Desplazamientos	444
A.5.1. Desplazamiento a la izquierda <<	444
A.5.2. Desplazamiento a la derecha >>	444
A.6. Resumen comparativo	445
B. Codificación de caracteres: de ASCII a Unicode	447
B.0.1. ASCII (7 bits): repertorio base 0–127	447

Prefacio

Parte I

Fundamentos Computacionales para Sistemas de Información

Capítulo 1

Sistemas de Información, Tecnología y Programación Aplicada

La formación en Ingeniería en Sistemas de Información exige una comprensión articulada entre estructuras organizacionales y mecanismos computacionales. La programación constituye el medio formal mediante el cual las necesidades operativas se traducen en soluciones ejecutables, integrando procesos, datos y reglas de negocio dentro de arquitecturas tecnológicas coherentes. La literatura especializada sostiene que el desarrollo de aplicaciones fortalece la capacidad de análisis estructurado y modelización de problemas empresariales [1], [2]. El dominio de fundamentos computacionales permite diseñar sistemas alineados con dinámicas administrativas reales y con entornos tecnológicos sujetos a evolución constante.

La disciplina integra personas, procesos y tecnología bajo un marco que combina análisis organizacional con implementación técnica. El software opera como mecanismo de materialización de políticas institucionales y controles operativos [3]. Desde la ingeniería de software se establecen criterios de trazabilidad, consistencia y verificabilidad que inciden directamente en la calidad del sistema construido [4]. El dominio de la programación habilita la construcción de aplicaciones capaces de gestionar inventarios, nóminas, transacciones financieras o registros académicos, ampliando el horizonte profesional del ingeniero y fortaleciendo su capacidad de intervención técnica en organizaciones de diversa escala.

La actividad del desarrollador trasciende la mera codificación; involucra modelado estructural, definición de reglas formales y validación sistemática de resultados. La automatización administrativa demanda control preciso de estructuras condicionales, ciclos y manejo de datos [5]. Los estándares técnicos sobre prácticas seguras de programación proporcionan lineamientos que reducen defectos y aumentan la confiabilidad del software [6].

La práctica sostenida consolida disciplina analítica, rigor metodológico y responsabilidad profesional.

Los sistemas de información transforman datos en insumos útiles para la gestión. La captura, almacenamiento y procesamiento algorítmico de registros digitales constituye la base operativa de dichos sistemas [7]. La experiencia directa con estructuras de control y funciones permite observar cómo los datos adquieren significado operativo cuando son sometidos a transformaciones formales. Estudios sobre ecosistemas informacionales muestran que la organización estructurada de datos incrementa la capacidad analítica institucional [8]. La programación habilita participación activa en la construcción de estos mecanismos informacionales.

La infraestructura computacional sostiene la ejecución de aplicaciones mediante la interacción coordinada entre procesadores, memoria y sistemas operativos [9]. La gestión de procesos y recursos, descrita en la teoría de sistemas operativos, determina el comportamiento observable del software en ejecución [10]. Conocer estos fundamentos permite escribir código que utilice racionalmente los recursos disponibles y que opere de manera estable en plataformas heterogéneas. La formación técnica proporciona criterio para evaluar rendimiento, eficiencia y adecuación arquitectónica.

La representación digital introduce al estudiante en la estructura formal de los datos en memoria. El estándar IEEE 754 define la organización de números en punto flotante mediante campos de signo, exponente y fracción [11]. El análisis clásico de errores de redondeo evidencia cómo pequeñas variaciones en la representación binaria influyen en cálculos posteriores [12]. Este conocimiento favorece decisiones técnicas informadas al diseñar modelos financieros, estadísticos o administrativos que dependen de precisión numérica.

La formulación rigurosa del problema computacional constituye la base del diseño de soluciones informacionales. La identificación sistemática de entradas, salidas y restricciones permite estructurar modelos verificables [13]. El diseño algorítmico proporciona criterios formales de finitud, precisión y corrección, garantizando coherencia interna en la solución implementada. La práctica continua en este ámbito fortalece el razonamiento estructurado y la capacidad de afrontar desafíos organizacionales mediante procedimientos definidos.

El modelo curricular Leidig2021Competency plantea que el profesional en Sistemas de Información debe integrar análisis organizacional con implementación técnica en plataformas reales [14], [15]. La programación actúa como puente entre teoría administrativa y ejecución tecnológica, permitiendo materializar procesos empresariales en sistemas operativos concretos. El dominio de estas competencias amplía oportunidades laborales y

consolida una formación integral orientada a la transformación digital organizacional.

1.1. Ingeniería en Sistemas de Información y Programación Aplicada

La Ingeniería en Sistemas de Información (IS) se orienta a comprender, diseñar e implementar soluciones socio-técnicas en las que personas, procesos, datos y reglas organizacionales se integran mediante sistemas de software. En este ámbito, el software no se concibe como un producto aislado, sino como un mecanismo de coordinación y control que operacionaliza políticas, procedimientos y restricciones bajo condiciones reales de uso (tiempos, recursos, variabilidad y excepciones). Por ello, la programación adquiere relevancia formativa y profesional: permite convertir descripciones organizacionales en comportamientos ejecutables, verificables y mantenibles sobre plataformas tecnológicas concretas [1], [2], [16].

En un curso de programación básica para IS, el objetivo no consiste únicamente en “aprender un lenguaje”, sino en adquirir una capacidad transferible: representar problemas con precisión, diseñar soluciones paso a paso y expresar dichas soluciones como algoritmos que un computador pueda ejecutar sin ambigüedades. Esta capacidad se relaciona con el pensamiento computacional, que enfatiza abstracción, descomposición y formulación sistemática de procedimientos, habilidades útiles para contextos organizacionales donde los problemas suelen estar incompletos o descritos informalmente [17], [18], [19].

Desde una perspectiva curricular basada en competencias, la programación se valida por su utilidad para intervenir en tareas reales de análisis, construcción e implementación de soluciones que atienden necesidades de actores y unidades organizacionales. En consecuencia, aprender a programar en IS implica articular conceptos (algoritmos, datos, control) con prácticas elementales de ingeniería (especificar, probar, documentar y mejorar), de modo que el estudiante avance desde ejercicios controlados hacia la resolución de problemas con significado organizacional [2], [14], [15].

En este capítulo, la programación se presenta como una herramienta que permite: (i) formalizar procesos organizacionales mediante reglas explícitas; (ii) automatizar operaciones repetitivas con criterios de calidad; y (iii) producir resultados consistentes que sostienen decisiones. El desarrollo se mantiene deliberadamente en un nivel conceptual y de aprendizaje, reservando para secciones posteriores el tratamiento detallado de transformación digital, responsabilidades profesionales y ciclo de vida del dato, con el fin de evitar redundancias y preservar la progresión pedagógica [1], [2], [16].

1.1.1. La programación como competencia profesional en Sistemas de Información

En IS, la programación se reconoce como una competencia transversal porque conecta el análisis del entorno organizacional con la implementación concreta de mecanismos de información, control y coordinación. Esta conexión no es abstracta: en la práctica profesional, decisiones operativas y administrativas quedan codificadas en reglas de validación, cálculos, flujos de trabajo y restricciones que el software ejecuta de manera consistente para múltiples usuarios y situaciones [1], [2], [16].

Para el estudiante que aprende a programar, esta transversalidad se traduce en una idea guía: un programa es una especificación ejecutable de un procedimiento. En términos formativos, ello exige dominar nociones fundamentales (variables, tipos, expresiones, estructuras de control, modularidad) y, al mismo tiempo, aprender a justificar la elección de una estrategia algorítmica frente a un requerimiento. Dicho aprendizaje se apoya en la disciplina algorítmica: claridad, finitud, corrección y trazabilidad entre el problema y la solución [5], [7], [13].

Además, la programación fortalece habilidades de abstracción y descomposición aplicables a dominios no exclusivamente informáticos. La capacidad de separar un problema en subproblemas, definir entradas y salidas, y construir soluciones incrementales constituye un fundamento metodológico para diseñar soluciones organizacionales robustas, incluso cuando los requisitos iniciales son ambiguos o cambian con el contexto [17], [18], [19].

Desde la perspectiva de Leidig2021Competency, el dominio técnico adquiere valor cuando se integra con comprensión organizacional, comunicación y pensamiento crítico para producir soluciones utilizables. En consecuencia, aprender a programar debe orientarse a producir artefactos comprensibles, verificables y evolucionables, no únicamente a “hacer que el código funcione” [14], [15], [16].

Programación como medio de formalización de procesos organizacionales

Los procesos organizacionales suelen describirse mediante lenguaje natural, documentos operativos o acuerdos tácitos. Sin embargo, para ser automatizables y auditables requieren una traducción a reglas explícitas, condiciones y secuencias de acciones que un sistema pueda ejecutar de forma consistente. La programación cumple aquí una función de formalización: convierte decisiones, excepciones, validaciones y dependencias en estructuras algorítmicas precisas que reducen ambigüedades [5], [7], [13].

En clave pedagógica, formalizar significa aprender a “cerrar” lo que el lenguaje natural deja abierto. Por ejemplo, una frase organizacional como “registrar la solicitud si está completa” obliga a precisar qué se entiende por completitud (campos obligatorios, formatos válidos, rangos aceptables) y qué ocurre si la condición falla (rechazo, corrección, notificación). Estos aspectos se expresan mediante condicionales, validaciones de entrada y manejo de casos límite, elementos básicos que estructuran el razonamiento del programador principiante [5], [6], [7].

La formalización también exige representar con precisión los estados del proceso y sus transiciones. Aunque el modelamiento de procesos se tratará de manera más extensa en otras secciones, aquí basta destacar que la programación permite materializar estados como valores y condiciones (por ejemplo, “pendiente”, “aprobado”, “rechazado”), y transiciones como reglas que dependen de eventos y verificaciones. Esta correspondencia entre proceso y control de flujo introduce al estudiante en el diseño de soluciones que no son lineales, sino gobernadas por decisiones y excepciones [1], [13], [16].

La formalización mediante programación aporta un lenguaje común entre lo organizacional y lo técnico. Modelos de análisis y diseño (por ejemplo, orientados a objetos o basados en UML) describen estructuras y comportamientos; la programación convierte dichas descripciones en componentes ejecutables en los que cada operación implementada representa una regla o decisión del proceso. La trazabilidad conceptual entre modelo e implementación se vuelve, así, una disciplina temprana que el estudiante debe aprender a cultivar desde ejercicios básicos [4], [16], [20].

Programación como instrumento de automatización operativa

La automatización operativa delega en procedimientos computacionales aquellas actividades repetitivas, intensivas en registro o propensas a error humano, con el fin de lograr ejecución uniforme y medible. En IS, esta automatización se expresa mediante programas que capturan datos, validan transacciones, aplican reglas y producen salidas operativas (cálculos, estados, comprobaciones, reportes), integrándose con los sistemas de información que sostienen la operación organizacional [1], [2], [3].

Desde el aprendizaje de programación, automatizar no equivale a “mecanizar” un procedimiento sin comprenderlo. Automatizar exige identificar entradas, decidir el orden de los pasos, definir condiciones de aceptación y establecer salidas verificables. En términos de construcción algorítmica, ello se traduce en seleccionar estructuras de control adecuadas (secuencia, selección, iteración), diseñar funciones para dividir responsabilidades y aplicar

pruebas simples para confirmar que el programa reproduce el procedimiento esperado [5], [7], [13].

La automatización también introduce un criterio didáctico esencial: la corrección debe sostenerse en la diversidad de casos, no solo en el “caso típico”. Por ello, el estudiante debe aprender a anticipar excepciones (datos incompletos, formatos inválidos, valores fuera de rango) y a diseñar validaciones y mensajes de error que hagan explícito el contrato del programa. Estos aspectos se relacionan con prácticas de programación responsable y con lineamientos que buscan reducir fallas recurrentes asociadas a suposiciones inseguras o entradas no controladas [4], [6], [7].

Automatizar con impacto organizacional requiere considerar calidad y mantenibilidad. La ingeniería de software destaca propiedades como corrección, claridad, modularidad y adecuación al propósito, que se vuelven críticas cuando un proceso depende del software para operar. En consecuencia, aprender a programar debe incluir hábitos tempranos de estructura y legibilidad (nombres, modularización, comentarios mínimos justificados), ya que estos hábitos condicionan la sostenibilidad del sistema más allá del primer funcionamiento [1], [4], [21].

Programación como soporte informacional para la toma de decisiones

La toma de decisiones organizacionales depende de transformar datos en información contextualizada y, posteriormente, en conocimiento accionable. Los sistemas de información sostienen esa transformación mediante mecanismos de captura, almacenamiento, procesamiento y presentación, donde los programas implementan operaciones que filtran, validan, agregan y derivan resultados que funcionan como evidencia para decidir [2], [8], [22].

En términos formativos, este rol informacional permite introducir al estudiante en tareas de programación con sentido: calcular totales, promedios, frecuencias, máximos y mínimos; clasificar registros; detectar inconsistencias; o generar resúmenes que hagan visible el estado de una operación. Estas tareas, aunque básicas, enseñan a pensar en términos de datos, reglas y resultados verificables, y facilitan comprender por qué la precisión en tipos, operaciones y validaciones afecta directamente la confiabilidad de la información producida [5], [7], [22].

El soporte informacional exige consistencia: decisiones distintas no deben originarse en cálculos inconsistentes o reglas implícitas. Por ello, la programación debe explicitar supuestos (por ejemplo, unidades, rangos, criterios de inclusión), y establecer controles que permitan reproducibilidad del procesamiento. Esta exigencia se conecta con la disciplina

algorítmica (precisión y finitud) y con criterios de calidad del software (comportamiento estable y verificable), aspectos que deben introducirse desde el nivel básico para evitar que el estudiante normalice resultados “aproximados” sin justificación [4], [13], [21].

Además, la confiabilidad informacional se asocia a la correcta representación de valores numéricos, especialmente cuando se realizan cálculos acumulativos o comparaciones. En programación, comprender límites de representación y el manejo apropiado de tipos numéricos fortalece la integridad de los resultados y reduce errores sutiles que afectan reportes e indicadores. Aunque los detalles de representación digital se abordarán posteriormente, aquí se subraya su impacto directo en la calidad del soporte informacional [7], [11], [12].

1.1.2. Relación entre procesos organizacionales y software

Un sistema de información puede entenderse como la materialización computacional de un conjunto de procesos organizacionales que gestionan recursos, eventos, registros y comunicaciones. En consecuencia, la calidad del software depende del entendimiento del proceso que modela y de la coherencia entre el procedimiento operativo y la lógica implementada, ya que el programa se convierte en un “ejecutor” de reglas organizacionales en escenarios reales [1], [2], [16].

Esta relación es bidireccional. Por un lado, los procesos condicionan requisitos y reglas del software; por otro, el software introduce capacidades de trazabilidad, validación sistemática y generación de evidencia que reconfiguran la forma en que el proceso se ejecuta y se controla. Para quien aprende a programar, esta bidireccionalidad se traduce en comprender que cada decisión de diseño (estructuras de datos, validaciones, orden de pasos) produce consecuencias operativas: habilita o restringe acciones, reduce o incrementa errores, y determina la calidad de la información disponible [2], [3], [4].

En el plano pedagógico, esta subsección cumple un propósito específico: ayudar a que el estudiante no programe “en el vacío”. Incluso en ejercicios introductorios, se busca que identifique qué parte del proceso representa el programa (qué se recibe, qué se decide, qué se produce) y que evalúe la solución por criterios de corrección y claridad, no solo por ejecución. Con ello, la programación se integra como una práctica de análisis aplicado, coherente con marcos de análisis y diseño de sistemas [1], [5], [16].

Proceso, procedimiento y sistema en entornos organizacionales

En IS resulta útil distinguir: (i) *proceso* como conjunto de actividades coordinadas orientadas a un resultado organizacional; (ii) *procedimiento* como la forma prescrita o normativa

de ejecutar parte del proceso; y (iii) *sistema* como el conjunto socio-técnico que integra personas, reglas y tecnología para ejecutar y controlar dichos procesos. Estas distinciones son relevantes porque el software no implementa “la organización” en abstracto, sino comportamientos concretos asociados a procedimientos y reglas dentro de un proceso delimitado [1], [2], [3].

Para aprender a programar, estas distinciones ofrecen una guía práctica: un programa suele representar un procedimiento (una forma específica de hacer) dentro de un proceso mayor. Por ello, antes de escribir código, el estudiante debe aprender a identificar el alcance: qué parte del proceso será automatizada, qué decisiones quedan dentro del programa y cuáles permanecen bajo control humano. Esta delimitación reduce errores de diseño típicos en principiantes, como querer resolver “todo el proceso” con una sola rutina o ignorar casos excepcionales relevantes [1], [5], [7].

La precisión conceptual evita errores de alcance en proyectos de software. Cuando no se distingue proceso de procedimiento, se tiende a sobredimensionar funciones del sistema o a automatizar actividades que requieren juicio humano o validación externa. Por ello, marcos de análisis y diseño recomiendan delimitar responsabilidades, entradas y salidas por función, y especificar la interacción de usuarios y componentes con el proceso organizacional [1], [16], [20].

En consecuencia, una práctica formativa adecuada consiste en traducir un procedimiento a una especificación mínima: entradas, reglas y salidas. Esta especificación se convierte en una estructura de programa con control de flujo explícito y validaciones claras, lo que refuerza la disciplina algorítmica y disminuye suposiciones implícitas sobre la dinámica organizacional [4], [6], [13].

Modelamiento de procesos y necesidades informacionales

El modelamiento constituye el puente entre la descripción de un proceso y su instrumentación computacional. En IS, modelar significa representar actividades, decisiones, actores, datos y reglas, con el fin de identificar qué información debe capturarse, en qué momento, bajo qué validaciones y con qué propósito operativo o gerencial. Esta representación no es un formalismo ornamental: orienta el diseño del programa, reduce ambigüedades y establece criterios para evaluar la solución [1], [2], [16].

En este sentido, la necesidad informacional no se reduce a “guardar datos”, sino a definir semántica y utilidad: qué atributos representan un evento, qué consistencia deben mantener, y cómo habilitan operaciones de control o seguimiento. Para el aprendizaje de

programación, ello implica entrenar al estudiante en la elección de estructuras de datos coherentes con el significado de la información y en la implementación de validaciones que preserven la integridad del registro [2], [8], [22].

El modelamiento también orienta decisiones de diseño: modularidad, responsabilidades y operaciones. En enfoques orientados a objetos, por ejemplo, el modelo facilita la identificación de entidades, relaciones y comportamientos que luego se traducen a clases y métodos; incluso cuando se programa de manera introductoria, esta perspectiva ayuda a organizar el código en partes comprensibles y reutilizables, evitando soluciones monolíticas difíciles de mantener [4], [16], [20].

Como consecuencia, el estudiante aprende que “programar” incluye un trabajo previo de clarificación: construir una representación mínima del proceso y de sus datos, a partir de la cual se derivan reglas y casos de prueba elementales. Esta práctica refuerza el vínculo entre lo que el proceso requiere y lo que el programa ejecuta, con criterios verificables de corrección [1], [5], [7].

De la necesidad organizacional a la solución computacional

La transición desde una necesidad organizacional hacia una solución computacional exige transformar objetivos (reducir errores, mejorar control, acelerar registro, producir reportes) en especificaciones operables: funciones, reglas y restricciones que puedan implementarse y verificarse. En análisis y diseño de sistemas, esta transición se estructura mediante identificación de requerimientos, definición de alcance y elaboración de modelos que conectan actores, procesos y datos con funcionalidades del software [1], [2], [16].

Desde la perspectiva del aprendizaje, esta transición se traduce en una secuencia disciplinada: (i) comprender el enunciado y sus restricciones; (ii) definir entradas y salidas con precisión; (iii) descomponer en subproblemas; (iv) diseñar un algoritmo; y (v) implementar de forma incremental con verificación continua. Este enfoque evita que el estudiante confunda la escritura de código con la solución del problema, y promueve que la implementación sea una consecuencia de un diseño algorítmico explícito [5], [7], [13].

La programación cierra el ciclo al convertir dichas especificaciones en algoritmos y estructuras de datos ejecutables, asegurando que el comportamiento real del sistema corresponda al comportamiento requerido. En términos formativos, esto implica habituarse a la verificación: comprobar resultados, probar escenarios alternativos y revisar supuestos. De este modo, el estudiante desarrolla un criterio profesional temprano: el programa es correcto cuando satisface el problema bajo condiciones relevantes, no cuando solo produce

una salida “razonable” [4], [13], [21].

Conviene enfatizar que, en esta sección, la relación se plantea a nivel conceptual y de trazabilidad: necesidad $\rightarrow$ modelo $\rightarrow$ implementación. Los métodos detallados para formular entradas, salidas, restricciones y criterios de aceptación, así como su articulación con otras dimensiones organizacionales, se desarrollan en secciones posteriores del capítulo. Esta decisión editorial evita redundancias y sostiene una progresión pedagógica que acompaña el aprendizaje de programación desde fundamentos hacia aplicaciones más integrales [1], [15], [16].

1.2. Rol Profesional del Desarrollador en Sistemas de Información

El rol profesional del desarrollador en Sistemas de Información (IS) se define por la capacidad de traducir necesidades organizacionales en soluciones de software que operan bajo restricciones reales: tiempo, presupuesto, normativas, interacción humana y evolución de procesos. Esta labor exige convertir descripciones funcionales en comportamientos ejecutables, con reglas explícitas, datos bien definidos y resultados verificables, de modo que el sistema sostenga operaciones y decisiones sin degradar la coherencia del proceso organizacional [1], [2], [16].

Dentro de este panorama, aprender a programar constituye una decisión formativa orientada a la empleabilidad y al desempeño profesional. La programación habilita al estudiante a pasar de la intención a la implementación: formular un algoritmo, elegir estructuras de datos, controlar la ejecución y validar resultados mediante pruebas. Esta progresión convierte el aprendizaje en una práctica acumulativa, donde cada concepto (secuencia, selección, iteración, modularidad) aporta un recurso concreto para construir soluciones aplicables en distintos contextos organizacionales [5], [7], [13].

La práctica profesional no se limita a producir código funcional; incluye asegurar calidad, mantenibilidad y seguridad desde etapas tempranas. Por esa razón, el desarrollo de competencias técnicas incorpora hábitos de diseño, lectura de requisitos, documentación mínima útil y revisión sistemática de defectos frecuentes. Este marco garantista se vincula con la ingeniería de software, donde la corrección técnica se evalúa junto con claridad, evolución y adecuación al propósito [4], [23], [24].

1.2.1. Automatización, digitalización y transformación organizacional

La automatización y la digitalización representan dos líneas de intervención que suelen confluir en iniciativas de transformación digital. En IS, el desarrollador participa en estas iniciativas implementando mecanismos que capturan información, normalizan registros, ejecutan reglas y coordinan actividades, buscando que el sistema reduzca fricción operativa y produzca evidencia útil para control y mejora [2], [16], [25].

El aprendizaje de programación contribuye directamente a este tipo de intervención. Un estudiante que domina estructuras de control y modelado básico de datos puede implementar validaciones, automatizar cálculos, generar reportes y construir prototipos que hagan visibles fallas del proceso o inconsistencias del registro. Esa experiencia temprana favorece un enfoque riguroso: cada decisión de diseño se expresa como una regla ejecutable y cada regla se verifica con casos de prueba y entradas variadas [5], [6], [7].

Paralelamente, la transformación organizacional demanda sensibilidad socio-técnica. El software modifica rutinas, tiempos y responsabilidades; por ello, el desarrollador debe anticipar efectos sobre usuarios y sobre la interacción con el sistema. Aun en programación básica, resulta pertinente introducir ejercicios que consideren uso real: mensajes de error informativos, flujos de entrada consistentes y salidas interpretables por usuarios no técnicos [1], [2], [26].

Digitalización de información y procesos

La digitalización consiste en representar información y actividades de manera que puedan almacenarse, consultarse y procesarse mediante sistemas computacionales. En organizaciones, este paso implica identificar qué datos describen un evento, qué atributos garantizan trazabilidad y qué reglas preservan integridad, evitando registros incompletos o ambiguos que comprometan la operación [2], [22], [27].

Para quien aprende a programar, la digitalización se materializa en decisiones concretas: definir tipos de datos, estructurar registros, validar entradas y diseñar formatos de salida. Un formulario, un archivo de datos o una estructura en memoria no son detalles secundarios; constituyen la base del comportamiento posterior del sistema. Por ello, el entrenamiento debe incluir prácticas de validación (rangos, formatos, obligatoriedad) y de coherencia semántica (nombres consistentes, unidades, estados), destacando que una buena representación reduce errores aguas abajo [5], [6], [7].

De manera complementaria, la digitalización se conecta con principios de gestión de

datos en sistemas de información. Modelos de datos y conceptos de bases de datos ayudan a evitar duplicidad, inconsistencias y pérdidas de significado al pasar de registros aislados a estructuras persistentes. Sin profundizar en administración de bases (tema desarrollado en otras unidades), conviene que el estudiante perciba la relación entre “cómo se programa” y “qué calidad tienen los datos” [27], [28], [29].

Automatización de operaciones y flujos de trabajo

La automatización se orienta a ejecutar tareas repetitivas con uniformidad, control y medición, integrando reglas operativas en procedimientos computacionales. En IS, esta automatización se implementa mediante programas que registran transacciones, calculan resultados, aplican restricciones y coordinan secuencias de trabajo. En organizaciones con alta carga operativa, pequeños incrementos de consistencia y velocidad producen mejoras acumulativas en calidad del servicio y trazabilidad [1], [2], [3].

El aprendizaje de programación ofrece un camino pedagógico natural para esta automatización. Primero se ejercita la especificación de pasos; luego se introduce la selección condicional para manejar decisiones; posteriormente se incorporan bucles para procesar conjuntos de registros; después se emplea modularidad para separar responsabilidades y facilitar pruebas. Esta ruta promueve un estilo de pensamiento donde el estudiante aprende a transformar un procedimiento narrado en un algoritmo que el computador ejecuta sin interpretaciones implícitas [5], [7], [13].

No obstante, la automatización requiere atención a fallos frecuentes: entradas inválidas, supuestos no verificados y manejo insuficiente de excepciones. La calidad operativa depende de validar datos, registrar errores de forma útil y conservar invariantes del proceso. Catálogos y lineamientos de codificación ayudan a identificar riesgos habituales y favorecen prácticas tempranas de programación responsable, incluso en ejercicios introductorios [4], [6], [24].

En distintos contextos organizacionales, la automatización también se implementa como orquestación de tareas y flujos. Sistemas de gestión de procesos (BPMS) y enfoques afines muestran que las reglas de negocio y los pasos de un flujo pueden representarse, monitorearse y refinarse. Para el estudiante, estas ideas se traducen en diseñar programas que hagan explícitas transiciones, estados y condiciones, preparando el terreno para trabajos más avanzados sin anticipar contenidos de unidades posteriores [1], [16], [30].

Transformación digital orientada al valor organizacional

La transformación digital orientada al valor se vincula con cambios en la forma de operar y decidir, apoyados por software, datos y rediseño de procesos. Su propósito no se reduce a informatizar tareas existentes; introduce nuevas capacidades de coordinación, medición y mejora continua, alineadas con objetivos de desempeño, servicio y sostenibilidad organizacional [25], [31], [32].

Desde la formación en programación, la conexión con el valor se enseña mediante problemas bien planteados: definir qué variable representa un indicador, qué regla determina una condición de alerta, qué estructura permite consolidar registros, y cómo se verifica que el resultado refleje la intención del proceso. La finalidad de ilustrar esta relación consiste en que el lector perciba que cada estructura de control y cada diseño de datos tienen impacto directo sobre la utilidad del sistema [2], [5], [7].

En contraste con una visión centrada en herramientas, el aprendizaje orientado al valor prioriza principios: consistencia, verificabilidad, trazabilidad y mantenibilidad. Un programa con resultados correctos pero difícil de leer o modificar impone costos de cambio; un programa legible y modular facilita adaptación ante cambios de política o de proceso. Esta relación entre diseño y evolución se encuentra documentada en fundamentos de ingeniería de software, con énfasis en calidad y ciclo de vida [4], [23], [24].

1.2.2. Responsabilidades y competencias técnicas en IS

El desarrollador en IS asume responsabilidades técnicas que exceden la escritura de código: interpretar requisitos, estructurar soluciones, verificar corrección y comunicar decisiones de diseño. Estas responsabilidades se ejercen con mayor solidez cuando el aprendizaje de programación se acompaña de hábitos de razonamiento: formular supuestos, delimitar alcance, definir contratos de entrada/salida y construir pruebas que respalden el resultado [1], [4], [16].

El modelo de competencias Leidig2021Competency destaca la integración entre capacidades técnicas, pensamiento analítico y comunicación profesional. Por ello, el aprendizaje de programación debe presentarse como una competencia de articulación: convierte una necesidad en un algoritmo, un algoritmo en una implementación, y una implementación en un componente operable dentro de un sistema socio-técnico [2], [14], [15].

Asimismo, la responsabilidad profesional incorpora prácticas de calidad que deben iniciarse desde programación básica. La estructura del código, la claridad de nombres, el uso de funciones y la revisión de errores comunes son prácticas que evitan deuda técnica

temprana y preparan al estudiante para proyectos colaborativos. La ingeniería de software provee criterios y técnicas para sostener estas prácticas en el tiempo [23], [24], [33].

Pensamiento lógico y abstracción aplicada

El pensamiento lógico aplicado se expresa en la habilidad de construir una solución paso a paso y justificar cada transición del algoritmo. En programación básica, esta habilidad se cultiva al formular procedimientos precisos, identificar condiciones y diseñar estructuras de control que representen decisiones del problema. La disciplina algorítmica proporciona un marco para evaluar si un procedimiento es correcto, finito y ejecutable [5], [7], [13].

La abstracción aplicada consiste en seleccionar lo relevante del problema y representarlo con estructuras mínimas: variables, colecciones, funciones y tipos. En IS, esta selección tiene una dimensión pragmática: se abstrae con el objetivo de operar sobre datos y reglas organizacionales sin perder significado. La formación debe favorecer ejercicios que obliguen a decidir qué información se guarda, qué se calcula, qué se valida y qué se omite por carecer de utilidad operacional [2], [18], [22].

El aprendizaje mejora cuando se incorpora una práctica incremental: comenzar con una versión simple que funcione, ampliar para cubrir casos adicionales y refactorizar para conservar claridad. Este patrón enseña al lector a programar con control del crecimiento del código, evitando soluciones extensas y frágiles. En ingeniería de software, esta idea se relaciona con modularidad y evolución de sistemas, aportando criterios para decidir cuándo dividir, renombrar o reorganizar [4], [24], [33].

Calidad, mantenibilidad y responsabilidad profesional

La calidad del software se manifiesta en corrección, legibilidad, seguridad y adaptabilidad. En IS, estas propiedades impactan operaciones y decisiones, ya que el sistema ejecuta reglas de negocio, valida transacciones y produce información para seguimiento. La responsabilidad profesional del desarrollador incluye prevenir defectos previsibles, documentar supuestos y asegurar que el sistema responda de manera consistente ante entradas diversas [4], [6], [23].

La mantenibilidad depende de estructuras comprensibles por terceros y por el propio autor en el tiempo. Para el lector que aprende a programar, esta idea se traduce en hábitos observables: nombres informativos, funciones con una responsabilidad definida, manejo explícito de errores y reducción de duplicación. Buenas prácticas de diseño y limpieza de código favorecen el aprendizaje porque convierten el programa en un objeto legible que puede discutirse, corregirse y mejorar con criterios explícitos [7], [24], [33].

La responsabilidad también incluye atención a fallas vinculadas a seguridad y robustez, incluso en programas pequeños. Validar entradas, controlar rangos y manejar condiciones inesperadas previene comportamientos indeseados y errores silenciosos que contaminan datos o producen resultados equivocados. Lineamientos de codificación y estándares de ingeniería proporcionan referencias para evitar patrones inseguros y para estructurar verificaciones de forma sistemática [4], [6], [23].

En síntesis, aprender a programar con orientación a IS implica adoptar una ética técnica: producir código que funcione, que pueda evolucionar y que proteja el significado de los datos y reglas que implementa. Esta ética se consolida cuando el estudiante aprende a justificar decisiones de diseño y a evaluar consecuencias, articulando práctica algorítmica con calidad de ingeniería [4], [13], [24].

Colaboración interdisciplinaria y documentación técnica

Los proyectos de IS requieren colaboración con actores de negocio, usuarios finales, analistas, administradores de datos y responsables de infraestructura. El desarrollador debe comunicar decisiones, restricciones y supuestos en un lenguaje accesible para perfiles no técnicos, manteniendo precisión suficiente para evitar interpretaciones erróneas. Por ello, la documentación técnica se consolida como un componente profesional del desarrollo, asociado a trazabilidad y a coordinación del trabajo [1], [2], [16].

Para quien aprende a programar, la colaboración se entrena mediante prácticas concretas: describir el algoritmo antes del código, comentar decisiones no obvias, registrar precondiciones y postcondiciones de funciones, y producir ejemplos de entrada/salida que sirvan como contrato. Estas prácticas fortalecen la capacidad de revisión y facilitan que otras personas validen el programa, reproduzcan resultados y detecten defectos de forma temprana [5], [7], [23].

Destacando la dimensión socio-técnica, la interacción con usuarios exige considerar usabilidad básica: mensajes, consistencia de opciones, prevención de errores y retroalimentación en tiempo de ejecución. En ejercicios introductorios, esta orientación puede incorporarse mediante requisitos simples de interacción y validación, promoviendo que el estudiante produzca programas que apoyen al usuario y reduzcan fricción operativa [1], [2], [26].

De manera complementaria, la documentación se apoya en representaciones estructuradas que conectan análisis y construcción. Modelos orientados a objetos y notaciones de diseño ofrecen un vocabulario compartido para describir entidades, responsabilidades

y relaciones, ayudando a que el código conserve coherencia con el diseño. Esta práctica prepara al estudiante para proyectos de mayor envergadura, donde la coordinación depende de artefactos que complementan el código [4], [16], [20].

1.3. Datos, Información y Conocimiento en Sistemas de Información

En Sistemas de Información (IS), la distinción entre datos, información y conocimiento estructura la forma en que las organizaciones operan, controlan procesos y orientan decisiones. Los sistemas computacionales gestionan registros, ejecutan transformaciones y producen resultados interpretables; sin embargo, esta secuencia solo adquiere sentido cuando el desarrollador comprende qué representa cada nivel y cómo se articulan entre sí [2], [16], [22]. Para quien aprende a programar, esta tríada constituye una guía conceptual que conecta estructuras de datos, algoritmos y salidas con impactos organizacionales concretos.

Dentro de este panorama, programar implica asumir responsabilidad sobre la calidad de los datos que ingresan, la coherencia de los procesos que los transforman y la utilidad de los resultados que se generan. Cada variable declarada, cada validación implementada y cada cálculo realizado influyen en la transición de dato a información y de información a conocimiento operativo. Esta relación se ha consolidado como fundamento en IS, donde la arquitectura de sistemas y el modelado de procesos se articulan con gestión de datos y análisis organizacional [1], [2], [27].

Aprender programación desde esta perspectiva transforma ejercicios técnicos en prácticas con sentido. El estudiante deja de manipular números o cadenas aisladas y comienza a interpretar registros, estados y eventos que describen actividades organizacionales. Esta orientación promueve un enfoque riguroso: cada decisión de diseño debe justificarse por su aporte a la coherencia informacional y a la trazabilidad del proceso [5], [7], [13].

1.3.1. Fundamentos conceptuales

La diferenciación conceptual entre dato, información y conocimiento organiza la reflexión sobre cómo operan los sistemas de información en distintos contextos organizacionales. Un sistema no agrega valor por almacenar registros de manera masiva; lo hace cuando transforma registros en resultados utilizables y alineados con decisiones y acciones. Esta transición ha sido abordada por la literatura de IS y de gestión de datos como una secuencia

que exige precisión técnica y claridad semántica [2], [8], [22].

Para entender esta secuencia en clave formativa, el estudiante debe identificar qué representa cada nivel en términos programáticos. El dato se vincula con estructuras elementales; la información surge del procesamiento controlado; el conocimiento emerge cuando los resultados se interpretan bajo reglas organizacionales. Programar, por tanto, no consiste en producir salidas arbitrarias, sino en diseñar transformaciones que respeten significado y contexto [1], [5], [27].

Asimismo, estos fundamentos orientan el aprendizaje hacia la precisión. Cuando un algoritmo procesa registros sin validación o sin definición clara de atributos, se degrada la calidad informacional. Este marco garantista exige que el estudiante adopte prácticas de validación y modelado coherentes con principios de ingeniería de software y administración de datos [4], [24], [28].

Dato como registro estructurado

El dato puede definirse como un registro estructurado que representa un hecho, evento o atributo dentro de un dominio organizacional. En IS, los datos adquieren significado cuando se asocian a entidades, relaciones y restricciones, lo cual exige definir tipos, formatos y dominios válidos para cada atributo [2], [27], [28]. Esta caracterización evita ambigüedades y facilita que el sistema procese registros de forma consistente.

Desde la perspectiva del aprendizaje de programación, el dato se materializa en variables, arreglos, registros o clases. Declarar correctamente un tipo, seleccionar una estructura adecuada y validar valores de entrada son decisiones que determinan la integridad del sistema. Por esa razón, los ejercicios iniciales deben insistir en rangos válidos, coherencia de formatos y prevención de valores fuera de dominio [5], [6], [7].

Paralelamente, la representación estructurada se relaciona con principios de normalización y modelado conceptual. Aunque la teoría detallada de bases de datos se abordará en otras unidades, resulta indispensable introducir la noción de que cada atributo responde a una definición precisa y a restricciones explícitas. Esta práctica favorece una visión holística donde el programador entiende que los datos constituyen la materia prima del sistema [27], [28], [29].

En síntesis, aprender a programar implica asumir que cada dato posee significado organizacional. No se trata de valores abstractos, sino de representaciones de hechos que deben mantenerse coherentes, completos y verificables a lo largo del ciclo de vida del sistema [2], [4], [22].

Información como procesamiento contextualizado

La información surge cuando los datos son procesados bajo reglas que incorporan contexto, relaciones y criterios de interpretación. En IS, esta transformación requiere operaciones de cálculo, agregación, filtrado y clasificación que dotan a los registros de significado operativo [2], [8], [22]. La diferencia entre un dato aislado y una información útil radica en el marco que orienta su procesamiento.

Para quien aprende a programar, esta transición se traduce en diseñar algoritmos que transformen entradas en resultados verificables. Calcular totales, identificar tendencias simples o validar condiciones son ejemplos de procesamiento contextualizado que enseñan a relacionar datos con reglas explícitas. Esta práctica promueve disciplina algorítmica y precisión en la definición de operaciones [5], [7], [13].

En contraste con el manejo superficial de registros, el procesamiento contextualizado exige que el estudiante justifique cada operación. ¿Por qué se filtra un conjunto? ¿Qué criterio define una categoría? Estas preguntas fortalecen el razonamiento estructurado y conectan programación con análisis organizacional [1], [2], [16].

De manera complementaria, la calidad de la información depende de la correcta implementación de validaciones y de la coherencia en los cálculos. Errores en tipos numéricos o en comparaciones pueden alterar indicadores y decisiones posteriores. Por ello, la enseñanza de programación debe integrar pruebas sistemáticas y revisión de resultados para garantizar consistencia [4], [11], [12].

Asimismo, la producción de información útil requiere claridad en la presentación. El formato de salida, la organización de resultados y la legibilidad influyen en la interpretación por parte de usuarios. Introducir estos criterios en ejercicios básicos fomenta sensibilidad hacia el impacto organizacional del código [1], [2], [26].

Conocimiento como soporte para decisiones organizacionales

El conocimiento organizacional emerge cuando la información se interpreta bajo objetivos estratégicos y reglas de negocio. En IS, los sistemas apoyan esta etapa proporcionando reportes, indicadores y alertas que orientan acciones y ajustes de procesos [2], [8], [22]. La generación de conocimiento exige coherencia entre cálculo, contexto y finalidad organizacional.

Desde el aprendizaje de programación, el paso hacia conocimiento se refleja en la capacidad de diseñar funciones que traduzcan información en criterios de decisión. Por ejemplo, un programa puede evaluar umbrales, clasificar estados o sugerir acciones en

función de reglas predefinidas. Esta práctica fortalece la habilidad de vincular estructuras lógicas con consecuencias operativas [5], [7], [13].

Destacando la dimensión estratégica, el conocimiento computacional requiere trazabilidad. Cada resultado debe poder vincularse con datos y reglas que lo originaron, evitando decisiones basadas en cálculos opacos. La ingeniería de software respalda esta necesidad mediante principios de diseño claro y documentación coherente [1], [4], [24].

Dentro de este panorama, aprender a programar con orientación a IS significa asumir que cada algoritmo puede influir en decisiones organizacionales. Esta responsabilidad impulsa al estudiante a diseñar soluciones robustas, verificables y alineadas con criterios explícitos [2], [14], [15].

1.3.2. Ciclo de vida del dato en entornos organizacionales

El ciclo de vida del dato describe la secuencia de etapas que atraviesa un registro desde su captura hasta su utilización estratégica. En IS, esta secuencia integra procesos técnicos y organizacionales que determinan calidad, accesibilidad y utilidad del dato a lo largo del tiempo [2], [28], [29].

Para el estudiante de programación, visualizar este ciclo permite ubicar su intervención en cada fase. Programar no se limita a procesar información existente; también implica diseñar mecanismos de captura, validación, almacenamiento y presentación coherentes con reglas y objetivos. Esta visión dinámica conecta fundamentos técnicos con impacto organizacional [1], [7], [16].

En este marco, adoptar prácticas responsables desde programación básica favorece sistemas más robustos en etapas posteriores. Validar entradas, estructurar datos con claridad y producir salidas interpretables constituyen hábitos que acompañan al desarrollador en proyectos de mayor envergadura [4], [6], [24].

Captura y validación de datos

La captura de datos inicia el ciclo informacional. En organizaciones, esta etapa puede involucrar formularios digitales, sensores o registros manuales digitalizados. La calidad del sistema depende de que los datos ingresen con coherencia y consistencia desde el primer momento [2], [27], [28].

Para aprender programación, la captura se traduce en leer entradas, verificar formatos y establecer condiciones mínimas de aceptación. Implementar validaciones evita que el

sistema procese información defectuosa y enseña disciplina en el diseño de algoritmos [5], [6], [7].

Paralelamente, la validación debe contemplar rangos, tipos y relaciones entre atributos. Un registro puede ser válido en forma individual, aunque inconsistente respecto a otros valores. Introducir estas verificaciones en ejercicios básicos fortalece el razonamiento lógico y la atención a detalles estructurales [4], [13], [24].

Asimismo, la captura responsable promueve trazabilidad. Registrar fecha, origen o responsable de un dato facilita auditoría y control posterior, aspectos relevantes en distintos contextos organizacionales [1], [2], [28].

Almacenamiento y procesamiento informacional

El almacenamiento organiza datos para su recuperación y análisis posterior. Sistemas de bases de datos proporcionan mecanismos de persistencia, integridad y consulta que permiten mantener coherencia a lo largo del tiempo [27], [28], [29]. La programación interactúa con estas estructuras mediante consultas, inserciones y actualizaciones controladas.

Para el estudiante, esta etapa se vincula con estructuras de datos persistentes y con operaciones que transforman conjuntos de registros. Diseñar algoritmos que recorran colecciones, filtren elementos y produzcan resultados agregados constituye un paso formativo hacia sistemas informacionales más complejos [5], [7], [13].

Destacando la importancia de integridad, el almacenamiento debe preservar restricciones definidas en el modelo conceptual. Claves, relaciones y dominios protegen coherencia del sistema y evitan inconsistencias acumulativas [4], [27], [28].

En síntesis, aprender a programar implica reconocer que el procesamiento no ocurre en el vacío: se apoya en estructuras persistentes que deben diseñarse con rigor y mantenerse coherentes a lo largo del ciclo de vida [2], [16], [24].

Presentación, análisis y uso estratégico

La etapa final del ciclo informacional consiste en presentar resultados de forma que apoyen análisis y acción. Reportes, visualizaciones y alertas traducen cálculos en representaciones interpretables por actores organizacionales [2], [8], [22]. La forma de presentación influye en la interpretación y en la calidad de decisiones derivadas.

Para quien aprende programación, esta fase introduce diseño de salidas legibles y estructuradas. Organizar información, etiquetar resultados y seleccionar formatos adecuados

forman parte del proceso algorítmico. Esta práctica fortalece sensibilidad hacia usuarios y hacia el impacto organizacional del código [1], [7], [26].

Asimismo, el análisis estratégico puede incorporar reglas adicionales que identifiquen patrones o condiciones de alerta. Implementar comparaciones, umbrales y clasificaciones enseña a vincular información con criterios decisionales [2], [5], [13].

Dentro de este panorama, aprender a programar desde la perspectiva de IS significa participar en todo el ciclo de vida del dato. Cada línea de código contribuye a capturar, transformar y presentar información que orienta acciones organizacionales. Esta visión integradora motiva al lector a desarrollar competencias técnicas con conciencia del impacto y con compromiso hacia la calidad del sistema [4], [14], [15].

Capítulo 2

Representación digital de la información

La representación digital define la forma en que un Sistema de Información (SI) convierte hechos organizacionales en símbolos manipulables por software. Esta conversión condiciona almacenamiento, transmisión y procesamiento, dado que toda operación computacional se ejecuta sobre codificaciones concretas que deben preservar significado y consistencia [1], [2], [34].

Aprender a programar con orientación a IS se fortalece cuando el lector domina las decisiones elementales de representación. Un error de codificación de texto, un desbordamiento en enteros o una interpretación incorrecta de números reales puede degradar reportes, alterar reglas de negocio y producir decisiones apoyadas en resultados inválidos [7], [11], [12].

Dentro de este panorama, la representación digital aporta un marco formativo que conecta teoría y práctica. Comprender bases numéricas, codificaciones y formatos conduce a programas más robustos, con validaciones mejor justificadas y salidas que preservan semántica organizacional [4], [5], [13].

Esta sección organiza la representación digital en tres perspectivas complementarias: fundamentos binarios, codificación de tipos de contenido y riesgos habituales de representación. La finalidad de ilustrar esta organización consiste en que el lector encuentre motivos técnicos y profesionales para programar con rigor, desde ejercicios básicos hasta escenarios organizacionales de mayor envergadura [2], [16], [24].

2.1. Fundamentos de representación binaria

La representación binaria constituye el lenguaje operativo del hardware. El procesador y la memoria trabajan con estados discretos, y el software se apoya en esos estados para representar números, texto y estructuras de datos que describen eventos organizacionales [2], [10], [34].

Aprender estos fundamentos permite que la programación deje de ser una actividad “mágica” y se convierta en una práctica justificable. Cuando el lector relaciona variables y estructuras con bytes reales en memoria, adquiere criterio para elegir tipos, anticipar límites y explicar fallas asociadas a rangos y conversiones [5], [7], [35].

La perspectiva binaria también facilita evaluar eficiencia. Representar información con estructuras apropiadas reduce consumo de memoria y costo de procesamiento, lo cual se refleja en tiempos de respuesta y escalabilidad del SI, especialmente cuando se procesan grandes volúmenes de registros [13], [34], [36].

No obstante, la utilidad formativa más directa se manifiesta en la depuración. Reconocer cómo se codifica un dato habilita rastrear errores de interpretación, fallos de lectura/escritura y resultados inesperados en operaciones aritméticas o lógicas, fortaleciendo hábitos de verificación [4], [7], [23].

Paralelamente, el manejo de bits introduce destrezas prácticas frecuentes en IS: validación mediante máscaras, compresión simple, control de permisos y codificación de estados. Estas destrezas ofrecen ejercicios motivadores para aprender a programar con problemas cercanos a escenarios reales [5], [13], [37].

2.1.1. Bits, bytes y almacenamiento

Un bit representa la unidad mínima de información en sistemas digitales, y un byte agrupa bits para formar una unidad práctica de almacenamiento. Esta organización responde a cómo hardware y sistema operativo direccionan memoria y gestionan transferencia de datos, lo cual incide en tamaño de variables, estructuras y archivos [2], [10], [34].

En programación, la noción de byte se conecta con el tipo de dato elegido. Un entero, un carácter o un valor booleano ocupan un número determinado de bytes; esa ocupación define rangos posibles y condiciona operaciones de lectura, escritura y serialización. Esta relación orienta al estudiante a seleccionar tipos con criterio y a justificar la representación en función del dominio organizacional [5], [7], [35].

El almacenamiento también se expresa en estructuras: arreglos, cadenas y registros

ocupan memoria de manera distinta. Reconocer esta diferencia favorece decisiones sobre cómo modelar colecciones de datos organizacionales, evitando estructuras infladas o diseños que dificulten procesamiento por recorridos repetidos [13], [36], [37].

En escenarios de IS, el almacenamiento se extiende a archivos y bases de datos. Aunque el estudio detallado de persistencia se desarrolla en otras unidades, resulta pertinente observar que un archivo codifica bytes y que una mala interpretación de esa codificación produce datos corruptos o inconsistentes [27], [28], [29].

De manera complementaria, el almacenamiento se relaciona con integridad y seguridad. Tamaños mal controlados, conversiones inseguras o supuestos erróneos sobre longitudes introducen fallas previsibles, por lo que se recomienda entrenar desde temprano validaciones y manejo seguro de entradas [4], [6], [23].

2.1.2. Sistemas de numeración

Los sistemas de numeración constituyen el fundamento matemático que permite representar cantidades mediante símbolos y reglas formales. En términos generales, un sistema de numeración define un conjunto de dígitos permitidos y un mecanismo para combinar dichos dígitos con el fin de expresar valores. Desde la perspectiva computacional, esta formalización es determinante porque toda información procesada por un Sistema de Información (SI) se expresa finalmente como secuencias numéricas codificadas en binario [13], [34], [37].

Existen sistemas posicionales y no posicionales. En los sistemas no posicionales, el valor de un símbolo no depende de su posición; ejemplos históricos como la numeración romana ilustran esta característica. En contraste, los sistemas posicionales asignan significado a cada posición en función de una potencia de la base. Esta distinción resulta indispensable para quien aprende a programar, ya que los sistemas digitales utilizan exclusivamente esquemas posicionales para almacenar y transformar datos [5], [7], [34].

En un sistema posicional de base b , los dígitos permitidos pertenecen al conjunto $\{0, 1, \dots, b-1\}$. El valor mínimo representable con un solo dígito es 0, y el valor máximo es $b-1$. Cuando se combinan n dígitos, el valor total se obtiene mediante la suma ponderada de cada dígito multiplicado por la potencia correspondiente de la base:

$$N = \sum_{i=0}^{n-1} d_i \cdot b^i,$$

donde d_i representa el dígito en la posición i . Esta formulación no constituye un formalismo abstracto; proporciona al estudiante un modelo exacto para diseñar algoritmos de conversión

y para justificar resultados obtenidos por el programa [5], [13], [37].

En computación, base 2 se emplea para operación interna, base 16 se utiliza para representación compacta y lectura técnica, y base 10 se mantiene como referencia organizacional para usuarios y reportes [2], [7], [34]. Dominar la noción de base permite al lector interpretar resultados de depuración, comprender límites de tipos numéricos y diseñar validaciones coherentes con el dominio del problema.

Representación en base decimal (10)

El sistema decimal utiliza base 10, con dígitos entre 0 y 9. Su valor máximo por posición es 9, y cada posición representa una potencia de 10. Por ejemplo:

$$372_{10} = 3 \cdot 10^2 + 7 \cdot 10^1 + 2 \cdot 10^0.$$

En programación, esta representación coincide con la forma habitual en que los usuarios ingresan datos numéricos, lo cual exige validaciones de rango y control de formato [5], [7].

Ejercicios propuestos:

1. Representar el número 5042_{10} como suma de potencias de 10.
2. Determinar el valor máximo representable con tres dígitos decimales.
3. Codificar en decimal el número ASCII del carácter 'A' (65) y verificar su interpretación como carácter.

Representación en base binaria (2)

La base 2 emplea dígitos $\{0, 1\}$. El valor máximo por posición es 1. Cada posición representa una potencia de 2. Por ejemplo:

$$1011_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 11_{10}.$$

Este sistema se ha consolidado como fundamento del hardware digital [7], [13], [34].

Ejercicios propuestos:

1. Convertir 110101_2 a decimal.
2. Determinar el valor máximo representable con 8 bits sin signo.
3. Representar en binario el código ASCII de la letra 'B' (66).

Representación en base octal (8)

El sistema octal utiliza base 8, con dígitos entre 0 y 7. El valor máximo por posición es 7. Su uso en informática histórica y en permisos de sistemas operativos lo convierte en un recurso práctico para quien aprende a programar [10], [34], [37]. Un número octal como:

$$157_8 = 1 \cdot 8^2 + 5 \cdot 8^1 + 7 \cdot 8^0$$

puede convertirse a decimal mediante el algoritmo posicional descrito anteriormente.

Ejercicios propuestos:

1. Convertir 345_8 a base decimal.
2. Determinar el mayor número representable con cuatro dígitos octales.
3. Representar en octal el código decimal 65 y relacionarlo con su carácter ASCII correspondiente.

Representación en base hexadecimal (16)

La base 16 utiliza dígitos $\{0, \dots, 9, A, \dots, F\}$, donde $A = 10$ hasta $F = 15$. El valor máximo por posición es 15. Esta base facilita lectura compacta de direcciones de memoria y datos binarios [7], [34], [35]. Por ejemplo:

$$2F_{16} = 2 \cdot 16^1 + 15 \cdot 16^0 = 47_{10}.$$

Ejercicios propuestos:

1. Convertir $3A7_{16}$ a decimal.
2. Determinar el máximo valor representable con dos dígitos hexadecimales.
3. Representar en hexadecimal el código ASCII de la palabra “SI” (S: 83, I: 73).

Aplicación formativa

El estudio de los sistemas de numeración fortalece pensamiento algorítmico, ya que cada conversión puede implementarse mediante procedimientos iterativos y verificables [5], [7], [13]. La conexión con datos alfanuméricos se evidencia en la codificación ASCII o Unicode,

donde cada carácter se representa como número entero que puede expresarse en distintas bases.

Desde la perspectiva de IS, dominar estas representaciones habilita interpretar identificadores, permisos, códigos de estado y estructuras de almacenamiento. Programar con criterio numérico reduce errores de conversión, facilita depuración y aporta fundamento técnico a decisiones de diseño que impactan directamente en la calidad del Sistema de Información [2], [24], [34].

2.1.3. Conversión de valores enteros entre distintos sistemas de numeración

La conversión entre bases se apoya en descomposición posicional. Este procedimiento constituye un ejemplo didáctico de algoritmo finito, con pasos verificables y resultados contrastables, por lo que se ha posicionado como práctica formativa adecuada para principiantes [5], [7], [13].

En programación, implementar conversiones refuerza dominio de divisiones enteras, residuos, recorridos y construcción incremental de resultados. Estos conceptos reaparecen en problemas típicos de IS, tales como generación de códigos, validación de identificadores y tratamiento de formatos de entrada variados [5], [7], [37].

Las conversiones entre bases se organizan en familias de procedimientos según su lógica de cálculo. En términos formativos, esta agrupación ayuda a programar las transformaciones con algoritmos simples, verificables y reutilizables, evitando memorizar reglas aisladas [5], [7], [13].

Familia A: conversión *desde decimal* mediante divisiones sucesivas

Este procedimiento se aplica cuando el número de entrada está en base 10 y se desea expresarlo en base b (por ejemplo, decimal→binario, decimal→octal, decimal→hexadecimal). Su lógica se apoya en cocientes y residuos, lo cual facilita una implementación directa con bucles [5], [7], [13].

1. Tomar el número decimal N como valor inicial.
2. Dividir N entre la base destino b .
3. Registrar el residuo $r = N \bmod b$ como dígito siguiente.

4. Actualizar $N \leftarrow \lfloor N/b \rfloor$.
5. Repetir pasos 2–4 hasta que $N = 0$.
6. Leer los residuos en orden inverso (del último al primero) para obtener el número en base b .
7. En base 16, reemplazar residuos 10–15 por A – F .

Casos cubiertos por esta familia:

- **Decimal** $\rightarrow$ **Binario** (base $b = 2$)
- **Decimal** $\rightarrow$ **Octal** (base $b = 8$)
- **Decimal** $\rightarrow$ **Hexadecimal** (base $b = 16$)

Ejercicios resueltos: Conversión desde decimal mediante divisiones sucesivas

A continuación se presentan dos ejercicios completos para cada caso cubierto por la familia de conversión desde base decimal hacia otra base utilizando divisiones sucesivas. En cada uno se siguen estrictamente los pasos establecidos.

1. Decimal $\rightarrow$ Binario (base $b = 2$)

Ejercicio 1: Convertir 45_{10} a binario

Paso 1: $N = 45$

N	$N \div 2$	Residuo
45	22	1
22	11	0
11	5	1
5	2	1
2	1	0
1	0	1

Paso final: Leer residuos en orden inverso:

$$45_{10} = 101101_2$$

Ejercicio 2: Convertir 156_{10} a binario

N	$N \div 2$	Residuo
156	78	0
78	39	0
39	19	1
19	9	1
9	4	1
4	2	0
2	1	0
1	0	1

$$156_{10} = 10011100_2$$

2. Decimal $\rightarrow$ Octal (base $b = 8$)

Ejercicio 1: Convertir 83_{10} a octal

N	$N \div 8$	Residuo
83	10	3
10	1	2
1	0	1

$$83_{10} = 123_8$$

Ejercicio 2: Convertir 250_{10} a octal

N	$N \div 8$	Residuo
250	31	2
31	3	7
3	0	3

$$250_{10} = 372_8$$

3. Decimal $\rightarrow$ Hexadecimal (base $b = 16$)

Ejercicio 1: Convertir 254_{10} a hexadecimal

N	$N \div 16$	Residuo
254	15	14
15	0	15

Reemplazando residuos:

$$14 = E, \quad 15 = F$$

$$254_{10} = FE_{16}$$

Ejercicio 2: Convertir 1023_{10} a hexadecimal

N	$N \div 16$	Residuo
1023	63	15
63	3	15
3	0	3

Reemplazando residuos:

$$15 = F$$

$$1023_{10} = 3FF_{16}$$

Familia B: conversión *hacia decimal* mediante expansión posicional

Este procedimiento se aplica cuando el número de entrada está en base b y se desea su equivalente en base 10 (por ejemplo, binario→decimal, octal→decimal, hexadecimal→decimal). Se fundamenta en el sistema posicional, donde cada dígito pondera una potencia de la base [13], [34], [37].

La conversión también funciona como ejercicio de pruebas. El estudiante puede verificar propiedades de reversibilidad (convertir y retornar), detectar discrepancias por redondeo y documentar supuestos sobre rangos. Esta práctica fortalece hábitos de verificación y depuración [4], [5], [23].

1. Identificar la base b del número de entrada (2, 8 o 16).
2. Enumerar los dígitos de derecha a izquierda, asignando exponente 0 al dígito menos significativo.
3. Para cada dígito d_i , calcular $d_i \cdot b^i$.
4. Sumar todos los productos obtenidos.
5. El resultado total corresponde al valor en base decimal.

Casos cubiertos por esta familia:

- **Binario** $\rightarrow$ **Decimal** (base $b = 2$)
- **Octal** $\rightarrow$ **Decimal** (base $b = 8$)
- **Hexadecimal** $\rightarrow$ **Decimal** (base $b = 16$, con $A = 10, \dots, F = 15$)

Ejercicios resueltos: Conversión hacia decimal mediante expansión posicional

A continuación se presentan dos ejercicios completos para cada caso cubierto por la familia de conversión hacia base decimal utilizando expansión posicional. En cada ejercicio se siguen estrictamente los pasos establecidos: identificar la base, enumerar dígitos de derecha a izquierda, multiplicar por potencias de la base y sumar los productos.

1. Binario $\rightarrow$ Decimal (base $b = 2$)**Ejercicio 1: Convertir 101101_2 a decimal**

Paso 1: Base identificada: $b = 2$

Paso 2: Enumerar dígitos de derecha a izquierda (exponente 0 al menos significativo)

Dígito (d_i)	Posición i	Potencia 2^i	Producto $d_i \cdot 2^i$
1	0	$2^0 = 1$	$1 \cdot 1 = 1$
0	1	$2^1 = 2$	$0 \cdot 2 = 0$
1	2	$2^2 = 4$	$1 \cdot 4 = 4$
1	3	$2^3 = 8$	$1 \cdot 8 = 8$
0	4	$2^4 = 16$	$0 \cdot 16 = 0$
1	5	$2^5 = 32$	$1 \cdot 32 = 32$

Paso final: Sumar productos:

$$1 + 0 + 4 + 8 + 0 + 32 = 45$$

$$101101_2 = 45_{10}$$

Ejercicio 2: Convertir 10011100_2 a decimal

Paso 1: Base identificada: $b = 2$

Dígito (d_i)	Posición i	Potencia 2^i	Producto $d_i \cdot 2^i$
0	0	$2^0 = 1$	$0 \cdot 1 = 0$
0	1	$2^1 = 2$	$0 \cdot 2 = 0$
1	2	$2^2 = 4$	$1 \cdot 4 = 4$
1	3	$2^3 = 8$	$1 \cdot 8 = 8$
1	4	$2^4 = 16$	$1 \cdot 16 = 16$
0	5	$2^5 = 32$	$0 \cdot 32 = 0$
0	6	$2^6 = 64$	$0 \cdot 64 = 0$
1	7	$2^7 = 128$	$1 \cdot 128 = 128$

Paso final: Sumar productos:

$$0 + 0 + 4 + 8 + 16 + 0 + 0 + 128 = 156$$

$$10011100_2 = 156_{10}$$

2. Octal $\rightarrow$ Decimal (base $b = 8$)

Ejercicio 1: Convertir 372_8 a decimal

Paso 1: Base identificada: $b = 8$

Dígito (d_i)	Posición i	Potencia 8^i	Producto $d_i \cdot 8^i$
2	0	$8^0 = 1$	$2 \cdot 1 = 2$
7	1	$8^1 = 8$	$7 \cdot 8 = 56$
3	2	$8^2 = 64$	$3 \cdot 64 = 192$

Paso final: Sumar productos:

$$2 + 56 + 192 = 250$$

$$372_8 = 250_{10}$$

Ejercicio 2: Convertir 123_8 a decimal

Paso 1: Base identificada: $b = 8$

Dígito (d_i)	Posición i	Potencia 8^i	Producto $d_i \cdot 8^i$
3	0	$8^0 = 1$	$3 \cdot 1 = 3$
2	1	$8^1 = 8$	$2 \cdot 8 = 16$
1	2	$8^2 = 64$	$1 \cdot 64 = 64$

Paso final: Sumar productos:

$$3 + 16 + 64 = 83$$

$$123_8 = 83_{10}$$

3. Hexadecimal $\rightarrow$ Decimal (base $b = 16$)

Ejercicio 1: Convertir FE_{16} a decimal

Paso 1: Base identificada: $b = 16$, con $F = 15$ y $E = 14$

Dígito (d_i)	Posición i	Potencia 16^i	Producto $d_i \cdot 16^i$
$E = 14$	0	$16^0 = 1$	$14 \cdot 1 = 14$
$F = 15$	1	$16^1 = 16$	$15 \cdot 16 = 240$

Paso final: Sumar productos:

$$14 + 240 = 254$$

$$FE_{16} = 254_{10}$$

Ejercicio 2: Convertir $3FF_{16}$ a decimal

Paso 1: Base identificada: $b = 16$, con $F = 15$

Dígito (d_i)	Posición i	Potencia 16^i	Producto $d_i \cdot 16^i$
$F = 15$	0	$16^0 = 1$	$15 \cdot 1 = 15$
$F = 15$	1	$16^1 = 16$	$15 \cdot 16 = 240$
3	2	$16^2 = 256$	$3 \cdot 256 = 768$

Paso final: Sumar productos:

$$15 + 240 + 768 = 1023$$

$$3FF_{16} = 1023_{10}$$

Familia C: conversiones entre bases potencia de 2 mediante agrupación de bits

Cuando las bases son potencias de 2, las conversiones pueden realizarse por *agrupación de bits* sin pasar por decimal, lo cual simplifica depuración y permite implementar conversores eficientes [13], [34], [35]. En particular, $8 = 2^3$ y $16 = 2^4$.

C1. Binario $\leftrightarrow$ Octal (grupos de 3 bits) Binario $\rightarrow$ Octal

1. Separar el número binario en grupos de 3 bits desde la derecha.
2. Si el grupo más a la izquierda queda incompleto, rellenar con ceros a la izquierda hasta completar 3 bits.
3. Convertir cada grupo de 3 bits a su valor decimal (0 a 7).
4. Escribir esos valores consecutivamente como dígitos octales.

Octal $\rightarrow$ Binario

1. Tomar cada dígito octal por separado (0 a 7).
2. Convertir cada dígito a su forma binaria de 3 bits (por ejemplo, $5_8 \rightarrow 101_2$).
3. Concatenar los grupos de 3 bits en el mismo orden para formar el binario final.
4. Si se desea una forma normalizada, eliminar ceros a la izquierda que no aporten valor.

C2. Binario $\leftrightarrow$ Hexadecimal (grupos de 4 bits) Binario $\rightarrow$ Hexadecimal

1. Separar el número binario en grupos de 4 bits desde la derecha.
2. Rellenar con ceros a la izquierda el grupo más significativo si fuera necesario.
3. Convertir cada grupo de 4 bits a su valor (0 a 15).
4. Reemplazar 10–15 por A – F .
5. Concatenar los dígitos resultantes para obtener el número hexadecimal.

Hexadecimal $\rightarrow$ Binario

1. Tomar cada dígito hexadecimal por separado (0–9, A – F).

2. Convertirlo a su valor entero (A=10, ..., F=15).
3. Representar ese valor como binario de 4 bits.
4. Concatenar los grupos de 4 bits en el mismo orden.
5. Normalizar quitando ceros a la izquierda si corresponde.

Ejercicios resueltos: Conversión entre bases potencia de 2 con longitudes no divisibles

En los siguientes ejercicios se emplean cantidades de bits que no son múltiplos de 3 ni de 4 (por ejemplo, 8, 16 y 32 bits). Esto obliga a aplicar correctamente el paso de relleno con ceros a la izquierda antes de agrupar, lo cual es relevante cuando se trabaja con registros completos de memoria o tipos de datos estándar en programación (byte de 8 bits, palabra de 16 bits, entero de 32 bits).

1. Binario $\rightarrow$ Octal (8, 16 y 32 bits)

Ejercicio 1: Convertir 10110110_2 (8 bits) a octal

Paso 1: Agrupar desde la derecha en bloques de 3 bits:

10 110 110

Paso 2: Rellenar con ceros a la izquierda:

010 110 110

Paso 3: Convertir cada grupo:

$$010_2 = 2, \quad 110_2 = 6, \quad 110_2 = 6$$

Paso 4: Resultado:

$$10110110_2 = 266_8$$

Observación adicional: En programación, un byte (8 bits) no coincide naturalmente con bloques de 3 bits; por ello, el relleno garantiza que la representación octal conserve el valor exacto del patrón binario.

Ejercicio 2: Convertir 1100110010101101_2 (16 bits) a octal**Paso 1:** $110\ 011\ 001\ 010\ 110\ 1$ **Paso 2:** Rellenar: $001\ 100\ 110\ 010\ 101\ 101$ **Paso 3:** $001 = 1, 100 = 4, 110 = 6, 010 = 2, 101 = 5, 101 = 5$ **Paso 4:** $1100110010101101_2 = 146255_8$

Observación adicional: En registros de 16 bits, la agrupación produce más dígitos octales que bytes disponibles, lo que demuestra que el tamaño de palabra y la base elegida influyen en la forma externa del número.

2. Octal $\rightarrow$ Binario (resultados de 8, 16 y 32 bits)**Ejercicio 1: Convertir 347_8 a binario****Paso 1:** $3, 4, 7$ **Paso 2:** $3 = 011, 4 = 100, 7 = 111$ **Paso 3:** 011100111_2 **Paso 4:** Normalizar: 11100111_2

Observación adicional: El resultado tiene 8 bits tras eliminar el cero inicial, coincidiendo con un byte completo.

Ejercicio 2: Convertir 15723_8 a binario

Paso 1:

1, 5, 7, 2, 3

Paso 2:

001, 101, 111, 010, 011

Paso 3:

001101111010011₂

Paso 4: Normalizar:

1101111010011₂

Observación adicional: Este resultado no es múltiplo de 8 bits; al almacenarlo en memoria real, el compilador ajustaría el tamaño al tipo de dato correspondiente.

3. Binario $\rightarrow$ Hexadecimal (8, 16 y 32 bits)

Ejercicio 1: Convertir 10101101_2 (8 bits) a hexadecimal

Paso 1: Agrupar en bloques de 4 bits:

1010 1101

Paso 2: No requiere relleno.

Paso 3:

$1010 = 10 = A, \quad 1101 = 13 = D$

Paso 4:

$10101101_2 = AD_{16}$

Observación adicional: Un byte equivale exactamente a dos dígitos hexadecimales; por ello, hexadecimal resulta especialmente práctico para inspección de memoria.

Ejercicio 2: Convertir 1100101011110001_2 (16 bits) a hexadecimal

Paso 1:

$$1100\ 1010\ 1111\ 0001$$

Paso 2:

$$1100 = 12 = C, 1010 = 10 = A, 1111 = 15 = F, 0001 = 1$$

Paso 3:

$$1100101011110001_2 = CAF1_{16}$$

Observación adicional: En 16 bits se obtienen cuatro dígitos hexadecimales; esta relación directa simplifica la depuración en lenguajes de bajo nivel.

4. Hexadecimal $\rightarrow$ Binario (32 bits)

Ejercicio 1: Convertir $1A3F_{16}$ a binario

Paso 1:

$$1, A, 3, F$$

Paso 2:

$$0001, 1010, 0011, 1111$$

Paso 3:

$$0001101000111111_2$$

Paso 4: Normalizar:

$$1101000111111_2$$

Observación adicional: Aunque el número puede escribirse sin ceros iniciales, en memoria podría almacenarse en 16 o 32 bits según el tipo de dato.

Ejercicio 2: Convertir $89ABCDEF_{16}$ a binario (32 bits)**Paso 1:**

8, 9, A, B, C, D, E, F

Paso 2:

1000, 1001, 1010, 1011, 1100, 1101, 1110, 1111

Paso 3:10001001101010111100110111101111₂

Observación adicional: Cada dígito hexadecimal representa exactamente 4 bits; por ello, 8 dígitos hexadecimales equivalen a 32 bits completos, tamaño típico de un entero estándar en muchos lenguajes de programación.

Familia D: conversiones Octal $\leftrightarrow$ Hexadecimal mediante puente binario

Octal y hexadecimal no son potencias directas entre sí, aunque ambas son potencias de 2. Un procedimiento estable consiste en convertir primero a binario y luego a la base destino, manteniendo pasos sistemáticos y fáciles de programar [5], [7], [34].

D1. Octal $\rightarrow$ Hexadecimal

1. Convertir cada dígito octal a 3 bits y concatenar para obtener el binario (procedimiento C1).
2. Agrupar el binario en bloques de 4 bits desde la derecha (procedimiento C2).
3. Convertir cada bloque a hexadecimal y concatenar.

D2. Hexadecimal $\rightarrow$ Octal

1. Convertir cada dígito hexadecimal a 4 bits y concatenar para obtener el binario (procedimiento C2).
2. Agrupar el binario en bloques de 3 bits desde la derecha (procedimiento C1).
3. Convertir cada bloque a octal y concatenar.

Mapa completo de conversiones (con familia aplicable)

- Decimal $\rightarrow$ Binario: Familia A
- Binario $\rightarrow$ Decimal: Familia B
- Decimal $\rightarrow$ Octal: Familia A
- Octal $\rightarrow$ Decimal: Familia B
- Decimal $\rightarrow$ Hexadecimal: Familia A
- Hexadecimal $\rightarrow$ Decimal: Familia B
- Binario $\rightarrow$ Octal: Familia C1
- Octal $\rightarrow$ Binario: Familia C1
- Binario $\rightarrow$ Hexadecimal: Familia C2
- Hexadecimal $\rightarrow$ Binario: Familia C2
- Octal $\rightarrow$ Hexadecimal: Familia D (puente binario)
- Hexadecimal $\rightarrow$ Octal: Familia D (puente binario)

Ejercicios resueltos: Conversión Octal $\leftrightarrow$ Hexadecimal mediante puente binario

A continuación se presentan dos ejercicios completos para cada caso de la Familia D. En cada uno se sigue estrictamente el procedimiento: conversión intermedia a binario, reagrupación y conversión final. Se emplean longitudes que no son múltiplos directos entre 3 y 4 para evidenciar la necesidad de relleno y normalización.

1. Octal $\rightarrow$ Hexadecimal (puente binario)**Ejercicio 1: Convertir 735_8 a hexadecimal**

Paso 1: Convertir cada dígito octal a 3 bits

$$7 = 111, \quad 3 = 011, \quad 5 = 101$$

$$735_8 \rightarrow 111011101_2$$

Paso 2: Agrupar en bloques de 4 bits desde la derecha

1110 1110 1

Rellenar a la izquierda:

0001 1101 1101

Paso 3: Convertir cada bloque a hexadecimal

$$0001 = 1, \quad 1101 = D, \quad 1101 = D$$

$$735_8 = 1DD_{16}$$

Observación adicional: Aunque el número original tenía 9 bits intermedios, el relleno garantiza alineación correcta en bloques de 4 bits, lo cual es indispensable al programar conversores automáticos.

Ejercicio 2: Convertir 12647_8 a hexadecimal

Paso 1:

$$1 = 001, \quad 2 = 010, \quad 6 = 110, \quad 4 = 100, \quad 7 = 111$$

$$12647_8 \rightarrow 001010110100111_2$$

Paso 2: Agrupar en bloques de 4 bits

0010 1011 0100 111

Rellenar:

0001 0101 1010 0111

Paso 3:

$$0001 = 1, \quad 0101 = 5, \quad 1010 = A, \quad 0111 = 7$$

$$12647_8 = 15A7_{16}$$

Observación adicional: Este procedimiento evidencia que la longitud binaria intermedia no coincide necesariamente con 8, 16 o 32 bits; la alineación depende del patrón generado por la base original.

2. Hexadecimal $\rightarrow$ Octal (puente binario)

Ejercicio 1: Convertir $2AF_{16}$ a octal

Paso 1: Convertir cada dígito hexadecimal a 4 bits

$$2 = 0010, \quad A = 1010, \quad F = 1111$$

$$2AF_{16} \rightarrow 001010101111_2$$

Paso 2: Agrupar en bloques de 3 bits desde la derecha

$$001\ 010\ 101\ 111$$

Paso 3: Convertir cada bloque a octal

$$001 = 1, \quad 010 = 2, \quad 101 = 5, \quad 111 = 7$$

$$2AF_{16} = 1257_8$$

Observación adicional: Obsérvese que 3 dígitos hexadecimales (12 bits) producen exactamente 4 dígitos octales (también 12 bits), lo cual facilita validaciones cruzadas en programas.

Ejercicio 2: Convertir $4C3D_{16}$ a octal

Paso 1:

$$4 = 0100, \quad C = 1100, \quad 3 = 0011, \quad D = 1101$$

$$4C3D_{16} \rightarrow 0100110000111101_2$$

Paso 2: Agrupar en bloques de 3 bits

010 011 000 011 110 1

Rellenar:

001 001 100 001 111 101

Paso 3:

001 = 1, 001 = 1, 100 = 4, 001 = 1, 111 = 7, 101 = 5

$$4C3D_{16} = 114175_8$$

Observación adicional: En este caso, 16 bits generan 6 dígitos octales tras el relleno; esta diferencia estructural es relevante cuando se diseñan funciones de conversión en lenguajes que manipulan enteros de tamaño fijo. No obstante, la conversión presenta límites cuando intervienen números reales. En esos casos, la representación depende de estándares de punto flotante, donde ciertos valores no pueden representarse exactamente. Reconocer esta limitación evita errores interpretativos en cálculos financieros o métricas operativas [11], [12], [38].

2.1.4. Conversión y representación de valores reales en distintos sistemas de numeración

Hasta este punto, los procedimientos descritos se han aplicado principalmente a números enteros. No obstante, en SI gran parte de los datos procesados corresponden a cantidades reales: montos financieros, promedios, tasas, porcentajes y métricas continuas. La conversión y representación de estos valores requiere extender los pasos anteriores, incorporando el tratamiento de la parte fraccionaria y considerando las limitaciones inherentes a la representación en punto flotante [11], [12], [38].

1. Extensión del modelo posicional a números reales

En un sistema posicional de base b , un número real puede expresarse como:

$$N = \sum_{i=-m}^n d_i \cdot b^i$$

donde los exponentes negativos representan posiciones fraccionarias. Por ejemplo, en base decimal:

$$45.375_{10} = 4 \cdot 10^1 + 5 \cdot 10^0 + 3 \cdot 10^{-1} + 7 \cdot 10^{-2} + 5 \cdot 10^{-3}$$

Esta formulación generaliza el esquema utilizado para enteros e introduce un aspecto formativo clave: la parte fraccionaria se construye mediante potencias negativas de la base [5], [13], [37]. Para quien aprende a programar, este modelo permite diseñar algoritmos sistemáticos de conversión que tratan por separado parte entera y parte decimal.

2. Conversión de decimal real a otra base

Cuando se convierte un número decimal real a base b (por ejemplo, decimal $\rightarrow$ binario), el procedimiento se divide en dos fases:

Parte entera (igual que antes):

1. Aplicar divisiones sucesivas entre la base b .
2. Registrar residuos.
3. Leer residuos en orden inverso.

Parte fraccionaria:

1. Tomar la parte decimal f .
2. Multiplicar f por la base b .
3. El entero resultante constituye el siguiente dígito fraccionario.
4. Conservar la nueva parte fraccionaria.
5. Repetir hasta que la fracción sea 0 o hasta alcanzar la precisión deseada.

Ejemplo conceptual: convertir 0.625_{10} a binario.

$$0.625 \times 2 = 1.25 \Rightarrow 1$$

$$0.25 \times 2 = 0.5 \Rightarrow 0$$

$$0.5 \times 2 = 1.0 \Rightarrow 1$$

Resultado: 0.101_2

Este procedimiento enseña al estudiante a manejar bucles controlados por precisión, introduciendo nociones de tolerancia y límite de iteraciones [5], [7], [38].

Ejemplos resueltos: Conversión de números reales. A continuación se presentan dos ejemplos completos de conversión de números reales (números con parte fraccionaria) desde base decimal hacia otra base. En ambos casos se aplican las dos fases del procedimiento: conversión de la parte entera mediante divisiones sucesivas y conversión de la parte fraccionaria mediante multiplicaciones sucesivas. El primer ejemplo produce una expansión finita; el segundo genera una expansión infinita periódica. Para representar una serie infinita se puede usar la siguiente notación $0.\overline{133}_3$, **133 se repite infinitamente y su base es 3**.

Ejemplo 1: Conversión finita

Convertir 13.625_{10} a binario

Parte entera

$$N = 13$$

N	$N \div 2$	Residuo
13	6	1
6	3	0
3	1	1
1	0	1

Leer residuos en orden inverso:

$$13_{10} = 1101_2$$

Parte fraccionaria

$$f = 0.625$$

$$0.625 \times 2 = 1.25 \Rightarrow 1$$

$$0.25 \times 2 = 0.5 \Rightarrow 0$$

$$0.5 \times 2 = 1.0 \Rightarrow 1$$

La fracción se hace 0, por lo tanto la conversión termina.

$$0.625_{10} = 0.101_2$$

Resultado final

$$13.625_{10} = 1101.101_2$$

Observación adicional: La conversión es finita porque $0.625 = \frac{5}{8}$ y el denominador es potencia de 2. Cuando la fracción decimal puede expresarse como $k/2^n$, su representación binaria termina.

Ejemplo 2: Conversión infinita periódica

Convertir 7.1_{10} a binario

Parte entera

$$N = 7$$

N	$N \div 2$	Residuo
7	3	1
3	1	1
1	0	1

$$7_{10} = 111_2$$

Parte fraccionaria

$$f = 0.1$$

$$0.1 \times 2 = 0.2 \Rightarrow 0$$

$$0.2 \times 2 = 0.4 \Rightarrow 0$$

$$0.4 \times 2 = 0.8 \Rightarrow 0$$

$$0.8 \times 2 = 1.6 \Rightarrow 1$$

$$0.6 \times 2 = 1.2 \Rightarrow 1$$

$$0.2 \times 2 = 0.4 \Rightarrow 0$$

Se observa que reaparece el valor 0.2, por lo que el patrón se repite.

$$0.1_{10} = 0.0001100110011 \dots_2$$

Resultado final

$$7.1_{10} = 111.0001100110011 \dots_2$$

Observación adicional: La expansión no termina porque $0.1 = \frac{1}{10}$ y el denominador contiene factores primos distintos de 2. En sistemas digitales, esta representación infinita se aproxima con un número finito de bits, lo que explica errores de redondeo en cálculos con punto flotante.

3. Conversión desde otra base hacia decimal (valores reales)

El proceso inverso aplica expansión posicional incluyendo potencias negativas:

Ejemplo: convertir 101.101_2 a decimal.

$$1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3}$$

$$= 4 + 0 + 1 + 0.5 + 0 + 0.125 = 5.625_{10}$$

El algoritmo consiste en:

1. Separar parte entera y fraccionaria.
2. Aplicar expansión posicional con exponentes positivos y negativos.
3. Sumar todos los términos.

Este procedimiento refuerza el entendimiento del sistema posicional extendido y promueve implementación modular en código [7], [13], [37].

Ejercicios resueltos: Conversión desde otra base hacia decimal (valores reales)

A continuación se presentan dos ejercicios completos aplicando expansión posicional con exponentes positivos y negativos. En ambos casos se separa parte entera y fraccionaria, y luego se suman todos los términos.

Ejemplo 1: Conversión finita

Convertir 1011.011_2 a decimal

Paso 1: Separar parte entera y fraccionaria

$$1011_2 \quad \text{y} \quad 0.011_2$$

Paso 2: Expandir parte entera

$$1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$$

$$= 8 + 0 + 2 + 1 = 11$$

Paso 3: Expandir parte fraccionaria

$$0 \cdot 2^{-1} + 1 \cdot 2^{-2} + 1 \cdot 2^{-3}$$

$$= 0 + 0.25 + 0.125 = 0.375$$

Resultado final

$$1011.011_2 = 11.375_{10}$$

Observación adicional: La conversión es finita porque la fracción binaria tiene un número limitado de dígitos; toda fracción binaria finita produce un decimal exacto.

Ejemplo 2: Conversión finita**Convertir 27.34_8 a decimal****Paso 1: Separar parte entera y fraccionaria**

$$27_8 \quad \text{y} \quad 0.34_8$$

Paso 2: Expandir parte entera

$$2 \cdot 8^1 + 7 \cdot 8^0$$

$$= 16 + 7 = 23$$

Paso 3: Expandir parte fraccionaria

$$3 \cdot 8^{-1} + 4 \cdot 8^{-2}$$

$$= 3 \cdot 0.125 + 4 \cdot 0.015625$$

$$= 0.375 + 0.0625 = 0.4375$$

Resultado final

$$27.34_8 = 23.4375_{10}$$

Observación adicional: Las fracciones en base 8 también son potencias de 2, por lo que su representación decimal es exacta.

Ejemplo 3: Conversión periódica**Convertir $0.333\dots_3$ a decimal**

Paso 1: Interpretar la fracción

$$0.333\dots_3$$

Paso 2: Expandir con potencias negativas

$$3 \cdot 3^{-1} + 3 \cdot 3^{-2} + 3 \cdot 3^{-3} + \dots$$

$$= 3 \left(\frac{1}{3} + \frac{1}{9} + \frac{1}{27} + \dots \right)$$

Se trata de una serie geométrica infinita:

$$= 3 \cdot \frac{\frac{1}{3}}{1 - \frac{1}{3}} = 3 \cdot \frac{\frac{1}{3}}{\frac{2}{3}} = 3 \cdot \frac{1}{2} = 1.5$$

Resultado final

$$0.333\dots_3 = 1.5_{10}$$

Observación adicional: Aunque la representación en base 3 es infinita, el valor decimal converge a un número racional exacto.

Ejemplo 4: Conversión periódica**Convertir $0.\bar{1}_3$ a decimal****Paso 1: Expansión**

$$1 \cdot 3^{-1} + 1 \cdot 3^{-2} + 1 \cdot 3^{-3} + \dots$$

$$= \frac{1}{3} + \frac{1}{9} + \frac{1}{27} + \dots$$

Serie geométrica:

$$= \frac{\frac{1}{3}}{1 - \frac{1}{3}} = \frac{\frac{1}{3}}{\frac{2}{3}} = \frac{1}{2}$$

Resultado final

$$0.\bar{1}_3 = 0.5_{10}$$

Observación adicional: La periodicidad en una base distinta puede producir un decimal exacto; este fenómeno ilustra cómo la representación depende de la relación entre los factores primos de la base original y la base destino.

4. Limitaciones fundamentales: representaciones no exactas

No todos los números reales pueden representarse exactamente en todas las bases. Por ejemplo, 0.1_{10} no tiene representación finita en binario. Este fenómeno se debe a que la fracción depende de factores primos distintos de la base. En consecuencia, la representación en binario puede ser infinita o periódica, y el sistema debe aproximarla [11], [12], [38].

En programación, esta limitación explica resultados como:

$$0.1 + 0.2 \neq 0.3$$

cuando se utiliza punto flotante binario. Este comportamiento no constituye un error del lenguaje, sino una consecuencia matemática de la representación digital [7], [11], [12].

5. Estándar IEEE 754 y punto flotante

La mayoría de los lenguajes utilizan el estándar IEEE 754 para representar números reales en binario. Este estándar define:

- 1 bit para el signo.
- Bits para exponente.
- Bits para mantisa.

La representación permite manejar un rango amplio de valores, aunque introduce redondeo y errores acumulativos en operaciones repetidas [11], [12], [34].

Para quien aprende a programar en IS, esta información resulta indispensable al trabajar con:

- Cálculos financieros.

- Indicadores estadísticos.
- Promedios y porcentajes.

6. Buenas prácticas en programación con reales

El manejo responsable de valores reales incluye:

1. Evitar comparaciones directas de igualdad.
2. Utilizar tolerancias: $|a - b| < \varepsilon$.
3. Controlar redondeo antes de mostrar resultados.
4. Documentar precisión esperada según dominio organizacional.

Estas prácticas se alinean con principios de calidad y robustez en ingeniería de software [4], [23], [24].

7. Implicación formativa

La representación de números reales amplía la visión del estudiante sobre la naturaleza finita de los sistemas digitales. Programar con conciencia de precisión fortalece criterio técnico y evita interpretaciones erróneas de resultados en entornos organizacionales. Este aprendizaje trasciende ejercicios académicos y prepara al desarrollador para diseñar soluciones confiables dentro de Sistemas de Información [2], [4], [15].

2.1.5. Operaciones bit a bit, máscaras y estados

Las operaciones bit a bit ofrecen un mecanismo directo para manipular estados codificados. En IS, estados, permisos y banderas pueden representarse en bits para compactar información y habilitar verificaciones rápidas mediante AND, OR, XOR y desplazamientos [13], [34], [37].

Operadores bit a bit.

Las operaciones bit a bit permiten manipular directamente los bits que representan un número entero. En programación, esto permite activar, desactivar o verificar estados codificados dentro de un valor numérico.

1. Operador OR (|) a nivel de bit **Problema:** Dado dos números enteros positivos, combinar sus bits utilizando la operación OR bit a bit y mostrar el resultado.

```

1  Algoritmo OperadorOR
2  Var
3  a, b, resultado : entero
4  Inicio
5  a <- 10          // 1010 en binario
6  b <- 12          // 1100 en binario
7
8  resultado <- a | b
9
10 Escribir("Resultado OR bit a bit : ", resultado)
11 Fin

```

Listing 2.1: Operador OR a nivel de bit

Explicación:

10 (1010)

12 (1100)

OR produce 1110 = 14

Si alguno de los bits es 1, el resultado es 1.

2. Operador AND (&) a nivel de bit **Problema:** Determinar qué bits están activos simultáneamente en dos números.

```

1  Algoritmo OperadorAND
2  Var
3  a, b, resultado : entero
4  inicio
5  a <- 10          // 1010
6  b <- 12          // 1100
7
8  resultado <- a & b
9

```

```
10  Escribir("Resultado AND: ", resultado)
11  Fin
```

Listing 2.2: Operador AND (&) a nivel de bit

Explicación:

Solo los bits que son 1 en ambos números permanecen en 1.

1010

1100

AND produce 1000 = 8

3. Operador XOR (^) Problema: Determinar qué bits son diferentes entre dos números.

```
1  Algoritmo OperadorXOR
2  Var
3  a, b, resultado : entero
4  Inicio
5  a <- 10          // 1010
6  b <- 12          // 1100
7
8  resultado <- a XOR b
9
10 Escribir("Resultado XOR: ", resultado)
11 Fin
```

Listing 2.3: Determininar los bits diferentes entre dos números

Explicación: El resultado es 1 cuando los bits son distintos.

1010

1100

XOR produce 0110 = 6

4. Desplazamiento Izquierdo Problema: Multiplicar un número entero por 2 usando desplazamiento de bits.

```
1  Algoritmo DesplazamientoIzquierdo
2  Var
3  numero, resultado : entero
4  Inicio
5  numero <- 5      // 0101
6
7  resultado <- numero << 1
8
9  Escribir("Resultado desplazamiento izquierdo: ", resultado)
10 Fin
```

Listing 2.4: Multiplicación entero por 2 con desplazamiento

Explicación: Desplazar a la izquierda equivale a multiplicar por 2. $0101 \ll 1$ produce $1010 = 10$

5. Desplazamiento Derecho Problema: Dividir un número entero entre 2 usando desplazamiento de bits.

```
1  Algoritmo DesplazamientoDerecho
2  Var
3  numero, resultado : ENTERO
4  Inicio
5  numero <- 8      // 1000
6
7  resultado <- numero >> 1
8
9  Escribir("Resultado desplazamiento derecho: ", resultado)
10 Fin
```

Listing 2.5: División de enteros para 2 por desplazamiento

Explicación: Desplazar a la derecha equivale a dividir entre 2.

$1000 \gg 1$ produce $0100 = 4$

6. Diferencia entre | y || Problema: Mostrar la diferencia entre OR lógico y OR bit a bit.

```
1  Algoritmo DiferenciaOR
2  Var
3  a, b : entero
4  inicio
5  a <- 5
6  b <- 0
7
8  Si (a | b) > 0 Entonces
9  Escribir("OR bit a bit produjo valor distinto de 0")
10 FinSi
11
12 Si (a > 0) OR (b > 0) Entonces
13 Escribir("OR logico verdadero")
14 FinSi
15 Fin
```

Listing 2.6: Diferencia entre OR bit a bit y OR lógico

Explicación:

- | combina bits.
- || evalúa condiciones lógicas.
- || produce VERDADERO o FALSO.
- | produce un número entero.

Estos algoritmos permiten que el estudiante observe cómo los números se manipulan a nivel binario, reforzando la relación entre representación digital y programación estructurada.

Para quien aprende a programar, las máscaras introducen un repertorio práctico de técnicas. Verificar si un estado está activo, activar o desactivar banderas y construir combinaciones controladas permiten ejercitar operadores lógicos y condicionales con un sentido operativo inmediato [5], [7], [35].

El uso de estados codificados se relaciona con diseño de datos. Definir qué bit representa qué condición requiere disciplina de especificación y documentación, lo que favorece programas más legibles y sostenibles. Esta práctica se conecta con principios de calidad y mantenibilidad en ingeniería de software [4], [23], [24].

Destacando la dimensión organizacional, las máscaras permiten modelar reglas sin inflar estructuras. En sistemas con gran cantidad de registros, representar atributos booleanos como bits reduce almacenamiento y acelera filtrado, lo que incide en desempeño del SI [2], [34], [36].

En síntesis, aprender operaciones bit a bit amplía la visión del lector sobre cómo el software controla estados reales y cómo una representación cuidadosa habilita soluciones robustas y eficientes [5], [7], [13].

2.2. Representación de Información en Sistemas Computacionales

La representación de la información constituye el mecanismo mediante el cual los sistemas computacionales transforman entidades del mundo real en estructuras tratables por hardware y software. Texto, números, imágenes, audio, fechas y estados deben expresarse mediante convenciones formales que permitan almacenamiento, transmisión, procesamiento y recuperación sin pérdida indebida de significado. En consecuencia, comprender cómo se representan los datos en un sistema no solo permite interpretar mejor su comportamiento interno, sino también anticipar restricciones de precisión, compatibilidad y validación que afectan directamente la confiabilidad del software [6], [10], [34].

Desde la perspectiva formativa, este tema permite vincular fundamentos de arquitectura, estructuras de datos y programación con problemas reales de calidad e integridad de la información. Elegir una codificación, definir un rango numérico, establecer una política de redondeo o interpretar correctamente un formato binario no son decisiones aisladas, sino operaciones que inciden en la corrección de cálculos, en la consistencia entre módulos y en la robustez del sistema frente a entradas inválidas o ambiguas. Por ello, estudiar la representación de la información en sistemas computacionales prepara al lector para programar con mayor precisión técnica, mejor criterio de diseño y una comprensión más profunda de la relación entre dato, formato e interpretación [11], [12], [24].

2.2.1. Representación de texto, números y contenidos multimedia

Los SI manejan contenidos heterogéneos: nombres, direcciones, descripciones, cantidades, fechas e incluso imágenes o audio. Cada tipo de contenido exige una codificación adecuada para preservar significado, permitir búsquedas y sostener reglas de negocio [2], [27], [28].

La representación de texto y números constituye una fuente frecuente de defectos cuando se programa sin criterios. Incompatibilidades de codificación, supuestos sobre longitudes o errores por redondeo pueden degradar integridad del sistema y generar resultados que usuarios interpretan como inconsistencias operativas [6], [11], [12].

Para el aprendizaje, estos contenidos permiten ejercicios orientados a IS: normalización de entradas textuales, validación de campos, tratamiento de identificadores, cálculo de métricas y generación de reportes. Cada ejercicio vincula estructuras de datos con reglas organizacionales [1], [5], [7].

De manera complementaria, la codificación de contenidos multimedia aparece en escenarios de gestión documental y sistemas que almacenan evidencias. Aunque su implementación completa suele ser avanzada, una introducción conceptual motiva al estudiante a reconocer que un archivo representa bytes con formato, cabeceras y restricciones de interpretación [2], [10], [34].

Este bloque se organiza en cinco líneas: codificación de caracteres, enteros, reales, formatos multimedia y representación temporal. Cada línea ofrece un motivo técnico para programar con precisión y un puente hacia prácticas de validación y prueba [4], [23], [24].

Codificación de caracteres

La codificación de caracteres define cómo símbolos textuales se representan en bytes. En IS, texto incluye nombres, descripciones y mensajes; una codificación inadecuada introduce pérdida de información, errores de visualización y fallos en comparaciones o búsquedas [2], [6], [10].

En programación, el lector debe reconocer que una cadena es una secuencia de códigos y que operaciones como longitud, recorte o concatenación dependen de la codificación subyacente. Esta perspectiva orienta a diseñar validaciones coherentes y evita suposiciones sobre “un carácter igual a un byte” en entornos multilingües [5], [7], [35].

La codificación también afecta intercambio entre sistemas. Cuando un SI integra fuentes diversas, la uniformidad en codificación resulta indispensable para mantener consistencia y evitar corrupción de datos en almacenamiento o transmisión [2], [27], [28].

No obstante, el problema no se limita a visualización. Una codificación inconsistente

puede alterar validaciones, ordenamientos y reglas de negocio basadas en texto, lo que se traduce en comportamientos incorrectos del sistema [1], [4], [6].

Dentro de los esquemas históricos de codificación, el código estándar americano para el intercambio de información (*American Standard Code for Information Interchange*, ASCII) ocupa un lugar central por haber establecido una correspondencia estandarizada entre valores numéricos y caracteres de uso común. En su versión original de 7 bits, ASCII permite representar 128 códigos, distribuidos entre caracteres de control y caracteres imprimibles. Los primeros intervienen en tareas asociadas al formato, la transmisión o el control del flujo de información, mientras que los segundos incluyen letras, dígitos, signos de puntuación y otros símbolos visibles. Desde la perspectiva de la programación, comprender esta organización permite interpretar cómo el texto es almacenado, procesado y transmitido en los sistemas digitales, y facilita el análisis de operaciones sobre cadenas, validaciones de entrada, tratamiento de caracteres especiales e interoperabilidad entre componentes de software [6], [10], [35].

Cuadro 2.1: Código ASCII estándar (0–127), organizado en dos bloques para consulta editorial y técnica.

Bloque 1				Bloque 2			
Dec	Hex	Nombre	Rep.	Dec	Hex	Nombre	Rep.
Tabla ASCII completa (códigos de control y caracteres imprimibles)							
0	00	NUL (Null)	—	64	40	Arroba	@
1	01	SOH	—	65	41	Letra A	A
2	02	STX	—	66	42	Letra B	B
3	03	ETX	—	67	43	Letra C	C
4	04	EOT	—	68	44	Letra D	D
5	05	ENQ	—	69	45	Letra E	E
6	06	ACK	—	70	46	Letra F	F
7	07	BEL	—	71	47	Letra G	G
8	08	BS	—	72	48	Letra H	H
9	09	TAB	—	73	49	Letra I	I
10	0A	LF	—	74	4A	Letra J	J
11	0B	VT	—	75	4B	Letra K	K
12	0C	FF	—	76	4C	Letra L	L
13	0D	CR	—	77	4D	Letra M	M

Continúa en la página siguiente

Tabla 2.1 (continuación)							
Bloque 1				Bloque 2			
Dec	Hex	Nombre	Rep.	Dec	Hex	Nombre	Rep.
14	0E	SO	—	78	4E	Letra N	N
15	0F	SI	—	79	4F	Letra O	O
16	10	DLE	—	80	50	Letra P	P
17	11	DC1	—	81	51	Letra Q	Q
18	12	DC2	—	82	52	Letra R	R
19	13	DC3	—	83	53	Letra S	S
20	14	DC4	—	84	54	Letra T	T
21	15	NAK	—	85	55	Letra U	U
22	16	SYN	—	86	56	Letra V	V
23	17	ETB	—	87	57	Letra W	W
24	18	CAN	—	88	58	Letra X	X
25	19	EM	—	89	59	Letra Y	Y
26	1A	SUB	—	90	5A	Letra Z	Z
27	1B	ESC	—	91	5B	Corchete izq.	[
28	1C	FS	—	92	5C	Barra inv.	\
29	1D	GS	—	93	5D	Corchete der.	]
30	1E	RS	—	94	5E	Circunflejo	^
31	1F	US	—	95	5F	Guion bajo	_
32	20	Espacio	□	96	60	Acento grave	‘
33	21	!	!	97	61	Letra a	a
34	22	”	”	98	62	Letra b	b
35	23	#	#	99	63	Letra c	c
36	24	\$	\$	100	64	Letra d	d
37	25	%	%	101	65	Letra e	e
38	26	&	&	102	66	Letra f	f
39	27	,	,	103	67	Letra g	g
40	28	(	(	104	68	Letra h	h
41	29	)	)	105	69	Letra i	i
42	2A	*	*	106	6A	Letra j	j
43	2B	+	+	107	6B	Letra k	k
44	2C	,	,	108	6C	Letra l	l
45	2D	-	-	109	6D	Letra m	m

Continúa en la página siguiente

Tabla 2.1 (continuación)							
Bloque 1				Bloque 2			
Dec	Hex	Nombre	Rep.	Dec	Hex	Nombre	Rep.
46	2E	.	.	110	6E	Letra n	n
47	2F	/	/	111	6F	Letra o	o
48	30	0	0	112	70	Letra p	p
49	31	1	1	113	71	Letra q	q
50	32	2	2	114	72	Letra r	r
51	33	3	3	115	73	Letra s	s
52	34	4	4	116	74	Letra t	t
53	35	5	5	117	75	Letra u	u
54	36	6	6	118	76	Letra v	v
55	37	7	7	119	77	Letra w	w
56	38	8	8	120	78	Letra x	x
57	39	9	9	121	79	Letra y	y
58	3A	:	:	122	7A	Letra z	z
59	3B	;	;	123	7B	{	{
60	3C	<	<	124	7C		
61	3D	=	=	125	7D	}	}
62	3E	>	>	126	7E	~	~
63	3F	?	?	127	7F	DEL	—

A medida que aumentó la necesidad de representar caracteres propios de distintas lenguas y símbolos adicionales no contemplados por ASCII estándar, surgieron extensiones basadas en 8 bits que ampliaron el rango de valores posibles de 128 a 255. Sin embargo, estos códigos extendidos no conforman un único estándar universal, sino que dependen de páginas de códigos o esquemas concretos, como ISO-8859-1 o Windows-1252. En consecuencia, un mismo valor numérico en ese intervalo puede corresponder a caracteres distintos según la codificación adoptada, lo que introduce riesgos de ambigüedad, incompatibilidad y corrupción de datos cuando no existe uniformidad entre los sistemas que intercambian información. Esta situación resulta especialmente relevante en aplicaciones multilingües, donde la representación correcta de vocales acentuadas, signos invertidos, letras especiales y otros símbolos textuales condiciona tanto la integridad de los datos como la calidad de la interacción con el usuario [6], [10], [39].

La Tabla 2.2 presenta, con fines de referencia técnica, una codificación extendida de 8 bits basada en ISO-8859-1, lo que permite comparar su estructura con la del ASCII estándar mostrado previamente. En ella se observa la incorporación de caracteres acentuados, signos tipográficos, símbolos monetarios y grafías utilizadas en diversas lenguas europeas, elementos que no forman parte del repertorio ASCII de 7 bits. Su revisión permite comprender tanto la utilidad histórica de estas extensiones como sus limitaciones en términos de portabilidad y compatibilidad, especialmente cuando diferentes plataformas o aplicaciones

interpretan un mismo valor binario bajo esquemas de codificación distintos [2], [27], [39].

Cuadro 2.2: Conjunto ASCII extendido (128–255), presentado según la codificación ISO-8859-1 para fines de referencia técnica y editorial.

Bloque 1				Bloque 2			
Dec	Hex	Nombre	Rep.	Dec	Hex	Nombre	Rep.
Tabla ASCII extendido / ISO-8859-1 (128–255)							
128	80	Carácter de control	—	192	C0	Letra A con acento grave	À
129	81	Carácter de control	—	193	C1	Letra A con acento agudo	Á
130	82	Carácter de control	—	194	C2	Letra A con circunflejo	Â
131	83	Carácter de control	—	195	C3	Letra A con tilde	Ã
132	84	Carácter de control	—	196	C4	Letra A con diéresis	Ä
133	85	Carácter de control	—	197	C5	Letra A con anillo	Å
134	86	Carácter de control	—	198	C6	Letra ae	Æ
135	87	Carácter de control	—	199	C7	Letra C con cedilla	Ç
136	88	Carácter de control	—	200	C8	Letra E con acento grave	È
137	89	Carácter de control	—	201	C9	Letra E con acento agudo	É
138	8A	Carácter de control	—	202	CA	Letra E con circunflejo	Ê
139	8B	Carácter de control	—	203	CB	Letra E con diéresis	Ë
140	8C	Carácter de control	—	204	CC	Letra I con acento grave	Ì
141	8D	Carácter de control	—	205	CD	Letra I con acento agudo	Í
142	8E	Carácter de control	—	206	CE	Letra I con circunflejo	Î
143	8F	Carácter de control	—	207	CF	Letra I con diéresis	Ï
144	90	Carácter de control	—	208	D0	Letra ETH	Ð
145	91	Carácter de control	—	209	D1	Letra N con tilde	Ñ
146	92	Carácter de control	—	210	D2	Letra O con acento grave	Ò
147	93	Carácter de control	—	211	D3	Letra O con acento agudo	Ó
148	94	Carácter de control	—	212	D4	Letra O con circunflejo	Ô
149	95	Carácter de control	—	213	D5	Letra O con tilde	Õ
150	96	Carácter de control	—	214	D6	Letra O con diéresis	Ö
151	97	Carácter de control	—	215	D7	Signo de multiplicación	×
152	98	Carácter de control	—	216	D8	Letra O con barra	Ø
153	99	Carácter de control	—	217	D9	Letra U con acento grave	Ù

Continúa en la página siguiente

Tabla 2.2 (continuación)							
Bloque 1				Bloque 2			
Dec	Hex	Nombre	Rep.	Dec	Hex	Nombre	Rep.
154	9A	Carácter de control	—	218	DA	Letra U con acento agudo	Ũ
155	9B	Carácter de control	—	219	DB	Letra U con circunflejo	Û
156	9C	Carácter de control	—	220	DC	Letra U con diéresis	Ü
157	9D	Carácter de control	—	221	DD	Letra Y con acento agudo	Ý
158	9E	Carácter de control	—	222	DE	Letra THORN	Þ
159	9F	Carácter de control	—	223	DF	Letra eszett	ß
160	A0	Espacio de no separación	□	224	E0	Letra a con acento grave	à
161	A1	Signo de exclamación invertido	¡	225	E1	Letra a con acento agudo	á
162	A2	Signo de centavo	¢	226	E2	Letra a con circunflejo	â
163	A3	Signo de libra esterlina	£	227	E3	Letra a con tilde	ã
164	A4	Signo monetario	¤	228	E4	Letra a con diéresis	ä
165	A5	Signo de yen	¥	229	E5	Letra a con anillo	å
166	A6	Barra vertical partida	¦	230	E6	Letra ae	æ
167	A7	Signo de sección	§	231	E7	Letra c con cedilla	ç
168	A8	Diéresis	¨	232	E8	Letra e con acento grave	è
169	A9	Signo de copyright	©	233	E9	Letra e con acento agudo	é
170	AA	Indicador ordinal femenino	ª	234	EA	Letra e con circunflejo	ê
171	AB	Signo de comillas angulares izquierdas	«	235	EB	Letra e con diéresis	ë
172	AC	Signo de negación	¬	236	EC	Letra i con acento grave	ì
173	AD	Guion discrecional		237	ED	Letra i con acento agudo	í
174	AE	Signo de marca registrada	®	238	EE	Letra i con circunflejo	î
175	AF	Barra superior	ˉ	239	EF	Letra i con diéresis	ï
176	B0	Signo de grado	°	240	F0	Letra eth	ð
177	B1	Signo más/menos	±	241	F1	Letra n con tilde	ñ
178	B2	Superíndice dos	²	242	F2	Letra o con acento grave	ò
179	B3	Superíndice tres	³	243	F3	Letra o con acento agudo	ó
180	B4	Acento agudo	´	244	F4	Letra o con circunflejo	ô
181	B5	Signo de micro	μ	245	F5	Letra o con tilde	õ
182	B6	Signo de párrafo	¶	246	F6	Letra o con diéresis	ö

Continúa en la página siguiente

Tabla 2.2 (continuación)							
Bloque 1				Bloque 2			
Dec	Hex	Nombre	Rep.	Dec	Hex	Nombre	Rep.
183	B7	Punto medio	·	247	F7	Signo de división	÷
184	B8	Cedilla	¸	248	F8	Letra o con barra	ø
185	B9	Superíndice uno	¹	249	F9	Letra u con acento grave	ù
186	BA	Indicador ordinal masculino	º	250	FA	Letra u con acento agudo	ú
187	BB	Signo de comillas angulares derechas	»	251	FB	Letra u con circunflejo	û
188	BC	Fracción un cuarto	¼	252	FC	Letra u con diéresis	ü
189	BD	Fracción un medio	½	253	FD	Letra y con acento agudo	ý
190	BE	Fracción tres cuartos	¾	254	FE	Letra thorn	þ
191	BF	Signo de interrogación invertido	¿	255	FF	Letra y con diéresis	ÿ

En la práctica cotidiana, especialmente en entornos Windows, se emplea de manera imprecisa la expresión “código ASCII” para referirse a valores numéricos superiores a 127 que generan caracteres adicionales mediante combinaciones como **Alt + número**. No obstante, desde una perspectiva formal, el estándar ASCII define exclusivamente un conjunto de 128 símbolos codificados en 7 bits, correspondientes al rango decimal 0–127. Los valores comprendidos entre 128 y 255 no pertenecen al estándar ASCII, sino que forman parte de extensiones dependientes de codificación, tales como ISO-8859-1 o Windows-1252, las cuales no son equivalentes entre sí y asignan representaciones distintas a los mismos valores numéricos. Tal como se evidencia en la Tabla 2.3, un mismo byte puede corresponder a diferentes símbolos según el esquema utilizado, lo que invalida cualquier interpretación universal de dichos códigos como “ASCII”. Esta confusión conceptual resulta especialmente crítica en programación, ya que la correcta interpretación, almacenamiento y transmisión de datos textuales depende de manera estricta de la codificación explícitamente definida en el sistema [6], [35], [39].

Cuadro 2.3: Comparación de representaciones en ASCII, Unicode, Windows-1252 y codificaciones Unicode para un conjunto ampliado de caracteres

Car.	AS-CII	Unicode	Win-1252	UTF-8	UTF-16	UTF-32	Observación
0	48	U+0030	48	30	0030	00000030	Coinciden en todos
1	49	U+0031	49	31	0031	00000031	Coinciden en todos
2	50	U+0032	50	32	0032	00000032	Coinciden en todos
3	51	U+0033	51	33	0033	00000033	Coinciden en todos
...	...	...	...	...	...	...	...
9	57	U+0039	57	39	0039	00000039	Coinciden en todos
A	65	U+0041	65	41	0041	00000041	ASCII básico
B	66	U+0042	66	42	0042	00000042	ASCII básico
C	67	U+0043	67	43	0043	00000043	ASCII básico
...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...
Z	90	U+005A	90	5A	005A	0000005A	ASCII básico
a	97	U+0061	97	61	0061	00000061	ASCII básico

Continúa en la página siguiente

Cuadro 2.3: Comparación de representaciones en ASCII, Unicode, Windows-1252 y codificaciones Unicode para un conjunto ampliado de caracteres*Continuación de la página anterior*

Car.	AS-CII	Unicode	Win-1252	UTF-8	UTF-16	UTF-32	Observación
b	98	U+0062	98	62	0062	00000062	ASCII básico
c	99	U+0063	99	63	0063	00000063	ASCII básico
...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...
z	122	U+007A	122	7A	007A	0000007A	ASCII básico
Á	—	U+00C1	193	C3 81	00C1	000000C1	Vocal mayúscula con acento agudo
É	—	U+00C9	201	C3 89	00C9	000000C9	Vocal mayúscula con acento agudo
Í	—	U+00CD	205	C3 8D	00CD	000000CD	Vocal mayúscula con acento agudo
Ó	—	U+00D3	211	C3 93	00D3	000000D3	Vocal mayúscula con acento agudo
Ú	—	U+00DA	218	C3 9A	00DA	000000DA	Vocal mayúscula con acento agudo
á	—	U+00E1	225	C3 A1	00E1	000000E1	Vocal minúscula con acento agudo
é	—	U+00E9	233	C3 A9	00E9	000000E9	Vocal minúscula con acento agudo
í	—	U+00ED	237	C3 AD	00ED	000000ED	Vocal minúscula con acento agudo
ó	—	U+00F3	243	C3 B3	00F3	000000F3	Vocal minúscula con acento agudo
ú	—	U+00FA	250	C3 BA	00FA	000000FA	Vocal minúscula con acento agudo
À	—	U+00C0	192	C3 80	00C0	000000C0	Vocal mayúscula con acento grave
È	—	U+00C8	200	C3 88	00C8	000000C8	Vocal mayúscula con acento grave
Ì	—	U+00CC	204	C3 8C	00CC	000000CC	Vocal mayúscula con acento grave
Ò	—	U+00D2	210	C3 92	00D2	000000D2	Vocal mayúscula con acento grave
Û	—	U+00D9	217	C3 99	00D9	000000D9	Vocal mayúscula con acento grave
à	—	U+00E0	224	C3 A0	00E0	000000E0	Vocal minúscula con acento grave
è	—	U+00E8	232	C3 A8	00E8	000000E8	Vocal minúscula con acento grave
ì	—	U+00EC	236	C3 AC	00EC	000000EC	Vocal minúscula con acento grave
ò	—	U+00F2	242	C3 B2	00F2	000000F2	Vocal minúscula con acento grave
ù	—	U+00F9	249	C3 B9	00F9	000000F9	Vocal minúscula con acento grave
Â	—	U+00C2	194	C3 82	00C2	000000C2	Vocal mayúscula con circunflejo
Ê	—	U+00CA	202	C3 8A	00CA	000000CA	Vocal mayúscula con circunflejo
Î	—	U+00CE	206	C3 8E	00CE	000000CE	Vocal mayúscula con circunflejo
Ô	—	U+00D4	212	C3 94	00D4	000000D4	Vocal mayúscula con circunflejo
Û	—	U+00DB	219	C3 9B	00DB	000000DB	Vocal mayúscula con circunflejo
â	—	U+00E2	226	C3 A2	00E2	000000E2	Vocal minúscula con circunflejo
ê	—	U+00EA	234	C3 AA	00EA	000000EA	Vocal minúscula con circunflejo
î	—	U+00EE	238	C3 AE	00EE	000000EE	Vocal minúscula con circunflejo
ô	—	U+00F4	244	C3 B4	00F4	000000F4	Vocal minúscula con circunflejo
û	—	U+00FB	251	C3 BB	00FB	000000FB	Vocal minúscula con circunflejo
Ä	—	U+00C4	196	C3 84	00C4	000000C4	Vocal mayúscula con diéresis
Ë	—	U+00CB	203	C3 8B	00CB	000000CB	Vocal mayúscula con diéresis
Ï	—	U+00CF	207	C3 8F	00CF	000000CF	Vocal mayúscula con diéresis
Ö	—	U+00D6	214	C3 96	00D6	000000D6	Vocal mayúscula con diéresis
Ü	—	U+00DC	220	C3 9C	00DC	000000DC	Vocal mayúscula con diéresis
ä	—	U+00E4	228	C3 A4	00E4	000000E4	Vocal minúscula con diéresis

Continúa en la página siguiente

Cuadro 2.3: Comparación de representaciones en ASCII, Unicode, Windows-1252 y codificaciones Unicode para un conjunto ampliado de caracteres*Continuación de la página anterior*

Car.	AS- CII	Unicode	Win- 1252	UTF-8	UTF-16	UTF-32	Observación
ë	—	U+00EB	235	C3 AB	00EB	000000EB	Vocal minúscula con diéresis
ï	—	U+00EF	239	C3 AF	00EF	000000EF	Vocal minúscula con diéresis
ö	—	U+00F6	246	C3 B6	00F6	000000F6	Vocal minúscula con diéresis
ü	—	U+00FC	252	C3 BC	00FC	000000FC	Vocal minúscula con diéresis
Ã	—	U+00C3	195	C3 83	00C3	000000C3	Vocal mayúscula con tilde
Õ	—	U+00D5	213	C3 95	00D5	000000D5	Vocal mayúscula con tilde
ã	—	U+00E3	227	C3 A3	00E3	000000E3	Vocal minúscula con tilde
õ	—	U+00F5	245	C3 B5	00F5	000000F5	Vocal minúscula con tilde
Å	—	U+00C5	197	C3 85	00C5	000000C5	Vocal mayúscula con anillo
å	—	U+00E5	229	C3 A5	00E5	000000E5	Vocal minúscula con anillo
Æ	—	U+00C6	198	C3 86	00C6	000000C6	Ligadura vocálica mayúscula
æ	—	U+00E6	230	C3 A6	00E6	000000E6	Ligadura vocálica minúscula
Ñ	—	U+00D1	209	C3 91	00D1	000000D1	Letra extendida del español
ñ	—	U+00F1	241	C3 B1	00F1	000000F1	Letra extendida del español
Ç	—	U+00C7	199	C3 87	00C7	000000C7	Letra con cedilla mayúscula
ç	—	U+00E7	231	C3 A7	00E7	000000E7	Letra con cedilla minúscula
Ý	—	U+00DD	221	C3 9D	00DD	000000DD	Letra mayúscula con acento agudo
ý	—	U+00FD	253	C3 BD	00FD	000000FD	Letra minúscula con acento agudo
ÿ	—	U+00FF	255	C3 BF	00FF	000000FF	Letra minúscula con diéresis
Ø	—	U+00D8	216	C3 98	00D8	000000D8	Letra mayúscula con barra
ø	—	U+00F8	248	C3 B8	00F8	000000F8	Letra minúscula con barra
Ð	—	U+00D0	208	C3 90	00D0	000000D0	Letra et mayúscula
ð	—	U+00F0	240	C3 B0	00F0	000000F0	Letra et minúscula
Þ	—	U+00DE	222	C3 9E	00DE	000000DE	Letra thorn mayúscula
þ	—	U+00FE	254	C3 BE	00FE	000000FE	Letra thorn minúscula
ß	—	U+00DF	223	C3 9F	00DF	000000DF	Eszett alemana
Α	—	U+0391	—	CE 91	0391	00000391	Alfa mayúscula griega; no existe en Win-1252
Β	—	U+0392	—	CE 92	0392	00000392	Beta mayúscula griega
Γ	—	U+0393	—	CE 93	0393	00000393	Gamma mayúscula griega
Δ	—	U+0394	—	CE 94	0394	00000394	Delta mayúscula griega
Ε	—	U+0395	—	CE 95	0395	00000395	Épsilon mayúscula griega
Ζ	—	U+0396	—	CE 96	0396	00000396	Zeta mayúscula griega
Η	—	U+0397	—	CE 97	0397	00000397	Eta mayúscula griega
Θ	—	U+0398	—	CE 98	0398	00000398	Theta mayúscula griega
Ι	—	U+0399	—	CE 99	0399	00000399	Iota mayúscula griega
Κ	—	U+039A	—	CE 9A	039A	0000039A	Kappa mayúscula griega
Λ	—	U+039B	—	CE 9B	039B	0000039B	Lambda mayúscula griega
Μ	—	U+039C	—	CE 9C	039C	0000039C	Mi mayúscula griega
Ν	—	U+039D	—	CE 9D	039D	0000039D	Ni mayúscula griega
Ξ	—	U+039E	—	CE 9E	039E	0000039E	Xi mayúscula griega
Ο	—	U+039F	—	CE 9F	039F	0000039F	Ómicron mayúscula griega

Continúa en la página siguiente

Cuadro 2.3: Comparación de representaciones en ASCII, Unicode, Windows-1252 y codificaciones Unicode para un conjunto ampliado de caracteres*Continuación de la página anterior*

Car.	AS-CII	Unicode	Win-1252	UTF-8	UTF-16	UTF-32	Observación
Π	—	U+03A0	—	CE A0	03A0	000003A0	Pi mayúscula griega
Ρ	—	U+03A1	—	CE A1	03A1	000003A1	Rho mayúscula griega
Σ	—	U+03A3	—	CE A3	03A3	000003A3	Sigma mayúscula griega
Τ	—	U+03A4	—	CE A4	03A4	000003A4	Tau mayúscula griega
Υ	—	U+03A5	—	CE A5	03A5	000003A5	Ípsilon mayúscula griega
Φ	—	U+03A6	—	CE A6	03A6	000003A6	Fi mayúscula griega
Χ	—	U+03A7	—	CE A7	03A7	000003A7	Ji mayúscula griega
Ψ	—	U+03A8	—	CE A8	03A8	000003A8	Psi mayúscula griega
Ω	—	U+03A9	—	CE A9	03A9	000003A9	Omega mayúscula griega
α	—	U+03B1	—	CE B1	03B1	000003B1	Alfa minúscula griega
β	—	U+03B2	—	CE B2	03B2	000003B2	Beta minúscula griega
γ	—	U+03B3	—	CE B3	03B3	000003B3	Gamma minúscula griega
δ	—	U+03B4	—	CE B4	03B4	000003B4	Delta minúscula griega
ε	—	U+03B5	—	CE B5	03B5	000003B5	Épsilon minúscula griega
ζ	—	U+03B6	—	CE B6	03B6	000003B6	Zeta minúscula griega
η	—	U+03B7	—	CE B7	03B7	000003B7	Eta minúscula griega
θ	—	U+03B8	—	CE B8	03B8	000003B8	Theta minúscula griega
ι	—	U+03B9	—	CE B9	03B9	000003B9	Iota minúscula griega
κ	—	U+03BA	—	CE BA	03BA	000003BA	Kappa minúscula griega
λ	—	U+03BB	—	CE BB	03BB	000003BB	Lambda minúscula griega
μ	—	U+03BC	—	CE BC	03BC	000003BC	Mi minúscula griega
ν	—	U+03BD	—	CE BD	03BD	000003BD	Ni minúscula griega
ξ	—	U+03BE	—	CE BE	03BE	000003BE	Xi minúscula griega
ο	—	U+03BF	—	CE BF	03BF	000003BF	Ómicron minúscula griega
π	—	U+03C0	—	CF 80	03C0	000003C0	Pi minúscula griega
ρ	—	U+03C1	—	CF 81	03C1	000003C1	Rho minúscula griega
ς	—	U+03C2	—	CF 82	03C2	000003C2	Sigma final minúscula griega
σ	—	U+03C3	—	CF 83	03C3	000003C3	Sigma minúscula griega
τ	—	U+03C4	—	CF 84	03C4	000003C4	Tau minúscula griega
υ	—	U+03C5	—	CF 85	03C5	000003C5	Ípsilon minúscula griega
φ	—	U+03C6	—	CF 86	03C6	000003C6	Fi minúscula griega
χ	—	U+03C7	—	CF 87	03C7	000003C7	Ji minúscula griega
ψ	—	U+03C8	—	CF 88	03C8	000003C8	Psi minúscula griega
ω	—	U+03C9	—	CF 89	03C9	000003C9	Omega minúscula griega
€	—	U+20AC	128	E2 82 AC	20AC	000020AC	No pertenece a ASCII; símbolo monetario moderno
£	—	U+00A3	163	C2 A3	00A3	000000A3	Símbolo monetario presente en Win-1252
¥	—	U+00A5	165	C2 A5	00A5	000000A5	Símbolo monetario presente en Win-1252
¢	—	U+00A2	162	C2 A2	00A2	000000A2	Símbolo monetario presente en Win-1252
©	—	U+00A9	169	C2 A9	00A9	000000A9	Símbolo legal presente en Win-1252
®	—	U+00AE	174	C2 AE	00AE	000000AE	Marca registrada
°	—	U+00B0	176	C2 B0	00B0	000000B0	Signo de grado

Continúa en la página siguiente

Cuadro 2.3: Comparación de representaciones en ASCII, Unicode, Windows-1252 y codificaciones Unicode para un conjunto ampliado de caracteres*Continuación de la página anterior*

Car.	AS- CII	Unicode	Win- 1252	UTF-8	UTF-16	UTF-32	Observación
±	—	U+00B1	177	C2 B1	00B1	000000B1	Signo más/menos
μ	—	U+00B5	181	C2 B5	00B5	000000B5	Signo micro en Win-1252; distinto de la letra griega
¿	—	U+00BF	191	C2 BF	00BF	000000BF	Signo de interrogación invertido
¡	—	U+00A1	161	C2 A1	00A1	000000A1	Signo de exclamación invertido
😊	—	U+1F600	—	F0 9F 98 80	D83D DE00	0001F600	Cara sonriente
😄	—	U+1F601	—	F0 9F 98 81	D83D DE01	0001F601	Cara sonriendo con ojos sonrientes
😂	—	U+1F602	—	F0 9F 98 82	D83D DE02	0001F602	Cara llorando de risa
🤩	—	U+1F923	—	F0 9F A4 A3	D83E DD23	0001F923	Cara revolcándose de risa
😊	—	U+1F60A	—	F0 9F 98 8A	D83D DE0A	0001F60A	Cara sonriente con ojos sonrientes
😍	—	U+1F60D	—	F0 9F 98 8D	D83D DE0D	0001F60D	Cara sonriente con ojos de corazón
😎	—	U+1F60E	—	F0 9F 98 8E	D83D DE0E	0001F60E	Cara con gafas de sol
😓	—	U+1F622	—	F0 9F 98 A2	D83D DE22	0001F622	Cara llorando
😭	—	U+1F62D	—	F0 9F 98 AD	D83D DE2D	0001F62D	Cara llorando intensamente
😡	—	U+1F621	—	F0 9F 98 A1	D83D DE21	0001F621	Cara enfadada
👍	—	U+1F44D	—	F0 9F 91 8D	D83D DC4D	0001F44D	Pulgar hacia arriba
👉	—	U+1F44E	—	F0 9F 91 8E	D83D DC4E	0001F44E	Pulgar hacia abajo
👏	—	U+1F44F	—	F0 9F 91 8F	D83D DC4F	0001F44F	Manos aplaudiendo
🙏	—	U+1F64F	—	F0 9F 99 8F	D83D DE4F	0001F64F	Manos en plegaria
❤️	—	U+2764 U+FE0F	—	E2 9D A4 EF B8 8F	2764 FE0F	00002764 0000FE0F	Corazón rojo con selector de variación
💔	—	U+1F494	—	F0 9F 92 94	D83D DC94	0001F494	Corazón roto
🔥	—	U+1F525	—	F0 9F 94 A5	D83D DD25	0001F525	Fuego
★	—	U+2B50	—	E2 AD 90	2B50	00002B50	Estrella
✅	—	U+2705	—	E2 9C 85	2705	00002705	Marca de verificación
❌	—	U+274C	—	E2 9D 8C	274C	0000274C	Marca de cruz

Continúa en la página siguiente

Cuadro 2.3: Comparación de representaciones en ASCII, Unicode, Windows-1252 y codificaciones Unicode para un conjunto ampliado de caracteres*Continuación de la página anterior*

Car.	AS- CII	Unicode	Win- 1252	UTF-8	UTF-16	UTF-32	Observación
	—	U+26A0 U+FE0F	—	E2 9A A0 EF B8 8F	26A0 FE0F	000026A0 0000FE0F	Señal de advertencia
	—	U+1F4A1	—	F0 9F 92 A1	D83D DCA1	0001F4A1	Bombilla
	—	U+1F4D8	—	F0 9F 93 98	D83D DCD8	0001F4D8	Libro azul
	—	U+1F4BB	—	F0 9F 92 BB	D83D DCBB	0001F4BB	Computador portátil

En síntesis, la codificación de caracteres constituye un componente fundamental en la representación y procesamiento de información textual dentro de los sistemas informáticos. Su comprensión no solo permite interpretar correctamente cómo los datos son almacenados y transmitidos, sino que también resulta determinante para garantizar la integridad, consistencia y validez de las operaciones que dependen del tratamiento de texto. En el ámbito de la programación, asumir que los caracteres poseen una representación subyacente específica posibilita diseñar soluciones más robustas, portables y seguras, especialmente en entornos donde intervienen múltiples lenguajes, plataformas y configuraciones regionales. Por tanto, el dominio de estos conceptos se convierte en un requisito esencial para el desarrollo de aplicaciones modernas, en las que la correcta gestión de la información textual impacta directamente en la calidad del software y en la experiencia del usuario [2], [6], [35].

Representación de números enteros

Los enteros suelen representarse en formato binario con un número fijo de bits, lo cual determina rangos y comportamiento ante desbordamiento. En IS, cantidades discretas como conteos, identificadores y estados se apoyan en enteros, por lo que el programador debe elegir tamaños coherentes con el dominio [5], [7], [34].

Para aprender a programar, los enteros constituyen un terreno idóneo para entrenar validaciones. Definir rangos, evitar conversiones peligrosas y controlar operaciones de incremento y multiplicación fortalece disciplina y reduce defectos previsibles [4], [6], [23].

El desbordamiento enseña un principio útil: la corrección depende de supuestos explícitos. Cuando el estudiante documenta límites y aplica verificaciones antes de operar, mejora robustez del programa y preserva integridad de resultados [5], [6], [24].

Dentro de este panorama, el uso de enteros se conecta con estructuras de datos. Índices, tamaños de arreglos y conteos de iteraciones dependen de enteros; un error en este nivel altera recorridos y acceso a memoria, generando fallos difíciles de depurar [7], [13], [36].

De manera complementaria, los enteros se usan en codificación de estados y máscaras. Esta práctica integra lógica, representación y control, proporcionando ejercicios motivadores para aprender operadores bit a bit con significado organizacional [2], [13], [37].

Representación de números reales

Los números reales se representan habitualmente mediante punto flotante, siguiendo estándares que definen signo, exponente y mantisa. Esta representación ofrece rango amplio, aunque introduce aproximaciones y errores de redondeo que el programador debe anticipar [11], [12], [34].

En IS, el impacto aparece en cálculos financieros, promedios y métricas de rendimiento. Comparaciones directas entre reales pueden fallar por diferencias mínimas de representación; por ello, resulta aconsejable entrenar criterios de tolerancia, redondeo controlado y pruebas con casos límite [11], [12], [38].

Aprender a programar con estos criterios forma un hábito técnico valioso: evaluar qué tipo numérico se ajusta al problema. En dominios sensibles, el uso de enteros escalados o representaciones decimales controladas reduce ambigüedad y mejora

consistencia de resultados [5], [23], [38].

No obstante, la selección de representación exige coherencia con almacenamiento y salida. Un valor que se calcula con punto flotante puede mostrarse redondeado para usuario y guardarse con otra precisión; el estudiante debe documentar y alinear estas decisiones para evitar discrepancias entre reporte y persistencia [2], [27], [28].

En contraste con el tratamiento de enteros, el punto flotante obliga a pensar en error numérico. Esta práctica fortalece criterio científico-técnico y promueve pruebas que evalúen estabilidad del resultado, no únicamente ejecución del programa [4], [11], [12].

Representación de imágenes, audio y video

Los contenidos multimedia se almacenan como secuencias de bytes que siguen formatos con cabeceras, metadatos y reglas de decodificación. En IS, estos contenidos aparecen en gestión documental, evidencia digital, sistemas educativos y registros audiovisuales asociados a procesos [2], [10], [34].

Para el aprendizaje, los formatos multimedia permiten introducir el concepto de “estructura dentro del archivo”. Leer una cabecera, verificar un identificador de formato y calcular tamaños de bloques enseña programación orientada a especificaciones, con validaciones precisas [5], [6], [7].

La compresión constituye otro punto formativo. Aunque el diseño de códecs es avanzado, reconocer que compresión implica transformar datos para reducir tamaño sin perder propiedades relevantes aporta criterio para elegir formatos en sistemas organizacionales [2], [24], [34].

Destacando el vínculo con integridad, un archivo multimedia corrupto puede ser ilegible o inducir fallas de procesamiento. Validar longitudes, controlar lecturas y verificar condiciones de consistencia se alinea con prácticas de robustez que deben entrenarse desde programación básica [4], [6], [23].

Representación temporal: fechas, horas y zonas

Los SI dependen de tiempo para auditoría, secuenciación de eventos y análisis histórico. Representar fechas y horas exige definir convenciones: zona horaria, formato, precisión y reglas de calendario, dado que discrepancias temporales afectan reportes y control de procesos [1], [2], [27].

En programación, el tiempo se expresa mediante tipos específicos o marcas numéricas. Esta representación obliga a manejar conversiones, comparaciones y cálculos de duración, aportando ejercicios ricos para reforzar estructuras de control y validación [5], [7], [13].

La consistencia temporal requiere validar entradas. Formatos ambiguos, fechas inexistentes o diferencias de zona introducen fallos sutiles; por ello, se recomienda diseñar rutinas de parseo y normalización con pruebas que cubran casos límite [5], [6], [23].

De manera complementaria, la representación temporal se vincula con almacenamiento y consulta. Índices por fecha, filtros por período y agregaciones temporales requieren coherencia entre formato persistido y formato de presentación, lo cual fortalece disciplina de diseño [2], [28], [29].

2.2.2. Riesgos de representación y prácticas de programación robusta

La representación digital ofrece potencia, aunque introduce riesgos previsibles cuando se programa sin criterios: desbordamientos, pérdidas de precisión, incompatibilidades de codificación y discrepancias de interpretación entre componentes. En SI, estos riesgos afectan integridad, trazabilidad y confianza del usuario en resultados [2], [4], [6].

Desde la formación, tratar estos riesgos aporta un beneficio directo: el estudiante aprende a programar con pruebas, validaciones y documentación de supuestos. Este marco garantista mejora capacidad de depuración y fortalece calidad del código desde etapas iniciales [7], [23], [24].

El diseño robusto también requiere reconocer que un SI integra múltiples tecnologías. Datos pueden circular entre aplicaciones, bases de datos y dispositivos; una representación válida en un componente puede fallar en otro si no existe alineación de formatos, rangos y codificaciones [1], [10], [28].

No obstante, la robustez no se limita a prevención de fallos. Programar con representaciones explícitas facilita mantenimiento: reglas bien documentadas y conversiones centralizadas reducen cambios dispersos y disminuyen defectos al evolucionar el sistema [4], [24], [33].

Este bloque se ha consolidado como una guía práctica para el lector: elegir tipos con criterio, validar entradas, controlar conversiones y probar casos límite. Tales prácticas acompañan a quien aprende a programar en problemas académicos y en escenarios organizacionales [5], [6], [7].

Desbordamiento, subdesbordamiento y rangos

El desbordamiento aparece cuando una operación excede el rango representable por un tipo. En enteros, puede producir valores inesperados; en punto flotante, puede generar infinitos o subdesbordamientos hacia cero. En IS, estos efectos alteran métricas, cálculos acumulativos y validaciones de reglas [6], [7], [34].

Para aprender a programar, este riesgo enseña a definir precondiciones. Verificar rangos antes de operar, usar tipos adecuados y registrar errores cuando un valor se sale del dominio fortalece el hábito de programar con contratos explícitos [4], [5], [23].

El tratamiento riguroso incluye pruebas. Diseñar casos en los límites del rango y observar comportamiento permite al estudiante indagar fallas que, de otro modo, se manifestarían en producción como inconsistencias difíciles de atribuir [6], [7], [24].

Orden de bytes, alineación y serialización

El **orden de bytes** (endianness) describe cómo se almacenan bytes de un valor multibyte. Sistemas distintos pueden ordenar de manera diferente; sin una serialización explícita, datos intercambiados pueden interpretarse de forma incorrecta [2], [10], [34].

Ejemplos prácticos de orden de bytes (Endianness). El orden de bytes determina cómo se almacenan en memoria los bytes que componen un valor multibyte (por ejemplo, enteros de 16, 32 o 64 bits). Existen dos esquemas principales:

- **Big-endian:** el byte más significativo se almacena primero.
- **Little-endian:** el byte menos significativo se almacena primero.

Ejemplo 1: Entero de 32 bits. Considérese el valor hexadecimal:

0x12345678

Este número está compuesto por 4 bytes:

12 34 56 78

Almacenamiento en Big-endian

Dirección 0	Dirección 1	Dirección 2	Dirección 3
12	34	56	78

Almacenamiento en Little-endian

Dirección 0	Dirección 1	Dirección 2	Dirección 3
78	56	34	12

Obsérvese que el valor lógico es el mismo, pero el orden físico en memoria cambia.

Ejemplo 2: Entero de 16 bits. Valor:

0x1234

Big-endian

12 34

Little-endian

34 12

Si un sistema little-endian envía estos bytes sin especificar el orden y un sistema big-endian los interpreta directamente, el valor leído sería:

0x3412

lo cual representa un número completamente distinto.

Ejemplo 3: Comunicación en redes. En redes, el estándar TCP/IP define el uso de **network byte order**, que es big-endian.

Si un procesador Intel (little-endian) envía el número:

0x12345678

debe convertirlo antes de transmitirlo.

En lenguaje C, esto se realiza con:

```
1      % La función htonl (Host TO Network Long) se utiliza para convertir un
      ↪ entero
2      % de 32 bits desde el orden de bytes del sistema anfitrión (host byte
      ↪ order)
3      % al orden de bytes de red (network byte order), que sigue el formato
      ↪ big-endian.
4      % Esta conversión es fundamental en aplicaciones distribuidas y
      ↪ sistemas de comunicación
5      % en red, ya que diferentes arquitecturas pueden representar los datos
      ↪ en memoria
6      % de manera distinta (little-endian o big-endian). El uso de htonl
      ↪ garantiza que
7      % los datos transmitidos sean interpretados correctamente por cualquier
      ↪ sistema receptor.
8
9      uint32_t valor = 0x12345678;
10     uint32_t red = htonl(valor);
```

Listing 2.7: Uso de la función `htonl` para garantizar la correcta representación de enteros de 32 bits en comunicaciones de red mediante la conversión al orden de bytes de red

Si no se realiza esta conversión, el receptor interpretará bytes invertidos.

Ejemplo 4: Archivos binarios. Supóngase que se guarda en un archivo binario el entero decimal 305419896, que en hexadecimal es:

$0x12345678$

Un sistema little-endian lo almacenará como:

78 56 34 12

Si el archivo se abre en un sistema big-endian sin conversión, el valor leído será:

$0x78563412$

equivalente a:

2018915346_{10}

Lo que demuestra cómo la ausencia de serialización explícita produce corrupción lógica de datos.

Ejemplo 5: Caso con UTF-16. El carácter Unicode:

$U + 00F1$

En UTF-16:

Big-endian

00 F1

Little-endian

F1 00

Para indicar el orden, algunos archivos incluyen un **BOM (Byte Order Mark)**:

FE FF (Big-endian)

FF FE (Little-endian)

Conclusión técnica. El problema del endianness no altera el valor abstracto, sino su representación física. Cuando se intercambian datos entre sistemas heterogéneos (arquitecturas x86, ARM, sistemas de red, almacenamiento binario), es imprescindible:

- Definir un orden estándar (ej. network byte order).
- Utilizar funciones de conversión.
- Especificar el formato de serialización.

En programación, la **serialización** traduce estructuras en secuencias de bytes para almacenamiento o transmisión. Implementarla enseña a definir formatos, incluir longitudes y validar consistencia, lo que fortalece habilidades de diseño orientadas a especificación [5], [6], [7].

Ejemplos prácticos de serialización En programación, la **serialización** consiste en transformar una estructura de datos en una secuencia ordenada de bytes para su almacenamiento o transmisión.

Diseñar un formato de serialización implica definir explícitamente:

- El orden de los campos.
- El tamaño en bytes de cada campo.
- El orden de bytes (endianness).
- La inclusión de longitudes para datos variables.
- Mecanismos de validación (checksum, versión, delimitadores).

Ejemplo 1: Serialización binaria simple. Supóngase la siguiente estructura:

```

1      struct Persona {
2          uint32_t id;
3          uint8_t edad;
4      };

```

Si:

$$id = 1000_{10} = 0x000003E8$$

$$edad = 25_{10} = 0x19$$

Formato definido (big-endian):

- id: 4 bytes
- edad: 1 byte

Secuencia serializada:

00 00 03 E8 19

Explicación. El diseño es explícito: siempre se esperan 5 bytes. Si se reciben menos, el mensaje es inválido.

Ejemplo 2: Inclusión de longitud en cadenas. Supóngase:

```

1          struct Usuario {
2              uint16_t longitudNombre;
3              char nombre[];
4          };

```

Si el nombre es:

"ANA"

En ASCII:

41 4E 41

Serialización

00 03 41 4E 41

Interpretación

- 00 03 indica que vienen 3 bytes.
- Luego se leen exactamente 3 bytes.

Este mecanismo evita ambigüedades y permite validar consistencia.

Ejemplo 3: Error por ausencia de longitud. Si solo se transmitiera:

41 4E 41

El receptor no sabría:

- Cuándo termina el nombre.
- Si vienen más campos después.
- Si los datos están incompletos.

Esto demuestra la importancia de definir especificaciones formales.

Ejemplo 4: Serialización con versión y checksum. Formato definido:

- 1 byte: versión
- 4 bytes: id
- 1 byte: edad
- 1 byte: checksum (suma módulo 256)

Datos:

$versin = 01$

$id = 00\ 00\ 03\ E8$

$edad = 19$

Suma de bytes:

$01 + 00 + 00 + 03 + E8 + 19$

Checksum:

$(01 + 00 + 00 + 03 + E8 + 19) \text{ mód } 256 = F3$

Mensaje final:

01 00 00 03 E8 19 F3

Si el receptor calcula un checksum distinto, el mensaje se descarta.

Ejemplo 5: Serialización en formato textual (JSON). Estructura:

```
{
  "id": 1000,
  "edad": 25
}
```

Ventaja

- Legible por humanos.
- Independiente de endianness.

Desventaja

- Mayor tamaño.
- Requiere análisis sintáctico.

Ejemplo 6: Problema de endianness. Si un sistema little-endian serializa:

0x12345678

como:

78 56 34 12

y otro sistema asume big-endian, interpretará:

0x78563412

lo cual altera completamente el valor.

Por ello, las especificaciones deben definir:

- Orden de bytes.
- Tamaño fijo de campos.
- Convenciones de codificación.

Conclusión conceptual. Implementar serialización enseña:

- Pensamiento estructural.
- Definición formal de formatos.
- Control de tamaño y memoria.
- Validación de integridad.
- Diseño orientado a especificación.

Estos principios son fundamentales en sistemas distribuidos, redes, bases de datos y arquitecturas IoT.

La **alineación de datos en memoria** también influye en desempeño y corrección cuando se trabaja cerca del hardware. Aunque muchos lenguajes abstraen esta complejidad, reconocer su existencia prepara al lector para interpretar fallos y optimizar estructuras en escenarios avanzados [10], [34], [35].

Ejemplos prácticos de alineación de datos en memoria. La **alineación** (alignment) describe la relación entre la dirección de memoria donde comienza un dato y el tamaño natural de acceso que la arquitectura utiliza para leerlo o escribirlo. En términos operativos, un dato de 4 bytes suele considerarse *alineado* si su dirección es múltiplo de 4; un dato de 8 bytes, si su dirección es múltiplo de 8. Esta regla surge de la forma en que la CPU y el subsistema de memoria transfieren información en bloques, y su incumplimiento puede introducir penalizaciones de desempeño o, en algunas arquitecturas, fallos de acceso [10], [34], [35].

Ejemplo 1: Alineación básica por múltiplos (16, 32 y 64 bits). **Objetivo del ejemplo:** identificar cuándo una dirección es alineada para distintos tamaños de palabra y por qué importa al leer datos multibyte [10], [34].

Supóngase un bloque de memoria donde se almacenan bytes en direcciones consecutivas:

Dir	1000	1001	1002	1003	1004	1005	1006	1007
Byte	.	.	.	.	.	.	.	.

Paso 1: considerar un entero de 32 bits (4 bytes). Si empieza en 1000, ocupa 1000–1003. Como $1000 \bmod 4 = 0$, el entero está alineado a 4 bytes.

Paso 2: si el mismo entero empieza en 1001, ocupa 1001–1004. Como $1001 \bmod 4 \neq 0$, el entero está *desalineado*.

Paso 3: para un entero de 64 bits (8 bytes), la dirección inicial debería cumplir $dir \bmod 8 = 0$. En el ejemplo, 1000 no es múltiplo de 8, por lo que un valor de 8 bytes comenzando en 1000 ya sería desalineado.

i

Observación adicional

Aun cuando una CPU permita accesos desalineados, puede requerir lecturas internas adicionales y reensamblado, introduciendo latencias [10], [34].

Ejemplo 2: Estructuras y relleno (padding) automático **Problema a resolver:** explicar por qué una estructura ocupa más bytes de lo esperado cuando el compilador inserta relleno para mantener alineación [34], [35].

Considérese la estructura:

```

1  typedef struct {
2      char  c;    // 1 byte
3      int   i;    // 4 bytes
4      char  d;    // 1 byte
5  } Registro;
```

Listing 2.8: Estructura con posibles bytes de relleno

Paso 1: *c* ocupa 1 byte. Si la estructura inicia en una dirección múltiplo de 4, después de *c* el siguiente byte queda en una dirección que normalmente no es múltiplo de 4.

Paso 2: para que *i* (4 bytes) comience en una dirección múltiplo de 4, el compilador inserta **3 bytes de relleno** tras *c*.

Paso 3: *i* ocupa 4 bytes. Luego *d* ocupa 1 byte.

Paso 4: para que el tamaño total sea múltiplo del mayor alineamiento del bloque (usualmente 4 en este caso), el compilador puede insertar relleno final (por ejemplo, 3 bytes) tras *d*.

Resultado típico: la estructura puede ocupar 12 bytes:

$$1 \text{ (c)} + 3 \text{ (padding)} + 4 \text{ (i)} + 1 \text{ (d)} + 3 \text{ (padding)} = 12$$

i

Observación adicional

Este relleno mejora el desempeño porque evita accesos desalineados repetidos cuando se procesan arreglos de estructuras [10], [34].

Ejemplo 3: Reordenamiento de campos para optimizar tamaño Problema a

resolver: reducir el tamaño de una estructura sin perder información, aplicando un criterio de diseño que también se aprende al programar con disciplina: ordenar campos por tamaño decreciente [10], [35]. A partir de 2.8, se propone:

```

1  typedef struct {
2      int    i;    // 4 bytes
3      char   c;    // 1 byte
4      char   d;    // 1 byte
5  } RegistroOpt;
```

Listing 2.9: Estructura reordenada para disminuir padding

Paso 1: *i* comienza alineado (múltiplo de 4).

Paso 2: *c* y *d* se ubican después; al agrupar bytes pequeños al final, se reduce el relleno intermedio.

Resultado típico: el tamaño baja, por ejemplo, a 8 bytes:

$$4 \text{ (i)} + 1 \text{ (c)} + 1 \text{ (d)} + 2 \text{ (padding final)} = 8$$

i

Observación adicional

En arreglos grandes, esta reducción disminuye consumo de memoria y mejora localidad de caché, lo que se traduce en operaciones más eficientes [10], [34].

Ejemplo 4: Corrección al leer bytes como enteros (acceso desalineado)

Problema a resolver: mostrar cómo un acceso desalineado puede ser incorrecto o fallar según la arquitectura, y cuál es el patrón de solución profesional [10], [34].

Supóngase un arreglo de bytes que representa un encabezado binario de red/archivo:

```

1  uint8_t buf[] = {0x01, 0x02, 0x03, 0x04, 0xFF, 0xEE};
```

```

2  uint32_t *p = (uint32_t*)&buf[1]; // comienza en dirección no múltiplo
   ↪ de 4
3  uint32_t x = *p;                  // acceso potencialmente desalineado

```

Listing 2.10: Lectura riesgosa por conversión directa de puntero

Paso 1: `&buf[1]` no garantiza alineación a 4 bytes.

Paso 2: el acceso `*p` puede:

- funcionar con penalización (CPU que soporta unaligned),
- generar excepción/fallo (CPU/ABI que no lo permite),
- producir resultados no portables (según compilador/optimización).

Solución directa y portable: copiar bytes con `memcpy` a una variable alineada y luego interpretar.

```

1  uint32_t x;
2  memcpy(&x, &buf[1], sizeof(uint32_t)); // copia a variable alineada

```

Listing 2.11: Lectura segura usando `memcpy`

i

Observación adicional

Esta práctica introduce una disciplina útil: separar representación en bytes de la representación como tipos, lo que prepara al lector para depuración en redes, archivos binarios y serialización [10], [34].

Ejemplo 5: Alineación y desempeño en arreglos (localidad y caché)

Problema a resolver: explicar por qué estructuras más compactas mejoran iteraciones y reducen fallos de caché, conectando el concepto con desempeño observable [10], [34].

Considérese un arreglo con un millón de registros. Si cada registro ocupa 12 bytes, el conjunto total consume más memoria que si ocupa 8. Al recorrer el arreglo secuencialmente, se cargan líneas de caché (por ejemplo, 64 bytes) que contienen menos registros cuando cada uno es más grande.

Paso 1: estructura grande $\Rightarrow$ menos elementos por línea de caché.

Paso 2: menos elementos por línea $\Rightarrow$ más lecturas desde memoria principal.

Paso 3: más lecturas $\Rightarrow$ mayor latencia acumulada.

i

Observación adicional

Aunque esta optimización no se exige en ejercicios iniciales, reconocer el fenómeno permite interpretar por qué un programa “equivalente” puede rendir distinto tras un cambio de estructura o compilador [10], [34], [35].

Cierre pedagógico La alineación y el relleno no se introducen para complicar el aprendizaje, sino para fortalecer una habilidad valiosa: razonar sobre cómo una abstracción (tipos y estructuras) se materializa en memoria. Esta visión orienta mejores decisiones al diseñar formatos binarios, estructuras de datos y módulos de serialización, y facilita diagnosticar fallos que aparecen únicamente en ciertos equipos o arquitecturas [10], [34], [35].

De manera complementaria, un formato de serialización bien definido mejora mantenibilidad. Centralizar conversiones reduce duplicación y facilita evolución de estructuras sin romper compatibilidad [4], [24], [33].

Normalización y consistencia de codificaciones

Las codificaciones inconsistentes producen fallos en comparación, búsqueda y validación. Un SI puede almacenar texto con una codificación y recibir entradas con otra; sin normalización, se generan registros duplicados y reglas que fallan por desigualdad de símbolos equivalentes [2], [6], [27].

Para el aprendizaje, esta situación refuerza la práctica de normalizar entradas. Recortar espacios, unificar mayúsculas/minúsculas según política y validar caracteres permitidos convierten ejercicios simples en prácticas profesionales aplicables a formularios y registros [1], [5], [7].

No obstante, normalizar requiere documentar criterios. Una regla de normalización cambia el significado percibido por los usuarios; por ello, se recomienda registrar decisiones y asegurar coherencia entre presentación y almacenamiento [2], [4], [24].

Ejemplo 1: Diferencias invisibles en Unicode

Considérese el nombre:

Jos

Este puede almacenarse de dos formas distintas en Unicode:

Forma 1 (precompuesta):

$U + 00E9$

Forma 2 (descompuesta):

$U + 0065 \ U + 0301$

Visualmente son idénticas, pero a nivel binario son distintas.

Problema práctico Si un sistema almacena la forma precompuesta y recibe la forma descompuesta, una comparación directa de bytes devuelve falso.

Solución Aplicar normalización Unicode (NFC o NFD) antes de almacenar o comparar.

Ejercicio guiado

1. Recibir dos cadenas visualmente iguales.
2. Mostrar sus códigos Unicode.
3. Comparar byte a byte.
4. Aplicar normalización NFC.
5. Comparar nuevamente.
6. Verificar igualdad.

Este ejercicio enseña que la igualdad visual no implica igualdad binaria.

Ejemplo 2: Espacios invisibles y registros duplicados Entrada A:`"ANA"`

Entrada B:

`"ANA "`

La segunda contiene un espacio final.

Problema Almacenar ambas entradas genera dos registros distintos.**Solución paso a paso**

1. Recortar espacios iniciales y finales.
2. Validar longitud mínima.
3. Aplicar política uniforme antes de persistir.

Ejercicio práctico Diseñar una función que:

- Elimine espacios laterales.
- Convierta a mayúsculas.
- Valide que solo contenga letras.

Probar con:

`"ana", "ANA", "Ana"`

Verificar que todas se almacenen como:

`"ANA"`**Ejemplo 3: Sensibilidad a mayúsculas y reglas de negocio** Supóngase un sistema donde:`"admin" ≠ "ADMIN"`**Problema** En un módulo, la comparación es sensible a mayúsculas; en otro no.

Esto genera:

- Inconsistencia en autenticación.
- Errores de búsqueda.
- Resultados inesperados.

Ejercicio analítico

1. Definir una política: ¿el sistema será case-sensitive?
2. Documentar la decisión.
3. Implementar función uniforme.
4. Probar búsquedas con combinaciones de mayúsculas/minúsculas.

Este ejercicio conecta decisiones técnicas con impacto organizacional.

Ejemplo 4: Codificaciones mixtas (UTF-8 vs ISO-8859-1) Supóngase que un sistema almacena:

Seor

En UTF-8:

53 65 C3 B1 6F 72

Pero otro módulo interpreta esos bytes como ISO-8859-1, mostrando:

Se±or

Problema La base de datos ahora contiene valores aparentemente distintos.

Ejercicio diagnóstico

1. Mostrar bytes almacenados.
2. Interpretar con codificación incorrecta.
3. Detectar incoherencia.
4. Forzar conversión explícita a UTF-8.

Este ejercicio refuerza la importancia de especificar codificación en interfaces.

Ejemplo 5: Normalización en bases de datos Tabla de clientes:

ID	Nombre
1	José
2	Jose
3	JOSE

Problema Búsquedas por nombre devuelven resultados inconsistentes.

Ejercicio de diseño

1. Definir política (conservar acentos o no).
2. Aplicar transformación antes de insertar.
3. Crear índice sobre campo normalizado.
4. Comparar rendimiento y consistencia.

Este ejercicio integra programación, reglas de negocio y diseño de datos.

Conclusión pedagógica Normalizar no es simplemente “modificar texto”; es definir una política coherente que garantice:

- Igualdad semántica.
- Integridad de datos.
- Coherencia entre módulos.
- Comportamiento predecible en comparaciones.

La práctica sistemática de normalización fortalece el pensamiento orientado a especificación y previene errores que, en entornos reales, pueden escalar a fallos operacionales significativos.

Precisión numérica, redondeo y comparación segura

La precisión numérica afecta cualquier indicador calculado a partir de reales. Operaciones repetidas acumulan error de redondeo; comparaciones directas pueden fallar al evaluar igualdad. Estas propiedades son inherentes al punto flotante, por lo que deben tratarse como parte del diseño [11], [12], [38].

Para aprender a programar, la comparación segura introduce un patrón didáctico: usar tolerancias y verificar magnitud de la diferencia. Este patrón se integra con pruebas y con documentación de supuestos sobre precisión requerida por el dominio [4], [5], [12].

La elección de estrategia de redondeo también importa. En reportes, un redondeo distinto puede generar discrepancias entre sistemas o entre períodos, afectando confianza en el SI. Un diseño coherente alinea cálculo interno, almacenamiento y presentación [2], [11], [27].

Ejemplo 1: Representación imperfecta de 0.1 **Problema a demostrar:** por qué 0.1 no se representa exactamente en binario.

$$0.1_{10} = 0.00011001100110011\dots_2$$

La expansión es infinita periódica, por lo que debe truncarse en memoria (IEEE 754).

Ejercicio paso a paso

1. Convertir 0.1 a binario mediante multiplicaciones sucesivas.
2. Observar repetición del patrón 0011.
3. Explicar que el almacenamiento es aproximado.
4. Mostrar que:

$$0.1 + 0.2 \neq 0.3$$

5. Calcular:

$$0.1 + 0.2 = 0.30000000000000004$$

Este ejercicio evidencia que la comparación directa puede fallar.

Ejemplo 2: Comparación insegura Código conceptual

```

1  double a = 0.1 + 0.2;
2  if (a == 0.3) {
3      printf("Iguales");
4  }
```

Resultado esperado: la condición puede no cumplirse.

Ejercicio correctivo

1. Calcular la diferencia:

$$|a - 0.3|$$

2. Definir tolerancia:

$$\epsilon = 10^{-9}$$

3. Comparar:

$$|a - 0.3| < \epsilon$$

4. Verificar que ahora la condición es verdadera.

Este patrón introduce el concepto de comparación segura.

Ejemplo 3: Acumulación de error Problema a demostrar: error acumulado en sumas repetidas.

```
1  double suma = 0.0;
2  for (int i = 0; i < 1000; i++) {
3      suma += 0.1;
4  }
```

Resultado teórico esperado:

100.0

Resultado real aproximado:

99.9999999998

Ejercicio analítico

1. Explicar que cada suma introduce pequeño error.
2. Multiplicar error por número de iteraciones.
3. Relacionar con indicadores financieros o científicos.

Este ejercicio muestra cómo operaciones repetidas amplifican desviaciones.

Ejemplo 4: Estrategias de redondeo Considérese el valor:

2.345

Redondeo tradicional (half-up)

2.35

Redondeo hacia abajo

2.34

Redondeo bancario (half-even)

2.34 si el dígito anterior es par

Ejercicio práctico

1. Calcular promedio mensual con distintos métodos de redondeo.
2. Comparar totales anuales.
3. Analizar discrepancias acumuladas.

Este ejercicio demuestra que la estrategia de redondeo impacta resultados organizacionales.

Ejemplo 5: Precisión en almacenamiento y presentación Supóngase un sistema financiero:

Monto real = 1234.56789

Almacenamiento interno: double con alta precisión.

Presentación al usuario: 2 decimales.

1234.57

Ejercicio de coherencia

1. Definir precisión interna.
2. Definir precisión de almacenamiento en base de datos.
3. Definir formato de presentación.
4. Documentar política.

Este ejercicio integra diseño técnico y confianza organizacional.

i

Conclusión aplicada

La precisión numérica no es un detalle de implementación sino una propiedad estructural del punto flotante. Aplicar tolerancias, definir estrategias de redondeo y documentar supuestos convierte ejercicios básicos en prácticas de ingeniería rigurosa. Esta disciplina previene errores invisibles que, en sistemas reales, pueden generar inconsistencias financieras, científicas o administrativas.

Integridad de datos: detección de errores y validación cruzada

La integridad requiere detectar errores durante captura, transmisión y almacenamiento. Validaciones de formato, verificaciones de longitud y controles cruzados entre campos reducen la probabilidad de registrar información defectuosa [2], [6], [28]. En programación básica, estas prácticas se implementan mediante funciones de validación y pruebas sistemáticas. El estudiante aprende a separar reglas de negocio de lógica de entrada, lo que mejora legibilidad y facilita mantenimiento [5], [7], [33].

De manera complementaria, la integridad se beneficia de restricciones en almacenamiento: claves, dominios y relaciones preservan coherencia cuando múltiples módulos escriben datos. Esta visión conecta programación con fundamentos de bases de datos [27], [28], [29]. No obstante, la validación cruzada debe diseñarse con criterio para evitar reglas contradictorias. Documentar decisiones y alinear validaciones con el proceso organizacional reduce fricción y fortalece trazabilidad [1], [4], [16].

Ejemplo 1: Validación de formato Problema: capturar una dirección de correo electrónico válida.

Regla de formato Debe contener:

- Un símbolo @
- Un dominio posterior
- Sin espacios

Ejercicio práctico

1. Solicitar entrada de usuario.
2. Verificar presencia de @.
3. Verificar que la longitud sea mayor que 5 caracteres.
4. Rechazar entradas como:

"usuario@", "@dominio.com", "usuariodominio.com"

Este ejercicio enseña validación de formato antes de almacenar.

Ejemplo 2: Verificación de longitud **Problema:** un código de producto debe tener exactamente 8 caracteres.

Ejercicio paso a paso

1. Leer código.
2. Calcular longitud.
3. Si longitud $\neq$ 8, mostrar error.
4. Permitir reingreso.

Esto previene registros incompletos o inconsistentes.

Ejemplo 3: Validación cruzada entre campos **Problema:** en un sistema académico, un estudiante no puede tener estado “Graduado” si créditos aprobados $<$ créditos requeridos.

Ejercicio de control cruzado

1. Leer créditos aprobados.
2. Leer estado académico.
3. Si estado = “Graduado” y créditos $<$ 240:

Error de coherencia

4. Registrar inconsistencia.

Este ejercicio demuestra que la validación no depende de un solo campo, sino de relaciones entre ellos.

Ejemplo 4: Separación de reglas de negocio **Problema:** no mezclar validación con lógica principal.

Diseño adecuado

- Función validarEdad(edad)
- Función validarCorreo(correo)
- Función registrarUsuario()

Ejercicio estructural

1. Crear función validarEdad (18–65).
2. Crear función validarCorreo.
3. En programa principal, llamar validaciones.
4. Si todas son correctas, registrar.

Esto mejora mantenimiento y claridad del código.

Ejemplo 5: Restricciones en almacenamiento (Base de datos) Supóngase una tabla de clientes:

ID	Email
----	-------

Restricciones necesarias

- ID como clave primaria.
- Email como único (UNIQUE).
- Dominio restringido (CHECK).

Ejercicio conceptual

1. Intentar insertar dos clientes con mismo email.
2. Verificar que la base rechaza duplicado.
3. Explicar por qué la restricción protege integridad.

Este ejercicio conecta programación con fundamentos de bases de datos.

Ejemplo 6: Validación contradictoria **Problema:** un módulo exige edad mínima 18, otro módulo acepta desde 16.

Ejercicio de análisis

1. Detectar inconsistencias entre módulos.
2. Documentar política oficial.
3. Ajustar validaciones para coherencia.

Este caso muestra la importancia de alinear reglas con el proceso organizacional.

Conclusión aplicada La integridad de datos no depende de una única validación, sino de un conjunto coherente de controles en múltiples niveles:

- Validación de entrada.
- Control cruzado.
- Restricciones de almacenamiento.
- Documentación formal de reglas.

Estos ejercicios transforman conceptos teóricos en prácticas aplicables a sistemas reales, fortaleciendo la capacidad del estudiante para diseñar software robusto y coherente.

Implicaciones de seguridad en la representación

La representación afecta seguridad cuando el programa interpreta entradas sin control, convierte tipos de manera insegura o asume longitudes sin verificarlas. Estas situaciones abren espacio a defectos y ataques basados en entradas maliciosas, incluso en programas sencillos [4], [6], [23].

Para aprender a programar, la seguridad se entrena como un hábito: validar, limitar, registrar y fallar de manera controlada. Esta práctica fortalece robustez y alinea el desarrollo con estándares que catalogan vulnerabilidades comunes y recomiendan mitigaciones [5], [6], [24].

El tratamiento de archivos y de formatos constituye un caso ilustrativo. Leer longitudes sin verificar o procesar cabeceras sin validar puede conducir a lecturas fuera de rango o a corrupción de memoria en lenguajes que exponen esos riesgos. La disciplina de validación reduce probabilidad de fallas [6], [10], [34].

En síntesis, la seguridad vinculada a representación invita al lector a programar con criterio profesional desde el inicio. La inversión en validación y pruebas produce sistemas más estables, facilita mantenimiento y fortalece confianza en resultados del SI [2], [4], [23].

Ejemplo 1: Conversión insegura de tipos Problema: convertir entrada de usuario directamente a entero sin validar.

```
1 char entrada[10];
2 scanf("%s", entrada);
3 int valor = atoi(entrada);
```

Si el usuario ingresa:

"999999999999"

Puede producir desbordamiento o comportamiento indefinido.

Ejercicio correctivo

1. Verificar que la entrada contenga solo dígitos.
2. Validar longitud máxima.
3. Usar función segura (por ejemplo, `strtol`).
4. Comprobar rango antes de asignar.

Este ejercicio demuestra que convertir tipos sin control abre vulnerabilidades.

Ejemplo 2: Desbordamiento por asumir longitud Problema: copiar cadena sin verificar tamaño.

```
1 char destino[8];
2 strcpy(destino, entrada);
```

Si `entrada` contiene más de 7 caracteres, se produce sobrescritura de memoria.

Ejercicio preventivo

1. Medir longitud antes de copiar.
2. Usar `strncpy`.
3. Limitar tamaño máximo permitido.
4. Registrar intento inválido.

Este caso ilustra cómo la representación incorrecta puede derivar en corrupción de memoria.

Ejemplo 3: Lectura insegura de archivo binario Supóngase un archivo cuyo encabezado contiene un campo de longitud:

```
1 uint32_t longitud;
2 fread(&longitud, sizeof(uint32_t), 1, archivo);
3 char buffer[100];
4 fread(buffer, 1, longitud, archivo);
```

Problema: si el archivo indica longitud = 5000, se intenta leer más allá del tamaño del buffer.

Ejercicio de validación

1. Verificar que longitud $\leq$ tamaño del buffer.
2. Validar que longitud sea coherente con tamaño real del archivo.
3. Rechazar archivo si condición falla.

Este ejercicio conecta representación binaria con riesgo de explotación.

Ejemplo 4: Interpretación incorrecta de formato **Problema:** asumir que todos los datos están en UTF-8.

Un atacante envía datos con codificación distinta, provocando interpretación errónea o bypass de filtros.

Ejercicio de mitigación

1. Detectar codificación.
2. Convertir explícitamente a formato interno estándar.
3. Validar caracteres permitidos.

Este ejercicio demuestra que la representación textual también impacta seguridad.

Ejemplo 5: Validación cruzada en autenticación **Problema:** aceptar credenciales sin validar coherencia entre usuario y rol.

Escenario Usuario: "admin" Rol: "usuario"

Ejercicio

1. Validar que el rol asignado corresponda al perfil almacenado.
2. Registrar intento inconsistente.
3. Rechazar operación.

Este caso muestra que la seguridad no depende solo del formato, sino de la coherencia semántica.

Ejemplo 6: Fallo controlado **Problema:** programa termina abruptamente ante error de entrada.

Ejercicio de diseño robusto

1. Detectar entrada inválida.
2. Mostrar mensaje claro.
3. Registrar evento.
4. Permitir reintento.

Fallar de manera controlada reduce impacto y mejora experiencia de usuario.

i

Conclusión aplicada

La representación no es neutra: afecta memoria, interpretación, seguridad y confiabilidad. Los ejercicios anteriores muestran que validar longitudes, tipos y coherencia semántica transforma programas simples en sistemas robustos. La disciplina de validar antes de procesar constituye una práctica profesional que previene defectos y mitiga vulnerabilidades desde etapas tempranas del desarrollo.

Capítulo 3

Tecnología Computacional como Soporte del Sistema de Información

La tecnología computacional constituye el entramado material y lógico que posibilita la operación de un Sistema de Información (SI). En organizaciones contemporáneas, hardware, software y componentes de control interactúan para capturar eventos, ejecutar reglas y producir resultados coherentes con objetivos operativos y estratégicos [1], [2], [16]. Desde la perspectiva formativa, este entramado adquiere sentido cuando el estudiante reconoce que cada programa escrito se ejecuta sobre una arquitectura concreta y que sus decisiones de diseño dialogan con restricciones técnicas reales.

Aprender a programar implica asumir que el código no existe en abstracto: se ejecuta en procesadores con capacidades definidas, interactúa con sistemas operativos que gestionan recursos y puede integrarse con dispositivos físicos que generan datos. Esta conciencia tecnológica orienta al lector hacia una práctica responsable, donde el rendimiento, la consistencia y la interacción con el entorno se consideran desde etapas iniciales del aprendizaje [5], [7], [10].

Dentro de este panorama, la comprensión del soporte tecnológico fortalece la calidad del desarrollo. Cuando el estudiante identifica cómo fluyen los datos desde un dispositivo hasta una aplicación y cómo se gestionan memoria y almacenamiento, adquiere una visión holística del sistema. Esta visión favorece decisiones algorítmicas coherentes con la infraestructura y evita supuestos infundados sobre disponibilidad de recursos [2], [4], [34].

Desde un enfoque riguroso, la tecnología computacional se convierte en aliada del aprendizaje. El conocimiento de sus componentes no persigue erudición técnica aislada; proporciona criterios para diseñar programas robustos, prever limitaciones y articular soluciones alineadas con la realidad organizacional [1], [16], [23].

3.0.1. Componentes Tecnológicos

Los componentes tecnológicos de un SI pueden agruparse en infraestructura de hardware, software de aplicación y sistemas operativos, así como firmware encargado del control directo de dispositivos. Cada componente cumple funciones específicas que, integradas, permiten capturar, procesar y distribuir información de manera coherente [2], [10], [34].

Para entender la relevancia de estos elementos en el aprendizaje de programación, es preciso reconocer que cada línea de código interactúa con ellos. Un programa gestiona memoria proporcionada por el sistema operativo, utiliza capacidades de procesamiento del hardware y puede depender de controladores que median la comunicación con dispositivos externos. Esta interdependencia sugiere que la formación del desarrollador en IS debe incorporar nociones básicas sobre arquitectura y gestión de recursos [4], [7], [10].

Asimismo, la articulación entre componentes tecnológicos respalda la confiabilidad del sistema. Un algoritmo correcto puede degradar su desempeño cuando ignora limitaciones de hardware o comportamientos del sistema operativo. Por ello, la programación con orientación a IS exige considerar latencia, almacenamiento y concurrencia como variables que influyen en la ejecución [23], [24], [34].

Infraestructura de Hardware

La infraestructura de hardware comprende procesadores, memoria principal, almacenamiento secundario y dispositivos periféricos que permiten entrada y salida de información. Estos elementos determinan capacidad de cómputo, velocidad de respuesta y volumen de datos que el sistema puede manejar [2], [34]. En organizaciones con operaciones intensivas, la selección y configuración del hardware impactan directamente en la eficiencia del SI.

Desde la perspectiva del aprendizaje, comprender nociones básicas de arquitectura favorece decisiones algorítmicas coherentes. La elección de estructuras de datos, el uso de ciclos anidados o la gestión de colecciones influyen en consumo de memoria y tiempo de ejecución. Esta relación introduce al estudiante en el análisis de eficiencia y en la importancia de evaluar costos computacionales [5], [7], [13].

De manera complementaria, el hardware interactúa con dispositivos que generan datos organizacionales: lectores, sensores o terminales de usuario. Programar implica considerar formatos de entrada y latencias asociadas a estos dispositivos, preparando al lector para entornos donde la tecnología trasciende la pantalla y se integra con procesos físicos [1], [2], [34].

Software de Aplicación y Sistemas Operativos

El software de aplicación implementa reglas de negocio y funcionalidades específicas del SI, mientras que el sistema operativo administra recursos, procesos y comunicación entre componentes. Esta distinción organiza la arquitectura lógica del entorno donde se ejecuta el código [2], [10], [24].

Para quien aprende a programar, entender esta separación favorece claridad conceptual. El código desarrollado en un lenguaje de alto nivel depende de servicios del sistema operativo para acceder a archivos, gestionar memoria o interactuar con red. Esta dependencia exige respetar convenciones, manejar excepciones y estructurar programas conforme a estándares del entorno [5], [7], [10].

Paralelamente, el software de aplicación refleja decisiones organizacionales. Cada módulo implementa funciones alineadas con procesos y requisitos definidos. Diseñar programas coherentes con estos requisitos fortalece la competencia del desarrollador y lo prepara para integrar su trabajo en sistemas de mayor envergadura [1], [16], [23].

Asimismo, la interacción entre aplicaciones y sistema operativo introduce al estudiante en conceptos como concurrencia y gestión de recursos compartidos. Aunque su estudio detallado corresponde a etapas posteriores, una noción inicial sobre cómo el entorno gestiona procesos enriquece la práctica de programación básica [10], [24], [34].

Firmware y Control de Dispositivos

El firmware constituye el nivel intermedio entre hardware y software de aplicación. Se trata de programas almacenados en dispositivos que controlan operaciones básicas y permiten comunicación con sistemas superiores [2], [34]. En entornos organizacionales con dispositivos inteligentes, el firmware coordina captura y transmisión de datos hacia aplicaciones centrales.

Aprender programación con conciencia de este nivel amplía la visión del estudiante. La interacción con dispositivos exige considerar protocolos, formatos y validaciones adicionales. Aunque el desarrollo de firmware puede exceder la programación inicial, reconocer su existencia fortalece la comprensión de la cadena completa de procesamiento [4], [16], [24].

En síntesis, el firmware ilustra que la programación no se limita a aplicaciones visibles; integra capas que permiten funcionamiento coordinado del sistema. Esta comprensión prepara al lector para escenarios donde software y hardware interactúan de manera dinámica y organizada [1], [2], [34].

3.0.2. Modelo funcional de entrada–proceso–salida en SI

El modelo funcional de entrada–proceso–salida (E–P–S) describe la estructura básica de operación de un SI. Datos ingresan al sistema, son transformados mediante reglas y algoritmos, y producen resultados destinados a usuarios o a otros sistemas [1], [2]. Este esquema sintetiza la lógica fundamental que el estudiante reproduce en cada ejercicio de programación.

Desde una perspectiva pedagógica, el modelo E–P–S actúa como guía para estructurar programas. Identificar claramente entradas, diseñar procesamiento coherente y definir salidas verificables ordena el razonamiento y facilita pruebas sistemáticas. Esta estructura fomenta disciplina y claridad en el diseño algorítmico [5], [7], [13].

Asimismo, el modelo evidencia que cada componente tecnológico participa en la secuencia. El hardware recibe datos, el software procesa información y los dispositivos presentan resultados. Esta integración subraya la relevancia de comprender cómo interactúan los niveles tecnológicos descritos anteriormente [2], [10], [34].

Entrada de datos organizacionales

La entrada de datos constituye el punto inicial del modelo E-P-S. Formularios, interfaces digitales y dispositivos de captura registran eventos y atributos relevantes para la operación organizacional [1], [2]. La calidad de esta etapa condiciona el procesamiento posterior.

Para quien aprende a programar, diseñar entradas implica validar tipos, rangos y formatos antes de ejecutar cálculos. Implementar estas verificaciones refuerza disciplina y reduce propagación de errores [5], [6], [7].

Asimismo, la entrada puede provenir de sistemas externos o de dispositivos automáticos. Integrar estas fuentes exige comprender protocolos básicos y estructuras de datos compatibles [2], [10], [34].

Destacando la dimensión organizacional, cada dato capturado representa un evento con significado. Reconocer esta conexión motiva al estudiante a programar con responsabilidad y coherencia [1], [4], [15].

Procesamiento de información

El procesamiento transforma entradas en resultados mediante algoritmos estructurados. Operaciones aritméticas, comparaciones, iteraciones y funciones constituyen la base de esta transformación [5], [7], [13]. En IS, este procesamiento implementa reglas de negocio y políticas operativas.

Aprender a diseñar algoritmos eficientes implica analizar pasos, evaluar complejidad y verificar consistencia de resultados. Esta práctica fortalece pensamiento lógico y prepara al lector para problemas de mayor escala [4], [23], [24].

De manera complementaria, el procesamiento puede involucrar interacción con almacenamiento persistente o con sistemas externos. Introducir estas consideraciones en ejercicios graduales permite visualizar la amplitud del entorno tecnológico [16], [28], [29].

Salida para usuarios y sistemas

La salida representa el resultado visible del modelo E-P-S. Puede consistir en mensajes en pantalla, reportes estructurados o datos transmitidos a otros sistemas [1], [2]. Su claridad y coherencia determinan utilidad organizacional.

Para el estudiante, diseñar salidas legibles y consistentes forma parte integral del algoritmo. Organizar resultados y explicar estados refuerza la conexión entre código y usuario [5], [7], [26].

Asimismo, la salida puede alimentar procesos posteriores. Programar con esta visión holística fomenta responsabilidad técnica y coherencia sistémica [4], [16], [24].

3.1. Formulación del problema informacional

La formulación del problema informacional establece la relación explícita entre una necesidad organizacional y una solución programable. En Ingeniería en Sistemas de Información, el software se concibe como un artefacto socio-técnico: integra procesos, datos y reglas operativas para producir resultados verificables que apoyan coordinación, control y toma de decisiones. Por ello, el problema informacional se define como la especificación de *qué* hechos del negocio deben registrarse, *bajo qué* reglas deben transformarse y *cómo* se presentan sus resultados con utilidad operativa y gerencial [1], [2], [24].

En formación inicial, esta actividad orienta el aprendizaje hacia programación con propósito. Cuando el estudiante parte de un enunciado organizacional y lo traduce a entradas, procesos y salidas, adquiere un criterio de diseño que trasciende la sintaxis: decide estructuras de datos por significado, elabora validaciones por política y delimita responsabilidades del programa por función. Esta práctica enlaza pensamiento algorítmico con análisis y diseño de sistemas, facilitando que el código refleje de manera rastreable el comportamiento esperado del proceso que representa [23], [40].

Una formulación rigurosa también fortalece verificabilidad. Estándares de ingeniería de requisitos insisten en claridad, ausencia de ambigüedad, trazabilidad y posibilidad de prueba; estos atributos permiten derivar casos de prueba, criterios de aceptación y evidencia de cumplimiento. Incluso en ejercicios básicos, adoptar esta disciplina reduce errores de interpretación, mejora la consistencia de soluciones y proporciona un marco estable para iterar sin perder control sobre el alcance [41], [42].

Desde la perspectiva de construcción del programa, formular el problema equivale a definir un contrato operativo: qué condiciones deben cumplir las entradas, qué transformaciones deben ejecutarse y qué salidas deben producirse con formato y significado precisos. De manera complementaria, esta formalización delimita estados del sistema y escenarios de error (datos

faltantes, formatos inválidos, combinaciones inconsistentes), lo que conduce a programas más robustos, más fáciles de probar y con depuración más directa, porque cada fallo puede asociarse a un punto de la especificación [5], [7], [13].

En Sistemas de Información Transaccionales (*Transactional Information Systems* TIS), la formulación incorpora propiedades que condicionan la programación desde el inicio. Cada operación relevante se expresa como transacción que modifica datos persistentes y, por tanto, debe preservar integridad y consistencia entre registros bajo uso concurrente. Este requisito obliga a especificar validaciones cruzadas, reglas de unicidad, relaciones entre entidades y comportamiento ante fallos parciales, conectando programación con fundamentos de bases de datos y control de errores. Dentro de este panorama, la formulación del problema informacional se consolida como una guía para implementar transacciones correctas y auditables, coherentes con reglas de negocio y con la estructura del almacenamiento [27], [28], [29].

3.1.1. Entradas del sistema

Las entradas del sistema constituyen el punto de contacto entre el entorno organizacional y el modelo computacional que lo representa. En Sistemas de Información en general, toda transacción o proceso depende de datos cuya calidad, formato y consistencia determinan la validez de los resultados posteriores. Por ello, el diseño de las entradas no se limita a la lectura técnica de datos, sino que implica definir reglas de validación, mecanismos de verificación cruzada y estrategias de control que preserven integridad y coherencia desde el momento mismo de la captura [1], [2], [24]. En el contexto formativo, comprender la naturaleza y diversidad de las entradas permite al estudiante anticipar errores, estructurar validaciones y desarrollar programas más robustos y confiables [5], [7].

Ingreso manual de datos (formularios e interfaces gráficas)

En entornos transaccionales, el ingreso manual ocurre en puntos de operación como cajas, ventanillas administrativas o módulos académicos. El usuario introduce identificadores, cantidades, fechas o selecciones que deben cumplir restricciones de dominio y formato. Programar este tipo de entrada exige implementar validaciones de obligatoriedad, control de rangos (por ejemplo, cantidades mayores que cero), verificación de formatos (identificaciones, fechas, códigos) y mensajes de retroalimentación claros que faciliten la corrección inmediata [5], [6], [7]. Esta práctica refuerza el principio de que la integridad comienza en la interfaz de captura.

Archivos estructurados (CSV, JSON, XML, TXT)

La carga desde archivos se utiliza en procesos por lote, migraciones de datos y conciliaciones contables. Aquí el programa debe validar estructura, número de columnas, tipos de datos, delimitadores y codificación de caracteres antes de incorporar la información al sistema. Además, resulta esencial registrar eventos anómalos (líneas incompletas, campos inválidos) para auditoría y trazabilidad [23], [28]. Este medio introduce al estudiante en la lectura secuencial, el manejo de excepciones y la importancia de verificar coherencia antes de persistir datos.

Bases de datos (consultas y recuperación persistente)

En OLTP, muchas entradas no provienen del exterior inmediato, sino del propio almacenamiento persistente. Por ejemplo, registrar una venta requiere consultar la existencia del cliente, verificar disponibilidad de inventario y recuperar precios vigentes. Estas operaciones deben garantizar consistencia referencial y sincronización con el estado actual de los datos [27], [28], [29]. Desde la perspectiva pedagógica, este medio enfatiza la diferencia entre datos temporales y datos persistentes, así como la necesidad de validar la existencia y unicidad de registros antes de proceder.

Consumo de servicios web (APIs e integraciones externas)

Las integraciones con sistemas externos —como verificación de identidad, pasarelas de pago o facturación electrónica— representan una fuente crítica de entrada. Programar estas interacciones implica manejar tiempos de espera, respuestas parciales, códigos de estado HTTP, autenticación y validación de esquemas de datos serializados [23], [24], [43]. En formación, este medio permite comprender que la estabilidad del sistema depende no solo del código propio, sino también del comportamiento de servicios externos, lo que exige mecanismos de reintento y control de errores.

Mensajería y colas de eventos

En arquitecturas orientadas a eventos, las entradas llegan como notificaciones asincrónicas (por ejemplo, confirmación de pago). En estos casos, el programa debe implementar procesamiento idempotente para evitar duplicidades y garantizar que una transacción no se ejecute más de una vez ante reenvíos del mismo mensaje [6], [24], [27]. Este enfoque introduce conceptos de concurrencia y control transaccional que amplían la comprensión del estudiante sobre integridad en entornos distribuidos.

Dispositivos y captura automatizada (lectores, sensores, terminales)

Algunos sistemas integran dispositivos físicos como lectores de códigos de barras, balanzas o sensores ambientales. Estas entradas exigen validación de tramas, control de rangos y sincronización inmediata con el flujo transaccional [10], [34]. Desde el punto de vista formativo, este medio ilustra la interacción entre hardware y software, y la necesidad de verificar coherencia antes de aceptar datos generados automáticamente.

i

Importante...

En conjunto, las entradas del sistema no deben entenderse como simples valores iniciales para un algoritmo, sino como representaciones formales de hechos organizacionales cuya precisión condiciona todo el ciclo de procesamiento. Diseñarlas adecuadamente implica aplicar criterios de validación, consistencia y control que reflejen las políticas del dominio y las restricciones técnicas del entorno. Así, el estudiante aprende que la calidad del resultado depende, en gran medida, de la calidad del dato capturado y validado desde su origen [2], [23], [29].

3.1.2. Salidas esperadas

Las salidas de un Sistema de Información representan la materialización observable del procesamiento realizado sobre los datos de entrada. En los TIS, la salida no se limita a mostrar un resultado, sino que constituye evidencia formal de que una operación fue validada, ejecutada y registrada conforme a reglas organizacionales y restricciones técnicas previamente definidas. Por ello, el diseño de las salidas exige precisión estructural, control de formato y definición explícita del medio de entrega, garantizando consistencia, trazabilidad y protección de la información según el rol del usuario [2], [23], [24].

Desde una perspectiva funcional, una primera categoría corresponde a **salidas interactivas en pantalla**. Estas incluyen confirmaciones, advertencias y detalles inmediatos de la transacción (por ejemplo, número de comprobante, subtotal, impuestos y total). Su correcta implementación requiere coherencia entre validaciones y mensajes, claridad en la retroalimentación y consistencia semántica con el estado del proceso ejecutado. En el ámbito formativo, este tipo de salida permite al estudiante consolidar la relación entre estructuras condicionales, control de errores y presentación de resultados comprensibles para el usuario final [1], [5], [7].

Una segunda categoría comprende la **persistencia estructurada de resultados**. En los TIS, la salida principal suele consistir en la escritura consistente de registros en bases de datos: inserción de cabeceras y detalles, actualización de inventarios o generación de bitácoras. Estas salidas son fundamentales para auditoría, trazabilidad histórica y generación posterior de reportes. La implementación exige garantizar integridad referencial, unicidad de identificadores y coherencia transaccional, aspectos que conectan programación con fundamentos de bases de datos y control de concurrencia [27], [28], [29].

Otra forma relevante corresponde a la generación de **documentos formales**, tales como facturas, tickets, certificados o reportes en PDF. Aquí se evidencia la separación entre cálculo y presentación: el programa realiza operaciones numéricas con precisión interna y posteriormente aplica reglas de formato, redondeo y representación monetaria para producir un documento legible y normativamente válido. Este proceso introduce al estudiante en consideraciones sobre precisión numérica y consistencia de representación, particularmente en contextos financieros [2], [11], [38].

Asimismo, las salidas pueden materializarse como **archivos de intercambio** (CSV, JSON, XML) destinados a conciliaciones contables o integración con otros sistemas. La correcta generación de estos archivos requiere controlar delimitadores, orden estable de campos, codificación de caracteres y validación de totales agregados para evitar discrepancias en procesos posteriores. Desde el punto de vista pedagógico, este formato refuerza la noción de interoperabilidad y la importancia de respetar contratos de datos definidos entre sistemas [6], [27], [28].

En arquitecturas distribuidas, la salida puede expresarse como **respuestas de servicios** o **mensajes asincrónicos**. En el primer caso, el sistema devuelve estructuras serializadas acompañadas de códigos de estado que indican éxito o error; en el segundo, publica eventos que desencadenan acciones en otros módulos (por ejemplo, notificaciones de stock bajo). Estas modalidades introducen conceptos de contrato de interfaz, consistencia eventual y control de idempotencia, ampliando la comprensión del estudiante sobre sistemas interconectados [6], [23], [24].

i

Resumiendo...

El diseño de salidas debe considerar la **segmentación por roles organizacionales**. La información generada no se distribuye indiscriminadamente: mientras un operador visualiza detalles operativos, un directivo requiere indicadores agregados, y un usuario final accede únicamente a datos que le competen. Este principio articula programación con gobierno de datos y control de acceso, reforzando la dimensión organizacional del sistema [1], [2], [27].

En conjunto, las salidas esperadas no constituyen meros productos finales del algoritmo, sino componentes estructurales que garantizan verificabilidad, consistencia y utilidad organizacional. Diseñarlas adecuadamente implica integrar criterios de precisión numérica, control transaccional, interoperabilidad y segmentación de información, consolidando así una visión profesional del desarrollo de Sistemas de Información.

3.1.3. Procesos

En el contexto de los TIS, el proceso constituye el núcleo transformador que convierte entradas validadas en resultados consistentes y persistentes. Desde una perspectiva formal, un proceso se define como una secuencia estructurada de operaciones gobernadas por reglas explícitas del dominio organizacional. En programación básica, esta estructura se materializa mediante decisiones condicionales, ciclos iterativos, funciones auxiliares y mecanismos de control de errores; sin embargo, en TIS el proceso adquiere una dimensión adicional al estar vinculado con estados persistentes y con la necesidad de preservar integridad bajo uso concurrente [2], [23], [24].

En términos operativos, el proceso transaccional se organiza como una transacción: una secuencia ordenada que (i) valida entradas, (ii) consulta el estado actual del almacenamiento persistente, (iii) aplica reglas de negocio, y (iv) confirma cambios de manera consistente. Esta estructura introduce al estudiante en principios fundamentales como atomicidad, consistencia e aislamiento, esenciales cuando múltiples usuarios interactúan simultáneamente con el sistema [27], [28], [29]. De este modo, el proceso deja de ser únicamente una transformación lógica y se convierte en un mecanismo formal de preservación del estado organizacional.

Desde una perspectiva didáctica, resulta conveniente estructurar el proceso como un *pipeline* de etapas claramente delimitadas: normalización de entradas, validación de formato y dominio, verificación cruzada contra datos persistentes, cálculo de resultados y persistencia controlada. Esta descomposición facilita pruebas unitarias, depuración incremental y trazabilidad entre requisito y código implementado [5], [7], [24]. Además, fomenta la modularidad y la claridad estructural, atributos recomendados en prácticas de ingeniería de software.

Un aspecto crítico en TIS es el tratamiento sistemático de excepciones. Condiciones como inexistencia de registros, duplicidad de identificadores, insuficiencia de recursos o fallos de comunicación externa deben gestionarse mediante rutas alternativas claramente definidas, evitando efectos colaterales no deseados. Programar excepciones fortalece la robustez del sistema y enseña a documentar condiciones límite, en coherencia con recomendaciones de programación segura [6], [10], [23].

Asimismo, la construcción de procesos requiere definir invariantes explícitos del dominio. Por ejemplo, en una venta transaccional: el total debe coincidir con la suma de sus ítems, el inventario no puede quedar en valores negativos y el identificador del comprobante debe ser único. Estas invariantes, implementadas como validaciones o aserciones, previenen inconsistencias silenciosas y refuerzan la disciplina de verificación interna del sistema [6], [27], [28].

i

Importante...

En escenarios multiusuario, los procesos deben considerar concurrencia y sincronización. Dos operaciones simultáneas sobre el mismo registro pueden generar condiciones de carrera si no existen mecanismos adecuados de aislamiento. El análisis y simulación de estos escenarios en el aula permite introducir nociones de bloqueo, control transaccional y manejo de conflictos, ampliando la comprensión del estudiante sobre el comportamiento real de los TIS [27], [29], [34].

Ejemplo aplicado (TIS): Registro de una venta con control de inventario.

Se considera el siguiente escenario transaccional: un operador registra una venta compuesta por varios productos; el sistema debe validar existencia de cliente y productos, verificar disponibilidad de inventario, calcular subtotales e impuestos, actualizar existencias y generar un comprobante persistente. En la Figura 3.1 se presenta el diagrama de flujo correspondiente al proceso de registro de una venta con control de inventario. El esquema describe de manera estructurada las decisiones, validaciones y operaciones transaccionales involucradas, permitiendo comprender con claridad la secuencia lógica de acciones necesarias para garantizar consistencia, verificación de stock y confirmación de la transacción.

Este ejemplo integra validación, cálculo, persistencia y control transaccional en una sola unidad coherente [2], [23], [29].

```
1  Algoritmo registrarVenta
2  Entrada: listaItems (productoId, cantidad), clienteId
3  Salida: comprobanteId, total, estado
4
5  {1) Validaciones iniciales}
6  Si clienteId es vacio Entonces
7      devolver Error("Cliente requerido")
8  FinSi
9
10 Para cada item en listaItems Haga
11     Si item.cantidad <= 0 Entonces
12         devolver Error("Cantidad invalida")
13     FinSi
14 FinPara
15
16 {2) Validacion cruzada contra almacenamiento persistente}
17 Si no existeCliente(clienteId) Entonces
18     devolver Error("Cliente inexistente")
19 FinSi
20
21 Para cada item en listaItems Haga
22     Si no existeProducto(item.productoId) Entonces
23         devolver Error("Producto inexistente")
24     FinSi
```

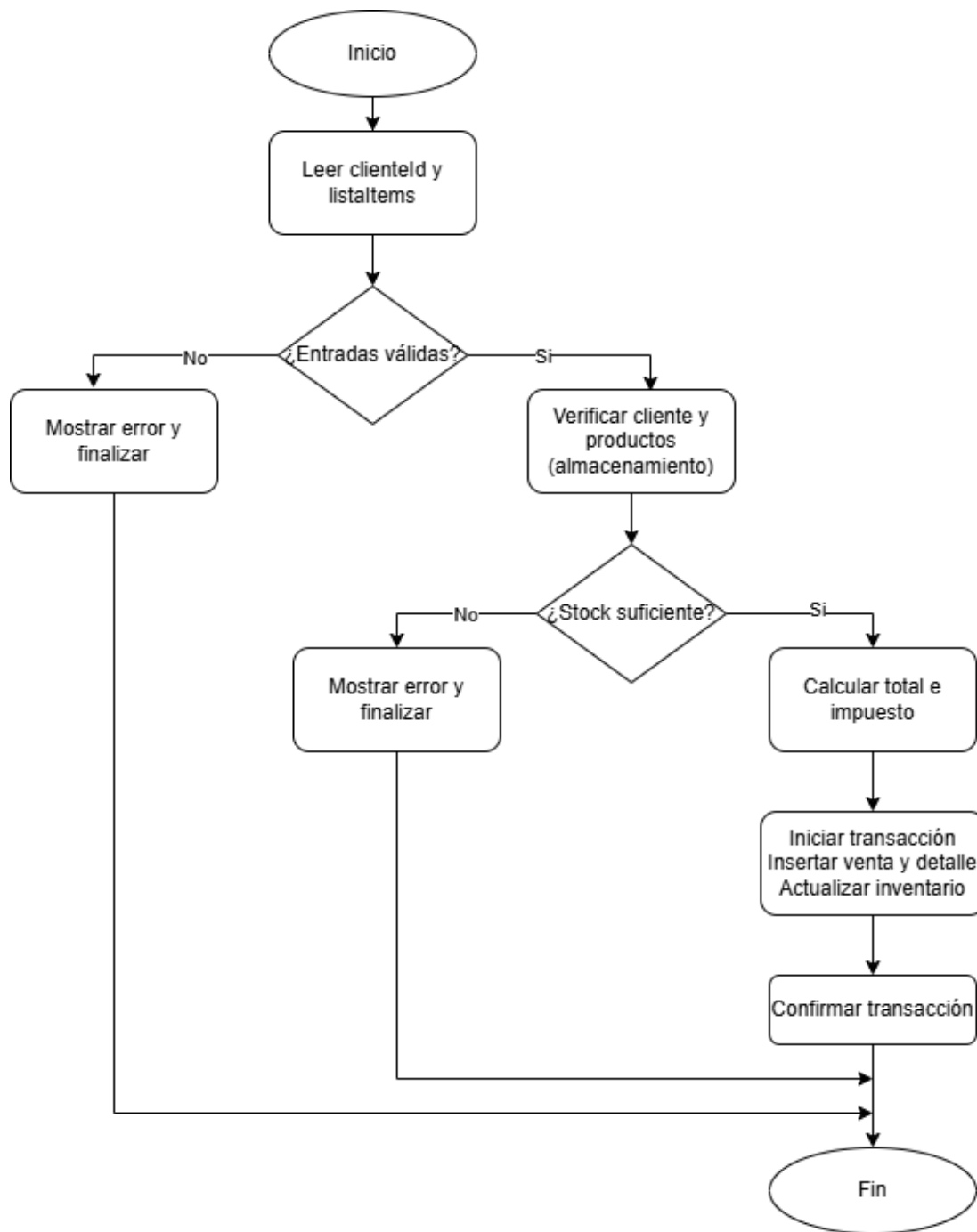


Figura 3.1: Flujo general del proceso transaccional en un TIS.

```

25     Si stock(item.productoId) < item.cantidad Entonces
26         devolver Error("Stock insuficiente")
27     FinSi
28 FinPara
  
```



```
29
30   {3) Calculo de montos}
31   total <- 0
32   Para cada item en listaItems Haga
33       precio <- precioVigente(item.productoId)
34       total <- total + (precio * item.cantidad)
35   FinPara
36   impuestos <- calcularImpuesto(total)
37   totalFinal <- total + impuestos
38
39   {4) Persistencia atomica}
40   IniciarTransaccion()
41   comprobanteId <- insertarVenta(clienteId, totalFinal, impuestos)
42   Para cada item en listaItems Haga
43       insertarDetalle(comprobanteId, item.productoId, item.cantidad,
44   ↪ precioVigente(item.productoId))
45       actualizarStock(item.productoId, stock(item.productoId) -
46   ↪ item.cantidad)
47   FinPara
48   ConfirmarTransaccion()
49
50   devolver OK(comprobanteId, totalFinal)
51   FinAlgoritmo
```

Listing 3.1: Pseudocódigo: Proceso transaccional registrarVenta

Utilidad formativa: este ejemplo permite ejercitar validación estructurada, ciclos, modularización y manejo de errores, a la vez que introduce el principio de atomicidad propio de los TIS: la transacción se aplica íntegramente o no se aplica en absoluto. Esta propiedad garantiza coherencia y evita estados intermedios inconsistentes [6], [27], [29].

3.1.4. Restricciones organizacionales y técnicas

Las restricciones organizacionales incluyen políticas internas, normativas legales y reglas institucionales. Por ejemplo, protección de datos personales según regulaciones vigentes.

Existen restricciones técnicas como capacidad de almacenamiento, rendimiento esperado o compatibilidad con sistemas heredados.

El presupuesto y los recursos humanos también limitan alcance y complejidad.

Las restricciones de seguridad exigen controles de autenticación y autorización.

Las restricciones temporales (plazos académicos o administrativos) condicionan la priorización de funcionalidades [24].

3.1.5. Del enunciado organizacional al modelo resoluble

Traducir un enunciado organizacional en modelo computacional implica abstraer la realidad en estructuras de datos y algoritmos. Por ejemplo, “gestionar biblioteca universitaria” se convierte en entidades: Libro, Usuario, Préstamo.

Stakeholders/Interesados

Los stakeholders incluyen usuarios finales, administradores, desarrolladores y directivos. Cada uno tiene expectativas distintas.

Por ejemplo, el desarrollador desea rapidez y claridad; el administrador requiere reportes consolidados.

Identificar interesados permite priorizar requisitos y definir criterios de aceptación [41].

Supuestos operativos

Los supuestos delimitan el contexto de operación. Ejemplo: “Se asume conexión estable a Internet” o “Todos los usuarios están previamente registrados”.

Documentar supuestos evita malentendidos y facilita validación posterior [24].

Casos de prueba representativos

Un caso de prueba para registro académico puede ser: Entrada: estudiante con identificación válida. Proceso: validación de requisitos académicos. Salida esperada: matrícula aprobada.

Otro caso: estudiante con deuda pendiente. Salida esperada: rechazo con mensaje explícito.

Estos casos permiten verificar funcionalidad antes de despliegue [23].

Criterios de aceptación funcional

Un criterio de aceptación debe ser medible. Ejemplo: “El sistema calcula correctamente el promedio con dos decimales de precisión”.

Otro criterio puede establecer tiempos máximos de respuesta.

Definir criterios explícitos fortalece calidad y coherencia entre requisito y solución implementada [40], [41].

Capítulo 4

Algoritmos y Modelado de Soluciones Informacionales

Los Sistemas de Información no operan sobre abstracciones aisladas, sino sobre datos que adquieren significado dentro de estructuras organizacionales, procesos y marcos normativos. La literatura en ciencias de la información ha demostrado que el paso de dato a información no es automático, sino que requiere contexto, estructura y validación semántica [44], [45], [46]. En este escenario, los algoritmos constituyen el mecanismo formal que transforma entradas estructuradas en salidas con valor organizacional. Sin un modelado riguroso, los datos permanecen como registros inertes; con un diseño adecuado, se convierten en soporte para decisiones operativas y estratégicas [47].

Desde la perspectiva de la ingeniería de software, el modelado algorítmico no es únicamente una técnica de programación, sino una disciplina de diseño que articula requisitos, procesos y estructuras de datos [23], [24]. En entornos de transformación digital, donde la automatización redefine procesos de negocio, la calidad del algoritmo impacta directamente en la eficiencia organizacional [48], [49]. Por ejemplo, un sistema de cálculo de inventarios mal modelado puede generar decisiones erróneas de reposición, afectando la cadena de suministro; uno correctamente diseñado puede optimizar tiempos y reducir costos operativos [50].

La modelación de soluciones informacionales exige comprender la arquitectura completa del sistema: datos, reglas de negocio, flujos de proceso y restricciones técnicas. Este enfoque coincide con los estándares de ingeniería de requisitos, que establecen que toda solución debe derivarse de especificaciones verificables y trazables [41]. Diseñar un algoritmo sin un análisis estructurado equivale a implementar código sin arquitectura conceptual, práctica ampliamente documentada como fuente de defectos y retrabajo [4], [51].

Este capítulo desarrolla un marco metodológico para transformar problemas organizacionales en modelos algorítmicos verificables. Se articula pensamiento computacional [17], fundamentos formales de algoritmos [52], [53] y principios de calidad de software [40]. El objetivo no es enseñar a escribir instrucciones aisladas, sino a estructurar soluciones que puedan sostener procesos reales, ser mantenidas en el tiempo y adaptarse a entornos de transformación digital.

4.1. Proceso de resolución de problemas en Sistemas de Información

Resolver un problema en Sistemas de Información implica formalizar una necesidad organizacional en una estructura ejecutable. Esta formalización requiere traducir reglas de negocio, restricciones legales y flujos operativos en representaciones precisas. La literatura sobre gestión de procesos de negocio (BPM) ha mostrado que la automatización solo produce beneficios cuando el proceso ha sido previamente modelado con claridad [30], [54]. Automatizar sin modelar equivale a escalar ineficiencias.

El proceso de resolución comienza con la delimitación del problema en términos de entradas, transformaciones y salidas.

Esta estructura responde a un patrón clásico en algoritmia formal [13], [53]. Sin embargo, en Sistemas de Información, las entradas no son únicamente valores numéricos; pueden incluir registros contables, eventos transaccionales o estados organizacionales. Por ejemplo, en un sistema de evaluación de desempeño, la entrada no es solo una puntuación, sino un conjunto de métricas ponderadas definidas por políticas institucionales.

Un segundo componente esencial es la identificación de reglas de decisión. Estas reglas representan políticas organizacionales formalizadas. Cuando no se modelan explícitamente, se introducen ambigüedades que degradan la calidad del sistema [41]. Por ejemplo, un algoritmo de aprobación crediticia debe explicitar umbrales, excepciones y condiciones acumulativas; omitir uno de estos elementos puede generar decisiones inconsistentes o sesgadas.

El proceso de resolución debe incluir validación estructurada. La ingeniería de software contemporánea enfatiza que la calidad se construye desde la etapa de diseño, no únicamente mediante pruebas posteriores [23], [40]. Por ejemplo, la revisión temprana de requisitos y modelos reduce significativamente defectos en etapas posteriores del ciclo de vida.

4.1.1. Análisis del problema

El análisis del problema constituye la fase donde se define la estructura conceptual de la solución. En términos epistemológicos, implica identificar qué entidades serán tratadas como datos y bajo qué condiciones adquirirán significado informacional [44], [55]. En Sistemas de Información, este análisis debe considerar simultáneamente dimensiones técnicas y organizacionales.

En problemas matemáticos, el análisis exige identificar relaciones formales entre variables. Por ejemplo, al modelar una proyección financiera, es necesario distinguir entre capital inicial, tasa de interés y período de capitalización. La omisión de una variable altera la función resultante. En términos algorítmicos, esto se traduce en estructuras incorrectas o incompletas [38].

En problemas financieros y contables, el análisis no solo implica cálculo, sino cumplimiento normativo. Por ejemplo, un sistema de amortización debe respetar reglas fiscales y estructuras contables. Aquí el algoritmo no representa únicamente una fórmula matemática, sino una política institucional formalizada [56]. Una implementación que ignore estas restricciones puede ser técnicamente correcta pero legalmente inválida.

En problemas organizacionales vinculados al talento humano, intervienen variables cualitativas y cuantitativas. Por ejemplo, en un sistema de incentivos, pueden intervenir métricas de desempeño, antigüedad y cumplimiento de objetivos. El análisis debe determinar si estas variables se combinan mediante reglas acumulativas, ponderaciones o umbrales discretos. Esta decisión modifica la estructura del algoritmo y su impacto organizacional.

En contextos de toma de decisiones estratégicas, el análisis requiere distinguir entre dato bruto e información contextualizada. La literatura DIKW ha demostrado que los datos carecen de significado sin un marco interpretativo [57], [58]. Por ejemplo, una cifra de ventas aislada no informa sobre rendimiento si no se compara con períodos anteriores o metas proyectadas.

Además, el análisis debe anticipar escenarios extremos y restricciones técnicas. Aspectos como representación numérica en punto flotante pueden introducir errores acumulativos si no se consideran adecuadamente [11], [12]. Por ejemplo, en sistemas financieros que realizan miles de operaciones, pequeñas imprecisiones pueden generar desviaciones significativas. Ignorar estos aspectos durante el análisis conduce a defectos estructurales difíciles de corregir posteriormente.

Identificación de variables relevantes y sus orígenes en un programa

Identificar variables relevantes implica reconocer qué elementos del problema pueden variar y cuáles influyen directamente en el resultado. Una variable relevante es aquella cuyo valor afecta el comportamiento del algoritmo o su resultado final. Si modificar un dato cambia el resultado, ese dato debe modelarse como variable [5], [59].

Un error común en principiantes es confundir variables con constantes. Una constante es un valor que no cambia durante la ejecución del algoritmo, como una tasa institucional fija del 12% o un límite mínimo de aprobación. En cambio, el salario de un empleado o la nota de un estudiante son variables porque pueden cambiar en cada ejecución.

Es importante distinguir entre una constante formal y una variable que en un caso particular no cambia. Por ejemplo, si en un ejercicio específico se utiliza una tasa de interés del 5%, esa tasa puede comportarse como constante dentro del algoritmo, pero conceptualmente sigue siendo una variable del dominio financiero. Esta distinción es clave para evitar confusiones estructurales y modelados rígidos que dificulten la evolución del sistema [23], [24].

Para identificar variables relevantes se recomienda formular preguntas como: ¿Qué datos pueden cambiar entre distintas ejecuciones? ¿Qué información influye en el resultado? ¿Qué valores dependen del usuario o del entorno? Este método analítico

permite traducir correctamente el problema real a una estructura programable [17].

Por ejemplo, en un sistema de cálculo de nómina, son variables relevantes el salario base, las horas trabajadas y el porcentaje de descuento. En cambio, el nombre de la empresa puede no ser relevante para el cálculo numérico, aunque sí lo sea para la presentación del resultado.

La correcta identificación de variables facilita la claridad del algoritmo, mejora su mantenibilidad y reduce errores conceptuales durante la implementación [51].

Origen de las variables en un programa En programación real, las variables no surgen únicamente del análisis conceptual; también deben clasificarse según su origen. Comprender el origen de una variable es fundamental porque determina cómo se valida, cómo se almacena y cómo se actualiza dentro del sistema.

1. Variables ingresadas manualmente (entrada por teclado). Son aquellas cuyo valor es proporcionado directamente por el usuario durante la ejecución del programa. Se utilizan en aplicaciones interactivas o educativas.

En el Listado de código 4.1 se muestra un ejemplo.

```
1 leer salarioBase
```

Listing 4.1: Pseudocódigo: Variable ingresada manualmente.

En este caso, el programa depende de la entrada humana. Estas variables requieren validación inmediata para evitar errores como datos fuera de rango o tipos incorrectos.

2. Variables provenientes de archivos. En aplicaciones más avanzadas, los datos pueden almacenarse previamente en archivos de texto o binarios (ver Listado de código 4.2).

Ejemplo conceptual:

```
1 Abrir archivo "notas.txt"
2 Leer nota1, nota2, nota3
3 Cerrar archivo
```

Listing 4.2: Pseudocódigo: Variables leídas desde un archivo.

Aquí, las variables se alimentan desde almacenamiento persistente. Este mecanismo es común en sistemas batch o en procesamiento masivo de datos.

3. Variables obtenidas desde bases de datos. En sistemas de información empresariales, la mayoría de las variables relevantes provienen de bases de datos [28], [29].

Ejemplo conceptual: Obtener el salario base del empleado con id = 15.

Consulta SQL:

```
SELECT salario_base FROM empleados WHERE id = 15;
```

Pseudocódigo estructurado siguiendo el flujo de JDBC

```
1 Inicio
2
```

```
3  Var
4      conexion: Conexion {Connection}
5      sentencia: Sebtebcia {Statement}
6      resultado: ConjuntoRegistros {ResultSet}
7      salarioBase: real
8      idEmpleado: entero
9
10     idEmpleado <- 15
11
12     {1. Cargar driver de base de datos}
13     cargarDriver("com.mysql.jdbc.Driver")
14
15     {2. Establecer conexión}
16     conexion <- obtenerConexion(
17         "jdbc:mysql://localhost:3306/empresa",
18         "usuario",
19         "clave")
20
21     {3. Preparar sentencia SQL}
22     sentencia <- conexion.prepararSentencia(
23         "SELECT salario_base
24         FROM empleados
25         WHERE id = ?")
26
27     {4. Asignar parámetro}
28     sentencia.asignarParametro(1, idEmpleado)
29
30     {5. Ejecutar consulta}
31     resultado <- sentencia.ejecutarConsulta()
32
33     {6. Procesar resultado}
34     Si resultado.tieneSiguiete() Entonces
35         salarioBase <- resultado.obtenerReal("salario_base")
36         escribir "Salario base: ", salarioBase
37     SiNo
38         escribir "Empleado no encontrado"
39     FinSi
```

```
40
41     cerrar(resultado)
42     cerrar(sentencia)
43     cerrar(conexion)
44
45     fin
```

Listing 4.3: Pseudocódigo: Variables leídas desde una base de datos.

Explicación pedagógica del flujo. En el pseudocódigo 4.3 no se han considerado los posibles errores inherentes al proceso de acceso a datos. A diferencia de los algoritmos puramente locales, la consulta a una base de datos implica establecer una comunicación entre dos sistemas independientes: el cliente (aplicación) y el servidor de base de datos. Esta interacción introduce riesgos asociados a la conectividad, la autenticación y la configuración del entorno.

Por esta razón, es imprescindible contemplar el manejo de excepciones. Pueden producirse fallos debido a credenciales incorrectas (nombre del servidor, puerto, nombre de la base de datos, usuario o contraseña), indisponibilidad del servicio, problemas de red, errores de tiempo de espera (timeout) o incluso inconsistencias en la consulta SQL. Ignorar estos escenarios conduce a aplicaciones frágiles que fallan abruptamente ante condiciones reales de operación.

El proceso sigue exactamente la arquitectura conceptual de Java para el acceso a datos de una base de datos (JDBC). El siguiente es un algoritmo en lenguaje natural:

```
1     1. Se carga el controlador (DriverManager).
2     2. Se establece una conexión (Connection).
3     3. Se prepara una sentencia parametrizada (PreparedStatement).
4     4. Se ejecuta la consulta.
5     5. Se obtiene un conjunto de resultados (ResultSet).
6     6. Se procesan los datos.
7     7. Se cierran los recursos para evitar fugas de memoria o conexiones
    ↪ abiertas.
```

Listing 4.4: Algoritmos en lenguaje natural: Datos leídos desde una base de datos.

El proceso descrito sigue la arquitectura conceptual de Java para el acceso a bases de datos mediante JDBC (*Java Database Connectivity*), la cual establece una secuencia estructurada: carga del controlador, establecimiento de conexión, preparación de sentencia, ejecución, procesamiento de resultados y liberación de recursos. Cada una de estas etapas puede generar excepciones que deben capturarse y gestionarse adecuadamente para garantizar robustez y estabilidad del sistema.

i

Observación importante para aplicarla en el futuro.

Nunca se debe concatenar directamente el valor del id dentro del SQL, como:

"SELECT salario_base as salarioBase FROM empleados WHERE id = " + idEmpleado

Esto puede producir vulnerabilidades de inyección SQL. Por eso se utilizan sentencias preparadas con parámetros.

En este escenario, la variable `salarioBase` no es ingresada manualmente, sino recuperada desde una estructura persistente. Esto introduce consideraciones adicionales como integridad, concurrencia y consistencia transaccional.

4. Variables provenientes de sensores o dispositivos. En sistemas instrumentados (IoT o monitorización), las variables pueden originarse en sensores físicos. En el Listado de código 4.5 ilustra un ejemplo conceptual sobre la variable temperatura que es leída de algún sensor.

```
1  temperatura <- leerSensor()
```

Listing 4.5: Variable leída desde un sensor.

En este caso, el dominio de la variable está condicionado por límites físicos y precisión del dispositivo.

5. Variables derivadas o calculadas. Son aquellas que no provienen directamente del entorno externo, sino que resultan de operaciones internas del algoritmo.

Un ejemplo claro es el que se muestra en el Listado de código 4.6, en el que se muestra la variable promedio calculada a partir de las 3 variables: nota1, nota2 y nota3.

```
1  promedio <- (nota1 + nota2 + nota3) / 3
```

Listing 4.6: Variable Calculada.

Estas variables dependen completamente de otras previamente definidas.

Clasificación estructural de variables: Desde el punto de vista del diseño del programa, las variables pueden clasificarse además como:

- **Variables de entrada** (input)
- **Variables de proceso** (intermedias)
- **Variables de salida** (resultado)

Esta clasificación ayuda a organizar el algoritmo y mejora su legibilidad, lo cual es un principio central de calidad del software [33].

Ejemplo integrador completo: Calcular el total a pagar por un cliente considerando el precio almacenado en base de datos y la cantidad ingresada por teclado.

Variables identificadas:

- precio (origen: base de datos)
- cantidad (origen: teclado)
- total (variable derivada)

Pseudocódigo:

```
1  precio <- consultarBaseDatos(idProducto)
2  leer cantidad
3  total <- precio * cantidad
4  escribir total
```

Listing 4.7: Variable Calculada.

El ejemplo del Listado de código 4.7 muestra el segmento de un programa real que combina múltiples orígenes de datos. Ignorar esta distinción conduce a diseños frágiles y difícilmente escalables.

i

Importante...

Comprender qué es una variable, distinguirla de una constante, determinar su dominio y reconocer su origen constituye una competencia esencial en la formación del programador. Esta capacidad permite modelar problemas con precisión, estructurar algoritmos coherentes y construir programas robustos que puedan evolucionar hacia sistemas de información completos [15], [40].

Definición de casos límite

En programación, un algoritmo correcto no es únicamente aquel que funciona en condiciones normales, sino aquel que mantiene su coherencia cuando enfrenta situaciones extremas o no previstas. Estas situaciones se denominan **casos límite** (boundary cases). Un caso límite es un valor o condición que se encuentra en los extremos del dominio permitido o en escenarios excepcionales que pueden provocar fallos si no son tratados explícitamente.

La literatura en ingeniería de software establece que una proporción significativa de errores en sistemas reales se origina en condiciones frontera no consideradas durante el diseño [23], [24]. Por esta razón, la definición de casos límite debe formar parte del análisis del problema y no dejarse exclusivamente a la fase de pruebas.

1. Casos límite por dominio numérico. Cuando una variable posee un rango definido, deben analizarse sus valores extremos.

Ejemplo: cálculo del promedio académico.

Dominio válido:

$$0 \leq \text{nota} \leq 10$$

Los casos límite son:

- $\text{nota} = 0$
- $\text{nota} = 10$
- $\text{nota} < 0$ (inválido)
- $\text{nota} > 10$ (inválido)

Algoritmo de validación:

```
1      leer nota
2
3      Si nota < 0 OR nota > 10 Entonces
4          escribir "Error: nota fuera de rango"
5      SiNo
6          escribir "Nota válida"
7      FinSi
```

Listing 4.8: Validación de nota dentro del dominio permitido.

Si el dominio no se valida, el algoritmo puede producir resultados matemáticamente correctos pero conceptualmente inválidos.

2. Casos límite por división entre cero. Uno de los errores más comunes en programación es la división por cero. Este caso ocurre cuando una variable utilizada como divisor puede tomar el valor cero.

Ejemplo: cálculo del promedio general.

```
1      leer suma
2      leer cantidad
3
4      Si cantidad = 0 Entonces
5          escribir "No se puede dividir entre cero"
6      SiNo
7          promedio <- suma / cantidad
8          escribir promedio
9      FinSi
```

Listing 4.9: Control de división por cero en cálculo de promedio.

Este caso límite debe anticiparse en la fase de diseño, no cuando el programa ya está en producción.

3. Casos límite en estructuras repetitivas. En ciclos, los extremos también deben analizarse:

- Número mínimo de iteraciones.
- Número máximo permitido.
- Condición inicial falsa.

Ejemplo: recorrer una lista de empleados.

```
1      leer totalEmpleados
2
3      Si totalEmpleados = 0 Entonces
4          escribir "No existen empleados registrados"
5      SiNo
6          para i <- 1 hasta totalEmpleados hacer
7              procesarEmpleado(i)
8      FinPara
9      FinSi
```

Listing 4.10: Recorrido de lista considerando lista vacía.

El caso límite aquí es una lista vacía. Si no se controla, el algoritmo puede omitir lógica crítica o producir errores de índice.

4. Casos límite en acceso a bases de datos. Cuando se realiza una consulta, también deben considerarse escenarios extremos:

- El registro no existe.
- La consulta devuelve múltiples registros inesperados.
- La conexión falla.

Ejemplo conceptual:

```
1      resultado <- ejecutarConsulta(idEmpleado)
2
3      Si resultado.tieneSiguiente() Entonces
4          salario <- resultado.obtener("salario_base")
5          escribir salario
6      SiNo
7          escribir "Empleado no encontrado"
8      FinSi
```

Listing 4.11: Control de registro inexistente en consulta.

El caso límite no es un error del algoritmo, sino una condición válida del sistema que debe ser gestionada explícitamente.

5. Casos límite por precisión numérica. En variables reales, la representación en punto flotante puede introducir pequeñas imprecisiones acumulativas [11], [12]. Esto puede afectar comparaciones directas.

Por ejemplo:

```
1      epsilon <- 0.0001
2
3      Si abs(valor1 - valor2) < epsilon Entonces
4          escribir "Son aproximadamente iguales"
5      FinSi
```

Listing 4.12: Comparación segura de números reales.

Comparar directamente números reales puede producir resultados inesperados si no se considera el error de representación.

6. Clasificación formal de casos límite Desde el punto de vista estructural, los casos límite pueden clasificarse como:

- Fronteras inferiores y superiores del dominio.
- Valores nulos o vacíos.
- Estados inexistentes.
- División por cero.

- Desbordamientos.
- Errores de precisión.

Identificar estos escenarios durante el diseño fortalece la robustez del algoritmo y reduce defectos posteriores, alineándose con buenas prácticas de ingeniería de software [40].

Un programador que anticipa casos límite diseña algoritmos resilientes. Un programador que los ignora produce código que funciona solo en condiciones ideales. La diferencia entre ambos radica en la calidad del análisis previo a la implementación.

Validación de requisitos funcionales

Cuando una persona empieza a programar, suele pensar que el objetivo principal es que el programa “funcione”. Sin embargo, en ingeniería de software el criterio es más exigente: el programa debe hacer exactamente lo que el problema solicita, ni más ni menos. A esto se le denomina **validación de requisitos funcionales** [23], [24].

Un requisito funcional describe qué debe hacer el sistema. Por ejemplo:

- “Calcular el promedio de tres notas.”
- “Mostrar si el estudiante aprueba o reprueba.”
- “Generar un reporte con el salario neto.”

Validar significa comprobar que el algoritmo implementa correctamente cada una de estas funciones.

1. Verificación directa del enunciado. El primer paso consiste en revisar cuidadosamente el enunciado del problema y preguntarse:

- ¿Estoy calculando exactamente lo que se pide?
- ¿Estoy mostrando la salida solicitada?
- ¿Estoy considerando todas las condiciones mencionadas?

Por ejemplo, si el problema indica: “*El estudiante aprueba con promedio mayor o igual a 7.0*”. El algoritmo debe reflejar exactamente esa condición.

```

1      Inicio
2      Var
3          nota1, nota2, nota3: entero
4          promedio: real
5
6      leer nota1
7      leer nota2
8      leer nota3
9
10     Si nota1 < 0 OR nota2 < 0 OR nota3 < 0 Entonces
11         escribir "Al menos una nota es menor a cero (0)"
12     SiNo
13         Si nota1 > 10 OR nota2 > 10 OR nota3 > 10 Entonces

```

```
14      escribir "Al menos una nota es mayor a diez (10)"
15      SiNo
16          promedio <- (nota1 + nota2 + nota3) / 3
17
18      Si promedio >= 7 Entonces
19          escribir "Aprobado"
20      SiNo
21          escribir "Reprobado"
22      FinSi
23  FinSi
24  FinSi
```

Listing 4.13: Validación directa del requisito de aprobación.

Si el estudiante programa ‘promedio > 7’ en lugar de ‘>= 7’, estaría incumpliendo el requisito. El programa funcionaría, pero sería incorrecto respecto al problema.

2. Trazabilidad entre requisito y algoritmo En proyectos profesionales se recomienda establecer trazabilidad: cada requisito debe poder localizarse dentro del algoritmo o del código [40]. Aunque el lector esté comenzando, es importante adquirir este hábito desde temprano.

Una técnica sencilla para estudiantes consiste en escribir los requisitos y marcarlos conforme se implementan:

- R1: Calcular promedio.
- R2: Determinar aprobación.
- R3: Mostrar resultado.

Luego verificar que cada uno tenga una instrucción correspondiente en el pseudocódigo.

3. Simulación manual (trazado) Una herramienta fundamental para principiantes es el **trazado manual**. Consiste en ejecutar el algoritmo paso a paso con valores de prueba, anotando cada cambio de variable [5].

Ejemplo:

Valores de prueba:

$$nota1 = 8, \quad nota2 = 6, \quad nota3 = 7$$

Cálculo:

$$promedio = (8 + 6 + 7) / 3 = 7$$

Resultado esperado: “Aprobado”.

Si el resultado obtenido coincide con el esperado, el requisito se cumple.

El trazado permite detectar errores lógicos antes de ejecutar el programa en una computadora.

4. Pruebas con escenarios normales y límite Validar no significa probar un solo caso. Deben evaluarse:

- Casos normales.
- Casos límite.
- Casos inválidos.

Por ejemplo:

- Promedio = 9 → Aprobado
- Promedio = 7 → Aprobado
- Promedio = 6.9 → Reprobado

Esta práctica está alineada con los principios básicos de pruebas en ingeniería de software [24], [40].

5. Requisitos implícitos En algunos problemas existen condiciones que no están explícitamente escritas, pero que deben considerarse.

Por ejemplo, si se solicita calcular un promedio, implícitamente se asume que el número de notas no puede ser cero. Por tanto, el algoritmo debe validar esa condición:

```

1  Leer cantidadNotas
2
3  Si cantidadNotas = 0 Entonces
4      escribir "No existen notas para calcular promedio"
5  SiNo
6      promedio <- suma / cantidadNotas
7      Escribir promedio
8  FinSi

```

Listing 4.14: Control de requisito implícito: evitar división por cero.

Detectar requisitos implícitos es una habilidad que se desarrolla con práctica y análisis cuidadoso del problema.

6. Relación con la calidad del software Validar requisitos funcionales desde las primeras etapas evita retrabajo posterior y reduce errores acumulativos [23]. Incluso en programas pequeños, esta disciplina fortalece el pensamiento estructurado y prepara al estudiante para proyectos de mayor complejidad.

i

Cerrando...

Una validación rigurosa no consiste únicamente en comprobar que el programa se ejecuta sin errores sintácticos, sino en garantizar que cumple fielmente los objetivos definidos en el problema. Aprender a validar desde el inicio forma programadores más precisos, analíticos y responsables.

4.1.2. Diseño de la solución

Diseñar una solución algorítmica implica transformar el análisis del problema en una estructura organizada de pasos lógicos que puedan implementarse posteriormente en un lenguaje de programación. Mientras que el análisis responde a la pregunta “¿qué debe hacerse?”, el diseño responde a “¿cómo debe hacerse?” [23], [24].

En esta etapa el programador principiante comienza a estructurar el algoritmo antes de escribir código definitivo. Esta práctica es fundamental, ya que evita improvisaciones y reduce errores lógicos. Diseñar no significa programar inmediatamente, sino construir una representación organizada del procedimiento que resolverá el problema [5].

El diseño adecuado facilita la claridad, la mantenibilidad y la evolución del programa. Incluso en ejercicios académicos sencillos, adoptar una estructura de diseño fortalece el pensamiento computacional [17] y prepara al estudiante para proyectos de mayor complejidad.

Estrategias de descomposición

La descomposición consiste en dividir un problema complejo en subproblemas más pequeños y manejables. Esta estrategia es uno de los principios centrales de la algoritmia y de la ingeniería de software [13], [53].

Un problema raramente se resuelve en un único bloque de instrucciones. Por ejemplo, si se desea desarrollar un sistema de gestión de estudiantes, pueden identificarse subproblemas como:

- Registrar estudiante.
- Calcular promedio.
- Determinar aprobación.
- Generar reporte.

Cada uno de estos elementos puede diseñarse de manera independiente. Esta técnica reduce la complejidad cognitiva y mejora la organización del algoritmo.

En programación estructurada, la descomposición se implementa mediante procedimientos o funciones. En términos formativos, el estudiante debe aprender a preguntarse:

- ¿Puede dividirse este problema en partes más simples?
- ¿Cada parte realiza una función claramente definida?

La correcta descomposición favorece la modularidad, un principio esencial en el desarrollo de software [40].

Refinamiento sucesivo

El refinamiento sucesivo, también conocido como diseño descendente (top-down), consiste en comenzar con una descripción general del algoritmo e ir detallándolo progresivamente hasta alcanzar instrucciones concretas [13].

Por ejemplo, un algoritmo general podría describirse inicialmente como:

```
1      1. Leer datos
2      2. Procesar información
3      3. Mostrar resultados
```

Listing 4.15: Descripción inicial del algoritmo mediante refinamiento sucesivo.

En un segundo nivel de detalle:

```
1      1. Leer nota1, nota2, nota3
2      2. Calcular promedio
3      3. Determinar si aprueba
4      4. Mostrar mensaje correspondiente
```

Listing 4.16: Segundo nivel de refinamiento.

El refinamiento continúa hasta que cada paso sea implementable directamente en un lenguaje de programación.

Este enfoque evita comenzar con detalles innecesarios y permite mantener una visión estructurada del problema. Es una técnica ampliamente reconocida en la literatura clásica de algoritmos [52].

Selección de estructuras lógicas

Una vez descompuesto y refinado el problema, el siguiente paso es seleccionar las estructuras lógicas apropiadas. En programación estructurada existen tres estructuras fundamentales:

- Secuencia
- Selección (condicional)
- Iteración (repetición)

Estas estructuras son suficientes para construir cualquier algoritmo computable [36], [53]. Por ejemplo:

- Secuencia: leer datos y calcular promedio.
- Selección: decidir si el estudiante aprueba.
- Iteración: recorrer una lista de estudiantes.

Elegir incorrectamente una estructura puede generar errores lógicos. Por ejemplo, utilizar un ciclo cuando solo se requiere una condición produce complejidad innecesaria.

La selección de estructuras lógicas adecuadas constituye una habilidad que se fortalece con práctica y análisis cuidadoso.

Paradigmas aplicables en el diseño de soluciones

El diseño de una solución puede abordarse desde distintos paradigmas de programación. Un paradigma es un enfoque conceptual que orienta la forma en que se organiza y estructura el programa [35].

1. Paradigma procedimental. En el enfoque procedimental, el programa se organiza como una secuencia de instrucciones agrupadas en procedimientos o funciones. Este paradigma enfatiza el flujo de control y la transformación de datos [60]. Es especialmente adecuado para problemas bien definidos y de tamaño reducido.

2. Orientación a procesos. En soluciones más amplias, especialmente en sistemas organizacionales, el diseño puede centrarse en procesos completos (por ejemplo, proceso de inscripción o proceso de facturación). Este enfoque está alineado con la gestión de procesos de negocio (BPM) y facilita la automatización estructurada [24].

3. Orientación a objetos. El paradigma orientado a objetos organiza el sistema en entidades llamadas objetos, que combinan datos y comportamiento [20]. En lugar de centrarse únicamente en funciones, se modelan entidades del mundo real como “Estudiante”, “Empleado” o “Cuenta”.

Este paradigma favorece la reutilización, la modularidad y la escalabilidad. Aunque puede ser complejo para principiantes, es importante introducir sus fundamentos desde etapas tempranas.

4. Orientación a contextos. En sistemas modernos, especialmente distribuidos o basados en servicios, el diseño puede estructurarse alrededor de contextos específicos (por ejemplo, contexto académico, contexto financiero). Este enfoque facilita la separación de responsabilidades y reduce dependencias innecesarias [40].

i

Importante...

Para estudiantes que inician en programación, el paradigma procedimental suele ser el punto de partida natural. A medida que el nivel de complejidad aumenta, pueden incorporarse enfoques orientados a objetos o a procesos. Comprender que existen distintos paradigmas amplía la visión del programador y le permite seleccionar la estrategia más adecuada según la naturaleza del problema.

4.1.3. Verificación y mejora continua

El desarrollo de una solución no concluye cuando el algoritmo compila o produce un resultado esperado en un caso de prueba. En programación, especialmente en contextos formativos, es fundamental comprender que la calidad del software se construye mediante un proceso continuo de verificación, corrección y mejora [23], [24].

La verificación responde a la pregunta: ¿el programa está funcionando correctamente según el diseño? La mejora continua responde a otra pregunta complementaria: ¿puede hacerse de manera más clara, más eficiente o más robusta?

Incluso en programas pequeños desarrollados por aprendices de programación, aplicar principios básicos de verificación y mejora fortalece el pensamiento disciplinado y prepara para prácticas profesionales formales [40].

Pruebas funcionales

Las pruebas funcionales consisten en ejecutar el programa con distintos valores de entrada para comprobar que produce los resultados esperados. Su objetivo es verificar que cada requisito funcional se cumple correctamente.

Para un estudiante que inicia, probar no significa simplemente “ejecutar el programa una vez”, sino diseñar casos de prueba representativos:

- Casos normales.
- Casos límite.
- Casos inválidos.

Por ejemplo, en un programa que determina si un estudiante aprueba:

```
1      leer promedio
2
3      Si promedio >= 7 Entonces
4          escribir "Aprobado"
5      SiNo
6          escribir "Reprobado"
7      FinSi
```

Listing 4.17: Algoritmo básico para determinar aprobación.

Los casos de prueba podrían ser:

- promedio = 9 → Aprobado
- promedio = 7 → Aprobado
- promedio = 6.9 → Reprobado

Diseñar estas pruebas permite verificar que la condición lógica está correctamente implementada.

Las pruebas funcionales forman parte del proceso de aseguramiento de calidad y se aplican en todas las etapas del ciclo de vida del software [40]. Aunque el lector esté aprendiendo, incorporar esta práctica desde el inicio reduce errores y fortalece la precisión conceptual.

Depuración

La depuración (debugging) es el proceso de localizar y corregir errores en un programa. Un error puede ser:

- Sintáctico (error en la escritura del código).
- Lógico (el programa compila, pero produce resultados incorrectos).
- De ejecución (por ejemplo, división por cero).

En el aprendizaje inicial, los errores lógicos son los más difíciles de detectar porque el programa “parece funcionar”. Considérese el siguiente algoritmo incorrecto:

```
1 promedio <- nota1 + nota2 + nota3 / 3
```

Listing 4.18: Ejemplo con error lógico en cálculo de promedio.

Aquí el error no es sintáctico, sino de precedencia de operadores. El resultado no será el promedio correcto, será la suma de las dos primeras calificaciones más la tercera dividida para 3.

La corrección sería:

```
1 promedio <- (nota1 + nota2 + nota3) / 3
```

Listing 4.19: Corrección del cálculo de promedio.

La depuración efectiva implica:

- Analizar paso a paso la ejecución.
- Utilizar valores simples para verificar cálculos.
- Revisar cuidadosamente condiciones lógicas.

En ingeniería de software, la depuración es considerada una actividad sistemática que complementa las pruebas [51]. Aprender a depurar fortalece la comprensión profunda del algoritmo.

Optimización básica

Una vez que el programa funciona correctamente, puede analizarse si su solución es eficiente. Optimizar no significa complicar el código, sino mejorar su desempeño o claridad sin alterar su comportamiento funcional.

Existen dos dimensiones básicas de optimización:

- Eficiencia temporal (tiempo de ejecución).
- Eficiencia espacial (uso de memoria).

Considérese el siguiente algoritmo para sumar los primeros n números:

```
1 suma <- 0
2 para i <- 1 hasta n hacer
3     suma <- suma + i
4 fin para
```

Listing 4.20: Suma iterativa de los primeros n números.

Este algoritmo tiene complejidad lineal $O(n)$. Sin embargo, existe una fórmula matemática equivalente:

$$suma = \frac{n(n+1)}{2}$$

Implementación optimizada:

```
1 suma <- n * (n + 1) / 2
```

Listing 4.21: Suma optimizada mediante fórmula matemática.

La segunda solución es más eficiente, ya que no requiere iteraciones. Este tipo de análisis corresponde a los fundamentos de algoritmos y complejidad computacional [36], [53].

No obstante, para estudiantes que inician, la prioridad debe ser primero la corrección y claridad del algoritmo. La optimización prematura puede introducir complejidad innecesaria.

i

Cerrando...

La verificación, la depuración y la optimización constituyen un ciclo continuo de mejora. Programar no es únicamente escribir instrucciones, sino revisar, corregir y perfeccionar la solución hasta que sea correcta, clara y eficiente. Este enfoque disciplinado diferencia la programación improvisada del desarrollo estructurado de software [23], [40].

4.2. Algoritmos: fundamento para soluciones organizacionales

4.2.1. Definición y propósito en IS

Un algoritmo es un conjunto finito y ordenado de pasos que describen cómo transformar entradas en salidas, cumpliendo condiciones de precisión, efectividad, finitud y determinismo [7], [23]. La definición formal de algoritmo, estudiada ampliamente en teoría computacional y programación, establece que cada instrucción debe ser clara e inequívoca, y que el procedimiento debe garantizar resultados correctos en un número finito de pasos [24], [34]. Los algoritmos constituyen la base de toda solución computacional, ya que definen la lógica que ejecutarán los programas [9].

4.2.2. Definición formal

En su definición formal, un algoritmo debe especificar pasos precisos, ejecutables y no ambiguos, siguiendo la teoría desarrollada en la literatura clásica de programación y sistemas [34]. Cada paso debe representar una instrucción que pueda ser interpretada por un agente computacional, ya sea humano o máquina, garantizando claridad en su ejecución [23]. Este principio de precisión evita interpretaciones múltiples y asegura consistencia en resultados [24].

Además, un algoritmo debe ser finito, es decir, debe concluir su ejecución en un número limitado de pasos [7]. La finitud distingue los algoritmos de procesos indefinidos o recursiones infinitas y permite que las soluciones computacionales sean

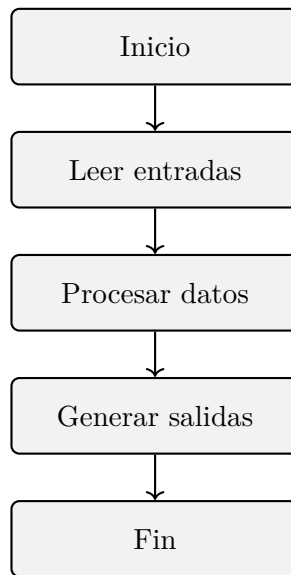


Figura 4.1: Estructura básica de un algoritmo [7], [9].

verificables y medibles [9]. La literatura técnica enfatiza que la ausencia de finitud puede provocar bloqueos, ciclos infinitos o fallas en sistemas críticos [23].

La efectividad del algoritmo implica que cada operación especificada debe ser realizable mediante recursos computacionales disponibles y debe producir el efecto esperado sobre los datos [34]. Esta propiedad garantiza que los algoritmos puedan implementarse en hardware real y se ejecuten dentro de tiempos razonables, conforme a los parámetros de eficiencia establecidos en ingeniería de software [24].

4.2.3. Propósito de los algoritmos en IS

En la carrera de Ingeniería en Sistemas de Información (IS), el aprendizaje de algoritmos no constituye un ejercicio abstracto desligado de la realidad organizacional, sino el fundamento técnico mediante el cual los futuros profesionales modelan, formalizan y automatizan procesos propios de los Sistemas de Información (SI). Un algoritmo, en este contexto, es la representación estructurada de una transformación organizada de datos en información significativa que respalda operaciones y decisiones [1], [2], [23].

Desde el punto de vista epistemológico, el algoritmo opera como mecanismo de formalización: convierte una situación organizacional —descrita en lenguaje natural— en una secuencia finita, precisa y verificable de pasos ejecutables. Esta formalización es esencial en IS, pues permite traducir reglas administrativas, financieras o académicas en estructuras lógicas que pueden ser implementadas computacionalmente [24], [40]. El estudiante aprende así que programar no es escribir instrucciones aisladas, sino modelar rigurosamente comportamientos organizacionales.

En los SI, los datos representan hechos registrados (ventas, calificaciones, inventarios, transacciones). Sin embargo, la información surge únicamente cuando esos datos son procesados mediante reglas definidas. El algoritmo constituye el núcleo de ese procesamiento. Por ejemplo, calcular un promedio académico, determinar el saldo disponible de una cuenta o validar una solicitud implica aplicar una secuencia de operaciones formales que transforman valores de entrada en resultados interpretables [2], [44]. Sin algoritmo, el SI se limita a almacenar datos; con algoritmo, genera conocimiento operativo.

Desde la perspectiva formativa en IS, el propósito de aprender algoritmos es desarrollar pensamiento estructurado. Diseñar un algoritmo implica descomponer un problema, identificar variables relevantes, definir condiciones y anticipar resultados posibles. Esta práctica fortalece el razonamiento lógico y la capacidad de análisis sistemático, competencias centrales en la disciplina [17], [53]. El estudiante comprende progresivamente que cada decisión algorítmica refleja una interpretación del problema organizacional.

Además, los algoritmos implementan reglas de negocio. Una política institucional, como aplicar un descuento específico,

validar un requisito mínimo o restringir operaciones fuera de horario, debe traducirse en condiciones explícitas dentro del código. Esta traducción exige claridad conceptual y precisión formal [23], [24]. La calidad del SI depende directamente de la fidelidad con que el algoritmo represente dichas reglas.

i

Concluyendo...

El propósito de los algoritmos en IS no se limita a producir resultados correctos desde el punto de vista técnico, sino a garantizar coherencia entre modelo organizacional y ejecución computacional. Aprender a programar dentro de la carrera implica asumir que cada algoritmo diseñado tiene implicaciones reales en el funcionamiento de los SI. Esta conciencia profesional diferencia la programación instrumental de la ingeniería disciplinada.

4.2.4. Los algoritmos y los SI

Un algoritmo no es simplemente una secuencia de instrucciones técnicas; en el contexto de los Sistemas de Información (SI), constituye el mecanismo formal mediante el cual los datos se transforman en información útil para la organización. Mientras que en matemáticas un algoritmo puede resolver una ecuación, en SI su propósito es apoyar procesos, decisiones y operaciones institucionales [1], [2].

Desde la perspectiva conceptual, los Sistemas de Información operan sobre datos que, por sí solos, carecen de significado operativo. La transformación de datos en información requiere estructura, contexto y procesamiento [44], [45]. El algoritmo es precisamente el instrumento que realiza esa transformación. Por ejemplo, un conjunto de números almacenados en una base de datos no constituye información hasta que un algoritmo calcula promedios, tendencias, totales o indicadores relevantes.

1. Algoritmos como transformadores de datos. En términos formales, un algoritmo puede representarse como una función:

$$f : \text{Entrada} \rightarrow \text{Salida}$$

En Sistemas de Información:

- Entrada: datos almacenados (notas, ventas, salarios, inventarios).
- Proceso: reglas de negocio implementadas mediante estructuras lógicas.
- Salida: información interpretada (promedios, reportes, alertas).

Por ejemplo, en un sistema académico:

```
1      promedio <- (nota1 + nota2 + nota3) / 3
2
3      Si promedio >= 7 Entonces
4          estado <- "Aprobado"
5      SiNo
6          estado <- "Reprobado"
7      FinSi
```

Listing 4.22: Algoritmo simple de transformación de datos en información.

Aquí, los datos (notas individuales) se convierten en información significativa (estado académico). Este proceso representa el propósito esencial del algoritmo dentro del sistema.

2. Algoritmos como formalización de reglas de negocio. En IS, los algoritmos implementan reglas organizacionales. Estas reglas pueden provenir de normativas institucionales, políticas internas o requisitos legales [24].

Por ejemplo:

- “Aplicar 12% de descuento a empleados permanentes.”
- “Rechazar solicitudes fuera del plazo establecido.”
- “Generar alerta cuando el inventario sea menor a 10 unidades.”

Cada una de estas reglas debe traducirse en condiciones lógicas dentro del algoritmo.

```
1      Si inventario < 10 Entonces
2          generarAlerta()
3      FinSi
```

Listing 4.23: Regla de negocio implementada como algoritmo.

El algoritmo actúa como un puente entre la política organizacional y la ejecución computacional.

3. Algoritmos como soporte a la toma de decisiones. Los Sistemas de Información no solo registran datos; también apoyan decisiones. Un algoritmo puede calcular indicadores que permitan evaluar desempeño, riesgos o tendencias [2].

Por ejemplo, en un sistema financiero:

```
1      utilidad <- ingresos - costos
2
3      Si utilidad > meta Entonces
4          escribir "Meta alcanzada"
5      SiNo
6          Escribir "Meta no alcanzada"
7      FinSi
```

Listing 4.24: Cálculo de utilidad como apoyo a decisiones.

El resultado no es únicamente un número, sino un insumo para decisiones administrativas.

4. Relación con el pensamiento computacional. El propósito de los algoritmos en IS también está vinculado al desarrollo del pensamiento computacional. Diseñar un algoritmo implica descomponer un problema, identificar patrones, definir reglas y estructurar soluciones paso a paso [17].

Esta habilidad no solo permite programar, sino también analizar procesos organizacionales desde una perspectiva estructurada.

5. Impacto en la calidad y confiabilidad del sistema. Un algoritmo incorrecto no solo produce errores técnicos; puede generar información errónea que afecte decisiones reales. Por ejemplo, un error en el cálculo de impuestos o nómina puede tener consecuencias financieras significativas.

Por ello, la calidad del algoritmo influye directamente en la confiabilidad del Sistema de Información [23], [40]. Diseñar algoritmos claros, verificables y correctamente validados es esencial para que el sistema cumpla su propósito organizacional.

i

Sintetizando...

En síntesis, el propósito de los algoritmos en Sistemas de Información es transformar datos en información significativa, implementar reglas de negocio y apoyar la toma de decisiones. Aprender a programar implica comprender que cada línea de código tiene un impacto potencial en la operación real de una organización. Esta conciencia otorga sentido y responsabilidad al proceso de diseño algorítmico.

4.2.5. Propiedades esenciales

Los algoritmos poseen propiedades esenciales que permiten evaluarlos: finitud, determinismo, efectividad y correctitud [23]. La finitud garantiza que el procedimiento terminará; el determinismo asegura que cada paso es claro y no admite ambigüedades; y la efectividad indica que cada operación es realizable mediante computación real [24]. Estas propiedades definen la naturaleza formal de los algoritmos y distinguen procedimientos correctos de descripciones vagas o indefinidas [9].

El determinismo implica que un algoritmo, dado un conjunto de entradas, debe producir siempre las mismas salidas siguiendo los mismos pasos, sin depender de factores externos o aleatoriedad [34]. Esta propiedad es fundamental en sistemas críticos, donde decisiones automáticas deben ser reproducibles y auditables [23]. Los modelos de evaluación de algoritmos analizan esta característica mediante pruebas de caja negra y caja blanca [24].

La efectividad está asociada a que cada instrucción del algoritmo corresponda a operaciones simples, realizables y comprensibles, evitando acciones abstractas imposibles de implementar en hardware o software [7]. Esta propiedad asegura la aplicabilidad del algoritmo en sistemas reales y guía el diseño de soluciones computacionales eficientes [9].

Finitud

La finitud indica que un algoritmo debe tener un número limitado de pasos y que su ejecución debe terminar necesariamente [23]. Este principio permite garantizar que los programas no queden atrapados en ciclos infinitos ni consuman recursos indefinidamente, problema recurrente en sistemas mal diseñados [24]. La literatura de algoritmia enfatiza que cualquier algoritmo que no garantice finitud no es computacionalmente válido [9].

Ejemplos clásicos incluyen algoritmos iterativos que usan condiciones de parada explícitas y algoritmos recursivos que definen casos base que permiten terminar la ejecución [34]. Por ejemplo, el cálculo del factorial de un número natural finaliza cuando se alcanza el caso base $0! = 1$, garantizando finitud y correctitud [7]. Esta propiedad debe verificarse durante el diseño para evitar comportamientos anómalos en ejecución [23].

El análisis de finitud también se aplica en validaciones formales mediante métodos matemáticos que permiten demostrar que el algoritmo no continuará indefinidamente, práctica documentada en estudios de verificación y análisis de programas [24]. Esta verificación es crucial en sistemas críticos, como control industrial o aeronáutica [9].

Determinismo

El determinismo establece que cada paso de un algoritmo debe definir una acción única y claramente especificada, de modo que no exista ambigüedad en su interpretación o ejecución [23]. Esta propiedad garantiza que, para un mismo conjunto de entradas, el algoritmo producirá siempre las mismas salidas siguiendo el mismo camino de ejecución, característica fundamental en sistemas predecibles y auditablemente correctos [24]. La ausencia de determinismo puede introducir incertidumbre en el comportamiento del sistema, generando riesgos en aplicaciones críticas como sistemas bancarios, médicos o aeronáuticos [9].

En la práctica, el determinismo implica que las operaciones deben estar definidas con precisión, evitando expresiones ambiguas como “ordenar adecuadamente” o “seleccionar un elemento adecuado”, que no especifican una acción concreta [34]. Los lenguajes de programación refuerzan este principio mediante reglas estrictas de sintaxis y semántica que permiten traducir instrucciones a operaciones de máquina sin interpretaciones inconsistentes [7]. La claridad fomentada por el determinismo mejora la mantenibilidad y facilita la verificación del software [23].

Asimismo, el determinismo posibilita la reproducibilidad, condición necesaria para validar algoritmos mediante pruebas y análisis formal [24]. Si un algoritmo no es determinista, los resultados podrían variar entre ejecuciones incluso bajo condiciones idénticas, dificultando su verificación y su uso en contextos que requieren alta confiabilidad [9]. Por estas razones, el determinismo constituye un pilar en la teoría clásica de algoritmos.

Efectividad

La efectividad se refiere a que cada operación descrita en un algoritmo debe ser realizable mediante un conjunto elemental de acciones ejecutables por un agente computacional [23]. Esto implica que las instrucciones deben ser suficientemente simples como para ser implementadas en hardware o software real, y deben garantizar que la acción especificada produzca el efecto esperado sobre los datos [24]. La efectividad evita la presencia de pasos vagos o imposibles de ejecutar, lo cual fortalecería la inconsistencia del algoritmo [9].

Los algoritmos efectivos se basan en operaciones elementales tales como asignación, comparación, aritmética básica, lectura y escritura, todas ellas compatibles con la arquitectura del computador descrita por modelos como la arquitectura de Von Neumann [34]. Por ejemplo, un algoritmo que requiera “adivinar el resultado adecuado” no sería efectivo porque no define una operación mecanizable; en cambio, un algoritmo que busque un elemento realizando comparaciones secuenciales sí lo es [7]. La efectividad garantiza que un algoritmo pueda ser concretado en un programa real.

Además, la efectividad contribuye al tiempo de ejecución razonable del algoritmo, dado que las operaciones elementales permiten prever el costo computacional de cada paso [23]. Esta característica se vincula estrechamente con el análisis de eficiencia, pues los algoritmos efectivos pueden evaluarse más fácilmente utilizando modelos formales de complejidad temporal y espacial [9]. En consecuencia, la efectividad es indispensable para asegurar que el algoritmo sea útil en la práctica.

4.2.6. Correctitud y eficiencia

La correctitud de un algoritmo implica que produce salidas válidas para todas las entradas válidas y que cumple con la especificación formal del problema [23]. Esta propiedad se fundamenta en la verificación de que cada paso contribuye exactamente al resultado esperado y de que las condiciones previas y posteriores a cada operación son coherentes con los requisitos del sistema [24]. La correctitud es un prerrequisito esencial, especialmente en sistemas donde los errores pueden provocar fallas graves o pérdidas económicas significativas [9].

La eficiencia, por su parte, se relaciona con el uso óptimo de recursos computacionales, principalmente tiempo de ejecución y memoria [34]. Algoritmos que realizan la misma tarea pueden diferir radicalmente en eficiencia; por ejemplo, ordenar una lista mediante selección tiene complejidad $O(n^2)$, mientras que algoritmos como Quicksort o Mergesort logran $O(n \log n)$ [23]. La eficiencia determina la escalabilidad de una solución y su aplicabilidad en escenarios de grandes volúmenes de datos [24].

La combinación de correctitud y eficiencia constituye el fundamento de la calidad algorítmica. Un algoritmo correcto pero ineficiente podría ser impracticable, mientras que uno eficiente pero incorrecto es inútil o peligroso [9]. Por ello, el diseño de algoritmos exige equilibrar claridad, precisión y rendimiento desde las etapas iniciales del proceso de programación [23].

Corrección parcial y total

La corrección parcial establece que, si el algoritmo termina, entonces su resultado es correcto de acuerdo con la especificación formal [24]. Este tipo de corrección verifica la validez del resultado, pero no garantiza que el algoritmo siempre concluya su ejecución [23]. Un algoritmo con corrección parcial podría presentar ciclos infinitos bajo ciertas condiciones, lo que hace necesario evaluar adicionalmente su finitud [9].

La corrección total combina la corrección parcial con la finitud, exigiendo que el algoritmo siempre termine y que lo haga produciendo el resultado correcto [34]. Esta propiedad es más estricta y se considera el estándar de calidad en sistemas donde la terminación es crítica, como en control industrial, sistemas bancarios y aplicaciones embebidas [24]. La literatura enfatiza que demostrar corrección total requiere analizar tanto la validez lógica de los pasos como las condiciones de terminación [23].

La verificación formal de algoritmos mediante métodos matemáticos, invariantes de bucles y pruebas estructuradas permite establecer corrección parcial y total con rigor científico [9]. Esta verificación es esencial en contextos de alta confiabilidad y se apoya en herramientas computacionales que automatizan parte del proceso [24]. Al aplicar estas técnicas, se garantiza que un algoritmo sea fiable, predecible y adecuado para sistemas complejos [23].

Eficiencia intuitiva

La eficiencia intuitiva se refiere a la capacidad del programador para estimar el rendimiento de un algoritmo basándose en su estructura, sin necesidad inmediata de análisis formal [23]. Esta intuición permite identificar rápidamente soluciones

potencialmente ineficientes, como bucles anidados innecesarios o recorridos repetitivos de estructuras de datos [24]. Aunque no reemplaza el análisis riguroso, la intuición sobre eficiencia guía la selección preliminar de técnicas algorítmicas [9].

Por ejemplo, un programador reconoce intuitivamente que ordenar una lista dos veces es menos eficiente que ordenar una sola vez, o que una búsqueda lineal es menos eficiente que una búsqueda binaria en listas ordenadas [34]. Este conocimiento se adquiere mediante experiencia, práctica y estudio de patrones de solución comunes [7]. La eficiencia intuitiva permite mejorar el rendimiento desde etapas tempranas de diseño antes de confirmar mediante análisis formal [23].

La eficiencia intuitiva promueve la toma de decisiones informadas sobre estructuras de datos y técnicas de programación, contribuyendo al desarrollo de algoritmos más robustos y escalables [24]. La literatura enfatiza que los programadores deben combinar intuición con herramientas formales como notación Big-O y análisis empírico para lograr soluciones equilibradas y eficientes [9].

4.3. Lenguajes formales y lenguajes de programación

Los lenguajes formales proporcionan reglas sintácticas y semánticas que definen cómo deben estructurarse las expresiones y cómo debe interpretarse su significado, constituyendo la base teórica de los lenguajes de programación [34]. Estos lenguajes permiten especificar algoritmos de manera precisa y verificable mediante gramáticas, alfabetos, producciones y modelos matemáticos, como autómatas y máquinas de Turing [24]. Su estudio es fundamental para comprender cómo los programas son analizados y traducidos por compiladores e intérpretes [9].

Los lenguajes de programación, contruidos sobre los principios de los lenguajes formales, permiten implementar algoritmos en una forma que pueda ejecutarse en una computadora [7]. Cada lenguaje incorpora reglas sintácticas estrictas y modelos semánticos que describen cómo deben ejecutarse las instrucciones, asegurando predictibilidad y consistencia en los resultados [23]. La elección de un lenguaje depende de factores como el dominio del problema, la eficiencia requerida, la portabilidad y la facilidad de mantenimiento [24].

4.3.1. Sintaxis

La sintaxis describe la estructura válida de instrucciones, expresiones y programas dentro de un lenguaje de programación, definiendo cómo deben organizarse los símbolos para formar construcciones correctas [34]. Los compiladores realizan análisis sintáctico para verificar que el código cumpla con estas reglas antes de traducirlo a instrucciones ejecutables, proceso ampliamente documentado en textos de construcción de compiladores [9]. La sintaxis evita ambigüedades y garantiza que el código sea comprensible por la máquina y por otros programadores [7].

Errores sintácticos como falta de paréntesis, comas incorrectas o instrucciones mal formadas son detectados en esta etapa, permitiendo corregirlos antes de la ejecución [23]. La claridad sintáctica contribuye a la mantenibilidad y legibilidad del código, y es un componente esencial en la calidad del software [24]. Por ello, los lenguajes modernos incorporan reglas explícitas y herramientas que ayudan a detectar errores sintácticos de forma temprana [9].

Reglas gramaticales

Las reglas gramaticales de un lenguaje de programación describen, mediante una gramática formal, cómo pueden combinarse los símbolos del lenguaje (palabras clave, identificadores, operadores, delimitadores) para formar construcciones válidas como sentencias, bloques y programas completos [34]. Estas gramáticas suelen especificarse en notaciones como BNF (Backus-Naur Form) o EBNF (Extended BNF), las cuales permiten expresar producciones del tipo **sentencia** ::= **if expresion then sentencia** [9]. El uso de gramáticas formales facilita el análisis automático del código por parte de compiladores e intérpretes [24].

Los analizadores sintácticos (*parsers*) utilizan estas reglas gramaticales para construir árboles de derivación o árboles sintácticos que representan la estructura jerárquica del programa [23]. Por ejemplo, una expresión aritmética como **a + b * c** puede representarse como un árbol donde la multiplicación tiene mayor precedencia que la suma, lo cual refleja las reglas gramaticales del lenguaje [34]. La correcta definición de estas reglas evita ambigüedades y garantiza interpretaciones consistentes del código [9].

En la literatura se enfatiza que las gramáticas de los lenguajes modernos se diseñan para ser no ambiguas o, al menos, para que la ambigüedad se resuelva mediante reglas adicionales de precedencia y asociatividad [24]. Esto permite que el

mismo programa sea interpretado de igual manera por diferentes compiladores conformes al estándar del lenguaje [23]. La precisión gramatical es por tanto un requisito para la portabilidad y la confiabilidad del software [9].

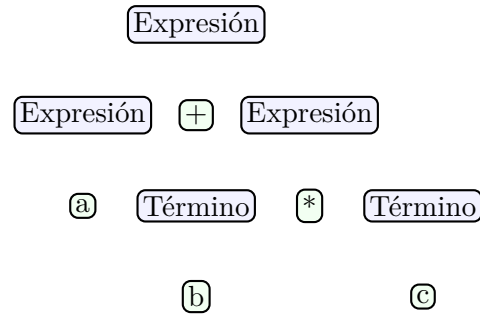


Figura 4.2: Árbol sintáctico para la expresión $a + b * c$ [9], [34].

Formación de expresiones

La formación de expresiones en un lenguaje de programación se rige por reglas que especifican cómo combinar operandos y operadores para producir valores [7]. Estas reglas definen qué operadores son válidos para cada tipo de dato, cuál es su precedencia y cómo se asocian en ausencia de paréntesis explícitos [34]. Por ejemplo, en la mayoría de lenguajes, la multiplicación tiene mayor precedencia que la suma, y operadores binarios como $+$ o $-$ suelen ser asociativos por la izquierda [9].

Las expresiones no se limitan a operaciones aritméticas; incluyen también expresiones lógicas, relacionales y de concatenación de cadenas, cada una con semánticas específicas [24]. Así, una expresión como `(edad > 18) && (pais == "ECUADOR")` combina operadores relacionales y lógicos para producir un valor booleano que controla el flujo de ejecución [7]. La correcta comprensión de estas reglas es esencial para evitar errores sutiles en condiciones y decisiones [23].

Además, muchos lenguajes permiten sobrecarga de operadores, lo cual significa que el mismo símbolo puede tener significados distintos según los tipos de sus operandos (por ejemplo, $+$ como suma numérica o concatenación de cadenas) [34]. Esta característica, aunque poderosa, puede introducir confusión si no se documenta y utiliza adecuadamente [24]. La literatura recomienda emplear la sobrecarga con moderación y claridad para preservar la legibilidad [23].

Las expresiones también interactúan con conversiones implícitas y explícitas de tipos (“casting”), por lo que una formación incorrecta puede producir pérdida de información o resultados inesperados [9]. Por ejemplo, dividir dos enteros puede generar un resultado entero truncado, mientras que dividir números de punto flotante conserva los decimales [7]. Estas diferencias deben considerarse cuidadosamente al diseñar algoritmos y escribir código [23].

4.3.2. Semántica

La semántica de un lenguaje de programación describe el significado de las construcciones sintácticamente válidas, indicando qué efecto tienen sobre el estado del programa y del sistema al ejecutarse [34]. Mientras la sintaxis responde a la pregunta “¿está bien formada esta instrucción?”, la semántica responde “¿qué hace esta instrucción?”, tal como se discute en la literatura de lenguajes formales y compiladores [9]. La semántica puede describirse de manera informal (texto natural), operacional (mediante máquinas abstractas) o denotacional (mediante funciones matemáticas) [24].

Comprender la semántica es fundamental para razonar sobre la corrección y el comportamiento del programa, especialmente en presencia de estructuras de control, llamadas a funciones, recursión y manejo de memoria [23]. Por ejemplo, dos fragmentos de código pueden ser sintácticamente válidos pero semánticamente diferentes si modifican variables de formas distintas o si manejan errores de manera divergente [7]. La semántica precisa permite predecir el efecto de las instrucciones y verificar la adherencia a los requisitos del sistema [24].

Significado operativo

El significado operativo describe la semántica de un programa en términos de los pasos que realiza una máquina abstracta al ejecutar sus instrucciones [34]. Esta perspectiva se formaliza mediante la definición de estados del programa (valores de

variables, posición en el código, contenido de memoria) y reglas de transición que indican cómo cada instrucción transforma dicho estado [9]. De este modo, es posible modelar la ejecución como una secuencia de estados que evoluciona siguiendo la lógica del programa [24].

Por ejemplo, una sentencia de asignación $x = x + 1$; puede interpretarse como una transición desde un estado en el que $x = n$ a otro en el que $x = n + 1$, manteniendo constantes el resto de variables [23]. De manera similar, una instrucción condicional define bifurcaciones en el espacio de estados según el valor de la condición evaluada [34]. Esta visión facilita el análisis de flujos de control, la detección de caminos no alcanzables y la verificación de propiedades como invariantes de bucle [9].

El significado operativo se utiliza tanto en la especificación formal de lenguajes como en herramientas de verificación y depuración, donde se simulan estados y transiciones para detectar errores [24]. La comprensión de esta semántica permite a los programadores anticipar cómo se comportará el código en diferentes escenarios de ejecución, contribuyendo a un diseño más robusto y seguro [23].

Errores semánticos

Los errores semánticos ocurren cuando el programa es sintácticamente correcto pero su comportamiento no coincide con la intención del programador o con la especificación del problema [7]. Por ejemplo, utilizar el operador de asignación en lugar del operador de comparación dentro de una condición, o aplicar la fórmula errónea para un cálculo, produce resultados incorrectos sin que el compilador detecte necesariamente un fallo [23]. Estos errores son especialmente peligrosos porque pueden pasar inadvertidos hasta que se observan resultados inconsistentes en producción [24].

La detección de errores semánticos requiere análisis cuidadoso, pruebas exhaustivas y, en algunos casos, revisión por pares [9]. Las técnicas de pruebas de caja negra y caja blanca, combinadas con revisiones de código y análisis estático, ayudan a identificar discrepancias entre el comportamiento observado y el comportamiento esperado [23]. La comprensión profunda de la semántica del lenguaje y del dominio del problema es un factor clave para prevenir y corregir estos errores [24].

4.3.3. Errores comunes en principiantes

Los lectores que se inician en la programación suelen cometer errores recurrentes derivados de dificultades para comprender la sintaxis, la semántica y la lógica de los programas [7]. Entre los más frecuentes se encuentran omitir delimitadores, confundir operadores, utilizar tipos de datos inadecuados, diseñar condiciones incorrectas y olvidar casos especiales en estructuras de control [24]. La literatura sobre enseñanza de la programación resalta la importancia de abordar explícitamente estos errores para favorecer el desarrollo de competencias sólidas [23].

Además, muchos principiantes tienden a centrarse en “hacer que el programa funcione” sin analizar si la solución es correcta, eficiente o mantenible [7]. Esta actitud puede conducir a código difícil de entender, duplicación de lógica y errores que reaparecen al introducir cambios [24]. Por ello, se recomienda enfatizar buenas prácticas desde los primeros cursos, incluyendo trazado de código, diseño previo, pruebas sistemáticas y reflexión sobre la solución [23].

Errores sintácticos

Los errores sintácticos se producen cuando el código viola las reglas de formación del lenguaje, como falta de paréntesis, llaves desbalanceadas, palabras clave mal escritas o uso incorrecto de signos de puntuación [7]. Estos errores impiden que el compilador o intérprete genere código ejecutable y suelen acompañarse de mensajes de error que indican la ubicación aproximada del problema [9]. En general, los errores sintácticos se detectan en fases tempranas del proceso de compilación [34].

Los principiantes a menudo encuentran dificultades para interpretar los mensajes de error del compilador, que pueden referirse a la línea detectada aunque el problema se origine en líneas previas [7]. La literatura didáctica recomienda desarrollar el hábito de revisar cuidadosamente el código cercano a la posición indicada, buscando patrones comunes como delimitadores faltantes o mal ubicados [24]. Comprender la relación entre gramática del lenguaje y mensajes de error facilita la corrección [23].

Para reducir errores sintácticos, se aconseja utilizar entornos de desarrollo que proporcionen resaltado de sintaxis, autocompletado y verificación incremental del código [9]. Estas herramientas ayudan a detectar problemas mientras se escribe el programa, evitando acumulación de errores y mejorando la productividad [23]. No obstante, la responsabilidad última de comprender la sintaxis recae en el programador [7].

Errores lógicos

Los errores lógicos ocurren cuando el programa se ejecuta sin generar errores sintácticos o de tiempo de ejecución, pero produce resultados incorrectos debido a fallos en el razonamiento o en el diseño de la solución [7]. Estos errores pueden surgir por condiciones mal formuladas, bucles que no cubren todos los casos, cálculos erróneos o uso inadecuado de variables [23]. A diferencia de los errores sintácticos, los errores lógicos no son detectados directamente por el compilador [24].

Un ejemplo típico de error lógico es utilizar un operador relacional equivocado en una condición, como $>$ en lugar de $>=$, lo que excluye un caso límite importante [7]. Otro caso frecuente es inicializar mal una variable de acumulación o contador, lo que afecta el resultado final de un cálculo iterativo [24]. Estos errores pueden permanecer ocultos si las pruebas no cubren adecuadamente los casos de frontera [23].

La literatura sobre buenas prácticas recomienda utilizar técnicas de trazado de código (*dry run*), depuración paso a paso y registro de valores intermedios para identificar el punto exacto en el que la lógica se desvía de lo esperado [7]. Estas estrategias permiten comparar el comportamiento real del programa con el comportamiento previsto en el diseño algorítmico [23]. En entornos educativos, el trazado manual es una herramienta clave para desarrollar comprensión profunda de la ejecución [24].

Además, el uso de pruebas sistemáticas, incluyendo casos típicos, extremos y casos erróneos, es esencial para detectar errores lógicos [23]. Las pruebas de caja negra se centran en la relación entrada-salida, mientras que las pruebas de caja blanca analizan rutas internas del código [24]. La combinación de ambas perspectivas incrementa la probabilidad de descubrir errores lógicos antes del despliegue [9].

La prevención de errores lógicos se apoya en un buen diseño algorítmico, en la claridad del código y en la revisión por pares [23]. Documentar decisiones, añadir comentarios significativos y mantener un estilo coherente facilitan la detección temprana de inconsistencias [24]. De este modo, la formación en pensamiento crítico y en metodologías de prueba resulta tan importante como el aprendizaje de la sintaxis del lenguaje [7].

4.4. Buenas prácticas para aprender programación

Aprender programación requiere no solo dominar la sintaxis de un lenguaje, sino también adoptar estrategias de estudio y prácticas sistemáticas que favorezcan el desarrollo del pensamiento algorítmico, la comprensión profunda del código y la capacidad de resolver problemas de forma estructurada [7]. La literatura en educación en computación destaca que los programadores que aplican métodos activos como trazado manual, depuración reflexiva y análisis de errores alcanzan niveles superiores de entendimiento y retención [24]. Asimismo, el aprendizaje colaborativo, el uso de herramientas de apoyo y la aceptación del error como parte natural del proceso son aspectos fundamentales para consolidar la competencia programadora [9], [23].

4.4.1. Métodos de estudio

Los métodos de estudio en programación deben orientarse hacia la comprensión de los conceptos fundamentales que subyacen al código, evitando la memorización superficial que conduce a errores y dificultades para resolver problemas novedosos [7]. La investigación educativa sugiere que las estrategias efectivas incluyen leer y analizar código antes de ejecutarlo, trazar el estado de las variables, realizar ejercicios progresivos y reflexionar sobre los resultados [24]. Estas prácticas fomentan el razonamiento lógico y la interpretación de algoritmos, elementos centrales en la formación de programadores [23].

Asimismo, se recomienda alternar entre estudio teórico y práctica deliberada, resolviendo ejercicios de complejidad creciente y comparando distintas aproximaciones a un mismo problema [9]. Esta combinación permite integrar conocimientos y desarrollar intuición algorítmica, habilidades que fortalecen la autonomía del programador en formación y su capacidad para enfrentar desafíos más avanzados [24]. El uso de técnicas reflexivas, como explicar en voz alta el funcionamiento de un programa, ha mostrado efectos positivos en la retención y comprensión [7].

Trazar código

Trazar código consiste en simular mental o manualmente la ejecución de un programa, registrando los cambios en las variables y el flujo de control paso a paso [7]. Esta técnica es fundamental para detectar errores lógicos, comprender

estructuras de control y verificar la coherencia entre el algoritmo y su implementación [24]. La literatura destaca que el trazado permite visualizar la dinámica del programa, facilitando la anticipación de comportamientos imprevistos [23].

El uso sistemático del trazado ayuda a comprender cómo se modifican los valores, cómo se recorren estructuras de datos y cómo se activan y desactivan diferentes ramas de ejecución [9]. Por ejemplo, en un bucle que calcula la suma de los primeros n números, trazar el estado de las variables `i` y `suma` permite verificar si las actualizaciones se realizan correctamente en cada iteración [7]. Este análisis detallado contribuye a desarrollar habilidades de depuración y razonamiento algorítmico [24].

El trazado también es útil para comparar la ejecución real del programa con la esperada según el diseño algorítmico [23]. Esta comparación revela discrepancias entre implementación e intención, lo que facilita corregir errores y mejorar la calidad del código [9]. Por ello, se considera una de las prácticas centrales en los cursos iniciales de programación [7].

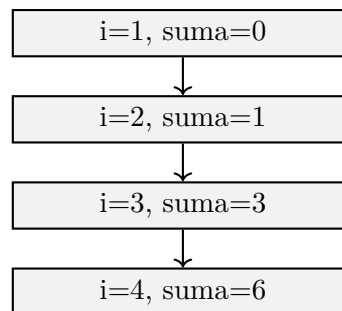


Figura 4.3: Trazado de un diagrama de flujo que suma los primeros n números naturales [7], [24].

Escribir antes que ejecutar

La práctica de escribir el algoritmo o el pseudocódigo antes de ejecutar el programa fomenta claridad conceptual y reduce errores comunes asociados a improvisación o dependencia excesiva del entorno de ejecución [23]. La literatura subraya que esta estrategia permite a los programadores organizar sus ideas, prever la estructura del programa y anticipar la interacción entre variables y estructuras de control [24]. De este modo, escribir antes de ejecutar fortalece la planificación lógica previa a la codificación [9].

Cuando los principiantes ejecutan el código sin un diseño previo, tienden a introducir cambios impulsivos que complican la detección de errores y deterioran la estructura del programa [7]. Al escribir primero, se puede analizar la solución con calma, aplicar principios de diseño modular y simplificar pasos innecesarios [23]. Este enfoque incrementa la calidad del código desde sus primeras versiones, disminuyendo la cantidad de errores arrastrados a etapas posteriores [24].

Además, escribir antes de ejecutar promueve la reflexión sobre alternativas algorítmicas, permitiendo comparar diferentes formas de resolver un mismo problema [9]. Por ejemplo, antes de implementar una búsqueda, el programador en formación puede evaluar si utilizará búsqueda lineal o binaria, dependiendo del tamaño y la estructura de los datos [34]. Esta reflexión previa contribuye al aprendizaje de patrones de solución y a la mejora de la eficiencia [23].

i

Cerrando...

Esta práctica facilita la comunicación y la revisión por pares, ya que el pseudocódigo es más accesible y menos dependiente de detalles del lenguaje que el código fuente completo [24]. Las revisiones tempranas basadas en pseudocódigo permiten detectar errores conceptuales sin el ruido adicional de la sintaxis [7]. Por estas razones, escribir antes de ejecutar es considerada una buena práctica tanto en contextos educativos como profesionales [23].

4.4.2. La importancia del error

El error desempeña un papel fundamental en el aprendizaje de la programación, ya que permite identificar malentendidos, explorar límites conceptuales y desarrollar habilidades de depuración [7]. La literatura educativa establece que los programadores que interpretan el error como una oportunidad de aprendizaje, y no como un fracaso, adquieren competencias más sólidas y persistentes en pensamiento computacional [24]. Las técnicas de depuración, análisis y revisión de código se apoyan directamente en esta disposición positiva frente al error [23].

Además, el error proporciona retroalimentación valiosa que guía la reestructuración de conceptos y corrige modelos mentales incorrectos [9]. Al equivocarse, el programador en formación debe revisar su razonamiento, contrastarlo con el comportamiento real del programa y ajustar su comprensión de la lógica computacional [23]. Este ciclo de autoevaluación y mejora continua es esencial tanto en entornos académicos como en el desarrollo profesional de software [24].

4.4.3. Trabajo colaborativo

El trabajo colaborativo favorece la construcción conjunta de conocimiento y mejora la capacidad de resolver problemas mediante discusión, análisis compartido y revisión mutua [24]. En programación, la colaboración permite que los programadores contrasten sus soluciones, detecten errores que individualmente pasarían desapercibidos y aprendan diferentes estilos y estrategias de razonamiento computacional [23]. Las metodologías ágiles y el desarrollo en pareja (*pair programming*) se apoyan en estos principios colaborativos [9].

Asimismo, la colaboración desarrolla competencias transversales como comunicación efectiva, negociación de ideas, organización de tareas y gestión conjunta del tiempo [24]. Estas habilidades resultan esenciales en entornos profesionales de desarrollo de software, donde la construcción de sistemas complejos requiere coordinación entre múltiples roles y equipos [23]. La literatura señala que el trabajo colaborativo incrementa significativamente la calidad del código y la motivación de los programadores [7].

4.4.4. Herramientas de apoyo

Las herramientas de apoyo facilitan el aprendizaje de la programación mediante asistencia sintáctica, ejecución controlada, visualización de estructuras y automatización de pruebas [7]. Entre las más utilizadas se encuentran los entornos de desarrollo integrado (IDE), que ofrecen resaltado de sintaxis, autocompletado, depuración paso a paso y monitoreo del estado de variables [23]. Estas funcionalidades permiten detectar errores más rápidamente y comprender el flujo de ejecución del programa [24].

Además, herramientas especializadas como simuladores de memoria, visualizadores de algoritmos y plataformas de aprendizaje interactivo ayudan a los programadores a desarrollar una comprensión más profunda de conceptos internos como pilas, colas, memoria dinámica o recursión [9]. Estas herramientas permiten observar visualmente lo que ocurre durante la ejecución, facilitando la relación entre teoría y práctica [7]. La literatura destaca que las visualizaciones incrementan la retención y la comprensión conceptual [24].

Los sistemas de control de versiones, como Git, también constituyen herramientas esenciales que permiten gestionar cambios, colaborar en proyectos, explorar alternativas y revertir errores [23]. Su uso temprano en la formación ayuda a fomentar disciplina técnica, trazabilidad y orden en el desarrollo [24]. De este modo, las herramientas de apoyo complementan el aprendizaje teórico y fortalecen las competencias prácticas en programación [9].

4.4.5. Lenguajes de programación en Sistemas de Información

En la formación en Ingeniería en Sistemas de Información (IS), los lenguajes de programación cumplen una doble función. Por un lado, constituyen herramientas pedagógicas para desarrollar pensamiento algorítmico y disciplina lógica. Por otro, representan instrumentos tecnológicos concretos para la construcción de Sistemas de Información (SI) organizacionales complejos, que actualmente integran bases de datos, servicios distribuidos, inteligencia artificial (IA), analítica avanzada e incluso dispositivos IoT.

Esta doble orientación obliga a distinguir entre lenguajes utilizados con fines formativos iniciales y lenguajes empleados en entornos empresariales reales. Ambos comparten fundamentos formales, pero difieren en complejidad, ecosistema tecnológico y propósito profesional [23], [24], [40].

En etapas iniciales del aprendizaje, el lenguaje es un medio para comprender estructuras de control, tipos de datos y modelado lógico. En contextos empresariales, el lenguaje se convierte en componente de arquitecturas mayores, integrándose con bases de datos, servicios web, contenedores, plataformas en la nube y modelos de IA. Comprender esta transición forma parte del desarrollo profesional dentro de IS.

Semántica

La semántica de un lenguaje de programación define el significado de sus construcciones. No basta con conocer la sintaxis; es necesario comprender qué ocurre realmente cuando una instrucción se ejecuta [24], [35].

En el aprendizaje inicial, la semántica permite entender, por ejemplo, la diferencia entre asignación y comparación, entre paso por valor y paso por referencia, o entre evaluación secuencial y condicional. Estos fundamentos desarrollan precisión conceptual y evitan errores comunes.

Por ejemplo:

```
1      x <- 5
2      y <- x
3      x <- 10
```

Listing 4.25: Ejemplo semántico básico.

Comprender la semántica implica saber que el valor de `y` permanece 5, porque la asignación copia el valor en ese momento. Este tipo de análisis es esencial para formar pensamiento computacional riguroso [17].

En lenguajes utilizados para desarrollar SI empresariales, la semántica adquiere mayor complejidad. Por ejemplo, operaciones asíncronas, concurrencia, transacciones o serialización de objetos requieren comprender cómo el lenguaje modela ejecución paralela, memoria compartida y comunicación entre procesos [10], [34].

Cuando un SI integra IA o IoT, la semántica también abarca la interpretación de flujos de datos en tiempo real, modelos entrenados y eventos distribuidos. Por ello, una comprensión profunda de la semántica no es opcional, sino requisito para garantizar coherencia y seguridad en sistemas organizacionales.

Paradigmas

Un paradigma de programación es un modelo conceptual que guía la forma en que se estructuran soluciones. En IS, comprender paradigmas permite seleccionar el enfoque más adecuado para cada tipo de problema [35], [40].

En el aprendizaje inicial, el paradigma procedimental es dominante. El estudiante organiza el algoritmo como una secuencia de instrucciones estructuradas en funciones y procedimientos. Este enfoque facilita comprender flujo de control y modularidad básica.

Posteriormente, la orientación a objetos introduce encapsulamiento, abstracción y reutilización. En SI organizacionales, este paradigma permite modelar entidades reales (cliente, factura, empleado) como objetos con atributos y comportamientos [24].

En entornos empresariales modernos, se incorporan además:

- Paradigma orientado a eventos (sistemas reactivos).
- Programación funcional (transformación de datos).
- Orientación a servicios (microservicios).
- Programación orientada a datos y aprendizaje automático.

Por ejemplo, en un SI que integra IoT, un evento como “sensor detecta temperatura alta” activa procesos automáticos. En sistemas con IA, los algoritmos incluyen modelos predictivos integrados en el flujo de decisión.

Para el estudiante de IS, comprender paradigmas no significa dominar todos simultáneamente, sino reconocer que cada uno responde a distintos niveles de complejidad y necesidades organizacionales.

Compilación e interpretación

Los lenguajes pueden clasificarse, de manera general, según su modelo de ejecución: compilados, interpretados o híbridos [34], [35].

Un lenguaje compilado traduce el código fuente a lenguaje máquina antes de ejecutarse. Esto suele producir mayor eficiencia en tiempo de ejecución. Lenguajes como C o C++ son ejemplos clásicos.

Un lenguaje interpretado ejecuta el código mediante un intérprete que traduce instrucciones en tiempo real. Esto facilita depuración y aprendizaje, aunque puede implicar menor rendimiento. Python es un ejemplo representativo.

Existen también modelos híbridos, como Java, que compilan a bytecode y ejecutan mediante una máquina virtual. Este modelo combina portabilidad y eficiencia.

En la formación inicial en IS, los lenguajes interpretados suelen facilitar el aprendizaje por su simplicidad sintáctica y retroalimentación inmediata. Sin embargo, en el desarrollo de SI empresariales, la elección del lenguaje depende de factores como:

- Escalabilidad.
- Rendimiento.
- Integración con bases de datos.
- Ecosistema de bibliotecas.
- Compatibilidad con IA y análisis de datos.
- Soporte para arquitecturas distribuidas.

Por ejemplo:

- Python es ampliamente utilizado en analítica e IA.
- Java y C# son comunes en SI empresariales.
- JavaScript es central en aplicaciones web.
- C/C++ y Rust pueden emplearse en dispositivos IoT.

Para el estudiante que aprende a programar, lo fundamental no es memorizar lenguajes, sino comprender que todos comparten fundamentos algorítmicos comunes. La sintaxis cambia; la lógica estructurada permanece.

i

Sintetizando...

En síntesis, los lenguajes de programación en IS cumplen una función dual: primero como instrumentos formativos para desarrollar pensamiento lógico estructurado y, posteriormente, como herramientas tecnológicas para construir SI organizacionales complejos que integran bases de datos, servicios distribuidos, inteligencia artificial e IoT. Comprender esta evolución permite al estudiante situar su aprendizaje inicial dentro de una trayectoria profesional coherente y escalable.

4.5. Representación en lenguaje natural estructurado

La representación en lenguaje natural estructurado constituye un punto intermedio entre la descripción informal de un problema y su especificación algorítmica formal. A diferencia del lenguaje natural convencional, este enfoque restringe las expresiones utilizadas, organiza la información en pasos claramente delimitados y prioriza la ausencia de ambigüedades. Su objetivo es facilitar el razonamiento secuencial, manteniendo la expresividad adecuada para que el algoritmo pueda ser comprendido por personas sin conocimientos avanzados de programación, lo cual es coherente con el propósito formativo de esta etapa del libro.

En concordancia con la estructura Entrada-Proceso-Salida (EPS) previamente desarrollada, el lenguaje natural estructurado permite describir la transformación de datos de forma ordenada y verificable antes de traducirla a pseudocódigo. Esta aproximación ha sido utilizada históricamente en metodologías de diseño algorítmico, pues permite identificar errores conceptuales, inconsistencias y omisiones antes de avanzar hacia representaciones más formales. Tal como ocurre en el análisis de algoritmos descrito por Cormen et al. [19], la claridad en las descripciones previas constituye un requisito para cualquier especificación computacional rigurosa.

En el ejemplo que se muestra en el listado de código 4.26, que funciona como referencia central para las subsecciones posteriores, presenta un algoritmo expresado íntegramente en lenguaje natural estructurado. Su extensión permite ilustrar decisiones secuenciales, condiciones y validaciones básicas, las mismas que se examinarán en cada apartado.

Listing 4.26: Algoritmo en lenguaje natural estructurado para calcular el índice de masa corporal (IMC)

Algoritmo CalcularIMC

Var

peso, estatura, imc: real

Entrada:

- peso (kilogramos)
- estatura (metros)

Proceso:

1. Verificar que peso sea mayor que 0.
2. Verificar que estatura sea mayor que 0.
3. Calcular imc como peso entre estatura al cuadrado: $imc \leftarrow peso / (estatura^2)$
4. Si $imc < 18.5$, entonces categoría $\leftarrow$ "Bajo peso".
5. Si imc está entre 18.5 y 24.9, entonces categoría $\leftarrow$ "Normal".
6. Si imc está entre 25 y 29.9, entonces categoría $\leftarrow$ "Sobrepeso".
7. Si $imc \geq 30$, entonces categoría $\leftarrow$ "Obesidad".

Salida:

- Mostrar imc.
- Mostrar categoría.

FinAlgoritmo

4.5.1. Pasos numerados

El empleo de pasos numerados, como se muestra en el algoritmo del listado 4.26, permite organizar el razonamiento del programador en formación mediante una secuencia explícita y ordenada de acciones. La numeración evita interpretaciones múltiples acerca del orden de ejecución y facilita la correlación posterior con las estructuras secuenciales del pseudocódigo. Además, constituye una práctica alineada con la descripción operacional de algoritmos presentada en secciones anteriores, donde cada acción representa una transición clara del estado del sistema.

Otra ventaja de la numeración consiste en su capacidad para descomponer problemas complejos en unidades más simples y verificables. En el ejemplo del IMC, cada validación y cada cálculo aparecen como pasos independientes, permitiendo identificar fácilmente qué ocurre si un dato de entrada es incorrecto o si una condición no se cumple. La literatura sobre diseño algorítmico destaca esta modularidad como un elemento clave para el análisis y depuración temprana de procedimientos [23], [53].

Finalmente, la numeración establece una relación directa con las estructuras de control estudiadas previamente, en particular la secuencia. Cada número indica un avance lineal en el flujo de ejecución, lo que permite a los programadores establecer paralelos entre el lenguaje natural estructurado y los bloques de código que implementarán más adelante. Esta transición gradual reduce la dificultad cognitiva y fortalece la comprensión del proceso algorítmico.

4.5.2. Precisión y claridad

La claridad en la redacción es un componente esencial del lenguaje natural estructurado. Tal como se observa en el listado 4.26, cada paso debe contener únicamente la información necesaria, evitando descripciones redundantes o informales. Una instrucción precisa permite que distintos lectores interpreten la misma acción de manera idéntica, reduciendo la ambigüedad y aumentando la reproducibilidad del algoritmo, principio central en la ingeniería del software según Pressman & Maxim [23].

La precisión también implica nombrar explícitamente las variables, operaciones y comparaciones involucradas. Por ejemplo, en el cálculo del IMC se indican claramente las restricciones de entrada y los rangos asociados a cada categoría. Estas expresiones eliminan posibles confusiones respecto a cómo se realiza la operación o qué intervalos determinan la clasificación. Como se enfatiza en Sommerville [53], esta especificación rigurosa es indispensable para garantizar la coherencia entre el diseño conceptual y la futura implementación.

Otro aspecto relevante es el uso consistente de conectores y palabras clave. Expresiones como “sí”, “entonces”, “verificar” o “calcular” deben emplearse siempre con el mismo significado y en contextos controlados. La estandarización terminológica permite que los programadores desarrollen un vocabulario compartido, facilitando el trabajo colaborativo y la revisión entre pares. Además, esta práctica es coherente con los estándares utilizados en notaciones semiformales de requisitos.

Finalmente, la claridad del algoritmo también depende de la estructura visual. Separar entradas, procesos y salidas; emplear listas; y organizar las condiciones de manera explícita, como se aprecia en el ejemplo del IMC, mejora la legibilidad y hace posible identificar errores lógicos antes de convertir el algoritmo en pseudocódigo o en un lenguaje de programación.

4.5.3. Evitar ambigüedades

Uno de los propósitos fundamentales del lenguaje natural estructurado es eliminar la ambigüedad inherente al lenguaje cotidiano. Expresiones informales como “aproximadamente”, “más o menos”, “grande”, “pequeño” o “rápido” deben evitarse por completo, pues no tienen un significado operativo dentro del diseño algorítmico. El ejemplo del listado 4.26 ilustra esta precisión al utilizar comparaciones numéricas claramente definidas para cada rango de clasificación.

Asimismo, las decisiones condicionales deben diferenciarse explícitamente. Frases como “si es necesario” o “en caso contrario” solo deben utilizarse cuando el contexto está plenamente determinado. La falta de precisión puede conducir a implementaciones inconsistentes o incompatibles con el objetivo del algoritmo. La claridad en los límites del problema es un requisito destacado por Cormen et al. [53], quienes enfatizan la necesidad de definir comportamientos específicos en cada situación posible.

4.6. Pseudocódigo

El pseudocódigo constituye una representación formalizada del algoritmo que, sin pertenecer a un lenguaje de programación real, adopta convenciones precisas que permiten describir con exactitud la lógica del procedimiento. A diferencia del lenguaje natural estructurado, el pseudocódigo utiliza palabras reservadas, bloques anidados y notaciones estandarizadas que favorecen la posterior traducción a un lenguaje de programación. Su función es, por tanto, actuar como puente entre el diseño conceptual y la implementación concreta.

La figura 4.4 muestra la estructura general de un algoritmo en pseudocódigo, destacando los bloques fundamentales: encabezado, entrada, proceso y salida. Esta estructura es coherente con el modelo EPS desarrollado previamente y se vincula directamente con la construcción de programas reales.

4.6.1. Elementos del lenguaje algorítmico (UPSAM 2.0)

El lenguaje algorítmico UPSAM 2.0 [61] define un conjunto de elementos fundamentales que permiten describir algoritmos de manera precisa, estructurada y comprensible, independientemente del lenguaje de programación en el que posteriormente se implementen. Estos elementos constituyen la base conceptual para el aprendizaje de la programación, ya que permiten modelar soluciones de forma lógica antes de enfrentarse a detalles sintácticos propios de lenguajes formales.

Desde un enfoque didáctico, la especificación de estos elementos facilita el desarrollo del pensamiento algorítmico, al promover la correcta identificación de datos, operaciones y estructuras de control [19]. A diferencia de una descripción informal en lenguaje natural, UPSAM 2.0 proporciona reglas claras que evitan ambigüedades y favorecen la transición progresiva hacia lenguajes de programación reales.

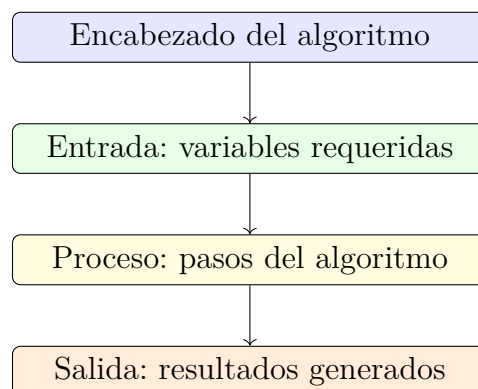


Figura 4.4: Estructura básica de un algoritmo expresado en pseudocódigo.

Identificadores

Los identificadores son nombres simbólicos que representan entidades del algoritmo como variables, constantes, funciones o procedimientos [35]. En UPSAM 2.0, un identificador debe comenzar con una letra, puede contener dígitos y el carácter de subrayado, y no debe coincidir con palabras reservadas del lenguaje. Esta restricción garantiza claridad y evita conflictos semánticos durante la interpretación del algoritmo.

El uso de identificadores significativos es una recomendación central en la programación estructurada, ya que mejora la legibilidad del algoritmo y facilita su mantenimiento [33]. Por ejemplo, un identificador como `total_pagar` transmite de forma inmediata su propósito, a diferencia de nombres genéricos como `x` o `aux`. Como ejemplo de uso, un algoritmo que calcula el promedio de notas puede emplear `nota1`, `nota2`, `nota3` y `promedio_final`, haciendo explícito el significado de cada dato manejado.

Comentarios

Los comentarios permiten documentar el algoritmo sin afectar su ejecución lógica [33], [35]. UPSAM 2.0 admite comentarios de una sola línea mediante `//` y comentarios multilínea delimitados por `{` y `}`. Estos comentarios son ignorados por el intérprete del algoritmo, pero resultan fundamentales para explicar decisiones de diseño, aclarar cálculos o describir el propósito de un bloque de instrucciones. Por ejemplo, puede indicarse mediante comentarios que una fórmula corresponde a un cálculo financiero o que una condición valida una restricción del problema planteado.

Tipos de datos básicos

Antes de presentar los tipos de datos, conviene recordar que la elección del tipo determina qué operaciones son válidas y cómo deben interpretarse los valores. En programación básica, esto permite evitar errores conceptuales (por ejemplo, intentar realizar operaciones aritméticas sobre cadenas) y mejora la claridad de los algoritmos al documentar la naturaleza de cada variable [7], [59]. En la práctica, los tipos de datos se evidencian en tareas cotidianas: contar elementos requiere enteros, promedios suelen requerir reales y las decisiones se modelan con valores lógicos.

Operadores del lenguaje algorítmico

Los operadores permiten construir expresiones y constituyen el núcleo del procesamiento algorítmico [35], [53], [59]. En UPSAM 2.0, los operadores se agrupan de forma clásica en aritméticos, relacionales y lógicos, lo que permite expresar cálculos, comparaciones y condiciones de control [61]. En programación básica, esta clasificación es esencial porque facilita identificar qué se está calculando (aritmética), qué se está comparando (relacional) y cómo se combinan decisiones (lógica) [53], [59], [62].

Cuadro 4.1: Tipos de datos en UPSAM 2.0 (ampliado con ejemplos)

Tipo	Definición mejorada	Ejemplo de uso
Entero	Número sin decimales (conteos, índices, control de ciclos)	<code>contador ← 0</code>
Real	Número con decimales (mediciones, promedios, finanzas)	<code>promedio ← 8.75</code>
Lógico	Valor de verdad (condiciones y decisiones)	<code>aprobado ← verdadero</code>
Carácter	Un símbolo (códigos, sexo, opciones)	<code>op ← 'A'</code>
Cadena	Texto (nombres, mensajes, descripciones)	<code>nombre ← "Ana"</code>

Operadores aritméticos. Antes de listar los operadores aritméticos, es útil notar que estos aparecen de manera recurrente en problemas matemáticos y financieros: sumatorias, promedios, porcentajes y tasas [7], [53], [59], [63]. Por ejemplo, un descuento comercial puede modelarse con multiplicación, y la validación de paridad se apoya en el operador residuo.

Cuadro 4.2: Operadores aritméticos (UPSAM 2.0) con ejemplos

Operador	Definición	Ejemplo de uso
-	Menos unitario (cambio de signo)	<code>a ← - b</code>
+	Suma dos valores	<code>suma ← a + b</code>
-	Resta dos valores	<code>dif ← a - b</code>
*	Multiplica valores	<code>area ← base * altura</code>
/	División real	<code>prom ← total / n</code>
div	División entera	<code>coc ← 7 div 2</code>
mod	Residuo de división	<code>res ← 7 mod 2</code>
**	Exponenciación	<code>potencia ← base ** exponente</code>

Operadores relacionales. Los operadores relacionales producen resultados lógicos y se usan principalmente en decisiones (`si... entonces`) y en el control de ciclos [53], [59], [63]. Un ejemplo típico es verificar rangos válidos, como `0 <= nota <= 10`, o validar restricciones de negocio, como `monto > 100` para aplicar un descuento.

Operadores lógicos. Los operadores lógicos permiten combinar condiciones simples para formar reglas de decisión compuestas. Esto es habitual al validar entradas (por ejemplo, que una nota esté en un rango) o al evaluar reglas institucionales (por ejemplo, matrícula permitida si no hay deudas y si cumple requisitos) [7], [35], [59].

Estructura general de un programa en UPSAM 2.0

La estructura de un programa en UPSAM 2.0 separa, de forma explícita, las declaraciones del cuerpo de instrucciones. Esta separación mejora la claridad, porque permite identificar primero los datos (constantes y variables) y luego el flujo de acciones (entrada, procesamiento y salida). En programación básica, esto ayuda a que el estudiante distinga entre el “qué datos existen” y el “qué hace el algoritmo” [61].

Cuadro 4.3: Operadores relacionales (UPSAM 2.0) con ejemplos

Operador	Definición	Ejemplo de uso
=	Igualdad	a = b
<>	Diferente	x <> 0
<	Menor que	edad < 18
<=	Menor o igual	nota <= 10
>	Mayor que	ventas > 1000
>=	Mayor o igual	prom >= 7

Cuadro 4.4: Operadores lógicos (UPSAM 2.0) con ejemplos

Operador	Definición	Ejemplo de uso
no	Negación lógica	no aprobado
y	Conjunción (AND)	(a>0) y (b>0)
o	Disyunción (OR)	(x=0) o (y=0)

Forma general. A continuación se presenta una forma general y didáctica de organizar un algoritmo, donde se define el nombre, se declaran constantes y variables, y finalmente se especifican instrucciones entre **inicio** y **fin** [61]. Por ejemplo, un algoritmo de cálculo de interés simple declarará variables como **capital**, **tasa** y **tiempo**, y luego realizará el cálculo dentro del bloque principal.

```

algoritmo NombreAlgoritmo
const
// Declaración de constantes
var
// Declaración de variables
inicio
// Instrucciones del algoritmo
fin

```

Declaración de constantes. Las constantes representan valores que no cambian durante la ejecución del algoritmo. Su uso resulta esencial para mejorar la claridad del algoritmo y evitar la repetición de valores “mágicos” en múltiples expresiones. Además, cuando un valor fijo debe modificarse (por ejemplo, una tasa institucional o un umbral de aprobación), basta con actualizarlo en un solo lugar, lo cual reduce errores y mejora la mantenibilidad [33], [51]. Como ejemplo de uso, $PI = 3.1416$ se utiliza como constante para cálculos geométricos, por ejemplo al calcular áreas o perímetros de circunferencias.

Declaración de variables. Las variables almacenan datos cuyo valor puede modificarse durante la ejecución del algoritmo [64]. En UPSAM 2.0, las variables se emplean para conservar entradas del usuario, resultados intermedios (sumas, acumuladores, contadores) y resultados finales. Por ejemplo, en un algoritmo de promedio, las calificaciones se almacenan en variables y la suma total se mantiene en un acumulador. La declaración explícita (cuando se usa le añade calidad a la solución) y el uso consistente de variables favorece la legibilidad y evita confusiones entre datos de distinta naturaleza.

Antes de mostrar ejemplos, conviene recalcar que los tipos de variables deben elegirse según el tipo de información: **entero** para conteos, **real** para mediciones y promedios, y **cadena** para texto. Como ejemplo contextualizado, un algoritmo

de nómina puede declarar `real : salario` y `real : descuento`, mientras que un algoritmo de inventarios podría declarar `entero : unidades`.

Cuadro 4.5: Ejemplos de declaración de variables en UPSAM 2.0

Tipo	Ejemplo
entero	<code>contador: entero</code>
real	<code>salario: real</code>
cadena	<code>mensaje ← "Hola": cadena</code>

Funciones predefinidas. UPSAM 2.0 incorpora una biblioteca de funciones que facilitan cálculos y manipulaciones comunes, reduciendo la necesidad de expresar procedimientos rutinarios paso a paso. En programación básica, estas funciones permiten concentrarse en la lógica general de la solución, manteniendo algoritmos más cortos y legibles. Por ejemplo, para calcular distancias o desviaciones se requiere raíz cuadrada, y para validaciones numéricas resulta útil el valor absoluto o la truncación de decimales [53].

Funciones aritméticas. Antes de presentar la tabla, nótese que estas funciones son especialmente relevantes en problemas matemáticos, estadísticos y financieros [38], [61]. Por ejemplo, `abs(x)` puede emplearse para medir error absoluto, `raiz2(x)` aparece en varianza y desviación estándar, y `trunc(x)` resulta útil al trabajar con valores monetarios o partes enteras de cantidades reales.

Cuadro 4.6: Funciones aritméticas predefinidas en UPSAM 2.0

Función	Descripción	Ejemplo
<code>abs(x)</code>	Devuelve el valor absoluto de un número	<code>abs(-5)</code>
<code>raiz2(x)</code>	Calcula la raíz cuadrada de un número	<code>raiz2(16)</code>
<code>exp(x)</code>	Calcula la función exponencial e^x	<code>exp(1)</code>
<code>ln(x)</code>	Calcula el logaritmo natural de un número	<code>ln(10)</code>
<code>log10(x)</code>	Calcula el logaritmo en base 10	<code>log10(100)</code>
<code>trunc(x)</code>	Elimina la parte decimal de un número	<code>trunc(8.9)</code>
<code>redondear(x)</code>	Redondea un número al entero más cercano	<code>redondear(3.6)</code>
<code>sen(x)</code>	Calcula el seno de un ángulo (en radianes)	<code>sen(1.57)</code>
<code>cos(x)</code>	Calcula el coseno de un ángulo (en radianes)	<code>cos(0)</code>
<code>tan(x)</code>	Calcula la tangente de un ángulo (en radianes)	<code>tan(0.78)</code>
<code>arcsen(x)</code>	Calcula el arco seno	<code>arcsen(0.5)</code>
<code>arccos(x)</code>	Calcula el arco coseno	<code>arccos(0.5)</code>
<code>arctan(x)</code>	Calcula el arco tangente	<code>arctan(1)</code>

Funciones de cadena. Las funciones de cadena permiten manipular texto, algo muy frecuente en sistemas de información, por ejemplo al procesar nombres, códigos, mensajes, identificadores o entradas del usuario. En programación

básica, estas funciones ayudan a introducir al estudiante en el tratamiento de datos no numéricos y en operaciones típicas como contar caracteres o extraer partes de un texto [59], [61].

Entrada y salida. Las operaciones de entrada y salida definen la interacción entre el algoritmo y su entorno. La entrada permite obtener datos desde el usuario o desde una fuente externa, mientras que la salida comunica resultados o mensajes. En programación básica, estas operaciones son esenciales porque conectan la lógica interna del algoritmo con el propósito práctico del programa: resolver un problema con datos reales. Como ejemplo, un algoritmo de cálculo de promedio primero lee las calificaciones y luego escribe el promedio final y el estado de aprobación [7].

4.6.2. Convenciones de escritura

El pseudocódigo utiliza un conjunto de convenciones que permiten representar algoritmos de forma independiente del lenguaje. Entre estas se encuentran las palabras reservadas (p. ej., **Algoritmo**, **FinAlgoritmo**, **Si**, **Entonces**, **Mientras**), la indentación para marcar bloques y el uso explícito de operaciones básicas. El objetivo es garantizar que cualquier lector pueda comprender el algoritmo sin conocer un lenguaje específico.

Por ejemplo, la instrucción que se muestra en el listado de código 4.27, representa de manera clara la estructura condicional, independientemente del lenguaje de implementación.

Listing 4.27: Estructura de Selección/Condicional como Parte de un Algoritmo

```
Si x > 0 Entonces
    escribir "Número positivo"
FinSi
```

Palabras reservadas

Las palabras reservadas constituyen el vocabulario fundamental del pseudocódigo, ya que cada una representa una operación o estructura de control específica. Su uso permite establecer una correspondencia directa con los lenguajes reales. Entre las más utilizadas se encuentran:

- Algoritmo / FinAlgoritmo — delimitan el inicio y fin del procedimiento.
- Si / Entonces / SiNo / FinSi — expresan decisiones.
- Mientras (hacer) / FinMientras — definen ciclos condicionales.
- Para / FinPara — definen ciclos contados.
- Escribir, Leer — representan operaciones de entrada/salida.
- <- — operador de asignación.

Estas palabras se emplean cuando el algoritmo requiere expresar control del flujo, iteraciones o decisiones lógicas, y su propósito es eliminar variaciones posibles del lenguaje natural. Al mantener este conjunto reducido, el pseudocódigo preserva claridad y evita interpretaciones ambiguas, alineándose con las recomendaciones formales de representación descritas en la sección 4.6.1.

Indentación

La indentación es una convención visual esencial para identificar bloques lógicos, anidamientos y alcances dentro del algoritmo. Su propósito es facilitar la lectura y permitir que el lector distinga rápidamente qué instrucciones pertenecen a qué estructura de control. Se utiliza siempre que se inicia una estructura como **Si**, **Mientras**, **Para**, entre otras.

Desde la perspectiva pedagógica, la indentación ayuda al programador en formación a internalizar los niveles jerárquicos dentro de un algoritmo, lo cual será fundamental cuando trabaje con lenguajes reales donde la organización del código influye en la mantenibilidad y comprensión general del programa.

4.6.3. Construcción de sentencias

Las sentencias en pseudocódigo permiten expresar operaciones básicas tales como asignación, entrada/salida, decisiones y ciclos. Su finalidad es representar el comportamiento del algoritmo de manera explícita, manteniendo correspondencia con los lenguajes reales de programación. A continuación se presentan ejemplos de cada tipo, vinculados lógicamente con el algoritmo del listado de código 4.26.

Asignación

La asignación permite establecer o actualizar el valor de una variable. En pseudocódigo se escribe mediante el operador `<-`. Existen asignaciones simples, matemáticas como la asignación del paso 3, y dependientes de condiciones previas como las que se muestran en los pasos 4, 5, 6 y 7 del listado de código 4.26. Según el mencionado algoritmo, en el segmento de código 4.28 se muestran las dos asignaciones, sin y con restricción:

Listing 4.28: Algoritmo en lenguaje natural estructurado para calcular el índice de masa corporal (IMC)

```
3. imc <- peso / (estatura^2)
.....
5. categoria <- "Normal"
```

En algoritmos más complejos, la asignación puede expresar actualizaciones dentro de ciclos o acumulaciones progresivas. A continuación se presentan dos ejercicios matemáticos.

Ejercicio 1: El objetivo de este ejercicio es calcular la suma de los primeros diez números enteros positivos utilizando un acumulador. Esta actividad permite comprender cómo una variable puede modificarse progresivamente dentro de un ciclo y muestra el rol fundamental de la asignación en procesos iterativos. El programador en formación debe identificar cómo el valor inicial del acumulador evoluciona en función del índice del ciclo, reforzando la noción de actualización de estado.

La solución implementa un ciclo contado que recorre los valores del 1 al 10 y acumula cada uno de ellos en la variable **total**. El algoritmo 4.29 ejemplifica adecuadamente el uso combinado de asignación inicial, actualización progresiva y salida final, permitiendo visualizar cómo la estructura de control dirige la transformación paso a paso del valor acumulado.

```
1      Algoritmo SumaAcumulada
2      1.      Var:
3      1.1.     total: real
4      1.2.     i: entero
5
6      2.      total <- 0
7      3.      Para i desde 1 hasta 10 hacer
8      3.1.     total <- total + i
9      4.      FinPara
10
11     5.      Escribir total
12     FinAlgoritmo
```

Listing 4.29: Cálculo de una suma acumulada

En el diseño de algoritmos, la elección de los nombres de las variables desempeña un papel fundamental en la comprensión tanto del problema como de la solución planteada. Si bien es habitual el uso de identificadores breves como *i*, *j* o *k* para

representar contadores o índices de control —especialmente en estructuras repetitivas o recorridos secuenciales—, esta práctica debe limitarse a contextos en los que la función de la variable resulte evidente. Cuando las variables almacenan información relevante del dominio del problema, es recomendable emplear *nombres descriptivos* que reflejen de manera directa su propósito, como `suma`, `promedio`, `totalVentas` o `nombreEstudiante`. Este criterio contribuye a que el algoritmo sea más legible y autoexplicativo, reduce la necesidad de comentarios adicionales y minimiza errores durante su interpretación o implementación. Asimismo, el uso adecuado de identificadores favorece la revisión, el mantenimiento y la traducción del algoritmo a lenguajes de programación formales, promoviendo una codificación más clara, consistente y alineada con el significado real de los datos manipulados.

Ejercicio 2: En este ejercicio se requiere evaluar una expresión polinómica para un valor específico de x . El propósito es mostrar cómo las asignaciones pueden utilizar operaciones aritméticas más complejas, incorporando exponentes, productos y sumas. Este tipo de cálculo es común en problemas físicos, económicos y matemáticos donde se necesita evaluar funciones de forma programática.

El algoritmo 4.30 presenta una asignación inicial para la variable x y posteriormente calcula el valor del polinomio mediante una expresión matemática directa. La solución refuerza el uso de la asignación como mecanismo para almacenar tanto valores iniciales como resultados intermedios y finales.

```
1  Algoritmo OperacionesMatematicas
2  1. Var:
3  1.1. x, y: real
4
5  2. x <- 3
6  3. y <- 2*x^2 + 5*x - 8
7  4. Escribir y
8  FinAlgoritmo
```

Listing 4.30: Asignación con potencias y productos

Entrada/Salida

Las sentencias de entrada y salida permiten la comunicación entre el usuario y el algoritmo. Se expresan mediante **Leer** y **Escribir**. Su diseño en pseudocódigo debe ser explícito, indicando qué datos se solicitan y qué valores se presentan.

El propósito de este ejercicio es solicitar al usuario un número entero y posteriormente mostrarlo por pantalla (ver listado de código 4.31). Esta actividad constituye el caso más elemental del flujo de entrada y salida, y permite comprender cómo un algoritmo interactúa con el usuario final. También puede introducir la necesidad de validar que el dato ingresado sea adecuado para su posterior procesamiento.

```
1  Leer peso
2  Escribir "El IMC es:", imc
```

Listing 4.31: Instrucciones de: lectura t escritura

Condicionales

Las estructuras condicionales permiten que un algoritmo seleccione un camino de ejecución dependiendo del resultado de una evaluación lógica. Su función es controlar el flujo del programa ante situaciones donde el comportamiento no puede ser lineal, lo cual es esencial para modelar decisiones. La literatura coincide en que la selección es uno de los tres pilares fundamentales de la computación algorítmica [53], [65].

El ejercicio presentado en el listado 4.26 evidencia cómo un conjunto de decisiones encadenadas permite clasificar un valor numérico dentro de rangos bien definidos. Este tipo de estructura se fundamenta en dividir el dominio de entrada en subconjuntos lógicos, asignando un comportamiento explícito a cada uno. Esta técnica es ampliamente utilizada en sistemas de decisión expertos y en validación de datos en aplicaciones reales [23], [24].

Las condicionales pueden ser simples, dobles o anidadas. Una condicional simple ejecuta una acción únicamente si se cumple la condición. En la condicional doble se introduce una ruta alternativa para cuando esta no se cumple, y en las condicionales anidadas se permite evaluar condiciones adicionales dentro de otras evaluaciones. Esta clasificación ha sido utilizada ampliamente en la enseñanza inicial de programación por su progresión cognitiva clara [4], [61].

i

Buenas prácticas

La claridad en la definición de cada condición es imprescindible para reducir ambigüedades. Las buenas prácticas recomiendan evitar expresiones vagas, rangos indefinidos o comparaciones incompletas, pues dificultan la futura implementación. Estudios en ingeniería del software señalan que gran parte de los errores en el código fuente provienen de especificaciones ambiguas o mal definidas en fases tempranas [21], [66].

i

Habilidades de Pensamiento

Las estructuras condicionales constituyen un puente conceptual que facilita al programador en formación la transición hacia lenguajes como *Python*, *Java* o *C/C++*, los cuales emplean operadores y sintaxis específicas pero basadas en los mismos principios. Este vínculo conceptual es clave en el desarrollo de habilidades de pensamiento computacional [17], [18], [67].

Ejercicio 1: Se desea diseñar un algoritmo que determine si un número ingresado por el usuario es positivo, negativo o cero. Este tipo de clasificación numérica permite introducir el uso de condicionales anidadas, las cuales son necesarias cuando un problema requiere múltiples comparaciones secuenciales dentro de un mismo proceso de decisión.

El algoritmo mostrado en el listado 4.32 aplica una estructura condicional doble con una anidación interna para evaluar los tres casos posibles. Esta solución refleja fielmente cómo las rutas alternativas dependen jerárquicamente de la condición original, reforzando la idea de bifurcación controlada del flujo.

```

1      Algoritmo ClasificarNumero
2      1. Var
3      1.1. n: entero
4      2. Leer n
5      3. Si n > 0 Entonces
6      4.   Escribir "Positivo"
7      5. SiNo
8      6.   Si n = 0 Entonces
```

```

9      7.      Escribir "Cero"
10     8.      SiNo
11     9.      Escribir "Negativo"
12    10.      FinSi
13    11. FinSi
14    FinAlgoritmo

```

Listing 4.32: Clasificación de un número

Como recomendación para quienes se inician en la programación, resulta fundamental comprender que el flujo o la secuencia de pasos de un algoritmo sigue un recorrido bien definido durante su ejecución. En particular, cuando se utiliza una estructura condicional, el algoritmo evalúa una condición lógica y, en ese instante, solo uno de los caminos posibles es ejecutado.

Si la condición resulta verdadera, se ejecutan las instrucciones asociadas a ese caso y, una vez concluidas, el flujo del algoritmo continúa con la instrucción inmediatamente posterior al cierre de la estructura condicional. En ese punto, la condición ya no se vuelve a evaluar, puesto que la decisión ha sido tomada para esa ejecución específica.

i

Consideración importante

Mientras que, durante la fase de diseño del algoritmo, es imprescindible considerar de manera explícita todas las alternativas posibles. Habitualmente, se comienza definiendo las instrucciones que se ejecutarán cuando la condición sea verdadera, concentrándose en ese escenario como si fuese el único. Una vez completado este bloque, se procede a especificar las instrucciones correspondientes al caso contrario, es decir, cuando la condición resulta falsa. Este enfoque secuencial facilita el razonamiento y evita omisiones en la lógica del algoritmo.

Esta consideración es especialmente relevante en problemas donde se requiere realizar acciones tanto en el caso verdadero como en el caso falso de una condición. Para un planteamiento correcto, debe asumirse que las instrucciones asociadas a la condición se ejecutarán siempre que esta sea verdadera, mientras que el conjunto alternativo cubrirá explícitamente el escenario contrario. Por lo tanto, en algunos casos el programador debe considerar negar la condición para que esto se cumpla.

Comprender este comportamiento —ya sea en estructuras del tipo Si–Entonces–FinSi o Si–Entonces–SiNo–FinSi— permite razonar con mayor claridad sobre el control del flujo del programa, reduce errores conceptuales frecuentes y favorece el diseño de algoritmos más claros, coherentes y predecibles.

Ejercicio 2: Se requiere determinar si un número entero es divisible por 3. Este ejercicio permite comprender la relación entre operaciones aritméticas y decisiones lógicas, mostrando cómo el operador módulo (`mod`) facilita verificar residuales en problemas de divisibilidad, ampliamente utilizados en análisis numérico y validación de datos.

El algoritmo del listado 4.33 evalúa si el residuo de dividir el número entre 3 es igual a cero. Esta verificación determina qué mensaje se imprime, lo cual ejemplifica el uso de condicionales simples y fortalece la comprensión del vínculo entre evaluación numérica y ramificación lógica.

```

1      Algoritmo VerificarDivisibilidad
2      1. Var
3      1.1. n: entero
4      2. Leer n
5      3. Si n mod 3 = 0 Entonces
6      4.      Escribir "Es divisible por 3"

```

```

7      5. SiNo
8      6.      Escribir "No es divisible por 3"
9      7. FinSi
10     FinAlgoritmo

```

Listing 4.33: Verificación de divisibilidad

En algunos casos, para encontrar la solución a ciertos problemas se pueden analizar en condiciones independientes. Hay que tener mucho cuidado especialmente, cuando se debe realizar una instrucción cuando la condición se evalúe como verdadera (y no se considere su caso falso).

Para ejemplificar esta situación, se considerará el siguiente ejercicio:

Ejercicio 3: En una institución educativa se desea desarrollar un programa que permita determinar la situación académica de un estudiante en una asignatura específica. El programa debe solicitar como datos de entrada el nombre del estudiante, el nombre de la asignatura y el promedio final obtenido por el estudiante en dicha asignatura, expresado en una escala de 0 a 10.

Con base en el promedio ingresado, el programa deberá mostrar por pantalla la condición académica correspondiente, de acuerdo con los siguientes criterios establecidos por la institución:

- **Aprueba:** si el promedio es mayor o igual a 6.0.
- **Suspense:** si el promedio es mayor o igual a 4.5 y menor que 6.0.
- **Reprueba:** si el promedio es menor que 4.5.

Una de sus soluciones es la que se muestra en el Listado de código 4.34.

```

1      Algoritmo VerificarDivisibilidad
2      1. Var
3      1.1. nombreEstudiante, nombreAsignatura: cadena
4      1.2. promedio: real
5      2. Mostrar "Ingrese el nombre del estudiante: "
6      3. Leer nombreEstudiante
7      4. Mostrar "Ingrese el nombre de la asignatura: "
8      5. Leer nombreAsignatura
9      6. Mostrar "El promedio de ", nombreEstudiante, " en ",
    ↪ nombreAsignatura
10     7. Leer promedio
11     8. Si promedio >= 6 Entonces Mostrar nombreEstudiante, " Aprueba,
    ↪ nombreAsignatura
12     9. Si promedio >= 4.5 AND promedio < 6 Entonces Mostrar
    ↪ nombreEstudiante, " debe rendir examen de suspense de ",
    ↪ nombreAsignatura
13    10. Si promedio < 4.5 Entonces Mostrar nombreEstudiante, " Reprueba ",
    ↪ nombreAsignatura

```

Listing 4.34: Verificación de divisibilidad

En soluciones como la mostrada en el Listado de código 4.34, cada condición se plantea de forma independiente, es decir, se analiza como si fuera la única posibilidad en el flujo del algoritmo. De este modo, para cada evaluación se asume temporalmente que dicha condición es la que se cumple, y se definen las instrucciones correspondientes sin considerar, en ese momento, las demás alternativas. Este enfoque facilita el diseño lógico del algoritmo y contribuye a una estructuración más clara y ordenada de las decisiones.

i

Consideración para abordar los problemas

Al abordar problemas que involucran decisiones basadas en condiciones, es recomendable adoptar una forma de razonamiento ordenada y deliberada, en la que cada escenario posible sea analizado con claridad antes de pensar en el conjunto completo. Resulta especialmente importante identificar si una condición requiere únicamente ejecutar una acción cuando se cumple, o si también es necesario contemplar explícitamente el comportamiento alternativo. Este ejercicio de análisis previo permite evitar solapamientos lógicos, omisiones en la toma de decisiones y errores en los límites de las comparaciones, fortaleciendo así la capacidad del programador para construir algoritmos correctos, legibles y coherentes desde las primeras etapas de su formación.

Ciclos

Los ciclos permiten repetir un conjunto de instrucciones mientras una condición sea verdadera o durante un número determinado de iteraciones. Esta estructura resulta esencial para resolver problemas que requieren procesamiento repetitivo o acumulativo, como sumas parciales, recorridos de listas o simulaciones computacionales [53], [61].

En su forma general, un ciclo puede controlarse mediante un contador (ciclo determinado) o mediante una condición lógica (ciclo indeterminado). Esta distinción responde a necesidades distintas: cuando se conoce el número de iteraciones, se emplea un ciclo contado; cuando se desconoce y depende del estado del sistema, se usa un ciclo condicionado [23], [24].

El uso adecuado de ciclos favorece la eficiencia algorítmica al evitar repeticiones innecesarias. De hecho, una gestión incorrecta de las condiciones de parada es responsable de errores comunes como ciclos infinitos, lo cual constituye una de las fallas típicas en la programación inicial [4], [21].

Los ciclos también permiten estructurar cálculos iterativos que dependen de valores ingresados por el usuario. Esto facilita desarrollar algoritmos interactivos que ajustan su comportamiento dinámicamente según las entradas recibidas, un aspecto clave en aplicaciones prácticas y simulaciones computacionales [17], [18], [67].

i

Comprensión de las Iteraciones

Los ciclos constituyen una herramienta expresiva indispensable para representar procesos repetitivos de la vida real, tales como sumatorias, barridos, comprobaciones sucesivas o acumulaciones. Su comprensión prepara al programador en formación para estructuras avanzadas como iteradores, generadores o recorridos de colecciones en lenguajes modernos [60], [68].

Ejercicio 1: El objetivo es imprimir los números del 1 al 10 utilizando un ciclo contado. Este tipo de estructura permite que el programador en formación aprenda cómo el algoritmo controla el avance del índice, asegurando que el número de iteraciones sea conocido desde el inicio.

El algoritmo 4.35 utiliza un ciclo **Para** que incrementa automáticamente la variable de control **i**. Cada iteración imprime su valor, lo que permite observar el comportamiento secuencial del ciclo contado.

```
1  Algoritmo ImprimirNumeros
2  1. Var
3  1.1. i: entero
4  2. Para i desde 1 hasta 10 hacer
5  2.1.     Escribir i
6  4. FinPara
7  FinAlgoritmo
```

Listing 4.35: Ciclo contado para imprimir números

Ejercicio 2: Se requiere construir un algoritmo que permita sumar números ingresados por el usuario hasta que la suma total alcance o supere el valor límite de 50. Este ejercicio enfatiza el uso de ciclos controlados por condición, donde la cantidad de iteraciones no es conocida de antemano.

El algoritmo 4.36 emplea un ciclo **Mientras**, repitiendo la lectura y el proceso de acumulación hasta cumplir la condición. Este enfoque refleja el carácter dinámico de los ciclos condicionados.

```
1  Algoritmo SumaHastaLimite
2  1. Var
3  1.1. total, x: entero
4  2. total <- 0
5  3. Mientras total < 50 hacer
6  3.1.     Escribir "Ingresa un número:"
7  3.2.     Leer x
8  3.3.     total <- total + x
9  4. FinMientras
10 5. Escribir "Suma final:", total
11 FinAlgoritmo
```

Listing 4.36: Ciclo condicional para sumar hasta superar un límite

Ejercicio 2: Se desea diseñar un algoritmo que permita calcular el promedio de n valores ingresados por el usuario. Para ello, el programa debe solicitar primero la cantidad total de datos y luego leer cada uno de los valores dentro de un ciclo contado. Este ejercicio permite comprender el uso de acumuladores y el control del flujo mediante un ciclo **Para**, reforzando la importancia de la estructura iterativa en tareas de agregación numérica.

El algoritmo presentado en el listado de código 4.37 utiliza un acumulador para sumar los valores ingresados y, posteriormente, divide la suma total entre la cantidad de datos registrados. El uso del ciclo **Para** garantiza que el algoritmo ejecute exactamente n iteraciones, lo cual facilita el control del proceso y evita condiciones indeterminadas.

```
1  Algoritmo PromedioNValoresIterativa
2  1. Escribir "Ingresa la cantidad de valores:"
```

```

3      2. Leer n
4      3. suma <- 0
5      4. Para i desde 1 hasta n hacer
6      4.1.   Escribir "Ingrese el valor ", i, ":"
7      4.2.   Leer x
8      4.3.   suma <- suma + x
9      5. FinPara
10     6. promedio <- suma / n
11     7. Escribir "El promedio es:", promedio
12     FinAlgoritmo

```

Listing 4.37: Cálculo del promedio de n valores

Ejercicio 3: Se desea diseñar un algoritmo que simule un cronómetro de cuenta regresiva. El usuario ingresa un número entero n que representa los segundos iniciales. El algoritmo debe mostrar cada segundo restante y, adicionalmente, emitir mensajes especiales cuando falten exactamente 10, 5 y 1 segundo. Este problema **solo puede resolverse adecuadamente con un ciclo decremental**, ya que los eventos dependen de la cercanía al final de la cuenta y no del progreso ascendente. Usar un ciclo ascendente generaría una lógica antinatural y obligaría a cálculos inversos innecesarios.

El algoritmo del listado 4.38 utiliza un ciclo **Para** que inicia en n y decrementa hasta 0. En cada iteración se evalúan los segundos restantes y, cuando coinciden con los puntos críticos definidos, se muestra la advertencia correspondiente. Gracias al recorrido descendente, la lógica se expresa de manera directa, clara y coherente con el funcionamiento real de un temporizador.

```

1      Algoritmo TemporizadorConEventos
2      1. Escribir "Ingrese el número de segundos iniciales:"
3      2. Leer n
4      3. Para i desde n hasta 0 hacer paso -1
5      3.1.   Escribir "Tiempo restante:", i, " segundos"
6      3.2.   Si i = 10 Entonces
7      3.2.1.   Escribir "¡Advertencia: quedan 10 segundos!"
8      3.3.   FinSi
9      3.4.   Si i = 5 Entonces
10     3.4.1.   Escribir "¡Atención: solo 5 segundos restantes!"
11     3.5.   FinSi
12     3.6.   Si i = 1 Entonces
13     3.6.1.   Escribir "¡Último segundo!"
14     3.7.   FinSi
15     4. FinPara
16     5. Escribir "Fin de la cuenta regresiva."
17     FinAlgoritmo

```

Listing 4.38: Cuenta regresiva con avisos en tiempos críticos

Antes de escribir el algoritmo, es fundamental comprender con claridad el comportamiento esperado del cronómetro. El problema no consiste únicamente en mostrar una cuenta regresiva, sino en identificar momentos específicos en los que deben emitirse mensajes especiales. Por ello, se recomienda distinguir entre las acciones que ocurren en cada iteración del ciclo (mostrar el tiempo restante) y aquellas que solo deben ejecutarse cuando el contador alcanza valores críticos como 10, 5 y 1 segundo.

Un segundo aspecto clave es analizar la entrada del algoritmo. El valor ingresado por el usuario representa los segundos iniciales y debe ser un número entero no negativo. Para un diseño más robusto, el programador en formación debería preguntarse qué ocurre si el usuario ingresa un valor negativo, cero o un número pequeño que no alcanza algunos de los tiempos críticos. Este análisis previo ayuda a prevenir errores lógicos y comportamientos inesperados durante la ejecución.

El núcleo del problema se resuelve mediante una estructura repetitiva decreciente. Un ciclo que vaya desde el valor inicial n hasta cero, disminuyendo de uno en uno, refleja de forma natural el funcionamiento de una cuenta regresiva. En este contexto, el uso de un ciclo **Para** descendente suele ser más claro y legible que otras alternativas, especialmente para quienes se inician en el diseño algorítmico.

Dentro del ciclo, es importante decidir el orden de las instrucciones. Una práctica común consiste en mostrar primero el tiempo restante y luego evaluar si se debe emitir algún mensaje especial. Los avisos se activan mediante condiciones exactas, por ejemplo, cuando el contador es igual a 10, 5 o 1. Este enfoque basado en comparaciones directas facilita la comprensión del algoritmo y evita confusiones con rangos o condiciones innecesariamente complejas.

i

Buenas prácticas

La numeración de las líneas de código no constituye un elemento esencial cuando el objetivo del análisis se centra en la comprensión conceptual de las sentencias y estructuras de control. En etapas formativas, lo relevante es identificar la lógica del flujo —condiciones, iteraciones y secuencias— más que la posición exacta de cada instrucción dentro del listado. La numeración puede facilitar la referencia puntual durante la explicación o depuración, pero no altera la semántica ni el comportamiento del algoritmo. Por ello, el énfasis debe situarse en la estructura lógica y en la coherencia del diseño antes que en aspectos formales de presentación.

Se recomienda verificar mentalmente o por escrito el funcionamiento del algoritmo con distintos valores de entrada antes de implementarlo. En el último Algoritmo, probar casos como $n = 12$, $n = 7$ o $n = 1$ permite confirmar que los mensajes especiales aparecen únicamente cuando corresponde y que la cuenta regresiva finaliza correctamente. Este hábito de validación temprana es esencial para desarrollar algoritmos correctos y confiables desde las primeras etapas de la formación en programación.

4.7. Diagramas de flujo

Los diagramas de flujo son representaciones gráficas del comportamiento lógico de un algoritmo mediante símbolos estandarizados y conectores que permiten visualizar el flujo de control. Su finalidad es clarificar la estructura del procedimiento, facilitar la comprensión del proceso y establecer una transición natural entre el análisis del problema y la implementación formal. Diversas fuentes coinciden en que los diagramas de flujo constituyen una herramienta pedagógica fundamental para reforzar el pensamiento computacional y el razonamiento algorítmico [13], [61].

La utilidad de los diagramas de flujo reside en su capacidad para eliminar ambigüedades al representar operaciones, decisiones y secuencias de forma visual. Esta característica facilita la revisión entre pares, el análisis de errores y la validación de la lógica antes de escribir pseudocódigo o código real en un lenguaje de programación. Además, permiten identificar estructuras como ciclos y condicionales de manera inmediata, lo cual favorece la comprensión del flujo de ejecución.

i

Buenas prácticas

Los diagramas de flujo sirven como documentación técnica, facilitando la comunicación entre equipos de desarrollo o entre docentes y estudiantes. Su estandarización ha permitido que se mantengan como un recurso esencial en el aprendizaje inicial de algoritmos, complementando otras representaciones formales como el pseudocódigo [23], [24].

4.7.1. Símbolos básicos

Los símbolos básicos constituyen el vocabulario visual fundamental de los diagramas de flujo, permitiendo representar las acciones, decisiones y operaciones que conforman un algoritmo. Cada símbolo posee una semántica particular y una forma estandarizada, lo cual contribuye a que la interpretación del diagrama sea clara e inequívoca, independientemente del lector o del contexto. La estandarización de estos símbolos facilita la enseñanza inicial de la programación y reduce la ambigüedad durante el análisis de sistemas [13], [61].

El uso correcto de los símbolos garantiza que el flujo del algoritmo sea comprensible tanto para estudiantes como para profesionales, actuando como una herramienta intermedia entre el lenguaje natural estructurado y el pseudocódigo. Asimismo, los diagramas de flujo permiten identificar de forma visual estructuras como secuencias, decisiones y ciclos, lo que resulta útil al validar la lógica del procedimiento antes de su codificación [23], [24].

Además, los símbolos básicos facilitan la construcción modular de algoritmos más complejos, debido a que cada componente puede combinarse con otros para representar procesos iterativos o ramificados. Esta modularidad gráfica favorece la depuración conceptual en etapas tempranas del diseño, previniendo errores de interpretación o inconsistencias lógicas [4].

i

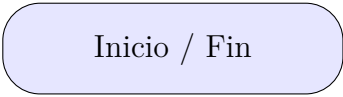
Buena práctica

La correcta disposición de los símbolos promueve la legibilidad del diagrama, permitiendo un flujo natural generalmente descendente y de izquierda a derecha. Seguir las convenciones visuales internacionales asegura que los diagramas puedan ser interpretados de manera uniforme, incluso por lectores externos al entorno académico o profesional del autor [21].

Inicio/fin

El inicio y el fin de un diagrama de flujo se representan mediante un óvalo o un rectángulo con bordes redondeados. En su interior se escribe la palabra correspondiente a la función del símbolo, ya sea *Inicio* o *Fin*. Este elemento indica los límites del algoritmo y asegura que el flujo esté correctamente definido desde su punto de entrada hasta su terminación. Su presencia es obligatoria en cualquier diagrama y constituye la base estructural sobre la cual se desarrolla el resto del flujo [13].

La Figura 4.5 ilustra el símbolo estándar para inicio/fin utilizado en diagramas de flujo, siguiendo la notación más empleada en textos de ingeniería y ciencias de la computación.



Inicio / Fin

Figura 4.5: Símbolo estándar de inicio y fin en un diagrama de flujo.

Proceso

El símbolo de proceso se representa mediante un rectángulo de bordes rectos y se utiliza para indicar una operación que el algoritmo debe ejecutar de forma determinista. Estas operaciones pueden incluir asignaciones, cálculos aritméticos, transformaciones o cualquier instrucción interna que no implique bifurcación del flujo. Su uso es esencial para expresar la secuencia lógica del procedimiento y constituye la base de la ejecución operacional del algoritmo [60], [68].

Como ejemplo, en la Figura 4.6, se muestra el símbolo de proceso ejecutando una operación de incremento:

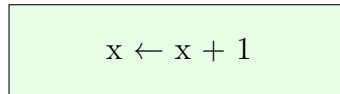


Figura 4.6: Símbolo de proceso para representar una operación interna.

Decisión

El símbolo de decisión se representa mediante un rombo y permite expresar bifurcaciones en el flujo del algoritmo. En su interior se escribe una condición lógica cuya evaluación determinará qué camino seguirá el flujo. Este símbolo es fundamental para modelar estructuras selectivas y tomar decisiones basadas en valores de entrada o estados intermedios. Los estudios sobre pensamiento computacional destacan que la bifurcación es un concepto clave para comprender la lógica condicional [17], [18], [67].

Caso 1: Rama verdadera. Cuando la condición se evalúa como verdadera, el flujo continúa por la rama etiquetada como *Sí*. Esta ruta contiene las acciones que deben ejecutarse únicamente cuando la condición se cumple. La Figura 4.7 ilustra un ejemplo simple.

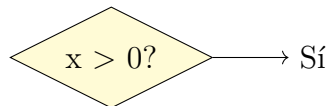


Figura 4.7: Rama verdadera del símbolo de decisión.

Caso 2: Rama falsa. Si la condición se evalúa como falsa, el flujo continúa por la rama *No*. Esto permite definir acciones alternativas que solo deben ejecutarse en ausencia del cumplimiento de la condición. La Figura 4.8 representa este comportamiento.

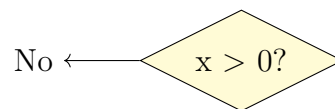


Figura 4.8: Rama falsa del símbolo de decisión.

Unión de ambas ramas y continuación del flujo. En muchos algoritmos, independientemente de si la condición evaluada resulta verdadera o falsa, ambas ramas deben converger posteriormente para continuar con un mismo conjunto de instrucciones. Esta convergencia representa el cierre lógico de una estructura condicional del tipo *Si-SiNo* y permite retomar el flujo secuencial del algoritmo. Desde el punto de vista conceptual, esta unión garantiza que el proceso no quede fragmentado y que todas las posibles rutas de ejecución conduzcan a un estado común, lo cual es esencial para la corrección y completitud del algoritmo [18], [67].

La Figura 4.9 muestra un ejemplo completo de un símbolo de decisión con sus dos ramas —verdadera y falsa— que, tras ejecutar acciones distintas, se unen nuevamente para continuar con el flujo normal del diagrama. Este patrón es uno de los más utilizados en la construcción de diagramas de flujo y tiene una correspondencia directa con las estructuras condicionales utilizadas en pseudocódigo y lenguajes de programación.

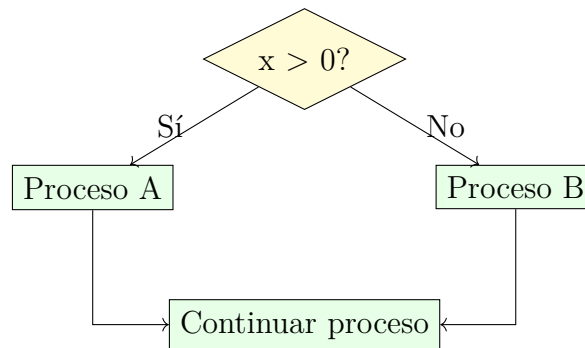


Figura 4.9: Decisión con ramas verdadera y falsa que convergen para continuar el flujo del algoritmo.

Entrada/salida

El símbolo de entrada/salida se representa mediante un paralelogramo y se utiliza para indicar tanto la captura de datos desde el usuario como la presentación de resultados. Este símbolo permite incorporar la interacción dentro del algoritmo y es esencial en aplicaciones que requieren comunicación con el entorno. La teoría de interacción humano-computadora destaca su importancia para representar interfaces básicas en modelos algorítmicos [26].

Caso 1: Lectura de datos. Cuando el símbolo representa una operación de entrada, se especifica dentro de él la acción de lectura, indicando el dato que debe ser proporcionado por el usuario. La Figura 4.10 muestra un ejemplo de lectura.

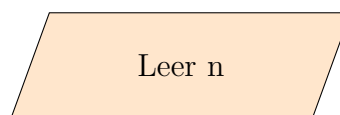


Figura 4.10: Símbolo de entrada para captura de datos.

Caso 2: Escritura de datos. Cuando representa una salida, el símbolo contiene el mensaje o dato que se desea mostrar al usuario. La Figura 4.11 ilustra este uso.

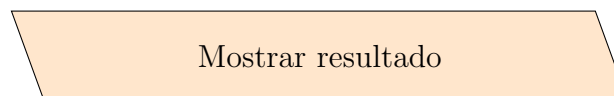


Figura 4.11: Símbolo de salida para presentación de información.

Entradas especializadas según el origen de los datos. En los diagramas de flujo, además de indicar la lectura de datos de forma genérica, es posible especificar el *origen* desde el cual se obtienen dichos datos. Esta diferenciación resulta especialmente relevante en algoritmos que interactúan con distintos medios de entrada, como dispositivos físicos o fuentes de almacenamiento. Representar explícitamente el origen de los datos mejora la comprensión del algoritmo y facilita su posterior implementación en sistemas reales [23], [37].

Entrada manual (teclado). La entrada manual representa la introducción de datos realizada directamente por una persona, generalmente mediante el uso del teclado u otro dispositivo de entrada manual. En los diagramas de flujo, este tipo de entrada se representa mediante el símbolo de *entrada manual*, caracterizado por tener la forma de un polígono similar a un rectángulo, pero con el borde superior inclinado y de menor longitud que el inferior. Esta forma distingue claramente la entrada manual de otras fuentes de datos, como archivos o dispositivos automáticos. La Figura 4.12 muestra el símbolo estándar utilizado para representar la entrada manual en diagramas de flujo, de acuerdo con la simbología ampliamente aceptada.

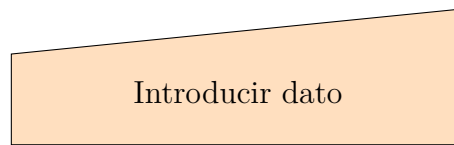


Figura 4.12: Símbolo de entrada manual (teclado) en un diagrama de flujo.

Salida por pantalla (visualización). La salida por pantalla representa una operación mediante la cual el sistema muestra información directamente al usuario a través de un dispositivo de visualización, como un monitor o una interfaz gráfica. En los diagramas de flujo, este tipo de salida no se representa con el símbolo genérico de entrada/salida, sino mediante el *símbolo de visualización (display)*, el cual permite distinguir la presentación visual de información de otros tipos de salida, como la impresión de documentos o el almacenamiento en archivos. Esta diferenciación resulta fundamental para una correcta interpretación del flujo de información dentro del sistema.

El símbolo de visualización se caracteriza por una forma similar a un rectángulo con uno de sus lados curvado, lo que indica que la información es mostrada al usuario de manera directa y no almacenada ni transmitida a otro medio. De acuerdo con la simbología empleada en herramientas de diagramación ampliamente aceptadas, como SmartDraw, este símbolo se utiliza específicamente para representar operaciones de salida visual, tales como mostrar resultados, mensajes o datos procesados en pantalla [69].

Desde el punto de vista del análisis y diseño de sistemas, la separación entre salidas visuales y otros tipos de salida contribuye a una mayor claridad del modelo y facilita la transición del diagrama de flujo hacia su implementación en software. Autores clásicos en el área de sistemas de información y programación estructurada recomiendan el uso explícito del símbolo de visualización cuando la interacción con el usuario se produce a través de la pantalla, ya que esto mejora la legibilidad del diagrama y reduce ambigüedades durante el desarrollo y mantenimiento del sistema [1], [37].

La Figura 4.13 muestra el símbolo de visualización empleado en los diagramas de flujo para representar la salida de información por pantalla, el cual será utilizado de forma consistente a lo largo de los ejemplos y ejercicios desarrollados en este capítulo.



Figura 4.13: Símbolo de visualización utilizado para representar la salida de información por pantalla en los diagramas de flujo.

Salida por impresora (documento impreso). La salida por impresora representa una operación mediante la cual el sistema genera información en un soporte físico, generalmente en forma de documentos impresos como reportes, comprobantes, listados o formularios [1], [61]. A diferencia de la salida por pantalla, este tipo de salida implica la materialización permanente de la información, lo que resulta especialmente relevante en contextos administrativos, legales o contables donde se requiere evidencia tangible de los resultados procesados por el sistema [1], [23].

En los diagramas de flujo, la salida por impresora no se representa mediante el símbolo de visualización, sino a través del *símbolo de documento*, el cual permite identificar de manera explícita que la información producida será impresa o generada como un archivo con estructura de documento [61], [69]. Este símbolo facilita la distinción entre salidas transitorias (visualización) y salidas persistentes (documentos), contribuyendo a una interpretación más precisa del flujo de información dentro del sistema [37].

La Figura 4.14 muestra el símbolo de documento utilizado en los diagramas de flujo para representar la salida de información destinada a impresión o generación de documentos, el cual será empleado de forma consistente en los ejemplos y ejercicios desarrollados en este capítulo [1], [61].



Figura 4.14: Símbolo de documento utilizado para representar la salida de información impresa o generada como documento en los diagramas de flujo.

Salida de múltiples documentos. La salida de múltiples documentos representa una operación mediante la cual el sistema genera simultáneamente más de un documento como resultado de un mismo proceso [1], [61]. Este tipo de salida es común en sistemas de información administrativos y organizacionales, por ejemplo, cuando se producen varios reportes, listados, comprobantes o formularios asociados a una misma transacción o evento [1], [23]. A diferencia de la salida de un único documento, este caso implica el manejo conjunto de un conjunto de documentos que deben ser considerados como una unidad lógica dentro del flujo del sistema [37].

En los diagramas de flujo, la salida de múltiples documentos se representa mediante el *símbolo de múltiples documentos*, el cual se construye a partir del símbolo de documento básico dispuesto de forma superpuesta [69]. Esta representación gráfica permite identificar de manera inmediata que la operación produce o utiliza varios documentos, evitando ambigüedades con respecto a salidas simples y facilitando la correcta interpretación del proceso modelado [37], [61].

La Figura 4.15 muestra el símbolo de múltiples documentos utilizado en los diagramas de flujo para representar la generación o tratamiento simultáneo de varios documentos impresos o estructurados. Este símbolo será empleado de manera consistente en los ejemplos y ejercicios desarrollados en este capítulo, contribuyendo a una lectura clara y precisa de los flujos de información en los sistemas analizados [1], [23].

Subproceso (Proceso Predefinido).

El subproceso, también denominado *proceso predefinido*, representa una operación cuya lógica interna se encuentra especificada en otro lugar del algoritmo o del sistema, y que es invocada como una unidad reutilizable dentro del flujo principal [37], [61]. En programación básica, este símbolo se utiliza para modelar funciones, procedimientos o módulos ya definidos, de modo que el diagrama de flujo se mantenga legible sin perder precisión, delegando los detalles de implementación a una descripción separada [23].

En los diagramas de flujo, el subproceso se distingue del símbolo de proceso estándar mediante un rectángulo con doble borde vertical (líneas paralelas a ambos lados) [61], [69]. Esta convención permite identificar visualmente que la actividad no se desarrolla paso a paso en el diagrama actual, sino que corresponde a una rutina invocada, cuya ejecución produce un resultado coherente y controlado antes de retornar al flujo principal [1]. Esta diferenciación resulta especialmente útil cuando el algoritmo incorpora tareas repetitivas o de complejidad moderada que conviene encapsular para facilitar mantenimiento y comprensión [23].

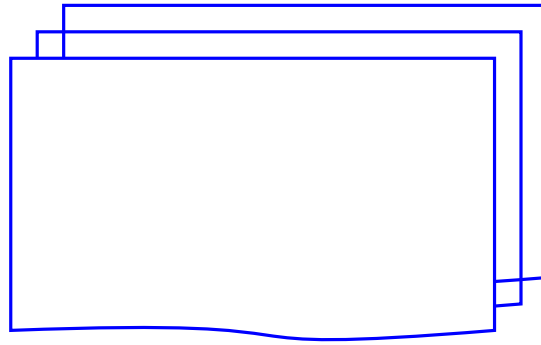


Figura 4.15: Símbolo de Multidocumento.

La Figura 4.16 muestra el símbolo de subproceso utilizado en los diagramas de flujo para representar procesos predefinidos que se ejecutan como módulos reutilizables. Este símbolo será empleado de manera consistente en los ejemplos y ejercicios desarrollados en este capítulo, particularmente en aquellos donde se estructuran algoritmos mediante funciones o procedimientos claramente delimitados [37], [61].



Figura 4.16: Símbolo de subproceso (proceso predefinido) utilizado para representar una rutina o módulo invocado desde el flujo principal.

Almacenamiento Interno

El almacenamiento interno representa una operación mediante la cual el sistema guarda datos de forma temporal o permanente dentro de su propia memoria o en estructuras internas de almacenamiento, tales como variables, arreglos, registros o áreas de memoria gestionadas por el programa [37], [61]. Este tipo de almacenamiento se utiliza para conservar información necesaria durante la ejecución del algoritmo, permitiendo su posterior recuperación y manipulación sin requerir interacción externa ni acceso a dispositivos físicos de entrada o salida [10].

En los diagramas de flujo, el símbolo de almacenamiento interno se emplea para indicar explícitamente que los datos permanecen dentro del sistema y son utilizados como parte del procesamiento lógico [61], [69]. Su uso resulta fundamental para distinguir entre información que se conserva internamente durante la ejecución del programa y aquella que se envía a medios externos, como archivos o dispositivos de salida [37]. Esta diferenciación contribuye a una mejor comprensión del ciclo de vida de los datos dentro del algoritmo y facilita el análisis del flujo de información [1].

Desde el punto de vista del análisis y diseño de sistemas, la identificación clara de las operaciones de almacenamiento interno permite modelar con mayor precisión el comportamiento del sistema, especialmente en algoritmos que dependen del manejo de estados, acumuladores o estructuras de datos intermedias [23]. En este sentido, la Figura 4.17 presenta el símbolo de almacenamiento interno utilizado en los diagramas de flujo para representar la retención y gestión de datos en memoria, el cual será empleado de manera consistente en los ejemplos y ejercicios desarrollados en este capítulo.

Base de Datos (Almacenamiento en Base de Datos)

La base de datos representa un mecanismo de almacenamiento estructurado y persistente mediante el cual el sistema conserva grandes volúmenes de información de forma organizada, coherente y accesible a lo largo del tiempo [28], [29]. A diferencia del almacenamiento interno, que suele emplearse para datos temporales o de uso inmediato durante la ejecución del algoritmo,

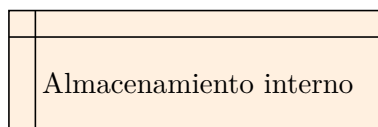


Figura 4.17: Símbolo de Almacenamiento interno.

la base de datos está diseñada para garantizar la permanencia de la información, su integridad y su disponibilidad para múltiples procesos o usuarios [27], [28].

En los diagramas de flujo, el símbolo de base de datos se utiliza para indicar operaciones de almacenamiento o recuperación de información desde un repositorio persistente gestionado por un sistema de gestión de bases de datos [1], [69]. Este símbolo permite diferenciar claramente las acciones que involucran memoria interna del programa de aquellas que implican acceso a datos almacenados de manera estructurada y duradera, como registros de clientes, transacciones, historiales académicos o inventarios [28], [29]. Su correcta identificación resulta esencial para comprender la arquitectura lógica del sistema y el flujo de datos entre los distintos componentes [1].

Desde la perspectiva del análisis y diseño de sistemas de información, el uso explícito del símbolo de base de datos contribuye a una modelación más precisa de los procesos que dependen de información persistente y compartida [1], [23]. La Figura 4.18 presenta el símbolo de base de datos empleado en los diagramas de flujo para representar el almacenamiento y acceso a datos persistentes, el cual será utilizado de manera consistente en los ejemplos y ejercicios desarrollados en este capítulo.

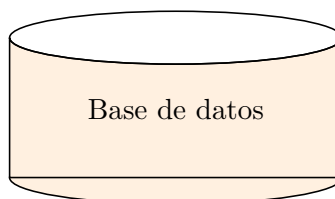


Figura 4.18: Símbolo de Base de datos (almacenamiento persistente).

Datos Externos

Los datos externos representan información que no es creada, almacenada ni gestionada directamente por el programa que se está modelando, pero que resulta necesaria para su funcionamiento o que es producida por él para ser utilizada fuera del sistema. En el contexto del análisis y diseño de sistemas, estos datos pertenecen a entidades, sistemas o fuentes externas al límite lógico del programa, aunque interactúan con él durante su ejecución [16].

En los diagramas de flujo, el símbolo de datos externos se emplea para indicar operaciones en las que el programa obtiene información desde fuentes externas o envía resultados hacia otros sistemas, sin asumir la responsabilidad sobre el almacenamiento, la persistencia o la administración de dichos datos. Desde una perspectiva introductoria, los estudiantes pueden comprender los datos externos como información que el algoritmo recibe o entrega, pero sobre la cual no tiene control directo, limitándose únicamente a su uso dentro del flujo lógico del programa [65].

Ejemplos comunes de datos externos incluyen información obtenida desde servicios web, aplicaciones remotas, plataformas externas o sistemas de información distintos, así como datos que el programa proporciona a otros sistemas mediante interfaces de comunicación. En estos casos, el algoritmo no describe cómo se almacenan o mantienen esos datos fuera del sistema, sino únicamente cómo se integran en el proceso de cálculo o decisión [16], [69].

La Figura 4.19 presenta el símbolo de datos externos utilizado en los diagramas de flujo. La forma con lados curvos hacia el interior sugiere gráficamente que la información atraviesa el límite del sistema, destacando que los datos no permanecen dentro del programa. El uso adecuado de este símbolo contribuye a que los estudiantes diferencien claramente entre datos internos del programa y datos que provienen de, o se dirigen hacia, otros sistemas, favoreciendo una comprensión más precisa del alcance y las responsabilidades del algoritmo [69].

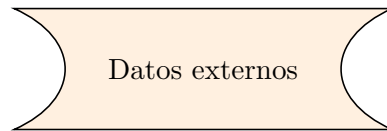


Figura 4.19: Símbolo de Datos externos.

Tira (paper tape)

El símbolo de *tira* (también denominado *paper tape*) se emplea en diagramas de flujo para representar el manejo de información en un medio *secuencial*, históricamente asociado a la cinta de papel perforada utilizada como soporte de entrada/salida en los primeros sistemas de cómputo. En términos didácticos, este símbolo permite enfatizar que los datos se registran o se leen siguiendo un orden lineal, sin acceso directo a posiciones arbitrarias, a diferencia de estructuras como memoria principal o bases de datos [61], [65].

En un curso de fundamentos de programación, la utilidad principal de este símbolo no radica en el dispositivo histórico, sino en la idea algorítmica que introduce: existen fuentes o destinos de datos cuyo acceso es predominantemente *por recorrido* (por ejemplo, una secuencia de registros, una lista de eventos, un flujo de lectura o un conjunto de resultados que se generan en orden). En consecuencia, cuando el estudiante modela un algoritmo que procesa elementos uno tras otro (lectura secuencial, acumulación progresiva, generación ordenada de salidas), la tira ayuda a reforzar la noción de *procesamiento secuencial* y a distinguirla de operaciones donde se consulta o actualiza información de manera aleatoria [65], [69].

Asimismo, en diagramas de flujo contemporáneos, el símbolo de tira puede interpretarse como una convención gráfica para señalar salidas impresas o registros generados en serie, así como entradas provenientes de un flujo continuo de información, siempre que se busque remarcar el carácter secuencial del manejo de datos. Esta interpretación resulta especialmente apropiada en ejercicios introductorios donde el objetivo es comprender el flujo de control y el recorrido de datos, más que modelar detalles de infraestructura [61], [69].

La Figura 4.20 presenta el símbolo de tira utilizado en los diagramas de flujo para representar el manejo secuencial de datos, el cual será empleado de manera consistente en los ejemplos y ejercicios desarrollados en este capítulo.

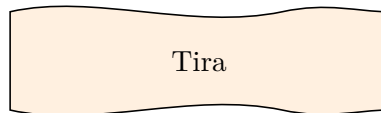


Figura 4.20: Símbolo de Tira (paper tape).

Operación manual

La operación manual representa una actividad dentro del proceso que es realizada directamente por una persona y no por el sistema computacional. A diferencia de las operaciones automáticas, que son ejecutadas por el programa mediante instrucciones precisas, una operación manual requiere intervención humana para completar una tarea específica, como clasificar información, revisar resultados, tomar una decisión externa al algoritmo o realizar una acción física fuera del sistema [37], [61].

En los diagramas de flujo, este símbolo se utiliza para indicar que el control del proceso se transfiere temporalmente al usuario u operador humano. Desde una perspectiva introductoria, resulta importante que el estudiante comprenda que no todas las actividades asociadas a un algoritmo pueden o deben ser automatizadas. Existen situaciones en las que el programa depende de acciones humanas, ya sea por razones de validación, supervisión, seguridad o simplemente porque la tarea no puede ser resuelta de forma automática [1].

En el contexto de la programación básica, la operación manual permite modelar correctamente procesos mixtos, en los cuales interactúan tanto el sistema como las personas. Por ejemplo, un algoritmo puede calcular resultados automáticamente, pero requerir que un usuario los interprete, los apruebe o introduzca ajustes antes de continuar. Representar estas acciones mediante el símbolo de operación manual ayuda a delimitar claramente las responsabilidades del programa y del usuario, favoreciendo una comprensión más realista del funcionamiento de los sistemas de información [61], [69].

La Figura 4.21 presenta el símbolo de operación manual empleado en los diagramas de flujo para indicar actividades que deben ser realizadas por una persona y no por el sistema computacional. Este símbolo será utilizado de manera consistente en los ejemplos y ejercicios desarrollados en este capítulo, con el fin de reforzar la distinción entre procesos automáticos y acciones humanas dentro de un algoritmo.

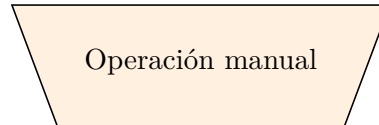


Figura 4.21: Símbolo de Operación manual.

Preparación

El símbolo de *preparación* se utiliza en los diagramas de flujo para representar operaciones de inicialización o configuración previa que deben realizarse antes de ejecutar un proceso principal o una estructura repetitiva. Estas operaciones establecen el estado inicial del algoritmo, definiendo valores de variables, contadores o acumuladores que serán utilizados posteriormente durante la ejecución del programa. En programación básica, este paso resulta fundamental para garantizar el correcto funcionamiento del algoritmo y evitar errores derivados de variables no inicializadas [61].

Desde una perspectiva didáctica, el símbolo de preparación permite al estudiante identificar claramente aquellas acciones que no forman parte del procesamiento repetitivo o del cálculo central, pero que son necesarias para preparar el entorno de ejecución. Por ejemplo, asignar un valor inicial a un contador, establecer el valor inicial de una suma acumulada o definir límites iniciales para un recorrido son acciones típicas que se representan mediante este símbolo. De este modo, el diagrama de flujo refleja de forma explícita la secuencia lógica que sigue el algoritmo desde su estado inicial hasta su ejecución normal [37], [61].

Es importante destacar que el símbolo de preparación no representa por sí mismo una estructura iterativa como el ciclo `for`, sino que modela la fase de inicialización que suele preceder a este tipo de estructuras. En un ciclo `for`, por ejemplo, la asignación inicial del índice se representa mediante el símbolo de preparación, mientras que la condición de repetición y la actualización del contador se modelan con otros símbolos del diagrama. Esta diferenciación ayuda a los estudiantes a comprender que una estructura iterativa está compuesta por varias partes lógicas y no por un único símbolo [1], [61].

La Figura 4.22 presenta el símbolo de preparación empleado en los diagramas de flujo para representar operaciones de inicialización o configuración previa, el cual será utilizado de manera consistente en los ejemplos y ejercicios desarrollados en este capítulo, especialmente en aquellos que incluyen estructuras repetitivas y control de variables.

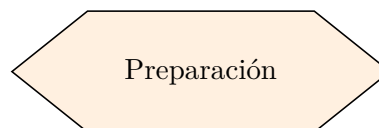


Figura 4.22: Símbolo de Preparación.

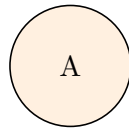
Conectores

Los conectores se emplean en los diagramas de flujo para indicar la continuidad lógica del proceso cuando el flujo no puede representarse de manera completamente lineal o cuando resulta conveniente interrumpirlo y retomarlo en otro punto del diagrama (ver Figura 4.23). Su uso permite evitar cruces innecesarios de líneas, mejorar la claridad visual y facilitar la comprensión del algoritmo, especialmente en procesos extensos o con múltiples ramificaciones [37], [61].

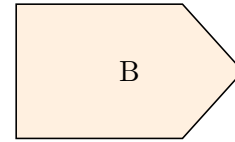
Los conectores ayudan al diseñador a comprender que el flujo de control de un algoritmo puede continuar en otra ubicación sin que ello implique una ruptura lógica. La Figura ?? muestra el *conector en la misma página*, representado por

un pequeño círculo, el cual se utiliza cuando el flujo se retoma dentro del mismo diagrama. Este tipo de conector se identifica mediante una letra o número que debe coincidir exactamente con el punto de continuidad correspondiente.

Por su parte, la Figura 4.23b presenta el *conector fuera de página*, empleado cuando el flujo continúa en otra página del diagrama. Este símbolo permite mantener la legibilidad del algoritmo cuando su extensión excede el espacio disponible, asegurando que la secuencia lógica se preserve de forma clara y ordenada. El uso correcto de ambos tipos de conectores contribuye a que los estudiantes diseñen diagramas más estructurados y comprendan mejor el recorrido del control en un programa [1], [61].



(a) Conector en la misma página.



(b) Conector fuera de página.

Figura 4.23: Símbolos de conectores utilizados en diagramas de flujo.

4.7.2. Construcción de diagramas

Los diagramas de flujo constituyen una herramienta fundamental para la representación gráfica de algoritmos, ya que permiten describir de manera clara y ordenada la secuencia de operaciones necesarias para resolver un problema. Su principal ventaja radica en que facilitan la comprensión del flujo lógico de un proceso, independientemente del lenguaje de programación que se utilice posteriormente para su implementación. En el contexto de la programación básica, los diagramas de flujo actúan como un puente entre el análisis del problema y la codificación de la solución, ayudando al estudiante a estructurar el pensamiento algorítmico antes de enfrentarse al código fuente [37], [61].

Desde el punto de vista del análisis de sistemas, los diagramas de flujo permiten identificar entradas, procesos, decisiones, salidas y almacenamiento de datos de forma visual, lo que reduce ambigüedades y errores conceptuales en etapas tempranas del desarrollo. Esta representación gráfica resulta especialmente útil para validar la lógica del algoritmo, comunicar la solución a otros integrantes del equipo y documentar el funcionamiento del sistema. Autores clásicos destacan que un diagrama de flujo bien construido contribuye a mejorar la calidad del software, al facilitar su revisión, mantenimiento y comprensión a lo largo del tiempo [1], [23].

En entornos educativos, el uso sistemático de diagramas de flujo favorece el aprendizaje progresivo de la programación, ya que permite a los estudiantes centrarse primero en la lógica del problema y no en la sintaxis de un lenguaje específico. De este modo, el estudiante desarrolla habilidades de análisis, descomposición de problemas y razonamiento secuencial, competencias esenciales para la formación en ingeniería y sistemas de información [61].

¿Cómo se construye un diagrama de flujo?

La construcción de un diagrama de flujo inicia con la comprensión completa del problema, identificando claramente qué se desea resolver y bajo qué condiciones. En esta etapa se determinan los datos de entrada, los resultados esperados y las restricciones del problema. A partir de esta información, se define la secuencia general de pasos que llevará desde las entradas hasta las salidas, empleando los símbolos normalizados descritos previamente en el documento [5], [7], [59].

Posteriormente, el diagrama se refina incorporando decisiones, cálculos intermedios, ciclos y subprocesos, según sea necesario. Cada paso debe representarse mediante el símbolo correspondiente, manteniendo una dirección clara del flujo y evitando cruces innecesarios de líneas [70]. El objetivo es obtener un diagrama legible, coherente y fiel a la lógica del algoritmo que se implementará más adelante.

Consejos para la Construcción de Diagramas de Flujo

- Utilizar siempre los símbolos normalizados y definidos en el capítulo.
- Mantener el flujo del diagrama de arriba hacia abajo y de izquierda a derecha.

- Evitar diagramas excesivamente largos; emplear conectores o subprocesos cuando sea necesario.
- Etiquetar claramente las decisiones y condiciones lógicas.
- Usar nombres significativos para variables y procesos.
- Revisar el diagrama para verificar que toda entrada conduce a una salida.
- Asegurar que el diagrama pueda entenderse sin explicaciones adicionales.

4.8. Resolver Problemas Mediante Algoritmos y/o Diagramas de Flujo

Resolver problemas mediante algoritmos implica transformar una necesidad expresada en lenguaje natural en una secuencia finita, ordenada y precisa de pasos que una computadora pueda ejecutar. Desde una perspectiva formativa, este proceso no se limita a obtener un resultado correcto, sino que busca desarrollar en el programador la capacidad de analizar, estructurar y razonar soluciones de manera sistemática. La literatura en ciencias de la computación enfatiza que un buen algoritmo comienza con una comprensión profunda del problema y no con la escritura inmediata de código [7], [52].

En el ámbito de los sistemas de información, los problemas suelen involucrar datos, reglas de negocio, validaciones y múltiples escenarios posibles. Por ello, resulta fundamental adoptar una metodología que permita identificar claramente las entradas, salidas, procesos y condiciones que gobiernan la solución. Este enfoque reduce errores conceptuales y facilita la implementación posterior en cualquier lenguaje de programación [23], [71].

4.8.1. Análisis del problema.

Para analizar y comprender correctamente un problema, el programador principiante debe leer cuidadosamente el planteamiento hasta entenderlo completamente [17], [59]. Esta lectura inicial no debe ser superficial; es necesario identificar con claridad qué se pide resolver, cuáles son los datos que intervienen, qué resultados se esperan y cuáles pueden ser las restricciones del problema [61]. El objetivo es lograr una comprensión suficiente del enunciado como para poder explicarlo con palabras propias.

Una vez alcanzado este nivel de comprensión, es recomendable interiorizar el problema —incluso memorizar su idea central— de modo que, en los pasos posteriores de diseño del algoritmo, no sea necesario regresar constantemente al planteamiento original. Evitar estas interrupciones permite mantener la concentración en el razonamiento lógico y en la construcción de la solución [7]. Muchos errores en programación ocurren precisamente porque el programador intenta diseñar el algoritmo mientras todavía está tratando de comprender el problema [23].

Luego, el programador debe plantearse ejercicios o casos típicos que estén dentro del contexto del problema. Estos ejercicios consisten en instancias concretas del problema general y deben resolverse manualmente, como si se tratara de situaciones específicas o puntuales [59]. Este proceso permite observar cómo deberían comportarse los datos y los resultados antes de formalizar la solución mediante un algoritmo [53].

Es importante que estos casos de prueba incluyan también los llamados *valores límite* o *casos frontera* [24]. Los valores límite corresponden a situaciones extremas o especiales que pueden provocar errores si no se consideran adecuadamente durante el diseño del algoritmo [23]. Por ejemplo, si el problema implica divisiones, debe analizarse qué ocurre cuando el divisor es cero; si se procesan cantidades, debe evaluarse qué sucede cuando el valor es cero o cuando alcanza su máximo permitido, especialmente con valores reales. Analizar estos casos permite anticipar posibles fallos y diseñar soluciones más robustas [7].

Estos problemas específicos, además de ayudar a comprender mejor el comportamiento del problema, proporcionan datos de prueba que posteriormente servirán para verificar la solución diseñada [24]. Es decir, los mismos valores utilizados durante el análisis manual pueden emplearse después para comprobar que el algoritmo o el diagrama de flujo realmente produce los resultados esperados.

En consecuencia, resolver manualmente varios casos representativos antes de escribir el algoritmo no solo facilita la comprensión del problema, sino que también constituye una primera forma de validación de la solución [23], [40]. Esta práctica permite al programador principiante desarrollar una visión clara del proceso que el algoritmo deberá seguir y reduce significativamente la probabilidad de cometer errores conceptuales durante la implementación.

4.8.2. Identificación de Datos de Entrada

Los datos de entrada constituyen la información que el algoritmo requiere para iniciar su ejecución y producir resultados significativos. Su correcta identificación exige analizar cuidadosamente el enunciado del problema y localizar aquellos valores que pueden variar entre distintas ejecuciones. Una regla práctica ampliamente aceptada establece que, si un valor influye en el resultado del algoritmo y no está fijado explícitamente por el problema, debe considerarse como un dato de entrada [7].

Desde un enfoque metodológico en el diseño algorítmico, resulta fundamental identificar con precisión dichos datos. Para ello, es recomendable formular preguntas analíticas como: ¿qué información es indispensable para que el algoritmo pueda ejecutarse y producir resultados válidos?, ¿qué datos dependen del contexto del problema y no pueden asumirse como constantes del sistema? Este razonamiento permite diferenciar claramente entre los valores que deben ser solicitados explícitamente al usuario y aquellos que pueden definirse internamente como parte de la lógica del algoritmo.

En este sentido, se consideran datos de entrada todos aquellos valores que influyen en el resultado y que no pueden ser obtenidos mediante cálculos internos ni generados automáticamente por el sistema. Estos datos deben ser proporcionados necesariamente por el usuario o por una fuente externa confiable. Por ejemplo, en el cálculo del promedio de calificaciones, las notas individuales constituyen datos de entrada, mientras que el criterio de aprobación —como una calificación mínima de 7/10— puede modelarse como una constante definida por las reglas del sistema [71]. Esta distinción favorece un diseño algorítmico más claro, coherente y alineado con las buenas prácticas de programación.

i

Para consideración...

La identificación de los datos de entrada implica también definir su tipo y sus restricciones. El programador en formación debe habituarse a determinar si un dato es entero, real, carácter o conjunto de valores, así como los rangos válidos que puede asumir. Esta práctica resulta esencial para prevenir errores lógicos y facilita la validación temprana de datos, una recomendación recurrente en la ingeniería de software contemporánea [24].

4.8.3. Identificación de Datos de Salida

Los datos de salida representan la información que el algoritmo entrega al usuario o a otro sistema una vez procesadas las entradas. Para identificarlos, es recomendable buscar en el enunciado expresiones como *mostrar*, *calcular*, *determinar* o *generar*. Estas palabras suelen indicar explícitamente qué resultados se esperan al finalizar la ejecución [7].

Desde una perspectiva formativa, resulta fundamental diferenciar entre salidas principales y salidas secundarias en el diseño de algoritmos. Las salidas principales representan el objetivo central del problema y corresponden a los resultados finales que el usuario espera obtener. En cambio, las salidas secundarias incluyen mensajes informativos, advertencias o resultados parciales que facilitan la comprensión del proceso o apoyan la interpretación de los resultados [7], [23].

Por ejemplo, en un algoritmo que simula un cronómetro, el tiempo restante mostrado en cada iteración constituye una salida informativa de carácter secundario, mientras que el mensaje final indica la culminación del proceso y actúa como salida principal. De manera similar, en la resolución del problema de calcular el promedio y la desviación estándar de un conjunto de valores, los resultados intermedios —como la suma de los datos y la varianza— pueden considerarse salidas secundarias si se muestran al usuario, mientras que el promedio y la desviación estándar representan las salidas principales del algoritmo [52], [71].

Esta distinción contribuye a estructurar de forma clara los resultados del algoritmo y permite al programador decidir qué información es esencial presentar y qué información cumple únicamente un rol de apoyo, favoreciendo así un diseño más comprensible y orientado a las necesidades del usuario.

Definir claramente las salidas permite verificar si el algoritmo realmente resuelve el problema planteado. En ingeniería de sistemas de información, una salida mal definida puede conducir a sistemas técnicamente correctos pero funcionalmente inútiles, ya que no proporcionan la información que el usuario necesita para tomar decisiones [23], [24].

4.8.4. Identificación de Cálculos Intermedios (Procesos Intermedios)

Los cálculos intermedios son operaciones necesarias para transformar los datos de entrada en resultados finales, aunque no siempre se muestran directamente como salida. Su función es preparar la información para decisiones posteriores o para cálculos finales más complejos. Por ejemplo, al calcular la desviación estándar, el promedio es un cálculo intermedio indispensable [52].

Para identificarlos, resulta útil descomponer el problema en pasos lógicos y observar qué resultados parciales se necesitan en cada etapa. Esta técnica, conocida como refinamiento sucesivo, permite manejar la complejidad de manera progresiva y es ampliamente recomendada en textos de programación y algoritmos [71].

Desde el punto de vista del diseño, los cálculos intermedios suelen almacenarse en variables auxiliares. Nombrar estas variables de forma clara y coherente mejora la legibilidad del algoritmo y facilita su mantenimiento, una práctica destacada como fundamental en el desarrollo de software de calidad [51].

4.8.5. Identificación de Cálculos Finales (Procesos Finales)

Los cálculos finales son aquellos que producen directamente el resultado solicitado por el problema. Normalmente se ejecutan una vez que se han completado todos los cálculos intermedios y se dispone de la información necesaria. Por ejemplo, dividir la suma total de valores entre la cantidad de datos para obtener un promedio constituye un cálculo final [7].

Un aspecto didáctico importante consiste en separar el cálculo del resultado de su presentación. Es decir, calcular un valor y mostrarlo con un formato adecuado son tareas distintas. Esta separación favorece un diseño más limpio y flexible, permitiendo modificar la forma de presentación sin afectar la lógica del algoritmo [23].

En sistemas reales, los cálculos finales suelen estar asociados a decisiones importantes, como aprobar o rechazar una solicitud, emitir una alerta o generar un reporte. Por ello, su correcta definición es esencial para garantizar que el algoritmo cumpla con los objetivos del sistema [24].

4.8.6. Identificación de Condiciones (Decisiones)

Las condiciones determinan si una acción se ejecuta o no dentro del algoritmo. Aparecen cuando el problema incluye reglas, restricciones o clasificación de casos. Para identificarlas, se recomienda localizar en el enunciado expresiones condicionales como *si*, *cundo*, *en caso de* o *solo si*. Estas expresiones suelen traducirse directamente en estructuras condicionales [71].

Desde una perspectiva formativa, es fundamental que el programador en formación considere todos los escenarios posibles. Una técnica útil consiste en enumerar casos y resultados esperados antes de escribir el algoritmo. Esto ayuda a evitar condiciones incompletas, uno de los errores más comunes en el diseño algorítmico [7].

Las condiciones bien definidas no solo garantizan la corrección del algoritmo, sino que también reflejan con precisión las reglas del dominio del problema. En sistemas de información, estas reglas suelen representar políticas organizacionales o normativas, por lo que su correcta implementación es crítica [24].

4.8.7. Identificación de Repeticiones o Iteraciones

Las repeticiones se utilizan cuando un conjunto de acciones debe ejecutarse múltiples veces. Para identificarlas, se deben buscar en el enunciado indicios como *para cada*, *mientras*, *repetir* o *hasta que*. Estos términos sugieren la necesidad de una estructura iterativa [71].

Un criterio clave para seleccionar el tipo de ciclo es determinar si se conoce de antemano el número de repeticiones. Si se conoce, un ciclo **Para** suele ser apropiado; si no, se recurre a ciclos condicionados como **Mientras**. Esta distinción es fundamental para escribir algoritmos claros y evitar errores como ciclos infinitos [7].

Además, cada ciclo debe incluir una modificación explícita de la variable de control o una condición que garantice su terminación. Este principio básico del diseño algorítmico es ampliamente enfatizado en la literatura para prevenir fallos lógicos difíciles de detectar [52].

4.8.8. Condiciones para Finalizar el Algoritmo

Todo algoritmo debe tener un punto claro de terminación. En algoritmos simples, este punto coincide con la ejecución de la última instrucción. En algoritmos que incluyen ciclos, la terminación depende del cumplimiento de una condición de salida correctamente definida [71].

Desde un enfoque didáctico, es recomendable analizar tanto la terminación normal como la terminación ante entradas inválidas. Aunque en programación básica no se aborden mecanismos avanzados de manejo de errores, prever validaciones simples mejora la robustez del algoritmo y refuerza buenas prácticas de diseño [23].

i

Prueba de Algoritmos

Comprender cuándo y por qué un algoritmo termina ayuda al programador a razonar sobre su corrección y eficiencia. Un algoritmo que no termina o que termina en un estado incorrecto no cumple su propósito, independientemente de la calidad de su implementación [24].

4.9. Ejercicios Resueltos mediante Diagramas de Flujo

4.9.1. Ejercicio 1: Problema a resolver

Se requiere diseñar un algoritmo que calcule el *promedio aritmético* de una cantidad N de valores numéricos ingresados por el usuario.

Pasos para Desarrollar la Solución

Los pasos para desarrollar la solución algorítmica de un problema, están encaminados a plasmar en papel lo que el programador principiante o cualquier persona hace para resolver un problema.

1. Análisis del problema. Para comprender el problema es necesario plantearse algunos casos puntuales y resolverlos de manera aislada, como si fuesen únicos.

Ejercicios de análisis manual con valores límite. Antes de diseñar el algoritmo, es recomendable resolver manualmente algunos casos representativos del problema. A continuación se presentan distintos conjuntos de datos que permiten verificar el comportamiento del algoritmo para el cálculo del promedio de una cantidad N de valores.

■ Caso 1: Caso normal

1. Datos de entrada

- cantidad = 3
- valores = 6, 8, 10

2. Resolución manual

$$suma = 6 + 8 + 10 = 24$$

$$promedio = \frac{24}{3} = 8$$

3. Resultado esperado

$$promedio = 8$$

- ##### 4. Observación.
- Este caso permite verificar el funcionamiento básico del algoritmo cuando los datos son valores típicos dentro del dominio esperado.

■ Caso 2: Caso con valores cero

1. Datos de entrada

- cantidad = 4
- valores = 0, 5, 10, 15

2. Resolución manual

$$suma = 0 + 5 + 10 + 15 = 30$$

$$promedio = \frac{30}{4} = 7.5$$

3. Resultado esperado

$$promedio = 7.5$$

4. **Observación:** Este caso permite comprobar que el algoritmo maneja correctamente valores iguales a cero dentro del conjunto de datos.

■ Caso 3: Caso con valores negativos**1. Datos de entrada**

- cantidad = 3
- valores = -5, 10, 15

2. Resolución manual

$$suma = -5 + 10 + 15 = 20$$

$$promedio = \frac{20}{3} \approx 6.67$$

3. Resultado esperado

$$promedio \approx 6.67$$

4. **Observación:** Este caso permite verificar que el algoritmo funciona correctamente cuando algunos valores son negativos.

■ Caso 4: Caso límite mínimo**1. Datos de entrada**

- cantidad = 1
- valores = 12

2. Resolución manual

$$suma = 12$$

$$promedio = \frac{12}{1} = 12$$

3. Resultado esperado

$$promedio = 12$$

4. Este caso representa el número mínimo posible de datos y permite verificar que el algoritmo funciona incluso cuando solo existe un valor.

■ Caso 5: Caso límite inválido (división por cero)**1. Datos de entrada**

- `cantidad = 0`

2. **Resolución:** No es posible calcular el promedio porque se produciría una división por cero.

3. **Resultado esperado.** El algoritmo debe detectar esta situación y mostrar un mensaje de error, por ejemplo:

- “No es posible calcular el promedio porque la cantidad de valores es cero.”
- **Observación:** Este caso límite permite identificar la necesidad de validar los datos de entrada antes de realizar el cálculo final.

2. Identificación de datos de entrada. Los datos de entrada corresponden a la información que el algoritmo necesita recibir desde el exterior para poder ejecutar el cálculo. En el diagrama/algoritmo, estos valores se capturan mediante operaciones de lectura.

- **cantidad (entero):** número total de valores a procesar (N). Para ello, esta lectura debe ir siempre antecedida por un mensaje que advierta al usuario qué se espera que ingrese. Por ejemplo: “Digite la cantidad de valores”.
- **valor (real):** cada dato numérico individual ingresado iterativamente (uno por cada repetición).

La variable **cantidad** determina el tamaño del proceso y controla la repetición; **valor** alimenta el cálculo acumulativo.

3. Identificación de datos de salida. Los datos de salida son los resultados que el algoritmo entrega al usuario una vez finalizado el procesamiento.

- **promedio (real):** promedio aritmético de los **cantidad** valores ingresados.
- **Mensaje informativo:** “El promedio de los **cantidad** valores es: **promedio**”.

La salida combina un valor numérico con un texto descriptivo, similar a la lectura de datos. Práctica recomendada para mejorar la interpretabilidad del resultado en programas introductorios [7], [59].

4. Identificación de cálculos intermedios (procesos intermedios). Los procesos intermedios son operaciones necesarias para llegar al resultado, pero que no constituyen todavía la respuesta final.

- **Inicialización:** `suma  $\leftarrow$  0` y `contador  $\leftarrow$  0`. Garantiza un estado inicial consistente del acumulador y del índice de iteración.
- **Acumulación:** `suma  $\leftarrow$  suma + valor`. Integra cada entrada al total acumulado.
- **Avance de iteración:** `contador  $\leftarrow$  contador + 1`. Registra cuántos valores han sido procesados.

Estos pasos implementan el patrón *contador-acumulador*, ampliamente empleado en algoritmos de recorrido de colecciones y procesamiento incremental [7], [61].

5. Identificación de cálculos finales (procesos finales). Los procesos finales producen directamente el resultado que se reporta al usuario.

- **Cálculo del promedio:** `promedio  $\leftarrow$  suma / cantidad`.

Esta operación se ejecuta únicamente cuando la repetición ha concluido, es decir, cuando ya se han incorporado los **cantidad** valores a **suma** [59], [61].

6. Identificación de condiciones (decisiones). Las decisiones son expresiones lógicas que determinan el flujo de control, en particular la continuidad o detención de la repetición.

- **Condición principal:** `contador < cantidad`. Mientras sea verdadera, se continúa solicitando y sumando valores.

La comparación entre un contador y un límite representa una forma canónica de control de bucles y se asocia directamente a la noción de *invariante de iteración*: al inicio de cada vuelta, **contador** indica cuántos datos ya fueron incorporados a **suma** [53].

7. Identificación de repeticiones o iteraciones. El diagrama contiene una estructura repetitiva explícita controlada por la condición `contador < cantidad`.

- **Iteración controlada por contador:** por cada vuelta se solicita un nuevo **valor**, se actualiza **suma** y se incrementa **contador**.

Este esquema equivale, en términos de programación estructurada, a un ciclo tipo **while** o **for**, según el lenguaje de implementación [7], [59].

8. Condiciones para finalizar el algoritmo. El algoritmo finaliza cuando se cumple simultáneamente que:

- la condición **contador < cantidad** deja de ser verdadera (es decir, **contador = cantidad**);
- se ejecuta el cálculo **promedio** $\leftarrow$ **suma** / **cantidad** y se imprime el resultado.

En consecuencia, el término del proceso está ligado al cumplimiento del número de entradas solicitadas, no al valor numérico de los datos [7], [61].

Propuesta de Solución

La Figura 4.24 describe una solución algorítmica estructurada para calcular el promedio de N valores. El diseño separa con claridad la *inicialización* (**suma** y **contador**), la *captura iterativa* de entradas (lectura de **valor**), el *procesamiento acumulativo* (actualización de **suma**) y la *obtención del resultado* (cálculo de **promedio**). Esta organización favorece la verificación manual del flujo y la posterior traducción a código en un lenguaje de programación, manteniendo correspondencia directa entre pasos del diagrama y sentencias de control [59], [72].

El Algoritmo 4.39 presenta una propuesta de solución estructurada para calcular el promedio de una cantidad N de valores ingresados por el usuario. En esta formulación se identifican explícitamente las variables necesarias para el procesamiento, así como las operaciones de entrada, acumulación y cálculo final del resultado. El algoritmo utiliza un esquema clásico de *contador-acumulador*, donde la variable **suma** permite ir acumulando los valores ingresados y la variable **contador** controla el número de iteraciones necesarias para procesar todos los datos. Una vez completada la lectura de los N valores, el algoritmo calcula el promedio mediante la división entre la suma total acumulada y la cantidad de datos procesados. Esta representación algorítmica facilita la comprensión del procedimiento lógico que posteriormente puede implementarse en cualquier lenguaje de programación, manteniendo correspondencia directa entre las estructuras del algoritmo y las estructuras de control utilizadas en los programas.

```

1      Algoritmo PromedioNValores
2      1.      Var:
3      1.1.      cantidad: entero
4      1.2.      valor: real
5      1.3.      suma: real
6      1.4.      contador: entero
7      1.5.      promedio: real
8
9      2.      Escribir "Digite la cantidad de valores"
10     3.      Leer cantidad
11
12     4.      suma <- 0
13     5.      contador <- 0
14
15     6.      Mientras contador < cantidad hacer
16     6.1.      Escribir "Digite un valor"
```

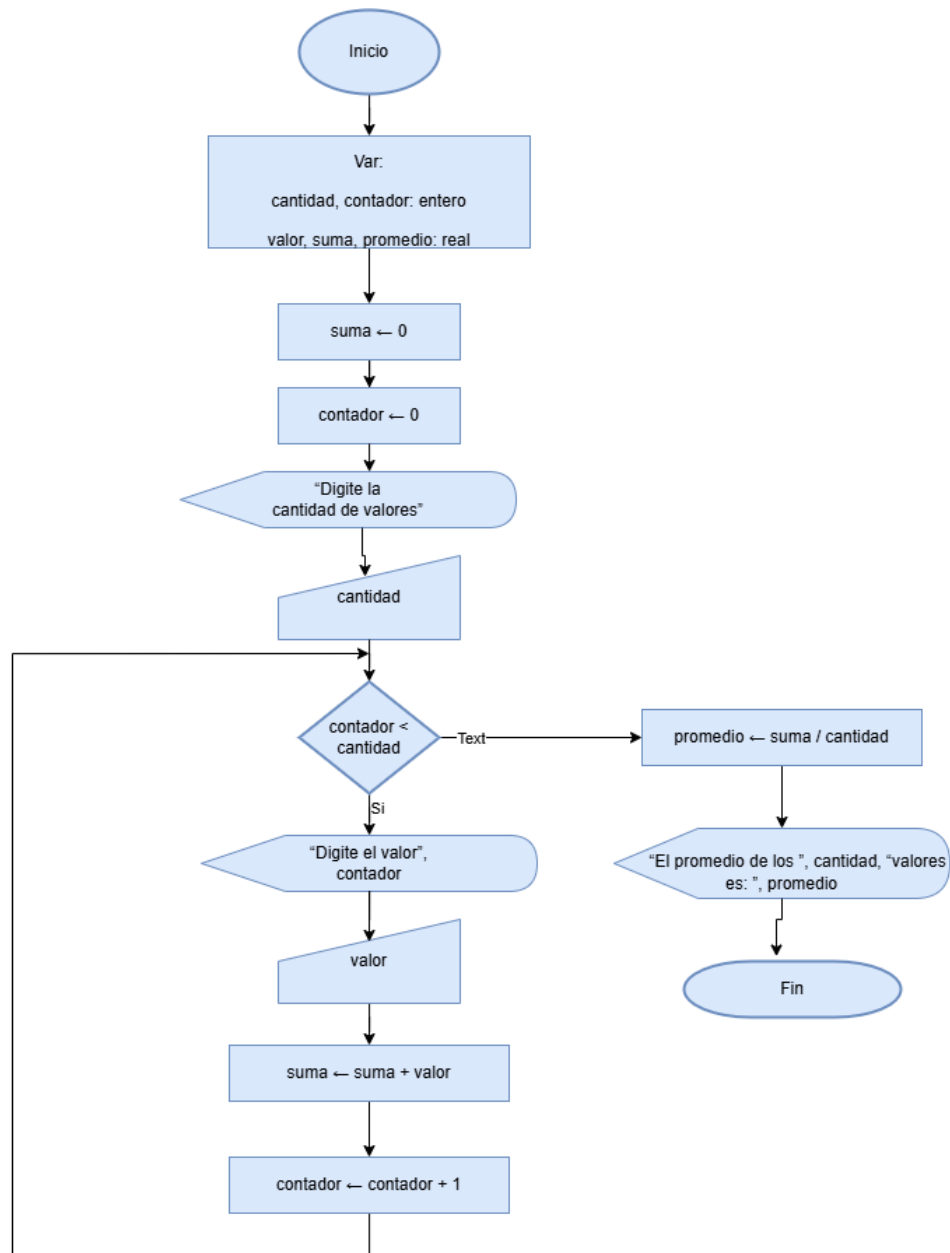


Figura 4.24: Diagrama de flujo para calcular el promedio de N valores ingresados por el usuario mediante un esquema contador-acumulador.

```

17      6.2.      Leer valor
18      6.3.      suma <- suma + valor
19      6.4.      contador <- contador + 1
20      7.        FinMientras
21
22      8.        Si cantidad <= 0 Entonces
23      8.1.      Escribir "No se puede calcular el promedio. No hay datos."
24      8.2.      SiNo
25      8.3.      promedio <- suma / cantidad
26      8.4.      Escribir "El promedio de los ", cantidad, " valores es: ",
    ↪ promedio
27      9.        FinSi
28      FinAlgoritmo

```

Listing 4.39: Cálculo del promedio de N valores

4.9.2. Ejercicio 2: Problema a resolver

Se requiere diseñar un algoritmo que, dados los coeficientes reales a , b y c de una ecuación de segundo grado de la forma $ax^2 + bx + c = 0$, determine si el modelo es efectivamente cuadrático ($a \neq 0$) y, en caso afirmativo, calcule sus raíces según el valor del discriminante $\Delta = b^2 - 4ac$: (i) dos raíces reales distintas si $\Delta > 0$, (ii) una raíz real doble si $\Delta = 0$, o (iii) raíces no reales si $\Delta < 0$ [7], [61].

Pasos para Desarrollar la Solución

Los pasos para desarrollar la solución algorítmica de un problema están encaminados a plasmar en papel el razonamiento que se sigue para resolverlo, separando con claridad la entrada de datos, las decisiones lógicas y los cálculos necesarios. En problemas con fórmulas matemáticas, esta separación es especialmente útil para evitar errores de interpretación y para sostener la correspondencia directa entre el diagrama/algoritmo y las estructuras de control del programa [59], [72].

1. Análisis del problema. Para comprender el problema es necesario plantearse casos puntuales y resolverlos manualmente como instancias concretas del caso general. Esta práctica permite anticipar el comportamiento esperado del algoritmo y distinguir condiciones especiales (por ejemplo, $a = 0$ o $\Delta < 0$) antes de codificar [7], [59].

Ejercicios de análisis manual con valores límite. Antes de diseñar la solución, es recomendable resolver manualmente casos representativos que cubran los distintos escenarios definidos por el discriminante y por la condición $a \neq 0$. A continuación se presentan conjuntos de datos que permiten verificar el comportamiento del algoritmo para la ecuación cuadrática.

▪ Caso 1: Caso normal con dos raíces reales distintas ($\Delta > 0$)

1. Datos de entrada

- $a = 1$, $b = -3$, $c = 2$

2. Resolución manual

$$\Delta = b^2 - 4ac = (-3)^2 - 4(1)(2) = 9 - 8 = 1$$

$$x_1 = \frac{-b + \sqrt{\Delta}}{2a} = \frac{3 + 1}{2} = 2, \quad x_2 = \frac{-b - \sqrt{\Delta}}{2a} = \frac{3 - 1}{2} = 1$$

3. Resultado esperado

$$x_1 = 2, \quad x_2 = 1$$

4. **Observación.** Este caso verifica el flujo principal cuando existen dos soluciones reales y distintas, lo cual exige calcular Δ y aplicar ambas variantes de la fórmula [61].

■ Caso 2: Caso con raíz real doble ($\Delta = 0$)

1. Datos de entrada

- $a = 1, b = 2, c = 1$

2. Resolución manual

$$\Delta = b^2 - 4ac = 2^2 - 4(1)(1) = 4 - 4 = 0$$

$$x = \frac{-b}{2a} = \frac{-2}{2} = -1$$

3. Resultado esperado

$$x = -1 \quad (\text{raíz doble})$$

4. **Observación.** Este caso obliga a distinguir $\Delta = 0$ como condición propia, porque la solución no debe calcular dos raíces distintas, sino reportar una única solución repetida [59].

■ Caso 3: Caso con raíces no reales ($\Delta < 0$)

1. Datos de entrada

- $a = 1, b = 2, c = 5$

2. Resolución manual

$$\Delta = b^2 - 4ac = 2^2 - 4(1)(5) = 4 - 20 = -16$$

3. **Resultado esperado.** No existen raíces reales; la solución debe informar que las raíces son imaginarias (no reales).
4. **Observación.** Este caso verifica la rama donde $\Delta < 0$ y, por tanto, no se debe invocar $\sqrt{\Delta}$ en el dominio real, sino detener el cálculo de raíces reales y reportar la condición [61].

■ Caso 4: Caso límite estructural ($a = 0$; no es ecuación cuadrática)

1. Datos de entrada

- $a = 0, b = 4, c = -8$

2. **Resolución manual.** Si $a = 0$, la expresión deja de ser cuadrática; el modelo se reduce a una ecuación lineal $bx + c = 0$.

3. **Resultado esperado.** La solución debe informar que *no es una ecuación cuadrática*, evitando aplicar fórmulas de segundo grado (divisiones para cero) [7], [59].

4. **Observación.** Este caso evidencia una validación previa indispensable: el coeficiente a determina la categoría del problema y controla la ruta lógica completa.

■ Caso 5: Caso numérico sensible (riesgo de precisión en coma flotante)

1. Datos de entrada

- $a = 1, b = 10^8, c = 1$

2. Resolución manual (ideal matemático)

$$\Delta = (10^8)^2 - 4(1)(1) = 10^{16} - 4$$

3. **Resultado esperado.** Existen dos raíces reales; en implementación con números de punto flotante pueden aparecer errores de redondeo por la representación finita y por cancelación numérica al sumar/restar cantidades muy distintas [11], [12].
4. **Observación.** Este caso no cambia la lógica de la solución, pero advierte que, al programar, las comparaciones con $\Delta = 0$ deben tratarse con cuidado cuando se usan `real/double` [12].

2. Identificación de datos de entrada. Los datos de entrada corresponden a la información que el algoritmo necesita recibir para decidir el tipo de solución y, si procede, calcular las raíces.

- **a (real):** coeficiente cuadrático.
- **b (real):** coeficiente lineal.
- **c (real):** término independiente.

Estos valores se capturan mediante operaciones de lectura precedidas por mensajes claros para el usuario, práctica recomendada en programación introductoria para evitar ambigüedades en la interacción [7], [59].

3. Identificación de datos de salida. Los datos de salida son los resultados comunicados al usuario según el caso:

- **Mensaje informativo:** “No es una ecuación cuadrática” cuando $a = 0$.
- **x1 y x2 (real):** dos raíces reales distintas cuando $\Delta > 0$.
- **x (real):** una raíz real doble cuando $\Delta = 0$.
- **Mensaje informativo:** “La ecuación tiene dos raíces imaginarias” cuando $\Delta < 0$ (en el dominio real).

4. Identificación de cálculos intermedios (procesos intermedios). Los procesos intermedios son operaciones necesarias para clasificar el problema antes de producir la respuesta final.

- **Validación estructural:** evaluar $a = 0$ para decidir si la ecuación es cuadrática.
- **Cálculo del discriminante:** `discriminante <- b^2 - 4*a*c`.

El discriminante actúa como variable de decisión porque determina el conjunto de soluciones reales posibles [61].

5. Identificación de cálculos finales (procesos finales). Los procesos finales son los que producen directamente las raíces cuando existen en $\mathbb{R}$.

- Si $\Delta > 0$:

$$x_1 = \frac{-b + \sqrt{\Delta}}{2a}, \quad x_2 = \frac{-b - \sqrt{\Delta}}{2a}$$

- Si $\Delta = 0$:

$$x = \frac{-b}{2a}$$

En implementación real, estas expresiones dependen de aritmética de punto flotante; por ello, la precisión numérica y los redondeos pueden influir en el resultado mostrado [11], [12].

6. Identificación de condiciones (decisiones). Las decisiones presentes en el problema hacia cada proceso de cálculo y salida, son:

- **Decisión 1:** $a = 0$. Si es verdadera, se reporta que no es ecuación cuadrática.
- **Decisión 2:** `discriminante > 0`. Si es verdadera, se calculan `x1` y `x2`.
- **Decisión 3:** `discriminante = 0`. Si es verdadera, se calcula una raíz real doble.
- **Rama restante:** `discriminante < 0`. Se informa que las raíces son imaginarias (no reales).

Estas bifurcaciones corresponden al patrón de selección múltiple (*if-else if-else*), esencial en la programación estructurada [7], [59].

7. Identificación de repeticiones o iteraciones. El problema no se soluciona usando estructuras repetitivas; su solución es de ejecución secuencial con decisiones. En términos de programación, se implementa con lecturas, asignaciones y una estructura de selección, sin ciclos [59].

8. Condiciones para finalizar el algoritmo. El algoritmo finaliza después de:

- leer a , b y c ;
- decidir si $a = 0$ (caso no cuadrático) o, si $a \neq 0$, calcular **discriminante**;
- ejecutar exactamente una de las salidas: reporte de no-cuadrática, dos raíces reales, raíz doble, o mensaje de raíces imaginarias.

En consecuencia, el término del proceso depende de completar la cadena de decisiones, no de una condición iterativa [7].

Propuesta de Solución

La Figura 4.25 describe una solución algorítmica estructurada para analizar y resolver una ecuación de segundo grado mediante decisiones basadas en a y en el discriminante. El diseño separa (i) la validación del modelo ($a \neq 0$), (ii) el cálculo de una variable de control (**discriminante**), y (iii) la selección de la fórmula adecuada para reportar el tipo de solución. Esta organización favorece la verificación manual del flujo y la traducción directa a código, manteniendo correspondencia entre cada rombo del diagrama y una condición lógica del programa [59], [72].

El Algoritmo 4.40 presenta una propuesta de solución estructurada coherente con el flujo de la Figura 4.25. En esta formulación se declaran explícitamente las variables, se valida primero la naturaleza cuadrática del modelo mediante la condición $a \neq 0$, y se utiliza el discriminante como criterio para seleccionar el cálculo final. Esta representación facilita a lectores principiantes relacionar cada decisión con una rama concreta del algoritmo y, posteriormente, implementar la solución en cualquier lenguaje manteniendo la estructura de selección de manera clara [7], [61].

```

1      Algoritmo EcuacionCuadratica
2      1.      Var:
3      1.1.      a, b, c: real
4      1.2.      discriminante: real
5      1.3.      x1, x2: real
6
7      2.      Escribir "Ingrese el valor de a:"
8      3.      Leer a
9      4.      Escribir "Ingrese el valor de b:"
10     5.      Leer b
11     6.      Escribir "Ingrese el valor de c:"
12     7.      Leer c
13
14     8.      Si a = 0 Entonces
15     8.1.      Escribir "No es una ecuación cuadrática"
16     8.2.      SiNo
17     8.3.      discriminante <- b^2 - 4*a*c

```

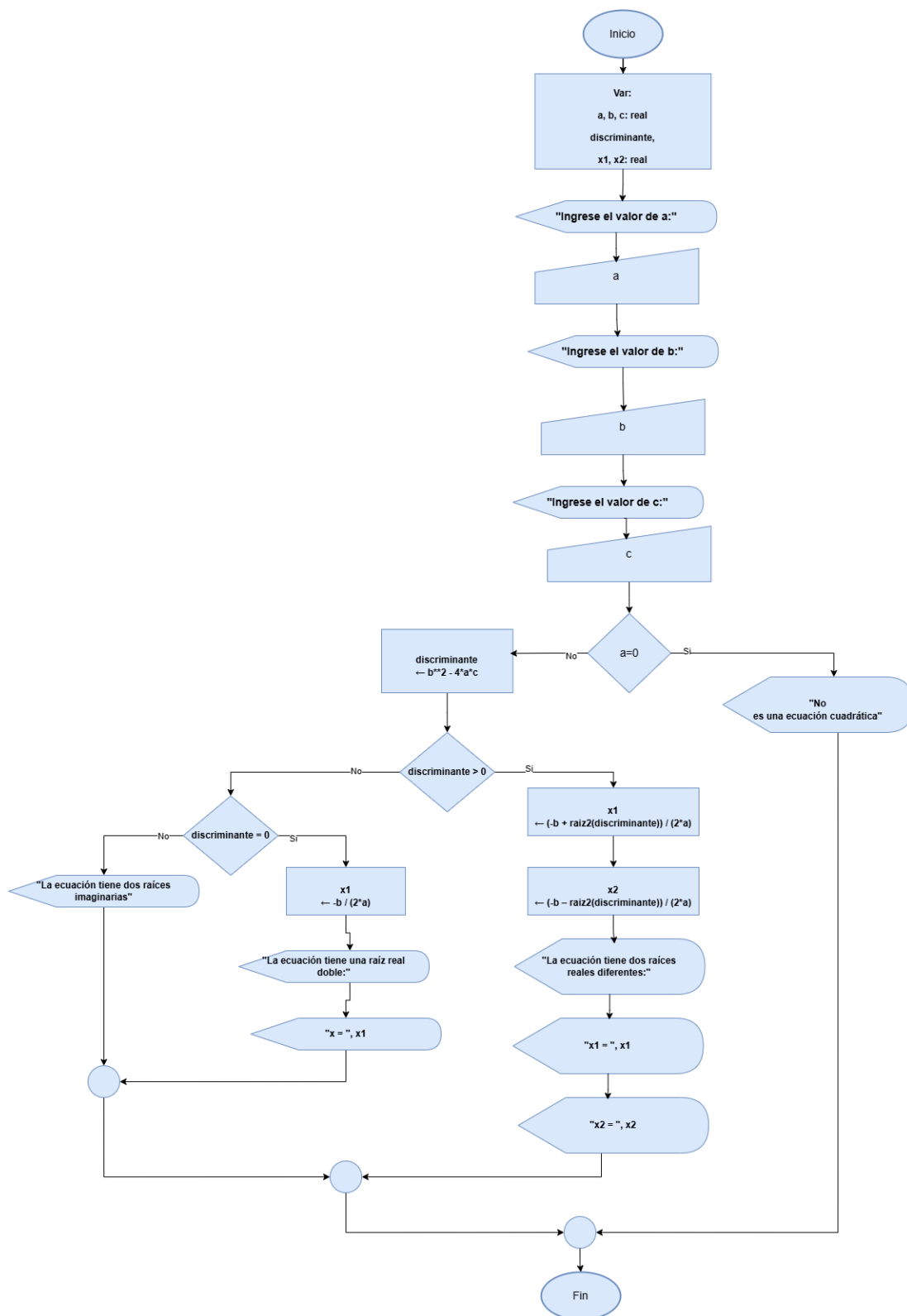


Figura 4.25: Diagrama de flujo para determinar y calcular las raíces de una ecuación cuadrática $ax^2 + bx + c = 0$ según el discriminante $\Delta = b^2 - 4ac$.

```

18
19      8.4.      Si discriminante > 0 Entonces
20      8.4.1.      x1 <- (-b + raiz2(discriminante)) / (2*a)
21      8.4.2.      x2 <- (-b - raiz2(discriminante)) / (2*a)
22      8.4.3.      Escribir "La ecuación tiene dos raíces reales
    ↪ diferentes:"
23      8.4.4.      Escribir "x1 = ", x1
24      8.4.5.      Escribir "x2 = ", x2
25      8.5.      SiNo
26      8.6.      Si discriminante = 0 Entonces
27      8.6.1.      x1 <- -b / (2*a)
28      8.6.2.      Escribir "La ecuación tiene una raíz real doble:"
29      8.6.3.      Escribir "x = ", x1
30      8.7.      SiNo
31      8.7.1.      Escribir "La ecuación tiene dos raíces imaginarias"
32      8.8.      FinSi
33      8.9.      FinSi
34      9.      FinSi
35      FinAlgoritmo

```

Listing 4.40: Determinación y cálculo de raíces de una ecuación cuadrática

4.9.3. Ejercicio 3: Problema a resolver

Se requiere diseñar un algoritmo que lea una *palabra* (cadena de caracteres) ingresada por el usuario, construya su versión invertida y determine si dicha palabra es *palíndroma*; es decir, si se lee igual de izquierda a derecha que de derecha a izquierda (por ejemplo, *reconocer* o *radar*) [7], [61].

Pasos para Desarrollar la Solución

Los pasos para desarrollar la solución algorítmica de un problema están orientados a representar con precisión el razonamiento necesario para resolverlo, distinguiendo con claridad: (i) qué datos se reciben, (ii) qué transformaciones se aplican, (iii) qué condiciones controlan el flujo, y (iv) cómo se produce la salida. Esta separación resulta especialmente útil en programación introductoria, porque permite al lector asociar cada decisión y cada repetición con una estructura de control que luego se implementará en un lenguaje real [7], [59].

1. Análisis del problema. Para comprender el problema es necesario trabajar con instancias concretas: elegir palabras específicas, invertir las manualmente y verificar si coinciden con el original. Esta práctica anticipa el comportamiento esperado de la solución y ayuda a detectar condiciones especiales antes de programar (por ejemplo, cadenas de longitud 1 o palabras con letras repetidas) [7], [59].

Ejercicios de análisis manual con valores límite. Antes de diseñar la solución, es recomendable resolver manualmente casos representativos que cubran longitudes pequeñas, longitudes pares e impares, y situaciones frontera donde el resultado es evidente. A continuación se presentan conjuntos de datos que permiten verificar el comportamiento del algoritmo para la detección de palíndromos.

■ Caso 1: Caso normal palíndromo (longitud impar)**1. Datos de entrada**

- palabra = radar

2. Resolución manual

invertida = radar

3. Resultado esperado

“es palabra palíndroma”

- 4. Observación.** Este caso verifica la ruta donde la palabra coincide exactamente con su inversión [61].

■ Caso 2: Caso normal no palíndromo (longitud impar)**1. Datos de entrada**

- palabra = casa

2. Resolución manual

invertida = asac

3. Resultado esperado

“NO es palabra palíndroma”

- 4. Observación.** Este caso valida la comparación final cuando las cadenas no coinciden.

■ Caso 3: Caso palíndromo (longitud par)**1. Datos de entrada**

- palabra = anna

2. Resolución manual

invertida = anna

3. Resultado esperado

“es palabra palíndroma”

- 4. Observación.** Este caso confirma que la solución no depende de que la longitud sea impar; depende de la igualdad palabra-inversión.

■ Caso 4: Caso límite mínimo (longitud 1)**1. Datos de entrada**

- palabra = a

2. Resolución manual

invertida = a

3. Resultado esperado

“es palabra palíndroma”

- 4. Observación.** Una palabra de un solo carácter siempre coincide con su inversión; este caso verifica la frontera inferior del dominio [7].

■ Caso 5: Caso con caracteres repetidos (no palíndromo)

1. Datos de entrada

- `palabra = aabb`

2. Resolución manual

`invertida = bbaa`

3. Resultado esperado

“NO es palabra palíndroma”

4. Observación. Este caso evita conclusiones incorrectas basadas únicamente en repeticiones de caracteres.

2. Identificación de datos de entrada. Los datos de entrada corresponden a la información que el algoritmo necesita recibir para ejecutar la verificación.

- **palabra (cadena):** texto ingresado por el usuario que se desea evaluar.

En programación introductoria, es recomendable que toda lectura sea precedida por un mensaje claro que indique al usuario qué se espera que ingrese [7], [59].

3. Identificación de datos de salida. Los datos de salida son los resultados comunicados al usuario:

- **Mensaje informativo (palíndroma):** *“palabra es palabra palíndroma”*.
- **Mensaje informativo (no palíndroma):** *“palabra NO es palabra palíndroma”*.

4. Identificación de cálculos intermedios (procesos intermedios). Los procesos intermedios son operaciones necesarias para producir el criterio de comparación final:

- **Cálculo de longitud:** `longPal <- longitud(palabra)`.
- **Inicialización de acumulador de cadena:** `invertida <- ""`.
- **Construcción incremental de la inversión:** concatenar, en orden inverso, cada carácter de `palabra` hacia `invertida`.

Este esquema corresponde a un patrón básico de procesamiento de secuencias: recorrer una estructura y construir un resultado acumulado, idea presente en el estudio de algoritmos y en la programación estructurada [53], [61].

5. Identificación de cálculos finales (procesos finales). El proceso final que permite decidir el resultado es:

- **Comparación final:** verificar si `palabra = invertida`.

La igualdad entre la entrada y su inversión define el criterio operativo de palíndromo en el contexto del problema [61].

6. Identificación de condiciones (decisiones). Las decisiones que controlan la salida del programa son:

- **Decisión principal:** `palabra = invertida`. Si es verdadera, se reporta que es palíndroma; de lo contrario, se reporta que no lo es.

Esta bifurcación corresponde al patrón *if-else*, esencial para expresar alternativas mutuamente excluyentes en programación [7], [59].

7. Identificación de repeticiones o iteraciones. El problema requiere una repetición para construir la inversión de la palabra:

- **Iteración decreciente:** recorrer el índice `i` desde `longPal` hasta 1, agregando en cada paso el carácter `palabra[i]` a `invertida`.

Este control por índice corresponde a un ciclo tipo `for` decreciente (o equivalente), y su corrección se razona mediante el hecho de que, en cada vuelta, `invertida` conserva como invariante ser el prefijo invertido ya construido con los caracteres procesados [53].

8. Condiciones para finalizar el algoritmo. El algoritmo finaliza después de:

- leer `palabra`;
- construir completamente `invertida` (cuando la iteración termina al completar el recorrido de índices);
- evaluar la condición `palabra = invertida` y mostrar exactamente uno de los dos mensajes de salida.

En consecuencia, el término del proceso depende de completar el recorrido de la cadena y la comparación final, no de una condición externa adicional [7].

Propuesta de Solución

La Figura 4.26 describe una solución algorítmica estructurada para determinar si una palabra es palíndroma mediante (i) la construcción de su inversión por recorrido decreciente y (ii) una comparación final entre la palabra original y la invertida. Esta organización favorece la verificación manual del flujo y la traducción directa a código, manteniendo correspondencia entre el recorrido por índices, la operación de concatenación y la estructura de selección que decide el mensaje mostrado al usuario [59], [72].

El Algoritmo 4.41 presenta una propuesta de solución estructurada coherente con el procedimiento descrito en la Figura 4.26. En esta formulación se declaran explícitamente las variables, se construye la inversión de la palabra recorriendo sus caracteres en orden inverso, y se utiliza una condición de igualdad para decidir el mensaje final. Esta representación facilita a lectores principiantes relacionar la repetición con la construcción de un resultado acumulado y la decisión con la clasificación final del caso [7], [61].

```

1      Algoritmo Palindromo
2      1.      Var:
3      1.1.      palabra, invertida: cadena
4      1.2.      i, longPal: entero
5
6      2.      Escribir "Ingresa una palabra"
7      3.      Leer palabra
8
9      4.      longPal <- longitud(palabra)
10     5.      invertida <- ""
11
12     6.      Para i desde longPal hasta 1 con paso -1 hacer
13     6.1.      invertida <- invertida + subcadena(palabra, i, 1)
14     7.      FinPara
15
16     8.      Si palabra = invertida Entonces
17     8.1.      Escribir palabra, " es palabra palíndroma"
18     8.2.      SiNo
19     8.3.      Escribir palabra, " NO es palabra palíndroma"
20     9.      FinSi
21     FinAlgoritmo

```

Listing 4.41: Determinación de palabra palíndroma mediante inversión de cadena

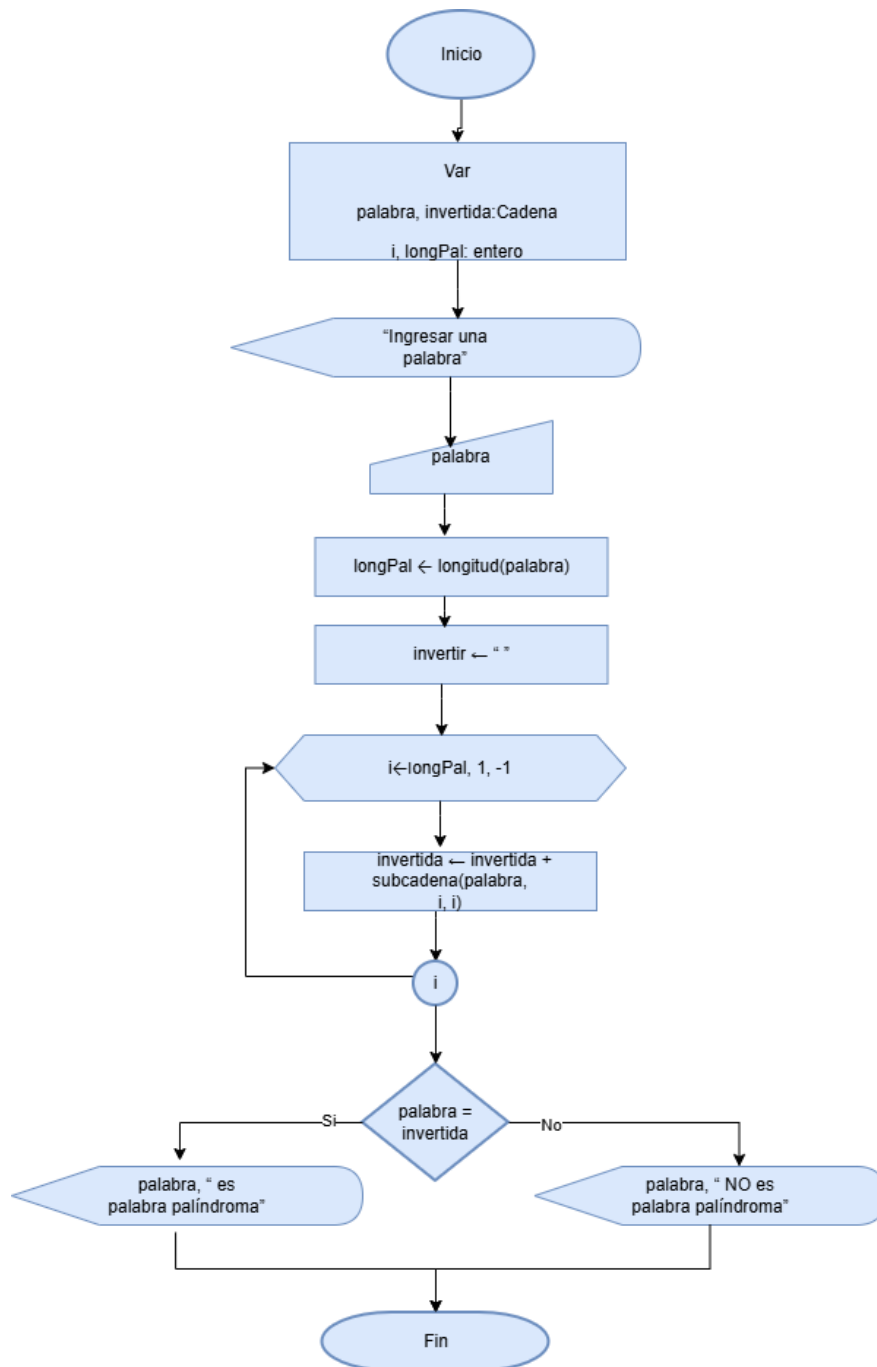


Figura 4.26: Solución algorítmica para determinar si una palabra es palíndroma mediante inversión de la cadena y comparación final.

4.9.4. Ejercicio 4: Cálculo de la Edad Exacta (Años, Meses y Días)

Se requiere diseñar un algoritmo que calcule la edad exacta de una persona en **años, meses y días**, a partir de su fecha de nacimiento ingresada por el usuario y de la fecha actual obtenida automáticamente por el sistema. El resultado debe expresarse como un conjunto ordenado (**edadAnio**, **edadMes**, **edadDia**) y debe garantizar consistencia lógica cuando el día o el mes actuales sean menores que los valores correspondientes de nacimiento, aplicando un mecanismo de *préstamo* (*borrowing*) entre unidades de tiempo [35], [73].

Pasos para Desarrollar la Solución

Los pasos para desarrollar la solución algorítmica de un problema, están encaminados a plasmar en papel lo que el programador principiante o cualquier persona hace para resolver un problema.

1. Análisis del problema. Para comprender el problema es necesario plantearse algunos casos puntuales y resolverlos de manera aislada, como si fuesen únicos. En este ejercicio, el algoritmo parte de una diferencia directa entre fecha actual y fecha de nacimiento, y luego corrige los valores negativos mediante *préstamos*: si los días resultan negativos, se suman días prestados y se decrementa un mes; si los meses resultan negativos, se suman 12 meses y se decrementa un año. Este patrón reduce errores conceptuales frecuentes en principiantes al separar el cálculo inicial de las correcciones por consistencia [74], [75], [76].

Ejercicios de análisis manual con valores límite. Antes de diseñar el algoritmo, es recomendable resolver manualmente algunos casos representativos del problema. A continuación se presentan conjuntos de datos que permiten verificar el comportamiento de la resta de fechas y del mecanismo de préstamo. Para mantener coherencia con el diagrama, se asume el ajuste de días con **30** como aproximación (préstamo fijo), criterio útil para aprendizaje inicial; en implementación profesional puede refinarse usando los días reales del mes [23], [24].

■ Caso 1: Caso normal (sin préstamos)

1. Datos de entrada

- fecha actual: 2026/03/20
- fecha nacimiento: 2000/01/10

2. Resolución manual

$$edadAnio = 2026 - 2000 = 26, \quad edadMes = 3 - 1 = 2, \quad edadDia = 20 - 10 = 10$$

3. Resultado esperado

$$(edadAnio, edadMes, edadDia) = (26, 2, 10)$$

4. **Observación.** Confirma el funcionamiento básico cuando día y mes actuales son mayores o iguales a los de nacimiento.

■ Caso 2: Préstamo de días (día actual menor)

1. Datos de entrada

- fecha actual: 2026/03/05
- fecha nacimiento: 2000/01/10

2. Resolución manual

$$edadAnio = 26, \quad edadMes = 2, \quad edadDia = 5 - 10 = -5$$

Como **edadDia** < 0, se aplica préstamo:

$$edadDia = -5 + 30 = 25, \quad edadMes = 2 - 1 = 1$$

3. Resultado esperado

$$(edadAnio, edadMes, edadDia) = (26, 1, 25)$$

4. **Observación.** Verifica la corrección de días negativos mediante decremento de mes, patrón típico de errores si no se formaliza [74].

■ Caso 3: Préstamo de meses (mes actual menor)

1. Datos de entrada

- fecha actual: 2026/02/20
- fecha nacimiento: 2000/04/10

2. Resolución manual

$$edadAnio = 26, \quad edadMes = 2 - 4 = -2, \quad edadDia = 10$$

Como $edadMes < 0$, se aplica préstamo:

$$edadMes = -2 + 12 = 10, \quad edadAnio = 26 - 1 = 25$$

3. Resultado esperado

$$(edadAnio, edadMes, edadDia) = (25, 10, 10)$$

4. **Observación.** Confirma que la corrección de meses negativos se resuelve sumando 12 y decrementando años.

■ Caso 4: Doble corrección (días y luego meses negativos)

1. Datos de entrada

- fecha actual: 2026/02/05
- fecha nacimiento: 2000/04/10

2. Resolución manual Diferencia inicial:

$$edadAnio = 26, \quad edadMes = 2 - 4 = -2, \quad edadDia = 5 - 10 = -5$$

Primero se corrigen días:

$$edadDia = -5 + 30 = 25, \quad edadMes = -2 - 1 = -3$$

Luego se corrigen meses:

$$edadMes = -3 + 12 = 9, \quad edadAnio = 26 - 1 = 25$$

3. Resultado esperado

$$(edadAnio, edadMes, edadDia) = (25, 9, 25)$$

4. **Observación.** Este caso prueba el orden correcto de correcciones: primero días, luego meses, lo cual evita resultados inconsistentes [35].

■ Caso 5: Cumpleaños hoy (valores borde)

1. Datos de entrada

- fecha actual: 2026/03/10

- fecha nacimiento: 2000/03/10

2. Resolución manual

$$\text{edadAnio} = 26, \quad \text{edadMes} = 0, \quad \text{edadDia} = 0$$

3. Resultado esperado

$$(\text{edadAnio}, \text{edadMes}, \text{edadDia}) = (26, 0, 0)$$

4. **Observación.** Verifica consistencia cuando mes y día coinciden exactamente.

■ Caso 6: Caso inválido (nacimiento en el futuro)

1. Datos de entrada

- fecha actual: 2026/03/10
- fecha nacimiento: 2027/01/01

2. **Resolución:** La fecha de nacimiento no puede ser posterior a la fecha actual.

3. **Resultado esperado.** El algoritmo debe detectar la situación y mostrar un mensaje de error, por ejemplo:

- *“La fecha de nacimiento no puede ser mayor que la fecha actual.”*
- **Observación.** Este caso obliga a incluir una validación de dominio, práctica esencial para robustez [23], [24].

2. Identificación de datos de entrada.

Los datos de entrada corresponden a la información que el algoritmo necesita recibir desde el exterior para poder ejecutar el cálculo:

- **anioNac (entero):** año de nacimiento ingresado por el usuario.
- **mesNac (entero):** mes de nacimiento ingresado por el usuario (1..12).
- **diaNac (entero):** día de nacimiento ingresado por el usuario (1..31, según el mes).

Adicionalmente, el algoritmo obtiene automáticamente la fecha actual:

- **anioHoy, mesHoy, diaHoy (enteros):** componentes de la fecha del sistema. Esta separación en componentes facilita el cálculo incremental y la detección de casos borde [35], [73].

3. **Identificación de datos de salida.** Los datos de salida son los resultados que el algoritmo entrega al usuario:

- **edadAnio (entero):** cantidad de años cumplidos.
- **edadMes (entero):** cantidad de meses adicionales.
- **edadDia (entero):** cantidad de días adicionales.
- **Mensaje informativo:** “Su edad es: **edadAnio** años, **edadMes** meses, **edadDia** días”.

Este formato combina datos numéricos con texto explicativo, práctica recomendada para mejorar interpretabilidad en programas introductorios y reducir ambigüedad en la salida [7], [59].

4. **Identificación de cálculos intermedios (procesos intermedios).** Los procesos intermedios son operaciones necesarias para llegar al resultado, pero que no constituyen todavía la respuesta final:

- **Extracción de fecha actual:** asignaciones **anioHoy**, **mesHoy**, **diaHoy** a partir de la fecha del sistema.
- **Diferencias iniciales:** $\text{edadAnio} \leftarrow \text{anioHoy} - \text{anioNac}$, $\text{edadMes} \leftarrow \text{mesHoy} - \text{mesNac}$, $\text{edadDia} \leftarrow \text{diaHoy} - \text{diaNac}$.
- **Corrección por préstamo de días:** si $\text{edadDia} < 0$, entonces $\text{edadDia} \leftarrow \text{edadDia} + 30$ y $\text{edadMes} \leftarrow \text{edadMes} - 1$.

- **Corrección por préstamo de meses:** si $\text{edadMes} < 0$, entonces $\text{edadMes} \leftarrow \text{edadMes} + 12$ y $\text{edadAnio} \leftarrow \text{edadAnio} - 1$.

El algoritmo separa una resta directa seguida de correcciones, lo cual disminuye la carga cognitiva inicial al convertir un problema de fechas en una secuencia de operaciones aritméticas controladas por condiciones [74], [76].

5. Identificación de cálculos finales (procesos finales). Los procesos finales producen directamente el resultado reportado al usuario:

- **Formateo de salida:** impresión de edadAnio , edadMes y edadDia con texto descriptivo.

Este paso final se ejecuta únicamente si las validaciones de dominio se cumplen, lo cual es consistente con prácticas de ingeniería de software orientadas a confiabilidad [23], [24].

6. Identificación de condiciones (decisiones). Las decisiones determinan correcciones y validaciones:

- **Condición 1 (préstamo de días):** $\text{edadDia} < 0$.
- **Condición 2 (préstamo de meses):** $\text{edadMes} < 0$.
- **Condición 3 (validez temporal):** la fecha de nacimiento no debe ser posterior a la fecha actual.

Estas condiciones formalizan reglas de consistencia que previenen resultados imposibles y facilitan el razonamiento y depuración del algoritmo [74], [75].

7. Identificación de repeticiones o iteraciones. El diagrama no contiene estructuras repetitivas: el cálculo se realiza en una única pasada con correcciones condicionales. Esta ausencia de iteraciones lo convierte en un ejercicio ideal para consolidar tipos, variables, expresiones y decisiones antes de introducir ciclos [7], [59].

8. Condiciones para finalizar el algoritmo. El algoritmo finaliza cuando:

- se capturan los datos de nacimiento;
- se obtiene la fecha actual;
- se calculan diferencias y se aplican correcciones (si corresponden);
- se presenta el resultado o un mensaje de error si la fecha es inválida.

En consecuencia, el término del proceso se encuentra ligado al cierre del bloque de cálculo y a la impresión del resultado, no a una condición de repetición [35].

Propuesta de Solución

La Figura 4.27 describe una solución algorítmica estructurada para calcular la edad exacta. El diseño separa con claridad (i) la captura de la fecha de nacimiento, (ii) la obtención de la fecha actual desde el sistema, (iii) el cálculo directo por diferencias y (iv) las correcciones por préstamos de días y de meses. Esta organización favorece la verificación manual del flujo y la posterior traducción a un lenguaje de programación, manteniendo correspondencia directa entre bloques del diagrama y sentencias de asignación/decisión [59], [72].

El Algoritmo 4.42 presenta una propuesta de solución estructurada para calcular la edad exacta. En esta formulación se identifican explícitamente las variables necesarias para el procesamiento, las operaciones de entrada y extracción de fecha del sistema, así como las condiciones que corrigen los resultados negativos mediante préstamos. La estrategia de préstamos convierte la resta de fechas en una secuencia de ajustes aritméticos gobernados por decisiones, lo cual reduce errores de interpretación y facilita el razonamiento en estudiantes que se inician en programación [74], [75], [76].

```

1      Algoritmo EdadExactaPrestamos
2      1.      Var:
3      1.1.      anioNac, mesNac, diaNac: entero
4      1.2.      anioHoy, mesHoy, diaHoy: entero

```

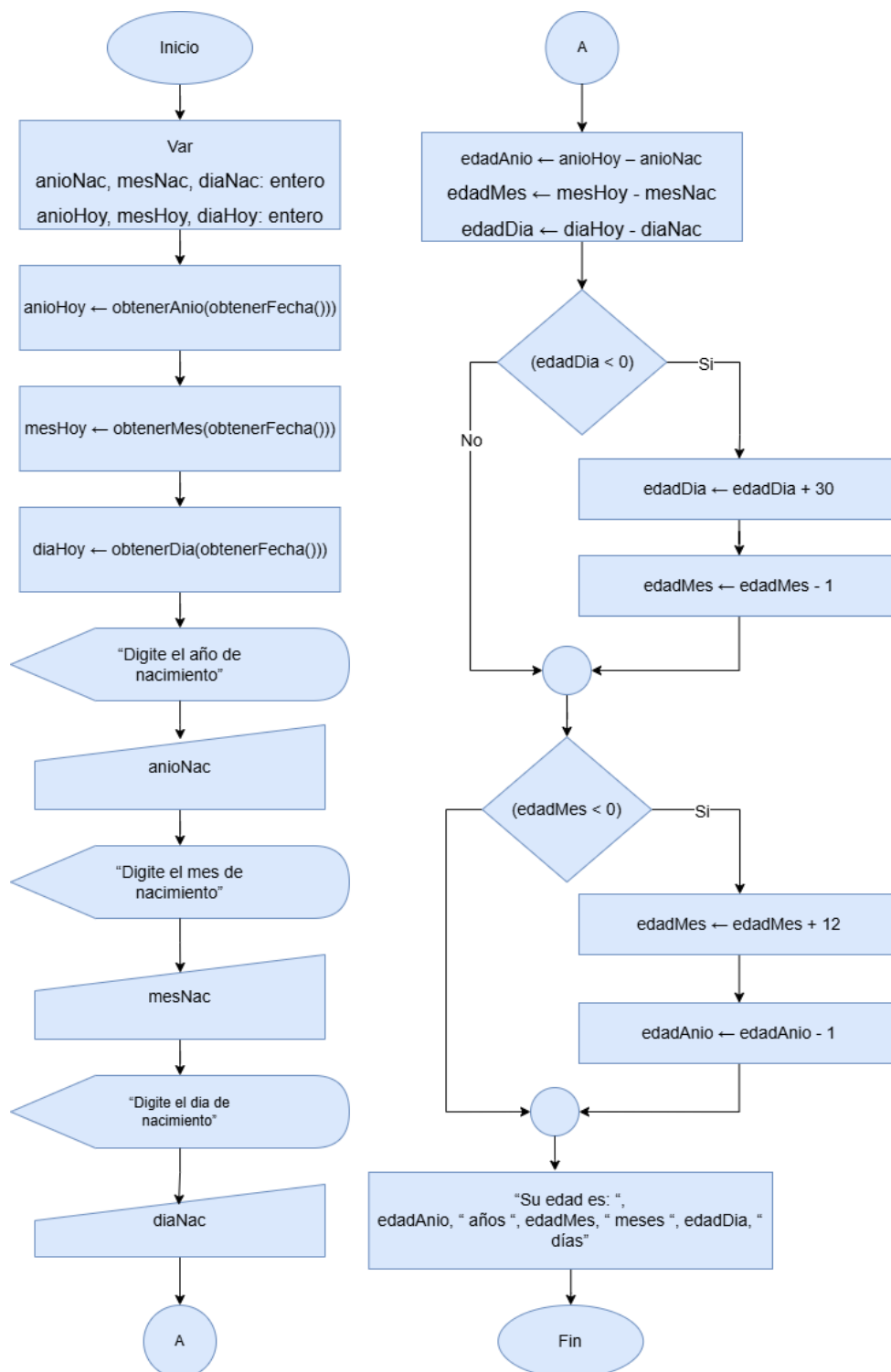



Figura 4.27: Diagrama de flujo para calcular la edad exacta (años, meses y días) aplicando préstamos cuando las diferencias de día o mes resultan negativas.

```

5      1.3.      edadAnio, edadMes, edadDia: entero
6
7      2.      anioHoy <- obtenerAnio(obtenerFecha())
8      3.      mesHoy  <- obtenerMes(obtenerFecha())
9      4.      diaHoy  <- obtenerDia(obtenerFecha())
10
11     5.      Escribir "Digite el anio de nacimiento"
12     6.      Leer anioNac
13     7.      Escribir "Digite el mes de nacimiento"
14     8.      Leer mesNac
15     9.      Escribir "Digite el dia de nacimiento"
16    10.      Leer diaNac
17
18    11.      Si (anioNac > anioHoy OR (anioNac = anioHoy AND mesNac > mesHoy)
    ↪ OR (anioNac = anioHoy AND mesNac = mesHoy AND diaNac > diaHoy))
    ↪ Entonces
19    11.1.     Escribir "La fecha de nacimiento no puede ser mayor que la
    ↪ fecha actual."
20    11.2.     FinAlgoritmo
21    12.      FinSi
22
23    13.      edadAnio <- anioHoy - anioNac
24    14.      edadMes  <- mesHoy  - mesNac
25    15.      edadDia  <- diaHoy  - diaNac
26
27    16.      Si edadDia < 0 Entonces
28    16.1.      edadDia <- edadDia + 30
29    16.2.      edadMes <- edadMes - 1
30    17.      FinSi
31
32    18.      Si edadMes < 0 Entonces
33    18.1.      edadMes <- edadMes + 12
34    18.2.      edadAnio <- edadAnio - 1
35    19.      FinSi
36
37    20.      Escribir "Su edad es: ", edadAnio, " años ", edadMes, " meses ",
    ↪ edadDia, " días"

```

Listing 4.42: Cálculo de la edad exacta con préstamos de días y meses

En el Algoritmo del Listado de código 4.42 se presenta la misma solución, incorporando además un *refuerzo de robustez*: la validación de consistencia temporal de la fecha de nacimiento respecto de la fecha actual. Esta verificación, implementada en los pasos 11 al 12, garantiza que el cálculo solo se ejecute cuando los datos pertenecen al dominio permitido (nacimiento no posterior a la fecha del sistema), evitando resultados incongruentes y fortaleciendo la confiabilidad del procedimiento [23], [24].

4.9.5. Ejercicio 5: Determinación del Valor Mayor entre Tres Números

Se requiere diseñar un algoritmo que permita determinar cuál es el **mayor** de tres valores numéricos ingresados por el usuario. El procedimiento debe solicitar tres números reales y, mediante comparaciones lógicas, identificar cuál de ellos posee el valor más alto. El resultado final debe mostrarse al usuario mediante un mensaje informativo que indique claramente el valor mayor encontrado [35], [73].

Pasos para Desarrollar la Solución

Los pasos para desarrollar la solución algorítmica de un problema están orientados a representar de manera ordenada el razonamiento que una persona seguiría al resolver la situación planteada.

1. Análisis del problema. Para comprender adecuadamente el problema es conveniente analizar distintos casos concretos. En este ejercicio, el objetivo consiste en comparar tres valores y determinar cuál de ellos es mayor. Este tipo de problema pertenece a la categoría de *selección o decisión*, donde el algoritmo debe evaluar condiciones lógicas para determinar cuál camino de ejecución seguir [74], [75], [76].

Ejercicios de análisis manual con valores límite. Antes de diseñar el algoritmo es recomendable analizar manualmente algunos casos representativos que permitan verificar el comportamiento esperado del procedimiento.

■ **Caso 1: Caso normal con valores distintos**

1. **Datos de entrada**

- $a = 8$
- $b = 5$
- $c = 3$

2. **Resolución manual**

$$8 > 5 \quad \text{y} \quad 8 > 3$$

3. **Resultado esperado**

$$\text{mayor} = 8$$

4. **Observación.** Este caso permite verificar el funcionamiento básico del algoritmo cuando uno de los valores es claramente mayor que los demás.

■ **Caso 2: El segundo valor es el mayor**

1. **Datos de entrada**

- $a = 4$
- $b = 9$
- $c = 6$

2. **Resolución manual**

$$9 > 4 \quad \text{y} \quad 9 > 6$$

3. **Resultado esperado**

$$\text{mayor} = 9$$

4. **Observación.** Permite comprobar que el algoritmo puede identificar correctamente cuando el segundo valor es el mayor de los tres.

■ **Caso 3: El tercer valor es el mayor**

1. **Datos de entrada**

- $a = 7$
- $b = 2$
- $c = 10$

2. **Resolución manual**

$$10 > 7 \quad \text{y} \quad 10 > 2$$

3. **Resultado esperado**

$$\text{mayor} = 10$$

4. **Observación.** Este caso confirma que el algoritmo debe evaluar también la posibilidad de que el tercer valor sea el mayor.

■ **Caso 4: Dos valores iguales y mayores**

1. **Datos de entrada**

- $a = 12$
- $b = 12$
- $c = 5$

2. **Resolución manual**

$$12 \geq 12 \quad \text{y} \quad 12 > 5$$

3. **Resultado esperado**

$$\text{mayor} = 12$$

4. **Observación.** Este caso permite comprobar que el algoritmo funciona correctamente incluso cuando dos valores poseen el mismo valor máximo.

■ **Caso 5: Todos los valores iguales**

1. Datos de entrada

- $a = 6$
- $b = 6$
- $c = 6$

2. Resolución manual

$$a = b = c$$

3. Resultado esperado

$$\text{mayor} = 6$$

4. **Observación.** Este caso límite permite verificar que el algoritmo sigue funcionando correctamente incluso cuando los tres valores son iguales.

2. Identificación de datos de entrada. Los datos de entrada corresponden a los valores que el algoritmo necesita recibir para poder realizar el proceso de comparación.

- **a (real):** primer valor numérico ingresado por el usuario.
- **b (real):** segundo valor numérico ingresado por el usuario.
- **c (real):** tercer valor numérico ingresado por el usuario.

Cada lectura de datos debe estar precedida por un mensaje que indique claramente al usuario qué información debe ingresar. Esta práctica mejora la interacción entre el usuario y el programa y reduce posibles errores de entrada [7], [59].

3. Identificación de datos de salida. El algoritmo produce un único resultado principal:

- **mayor (real):** el valor más grande entre los tres números ingresados.
- **Mensaje informativo:** “El valor mayor es: `mayor`”.

Este resultado permite comunicar de forma clara el valor que satisface la condición de mayor magnitud entre los datos analizados.

4. Identificación de cálculos intermedios (procesos intermedios). En este problema no se realizan cálculos aritméticos complejos; el procesamiento consiste principalmente en evaluaciones lógicas mediante operadores de comparación.

- **Comparación entre valores:** se evalúan expresiones lógicas como $a \geq b$ y $a \geq c$.
- **Asignación del valor mayor:** dependiendo del resultado de las comparaciones, se asigna a la variable `mayor` el valor correspondiente.

Estas operaciones constituyen ejemplos clásicos de algoritmos basados en *estructuras de decisión*, donde el flujo del programa depende del resultado de expresiones lógicas [35].

5. Identificación de cálculos finales (procesos finales). El proceso final consiste en mostrar el resultado obtenido al usuario.

- **Salida del resultado:** impresión del valor almacenado en la variable `mayor`.

Este paso final permite comunicar el resultado del análisis comparativo realizado por el algoritmo.

6. Identificación de condiciones (decisiones). Las decisiones constituyen el núcleo del algoritmo. En este caso se evalúan comparaciones entre los valores ingresados.

- **Condición principal:** verificar si $a \geq b$ y $a \geq c$. Si ambas condiciones se cumplen, entonces `a` es el mayor.
- **Condición secundaria:** si la condición anterior no se cumple, se evalúa si $b \geq a$ y $b \geq c$. Si es verdadera, entonces `b` es el mayor.

- **Caso restante:** si ninguna de las condiciones anteriores se cumple, entonces el mayor corresponde a c.

Estas comparaciones permiten cubrir todas las combinaciones posibles entre los tres valores.

7. Identificación de repeticiones o iteraciones. Este problema no requiere estructuras repetitivas. El algoritmo se ejecuta de manera secuencial, solicitando tres valores y realizando un conjunto finito de comparaciones para determinar el resultado.

8. Condiciones para finalizar el algoritmo. El algoritmo finaliza cuando:

- se han leído los tres valores de entrada;
- se han evaluado las condiciones de comparación;
- se ha determinado el valor mayor;
- se muestra el resultado al usuario.

Propuesta de Solución

La Figura 4.28 presenta una propuesta de solución algorítmica para determinar el valor mayor entre tres números ingresados por el usuario. El procedimiento inicia con la declaración de las variables necesarias y la captura de los tres valores de entrada. Posteriormente se realizan comparaciones lógicas entre los valores para identificar cuál de ellos es el mayor. Finalmente, el algoritmo muestra el resultado obtenido mediante un mensaje informativo.

El Algoritmo 4.43 presenta una posible implementación estructurada para resolver el problema. En esta formulación se identifican explícitamente las variables utilizadas y las condiciones necesarias para determinar cuál de los tres valores es el mayor.

```

1      Algoritmo MayorTresValores
2      1.      Var:
3      1.1.      a, b, c: real
4      1.2.      mayor: real
5
6      2.      Escribir "Ingrese el primer valor"
7      3.      Leer a
8
9      4.      Escribir "Ingrese el segundo valor"
10     5.      Leer b
11
12     6.      Escribir "Ingrese el tercer valor"
13     7.      Leer c
14
15     8.      Si a >= b Y a >= c Entonces
16     8.1.      mayor <- a
17     9.      SiNo
18     9.1.      Si b >= a Y b >= c Entonces
19     9.2.      mayor <- b
20     9.3.      SiNo
21     9.4.      mayor <- c

```

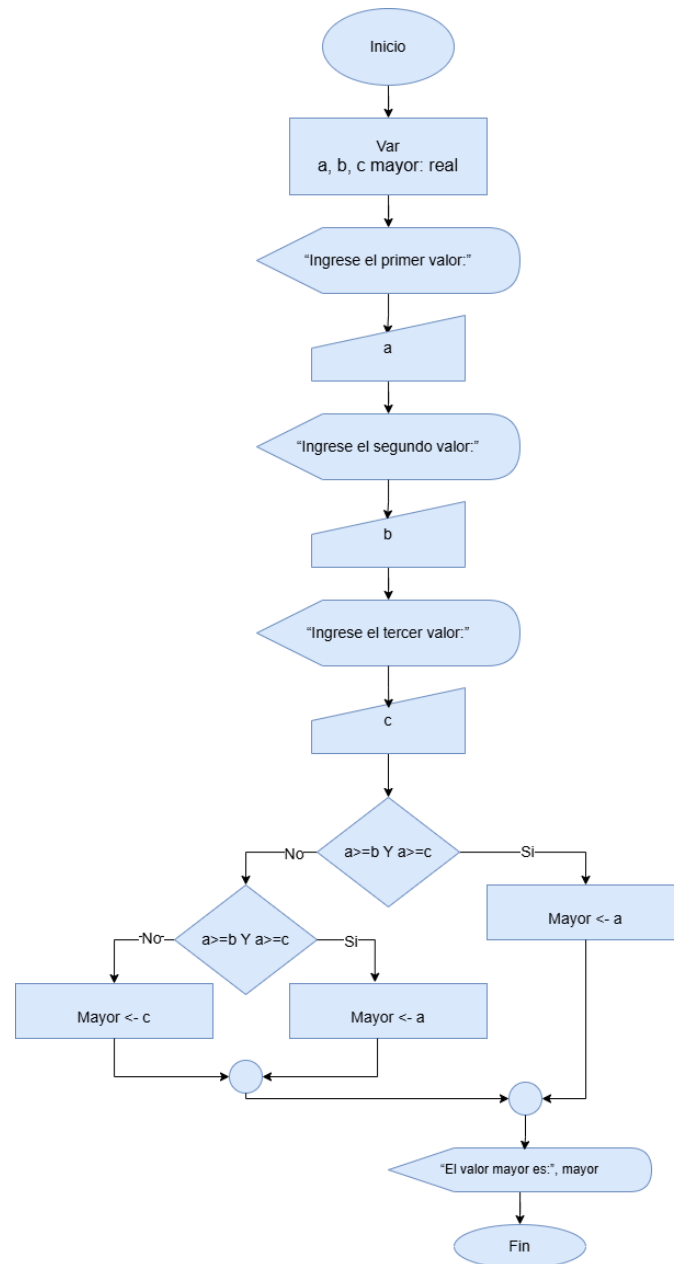


Figura 4.28: Determinación del valor mayor entre tres números ingresados por el usuario.

```

22      9.5.      FinSi
23      10.      FinSi
24
25      11.      Escribir "El valor mayor es: ", mayor
26
27      FinAlgoritmo

```

Listing 4.43: Determinación del valor mayor entre tres números

4.9.6. Ejercicio 6: Cálculo del Máximo Común Divisor (MCD)

Se requiere diseñar un algoritmo que calcule el **máximo común divisor (MCD)** de dos números enteros ingresados por el usuario. El procedimiento debe utilizar el *algoritmo de Euclides*, el cual se basa en la operación módulo (MOD) para reducir progresivamente el problema hasta que el segundo número se vuelva cero; en ese momento, el primer número corresponde al MCD. Este método es ampliamente utilizado por su eficiencia y por su claridad para introducir la idea de iteración controlada por una condición de parada [35], [53].

Pasos para Desarrollar la Solución

Los pasos para desarrollar la solución algorítmica de un problema están encaminados a plasmar en papel lo que el programador principiante o cualquier persona hace para resolver un problema.

1. Análisis del problema. Para comprender el problema es necesario plantearse algunos casos puntuales y resolverlos de manera aislada, como si fuesen únicos. En este ejercicio, el algoritmo aplica repetidamente la regla: *el MCD de num1 y num2 es el mismo que el MCD de num2 y num1 MOD num2*. La iteración termina cuando num2 llega a cero, lo cual reduce errores conceptuales frecuentes en principiantes al convertir un razonamiento matemático en una secuencia mecánica de asignaciones y decisiones [74], [75], [76].

Ejercicios de análisis manual con valores límite. Antes de diseñar el algoritmo, es recomendable resolver manualmente algunos casos representativos del problema. A continuación se presentan conjuntos de datos que permiten verificar el comportamiento de la operación MOD y la correcta aplicación del intercambio inicial cuando el primer número es menor que el segundo.

■ **Caso 1: Caso normal (valores típicos)**

1. **Datos de entrada**

- num1 = 48
- num2 = 18

2. **Resolución manual**

$$48 \bmod 18 = 12, \quad 18 \bmod 12 = 6, \quad 12 \bmod 6 = 0$$

Cuando el residuo llega a cero, el divisor vigente es el MCD.

3. **Resultado esperado**

$$\text{MCD}(48, 18) = 6$$

4. **Observación.** Este caso confirma la reducción progresiva del problema mediante residuos.

■ **Caso 2: Caso con intercambio inicial (primer valor menor)**

1. **Datos de entrada**

- $\text{num1} = 12$
- $\text{num2} = 30$

2. **Resolución manual** Como $\text{num1} < \text{num2}$, se intercambian:

$$(\text{num1}, \text{num2}) \leftarrow (30, 12)$$

Luego:

$$30 \bmod 12 = 6, \quad 12 \bmod 6 = 0$$

3. **Resultado esperado**

$$\text{MCD}(12, 30) = 6$$

4. **Observación.** Verifica que el intercambio evita un primer módulo innecesario y simplifica el flujo.

■ **Caso 3: Caso con números iguales**

1. **Datos de entrada**

- $\text{num1} = 25$
- $\text{num2} = 25$

2. **Resolución manual**

$$25 \bmod 25 = 0$$

3. **Resultado esperado**

$$\text{MCD}(25, 25) = 25$$

4. **Observación.** Caso borde que confirma que el MCD de un número consigo mismo es el mismo número.

■ **Caso 4: Uno de los valores es cero (borde)**

1. **Datos de entrada**

- $\text{num1} = 19$
- $\text{num2} = 0$

2. **Resolución manual** Si $\text{num2} = 0$, no se debe aplicar MOD; el MCD corresponde a num1 .

3. **Resultado esperado**

$$\text{MCD}(19, 0) = 19$$

4. **Observación.** Este caso exige que la condición de parada contemple el cero para evitar división por cero [23], [24].

■ **Caso 5: Caso con coprimos (MCD = 1)**

1. **Datos de entrada**

- $\text{num1} = 35$
- $\text{num2} = 64$

2. **Resolución manual**

$$64 \bmod 35 = 29, \quad 35 \bmod 29 = 6, \quad 29 \bmod 6 = 5, \quad 6 \bmod 5 = 1, \quad 5 \bmod 1 = 0$$

3. **Resultado esperado**

$$\text{MCD}(35, 64) = 1$$

4. **Observación.** Confirma el comportamiento cuando no existen divisores comunes distintos de 1.

■ **Caso 6: Caso inválido (valores negativos)**

1. **Datos de entrada**

- $\text{num1} = -24$
- $\text{num2} = 18$

2. **Resolución:** Para un tratamiento introductorio consistente, se recomienda validar que ambos números sean positivos antes de ejecutar el cálculo, o bien normalizarlos con valor absoluto.

3. **Resultado esperado.** El algoritmo debe advertir la situación, por ejemplo:

■ *“Error: los valores deben ser enteros positivos.”*

■ **Observación.** Este caso fuerza a considerar validaciones de dominio para robustez [23], [24].

2. Identificación de datos de entrada. Los datos de entrada corresponden a la información que el algoritmo necesita recibir desde el exterior para poder ejecutar el cálculo. En el algoritmo, estos valores se capturan mediante operaciones de lectura.

- **num1 (entero):** primer número ingresado por el usuario.
- **num2 (entero):** segundo número ingresado por el usuario.

Cada lectura debe ir antecedida por un mensaje que indique al usuario qué se espera que ingrese. Esta práctica reduce ambigüedad y mejora la interacción en programas introductorios [7], [59].

3. Identificación de datos de salida. Los datos de salida son los resultados que el algoritmo entrega al usuario una vez finalizado el procesamiento.

- **MCD (entero):** máximo común divisor de num1 y num2 .
- **Mensaje informativo:** “El MCD es: num1 ” (al finalizar, num1 contiene el resultado).

La salida combina un valor numérico con un texto descriptivo para facilitar la interpretación del resultado [7], [59].

4. Identificación de cálculos intermedios (procesos intermedios). Los procesos intermedios son operaciones necesarias para llegar al resultado, pero que no constituyen todavía la respuesta final.

- **Intercambio inicial:** si $\text{num1} < \text{num2}$, se utiliza aux para intercambiar valores y asegurar que num1 sea el mayor. Este paso mejora consistencia del flujo y reduce confusiones al operar con el módulo.
- **Cálculo del residuo:** $\text{resto} \leftarrow \text{num1} \bmod \text{num2}$. Este residuo representa la parte que queda al dividir num1 entre num2 .
- **Desplazamiento de variables:** $\text{num1} \leftarrow \text{num2}$ y $\text{num2} \leftarrow \text{resto}$. Esta actualización prepara la siguiente iteración, preservando la propiedad del algoritmo de Euclides [53].

Estos pasos implementan un patrón de iteración por reducción: en cada ciclo, el segundo valor disminuye hasta alcanzar cero, lo que facilita la verificación manual del estado [74], [76].

5. Identificación de cálculos finales (procesos finales). Los procesos finales producen directamente el resultado que se reporta al usuario.

- **Presentación del resultado:** cuando la condición de parada se cumple, se imprime el mensaje “El MCD es: num1 ”.

Este paso final se ejecuta únicamente cuando se ha concluido la reducción iterativa, garantizando que el valor mostrado corresponde al máximo divisor común [35], [53].

6. Identificación de condiciones (decisiones). Las decisiones son expresiones lógicas que determinan el flujo de control.

- **Condición de intercambio:** $\text{num1} < \text{num2}$. Si es verdadera, se realiza el intercambio usando aux .

- **Condición de parada:** $\text{num2} \leq 0$. Si es verdadera, el algoritmo finaliza mostrando el resultado; si es falsa, se ejecuta una nueva iteración.

Estas condiciones formalizan reglas simples que evitan operaciones inválidas (como MOD con divisor cero) y hacen explícita la lógica de terminación, práctica recomendable en desarrollo de software confiable [23], [24].

7. Identificación de repeticiones o iteraciones. El algoritmo contiene una estructura repetitiva implícita controlada por la condición $\text{num2} \leq 0$. Mientras dicha condición sea falsa, el algoritmo continúa calculando residuos y actualizando variables.

- **Iteración por reducción:** en cada repetición se calcula **resto** y se actualizan **num1** y **num2**. Este esquema corresponde a un ciclo tipo **while** en lenguajes de programación estructurada [35], [59].

Este ejercicio consolida el uso de variables, operadores relacionales y la operación MOD, además de introducir una iteración con condición de parada clara, componente clave para aprendizaje inicial [74], [75], [76].

8. Condiciones para finalizar el algoritmo. El algoritmo finaliza cuando:

- se han leído los dos números enteros;
- se ha realizado el intercambio inicial si corresponde;
- la condición $\text{num2} \leq 0$ se vuelve verdadera;
- se imprime el resultado almacenado en **num1**.

En consecuencia, el término del proceso está ligado a la condición de parada del algoritmo de Euclides y no al número fijo de iteraciones, lo cual refuerza la comprensión de bucles controlados por estado [35], [53].

Propuesta de Solución

La Figura 4.29 presenta una solución algorítmica estructurada para calcular el MCD de dos enteros. El diseño separa con claridad (i) la captura de datos, (ii) la normalización inicial mediante intercambio, (iii) la iteración por residuos usando MOD y (iv) la condición de parada que determina cuándo el resultado ya se encuentra en **num1**. Esta organización favorece la verificación manual del flujo y la posterior traducción a un lenguaje de programación, manteniendo correspondencia directa entre asignaciones, decisiones e iteración [59], [72].

El Algoritmo 4.44 presenta una propuesta de solución estructurada para calcular el MCD. En esta formulación se identifican explícitamente las variables auxiliares (**aux** y **resto**) y las condiciones que controlan el intercambio inicial y la terminación del proceso. El uso de una actualización sistemática del par (**num1**, **num2**) reduce la complejidad del razonamiento y facilita depuración, ya que en cada iteración se conserva la propiedad del MCD mientras se reduce el tamaño del problema [53], [74].

```

1      Algoritmo MCD Euclides
2      1.      Var:
3          1.1.      num1, num2: entero
4          1.2.      resto, aux: entero
5
6      2.      Escribir "Digite el primer numero"
7      3.      Leer num1
8
9      4.      Escribir "Ingrese el segundo valor"
10     5.      Leer num2
11
12     6.      Si (num1 < 0) Entonces

```

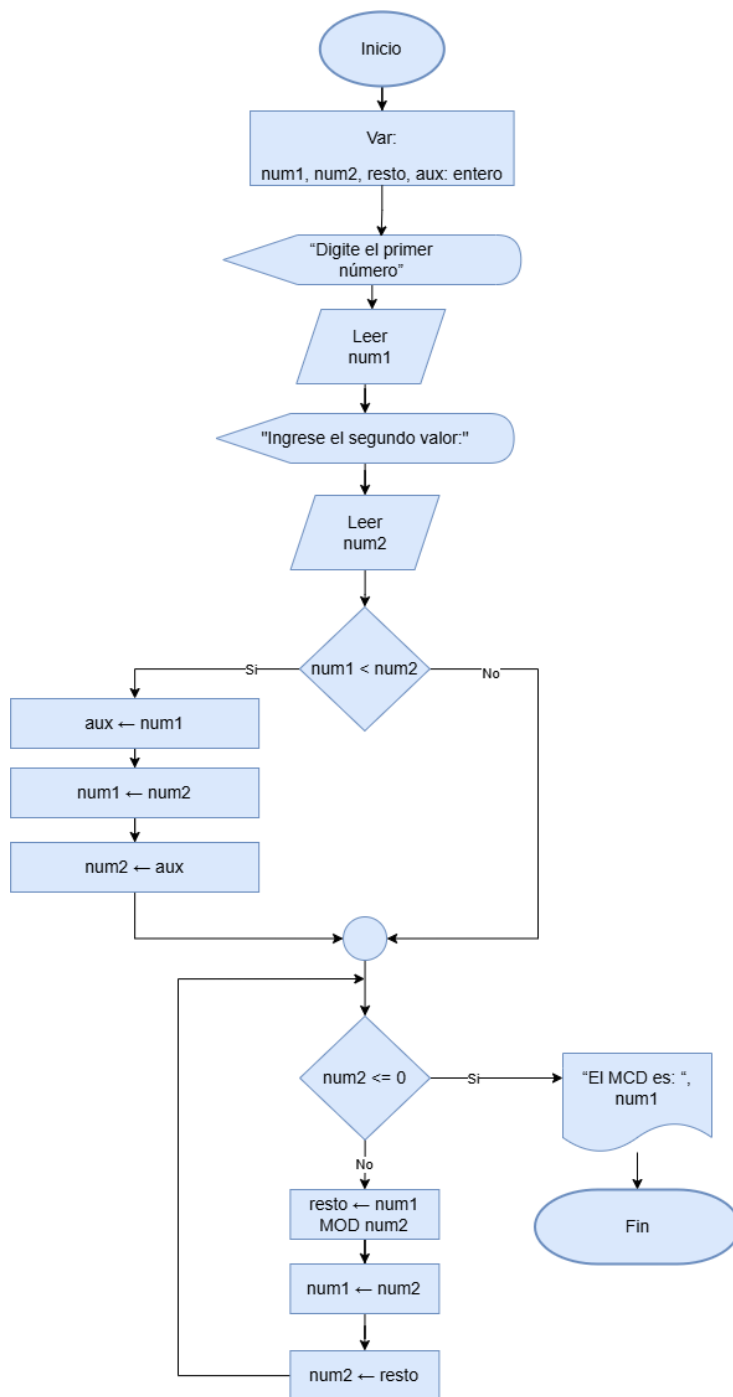


Figura 4.29: Cálculo del MCD de dos enteros mediante el algoritmo de Euclides utilizando la operación módulo.

```

13      6.1.      num1 <- abs(num1)
14      7.      FinSi
15      8.      Si (num2 < 0) Entonces
16      8.1.      num2 <- abs(num2)
17      9.      FinSi
18
19      10.     Si num1 < num2 Entonces
20      10.1.     aux  <- num1
21      10.2.     num1 <- num2
22      10.3.     num2 <- aux
23      11.     FinSi
24
25      12.     Mientras num2 > 0 Hacer
26      12.1.     resto <- num1 MOD num2
27      12.2.     num1  <- num2
28      12.3.     num2  <- resto
29      13.     FinMientras
30
31      14.     Escribir "El MCD es: ", num1
32      FinAlgoritmo

```

Listing 4.44: Cálculo del MCD mediante el algoritmo de Euclides

En el Listado de código 4.44 la solución incorpora además un *mecanismo de robustez orientado al dominio de los datos*: la normalización de los valores ingresados mediante la función de valor absoluto. Las instrucciones comprendidas entre los pasos 6 y 9 verifican si alguno de los números introducidos es negativo y, de ser así, lo transforman en su correspondiente valor positivo antes de iniciar el proceso iterativo del algoritmo de Euclides. Esta operación garantiza la coherencia matemática del procedimiento, dado que el máximo común divisor se define convencionalmente como un entero no negativo y cumple la propiedad $\gcd(a, b) = \gcd(|a|, |b|)$. De esta forma, el algoritmo mantiene su corrección lógica incluso cuando el usuario introduce valores fuera del dominio típico del problema, evitando comportamientos inesperados durante la operación MOD y fortaleciendo la confiabilidad general de la solución [23], [24], [77], [78].

4.9.7. Ejercicio 7: Cálculo del Mínimo Común Múltiplo (MCM)

Se requiere diseñar un algoritmo que calcule el **mínimo común múltiplo (MCM)** de dos números enteros ingresados por el usuario. El procedimiento debe contemplar el caso en que alguno de los valores sea cero, situación en la cual el resultado se define como cero dentro del tratamiento introductorio del problema. Cuando ambos valores son distintos de cero, la solución puede construirse a partir de una propiedad clásica de teoría de números: para dos enteros a y b , el mínimo común múltiplo puede calcularse mediante la expresión $\text{MCM}(a, b) = \frac{|a \cdot b|}{\text{MCD}(a, b)}$. Esta formulación resulta especialmente valiosa desde el punto de vista algorítmico, ya que reutiliza el cálculo del máximo común divisor como subproceso y permite introducir la idea de descomposición modular de la solución [35], [53].

Pasos para Desarrollar la Solución

Los pasos para desarrollar la solución algorítmica de un problema permiten transformar un razonamiento matemático o cotidiano en una secuencia ordenada de instrucciones. En este ejercicio, el cálculo del MCM no se realiza mediante búsqueda exhaustiva de múltiplos, sino a partir de una relación directa con el máximo común divisor. Este enfoque reduce la complejidad conceptual del problema y, al mismo tiempo, muestra al estudiante que una solución puede construirse reutilizando resultados obtenidos por otros algoritmos previamente conocidos.

1. Análisis del problema. Para comprender el problema es conveniente partir de algunos casos concretos y resolverlos manualmente. En este ejercicio, el algoritmo sigue dos ideas principales. La primera consiste en verificar si alguno de los números es igual a cero; si esto ocurre, el resultado se establece inmediatamente como cero, evitando operaciones innecesarias. La segunda consiste en normalizar los valores mediante valor absoluto y luego calcular el MCD de ambos números para obtener finalmente el MCM usando la relación $MCM(a, b) = \frac{a \cdot b}{MCD(a, b)}$. Esta descomposición facilita el aprendizaje porque convierte un problema aparentemente nuevo en una extensión natural de uno ya conocido, fortaleciendo la capacidad del estudiante para reutilizar soluciones algorítmicas previas [74], [75], [76].

Ejercicios de análisis manual con valores representativos. Antes de diseñar el algoritmo, es recomendable resolver manualmente algunos casos que permitan verificar la validez de la fórmula y el comportamiento de la condición especial cuando interviene el cero.

■ Caso 1: Caso normal (valores típicos)

1. Datos de entrada

- a = 12
- b = 18

2. Resolución manual

$$MCD(12, 18) = 6$$

$$MCM(12, 18) = \frac{12 \cdot 18}{6} = 36$$

3. Resultado esperado

$$MCM(12, 18) = 36$$

4. Observación.

Este caso verifica la aplicación directa de la fórmula basada en el MCD.

■ Caso 2: Caso con números iguales

1. Datos de entrada

- a = 15
- b = 15

2. Resolución manual

$$MCD(15, 15) = 15$$

$$MCM(15, 15) = \frac{15 \cdot 15}{15} = 15$$

3. Resultado esperado

$$MCM(15, 15) = 15$$

4. Observación.

El mínimo común múltiplo de un número consigo mismo es el propio número.

■ Caso 3: Caso con uno de los valores igual a cero

1. Datos de entrada

- a = 0
- b = 9

2. **Resolución manual** Como uno de los operandos es cero, el algoritmo define:

$$\text{MCM}(0, 9) = 0$$

3. **Resultado esperado**

$$\text{MCM}(0, 9) = 0$$

4. **Observación.** Este caso evita invocar innecesariamente el cálculo del MCD y refuerza el control del dominio de entrada.

■ **Caso 4: Caso con valores negativos**

1. **Datos de entrada**

- $a = -8$
- $b = 20$

2. **Resolución manual** Primero se normalizan los valores:

$$|a| = 8, \quad |b| = 20$$

Luego:

$$\text{MCD}(8, 20) = 4$$

$$\text{MCM}(8, 20) = \frac{8 \cdot 20}{4} = 40$$

3. **Resultado esperado**

$$\text{MCM}(-8, 20) = 40$$

4. **Observación.** Este caso confirma la utilidad de la normalización con valor absoluto antes del cálculo.

■ **Caso 5: Caso con números coprimos**

1. **Datos de entrada**

- $a = 7$
- $b = 9$

2. **Resolución manual**

$$\text{MCD}(7, 9) = 1$$

$$\text{MCM}(7, 9) = \frac{7 \cdot 9}{1} = 63$$

3. **Resultado esperado**

$$\text{MCM}(7, 9) = 63$$

4. **Observación.** Cuando los números son coprimos, el MCM coincide con el producto de ambos.

■ **Caso 6: Ambos valores iguales a cero**

1. **Datos de entrada**

- $a = 0$
- $b = 0$

2. **Resolución manual** En el tratamiento introductorio del problema, el algoritmo asigna:

$$\text{MCM}(0, 0) = 0$$

3. Resultado esperado

$$\text{MCM}(0, 0) = 0$$

4. **Observación.** Este caso fuerza la inclusión de una decisión temprana que evite divisiones indeterminadas o llamadas innecesarias al subproceso del MCD [23], [24].

2. Identificación de datos de entrada. Los datos de entrada corresponden a la información que el algoritmo debe recibir para poder realizar el cálculo.

- **a (entero):** primer número ingresado por el usuario.
- **b (entero):** segundo número ingresado por el usuario.

Cada lectura debe ir precedida por un mensaje claro que indique al usuario el dato esperado. Esta práctica mejora la interacción, reduce ambigüedades y facilita la interpretación del programa en contextos de aprendizaje inicial [7], [59].

3. Identificación de datos de salida. Los datos de salida son los resultados que el algoritmo entrega una vez concluido el procesamiento.

- **MCM (entero):** mínimo común múltiplo de a y b.
- **Mensaje informativo:** “El mínimo común múltiplo es: mcm”.

La salida combina el valor numérico con un texto descriptivo para facilitar su interpretación por parte del usuario [7], [59].

4. Identificación de cálculos intermedios (procesos intermedios). Los procesos intermedios son operaciones necesarias para obtener el resultado final.

- **Validación del cero:** si alguno de los valores es cero, se asigna directamente `mcm <- 0`.
- **Normalización de signos:** `a <- abs(a)` y `b <- abs(b)`. Esta operación garantiza que el cálculo se realice con valores no negativos.
- **Cálculo del MCD:** `divisor <- MCD(a,b)`. Este subproceso reutiliza el algoritmo de Euclides.
- **Cálculo del MCM:** `mcm <- a*b/divisor`. Esta fórmula expresa la relación entre producto, MCD y MCM.

Estos pasos muestran que una solución puede construirse de manera modular: en lugar de diseñar el cálculo del MCM desde cero, el algoritmo reutiliza un procedimiento conocido para el MCD y lo integra dentro de una solución más amplia [35], [53].

5. Identificación de cálculos finales (procesos finales). Los procesos finales son aquellos que producen directamente la respuesta que se mostrará al usuario.

- **Asignación final de mcm:** ya sea por la condición especial del cero o mediante la fórmula $\frac{a \cdot b}{\text{MCD}(a,b)}$.
- **Presentación del resultado:** impresión del mensaje “El mínimo común múltiplo es: mcm”.

Estas operaciones solo se ejecutan una vez que el algoritmo ha determinado correctamente el valor del MCM, garantizando coherencia entre el procesamiento previo y la salida mostrada al usuario.

6. Identificación de condiciones (decisiones). Las decisiones son expresiones lógicas que controlan el flujo del algoritmo.

- **Condición especial:** `a == 0 o b == 0`. Si es verdadera, el algoritmo asigna `mcm <- 0`.
- **Condición alternativa:** si ambos valores son distintos de cero, se normalizan los signos y se procede al cálculo del MCD y del MCM.

Estas decisiones permiten distinguir con claridad entre un caso borde y el caso general del problema, práctica importante en el diseño de software confiable [23], [24].

7. Identificación de repeticiones o iteraciones. El diagrama de flujo principal no muestra explícitamente una iteración dentro del cálculo del MCM, ya que la parte repetitiva queda encapsulada en la llamada al subproceso `MCD(a,b)`. Por ello, la repetición no aparece como un ciclo visible en el algoritmo principal, sino como una funcionalidad delegada al algoritmo de Euclides previamente estudiado.

- **Iteración encapsulada:** el cálculo del MCD contiene internamente una repetición por residuos, aunque desde la perspectiva del algoritmo principal del MCM se comporta como una operación única.

Este aspecto resulta especialmente valioso desde el punto de vista didáctico, ya que introduce al estudiante en la idea de modularización y reutilización de subprocesos, una práctica central en programación estructurada y en ingeniería de software [35], [59].

8. Condiciones para finalizar el algoritmo. El algoritmo finaliza cuando:

- se han leído los dos números enteros;
- se ha evaluado la condición especial asociada al cero;
- se ha calculado el valor del MCM, ya sea directamente o a través del MCD;
- se imprime el resultado almacenado en `mcm`.

En consecuencia, la finalización del proceso depende de una decisión inicial simple y, en el caso general, de la correcta culminación del subproceso encargado de calcular el MCD.

Propuesta de Solución

La Figura 4.30 presenta una solución algorítmica estructurada para calcular el mínimo común múltiplo de dos enteros. El diseño separa con claridad (i) la captura de datos, (ii) la validación del caso especial asociado al cero, (iii) la normalización de los operandos mediante valor absoluto, (iv) la reutilización del cálculo del MCD como subproceso y (v) la obtención final del resultado mediante la fórmula $MCM(a, b) = \frac{a \cdot b}{MCD(a, b)}$. Esta organización favorece la comprensión del problema y muestra cómo una solución puede construirse reutilizando algoritmos previamente desarrollados.

El Algoritmo 4.45 presenta una propuesta de solución estructurada para calcular el MCM. En esta formulación se identifican explícitamente la condición especial cuando alguno de los valores es cero, la normalización de signos mediante valor absoluto y el uso del MCD como subproceso auxiliar. Esta construcción reduce la complejidad del razonamiento, ya que evita la búsqueda directa de múltiplos y transforma el problema en una secuencia breve de decisiones y cálculos aritméticos. Además, introduce al estudiante en una idea importante de diseño algorítmico: resolver un problema nuevo a partir de la reutilización de soluciones ya conocidas [53], [74].

```

1      Algoritmo MCMConMCD
2      1.      Var:
3      1.1.      a, b: entero
4      1.2.      divisor, mcm: entero
5
6      2.      Escribir "Digite el primer numero"
7      3.      Leer a
8
9      4.      Escribir "Digite el segundo numero"
10     5.      Leer b
11
12     6.      Si (a = 0 O b = 0) Entonces
13     6.1.      mcm <- 0
14     6.2.      SiNo
15     6.3.      a <- abs(a)
```

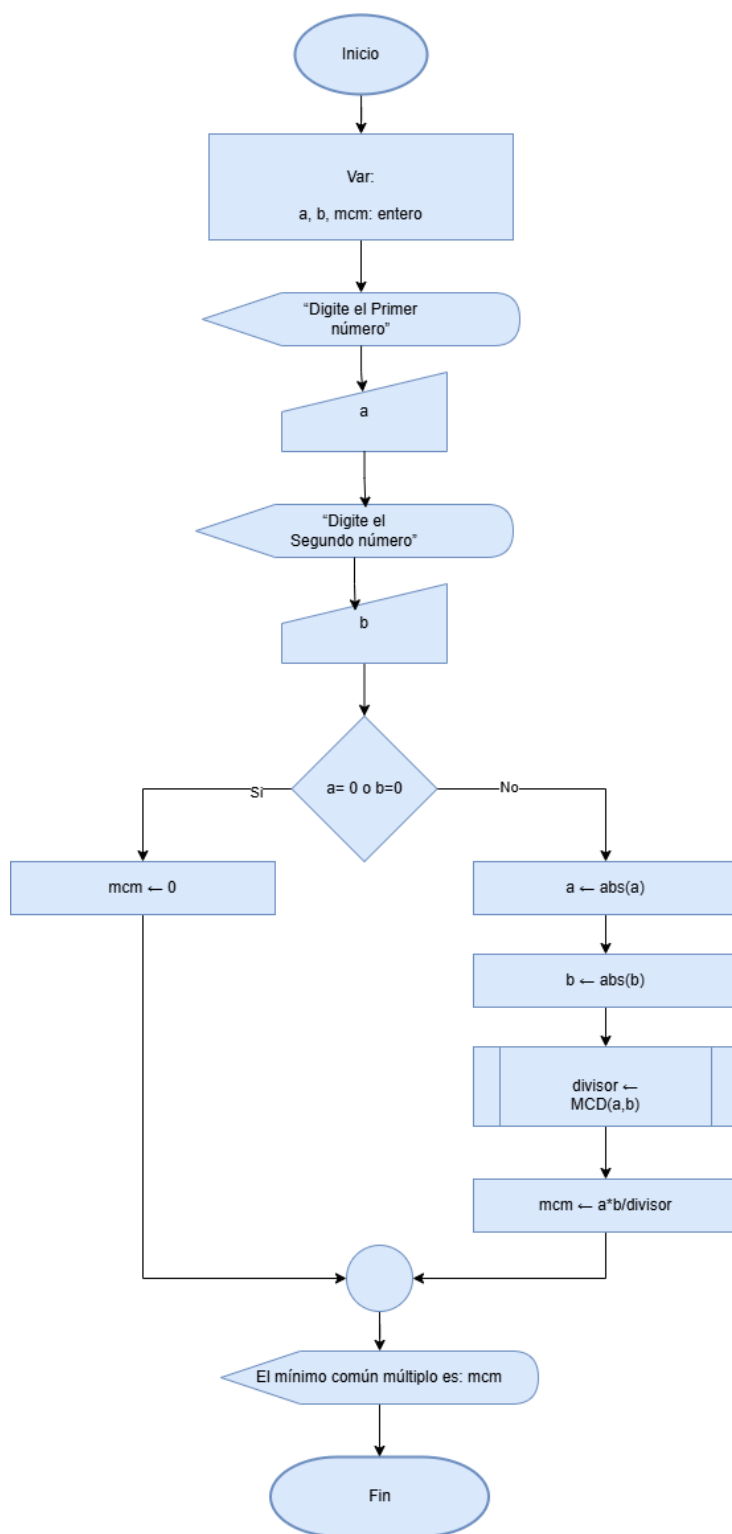


Figura 4.30: Cálculo del mínimo común múltiplo de dos enteros a partir del máximo común divisor.

```

16      6.4.      b <- abs(b)
17      6.5.      divisor <- MCD(a,b)
18      6.6.      mcm <- a*b/divisor
19      7.      FinSi
20
21      8.      Escribir "El minimo comun multiplo es: ", mcm
22      FinAlgoritmo

```

Listing 4.45: Cálculo del mínimo común múltiplo a partir del MCD

En el Listado de código 4.45 la solución incorpora un mecanismo de robustez orientado al dominio de los datos al normalizar los valores mediante la función de valor absoluto antes de calcular el MCD y el MCM. Esta decisión garantiza que el procedimiento opere sobre enteros no negativos, lo cual resulta coherente con la definición usual del mínimo común múltiplo como entero positivo o nulo. Asimismo, la validación temprana del caso en que alguno de los operandos es cero evita operaciones innecesarias y refuerza la confiabilidad general del algoritmo. En conjunto, estas decisiones muestran cómo una formulación matemática puede enriquecerse con controles sencillos que mejoran la robustez de la solución cuando se implementa en un programa real [23], [24].

4.9.8. Ejercicio 8: Ordenamiento de un Vector de Valores

Se requiere diseñar un algoritmo que permita ordenar un conjunto de valores numéricos reales ingresados por teclado. El procedimiento debe solicitar primero la cantidad de elementos que se desean procesar, validar que dicha cantidad sea positiva y, posteriormente, capturar cada uno de los valores. Una vez completada la carga de datos, el algoritmo debe reorganizar los valores de manera ordenada mediante comparaciones sucesivas entre los elementos.

En problemas sencillos de programación es común trabajar con variables individuales para almacenar datos. Sin embargo, cuando se requiere manejar múltiples valores de la misma naturaleza, el uso de variables independientes se vuelve impráctico y dificulta el tratamiento sistemático de la información. Para resolver este tipo de situaciones se utilizan **arreglos** o **vectores**, estructuras de datos que permiten almacenar un conjunto de elementos del mismo tipo bajo un único nombre y acceder a cada uno de ellos mediante una posición numérica denominada *índice*. Esta forma de organización facilita el manejo de colecciones de datos y constituye uno de los conceptos fundamentales en la programación estructurada [7], [35].

Por ejemplo, el **Listado de código 4.46** muestra una declaración simple de un vector de números reales en pseudocódigo. En este caso, el arreglo se denomina **vector** y permite almacenar múltiples valores dentro de una única estructura.

```

1      Var:
2      vector[ ] : real

```

Listing 4.46: Declaración conceptual de un vector

Un vector puede entenderse como una secuencia ordenada de posiciones contiguas en memoria, donde cada posición almacena un valor del mismo tipo. Para acceder a un elemento específico del vector se utiliza un índice que indica su posición dentro de la estructura. Por ejemplo, si un vector se denomina **vector**, el elemento ubicado en la posición *i* se expresa como **vector[i]**. El uso de índices permite recorrer el conjunto completo de datos mediante estructuras repetitivas, lo que resulta esencial cuando se desea realizar operaciones como búsqueda, cálculo o reordenamiento de los valores almacenados [35], [53].

El acceso a los elementos del vector puede observarse en el **Listado de código 4.47**, donde se obtiene el valor almacenado en una posición determinada del arreglo.

```
1 valor <- vector[i]
```

Listing 4.47: Acceso a un elemento de un vector mediante índice

La Figura 4.31 muestra la operación de acceso a un elemento del arreglo y la copia de su contenido hacia una variable. En este caso, el índice i identifica la posición desde la cual se extrae el dato, y el valor almacenado en dicha celda es transferido a una variable auxiliar. Este tipo de operación aparece con frecuencia en algoritmos que requieren comparar, procesar o intercambiar elementos del vector. Comprender esta idea resulta esencial antes de abordar procedimientos de ordenamiento, ya que muchas de sus operaciones internas consisten precisamente en leer un valor de una posición, almacenarlo temporalmente y luego reubicarlo en otra.

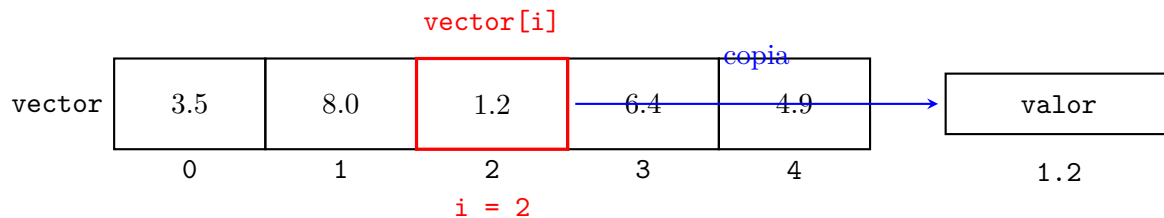


Figura 4.31: Extracción de un elemento del arreglo y copia de su contenido a una variable auxiliar.

Para trabajar con todos los elementos del arreglo es necesario recorrerlo utilizando una estructura repetitiva. El **Listado de código 4.48** muestra un ejemplo simple de recorrido secuencial del vector mediante un índice que se incrementa progresivamente.

```
1 i <- 0
2 Mientras i < cantidad Hacer
3   Escribir vector[i]
4   i <- i + 1
5 FinMientras
```

Listing 4.48: Recorrido secuencial de un vector

En este ejercicio se utilizará un vector para almacenar los valores introducidos por el usuario y posteriormente reorganizarlos mediante un procedimiento de comparación entre pares de elementos. Cuando se detecta que dos posiciones no cumplen con el orden deseado, sus valores se intercambian utilizando una variable auxiliar. Este mecanismo de comparación e intercambio constituye la base de varios algoritmos clásicos de ordenamiento y permite comprender cómo se reorganizan progresivamente los datos dentro de una estructura de almacenamiento [53].

La Figura 4.32 representa de manera conceptual cómo se almacenan los datos dentro de un arreglo unidimensional. Cada valor ocupa una posición específica de la estructura y puede identificarse mediante un índice entero. Esta representación resulta útil para comprender que un vector no es una colección desordenada de datos, sino una secuencia de posiciones consecutivas en las que cada elemento queda asociado a una ubicación concreta. Desde el punto de vista formativo, esta idea permite al estudiante reconocer que el nombre del arreglo representa a toda la estructura, mientras que la expresión indexada, por ejemplo `vector[i]`, permite acceder a uno de sus elementos particulares.

El **Listado de código 4.49** ilustra la operación de intercambio entre dos posiciones del vector utilizando una variable auxiliar. Esta técnica es fundamental en los algoritmos de ordenamiento.

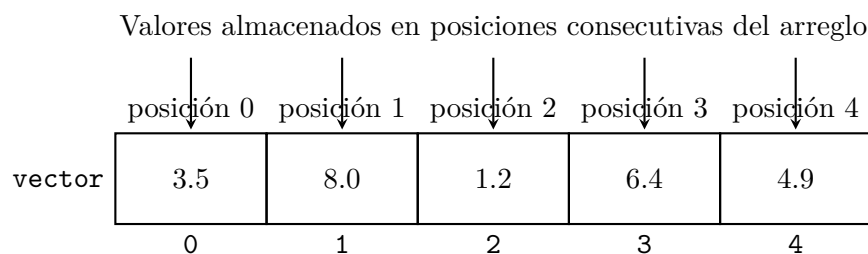


Figura 4.32: Representación conceptual del almacenamiento de valores en un arreglo unidimensional.

```

1  auxiliar <- vector[i]
2  vector[i] <- vector[j]
3  vector[j] <- auxiliar

```

Listing 4.49: Intercambio de dos elementos de un vector

La Figura 4.33 ilustra el procedimiento utilizado para intercambiar dos elementos de un arreglo. El proceso se realiza en tres pasos: primero se copia el contenido de `vector[i]` en una variable auxiliar; posteriormente, el valor de `vector[j]` se mueve hacia la posición `vector[i]`; finalmente, el contenido almacenado temporalmente en `auxiliar` se copia en la posición `vector[j]`. Este mecanismo permite reordenar los elementos del arreglo sin perder información durante el proceso de actualización de las posiciones.

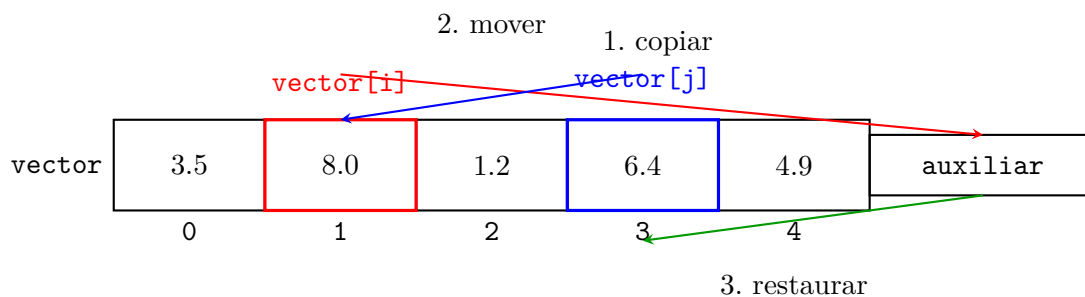


Figura 4.33: Intercambio de dos elementos de un arreglo utilizando una variable auxiliar.

Desde el punto de vista formativo, este problema resulta especialmente relevante porque introduce simultáneamente tres conceptos fundamentales en la programación inicial: el almacenamiento de múltiples datos en un arreglo, el recorrido de la estructura mediante índices y el uso de comparaciones para transformar la organización interna de los datos. Estos elementos constituyen la base de numerosos algoritmos utilizados posteriormente en el desarrollo de software [74], [75], [76].

Pasos para Desarrollar la Solución

Los pasos para desarrollar la solución algorítmica de un problema permiten convertir un procedimiento intuitivo en una secuencia de instrucciones precisas. En este ejercicio, la solución no se limita a leer y mostrar datos, sino que incorpora una fase de reorganización interna de los elementos almacenados. Esto resulta especialmente valioso en programación básica, porque obliga a distinguir entre la etapa de entrada de datos, la etapa de procesamiento interno y la etapa de salida del

resultado final. Además, introduce al estudiante en una idea central del desarrollo de algoritmos: trabajar no solo con variables simples, sino con colecciones de datos que deben recorrerse y modificarse de manera controlada [19], [74], [75], [76].

1. Análisis del problema. Para comprender el problema conviene partir de pequeños conjuntos de datos y ordenarlos manualmente. En este ejercicio, el algoritmo sigue una estrategia de comparación por pares: para cada posición i del vector, se comparan los elementos ubicados en las posiciones posteriores j . Si se detecta que el valor de `vector[i]` es menor que el de `vector[j]`, ambos valores se intercambian utilizando una variable auxiliar. De esta manera, en cada pasada el valor mayor de la sección aún no ordenada se desplaza hacia la posición actual. Esta lógica corresponde a un esquema elemental de ordenamiento por selección/comparación, muy útil en contextos formativos porque hace explícitas las relaciones entre índices, comparaciones y movimientos de datos [35], [53].

Ejercicios de análisis manual con valores representativos. Antes de diseñar el algoritmo, es recomendable resolver manualmente algunos casos representativos que permitan verificar el comportamiento del procedimiento de ordenamiento y el efecto de los intercambios entre posiciones del vector.

■ **Caso 1: Caso normal con valores distintos**

1. **Datos de entrada**

- `cantidad` = 4
- `vector` = [3.5, 8.0, 1.2, 6.4]

2. **Resolución manual** Se compara el elemento de cada posición con los que están a su derecha. Cuando un valor posterior es mayor que el actual, se realiza un intercambio. El proceso continúa hasta completar todas las posiciones relevantes.

3. **Resultado esperado**

[8.0, 6.4, 3.5, 1.2]

4. **Observación.** Este caso confirma que el algoritmo organiza los datos en orden descendente.

■ **Caso 2: Vector ya ordenado**

1. **Datos de entrada**

- `cantidad` = 5
- `vector` = [9, 7, 5, 2, 1]

2. **Resolución manual** Las comparaciones se realizan normalmente, pero no se ejecutan intercambios porque el orden requerido ya se cumple desde el inicio.

3. **Resultado esperado**

[9, 7, 5, 2, 1]

4. **Observación.** Este caso permite verificar que el algoritmo no altera innecesariamente un vector correctamente ordenado.

■ **Caso 3: Vector en orden inverso**

1. **Datos de entrada**

- `cantidad` = 4
- `vector` = [1, 2, 3, 4]

2. **Resolución manual** El algoritmo debe realizar múltiples intercambios hasta desplazar los mayores valores hacia las primeras posiciones.

3. **Resultado esperado**

[4, 3, 2, 1]

4. **Observación.** Este caso exige la mayor cantidad de movimientos y permite observar con claridad el funcionamiento del intercambio.

▪ **Caso 4: Valores repetidos**

1. **Datos de entrada**

- `cantidad` = 5
- `vector` = [4, 7, 4, 2, 7]

2. **Resolución manual** Los valores se comparan normalmente y solo se intercambian cuando el posterior es mayor que el actual.

3. **Resultado esperado**

[7, 7, 4, 4, 2]

4. **Observación.** Este caso confirma que el algoritmo admite elementos repetidos sin perder consistencia.

▪ **Caso 5: Un solo elemento**

1. **Datos de entrada**

- `cantidad` = 1
- `vector` = [12.8]

2. **Resolución manual** No existen comparaciones ni intercambios, ya que un vector con un único elemento ya está ordenado.

3. **Resultado esperado**

[12.8]

4. **Observación.** Este caso borde permite verificar que las condiciones de los ciclos no generen recorridos innecesarios.

▪ **Caso 6: Cantidad no válida**

1. **Datos de entrada**

- `cantidad` = 0

2. **Resolución manual** Como no es posible ordenar cero elementos, el algoritmo debe volver a solicitar una cantidad válida antes de continuar.

3. **Resultado esperado**

4. *“La cantidad debe ser mayor que cero.”*

5. **Observación.** Este caso obliga a incorporar una validación inicial del tamaño del vector, fortaleciendo la robustez de la solución [23], [24].

2. Identificación de datos de entrada. Los datos de entrada corresponden a la información que el algoritmo necesita recibir para ejecutar el proceso de ordenamiento.

- **cantidad (entero):** número de valores que se almacenarán en el vector.
- **vector[i] (real):** cada uno de los valores numéricos ingresados por el usuario.

La cantidad debe validarse antes de proceder a la carga de los datos, de modo que el vector se construya únicamente cuando exista un número positivo de elementos. Cada lectura debe ir acompañada de un mensaje claro que indique al usuario qué valor se espera en cada iteración [7], [59].

3. Identificación de datos de salida. Los datos de salida son los resultados que el algoritmo entrega una vez completado el procesamiento.

- **Vector ordenado:** secuencia final de los valores almacenados en el arreglo después del proceso de comparación e intercambio.

- **Presentación secuencial de los elementos:** salida de cada valor ordenado según el recorrido final del vector.

La salida no consiste en un único valor, sino en la exhibición de todos los elementos del arreglo ya reorganizados. Esto permite al usuario verificar que el ordenamiento se realizó correctamente [7], [59].

4. Identificación de cálculos intermedios (procesos intermedios). Los procesos intermedios son las operaciones necesarias para transformar los datos originales en un vector ordenado.

- **Inicialización del índice de carga:** $i \leftarrow 0$. Permite comenzar el recorrido del vector desde la primera posición.
- **Lectura sucesiva de los elementos:** mientras $i < \text{cantidad}$, cada valor ingresado se almacena en `vector[i]` y luego se incrementa el índice.
- **Inicialización del índice externo de ordenamiento:** $i \leftarrow 0$. Marca la posición desde la cual se buscará el mayor valor restante.
- **Inicialización del índice interno:** $j \leftarrow i + 1$. Permite comparar el elemento actual con los valores ubicados a su derecha.
- **Comparación entre elementos:** si `vector[i] < vector[j]`, se realiza un intercambio.
- **Intercambio mediante variable auxiliar:** `auxiliar ← vector[i]`, `vector[i] ← vector[j]`, `vector[j] ← auxiliar`. Este paso reubica los valores en el orden requerido.

Estos procesos muestran con claridad cómo una tarea aparentemente simple, como ordenar una lista de valores, requiere en realidad varios subprocesos coordinados: almacenamiento, recorrido con índices, comparación y actualización del contenido del vector [35], [53].

5. Identificación de cálculos finales (procesos finales). Los procesos finales son aquellos que producen directamente el resultado visible para el usuario.

- **Recorrido del vector ordenado:** una vez concluido el algoritmo de ordenamiento, se recorre el arreglo para mostrar sus elementos.
- **Presentación de cada valor:** cada `vector[i]` se imprime en pantalla en el nuevo orden resultante.

Estas operaciones permiten transformar el estado interno final del vector en una salida observable, completando así la interacción entre el algoritmo y el usuario.

6. Identificación de condiciones (decisiones). Las decisiones son expresiones lógicas que controlan el flujo del algoritmo.

- **Validación de la cantidad:** $\text{cantidad} > 0$. Permite continuar solo cuando el tamaño del vector es válido.
- **Condición de carga:** $i < \text{cantidad}$. Determina si todavía faltan elementos por ingresar.
- **Condición del ciclo externo:** $i < \text{cantidad} - 1$. Determina si aún quedan posiciones por ordenar.
- **Condición del ciclo interno:** $j < \text{cantidad}$. Controla el recorrido comparativo de las posiciones posteriores.
- **Condición de intercambio:** `vector[i] < vector[j]`. Si se cumple, se intercambian los valores.

Estas decisiones estructuran completamente el comportamiento del algoritmo y permiten distinguir entre la fase de carga, la fase de ordenamiento y la fase de salida. Además, refuerzan el aprendizaje de expresiones relacionales aplicadas a índices y datos almacenados [35], [59].

7. Identificación de repeticiones o iteraciones. El algoritmo contiene varias estructuras repetitivas claramente diferenciadas.

- **Repetición para validación de la cantidad:** mientras el valor ingresado no sea positivo, se solicita nuevamente.
- **Repetición para la carga del vector:** recorre todas las posiciones desde 0 hasta `cantidad-1`.
- **Repetición externa de ordenamiento:** controla cada posición del vector que debe ser fijada en su valor correcto.
- **Repetición interna de comparación:** compara la posición actual con las posiciones posteriores para decidir posibles intercambios.

- **Repetición final de salida:** recorre el vector ordenado para mostrar sus elementos.

La presencia de varios ciclos en un mismo algoritmo resulta especialmente instructiva, pues permite al estudiante distinguir entre iteraciones destinadas a capturar datos, iteraciones destinadas a transformar información y iteraciones orientadas a producir la salida final [74], [75], [76].

8. Condiciones para finalizar el algoritmo. El algoritmo finaliza cuando:

- se ha recibido una cantidad válida de elementos;
- todos los valores han sido leídos y almacenados en el vector;
- se han completado los ciclos de comparación e intercambio;
- se han mostrado los elementos del arreglo ya ordenados.

En consecuencia, la finalización del proceso depende de la culminación ordenada de varias etapas sucesivas, lo que convierte este ejercicio en un ejemplo particularmente útil para mostrar al estudiante cómo varios ciclos y decisiones pueden integrarse dentro de una única solución algorítmica.

Propuesta de Solución

La Figura 4.34 presenta una solución algorítmica estructurada para ordenar un vector de números reales. El diseño separa con claridad (i) la validación del tamaño del arreglo, (ii) la captura secuencial de los datos, (iii) el proceso de ordenamiento basado en comparaciones e intercambios entre posiciones y (iv) la presentación final del vector ya ordenado. Esta organización favorece la comprensión del flujo completo del problema y permite apreciar cómo el tratamiento de arreglos exige coordinar índices, decisiones y actualizaciones del contenido de la estructura de datos [59], [72].

El Algoritmo 4.50 presenta una propuesta de solución estructurada para ordenar un vector de números reales en orden descendente. En esta formulación se identifican explícitamente la validación inicial del tamaño del arreglo, la carga de datos en posiciones consecutivas del vector y el proceso de comparación entre elementos utilizando dos índices. El uso de una variable auxiliar para efectuar intercambios permite modificar el contenido del arreglo sin pérdida de información, mientras que la organización mediante un ciclo externo y uno interno facilita comprender cómo se construye progresivamente el orden final del vector [53], [74].

1	Algoritmo OrdenamientoVectorDescendente
2	1. Var:
3	1.1. cantidad, i, j: entero
4	1.2. vector[], auxiliar: real
5	
6	2. i <- 0
7	3. Escribir "Digite la cantidad de valores a
8	↪ ordenar"
9	4. Leer cantidad
10	5. Mientras cantidad <= 0 Hacer
11	5.1. Escribir "La cantidad debe ser mayor
12	↪ que cero"
	5.2. Escribir "Digite la cantidad de
	↪ valores a ordenar"
	5.3. Leer cantidad

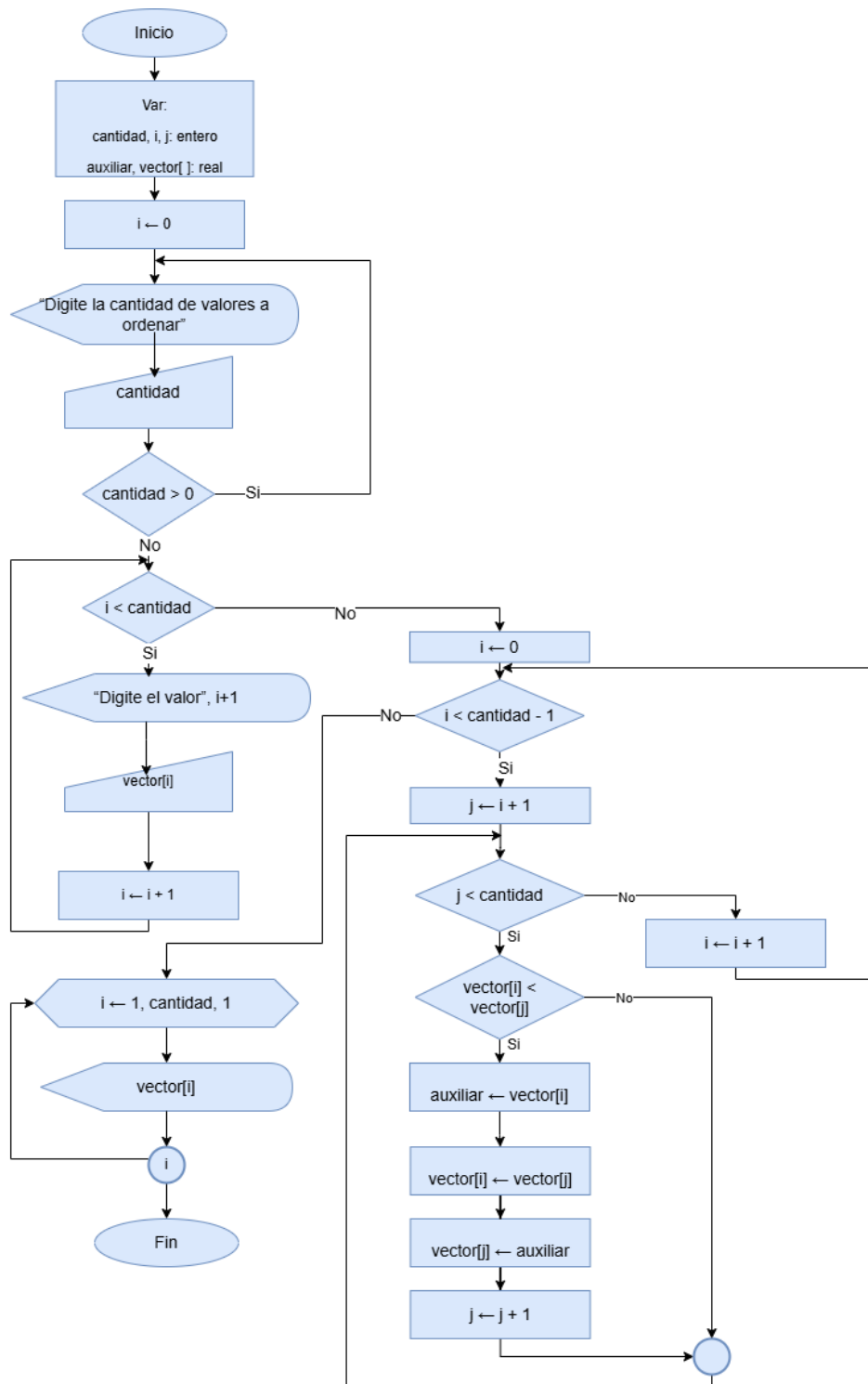


Figura 4.34: Ordenamiento de un vector de números reales mediante comparaciones e intercambios sucesivos.

```

13      6.      FinMientras
14
15      7.      Mientras i < cantidad Hacer
16      7.1.      Escribir "Digite el valor ", i+1
17      7.2.      Leer vector[i]
18      7.3.      i <- i + 1
19      8.      FinMientras
20
21      9.      i <- 0
22      10.     Mientras i < cantidad - 1 Hacer
23      10.1.     j <- i + 1
24      10.2.     Mientras j < cantidad Hacer
25      10.2.1.     Si vector[i] < vector[j] Entonces
26      10.2.1.1.     auxiliar <- vector[i]
27      10.2.1.2.     vector[i] <- vector[j]
28      10.2.1.3.     vector[j] <- auxiliar
29      10.2.2.     FinSi
30      10.2.3.     j <- j + 1
31      10.3.     FinMientras
32      10.4.     i <- i + 1
33      11.     FinMientras
34
35      12.     Para i <- 0 Hasta cantidad - 1 Hacer
36      12.1.     Escribir vector[i]
37      13.     FinPara
38      FinAlgoritmo
39

```

Listing 4.50: Ordenamiento descendente de un vector de números reales

El algoritmo presentado en el Listado de código 4.50 contempla una comprobación inicial del número de elementos a ingresar. Este control evita uno de los errores más frecuentes en programas introductorios: intentar recorrer o procesar estructuras de datos cuyo tamaño no ha sido definido correctamente. La verificación temprana de los datos de entrada constituye una práctica recomendada en programación básica, ya que permite prevenir comportamientos inesperados durante la ejecución y facilita el manejo seguro de estructuras de datos como arreglos o listas [7], [59], [79].

4.9.9. Ejercicio 9: Cálculo de Medidas de Tendencia Central

Se requiere diseñar un algoritmo que permita calcular el **promedio**, la **varianza** y la **desviación estándar** de un conjunto de valores numéricos reales ingresados por teclado. El procedimiento debe solicitar primero la cantidad de datos que se desean

procesar, verificar que exista un número suficiente de observaciones para realizar el cálculo y, posteriormente, capturar cada uno de los valores. Una vez almacenados los datos, el algoritmo debe obtener la suma total para calcular el promedio, recorrer nuevamente el conjunto de valores para acumular las desviaciones cuadráticas respecto del promedio y, finalmente, determinar la varianza y la desviación estándar a partir de dichas operaciones [7], [35], [53].

En este ejercicio vuelve a ser necesario el uso de un **arreglo o vector**, ya que no se trabaja con un único valor, sino con una colección de datos del mismo tipo. El vector permite conservar todos los elementos introducidos por el usuario para reutilizarlos en más de una fase del algoritmo. En efecto, el promedio requiere recorrer el conjunto completo para sumar sus valores, mientras que la varianza exige una segunda pasada sobre los mismos datos para calcular las diferencias respecto de la media. Esto hace que el uso de variables individuales resulte insuficiente y que el arreglo se convierta en la estructura natural para almacenar y recuperar cada observación mediante índices [7], [35].

Este problema resulta especialmente importante porque introduce al estudiante en el cálculo de medidas estadísticas básicas a partir de un procedimiento algorítmico organizado en varias etapas. A diferencia de ejercicios centrados únicamente en operaciones directas, aquí la solución requiere conservar los datos, recorrerlos más de una vez y coordinar cálculos sucesivos cuya dependencia lógica debe mantenerse con precisión. Por ello, este ejercicio no solo amplía el tratamiento de arreglos, sino que también refuerza la comprensión de acumuladores, recorridos secuenciales, operaciones aritméticas compuestas y reutilización de resultados intermedios dentro de una misma solución [19], [74], [75], [76].

Pasos para Desarrollar la Solución

Los pasos para desarrollar la solución algorítmica de un problema permiten convertir un procedimiento matemático en una secuencia ordenada de operaciones que puedan ejecutarse de manera mecánica. En este ejercicio, la solución requiere dos recorridos sobre el mismo conjunto de datos: el primero para obtener la suma total y calcular el promedio, y el segundo para acumular los cuadrados de las desviaciones de cada dato respecto de la media. Esta organización introduce al estudiante en una idea fundamental del desarrollo de algoritmos: no siempre basta con una sola pasada sobre la información, sino que en ciertos problemas es necesario almacenar los datos y reutilizarlos posteriormente.

1. Análisis del problema. Para comprender el problema conviene partir de pequeños conjuntos de datos y calcular manualmente las medidas estadísticas requeridas. En esta propuesta se trabaja con la versión poblacional de la varianza, por lo que la suma de cuadrados se divide entre n . El procedimiento algorítmico sigue una secuencia clara: primero se valida que exista un número suficiente de datos, luego se leen y almacenan las observaciones, después se calcula el promedio y finalmente se recorren nuevamente los valores para obtener la suma de las diferencias cuadráticas respecto de la media. A partir de este resultado se calcula la varianza y, posteriormente, la desviación estándar como raíz cuadrada de la varianza. Esta descomposición facilita el aprendizaje porque separa con nitidez la etapa de captura, la etapa de cálculo de la media y la etapa de dispersión estadística [35], [53], [74].

Ejercicios de análisis manual con valores representativos. Antes de diseñar el algoritmo, es recomendable resolver manualmente algunos casos que permitan verificar el cálculo del promedio, la varianza y la desviación estándar, así como el comportamiento de la validación inicial del número de datos.

■ Caso 1: Caso normal con valores conocidos

1. Datos de entrada

- $n = 8$
- $\text{datos} = [2, 4, 4, 4, 5, 5, 7, 9]$

2. Resolución manual

$$\begin{aligned}\text{promedio} &= \frac{2 + 4 + 4 + 4 + 5 + 5 + 7 + 9}{8} = 5 \\ \sum (x_i - \bar{x})^2 &= (2 - 5)^2 + (4 - 5)^2 + \dots + (9 - 5)^2 = 32 \\ \text{varianza} &= \frac{32}{8} = 4, \quad \text{desviación estándar} = \sqrt{4} = 2\end{aligned}$$

3. Resultado esperado

$$\text{promedio} = 5, \quad \text{varianza} = 4, \quad \text{desviación estándar} = 2$$

4. **Observación.** Este caso verifica un conjunto de datos donde los resultados son enteros y fáciles de comprobar manualmente.

■ **Caso 2: Dos valores**

1. **Datos de entrada**

- $n = 2$
- $\text{datos} = [1, 3]$

2. **Resolución manual**

$$\text{promedio} = \frac{1+3}{2} = 2$$

$$\sum (x_i - \bar{x})^2 = (1-2)^2 + (3-2)^2 = 2$$

$$\text{varianza} = \frac{2}{2} = 1, \quad \text{desviación estándar} = \sqrt{1} = 1$$

3. **Resultado esperado**

$$\text{promedio} = 2, \quad \text{varianza} = 1, \quad \text{desviación estándar} = 1$$

4. **Observación.** Este caso muestra el comportamiento mínimo aceptado por la propuesta al trabajar con al menos dos datos.

■ **Caso 3: Todos los valores iguales**

1. **Datos de entrada**

- $n = 4$
- $\text{datos} = [6, 6, 6, 6]$

2. **Resolución manual**

$$\text{promedio} = 6$$

$$\sum (x_i - \bar{x})^2 = 0$$

$$\text{varianza} = 0, \quad \text{desviación estándar} = 0$$

3. **Resultado esperado**

$$\text{promedio} = 6, \quad \text{varianza} = 0, \quad \text{desviación estándar} = 0$$

4. **Observación.** Cuando no existe dispersión entre los datos, tanto la varianza como la desviación estándar son cero.

■ **Caso 4: Valores negativos y positivos**

1. **Datos de entrada**

- $n = 3$
- $\text{datos} = [-2, 0, 2]$

2. **Resolución manual**

$$\text{promedio} = \frac{-2+0+2}{3} = 0$$

$$\sum (x_i - \bar{x})^2 = (-2)^2 + 0^2 + 2^2 = 8$$

$$\text{varianza} = \frac{8}{3}, \quad \text{desviación estándar} = \sqrt{\frac{8}{3}}$$

3. Resultado esperado

$$\text{promedio} = 0, \quad \text{varianza} = \frac{8}{3}, \quad \text{desviación estándar} \approx 1.633$$

4. **Observación.** Este caso confirma que el algoritmo debe operar correctamente con valores reales de distinto signo.

■ Caso 5: Valores decimales

1. Datos de entrada

- $n = 4$
- $\text{datos} = [1.5, 2.5, 3.0, 5.0]$

2. Resolución manual

$$\text{promedio} = \frac{1.5 + 2.5 + 3.0 + 5.0}{4} = 3$$

$$\sum (x_i - \bar{x})^2 = (1.5 - 3)^2 + (2.5 - 3)^2 + (3 - 3)^2 + (5 - 3)^2 = 6.5$$

$$\text{varianza} = \frac{6.5}{4} = 1.625, \quad \text{desviación estándar} = \sqrt{1.625} \approx 1.275$$

3. Resultado esperado

$$\text{promedio} = 3, \quad \text{varianza} = 1.625, \quad \text{desviación estándar} \approx 1.275$$

4. **Observación.** Este caso confirma que el algoritmo debe tratar correctamente datos reales no enteros.

■ Caso 6: Cantidad no válida

1. Datos de entrada

- $n = 1$

2. **Resolución manual** Como la propuesta exige al menos dos datos para desarrollar el procedimiento, el algoritmo debe solicitar nuevamente un valor válido de n .

3. Resultado esperado

4. *“Error: la cantidad de valores debe ser mayor o igual que 2. Intente de nuevo.”*

5. **Observación.** Este caso obliga a incorporar una validación inicial del tamaño del arreglo y evita que el procesamiento continúe con un conjunto insuficiente de datos [23], [24].

2. Identificación de datos de entrada. Los datos de entrada corresponden a la información que el algoritmo necesita recibir para ejecutar los cálculos estadísticos.

- **n (entero):** cantidad de datos que serán procesados.
- **datos[i] (real):** cada uno de los valores numéricos almacenados en el arreglo.

La cantidad de datos debe validarse antes de iniciar la captura de valores, de manera que el arreglo se construya solo cuando exista un número suficiente de observaciones. Además, cada lectura debe estar precedida por un mensaje claro que identifique al usuario qué dato se espera en cada iteración [7], [59].

3. Identificación de datos de salida. Los datos de salida son los resultados que el algoritmo entrega una vez concluidos los cálculos.

- **Promedio (real):** media aritmética de los datos ingresados.
- **Varianza (real):** medida de dispersión obtenida a partir de las desviaciones cuadráticas respecto del promedio.

- **Desviación estándar (real):** raíz cuadrada de la varianza.

La salida combina tres resultados numéricos complementarios que describen tanto la tendencia central como la dispersión del conjunto de datos, lo que permite al estudiante apreciar que un mismo arreglo puede ser procesado con distintos fines dentro de un solo algoritmo.

4. Identificación de cálculos intermedios (procesos intermedios). Los procesos intermedios son las operaciones necesarias para obtener las medidas estadísticas finales.

- **Validación inicial de n:** verificar que el número de datos sea al menos dos.
- **Inicialización de acumuladores:** `suma <- 0` y `sumaCuadrados <- 0`.
- **Primer recorrido del vector:** lectura de cada `datos[i]` y acumulación en `suma`.
- **Cálculo del promedio:** `promedio <- suma / n`.
- **Segundo recorrido del vector:** acumulación de $(\text{datos}[i] - \text{promedio})^2$ en `sumaCuadrados`.
- **Cálculo de la varianza:** `varianza <- sumaCuadrados / n`.
- **Cálculo de la desviación estándar:** `desviacion <- raiz2(varianza)`.

Estos pasos muestran con claridad que el problema se resuelve mediante una secuencia de operaciones acumulativas y no mediante una única fórmula aplicada directamente sobre los datos. Esta organización permite dividir el cálculo en fases comprensibles y facilita su implementación progresiva en un lenguaje de programación [35], [53].

5. Identificación de cálculos finales (procesos finales). Los procesos finales son aquellos que producen directamente la respuesta que se mostrará al usuario.

- **Obtención del promedio:** resultado del primer acumulador normalizado por el número de datos.
- **Obtención de la varianza:** resultado del segundo acumulador normalizado por `n`.
- **Obtención de la desviación estándar:** raíz cuadrada de la varianza.
- **Presentación de resultados:** impresión de las tres medidas estadísticas.

Estas operaciones convierten los acumuladores intermedios en información estadística interpretable, completando la transición desde los datos crudos hasta un resumen cuantitativo del conjunto analizado.

6. Identificación de condiciones (decisiones). Las decisiones son expresiones lógicas que controlan el flujo del algoritmo.

- **Validación de la cantidad de datos:** `n >= 2`. Permite continuar solo cuando existe un número suficiente de observaciones.
- **Condición del primer recorrido:** controla la captura y acumulación de los datos.
- **Condición del segundo recorrido:** controla la acumulación de las desviaciones cuadráticas.

Estas decisiones separan con claridad la fase de validación de la fase de cálculo y hacen explícito que ciertos procesos solo deben ejecutarse cuando las condiciones iniciales del problema se encuentran satisfechas [35], [59].

7. Identificación de repeticiones o iteraciones. El algoritmo contiene varias estructuras repetitivas claramente diferenciadas.

- **Repetición para la validación de n:** mientras no se cumpla la condición mínima requerida.
- **Primer recorrido del arreglo:** lectura de cada dato y acumulación de la suma total.
- **Segundo recorrido del arreglo:** cálculo de las diferencias cuadráticas respecto del promedio.

La presencia de más de un recorrido sobre el mismo vector constituye uno de los aportes didácticos más importantes de este ejercicio, ya que enseña que una estructura de datos puede reutilizarse en distintas fases del algoritmo para resolver problemas de mayor complejidad [74], [75], [76].

8. Condiciones para finalizar el algoritmo. El algoritmo finaliza cuando:

- se ha ingresado una cantidad válida de datos;
- todos los valores han sido leídos y almacenados en el arreglo;
- se han completado los dos recorridos necesarios para el cálculo;
- se han obtenido y mostrado el promedio, la varianza y la desviación estándar.

En consecuencia, la finalización del proceso depende de la culminación ordenada de varias etapas sucesivas, lo que convierte este ejercicio en un ejemplo adecuado para mostrar cómo un mismo conjunto de datos puede ser utilizado en distintas fases de procesamiento dentro de una única solución algorítmica.

Propuesta de Solución

La Figura 4.35 presenta una solución algorítmica estructurada para calcular el promedio, la varianza y la desviación estándar de un conjunto de números reales almacenados en un arreglo. El diseño separa con claridad (i) la validación del número de datos, (ii) la inicialización de acumuladores, (iii) la captura y almacenamiento de cada observación, (iv) el cálculo del promedio a partir de la suma total, (v) el segundo recorrido del vector para acumular las desviaciones cuadráticas y (vi) la obtención final de la varianza y la desviación estándar. Esta organización favorece la comprensión del flujo completo del problema y permite apreciar cómo el uso de arreglos y múltiples recorridos se integra en una misma solución [59], [72].

El Algoritmo 4.51 presenta una propuesta de solución estructurada para calcular estas tres medidas estadísticas. En esta formulación se distinguen explícitamente la validación inicial del número de datos, la carga del arreglo, el uso de dos acumuladores y la separación del proceso en dos recorridos del mismo vector. Esta organización reduce la complejidad del razonamiento, ya que evita mezclar en una sola etapa el cálculo de la media con el de la dispersión, y permite al estudiante reconocer con claridad la dependencia existente entre promedio, varianza y desviación estándar [53], [74].

```

1      Algoritmo EstadisticasBasicasVector
2      1.      Var:
3      1.1.      n, i: entero
4      1.2.      datos[ ]: real
5      1.3.      suma, sumaCuadrados, promedio, varianza, desviacion: real
6
7      2.      Escribir "Ingrese la cantidad de datos (n):"
8      3.      Leer n
9      4.      Mientras n < 2 Hacer
10     4.1.      Escribir "Error: la cantidad de valores debe ser mayor o
    ↪ igual que 2. Intente de nuevo."
11     4.2.      Escribir "Ingrese la cantidad de datos (n):"
12     4.3.      Leer n
13     5.      FinMientras
14
15     6.      suma <- 0
16     7.      sumaCuadrados <- 0
17
18     8.      Para i <- 1 Hasta n Hacer
19     8.1.      Escribir "Ingrese el dato ", i, ":"

```

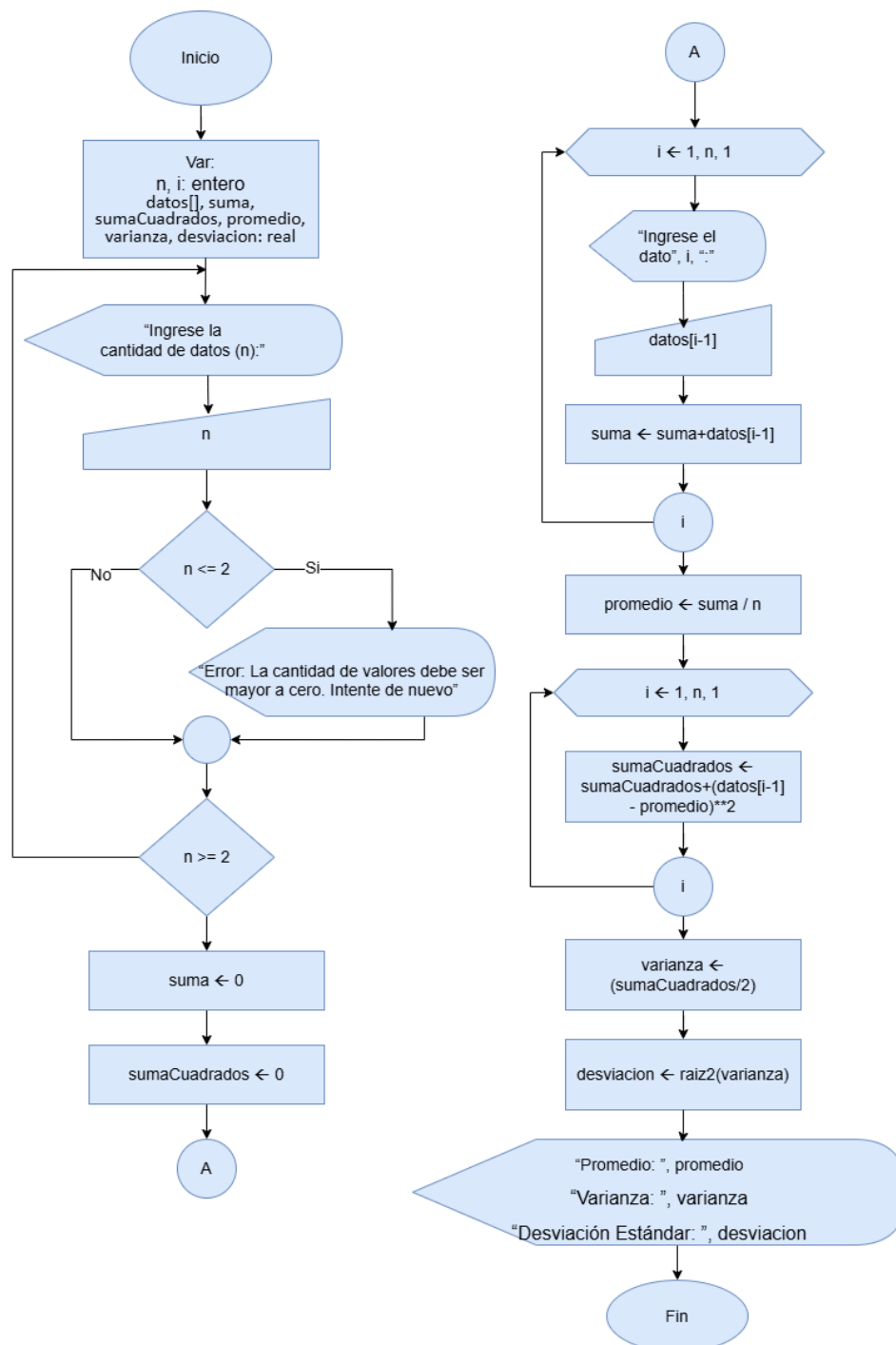



Figura 4.35: Cálculo del promedio, la varianza y la desviación estándar a partir de un arreglo de datos reales.

```

20      8.2.      Leer datos[i-1]
21      8.3.      suma <- suma + datos[i-1]
22      9.        FinPara
23
24      10.       promedio <- suma / n
25
26      11.       Para i <- 1 Hasta n Hacer
27      11.1.     sumaCuadrados <- sumaCuadrados + (datos[i-1] - promedio)^2
28      12.       FinPara
29
30      13.       varianza <- sumaCuadrados / n
31      14.       desviacion <- raiz2(varianza)
32
33      15.       Escribir "Promedio: ", promedio
34      16.       Escribir "Varianza: ", varianza
35      17.       Escribir "Desviacion estandar: ", desviacion
36      FinAlgoritmo

```

Listing 4.51: Cálculo del promedio, la varianza y la desviación estándar de un conjunto de datos

El algoritmo presentado en el Listado de código 4.51 organiza el cálculo en dos recorridos claramente diferenciados sobre el mismo arreglo. En el primero se capturan los datos y se acumula su suma total; en el segundo se reutilizan las observaciones almacenadas para calcular la suma de las desviaciones cuadráticas respecto del promedio. Esta separación mejora la legibilidad del procedimiento y ayuda a comprender que ciertas medidas estadísticas dependen de resultados previos que deben obtenerse antes de continuar con el resto del procesamiento. Además, la validación inicial de **n** evita que el algoritmo avance con una cantidad insuficiente de observaciones, lo que da mayor coherencia al flujo general de la solución [7], [23], [24], [59].

4.9.10. Ejercicio 10: Conversión de un Número Entero en base 10 a Otra Base Menor que 10

Se requiere diseñar un algoritmo que permita convertir un número entero expresado en **base 10** a otra base numérica entera menor que 10. El procedimiento debe solicitar primero el número que se desea convertir, luego pedir la base de destino y verificar que esta pertenezca al dominio permitido. Una vez validada la base, el algoritmo debe aplicar divisiones sucesivas entre la base elegida, registrar los residuos obtenidos en cada iteración y construir con ellos la representación final del número en el nuevo sistema de numeración. Este tipo de problema es especialmente útil en la formación inicial porque integra validación de entradas, operaciones enteras, uso de residuos, concatenación de resultados y control de iteraciones mediante una condición de parada [7], [35], [59].

Desde el punto de vista matemático, la conversión de un número decimal a otra base se fundamenta en un procedimiento clásico de divisiones sucesivas. En cada paso, el residuo de la división entre el número actual y la base de destino produce uno de los dígitos de la nueva representación. Posteriormente, el cociente entero se convierte en el nuevo valor a procesar. Este mecanismo se repite hasta que el cociente sea menor que la base. La secuencia de residuos generados debe reconstruirse en orden inverso para obtener el resultado final, razón por la cual el algoritmo concatena cada nuevo dígito al inicio de la cadena de salida [35], [53]. En consecuencia, este ejercicio no solo introduce la conversión entre sistemas de numeración, sino que

también muestra cómo una propiedad matemática puede transformarse en una estrategia computacional precisa y repetible.

Este problema también resulta valioso porque obliga a distinguir entre el **valor numérico** del dato y su **representación textual** en otra base. Mientras las divisiones, residuos y cocientes se realizan sobre variables enteras, la representación final del número convertido debe almacenarse como una cadena, ya que el resultado no se construye mediante una suma aritmética, sino mediante la concatenación ordenada de dígitos. Esta separación entre cálculo numérico y presentación textual constituye una idea importante en programación básica, pues muestra que no todos los resultados se representan internamente del mismo modo [19], [74], [75], [76].

Pasos para Desarrollar la Solución

Los pasos para desarrollar la solución algorítmica de un problema permiten convertir un procedimiento matemático en una secuencia ordenada de operaciones ejecutables. En este ejercicio, la conversión se construye a partir de una iteración por reducción: en cada paso el número disminuye mediante división entera, mientras los residuos obtenidos se van acumulando como parte del resultado final. Esta organización permite al estudiante comprender que ciertos problemas no se resuelven mediante una única fórmula directa, sino a través de una secuencia de transformaciones sucesivas sobre el dato original.

1. Análisis del problema. Para comprender el problema conviene partir de algunos ejemplos pequeños y ejecutar manualmente las divisiones sucesivas. El algoritmo sigue una secuencia clara: primero se lee el número original, luego se obtiene su valor absoluto para trabajar con una magnitud no negativa durante la conversión, después se solicita la base de destino y se verifica que sea menor que 10. Una vez satisfechas estas condiciones, se inicializa una cadena vacía y se repite el siguiente proceso: calcular el residuo de dividir el número actual entre la base, convertir ese residuo en texto, concatenarlo al inicio del resultado parcial y reemplazar el número actual por el cociente entero de la división. Cuando el cociente ya no supera la base, el proceso concluye agregando el último dígito y, si el número original era negativo, anteponiendo el signo menos al resultado final. Esta descomposición facilita el aprendizaje porque separa con nitidez la etapa de validación, la etapa de reducción iterativa y la etapa de presentación del resultado [35], [53], [74].

Ejercicios de análisis manual con valores representativos. Antes de diseñar el algoritmo, es recomendable resolver manualmente algunos casos que permitan verificar el comportamiento de la división entera, el uso del residuo y la concatenación de dígitos.

■ Caso 1: Conversión de un valor positivo a base 2

1. Datos de entrada

- `numero = 13`
- `base = 2`

2. Resolución manual

$$13 \div 2 = 6 \text{ residuo } 1, \quad 6 \div 2 = 3 \text{ residuo } 0,$$

$$3 \div 2 = 1 \text{ residuo } 1, \quad 1 \div 2 = 0 \text{ residuo } 1$$

Los residuos leídos en orden inverso forman la representación final:

$$13_{10} = 1101_2$$

3. Resultado esperado

$$1101_2$$

4. Observación. Este caso permite verificar el funcionamiento elemental del método de divisiones sucesivas.

■ Caso 2: Conversión a base 8

1. Datos de entrada

- `numero = 100`
- `base = 8`

2. **Resolución manual**

$$100 \div 8 = 12 \text{ residuo } 4, \quad 12 \div 8 = 1 \text{ residuo } 4$$

$$1 \div 8 = 0 \text{ residuo } 1$$

Por lo tanto:

$$100_{10} = 144_8$$

3. **Resultado esperado**

$$144_8$$

4. **Observación.** Este caso muestra que el mismo procedimiento se aplica a cualquier base válida menor que 10.

■ **Caso 3: Número menor que la base**1. **Datos de entrada**

- numero = 5
- base = 7

2. **Resolución manual** Como el número ya es menor que la base, no se requieren iteraciones completas de división sucesiva; su representación en la base de destino es el mismo dígito.

$$5_{10} = 5_7$$

3. **Resultado esperado**

$$5_7$$

4. **Observación.** Este caso permite verificar la condición de parada cuando el valor actual ya no necesita reducción adicional.

■ **Caso 4: Número negativo**1. **Datos de entrada**

- numero = -25
- base = 2

2. **Resolución manual** Primero se trabaja con el valor absoluto:

$$25_{10} = 11001_2$$

Luego se antepone el signo:

$$-25_{10} = -11001_2$$

3. **Resultado esperado**

$$-11001_2$$

4. **Observación.** Este caso confirma la conveniencia de separar el tratamiento del signo del proceso de conversión de magnitud.

■ **Caso 5: Número cero**1. **Datos de entrada**

- numero = 0
- base = 3

2. **Resolución manual** El valor cero se representa como cero en cualquier base válida:

$$0_{10} = 0_3$$

3. **Resultado esperado**

$$0_3$$

4. **Observación.** Este caso exige que el algoritmo trate correctamente un valor cuyo proceso de reducción no genera residuos sucesivos relevantes.

■ **Caso 6: Base no válida**

1. **Datos de entrada**

- `numero = 18`
- `base = 12`

2. **Resolución manual** Como el ejercicio restringe la conversión a bases menores que 10, el algoritmo debe solicitar nuevamente la base.

3. **Resultado esperado**

4. *“Error: la base debe ser menor que 10.”*

5. **Observación.** Este caso obliga a incorporar una validación inicial del dominio permitido para la base [23], [24].

2. Identificación de datos de entrada. Los datos de entrada corresponden a la información que el algoritmo necesita recibir para ejecutar la conversión.

- **numero (entero):** valor original expresado en base 10.
- **base (entero):** base de destino a la cual se desea convertir el número.

La base debe validarse antes de iniciar la conversión, de modo que el procedimiento continúe únicamente cuando el valor ingresado pertenezca al dominio previsto. Asimismo, cada lectura debe estar precedida por un mensaje claro que indique al usuario qué dato se espera en cada etapa del proceso [7], [59].

3. Identificación de datos de salida. Los datos de salida son los resultados que el algoritmo entrega una vez concluido el procesamiento.

- **Representación convertida (cadena):** número equivalente al valor original expresado en la base seleccionada.
- **Mensaje informativo:** salida textual que presenta el número original, la base de destino y la representación final obtenida.

La salida combina un resultado textual con información contextual, lo que permite interpretar correctamente la conversión realizada y distinguir el sistema numérico de destino.

4. Identificación de cálculos intermedios (procesos intermedios). Los procesos intermedios son las operaciones necesarias para obtener la representación final del número en la nueva base.

- **Normalización del signo:** `num <- abs(numero)`. Permite trabajar con una magnitud no negativa durante la conversión.
- **Inicialización del resultado:** `resultado <- ""`. La representación final se construye progresivamente como cadena.
- **Cálculo del residuo:** `residuo <- num MOD base`. Cada residuo representa un dígito de la nueva base.
- **Concatenación del dígito:** `resultado <- concatenar(convertirATexto(residuo), resultado)`. El nuevo dígito se agrega al inicio de la cadena parcial.
- **Actualización del número:** `num <- num DIV base`. El cociente entero se convierte en el nuevo valor a procesar.

Estos pasos muestran con claridad cómo el problema se resuelve mediante una reducción iterativa del valor inicial, mientras se construye simultáneamente una representación textual ordenada de sus dígitos. Esta organización separa la parte aritmética de la parte de presentación y facilita la comprensión del procedimiento [35], [53].

5. Identificación de cálculos finales (procesos finales). Los procesos finales son aquellos que producen directamente la respuesta visible para el usuario.

- **Incorporación del último dígito:** cuando el número ya no requiere más divisiones, se agrega el valor restante al inicio del resultado.
- **Tratamiento del signo:** si el número original era negativo, se antepone el signo menos a la representación convertida.
- **Presentación del resultado:** impresión del número original, la base de destino y la representación final obtenida.

Estas operaciones completan la transición desde el valor decimal inicial hasta su equivalente textual en la nueva base, permitiendo cerrar el proceso con una salida interpretable y consistente.

6. Identificación de condiciones (decisiones). Las decisiones son expresiones lógicas que controlan el flujo del algoritmo.

- **Validación de la base:** controla que el valor ingresado sea menor que 10 y, didácticamente, mayor que 1.
- **Condición del ciclo de conversión:** determina si el valor actual aún requiere nuevas divisiones.
- **Condición del signo:** verifica si el número original era negativo para decidir si debe anteponerse el símbolo menos al resultado.

Estas decisiones separan con claridad la fase de validación de la fase de cálculo y hacen explícito que la conversión solo debe continuar cuando las entradas pertenecen al dominio previsto [35], [59].

7. Identificación de repeticiones o iteraciones. El algoritmo contiene dos estructuras repetitivas bien diferenciadas.

- **Repetición para la validación de la base:** mientras la base no pertenezca al dominio permitido, se solicita nuevamente.
- **Repetición para la conversión:** mientras el número actual siga siendo reducible por divisiones sucesivas, se calculan residuos y se actualiza el cociente.

La presencia de estas dos repeticiones muestra que el problema combina control de entrada con transformación progresiva del dato. Esta combinación resulta didácticamente útil porque enseña que un algoritmo puede contener distintos ciclos con propósitos claramente diferenciados [74], [75], [76].

8. Condiciones para finalizar el algoritmo. El algoritmo finaliza cuando:

- se ha ingresado una base válida;
- el número ha sido reducido completamente mediante divisiones sucesivas;
- se ha construido la cadena con la representación convertida;
- se ha presentado el resultado final con o sin signo negativo.

En consecuencia, la finalización del proceso depende tanto de la validez de la base ingresada como de la culminación ordenada de la reducción iterativa del número original.

Propuesta de Solución

La Figura 4.36 presenta una solución algorítmica estructurada para convertir un número entero en base 10 a otra base menor que 10. El diseño separa con claridad (i) la lectura del número original, (ii) la normalización de su magnitud mediante valor absoluto, (iii) la validación de la base de destino, (iv) la reducción iterativa por división entera, (v) la construcción textual del resultado a partir de residuos y (vi) la incorporación final del signo cuando corresponde. Esta organización favorece la comprensión del flujo completo del problema y permite apreciar cómo una operación matemática recurrente puede traducirse en un procedimiento algorítmico bien estructurado [59], [72].

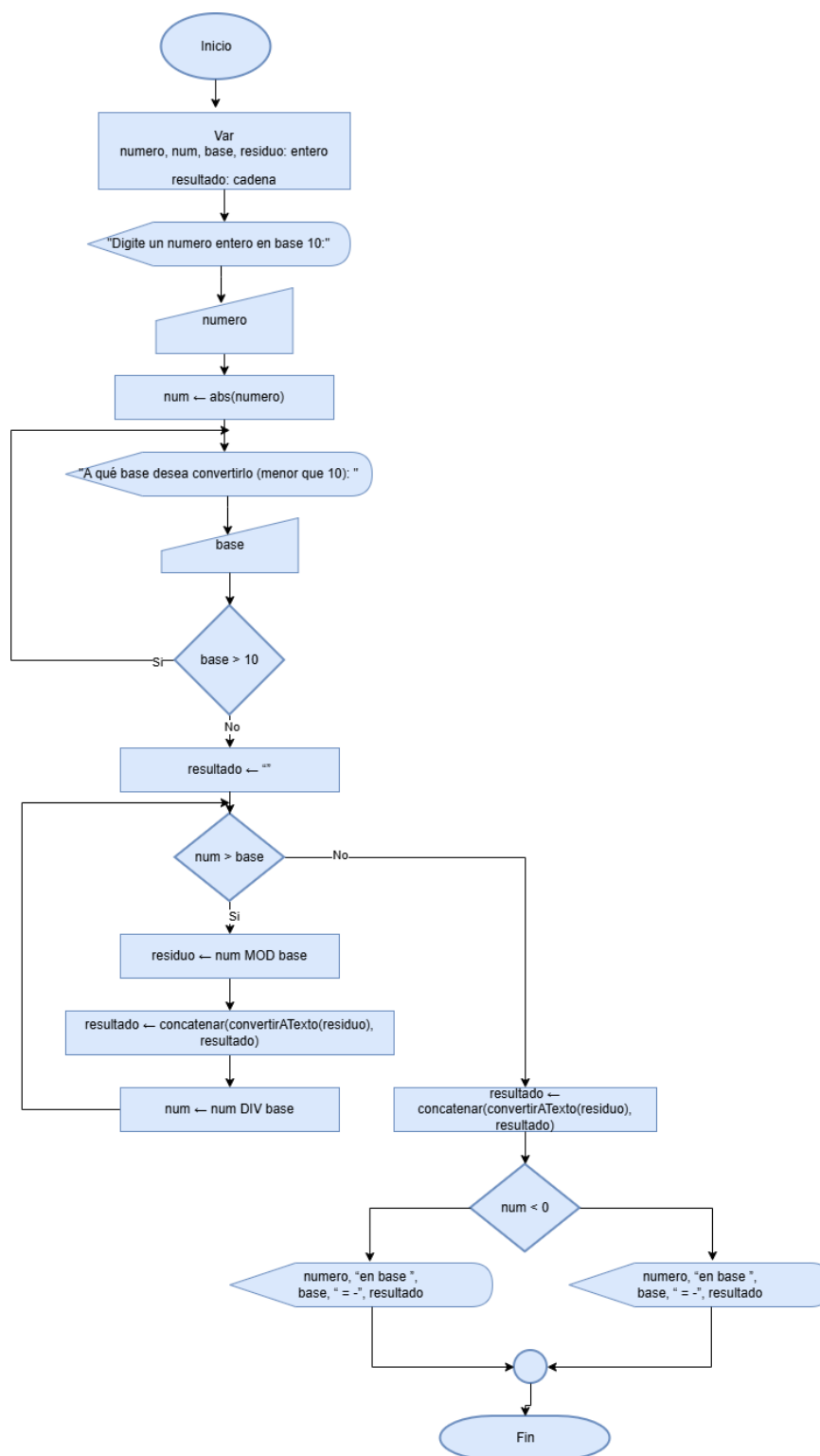


Figura 4.36: Conversión de un número entero en base 10 a otra base menor que 10 mediante divisiones sucesivas.

El Algoritmo 4.52 presenta una propuesta de solución estructurada para realizar esta conversión. En esta formulación se distinguen explícitamente la validación de la base, el tratamiento separado del signo, la construcción progresiva del resultado como cadena y el uso de las operaciones MOD y DIV para obtener los dígitos de la nueva base. Esta organización reduce la complejidad del razonamiento, ya que evita mezclar el proceso aritmético de reducción con la presentación final del número convertido y permite al estudiante reconocer con claridad cómo los residuos de las divisiones sucesivas se transforman en los dígitos de la nueva representación [53], [74].

```

1      Algoritmo ConversionBaseMenorQueDiez
2      1.      Var:
3      1.1.      numero, num, base, residuo: entero
4      1.2.      resultado: cadena
5
6      2.      Escribir "Digite un numero entero en base 10:"
7      3.      Leer numero
8
9      4.      num <- abs(numero)
10
11     5.      Escribir "A que base desea convertirlo (menor que 10):"
12     6.      Leer base
13     7.      Mientras base >= 10 O base <= 1 Hacer
14     7.1.      Escribir "Error: la base debe ser mayor que 1 y menor que
15     ↪ 10."
16     7.2.      Escribir "A que base desea convertirlo (menor que 10):"
17     7.3.      Leer base
18     8.      FinMientras
19
20     9.      resultado <- ""
21
22     10.     Si num = 0 Entonces
23     10.1.      resultado <- "0"
24     10.2.     SiNo
25     10.3.     Mientras num >= base Hacer
26     10.3.1.      residuo <- num MOD base
27     10.3.2.      resultado <- concatenar(convertirATexto(residuo),
28     ↪ resultado)
29     10.3.3.      num <- num DIV base
30     10.4.     FinMientras
31     10.5.     resultado <- concatenar(convertirATexto(num), resultado)

```



```

30      11.   FinSi
31
32      12.   Si numero < 0 Entonces
33      12.1.     resultado <- concatenar("-", resultado)
34      13.   FinSi
35
36      14.   Escribir numero, " en base ", base, " = ", resultado
37      FinAlgoritmo

```

Listing 4.52: Conversión de un entero decimal a una base menor que 10

El algoritmo presentado en el Listado de código 4.52 separa de manera explícita el tratamiento de la magnitud del número y el manejo del signo. Durante la conversión, el procedimiento trabaja con el valor absoluto del dato original para simplificar el cálculo de residuos y cocientes; una vez construida la representación en la nueva base, el signo negativo se incorpora solo si el número inicial era menor que cero. Esta organización mejora la claridad del flujo y evita mezclar la lógica aritmética de la conversión con la presentación final del resultado. Además, la validación previa de la base impide continuar con valores fuera del dominio previsto, lo que aporta coherencia al procedimiento general y hace la solución más comprensible en contextos de aprendizaje inicial [7], [23], [24], [59].

4.10. Estrategias para el aprendizaje técnico en IS

El aprendizaje técnico en la carrera de Ingeniería en Sistemas de Información (IS) exige transformar conocimientos declarativos (definiciones, conceptos, reglas) en habilidades procedimentales verificables (resolver problemas, diseñar algoritmos, implementar y depurar programas). En programación, dicha transformación no ocurre por exposición pasiva a ejemplos, sino por la construcción iterativa de soluciones y la retroalimentación sistemática sobre los errores, hasta estabilizar patrones de razonamiento y de control de flujo [7], [40], [59]. En consecuencia, las estrategias didácticas deben articular práctica deliberada, técnicas cognitivas de verificación (trazado, casos límite) y mecanismos de control de calidad personal (registro de fallos y refactorización incremental), alineados a principios de ingeniería de software y a la necesidad de mantenibilidad de las soluciones [23], [24], [80].

Resulta útil distinguir dos planos complementarios: (i) la adquisición de fundamentos para aprender a programar (lógica, estructuras de control, representación de datos), y (ii) la aplicación de estas habilidades al desarrollo de soluciones organizacionales propias de los sistemas de información (SI), donde los programas interactúan con bases de datos, servicios, reglas de negocio, seguridad y restricciones del entorno [1], [2], [41]. Las estrategias que se presentan a continuación se orientan al primer plano sin perder de vista el segundo: la destreza algorítmica se consolida cuando el estudiante puede justificar decisiones, anticipar fallos y validar requisitos de manera explícita, tal como se exige en proyectos reales [23], [40], [41].

4.10.1. Práctica deliberada

La práctica deliberada consiste en entrenar habilidades específicas con objetivos claros, tareas de dificultad controlada y retroalimentación inmediata, evitando la repetición mecánica sin aprendizaje [7], [40]. En programación introductoria, esto implica seleccionar micro-competencias observables (por ejemplo, construir un acumulador, formular una condición de parada, validar una entrada) y practicar cada una en problemas variados, de modo que el estudiante pueda reconocer el patrón subyacente más allá del enunciado [59], [61]. La práctica deliberada también favorece el tránsito desde soluciones informales hacia soluciones estructuradas, donde cada paso se puede rastrear y defender mediante reglas del lenguaje y del dominio del problema [23], [53].

En IS, la práctica deliberada debe incorporar desde etapas tempranas la noción de calidad: claridad, corrección, control de errores y posibilidad de mantenimiento. Incluso en ejercicios sencillos, el estudiante debe aprender a producir artefactos

legibles y verificables (pseudocódigo, algoritmos, pruebas), porque esa disciplina se traslada de forma directa al desarrollo de SI, donde los cambios son frecuentes y el software se mantiene durante largos periodos [24], [80], [81]. En consecuencia, practicar no equivale a “hacer muchos ejercicios”, sino a fortalecer la precisión del razonamiento y la consistencia del método.

Frecuencia y consistencia

La frecuencia se refiere a cuántas veces se practica, mientras que la consistencia describe la estabilidad del hábito y la regularidad del esfuerzo. En programación, la consistencia es crítica porque las habilidades se apoyan en esquemas mentales que se degradan si la exposición es esporádica: el estudiante puede recordar definiciones, pero pierde fluidez para construir soluciones paso a paso [7], [59]. Por ello, se recomienda una práctica distribuida y recurrente, con sesiones relativamente cortas pero frecuentes, que incluyan tanto resolución como verificación.

Para principiantes, la consistencia se fortalece mediante rutinas simples: (i) lectura comprensiva del problema, (ii) identificación explícita de entradas, salidas y restricciones, (iii) propuesta de casos de prueba, (iv) elaboración del algoritmo, y (v) verificación manual del comportamiento esperado. Este orden reduce la carga cognitiva, porque evita intentar programar sin haber estabilizado el modelo del problema [59], [61]. Además, promueve hábitos alineados con prácticas de ingeniería: validar primero el objetivo antes de optimizar o “codificar rápido” [23], [41].

En términos prácticos, la consistencia también exige variar el contexto sin variar el objetivo. Por ejemplo, si el objetivo es dominar condicionales anidados, los ejercicios deben cambiar el dominio (notas, nómina, inventario) manteniendo la estructura lógica esencial. Este enfoque favorece la transferencia: el estudiante aprende a identificar la estructura del problema y no solo a memorizar una solución particular [7], [61]. A medida que progresa, se incorporan restricciones realistas (validación, manejo de errores, datos faltantes), anticipando escenarios típicos de SI [1], [2].

Resolución guiada e independiente

La resolución guiada consiste en resolver problemas con apoyo explícito (plantillas, pasos, ejemplos análogos), mientras que la resolución independiente exige que el estudiante ejecute el proceso completo sin intervención. Ambas son necesarias: la primera establece el método y reduce ambigüedades; la segunda verifica autonomía y consolida el aprendizaje [7], [59]. En programación, la guía debe centrarse en el *proceso* (cómo analizar, cómo decidir, cómo verificar) más que en el resultado final, pues el riesgo común es que el estudiante copie una solución sin comprender la lógica que la sostiene.

Una estrategia efectiva es la “transferencia gradual de control”: al inicio, el docente o el material provee una estructura (por ejemplo, identificar entradas/salidas y proponer casos); luego, el estudiante completa partes del algoritmo; finalmente, produce el diseño completo y justifica cada decisión. Este esquema se alinea con la idea de construir soluciones verificables y trazables, donde cada requisito se puede mapear a un paso del algoritmo [23], [41]. En IS, esta trazabilidad se vuelve crucial cuando el problema incorpora reglas de negocio, validaciones y excepciones [1], [2].

En la resolución independiente, el criterio de logro no debe limitarse a “funciona”, sino a “funciona y se puede explicar”. Un estudiante que solo obtiene un resultado correcto, pero no puede justificar por qué el ciclo termina o por qué una condición cubre todos los casos, aún no domina la competencia. Por ello, se recomienda que cada solución independiente incluya una explicación estructurada del flujo y un conjunto mínimo de pruebas, incluyendo casos límite [53], [61]. Esta exigencia introduce tempranamente prácticas de calidad y reduce errores conceptuales que luego son costosos en proyectos de SI [24], [40].

Registro de errores

El registro de errores es un instrumento formativo para convertir fallos en conocimiento operativo. En programación, los errores rara vez son aleatorios: se agrupan en patrones (confusión de tipos, límites de bucle, inicializaciones faltantes, condiciones incompletas, manejo inadecuado de entradas). Cuando el estudiante documenta el error, su causa y su corrección, construye un repositorio personal de lecciones que acelera el aprendizaje y disminuye la reincidencia [23], [40]. Esta práctica es coherente con la ingeniería de software: la calidad se incrementa cuando se gestionan defectos y se aprende de ellos [24].

Para principiantes, el registro debe ser sencillo y estandarizado. Se recomienda anotar, al menos: (i) enunciado del problema, (ii) síntoma observado (salida incorrecta, bucle infinito, error de ejecución), (iii) condición que lo produce (datos específicos), (iv) causa identificada (por ejemplo, variable no inicializada), y (v) corrección aplicada. Cuando el error se asocia a datos concretos, se facilita la reproducción y, por tanto, la depuración [7], [59]. Adicionalmente, el registro permite detectar debilidades recurrentes y planificar práctica deliberada focalizada.

En niveles posteriores, el registro de errores se conecta con refactorización y pruebas. Si una corrección obliga a reestructurar el código o el algoritmo para hacerlo más claro y menos propenso a fallos, la refactorización se vuelve una extensión natural del aprendizaje técnico [33], [80], [81]. Así, el estudiante incorpora una visión de mejora continua: no solo reparar el fallo puntual, sino reducir la probabilidad de fallos futuros mediante mejor diseño, nombres más claros y estructura controlada [24], [51].

4.10.2. Estrategias cognitivas aplicadas

Las estrategias cognitivas aplicadas son técnicas explícitas para controlar el razonamiento durante la solución de problemas: ayudan a “ver” la lógica antes de programar, anticipar resultados y detectar inconsistencias. En programación introductoria, estas estrategias son esenciales porque el estudiante todavía no dispone de automatismos; por tanto, necesita métodos externos de verificación que reduzcan la incertidumbre [7], [59]. Entre las más relevantes se encuentran el trazado manual, las pruebas con casos límite y la explicación estructurada, las cuales se corresponden con prácticas de validación y verificación en ingeniería [23], [40].

En IS, estas estrategias también preparan para escenarios donde los datos no provienen únicamente del teclado, sino de archivos, bases de datos y servicios. Por ejemplo, un algoritmo correcto puede fallar si no considera valores faltantes, datos fuera de rango o condiciones de conectividad, lo cual obliga a pensar en restricciones y excepciones como parte del razonamiento, no como un añadido posterior [28], [29], [82]. En consecuencia, enseñar estrategias cognitivas significa enseñar a razonar con condiciones reales de operación, sin perder la claridad necesaria para principiantes.

Trazado manual

El trazado manual (también llamado simulación paso a paso) consiste en ejecutar el algoritmo “a mano” con un conjunto de datos concreto, registrando el valor de las variables en cada paso. Esta técnica obliga a explicitar el estado del programa y revela errores frecuentes: contadores mal inicializados, condiciones incorrectas, actualizaciones que no ocurren, o resultados que cambian en el momento equivocado [7], [59]. Para principiantes, el trazado manual es especialmente valioso porque convierte la ejecución —que en un computador es rápida e invisible— en una secuencia comprensible de estados.

Un trazado bien realizado requiere disciplina: se recomienda construir una tabla con columnas para variables relevantes (por ejemplo, acumuladores y contadores) y filas para cada iteración o punto de decisión. Además, debe incluir el “evento” que ocurre en cada paso: lectura, actualización, comparación. Esta forma de registro se vincula con la noción de invariante de iteración: al inicio de cada vuelta, ciertas condiciones deben mantenerse verdaderas (por ejemplo, “el contador indica cuántos datos ya se han acumulado”) [53]. Cuando el invariante se rompe, el trazado muestra exactamente dónde ocurrió.

En problemas con entrada externa (por ejemplo, lectura desde base de datos), el trazado manual también puede incluir estados del entorno: “conexión establecida”, “consulta ejecutada”, “resultado vacío”. Aunque estos estados no sean variables numéricas, ayudan a comprender el flujo del algoritmo y a justificar la necesidad de manejo de errores [28], [29]. Así, el trazado manual no solo sirve para cálculos aritméticos, sino para razonar sobre procesos de software donde el control de excepciones es parte del comportamiento correcto.

Pruebas con casos límite

Las pruebas con casos límite consisten en diseñar entradas que estresan las fronteras del dominio del problema: mínimos, máximos, ceros, valores negativos, cadenas vacías, colecciones de tamaño 1, datos faltantes o condiciones inválidas (por ejemplo, división por cero). Estos casos son decisivos porque muchos algoritmos parecen correctos en casos “normales”, pero fallan en fronteras donde cambian las condiciones lógicas [7], [61]. En programación, aprender a proponer casos límite es aprender a anticipar errores antes de codificar, lo cual reduce retrabajo y fortalece la calidad.

Desde la perspectiva de ingeniería de software, los casos límite actúan como una forma inicial de especificación operacional: describen cómo debe comportarse la solución bajo condiciones críticas. Esto se alinea con la validación de requisitos, ya que fuerza a explicitar qué ocurre cuando una condición no permite el cálculo (por ejemplo, “no se puede calcular promedio si $N = 0$ ”) [23], [41]. En IS, donde las reglas de negocio suelen incluir excepciones (límites, umbrales, autorizaciones), esta práctica se vuelve todavía más relevante [1], [2].

En contextos de datos reales, los casos límite deben considerar también el origen de los datos. Si la entrada proviene de teclado, el caso límite puede ser “usuario ingresa un texto cuando se espera un número”. Si proviene de archivo, puede

ser “archivo vacío” o “línea con formato incorrecto”. Si proviene de base de datos, un caso límite típico es “consulta sin resultados” (conjunto vacío) o “campo nulo”. Estos escenarios requieren decisiones: mostrar mensajes, detener el proceso, solicitar reingreso o registrar el error [28], [29]. Por tanto, el aprendizaje de casos límite desarrolla la capacidad de diseñar algoritmos robustos, no solo correctos en condiciones ideales.

En problemas con aritmética de punto flotante, existen además casos límite asociados a representación numérica y redondeo. Comparaciones como “igual a cero” pueden ser inestables si el valor calculado es un aproximado debido a precisión finita, por lo que conviene advertir al estudiante que los resultados computacionales pueden diferir del cálculo exacto y que las condiciones deben diseñarse con prudencia [11], [12]. Esta advertencia no pretende complejizar innecesariamente, sino introducir una idea fundamental: en computación, los números reales se representan con aproximaciones y ello impacta decisiones y resultados.

Explicación estructurada

La explicación estructurada consiste en presentar la solución como una secuencia ordenada de componentes: problema, entradas, salidas, procesos intermedios, decisiones, repeticiones y condición de término. Para principiantes, esta estructura funciona como un “andamiaje” que evita omisiones y ayuda a justificar cada parte del algoritmo [7], [59]. Además, mejora la comunicación técnica: una solución explicada de forma estructurada es más fácil de revisar, corregir y traducir a un lenguaje de programación.

En términos de calidad, la explicación estructurada introduce la idea de especificación y trazabilidad. Cuando un estudiante afirma “esto es lo que entra” y “esto es lo que sale”, establece límites claros del algoritmo. Cuando indica “estas son las decisiones”, identifica el núcleo lógico que controla el flujo. Esta forma de describir se alinea con prácticas formales de requisitos y con la necesidad de que cada requisito tenga una representación explícita en la solución [23], [41]. En IS, donde los requisitos organizacionales suelen ser extensos, esta disciplina reduce interpretaciones ambiguas y mejora la consistencia del diseño [1], [2].

La explicación estructurada también favorece el aprendizaje de paradigmas y estructuras de control. Al describir una solución, el estudiante reconoce si está usando secuencia, selección o repetición, y puede relacionar estas formas con construcciones del lenguaje (por ejemplo, `if`, `while`, `for`) [59], [61]. De esta manera, el paso de “pensar en algoritmo” a “escribir código” se vuelve una traducción controlada, no un salto improvisado.

Por último, la explicación estructurada facilita la mejora continua: al revisar una solución, el estudiante puede localizar con precisión dónde ajustar (por ejemplo, en la validación de entradas o en la condición de parada) sin reescribir todo. Esta práctica es coherente con la idea de mantenimiento y refactorización como parte natural del ciclo de vida del software, incluso en ejercicios académicos [24], [33], [80]. En consecuencia, explicar no es un requisito “redaccional”, sino un mecanismo técnico para pensar mejor, verificar mejor y construir soluciones más confiables.

Cuadro 4.7: Funciones de cadena predefinidas en UPSAM 2.0

Función	Descripción	Tipo de retorno	Ejemplo
longitud (c)	Devuelve el número de caracteres de una cadena.	Entero	longitud (nombre)
subcadena (c, i, l)	Extrae una subcadena a partir de la posición <i>i</i> con longitud <i>l</i> .	Cadena	subcadena ("Hola", 1, 2)
concatenar (c1, c2)	Une dos cadenas en una sola cadena resultante.	Cadena	concatenar ("Hola", "Mundo")
reemplazar (c, sub1, sub2)	Reemplaza todas las apariciones de una subcadena por otra dentro de una cadena.	Cadena	reemplazar ("Hola mundo", "mundo", "UPSAM")
contar (c, sub)	Cuenta cuántas veces aparece una subcadena o carácter dentro de una cadena.	Entero	contar ("banana", "a")
posicion (c, sub)	Devuelve la posición de la primera aparición de una subcadena en una cadena.	Entero	posicion ("programacion", "gra")
ultimaPosicion	Devuelve la posición de la última aparición de una subcadena en una cadena.	Entero	ultimaPosicion("banana", "a")
mayusculas (c)	Convierte todos los caracteres de una cadena a letras mayúsculas.	Cadena	mayusculas ("hola")
minusculas (c)	Convierte todos los caracteres de una cadena a letras minúsculas.	Cadena	minusculas ("HOLA")
igual (c1, c2)	Compara dos cadenas y devuelve verdadero si son idénticas.	Lógico	igual("abc", "abc")
empiezaCon (c, sub)	Verifica si una cadena comienza con una subcadena dada.	Lógico	empiezaCon ("algoritmo", "algo")
terminaCon (c, sub)	Verifica si una cadena finaliza con una subcadena dada.	Lógico	terminaCon ("algoritmo", "ritmo")

Cuadro 4.8: Procedimientos de entrada y salida en UPSAM 2.0

Procedimiento	Uso	Ejemplo
leer	Entrada de datos	<code>leer(edad)</code>
escribir	Salida de datos	<code>escribir("Resultado:", total)</code>

Parte II

Desarrollo de aplicaciones para Sistemas de Información

Capítulo 5

Lenguajes de Programación y Desarrollo de Aplicaciones

La formación en programación dentro de la carrera de Ingeniería en Sistemas de Información no se limita al aprendizaje de la sintaxis de un lenguaje o a la construcción de pequeños programas aislados. En el contexto profesional, el objetivo es comprender cómo las soluciones algorítmicas se convierten en aplicaciones que apoyan procesos organizacionales, gestionan datos y contribuyen a la toma de decisiones. Por esta razón, el tránsito desde el algoritmo hacia el desarrollo de aplicaciones constituye un paso fundamental en la formación técnica del estudiante [2], [23], [24].

En capítulos anteriores se ha trabajado con algoritmos, diagramas de flujo y razonamientos paso a paso que permiten resolver problemas de manera estructurada. Sin embargo, en un entorno organizacional real, estas soluciones deben implementarse en lenguajes de programación concretos, ejecutarse en plataformas tecnológicas específicas y manejar datos provenientes de usuarios, archivos o bases de datos. En consecuencia, aprender a programar implica comprender tanto la lógica algorítmica como los mecanismos técnicos mediante los cuales dicha lógica se transforma en software funcional dentro de un sistema de información [1], [40], [41].

Desde la perspectiva de la ingeniería de software, los programas no son únicamente artefactos computacionales que realizan cálculos. Constituyen componentes de sistemas más amplios que integran interfaces de usuario, servicios, bases de datos y reglas de negocio. Por ello, la programación debe abordarse desde una visión sistemática que conecte el diseño algorítmico con el desarrollo de aplicaciones organizacionales, manteniendo principios de claridad, mantenibilidad y confiabilidad [23], [24], [33], [51].

Los lenguajes de programación constituyen sistemas formales diseñados para expresar algoritmos, estructuras de datos y reglas de transformación de información de manera que una máquina pueda ejecutar, interpretar o traducir dichas descripciones. En términos conceptuales, un lenguaje no solo aporta un vocabulario sintáctico, sino también una manera de modelar problemas, organizar instrucciones y representar procesos computacionales. Esta distinción resulta fundamental, porque el aprendizaje de la programación no depende únicamente de memorizar palabras reservadas, sino de comprender cómo un determinado lenguaje permite descomponer una situación, identificar datos, establecer operaciones válidas y producir resultados verificables [35], [75], [83].

La diversidad de lenguajes existente en la informática no obedece a una proliferación arbitraria, sino a la evolución histórica de necesidades técnicas distintas. Algunas aplicaciones requieren cercanía al hardware, otras privilegian la legibilidad, otras buscan robustez para grandes sistemas, mientras que otras priorizan el manejo declarativo de datos o el razonamiento matemático sobre funciones. Por ello, el estudio comparado de los lenguajes permite reconocer que programar no equivale a escribir instrucciones de una sola manera, sino a seleccionar una representación adecuada para cada clase de problema [35], [83], [84].

Con el fin de mantener la comparación conceptual bajo un mismo referente, se empleará a lo largo del capítulo un problema común: procesar las calificaciones de un curso, calcular el promedio, identificar la nota máxima y mínima, contar el número de aprobaciones y emitir un reporte final. Este problema se conservará deliberadamente en todos los enfoques, de modo que las diferencias observables provengan del paradigma y del lenguaje utilizados, y no de cambios en el dominio del

problema. De esta forma, el análisis permitirá reconocer cómo una misma necesidad puede expresarse mediante secuencias imperativas, objetos colaborativos, funciones puras o consultas declarativas [73], [75], [85].

5.1. Clasificación general de los lenguajes de programación

La clasificación de los lenguajes de programación puede realizarse según diversos criterios: nivel de abstracción, paradigma dominante, estrategia de ejecución, dominio de aplicación o cercanía con la arquitectura física del computador. En una introducción sistemática resulta especialmente útil distinguir, en primer lugar, entre lenguajes de bajo nivel y lenguajes de alto nivel. Esta separación permite comprender que algunos lenguajes se aproximan al funcionamiento interno del procesador, mientras que otros operan en un plano de abstracción más cercano a los conceptos del problema que se desea resolver [9], [34], [35].

La diferencia entre bajo y alto nivel no implica una jerarquía de calidad, sino una diferencia de propósito. Un lenguaje de bajo nivel permite control preciso sobre registros, memoria e instrucciones, lo cual resulta indispensable en sistemas embebidos, controladores, firmware y partes críticas de sistemas operativos. Un lenguaje de alto nivel, en cambio, permite describir reglas de negocio, procesos administrativos, cálculos científicos o aplicaciones interactivas con una sintaxis más expresiva y con un esfuerzo cognitivo menor respecto de los detalles físicos de la máquina. Ambos tipos son necesarios; lo importante es comprender qué problemas resuelven mejor y qué exigencias imponen en el proceso de programación [9], [34], [83].

5.1.1. Lenguajes de bajo nivel y lenguajes de alto nivel

La distinción entre bajo nivel y alto nivel constituye una puerta de entrada decisiva para comprender cómo los programas se relacionan con el hardware y con las abstracciones propias del software. Un mismo algoritmo puede imaginarse como una secuencia de instrucciones que manipulan directamente registros y direcciones de memoria, o como una solución expresada en términos de variables, listas, clases, objetos, funciones o consultas. La diferencia entre ambas aproximaciones no reside únicamente en la sintaxis, sino en el grado de proximidad respecto del procesador y en la carga conceptual que se delega al compilador, al intérprete o al entorno de ejecución [34], [35], [83].

Desde una perspectiva formativa, esta clasificación permite aclarar por qué algunos lenguajes exigen describir operaciones elementales de forma casi mecánica, mientras que otros permiten concentrarse en la estructura lógica del problema. Cuando se comprende esta diferencia, resulta más fácil interpretar por qué un programa escrito en ensamblador suele ser más extenso y detallado que uno escrito en Java o Python, o por qué una misma tarea puede resolverse mediante pocas instrucciones declarativas en SQL y, sin embargo, requerir múltiples ciclos, condiciones y variables acumuladoras en un enfoque imperativo clásico [9], [34], [84].

Lenguajes de bajo nivel

Los lenguajes de bajo nivel se caracterizan por su proximidad con la máquina física. En este tipo de lenguajes, las operaciones representadas se corresponden de manera mucho más directa con instrucciones ejecutables por el procesador. Esta cercanía proporciona un control detallado sobre memoria, registros, direccionamiento y flujo de instrucciones, pero también incrementa la complejidad de la escritura, lectura y mantenimiento del programa. En otras palabras, cuanto menor es la abstracción, mayor es la responsabilidad de describir de forma explícita aspectos que en otros lenguajes son resueltos automáticamente por compiladores o entornos de ejecución [9], [34], [35].

En este nivel, cada decisión del programa suele estar fuertemente vinculada con la arquitectura sobre la cual se ejecutará. Eso significa que las instrucciones válidas, los registros disponibles y hasta el orden en que ciertos datos deben ubicarse en memoria dependen de la familia de procesadores utilizada. Por esa razón, los lenguajes de bajo nivel no suelen emplearse para aplicaciones informacionales generales, sino para contextos donde el rendimiento, el control de hardware o la interacción con dispositivos requieren precisión extrema [9], [34].

Lenguaje máquina

El lenguaje máquina es la forma más básica de representación de un programa. Cada instrucción se expresa directamente como una secuencia binaria que el procesador puede decodificar y ejecutar. Estas secuencias no contienen palabras comprensibles para la escritura ordinaria, sino combinaciones de bits que representan códigos de operación y operandos. Por esta razón, el lenguaje máquina resulta difícil de leer y escribir, ya que cada cambio de valor o de instrucción implica manipular patrones binarios precisos [9], [34].

Un programa en lenguaje máquina puede concebirse como una sucesión de bloques binarios, donde una parte de los bits indica la operación a realizar y otra parte el dato o la dirección sobre la cual se aplica esa operación. En términos pedagógicos, este hecho permite comprender una idea fundamental: el procesador no “entiende” palabras, sino patrones binarios. Todo lenguaje de mayor nivel termina siendo traducido, directa o indirectamente, a este tipo de representación.

A manera de ejemplo conceptual, puede considerarse un programa extremadamente simple cuyo objetivo sea cargar un valor numérico en un registro y luego detener la ejecución. Una representación binaria simplificada podría escribirse de la siguiente manera:

```
1      10110000 00000101
2      11110100
3
```

Listing 5.1: Ejemplo conceptual de instrucciones en lenguaje máquina

En esta representación conceptual, la primera línea puede interpretarse como “cargar el valor 5 en un registro” y la segunda como “detener la ejecución”. No se trata aquí de memorizar una arquitectura específica, sino de reconocer que el programa se escribe literalmente como un conjunto de bits. La escritura, por tanto, no se realiza mediante palabras clave como `if`, `while` o `class`, sino mediante secuencias binarias cuya semántica depende del conjunto de instrucciones del procesador [9], [34].

Para relacionar este nivel con el problema común de las calificaciones, basta advertir que operaciones aparentemente sencillas, como sumar notas o comparar si una nota supera un umbral, requerirían múltiples instrucciones binarias: cargar datos desde memoria, compararlos, saltar a otra dirección si se cumple una condición y almacenar resultados intermedios. Esto demuestra por qué el lenguaje máquina, aunque fundamental para comprender la base material de la computación, no es adecuado para expresar problemas de negocio o algoritmos de mediana complejidad en forma directa.

Lenguaje ensamblador

El lenguaje ensamblador representa un paso de abstracción respecto del lenguaje máquina. En lugar de escribir secuencias de bits de manera directa, se emplean mnemónicos simbólicos que representan instrucciones del procesador, como `MOV`, `ADD`, `CMP` o `JMP`. Esto no elimina la dependencia con la arquitectura, pero sí mejora sustancialmente la legibilidad del programa [9], [34], [35].

El valor pedagógico del ensamblador reside en que permite ver con relativa claridad cómo las operaciones abstractas terminan convirtiéndose en acciones concretas sobre registros y memoria. Mientras que en un lenguaje de alto nivel basta escribir una asignación como `total = total + nota;`, en ensamblador dicha operación requiere cargar el valor de `total`, sumar `nota`, y volver a almacenar el resultado. Cada paso se vuelve explícito.

Lenguajes representativos de este nivel.

- **x86 Assembly:** utilizado históricamente en computadores personales basados en procesadores Intel y AMD.
- **ARM Assembly:** dominante en dispositivos móviles, sistemas embebidos y numerosos entornos contemporáneos.
- **MIPS Assembly:** ampliamente usado en contextos educativos para explicar organización de computadores.
- **RISC-V Assembly:** cada vez más relevante en investigación, enseñanza y diseños abiertos de arquitectura.

En el listado 5.2 ejemplo ensamblador simplificado ilustra la idea de leer dos valores almacenados en memoria, sumarlos y guardar el resultado:

```
1      section .data
2          nota1 db 8
3          nota2 db 7
4          total db 0
5
6      section .text
7          global _start
8
9      _start:
10         mov al, [nota1]    ; cargar la primera nota
11         add al, [nota2]    ; sumar la segunda nota
12         mov [total], al    ; guardar el resultado
13
14         mov eax, 1         ; llamada al sistema para finalizar
15         xor ebx, ebx
16         int 0x80
```

Listing 5.2: Ejemplo simple en ensamblador x86

En este programa, la línea `mov al, [nota1]` indica que el valor almacenado en la posición `nota1` se copia al registro `AL`. Luego, `add al, [nota2]` suma al contenido del registro el valor almacenado en `nota2`. Por último, `mov [total], al` devuelve el resultado a memoria. El ejemplo muestra con claridad que el ensamblador obliga a pensar en registros, celdas de memoria y pasos elementales de cálculo, lo que explica tanto su potencia como su dificultad para programas extensos [9], [34].

Si el mismo problema común de las calificaciones se escribiera íntegramente en ensamblador, sería necesario definir regiones de memoria para almacenar notas, contadores y resultados, usar instrucciones de comparación como `CMP`, emplear saltos condicionales como `JG` o `JL`, e implementar bucles mediante etiquetas y saltos. Esto aclara por qué el ensamblador es útil para comprender el funcionamiento interno del computador, pero no suele ser la opción apropiada para aplicaciones administrativas o académicas.

5.1.2. Lenguajes de alto nivel

Los lenguajes de alto nivel se diseñan para expresar soluciones de forma más cercana al razonamiento humano y más alejada de los detalles eléctricos o arquitectónicos del procesador. En ellos, la sintaxis se construye mediante estructuras más legibles, palabras reservadas comprensibles, tipos de datos abstractos, mecanismos de modularización y bibliotecas reutilizables. Esta abstracción reduce drásticamente la cantidad de detalles operativos que deben describirse manualmente [35], [83], [84].

El paso a un lenguaje de alto nivel permite concentrar la atención en preguntas sustanciales: qué datos se necesitan, qué transformaciones deben aplicarse, cómo estructurar la lógica de la solución y qué formato debe tener la salida. De este modo, la programación deja de ser una sucesión de manipulaciones de registros y se convierte en una actividad de modelado de problemas. Por ello, los lenguajes de alto nivel dominan en el desarrollo de sistemas empresariales, aplicaciones web, aplicaciones móviles, análisis de datos, software científico y entornos de automatización [35], [83].

Conviene destacar que “alto nivel” no significa “simple” en sentido absoluto. Significa, más bien, que el lenguaje abstraer

muchos detalles de bajo nivel y ofrece construcciones más expresivas. Aun así, cada lenguaje de alto nivel adopta decisiones de diseño propias: algunos enfatizan la orientación a objetos, otros la sencillez sintáctica, otros la concurrencia, otros la seguridad de tipos, y otros la escritura declarativa de consultas o transformaciones.

Lenguajes representativos de alto nivel. Los lenguajes de alto nivel no forman un conjunto homogéneo, sino una familia amplia de herramientas diseñadas para atender necesidades distintas dentro del desarrollo de software. Algunos priorizan la eficiencia y la cercanía relativa al sistema, otros favorecen la expresividad sintáctica, otros se orientan a ecosistemas empresariales, científicos o web, y otros destacan por sus mecanismos de seguridad, tipado o concurrencia. Por ello, antes de enumerarlos conviene advertir que su comparación no debe hacerse únicamente por popularidad o facilidad aparente, sino por el tipo de problemas que permiten modelar con mayor claridad, por las abstracciones que ofrecen y por el contexto técnico en el que suelen emplearse [35], [83], [84].

- **C:** lenguaje de alto nivel cercano al sistema, ampliamente utilizado en compiladores, sistemas operativos y software de infraestructura. Para una comprensión inicial del manejo de memoria y estructuras básicas, el lector puede revisar obras introductorias como [86]. Asimismo, resulta conveniente explorar ejemplos prácticos sobre punteros y gestión de memoria dinámica para consolidar los fundamentos.
- **Pascal:** lenguaje históricamente asociado a la enseñanza de programación estructurada por su sintaxis organizada. Se recomienda consultar textos clásicos como [13] para comprender los principios de diseño estructurado. Este lenguaje permite afianzar conceptos como control de flujo y modularidad de manera clara.
- **Java:** lenguaje orientado a objetos, fuertemente tipado y muy utilizado en sistemas empresariales, servicios y aplicaciones Android. Para profundizar en sus fundamentos, el lector puede apoyarse en [87]. También es recomendable estudiar conceptos como clases, herencia y manejo de excepciones mediante ejemplos prácticos.
- **Python:** lenguaje multiparadigma que privilegia la legibilidad y la rapidez de desarrollo. Para iniciar, se sugiere revisar materiales como [88], que presentan el lenguaje de forma progresiva. Además, explorar scripts sencillos ayuda a comprender estructuras de control y tipos de datos dinámicos.
- **C#:** lenguaje moderno de la plataforma .NET, con amplio uso en aplicaciones empresariales, servicios y desarrollo de interfaces. El lector puede apoyarse en [89] para comprender su sintaxis y características. Se aconseja practicar con aplicaciones simples para familiarizarse con el entorno .NET.
- **JavaScript:** lenguaje central en el desarrollo web del lado del cliente y, con Node.js, también del lado del servidor. Para ampliar su comprensión, se puede revisar [90]. Es recomendable experimentar con ejemplos en el navegador para entender el modelo de ejecución y los eventos.
- **Kotlin:** lenguaje moderno de la JVM, con fuerte presencia en desarrollo Android. Para iniciarse, el lector puede consultar la documentación oficial y guías prácticas como [91]. Resulta útil comparar su sintaxis con Java para identificar mejoras en seguridad y concisión.
- **Swift:** lenguaje orientado a plataformas Apple, con énfasis en seguridad y expresividad. Se sugiere revisar [92] como referencia oficial. Explorar pequeños proyectos permite comprender conceptos como opcionales y tipado seguro.
- **Ruby:** lenguaje orientado a objetos, reconocido por su sintaxis expresiva y su empleo en desarrollo web. Para comprender su filosofía, el lector puede apoyarse en [93]. Es recomendable analizar ejemplos en Ruby on Rails para observar su aplicación práctica.
- **Go:** lenguaje compilado de sintaxis sobria, orientado a concurrencia y herramientas de infraestructura. Para profundizar, se sugiere revisar [94]. Asimismo, resulta pertinente estudiar el uso de goroutines y canales para comprender su modelo concurrente.

La comprensión de estos lenguajes mejora cuando se observa cómo un mismo problema puede implementarse en diferentes estilos manteniendo el mismo objetivo computacional. A continuación se presentan dos ejemplos de alto nivel sobre el problema común de las calificaciones. El primero se formula en Python, con énfasis en la legibilidad y el trabajo directo con listas. El segundo se formula en JavaScript, mostrando un estilo ampliamente utilizado en aplicaciones web y automatización.

Ejemplo en Python

Python ofrece una sintaxis directa y una colección de funciones integradas que permiten expresar con claridad operaciones frecuentes sobre secuencias. En el problema de las calificaciones, la lista de notas puede almacenarse como una estructura homogénea y luego procesarse mediante recorridos simples y funciones como `sum()`, `max()` y `min()`. Esta combinación hace visible la lógica del problema sin exigir demasiada sintaxis accesoria [95], [96], [97].

```
1  def procesar_calificaciones(notas):
2      if not notas:
3          print("No existen calificaciones registradas.")
4      return
5
6      promedio = sum(notas) / len(notas)
7      maxima = max(notas)
8      minima = min(notas)
9      aprobados = len([n for n in notas if n >= 7.0])
10
11     print("=== REPORTE DE CALIFICACIONES ===")
12     print(f"Notas registradas: {notas}")
13     print(f"Promedio: {promedio:.2f}")
14     print(f"Nota maxima: {maxima:.2f}")
15     print(f"Nota minima: {minima:.2f}")
16     print(f"Cantidad de aprobados: {aprobados}")
17     print(f"Cantidad de reprobados: {len(notas) - aprobados}")
18
19     notas_curso = [8.5, 6.2, 9.1, 5.8, 7.4, 10.0, 6.9]
20
21     procesar_calificaciones(notas_curso)
```

Listing 5.3: Procesamiento de calificaciones en Python

En este código, la expresión `sum(notas) / len(notas)` sintetiza en una línea una idea que en lenguajes más cercanos al hardware requeriría un acumulador y un contador explícitos. De forma análoga, la construcción `[n for n in notas if n >= 7.0]` permite contar aprobaciones mediante una lista filtrada. Estas características hacen de Python un lenguaje idóneo para introducir procesamiento de colecciones y reglas simples de negocio.

Ejemplo en JavaScript

JavaScript combina un modelo de programación flexible con amplio soporte para arreglos, funciones y objetos. Su presencia en navegadores y servidores lo convierte en un lenguaje particularmente relevante para el desarrollo actual. Sobre el mismo problema, es posible construir una solución apoyada en recorridos y acumulaciones, pero con una sintaxis diferente a la de Python [83], [98].

```
1  function procesarCalificaciones(notas) {
2      if (notas.length === 0) {
3          console.log("No existen calificaciones registradas.");
4          return;
5      }
6
7      let suma = 0;
8      let maxima = notas[0];
9      let minima = notas[0];
10     let aprobados = 0;
11
12     for (let i = 0; i < notas.length; i++) {
13         suma += notas[i];
14
15         if (notas[i] > maxima) {
16             maxima = notas[i];
17         }
18
19         if (notas[i] < minima) {
20             minima = notas[i];
21         }
22
23         if (notas[i] >= 7.0) {
24             aprobados++;
25         }
26     }
27
28     let promedio = suma / notas.length;
29
30     console.log("=== REPORTE DE CALIFICACIONES ===");
31     console.log("Notas registradas:", notas);
32     console.log("Promedio:", promedio.toFixed(2));
33     console.log("Nota maxima:", maxima.toFixed(2));
34     console.log("Nota minima:", minima.toFixed(2));
35     console.log("Cantidad de aprobados:", aprobados);
36     console.log("Cantidad de reprobados:", notas.length - aprobados);
```

```
37     }  
38  
39     const notasCurso = [8.5, 6.2, 9.1, 5.8, 7.4, 10.0, 6.9];  
40     procesarCalificaciones(notasCurso);
```

Listing 5.4: Procesamiento de calificaciones en JavaScript

La instrucción `for (let i = 0; i < notas.length; i++)` expresa el recorrido de la colección, mientras que la condición `if (notas[i] >= 7.0)` incorpora una regla simple de aprobación. El ejemplo permite apreciar que los lenguajes de alto nivel pueden compartir estructuras de pensamiento semejantes, aunque su sintaxis y sus bibliotecas difieran. Esta constatación resulta importante porque aclara que el aprendizaje sólido no depende de una sola notación, sino del dominio de ideas computacionales transferibles.

5.2. Paradigma imperativo

El paradigma imperativo organiza la solución como una secuencia de instrucciones que modifican un estado. Su rasgo distintivo es que el programa avanza mediante órdenes explícitas: asignar, comparar, repetir, sumar, decrementar, detener. En este enfoque, la solución se concibe como un proceso que transforma el contenido de variables a lo largo del tiempo hasta alcanzar un resultado final. Por ello, gran parte de la claridad del programa depende de la correcta gestión del estado y del orden de ejecución [35], [83], [99].

El enfoque imperativo conserva gran relevancia porque muchas arquitecturas mentales iniciales sobre la programación se construyen precisamente a partir de la idea de ejecutar pasos en orden. La noción de variable como contenedor de datos, el uso de ciclos para recorrer estructuras y el empleo de condiciones para decidir caminos de ejecución forman parte del núcleo de este paradigma. En consecuencia, una comprensión rigurosa del mismo resulta decisiva incluso cuando posteriormente se adopten estilos orientados a objetos, funcionales o declarativos.

5.2.1. Idea central

La esencia del paradigma imperativo consiste en expresar *cómo* debe ejecutarse una tarea. En lugar de describir únicamente el resultado deseado, se especifica la secuencia concreta de operaciones que lo hará posible. Esta secuencia puede incluir la lectura de datos, la inicialización de variables, la actualización paso a paso de acumuladores, la comparación de valores, la repetición de operaciones dentro de bucles y la presentación del resultado final [35], [83].

La importancia de esta idea no radica únicamente en su uso práctico, sino en que ofrece un modelo claro de causalidad computacional. Cada instrucción produce un efecto concreto sobre el estado del programa, y ese efecto condiciona las instrucciones siguientes. Por ello, el paradigma imperativo permite comprender con especial nitidez conceptos como trazabilidad, flujo de control y dependencia entre operaciones.

En este paradigma pueden distinguirse tres patrones fundamentales: **inicialización**, **actualización** y **verificación**. La inicialización es indispensable porque todo cálculo requiere un punto de partida definido. Sin una suma inicial, no puede existir acumulación; sin un índice inicial, no puede existir recorrido; sin una variable base para máximo o mínimo, no puede existir comparación significativa. La actualización es fundamental porque los resultados parciales deben cambiar conforme el programa procesa nuevos datos. Finalmente, la verificación resulta necesaria porque el programa necesita decidir si debe continuar, cambiar de camino o finalizar. Estos tres patrones son esenciales porque permiten estructurar prácticamente cualquier algoritmo imperativo de forma coherente y predecible [75], [100].

5.2.2. Cómo abordar un problema en el paradigma imperativo

Abordar un problema en el paradigma imperativo exige partir de una descomposición ordenada. Primero se identifica con claridad qué datos entran al programa y qué resultados deben producirse. Después se determinan las variables necesarias

para almacenar estados intermedios. A continuación se decide el orden de las operaciones: qué debe calcularse primero, qué debe repetirse y qué condiciones alteran el flujo normal. Solo después de esa planificación conviene escribir el código [35], [83].

En términos generales, este abordaje puede formularse en una secuencia de razonamiento: identificar entradas, definir variables de control y de resultado, establecer valores iniciales, diseñar la estructura de repetición adecuada, introducir verificaciones dentro del recorrido y consolidar los resultados finales. Este procedimiento no depende de un lenguaje concreto; constituye una manera de pensar propia del paradigma.

Particularizando el problema de las calificaciones, el razonamiento imperativo puede describirse así. Se parte de una colección de notas. Se declara una variable para la suma total, otra para el conteo de aprobaciones y otras dos para conservar la nota máxima y mínima. Se inicializan dichos valores de manera coherente. Luego se recorre la colección. En cada iteración se actualiza la suma, se compara la nota actual con los extremos y se verifica si la nota cumple la condición de aprobación. Al concluir el recorrido, se calcula el promedio y se presenta el reporte. Este esquema permite ver con claridad cómo un problema general se convierte en una serie ordenada de transformaciones de estado.

5.2.3. Lenguajes representativos del paradigma imperativo

Históricamente, el paradigma imperativo ha estado en la base de numerosos lenguajes de programación. Su expansión está estrechamente relacionada con los primeros compiladores y con la necesidad de expresar algoritmos secuenciales de manera comprensible y verificable. Muchos lenguajes posteriores incorporaron otros paradigmas, pero conservaron una base imperativa debido a su utilidad para describir cálculos paso a paso y controlar flujos de ejecución [35], [83].

Entre los lenguajes imperativos clásicos y contemporáneos se encuentran los siguientes:

- **Pascal:** diseñado con fines de claridad estructural y ampliamente asociado a la enseñanza de algoritmos.
- **Fortran:** uno de los lenguajes más influyentes en cálculo científico y computación numérica.
- **Ada:** lenguaje con fuerte énfasis en robustez, tipado y sistemas críticos.
- **Basic / Visual Basic:** históricamente empleado en entornos educativos y aplicaciones de oficina.
- **Go:** aunque más moderno y multipropósito, conserva una base fuertemente imperativa y procedural.

A continuación se emplearán dos lenguajes distintos de C para resolver el problema común: **Pascal** y **Go**. Esta elección permite observar tanto una formulación imperativa clásica como una formulación imperativa contemporánea.

Ejemplo en Pascal

Pascal destaca por una sintaxis explícita y estructurada. Palabras como **begin**, **end**, **for** e **if** permiten visualizar con claridad los bloques del programa. En el problema de las calificaciones, esa estructura facilita identificar dónde se inicializan variables, dónde se realiza el recorrido y dónde se consolida el reporte.

```
1    program ReporteCalificaciones;
2
3    var
4        notas: array[1..7] of real = (8.5, 6.2, 9.1, 5.8, 7.4, 10.0, 6.9);
5        i, aprobados: integer;
6        suma, promedio, maxima, minima: real;
7
8    begin
9        suma := 0;
10       aprobados := 0;
```

```
11     maxima := notas[1];
12     minima := notas[1];
13
14     for i := 1 to 7 do
15     begin
16         suma := suma + notas[i];
17
18         if notas[i] > maxima then
19             maxima := notas[i];
20
21         if notas[i] < minima then
22             minima := notas[i];
23
24         if notas[i] >= 7.0 then
25             aprobados := aprobados + 1;
26     end;
27
28     promedio := suma / 7;
29
30     writeln('=== REPORTE DE CALIFICACIONES ===');
31     writeln('Promedio: ', promedio:0:2);
32     writeln('Nota maxima: ', maxima:0:2);
33     writeln('Nota minima: ', minima:0:2);
34     writeln('Aprobados: ', aprobados);
35     writeln('Reprobados: ', 7 - aprobados);
36 end.
```

Listing 5.5: Resolución imperativa del problema de calificaciones en Pascal

La línea `suma := suma + notas[i];` muestra de forma muy clara el principio de acumulación, mientras que las condiciones `if notas[i] > maxima then` y `if notas[i] >= 7.0 then` ilustran la doble función de la verificación: comparar para actualizar un extremo y evaluar una regla de negocio. La sintaxis de Pascal favorece una lectura secuencial muy limpia, lo cual explica su valor histórico en la enseñanza de programación estructurada.

Ejemplo en Go

Go ofrece una sintaxis más compacta que Pascal, pero conserva una lógica imperativa nítida. Su relevancia actual en servicios, herramientas de infraestructura y software concurrente lo convierte en un lenguaje útil para mostrar que el paradigma imperativo no pertenece únicamente a la historia de la computación, sino que sigue vigente en entornos modernos.

```
1  package main
2
3  import "fmt"
4
5  func main() {
6      notas := []float64{8.5, 6.2, 9.1, 5.8, 7.4, 10.0, 6.9}
7
8      suma := 0.0
9      aprobados := 0
10     maxima := notas[0]
11     minima := notas[0]
12
13     for _, nota := range notas {
14         suma += nota
15
16         if nota > maxima {
17             maxima = nota
18         }
19
20         if nota < minima {
21             minima = nota
22         }
23
24         if nota >= 7.0 {
25             aprobados++
26         }
27     }
28
29     promedio := suma / float64(len(notas))
30
31     fmt.Println("=== REPORTE DE CALIFICACIONES ===")
32     fmt.Printf("Promedio: %.2f\n", promedio)
33     fmt.Printf("Nota maxima: %.2f\n", maxima)
34     fmt.Printf("Nota minima: %.2f\n", minima)
35     fmt.Printf("Aprobados: %d\n", aprobados)
36     fmt.Printf("Reprobados: %d\n", len(notas)-aprobados)
37 }
```

Listing 5.6: Resolución imperativa del problema de calificaciones en Go

La instrucción `for _, nota := range notas` representa el recorrido de la colección, mientras que operadores como `+=` y `++` muestran formas abreviadas de actualización del estado. Aunque el estilo es distinto al de Pascal, la lógica subyacente es la misma: inicializar, recorrer, actualizar y verificar. Esta coincidencia permite confirmar que el paradigma se mantiene aun cuando la notación concreta cambie.

5.2.4. Análisis del enfoque imperativo

El paradigma imperativo posee una gran fortaleza explicativa porque permite seguir la trayectoria exacta de cada dato durante la ejecución. En el ejemplo de Pascal, la variable `suma` comienza en cero y se modifica en cada iteración. En el ejemplo de Go, la variable `aprobados` cambia únicamente cuando se satisface la condición `nota >= 7.0`. Esta visibilidad del estado convierte al paradigma imperativo en un entorno apropiado para aprender trazado de algoritmos, depuración y validación de pasos intermedios [75], [100].

Su principal limitación aparece cuando la solución crece en tamaño y complejidad. Si demasiadas variables comparten responsabilidades o si múltiples procedimientos modifican estados comunes sin disciplina de diseño, el razonamiento se vuelve más difícil. Por ello, aunque el enfoque imperativo es excelente para consolidar la lógica procedural, suele complementarse posteriormente con abstracciones orientadas a objetos, funcionales o declarativas.

5.3. Paradigma orientado a objetos

La programación orientada a objetos organiza la solución en entidades que reúnen estado y comportamiento. La documentación oficial de Java presenta este enfoque a través de nociones como clase, objeto, campo, método y constructor, mostrando que una clase funciona como un molde para crear objetos con características y operaciones definidas [101], [102], [103]. Esta forma de estructurar el software modifica profundamente el modo de abordar un problema, porque desplaza la atención desde una simple secuencia de pasos hacia la identificación de responsabilidades y relaciones entre componentes.

En este paradigma, el programa se entiende como una red de colaboraciones entre objetos. Cada objeto tiene información propia y ofrece servicios concretos mediante métodos. La solución deja de residir en un solo bloque de instrucciones y pasa a distribuirse entre entidades con propósitos diferenciados. Esta distribución mejora la organización, facilita el mantenimiento y hace posible extender el sistema con menor impacto sobre el resto del código [35], [83], [101].

5.3.1. Idea central

La idea central del paradigma orientado a objetos consiste en modelar el dominio del problema mediante clases y objetos. Una clase describe qué atributos y comportamientos comparte un conjunto de entidades semejantes. Un objeto, en cambio, es una instancia concreta de esa clase, con valores propios para sus atributos. Esta distinción permite traducir conceptos del problema a componentes del programa de forma más organizada.

El principio resulta especialmente poderoso cuando el problema posee entidades claramente diferenciables. En el caso de las calificaciones, aparecen de inmediato nociones como curso, estudiante, nota y reporte. En lugar de construir una única secuencia de cálculos, la orientación a objetos permite decidir qué parte del trabajo corresponde a cada entidad. El curso puede agrupar estudiantes; el estudiante puede conservar nombre y nota; el reporte puede condensar resultados estadísticos.

Además, este paradigma incorpora principios de diseño de gran relevancia. La **encapsulación** protege los datos internos de una clase y obliga a interactuar mediante métodos definidos; la **abstracción** permite mostrar solo los aspectos relevantes de una entidad; la **herencia** posibilita extender comportamientos comunes; y el **polimorfismo** admite distintas implementaciones para una misma interfaz conceptual. Aunque no todos esos mecanismos sean necesarios en un ejemplo simple, forman parte del marco conceptual que otorga potencia al paradigma [35], [101], [102].

5.3.2. Cómo abordar un problema en orientación a objetos

Abordar un problema en orientación a objetos exige comenzar por las entidades del dominio, no por las instrucciones sueltas. El primer paso consiste en identificar qué cosas del problema merecen convertirse en clases. Después debe determinarse qué datos conserva cada clase y qué operaciones realiza. Finalmente, se define cómo interactúan esas clases para producir el resultado final.

Particularizando el problema de las calificaciones, el razonamiento puede describirse así. Una clase **Estudiante** representa una entidad con nombre y nota. Una clase **Curso** conserva un conjunto de estudiantes y ofrece operaciones para agregarlos y calcular estadísticas. Una clase principal coordina la creación de objetos y solicita la emisión del reporte. Esta estructura distribuye el problema en partes más coherentes: la nota deja de ser un número aislado y pasa a formar parte de una entidad; el cálculo del promedio deja de ser una operación suelta y se convierte en responsabilidad del curso.

Este enfoque no solo organiza mejor el código, sino que facilita su evolución. Si en una etapa posterior se incorpora asistencia, evaluación por parciales o categorías de aprobación, la estructura orientada a objetos permite añadir atributos y métodos con menos reescritura global. Por ello, la orientación a objetos resulta particularmente apropiada para sistemas que crecen con el tiempo o que representan dominios con entidades y reglas relativamente estables.

5.3.3. Lenguajes representativos del paradigma orientado a objetos

La orientación a objetos adquirió gran proyección histórica debido a su capacidad para estructurar sistemas extensos. Muchos lenguajes contemporáneos la incorporan como paradigma central o como parte importante de su diseño. Entre los más representativos se encuentran los siguientes:

- **Java**: referencia central para sistemas empresariales, servicios y aplicaciones multiplataforma.
- **C#**: lenguaje moderno del ecosistema .NET con soporte robusto para programación orientada a objetos.
- **C++**: combina elementos de programación procedural y orientación a objetos con gran control de recursos.
- **Kotlin**: lenguaje moderno sobre la JVM, con fuerte énfasis en expresividad y seguridad.
- **Ruby**: lenguaje de sintaxis expresiva donde prácticamente todo se modela como objeto.
- **Swift**: lenguaje orientado a plataformas Apple con mecanismos modernos de diseño y seguridad.

Para mostrar la aplicación del paradigma se emplearán dos lenguajes: **Java** y **C#**. Ambos permiten expresar con claridad clases, métodos y encapsulación, y constituyen ejemplos especialmente adecuados para observar similitudes y diferencias de sintaxis dentro del mismo enfoque conceptual.

Ejemplo en Java

```
1  import java.util.ArrayList;
2  import java.util.List;
3
4  class Estudiante {
5      private String nombre;
6      private double nota;
7
8      public Estudiante(String nombre, double nota) {
9          this.nombre = nombre;
```

```
10         this.nota = nota;
11     }
12
13     public String getNombre() {
14         return nombre;
15     }
16
17     public double getNota() {
18         return nota;
19     }
20
21     public boolean aprobo() {
22         return nota >= 7.0;
23     }
24 }
25
26 class Curso {
27     private String nombreCurso;
28     private List<Estudiante> estudiantes;
29
30     public Curso(String nombreCurso) {
31         this.nombreCurso = nombreCurso;
32         this.estudiantes = new ArrayList<>();
33     }
34
35     public void agregarEstudiante(Estudiante estudiante) {
36         estudiantes.add(estudiante);
37     }
38
39     public double calcularPromedio() {
40         double suma = 0.0;
41         for (Estudiante e : estudiantes) {
42             suma += e.getNota();
43         }
44         return estudiantes.isEmpty() ? 0.0 : suma / estudiantes.size();
45     }
46 }
```

```
47     public double calcularMaxima() {
48         double maxima = Double.NEGATIVE_INFINITY;
49         for (Estudiante e : estudiantes) {
50             if (e.getNota() > maxima) {
51                 maxima = e.getNota();
52             }
53         }
54         return maxima;
55     }
56
57     public double calcularMinima() {
58         double minima = Double.POSITIVE_INFINITY;
59         for (Estudiante e : estudiantes) {
60             if (e.getNota() < minima) {
61                 minima = e.getNota();
62             }
63         }
64         return minima;
65     }
66
67     public int contarAprobados() {
68         int contador = 0;
69         for (Estudiante e : estudiantes) {
70             if (e.aprobo()) {
71                 contador++;
72             }
73         }
74         return contador;
75     }
76
77     public void mostrarReporte() {
78         System.out.println("=== REPORTE DEL CURSO " + nombreCurso + "
↪ ===");
79         System.out.println("Promedio: " + calcularPromedio());
80         System.out.println("Nota maxima: " + calcularMaxima());
81         System.out.println("Nota minima: " + calcularMinima());
82         System.out.println("Aprobados: " + contarAprobados());
```

```

83
84         for (Estudiante e : estudiantes) {
85             System.out.println(e.getNombre() + " -> " + e.getNota());
86         }
87     }
88 }
89
90 public class GestionCurso {
91     public static void main(String[] args) {
92         Curso curso = new Curso("Programacion");
93
94         curso.agregarEstudiante(new Estudiante("Ana", 8.5));
95         curso.agregarEstudiante(new Estudiante("Luis", 6.2));
96         curso.agregarEstudiante(new Estudiante("Maria", 9.1));
97         curso.agregarEstudiante(new Estudiante("Carlos", 5.8));
98         curso.agregarEstudiante(new Estudiante("Elena", 7.4));
99
100         curso.mostrarReporte();
101     }
102 }

```

Listing 5.7: Resolución orientada a objetos del problema de calificaciones en Java

La línea `private double nota;` muestra la encapsulación del estado interno del objeto, mientras que el método `public boolean aprobo()` convierte una regla del dominio en una operación propia de la clase. A su vez, la clase `Curso` asume responsabilidades agregadas, como calcular el promedio o contar aprobaciones. Esta separación expresa con claridad el principio de distribución de responsabilidades.

Ejemplo en C#

Listing 5.8: Resolución orientada a objetos del problema de calificaciones en C#

```

using System;
using System.Collections.Generic;
using System.Linq;

class Estudiante
{
    public string Nombre { get; set; }
    public double Nota { get; set; }

    public Estudiante(string nombre, double nota)
    {

```



```
        Nombre = nombre;
        Nota = nota;
    }

    public bool Aprobo()
    {
        return Nota >= 7.0;
    }
}

class Curso
{
    public string NombreCurso { get; set; }
    private List<Estudiante> estudiantes = new List<Estudiante>();

    public Curso(string nombreCurso)
    {
        NombreCurso = nombreCurso;
    }

    public void AgregarEstudiante(Estudiante estudiante)
    {
        estudiantes.Add(estudiante);
    }

    public double CalcularPromedio()
    {
        return estudiantes.Average(e => e.Nota);
    }

    public double CalcularMaxima()
    {
        return estudiantes.Max(e => e.Nota);
    }

    public double CalcularMinima()
    {
        return estudiantes.Min(e => e.Nota);
    }

    public int ContarAprobados()
    {
        return estudiantes.Count(e => e.Aprobo());
    }

    public void MostrarReporte()
    {
        Console.WriteLine("===REPORTE DEL CURSO" + NombreCurso + "===");
        Console.WriteLine("Promedio:" + CalcularPromedio());
        Console.WriteLine("Nota maxima:" + CalcularMaxima());
        Console.WriteLine("Nota minima:" + CalcularMinima());
        Console.WriteLine("Aprobados:" + ContarAprobados());
    }
}
```

```

        foreach (var e in estudiantes)
        {
            Console.WriteLine(e.Nombre + "─>" + e.Nota);
        }
    }

    class Programa
    {
        static void Main()
        {
            Curso curso = new Curso("Programacion");

            curso.AgregarEstudiante(new Estudiante("Ana", 8.5));
            curso.AgregarEstudiante(new Estudiante("Luis", 6.2));
            curso.AgregarEstudiante(new Estudiante("Maria", 9.1));
            curso.AgregarEstudiante(new Estudiante("Carlos", 5.8));
            curso.AgregarEstudiante(new Estudiante("Elena", 7.4));

            curso.MostrarReporte();
        }
    }

```

En C#, las propiedades `Nombre` y `Nota` muestran una forma idiomática de exponer atributos con control de acceso. Además, expresiones como `Average(e => e.Nota)` o `Count(e => e.Aprobo())` condensan recorridos comunes sin abandonar el paradigma orientado a objetos. Esto muestra cómo un mismo enfoque conceptual puede expresarse con estilos más tradicionales o con bibliotecas modernas que elevan el nivel de expresividad.

5.3.4. Análisis del enfoque orientado a objetos

La fortaleza principal de este paradigma radica en su capacidad para organizar problemas complejos en partes coherentes. Mientras que el paradigma imperativo tiende a reunir la lógica en secuencias de pasos, la orientación a objetos obliga a decidir qué entidad debe hacer qué cosa. Esa exigencia conduce a soluciones con mejor separación de responsabilidades y con mayor facilidad de extensión [35], [101], [103].

Además, la orientación a objetos resulta especialmente pertinente en lenguajes como Java, donde la biblioteca estándar y las estructuras fundamentales del ecosistema presuponen ese modo de organización. Por ello, una comprensión sólida del paradigma no solo mejora la calidad del código, sino que facilita el uso correcto de colecciones, clases auxiliares, interfaces y arquitecturas modulares.

5.4. Paradigma funcional

El paradigma funcional se fundamenta en la idea de construir programas como composiciones de funciones. La documentación de Python reconoce explícitamente que el lenguaje dispone de rasgos aptos para programación en estilo funcional, mientras que Haskell se define oficialmente como un lenguaje puramente funcional, basado en cálculo lambda, transparencia referencial, inmutabilidad y evaluación perezosa [96], [104], [105]. Esta caracterización convierte al paradigma funcional en una forma particularmente rigurosa de pensar la programación como transformación de datos.

A diferencia del enfoque imperativo, donde el interés principal recae en cómo cambia el estado, el enfoque funcional privilegia la relación entre entrada y salida. Una función recibe argumentos y produce un resultado sin necesidad de alterar variables globales o estructuras compartidas. Esta forma de modelado favorece el razonamiento matemático, reduce ciertos tipos de errores asociados al estado mutable y hace visibles patrones de composición muy poderosos [83], [84], [104].

5.4.1. Idea central

La idea central del paradigma funcional consiste en considerar que una gran parte de los problemas computacionales puede expresarse como composición de funciones pequeñas y bien definidas. Cada función realiza una transformación concreta y, al combinarse con otras, contribuye a construir la solución total. En lugar de pensar primero en ciclos y asignaciones, se piensa en operaciones como transformar, filtrar, agrupar, reducir y componer.

Esta perspectiva resulta especialmente útil cuando el problema involucra colecciones de datos. Por ejemplo, calcular el promedio de un conjunto de notas puede verse como la combinación de dos operaciones: sumar la colección y dividir entre su longitud. Contar aprobaciones puede entenderse como filtrar los elementos que satisfacen una condición y medir cuántos quedaron. Hallar un máximo o un mínimo puede expresarse como una reducción sobre la colección. Estas ideas permiten observar el problema desde una lógica menos procedimental y más algebraica.

También debe destacarse el papel de la **inmutabilidad**. Cuando una estructura no cambia de manera destructiva, el razonamiento sobre el comportamiento del programa se simplifica, porque una función aplicada a los mismos argumentos siempre produce el mismo resultado. Esta propiedad, conocida como transparencia referencial, es uno de los rasgos más característicos del enfoque funcional puro [84], [104], [105].

5.4.2. Cómo abordar un problema en el paradigma funcional

Abordar un problema en el paradigma funcional requiere comenzar por identificar transformaciones de datos y no variables de estado. El primer paso consiste en preguntarse qué función representa el resultado buscado. El segundo paso es descomponer ese resultado en subfunciones más simples. El tercero es determinar cómo se combinan dichas funciones sobre entradas o colecciones.

Particularizando el problema de las calificaciones, el promedio puede modelarse como una función `promedio(notas)`, el conteo de aprobados como una función que filtra y cuenta, y el reporte como una composición de transformaciones sobre la misma colección. En lugar de pensar “recorrer y actualizar”, se piensa “tomar una lista y obtener de ella una nueva información derivada”.

Las buenas prácticas más importantes en este enfoque consisten en definir funciones de propósito claro, evitar dependencias laterales innecesarias, usar transformaciones expresivas sobre colecciones y conservar consistencia semántica en la composición. Cuando este paradigma se aplica con disciplina, la solución tiende a ser más modular y más fácil de verificar conceptualmente.

5.4.3. Lenguajes representativos del paradigma funcional

El paradigma funcional ha dado origen a varios lenguajes emblemáticos y también ha influido en lenguajes multiparadigma contemporáneos. Entre los más representativos se encuentran:

- **Haskell**: referencia principal del paradigma funcional puro.
- **Lisp**: familia histórica de lenguajes fuertemente vinculada a la manipulación simbólica y a la programación funcional.
- **Scheme**: variante de Lisp de gran influencia académica.
- **F#**: lenguaje funcional sobre la plataforma .NET.
- **OCaml**: lenguaje funcional con sistema de tipos expresivo.
- **Scala**: lenguaje multiparadigma con soporte muy sólido para composición funcional.
- **Elixir**: lenguaje funcional moderno construido sobre la máquina virtual de Erlang.

A continuación se desarrollará el problema común en **Haskell** y **Scala**. Esta elección permite comparar un lenguaje funcional puro con otro multiparadigma donde el estilo funcional ocupa una posición central.

Ejemplo en Haskell

```
1  type Nombre = String
2  type Nota = Double
3
4  promedio :: [Nota] -> Double
5  promedio xs = sum xs / fromIntegral (length xs)
6
7  maxima :: [Nota] -> Nota
8  maxima = maximum
9
10 minima :: [Nota] -> Nota
11 minima = minimum
12
13 cantidadAprobados :: [Nota] -> Int
14 cantidadAprobados xs = length (filter (>= 7.0) xs)
15
16 estado :: Nota -> String
17 estado n
18   | n >= 7.0  = "Aprobado"
19   | otherwise = "Reprobado"
20
21 mostrarDetalle :: [(Nombre, Nota)] -> IO ()
22 mostrarDetalle estudiantes =
23   mapM_ \(nom, nota) -> putStrLn (nom ++ " -> " ++ show nota ++ " (" ++
24   ↪ estado nota ++ ")")) estudiantes
25
26 main :: IO ()
27 main = do
28   let estudiantes =
29     [ ("Ana", 8.5)
30     , ("Luis", 6.2)
31     , ("Maria", 9.1)
32     , ("Carlos", 5.8)
33     , ("Elena", 7.4)
34     ]
35   notas = map snd estudiantes
```

```
35
36     putStrLn "=== REPORTE FUNCIONAL EN HASKELL ==="
37     putStrLn ("Promedio: " ++ show (promedio notas))
38     putStrLn ("Nota maxima: " ++ show (maxima notas))
39     putStrLn ("Nota minima: " ++ show (minima notas))
40     putStrLn ("Aprobados: " ++ show (cantidadAprobados notas))
41     mostrarDetalle estudiantes
```

Listing 5.9: Resolución funcional del problema de calificaciones en Haskell

La función `cantidadAprobados xs = length (filter (>= 7.0) xs)` resume con notable precisión una idea que, en un enfoque imperativo, requeriría varias líneas de recorrido, condición y contador. De igual modo, `map snd estudiantes` extrae la segunda componente de cada par, es decir, las notas, sin modificar la colección original. Esta forma de escribir destaca la expresividad del paradigma funcional cuando se trabaja con listas y transformaciones bien definidas.

Ejemplo en Scala

```
1     object ReporteCalificacionesScala {
2         case class Estudiante(nombre: String, nota: Double)
3
4         def promedio(notas: List[Double]): Double =
5             notas.sum / notas.length
6
7         def maxima(notas: List[Double]): Double =
8             notas.max
9
10        def minima(notas: List[Double]): Double =
11            notas.min
12
13        def cantidadAprobados(notas: List[Double]): Int =
14            notas.count(_ >= 7.0)
15
16        def main(args: Array[String]): Unit = {
17            val estudiantes = List(
18                Estudiante("Ana", 8.5),
19                Estudiante("Luis", 6.2),
20                Estudiante("Maria", 9.1),
21                Estudiante("Carlos", 5.8),
```

```
22     Estudiante("Elena", 7.4)
23     )
24
25     val notas = estudiantes.map(_.nota)
26
27     println("=== REPORTE FUNCIONAL EN SCALA ===")
28     println(s"Promedio: ${promedio(notas)}")
29     println(s"Nota maxima: ${maxima(notas)}")
30     println(s"Nota minima: ${minima(notas)}")
31     println(s"Aprobados: ${cantidadAprobados(notas)}")
32
33     estudiantes.foreach(e =>
34         println(s"${e.nombre} -> ${e.nota} (${if (e.nota >= 7.0)
↪      "Aprobado" else "Reprobado"}")
35     )
36 }
37 }
```

Listing 5.10: Resolución funcional del problema de calificaciones en Scala

En Scala, expresiones como `notas.count(_ >= 7.0)` y `estudiantes.map(_.nota)` muestran la integración de operaciones funcionales sobre colecciones dentro de un lenguaje multiparadigma. El uso de `case class` también evidencia cómo el estilo funcional puede convivir con nociones estructuradas de modelado de datos.

5.4.4. Análisis del enfoque funcional

La mayor fortaleza del paradigma funcional consiste en que expresa la solución como un conjunto de relaciones de transformación bien delimitadas. Esto reduce la necesidad de rastrear manualmente cambios de estado y hace posible razonar sobre partes del programa con mayor independencia. Además, las soluciones funcionales suelen ser altamente composables: una función que hoy calcula aprobaciones puede reutilizarse mañana dentro de un reporte más amplio sin necesidad de reestructurar la lógica [84], [96], [104].

La dificultad principal radica en que este enfoque exige un cambio de perspectiva. La ausencia de acumuladores explícitos y la preferencia por funciones sobre datos inmutables pueden parecer menos intuitivas al comienzo. Sin embargo, una vez consolidado, este estilo proporciona herramientas muy valiosas para el diseño de software confiable y expresivo.

5.5. Paradigma declarativo

El paradigma declarativo se distingue por describir el resultado buscado sin detallar paso a paso el procedimiento de ejecución. En este enfoque, el centro del programa no es una secuencia explícita de instrucciones, sino una especificación de condiciones, relaciones o consultas. Esta forma de programación resulta especialmente poderosa en contextos donde el motor subyacente puede encargarse del orden, la optimización y la estrategia concreta para producir el resultado [84], [99], [106].

La diferencia con el paradigma imperativo es profunda. Mientras el enfoque imperativo responde a la pregunta “cómo se hace”, el declarativo se orienta a la pregunta “qué se desea obtener”. Esta distinción no implica que desaparezca la lógica,

sino que la lógica se traslada a la especificación del objetivo, a las restricciones y a las relaciones entre datos. Bases de datos, motores de inferencia y lenguajes lógicos constituyen escenarios paradigmáticos para esta forma de programación.

5.5.1. Idea central

La idea central del paradigma declarativo consiste en formular propiedades del resultado deseado y delegar la estrategia de resolución al sistema. En SQL, por ejemplo, se indica qué columnas se desean, qué condiciones deben cumplirse y cómo deben agruparse los datos; el motor de base de datos decide los planes de ejecución concretos. En Prolog, se expresan hechos y reglas, y el motor de inferencia determina qué respuestas satisfacen las consultas.

Esta forma de trabajo obliga a razonar con precisión sobre la semántica del problema. En lugar de pensar primero en contadores o bucles, conviene identificar relaciones, criterios, restricciones y condiciones de pertenencia. Así, el problema de las calificaciones puede expresarse como una consulta de promedio, una consulta de máximos y mínimos, o una regla que define cuándo una calificación constituye una aprobación.

5.5.2. Cómo abordar un problema en el paradigma declarativo

El abordaje general comienza por decidir cuál es la unidad declarativa adecuada: una consulta, una regla o una restricción. Luego se determina qué datos de base o hechos son necesarios. Finalmente se formulan las expresiones que permiten obtener el resultado deseado. Esta estrategia requiere especial claridad semántica, ya que una especificación ambigua o incompleta puede conducir a resultados inesperados.

Aplicando esta lógica al problema común, la pregunta no es “cómo recorrer las calificaciones”, sino “qué consulta devuelve el promedio”, “qué condición separa aprobados de reprobados” o “qué regla define el estado de cada registro”. Esta reorganización conceptual resulta muy útil para problemas centrados en información tabular, relaciones lógicas o recuperación de conocimiento a partir de hechos declarados.

5.5.3. Lenguajes representativos del paradigma declarativo

El paradigma declarativo reúne varios lenguajes con propósitos distintos, pero con una misma idea de base: expresar resultados y relaciones en lugar de secuencias detalladas de control. Entre los más representativos se encuentran:

- **SQL**: lenguaje de consulta y manipulación de datos relacionales.
- **Prolog**: lenguaje lógico basado en hechos y reglas.
- **Datalog**: variante lógica especializada en consultas y reglas sobre datos.
- **XQuery**: orientado a consulta y transformación de documentos XML.
- **Mercury**: lenguaje lógico/declarativo con fuerte énfasis en tipos y modos.

A continuación se desarrollará el problema común en **SQL** y **Prolog**, pues ambos muestran con claridad dos variantes del enfoque declarativo: la consulta sobre tablas y la inferencia basada en reglas.

Ejemplo en SQL

```
1 CREATE TABLE calificaciones (  
2     id_estudiante INT PRIMARY KEY,  
3     nombre VARCHAR(80),  
4     nota DECIMAL(4,2)  
5 );  
6
```

```
7  INSERT INTO calificaciones
8      (id_estudiante, nombre, nota)
9  VALUES
10     (1, 'Ana', 8.50),
11     (2, 'Luis', 6.20),
12     (3, 'Maria', 9.10),
13     (4, 'Carlos', 5.80),
14     (5, 'Elena', 7.40);
15
16  SELECT AVG(nota) AS promedio,
17         MAX(nota) AS nota_maxima,
18         MIN(nota) AS nota_minima
19  FROM calificaciones;
20
21  SELECT COUNT(*) AS aprobados
22  FROM calificaciones
23  WHERE nota >= 7.0;
24
25  SELECT nombre, nota,
26         CASE
27             WHEN nota >= 7.0 THEN 'Aprobado'
28             ELSE 'Reprobado'
29         END AS estado
30  FROM calificaciones
31  ORDER BY nota DESC;
```

Listing 5.11: Resolución declarativa del problema de calificaciones en SQL

Aquí no aparece ninguna instrucción que recorra explícitamente fila por fila. El motor de base de datos se encarga de ello. La cláusula `WHERE nota >= 7.0` define un criterio de selección; `AVG(nota)` expresa un agregado; `CASE` formula una regla de clasificación. El estilo declarativo se evidencia precisamente en esa ausencia de control paso a paso.

Ejemplo en Prolog

```
1  calificacion(ana, 8.5).
2  calificacion(luis, 6.2).
3  calificacion(maria, 9.1).
4  calificacion(carlos, 5.8).
```



```
5   calificacion(elena, 7.4).
6
7   aprobado(Estudiante) :-
8   calificacion(Estudiante, Nota),
9   Nota >= 7.0.
10
11  reprobado(Estudiante) :-
12  calificacion(Estudiante, Nota),
13  Nota < 7.0.
14
15  estado(Estudiante, aprobado) :-
16  aprobado(Estudiante).
17
18  estado(Estudiante, reprobado) :-
19  reprobado(Estudiante).
```

Listing 5.12: Resolución declarativa del problema de calificaciones en Prolog

En Prolog, las líneas iniciales representan hechos. Las reglas posteriores describen condiciones lógicas. Una consulta como `aprobado(X)` permite obtener todas las entidades que satisfacen la regla de aprobación. El valor didáctico de este ejemplo reside en que la programación aparece aquí como formulación de conocimiento y no como descripción de una secuencia de ejecución.

5.5.4. Análisis del enfoque declarativo

El paradigma declarativo es especialmente poderoso cuando el problema se expresa mejor como una relación entre datos que como una secuencia de pasos. Esta cualidad lo hace muy apropiado para bases de datos, motores de reglas, validaciones complejas y consultas sobre estructuras ya organizadas. Además, fortalece la capacidad de especificar con precisión condiciones y resultados [84], [99].

Su principal exigencia conceptual consiste en pensar el problema con claridad semántica. Una especificación deficiente no suele manifestarse como un error sintáctico evidente, sino como una consulta o una regla que produce resultados erróneos o incompletos. Por ello, el paradigma declarativo desarrolla una disciplina de precisión en la formulación del problema.

5.6. Paradigmas y estrategias de resolución del mismo problema

Comparar paradigmas sobre un mismo problema permite observar que la diferencia entre lenguajes no se agota en la apariencia del código. Cada paradigma selecciona qué aspectos del problema deben hacerse explícitos y cuáles se delegan al lenguaje, al compilador o al motor de ejecución. Esta comparación es especialmente valiosa porque muestra que la programación no es una habilidad monolítica, sino una familia de formas de modelado [84], [85], [99].

5.6.1. Diferencia de enfoque

En el paradigma imperativo, la atención se concentra en la evolución del estado. La solución se evalúa por la corrección del orden de las instrucciones y por la consistencia con que las variables cambian a lo largo del tiempo. En consecuencia, los errores más frecuentes suelen relacionarse con inicializaciones incorrectas, condiciones defectuosas o actualizaciones omitidas. El valor de este enfoque consiste en que hace visibles las operaciones elementales que producen el resultado.

En el paradigma orientado a objetos, la diferencia esencial reside en la distribución de responsabilidades. El mismo problema deja de verse como una única secuencia de pasos y pasa a entenderse como una colaboración entre entidades. La calidad de la solución depende entonces menos de una traza lineal y más de la coherencia entre clases, atributos y métodos. Este enfoque aporta una visión estructural del software: importa no solo qué se calcula, sino también dónde debe residir cada parte de la lógica.

En el paradigma funcional, la diferencia surge de la prioridad otorgada a las transformaciones y a la composición. El problema común deja de formularse como una serie de cambios sobre variables y se expresa como una relación entre colecciones y funciones. Este enfoque favorece la modularidad fina, la reutilización de transformaciones y la separación entre definición de operaciones y efectos de ejecución.

En el paradigma declarativo, la diferencia más profunda se encuentra en que el procedimiento se vuelve implícito. La pregunta dominante ya no es qué paso sigue, ni qué objeto asume la responsabilidad, ni qué función transforma una entrada, sino qué propiedad debe cumplir el resultado. Esta forma de resolución resulta especialmente adecuada cuando el dominio se expresa naturalmente mediante consultas, relaciones o reglas.

5.6.2. Buenas prácticas para aprender a resolver problemas desde diferentes paradigmas

Una práctica valiosa consiste en formular primero el problema en lenguaje natural con total precisión antes de elegir un paradigma. Si el problema no está claramente delimitado, ningún lenguaje compensará esa deficiencia. Conviene explicitar entradas, salidas, restricciones, reglas y posibles casos límite. Una vez definido ese marco, la elección del paradigma se vuelve más consciente, porque ya no depende de la costumbre sintáctica, sino de la estructura del problema.

Otra práctica importante consiste en resolver el mismo problema en más de un paradigma y comparar después las decisiones tomadas. Esta comparación permite identificar qué partes del problema permanecen invariantes y cuáles dependen del estilo de modelado. Tal contraste mejora la comprensión profunda, porque obliga a distinguir entre la esencia algorítmica del problema y su forma de codificación.

También resulta recomendable conservar una disciplina de refinamiento progresivo. En un primer nivel conviene expresar la solución de manera simple, aunque aún no sea la más elegante. Después puede reorganizarse según el paradigma: extrayendo funciones en el enfoque funcional, distribuyendo responsabilidades en orientación a objetos, o reformulando reglas y consultas en el enfoque declarativo. Este refinamiento gradual fortalece la calidad conceptual de la solución sin perder de vista el problema original.

Finalmente, conviene revisar cada solución preguntándose no solo si produce el resultado correcto, sino también qué tipo de razonamiento exige para ser entendida y mantenida. Una solución imperativa puede ser correcta pero difícil de extender; una orientada a objetos puede estar bien estructurada pero ser innecesariamente compleja; una funcional puede ser elegante pero demasiado condensada; una declarativa puede ser expresiva pero exigir conocimiento detallado del dominio de consulta. Reconocer estas tensiones forma parte esencial del aprendizaje riguroso de la programación.

5.6.3. Tabla comparativa de enfoques

Cuadro 5.1: Diferencias entre paradigmas de programación en la resolución de un mismo problema

Paradigma	Pregunta dominante	Unidad principal de construcción	Riesgo conceptual más frecuente
Imperativo	¿Qué pasos deben ejecutarse y en qué orden?	Variables, asignaciones, condiciones y ciclos	Inicialización incorrecta, actualización defectuosa o control de flujo inconsistente
Orientado a objetos	¿Qué entidades existen y qué responsabilidad corresponde a cada una?	Clases, objetos, atributos y métodos	Distribución deficiente de responsabilidades o exceso de complejidad estructural
Funcional	¿Qué transformaciones deben aplicarse a los datos?	Funciones, composición, mapeo, filtrado y reducción	Abstracciones demasiado condensadas o pérdida de claridad para transformaciones complejas
Declarativo	¿Qué resultado o relación debe satisfacerse?	Consultas, reglas y restricciones	Especificaciones incompletas, ambiguas o semánticamente incorrectas

5.7. Conclusión

El estudio de los lenguajes de programación adquiere mayor profundidad cuando se entiende que cada paradigma no solo introduce una sintaxis diferente, sino una manera distinta de organizar el pensamiento computacional. Los lenguajes de bajo nivel revelan cómo los programas se materializan en instrucciones concretas para el procesador; los lenguajes de alto nivel trasladan la atención hacia la lógica del problema; el paradigma imperativo hace visible la transformación del estado; la orientación a objetos organiza la solución en entidades con responsabilidades; el paradigma funcional expresa transformaciones y composiciones; y el paradigma declarativo privilegia la especificación del resultado [35], [83], [84].

La comparación del mismo problema en varios enfoques permite distinguir con nitidez entre *problema* y *forma de representación*. Esa distinción constituye una de las bases más sólidas para aprender programación con rigor, porque impide reducir el conocimiento a un conjunto de instrucciones memorizadas y favorece la comprensión de principios transferibles entre lenguajes. Cuando se domina esa diferencia, resulta más sencillo analizar nuevos entornos de programación, adaptar estrategias y seleccionar herramientas con criterio técnico [75], [85], [100].

Capítulo 6

Del Algoritmo a la Aplicación Organizacional

El paso del algoritmo a la aplicación constituye uno de los procesos más importantes en la formación de un ingeniero en sistemas de información. Mientras que un algoritmo describe de manera abstracta una secuencia de pasos para resolver un problema, una aplicación es la implementación concreta de esa solución dentro de un entorno computacional que interactúa con usuarios, sistemas externos y estructuras de datos persistentes.

En el contexto de los sistemas de información organizacionales, los programas deben procesar datos provenientes de múltiples fuentes, validar reglas de negocio y producir resultados que apoyen procesos administrativos o estratégicos. Por ejemplo, un algoritmo que calcula un promedio puede convertirse en un módulo de evaluación académica dentro de un sistema universitario; un algoritmo que valida condiciones puede transformarse en un componente de control de acceso o en una regla de aprobación dentro de un sistema financiero [1], [2].

Por esta razón, la programación en Ingeniería en Sistemas de Información no se orienta únicamente a escribir código que funcione, sino a construir soluciones que puedan integrarse dentro de arquitecturas más amplias, manteniendo claridad lógica, trazabilidad de requisitos y capacidad de mantenimiento a largo plazo. En otras palabras, el objetivo no es solo resolver el problema, sino hacerlo de una forma que pueda sostenerse en el tiempo dentro de un sistema organizacional real [23], [24].

i

Para considerar...

Un *algoritmo* describe una solución en términos de pasos lógicos: qué datos se reciben, cómo se procesan y qué resultados se producen. Una *aplicación organizacional*, en cambio, es la materialización de esa solución en software ejecutable, integrado a procesos reales, con entradas provenientes de usuarios o sistemas y con salidas que deben ser útiles, verificables y oportunas para distintos roles dentro de la organización [1], [2], [23].

Para que el paso del algoritmo a la aplicación sea formativo (y no meramente mecánico), conviene trabajar con un ejercicio organizacional que sea suficientemente realista pero didáctico. En esta sección se utilizará un ejemplo coherente con entornos empresariales y con sistemas de información modernos: **registro de ingreso del personal**. Este problema permite conectar: (i) reglas de negocio (puntualidad, tolerancias, estados), (ii) estructuras de control (decisiones y validaciones), (iii) persistencia (registro en archivo o base de datos), y (iv) distribución del resultado (mensajes para usuarios y auditoría) [24], [40].

Problema organizacional (registro de ingreso). Se requiere diseñar un programa que registre el ingreso del personal a una organización. El programa debe recibir un `idEmpleado` y la `horaIngreso` (por ejemplo, en formato HH:mm), validar que los datos sean correctos, calcular los `minutosTarde` con respecto a una `horaInicioJornada` (por ejemplo, 08:00), clasificar el ingreso como PUNTUAL o TARDANZA (usando una tolerancia configurable), y registrar el evento. La salida no

se reduce a “mostrar un resultado”: debe generar información que pueda consumirse por distintos destinatarios (empleado, supervisor, auditoría) según su necesidad [1], [2].

Este mismo problema puede evolucionar a escenarios más avanzados sin cambiar su núcleo: el `idEmpleado` puede provenir de un teclado, de un lector de credenciales, de un archivo de marcaciones, o de un evento IoT empresarial (lector RFID/biométrico); y el registro puede almacenarse en archivo o en base de datos. La idea didáctica consiste en conservar una lógica central clara y extender las fuentes/destinos de datos progresivamente [23], [24].

6.1. Estructura Mínima de un Programa

Todo programa, incluso el más simple, tiene una estructura mínima: **declaración y organización del código, entrada de datos, procesamiento, salida de resultados y finalización**. Esta estructura se conserva aunque cambie el lenguaje, porque responde a la lógica de transformar entradas en salidas por medio de operaciones bien definidas [7], [59].

En la formación inicial en programación es fundamental que el estudiante comprenda esta estructura mínima antes de abordar aspectos más complejos como la modularidad, la arquitectura del software o la interacción con bases de datos. Comprender esta base permite reconocer patrones comunes entre distintos lenguajes de programación y facilita la transición hacia tecnologías más avanzadas.

En Java esta estructura debe comprenderse a la luz de que Java es un lenguaje *orientado a objetos*. Esto significa que, en la práctica, un “programa” se organiza como clases que encapsulan datos y comportamiento; la ejecución inicia típicamente en un método `main`, y desde allí se invocan métodos que realizan tareas específicas. Hablar de la estructura mínima de un programa en Java implica reconocer: (i) una clase principal, (ii) un punto de entrada (`main`), (iii) bloques de código con responsabilidades claras, y (iv) uso de tipos y librerías estándar para validar, convertir y procesar datos [20], [23], [35].

En el Listado de código 6.1 se presenta una primera implementación *didáctica* del problema organizacional (registro de ingreso) en Java, con entrada por teclado y salida por consola. Más adelante se ilustran variantes de entrada/salida (archivo y base de datos) manteniendo la misma lógica central.

```

1      /*
2      * Registro de ingreso del personal.
3      * Entrada: idEmpleado (entero), horaIngreso (HH:mm).
4      * Proceso: calcular minutos de tardanza respecto a horaInicioJornada.
5      * Salida: estado (PUNTUAL/TARDANZA) y mensajes para distintos
        ↪ destinatarios.
6      */
7      import java.time.LocalDateTime;
8      import java.time.format.DateTimeParseException;
9      import java.util.Scanner;
10
11     // Importa la clase LocalDateTime para manejar horas del día
12     import java.time.LocalDateTime;
13     // Importa la excepción que se produce si el formato de hora es
        ↪ incorrecto
14     import java.time.format.DateTimeParseException;
15     // Importa la clase Scanner para leer datos desde el teclado
16     import java.util.Scanner;
```

```
17
18 // Declaración de la clase principal del programa
19 public class RegistroIngresoApp {
20
21     // Método principal donde inicia la ejecución del programa
22     public static void main(String[] args) {
23
24         // -----
25         // Declaración de constantes
26         // -----
27
28         // Define la hora oficial de inicio de la jornada laboral
29         ↪ (08:00)
30         final LocalTime horaInicioJornada = LocalTime.of(8, 0);
31
32         // Define la tolerancia máxima permitida en minutos antes de
33         ↪ considerar tardanza
34         final int toleranciaMinutos = 5;
35
36         // -----
37         // Creación del objeto Scanner
38         // -----
39
40         // Crea un objeto Scanner para capturar datos ingresados por el
41         ↪ usuario desde el teclado
42         Scanner sc = new Scanner(System.in);
43
44         // -----
45         // Ingreso de datos
46         // -----
47
48         // Muestra un mensaje solicitando el identificador del empleado
49         System.out.print("Digite el ID del empleado: ");
50
51         // Lee el texto ingresado por el usuario y lo almacena en la
52         ↪ variable idTexto
53         String idTexto = sc.nextLine();
54     }
55 }
```

```

50
51      // Solicita al usuario la hora de ingreso en formato HH:mm
52      System.out.print("Digite la hora de ingreso (HH:mm): ");
53
54      // Lee el texto ingresado y lo guarda en la variable horaTexto
55      String horaTexto = sc.nextLine();
56
57      // -----
58      // Validación y conversión del ID
59      // -----
60
61      // Declara una variable que almacenará el ID convertido a
↪ número entero
62      Integer idEmpleado = null;
63
64      // Inicia un bloque de control de errores
65      try {
66          // Convierte el texto ingresado a número entero eliminando
↪ espacios en blanco
67          idEmpleado = Integer.parseInt(idTexto.trim());
68      }
69      // Captura el error si el texto no representa un número válido
70      catch (NumberFormatException ex) {
71          // Si ocurre error, asigna null para indicar que la
↪ conversión falló
72          idEmpleado = null;
73      }
74
75      // -----
76      // Validación y conversión de la hora
77      // -----
78
79      // Declara una variable para almacenar la hora convertida al
↪ tipo LocalTime
80      LocalTime horaIngreso = null;
81
82      // Inicia bloque de control de errores para la conversión de la

```



```

↪ hora
83         try {
84             // Convierte el texto ingresado a un objeto LocalTime
85             horaIngreso = LocalTime.parse(horaTexto.trim());
86         }
87         // Captura la excepción si el formato de la hora es incorrecto
88         catch (DateTimeParseException ex) {
89             // Si ocurre error, se asigna null para indicar que la hora
↪ es inválida
90             horaIngreso = null;
91         }
92
93         // -----
94         // Verificación de datos inválidos
95         // -----
96
97         // Verifica si el ID o la hora no pudieron convertirse
↪ correctamente
98         if (idEmpleado == null || horaIngreso == null) {
99             // Muestra mensaje de error al usuario
100            System.out.println("ERROR: datos inválidos. Verifique ID y
↪ formato de hora.");
101
102            // Termina inmediatamente la ejecución del programa
103            return;
104        }
105
106        // -----
107        // Cálculo de minutos de tardanza
108        // -----
109
110        // Convierte la hora de ingreso a segundos desde medianoche
111        int segundosIngreso = horaIngreso.toSecondOfDay();
112
113        // Convierte la hora oficial de inicio a segundos desde
↪ medianoche
114        int segundosInicio = horaInicioJornada.toSecondOfDay();

```

```

115
116      // Calcula la diferencia en segundos entre la hora de ingreso y
    ↪ la hora oficial
117      int diferencia = segundosIngreso - segundosInicio;
118
119      // Declara una variable que almacenará los minutos de tardanza
120      int minutosTarde;
121
122      // Si la diferencia es cero o negativa significa que llegó
    ↪ puntual o antes
123      if (diferencia <= 0) {
124          // En ese caso no existe tardanza
125          minutosTarde = 0;
126      }
127      else // Si la diferencia es positiva
128      {
129
130          // Convierte la diferencia de segundos a minutos
131          minutosTarde = diferencia / 60;
132      }
133
134      // -----
135      // Determinación del estado
136      // -----
137
138      // Declara una variable para almacenar el estado del ingreso
139      String estado;
140
141      // Si los minutos de tardanza están dentro del rango de
    ↪ tolerancia
142      if (minutosTarde <= toleranciaMinutos) {
143          // El empleado se considera puntual
144          estado = "PUNTUAL";
145      }
146      else // Si supera la tolerancia establecida
147      {
148          // Se clasifica como tardanza

```

```

149         estado = "TARDANZA";
150     }
151
152     // -----
153     // Salidas para distintos destinatarios
154     // -----
155
156     System.out.println("[Empleado] Estado de ingreso: " + estado);
157     ↪ // Informa al empleado si su ingreso fue puntual o tardío
158
159     System.out.println("[Supervisor] Minutos tarde: " +
160     ↪ minutosTarde); // Muestra al supervisor la cantidad exacta de minutos
161     ↪ de tardanza
162
163     // -----
164     // Registro de auditoría
165     // -----
166
167     System.out.println("[Auditoría] Registro generado para id="
168     ↪ + idEmpleado + ", hora=" + horaIngreso); // Simula el registro
169     ↪ del evento en un sistema de auditoría
170
171     sc.close(); // Cierra el objeto Scanner liberando el recurso de
172     ↪ entrada de datos
173
174 } // Fin del método main
175
176 } // Fin de la clase

```

Listing 6.1: Programa Java mínimo para registrar ingreso del personal (entrada por teclado).

Este ejemplo cumple un rol pedagógico: conserva la estructura mínima, introduce validación, separa responsabilidades (métodos), y demuestra que la salida puede orientarse a distintos destinatarios. En sistemas organizacionales, este último aspecto es esencial: un mismo resultado puede requerir *presentación diferente* según el rol, y no toda salida debe ser visible para todos (principio de mínima exposición y utilidad de la información) [1], [2], [23].

6.1.1. Entradas y Salidas

Las entradas representan los datos que el programa recibe para realizar su procesamiento. Estos datos pueden provenir de diversas fuentes, como la interacción directa con el usuario, archivos almacenados en el sistema, sensores o servicios externos.

En aplicaciones organizacionales, las entradas suelen corresponder a registros de información que describen eventos o estados del negocio.

Las salidas, por su parte, corresponden a los resultados que el programa produce después de ejecutar el procesamiento correspondiente. Estas salidas pueden presentarse como mensajes en pantalla, informes, registros almacenados en bases de datos o respuestas enviadas a otros sistemas.

Comprender la relación entre entradas y salidas es esencial para diseñar programas correctos. Todo algoritmo puede interpretarse como una función que transforma datos de entrada en resultados de salida. Por ello, en el desarrollo de aplicaciones para sistemas de información es imprescindible definir claramente qué datos se reciben, qué transformaciones se realizan y qué resultados se deben producir [2], [23].

Entradas del Programa del Ejemplo

En el Listado 6.1, las entradas son:

- **idEmpleado (entero):** identifica al empleado en el SI. En un entorno real suele validarse contra un catálogo (base de datos) [28], [29].
- **horaIngreso (HH:mm):** dato temporal capturado por usuario o dispositivo (marcación).

Orígenes típicos de las entradas

Para un estudiante principiante, es importante entender que el *mismo algoritmo* puede alimentarse desde distintas fuentes:

1. **Ingreso manual (teclado):** ideal para aprendizaje inicial y pruebas rápidas [7], [59].
2. **Archivo de texto/CSV:** usado para cargas masivas o integración con sistemas legados.
3. **Base de datos:** fuente principal en SI organizacionales, por consistencia y trazabilidad [28], [29].
4. **Servicios o APIs:** entradas provenientes de otros módulos (por ejemplo, un servicio de autenticación).
5. **Dispositivos/IoT empresarial:** lector de credenciales, RFID, biometría; el programa recibe eventos o lecturas en tiempo real. En control de ingreso del personal, un lector registra el **idEmpleado** y la hora automáticamente, reduciendo errores de captura [32], [48], [50].

Salidas del programa del ejemplo

Las salidas no se limitan a “imprimir”; en SI se espera que la información llegue al *destinatario correcto*:

- **Salida principal (operativa):** estado de ingreso (PUNTUAL/TARDANZA).
- **Salida secundaria (gestión):** minutos de tardanza para supervisión.
- **Salida secundaria (auditoría):** registro persistente del evento (quién, cuándo, estado).

Esta separación ayuda a mantener claridad y evita que un usuario reciba información que no necesita o que no le corresponde. En términos organizacionales, un SI debe producir información útil y contextualizada, y su valor depende de su calidad, trazabilidad y asignación al proceso/rol adecuado [1], [2], [22].

Fragmentos de código: otras entradas y otras salidas (mismo algoritmo)

Entrada desde archivo (ejemplo mínimo).

```
1 List<String> lineas = Files.readAllLines(Paths.get("marcaciones.txt"));
2 for (String linea : lineas) {
3     // Formato: idEmpleado;HH:mm
```

```
4      String[] partes = linea.split(";");
5      Integer idEmpleado = parseEntero(partes[0]);
6      LocalTime horaIngreso = parseHora(partes[1]);
7      // Reutilizar: calcularMinutosTarde(...) y clasificar estado
8  }
```

Listing 6.2: Lectura de marcaciones desde archivo (ejemplo mínimo).

Salida a archivo (auditoría).

```
1      String registro = idEmpleado + ";" + horaIngreso + ";" + estado + ";" +
    ↪ minutosTarde;
2      Files.write(Paths.get("auditoria_ingresos.txt"),
3      (registro + System.lineSeparator()).getBytes(),
4      StandardOpenOption.CREATE, StandardOpenOption.APPEND);
```

Listing 6.3: Registro de auditoría en archivo (salida persistente).

Salida a base de datos (idea general).

```
1      String sql = "INSERT INTO ingresos(id_empleado, hora_ingreso, estado,
    ↪ minutos_tarde) VALUES(?,?,?,?)";
2      // Preparar sentencia, asignar parametros, ejecutar
```

Listing 6.4: Idea general de inserción en BD (salida).

Estas variantes introducen un principio clave: *separar la lógica del cálculo* (minutos tarde y estado) *de la logística de datos* (de dónde provienen y hacia dónde van). Esta separación simplifica pruebas, reduce errores y facilita mantenimiento [23], [24], [33].

6.1.2. Bloques y alcance

En programación, los bloques de código representan agrupaciones de instrucciones que se ejecutan como una unidad lógica. Estos bloques permiten organizar el programa en secciones coherentes, facilitando la comprensión y el mantenimiento del código.

El concepto de alcance (*scope*) se refiere al contexto en el cual una variable o una instrucción es visible y puede ser utilizada. Comprender el alcance es fundamental para evitar errores relacionados con variables no definidas, conflictos de nombres o comportamientos inesperados durante la ejecución del programa.

En aplicaciones organizacionales, la correcta gestión del alcance contribuye a mantener una estructura modular del software, donde cada componente cumple una función específica sin interferir con otras partes del sistema. Este principio se relaciona con prácticas fundamentales de la ingeniería de software como la encapsulación y la separación de responsabilidades [23], [24].

i

Resumiendo

Un *bloque* es un conjunto de sentencias agrupadas, usualmente delimitadas por llaves { } en Java. El *alcance* (*scope*) define dónde una variable es visible y por cuánto tiempo existe durante la ejecución. Para principiantes, este concepto evita confusiones típicas como “la variable no existe” o “toma un valor inesperado” [7], [59].

En el Listado de código 6.1 pueden observarse bloques representativos:

- **Bloque de clase:** delimita la unidad principal de organización del programa (`class RegistroIngresoApp`).
- **Bloque del método `main`:** contiene la secuencia de ejecución principal.
- **Bloque `if` de validación:** decide si continuar o finalizar (control de flujo por condición).
- **Bloque `try-catch`:** maneja errores de conversión (control de flujo por excepción).

A nivel de alcance, conviene distinguir tres niveles frecuentes:

1. **Variables locales:** declaradas dentro de un método (por ejemplo, `minutosTarde` en `main`); existen durante la ejecución del método.
2. **Constantes locales:** declaradas con `final` (por ejemplo, `horaInicioJornada`); ayudan a expresar reglas del negocio de forma explícita.
3. **Miembros de clase:** no usados en el ejemplo mínimo, pero comunes cuando el programa crece (por ejemplo, un repositorio de datos o un servicio).

Para un estudiante que inicia, la recomendación práctica es: *declarar las variables lo más cerca posible de donde se usan* y evitar reutilizar nombres para significados distintos. Esto reduce ambigüedad y hace más legible la intención del programa [33], [51].

6.1.3. Ejecución paso a paso

La ejecución de un programa ocurre como una secuencia ordenada de instrucciones que el computador procesa una a una. Este modelo de ejecución secuencial es fundamental para comprender cómo se comportan las estructuras de control como condicionales y ciclos.

Para los que se inician en el desarrollo (comienzan a programar), visualizar la ejecución paso a paso resulta especialmente útil. Técnicas como el trazado manual o el uso de herramientas de depuración permiten observar cómo cambian los valores de las variables a lo largo del programa y cómo se toman las decisiones dentro del flujo de control.

Este enfoque facilita la identificación de errores y permite comprender con mayor claridad la relación entre el diseño algorítmico y su implementación en un lenguaje de programación [7], [59].

i

Para aplicarlo...

Comprender la ejecución paso a paso permite conectar el algoritmo (idea) con el programa (comportamiento real). El trazado manual y la depuración cumplen el mismo objetivo: observar cómo cambian las variables conforme avanzan las instrucciones [7], [59], [72].

Con el fin de reforzar una comprensión operacional del programa (más allá de leer el código como texto), conviene representar la ejecución como un flujo controlado por tres elementos: (i) *secuencia* de instrucciones, (ii) *decisiones* que bifurcan el recorrido según condiciones lógicas, y (iii) *puntos de entrada/salida* por los que circula la información. Esta representación permite al estudiante principiante construir un modelo mental correcto de lo que hace la máquina: avanzar instrucción por

instrucción, evaluar condiciones y actualizar el estado interno del programa mediante cambios en variables. En consecuencia, el trazado manual y la depuración en el IDE se interpretan como dos formas de observar el mismo fenómeno: el recorrido del flujo de control y la evolución del estado del programa [7], [51], [59], [68].

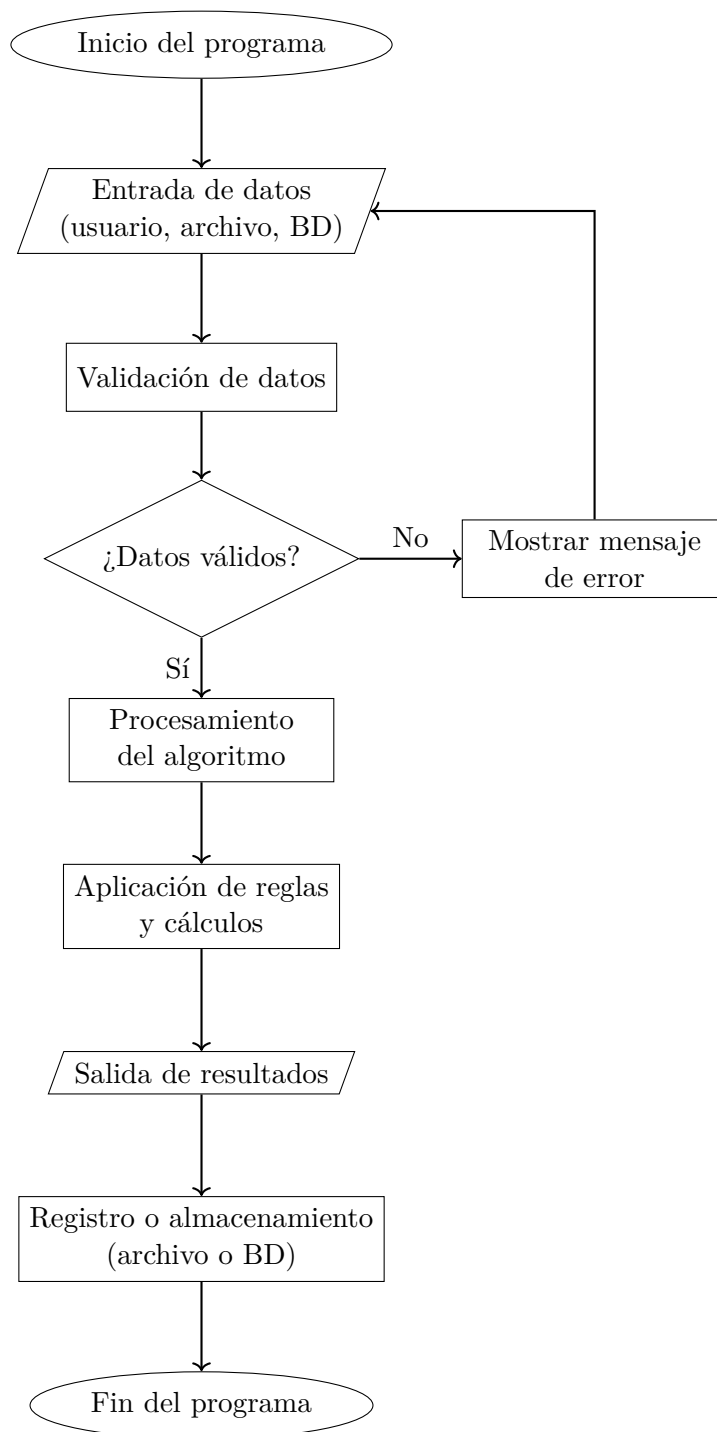


Figura 6.1: Flujo general de ejecución de un programa en una aplicación de sistemas de información.

A partir de este flujo, el siguiente paso consiste en observar de manera explícita cómo cambia el estado del programa; para ello, se registra la evolución de variables clave en una traza paso a paso.

A continuación se muestra una traza didáctica del ejemplo, usando entradas:

$idEmpleado = 15$, $horaIngreso = 08:12$, $horaInicioJornada = 08:00$, $tolerancia = 5$

Cuadro 6.1: Ejecución paso a paso del registro de ingreso (traza simplificada).

Paso	Acción	Estado de variables (valor)
1	Se definen constantes	horaInicioJornada=08:00, toleranciaMinutos=5
2	Se leen textos	idTexto="15", horaTexto="08:12"
3	Se convierte idTexto	idEmpleado=15
4	Se convierte horaTexto	horaIngreso=08:12
5	Validación de nulos	idEmpleado!=null, horaIngreso!=null ⇒ continúa
6	Cálculo diferencia	diffSeg=720 (12 min × 60)
7	Minutos tarde	minutosTarde=12
8	Clasificación por tolerancia	estado="TARDANZA" (12 > 5)
9	Salidas por rol	Empleado: TARDANZA; Supervisor: 12; Auditoría: registro

Este tipo de tabla es útil porque obliga a identificar: (i) qué variables existen, (ii) cuándo reciben valor por primera vez, (iii) en qué paso cambian, y (iv) qué condición determina el camino tomado. Esa disciplina de trazado es una forma temprana de verificación [7], [53], [72].

6.2. Errores frecuentes en el desarrollo inicial

Durante las primeras etapas del aprendizaje de la programación es común que los estudiantes enfrenten distintos tipos de errores. Comprender la naturaleza de estos errores constituye una parte esencial del proceso formativo, ya que permite desarrollar habilidades de diagnóstico, razonamiento y depuración. En la práctica profesional del desarrollo de software, una parte significativa del trabajo del programador consiste precisamente en analizar comportamientos inesperados del programa, identificar su causa y aplicar correcciones adecuadas. Por esta razón, aprender a reconocer los distintos tipos de errores desde las primeras etapas del aprendizaje contribuye a desarrollar una comprensión más profunda del funcionamiento de los programas y del proceso de construcción del software [7], [23], [24].

En el contexto de la Ingeniería en Sistemas de Información, esta capacidad resulta especialmente relevante porque los programas desarrollados forman parte de sistemas organizacionales más amplios que gestionan información crítica para las operaciones institucionales. Un error en el software puede provocar desde resultados incorrectos hasta fallos en procesos administrativos o decisiones basadas en información errónea. Por ello, el estudio sistemático de los errores en programación no solo tiene un valor pedagógico, sino también un valor profesional, ya que contribuye a formar desarrolladores capaces de producir software confiable y mantenible [2], [23], [40].

Desde el punto de vista conceptual, los errores en programación suelen clasificarse en tres categorías principales: errores de sintaxis, errores lógicos y errores de ejecución. Esta clasificación es ampliamente utilizada en la literatura de programación y en la enseñanza de los lenguajes de programación, ya que permite comprender la naturaleza de los problemas que pueden surgir durante el desarrollo del software y aplicar estrategias específicas para su detección y corrección [7], [59], [87].

La clasificación anterior puede comprenderse con mayor claridad mediante una representación conceptual de la relación entre los distintos tipos de errores que pueden aparecer durante el desarrollo de un programa. La Figura 6.2 muestra cómo los errores de sintaxis, los errores lógicos y los errores de ejecución se relacionan con distintas etapas del proceso de construcción y ejecución del software. Mientras que los errores de sintaxis se detectan durante la fase de compilación o verificación del código fuente, los errores lógicos se manifiestan en la incorrecta formulación del algoritmo y solo se evidencian cuando el programa produce resultados equivocados. Por su parte, los errores de ejecución aparecen cuando el programa interactúa con datos o recursos del entorno que generan condiciones inesperadas durante su funcionamiento. Esta representación ayuda al estudiante a comprender que cada tipo de error surge en momentos distintos del ciclo de desarrollo del software y requiere estrategias específicas de diagnóstico y corrección [7], [23], [24].



Figura 6.2: Relación entre los principales tipos de errores en programación. Los errores de sintaxis se detectan durante la compilación del código, los errores lógicos se originan en el diseño incorrecto del algoritmo y los errores de ejecución aparecen cuando el programa interactúa con datos o recursos del entorno durante su funcionamiento.

Los errores de sintaxis se producen cuando el código fuente escrito por el programador no cumple con las reglas gramaticales del lenguaje de programación. Los lenguajes formales poseen una sintaxis estricta que define cómo deben estructurarse las instrucciones, cómo se utilizan las palabras reservadas y cómo deben escribirse las expresiones. Cuando estas reglas se violan, el compilador o intérprete del lenguaje genera mensajes de error que impiden la ejecución del programa. Este tipo de errores suele ser frecuente en las primeras etapas del aprendizaje, cuando el estudiante aún se está familiarizando con la estructura formal del lenguaje [7], [87].

Los errores lógicos, en cambio, no están relacionados con la sintaxis del lenguaje, sino con el razonamiento utilizado para diseñar el algoritmo. En estos casos, el programa puede compilarse y ejecutarse correctamente, pero produce resultados incorrectos porque la lógica implementada no refleja adecuadamente el problema que se pretende resolver. Este tipo de errores suele ser más difícil de detectar, ya que el programa no genera mensajes de error explícitos; en su lugar, se requiere analizar cuidadosamente el comportamiento del programa y verificar sus resultados mediante pruebas con distintos conjuntos de datos [23], [24].

Los errores de ejecución (también conocidos como *runtime errors*) ocurren cuando el programa encuentra una situación que impide continuar su ejecución normal. Estos errores aparecen durante la ejecución del programa y pueden estar relacionados con situaciones como divisiones por cero, acceso a posiciones inválidas en estructuras de datos, fallos en la lectura de archivos o problemas en la comunicación con otros sistemas. En aplicaciones organizacionales, este tipo de errores puede afectar directamente la integridad de los datos o la continuidad de los procesos del negocio, por lo que es fundamental implementar mecanismos de validación y manejo de excepciones que permitan detectar y gestionar estas situaciones de forma controlada [23], [24], [87].

Comprender esta clasificación de errores permite a los estudiantes adoptar un enfoque más sistemático durante el desarrollo de programas. En lugar de percibir los errores como simples fallos del código, el programador aprende a interpretarlos como indicadores que revelan aspectos específicos del proceso de construcción del software: la corrección sintáctica del lenguaje, la coherencia lógica del algoritmo o la robustez del programa frente a situaciones inesperadas. Esta perspectiva constituye una competencia fundamental para el desarrollo profesional en el ámbito de la ingeniería de software y de los sistemas de información [40], [51].

i

Importante...

En el aprendizaje de la programación, los errores no son un accidente: son parte del proceso de adquirir precisión. Para un principiante es fundamental diferenciar *qué tipo de error* se está enfrentando, porque cada tipo exige una estrategia distinta: corregir sintaxis, revisar lógica o manejar condiciones de ejecución y recursos externos [7], [23], [59].

A continuación se muestran ejemplos de errores usando el mismo ejercicio (registro de ingreso). Se presentan como fragmentos *incorrectos* y su explicación.

6.2.1. Errores de sintaxis

En el proceso de desarrollo de software, los errores de sintaxis representan una de las primeras dificultades que enfrentan los programadores al escribir código en un lenguaje formal. A diferencia de los errores conceptuales o de diseño, los errores sintácticos se relacionan con la forma en que las instrucciones están escritas respecto a las reglas gramaticales del lenguaje de programación. Cada lenguaje define una sintaxis precisa que establece cómo deben escribirse las declaraciones, cómo se estructuran las expresiones y qué símbolos delimitan los distintos componentes del programa. Cuando el código no respeta estas reglas formales, el compilador o el intérprete impide su ejecución y genera mensajes que indican la presencia del error.

En lenguajes compilados como Java, el proceso de compilación analiza el código fuente antes de que el programa pueda ejecutarse. Durante esta fase, el compilador realiza un análisis léxico y sintáctico del programa para verificar que las instrucciones cumplan con la gramática del lenguaje. Si detecta inconsistencias, el proceso de compilación se detiene y se generan mensajes de error que señalan la línea del código donde se identificó el problema. Aunque estos mensajes no siempre indican con exactitud la causa del error, proporcionan pistas que permiten al programador localizar y corregir la instrucción problemática [7], [87].

Entre los errores de sintaxis más comunes se encuentran la omisión de signos de puntuación obligatorios (como punto y coma en Java), el uso incorrecto de paréntesis o llaves, la escritura incorrecta de palabras reservadas del lenguaje, o la utilización de estructuras que no corresponden con la gramática definida por el compilador. Estos errores suelen detectarse rápidamente una vez que el programador aprende a interpretar los mensajes generados por el compilador y a revisar cuidadosamente la estructura del código [7], [59].

En el desarrollo moderno de software, los entornos de desarrollo integrados (*Integrated Development Environments*, IDE) han reducido considerablemente la dificultad asociada con la identificación de este tipo de errores. Herramientas como IntelliJ IDEA, Eclipse o Visual Studio Code incorporan analizadores sintácticos que examinan el código en tiempo real mientras el programador escribe. Gracias a estos mecanismos, el IDE puede resaltar inmediatamente posibles errores mediante subrayados, colores o indicadores visuales, permitiendo que el programador los detecte antes incluso de intentar compilar el programa.

Además de la detección temprana, los IDE modernos proporcionan diversas funcionalidades que facilitan la corrección de errores sintácticos. Entre estas herramientas se encuentran el autocompletado de código, la sugerencia automática de

correcciones, la inserción automática de estructuras del lenguaje y la verificación continua de la coherencia del código. Por ejemplo, cuando el programador comienza a escribir una instrucción, el IDE puede sugerir métodos, clases o palabras reservadas válidas dentro del contexto actual del programa, reduciendo así la probabilidad de cometer errores de escritura.

Otra funcionalidad importante de los IDE contemporáneos es la capacidad de navegar rápidamente entre los errores detectados. El entorno puede mostrar una lista estructurada de problemas en el código, permitiendo al desarrollador desplazarse directamente a la línea correspondiente. Esta integración entre editor, compilador y herramientas de análisis estático constituye una ventaja significativa en el aprendizaje de la programación, ya que permite al estudiante comprender de manera inmediata cómo las reglas sintácticas del lenguaje influyen en la construcción correcta de los programas [23], [51].

En consecuencia, aunque los errores de sintaxis son inevitables durante el proceso de aprendizaje y desarrollo del software, las herramientas modernas de programación han transformado la forma en que estos errores se detectan y corrigen. El programador ya no depende únicamente del proceso de compilación para descubrir problemas en el código; en cambio, dispone de asistencia automática que contribuye a mejorar la precisión del código y a acelerar el proceso de desarrollo.

i

Afirmación comprobada

Los errores de sintaxis son violaciones a la “gramática” de Java y suelen impedir compilar. En un IDE se detectan temprano porque el editor subraya el problema y el compilador emite un mensaje [35], [68].

1) Signos de puntuación (falta de punto y coma)

El fragmento mostrado en el Listado de código 6.5 ilustra uno de los errores de sintaxis más frecuentes en Java: la omisión del punto y coma (;) al final de una instrucción. En este lenguaje, el punto y coma actúa como delimitador que indica al compilador dónde termina una sentencia. Cuando dicho símbolo no aparece donde corresponde, el compilador no puede determinar correctamente los límites de la instrucción y genera un error durante la fase de compilación.

Este tipo de problema es especialmente común en las primeras etapas del aprendizaje de la programación, ya que los principiantes tienden a concentrarse en la lógica del programa y pueden pasar por alto detalles formales de la sintaxis. En la práctica, la lectura cuidadosa del mensaje emitido por el compilador permite identificar rápidamente la línea donde falta el delimitador y corregir la instrucción correspondiente [7], [87].

```
1      int toleranciaMinutos = 5    // <-- falta ;
```

Listing 6.5: Error de sintaxis: falta de punto y coma.

2) Paréntesis/llaves desbalanceadas

El Listado de código 6.6 muestra un ejemplo de error sintáctico provocado por el desbalanceo de llaves en una estructura de control. En Java, las llaves ({}) se utilizan para delimitar bloques de instrucciones asociados a estructuras como condicionales, bucles o métodos. Cuando una llave de apertura no posee su correspondiente llave de cierre, el compilador pierde la referencia del bloque que se está definiendo y detiene el proceso de compilación indicando la inconsistencia. Este tipo de error puede resultar difícil de detectar visualmente en programas extensos, por lo que los entornos de desarrollo integrados suelen ayudar resaltando las parejas de llaves o mostrando indicadores visuales que permiten verificar rápidamente la correspondencia entre ellas [23], [35].

```
1      if (minutosTarde > toleranciaMinutos) {  
2          estado = "TARDANZA";  
3          // <-- falta cerrar }
```

```
4      System.out.println(estado);
```

Listing 6.6: Error de sintaxis: llaves desbalanceadas.

3) Uso incorrecto de palabras reservadas

En el Listado de código 6.7 se presenta un error derivado del uso indebido de una palabra reservada del lenguaje. En Java, términos como `class`, `if`, `while` o `return` forman parte del conjunto de palabras clave que poseen un significado específico dentro de la gramática del lenguaje.

Debido a su función sintáctica, estas palabras no pueden utilizarse como identificadores para variables, métodos o clases. Cuando el programador intenta emplear una palabra reservada como nombre de una variable —como ocurre en el ejemplo— el compilador detecta la infracción y genera un error de sintaxis.

Comprender el papel de estas palabras dentro del lenguaje resulta esencial para evitar conflictos entre los identificadores definidos por el programador y los elementos estructurales del lenguaje [35], [87].

```
1      int class = 10; // "class" es palabra reservada en Java
```

Listing 6.7: Error de sintaxis: palabra reservada mal utilizada.

4) Literales de texto mal cerrados

El ejemplo presentado en el Listado de código 6.8 muestra un error sintáctico relacionado con la definición incorrecta de un literal de texto. En Java, las cadenas de caracteres deben delimitarse mediante comillas dobles (`"`). Si una de estas comillas no se cierra adecuadamente, el compilador interpreta que el literal continúa hasta encontrar el siguiente delimitador válido, lo que provoca una inconsistencia en la estructura del programa.

Este tipo de error suele aparecer cuando se escriben mensajes largos o cuando se modifican cadenas existentes sin revisar cuidadosamente la presencia de las comillas de apertura y cierre. Los IDE modernos suelen detectar inmediatamente este problema resaltando la cadena incompleta o indicando la ausencia del delimitador correspondiente, lo que facilita su corrección durante la edición del código [7], [59].

```
1      System.out.println("Digite el ID del empleado:);
```

Listing 6.8: Error de sintaxis: comillas no cerradas.

i

Importante para principiantes...

La recomendación didáctica para principiantes es: *leer el mensaje del compilador y ubicar el primer error reportado*. Con frecuencia, un error temprano produce una cascada de errores secundarios; al corregir el primero, desaparecen varios [7], [68].

6.2.2. Errores lógicos

Los errores lógicos se producen cuando el programa se ejecuta correctamente desde el punto de vista sintáctico, pero produce resultados incorrectos debido a fallas en el razonamiento del algoritmo.

Este tipo de errores suele ser más difícil de detectar, ya que el programa no genera mensajes de error. En su lugar, produce resultados inesperados o inconsistentes. Para identificar estos errores es necesario analizar cuidadosamente la lógica del programa y verificar su comportamiento con distintos conjuntos de datos. Su detección depende de pruebas y trazado manual (como la Tabla 6.1) [7], [59].

Ejemplo 1: condición mal planteada (tolerancia)

Si el requisito es “hasta 5 minutos es puntual”, entonces `minutosTarde <= 5` es puntual. Un error típico es usar `< 5` y declarar tardanza a alguien con 5 minutos exactos.

Considérese el fragmento presentado en el Listado de código 6.9. En este ejemplo se utiliza una expresión condicional para determinar si el ingreso de una persona debe clasificarse como puntual o como tardanza. Aunque la instrucción es válida desde el punto de vista sintáctico y el programa se ejecuta sin generar errores, la lógica aplicada no refleja con precisión el requisito establecido por la regla del sistema. El problema surge al emplear el operador relacional `<` en lugar de `<=`. Como consecuencia, cuando el valor de `minutosTarde` es exactamente igual al valor permitido de tolerancia, el sistema clasifica incorrectamente el caso como tardanza. Este tipo de errores aparece con frecuencia cuando las condiciones de negocio no se traducen con exactitud al código y pone de manifiesto la importancia de analizar cuidadosamente los valores límite en las condiciones lógicas que gobiernan el comportamiento del programa [7], [59].

```
1      String estado = (minutosTarde < toleranciaMinutos) ? "PUNTUAL" :  
      ↪ "TARDANZA";  
2      // Con tolerancia=5, minutosTarde=5 se clasifica como TARDANZA  
      ↪ (incorrecto).
```

Listing 6.9: Error lógico: condición incorrecta.

Ejemplo 2: cálculo mal interpretado (minutos)

Un ejemplo adicional que ilustra otra situación representativa de error lógico, es el segmento de código del Listado 6.10. En este caso relacionada con la interpretación incorrecta de las unidades de medida utilizadas en un cálculo. En el ejemplo, el programa calcula la diferencia entre dos instantes de tiempo utilizando el método `toSecondOfDay()`, el cual devuelve el número total de segundos transcurridos desde el inicio del día.

Sin embargo, el resultado obtenido se asigna directamente a una variable denominada `minutosTarde`, lo que implica interpretar el valor calculado en segundos como si representara minutos. Aunque la operación es sintácticamente correcta y el programa no genera ninguna excepción durante su ejecución, el resultado producido es conceptualmente incorrecto. Este tipo de errores evidencia la necesidad de comprender con precisión el significado de los valores que se manipulan en un algoritmo y de verificar que las transformaciones entre unidades —por ejemplo, dividir los segundos entre 60 para obtener minutos— se realicen adecuadamente durante la implementación [24], [87].

```
1      int diff = ingreso.toSecondOfDay() - inicio.toSecondOfDay();  
2      int minutosTarde = diff; // incorrecto: diff esta en segundos
```

Listing 6.10: Error lógico: no convertir segundos a minutos.

Estos errores se previenen con: (i) casos de prueba con valores límite (por ejemplo, ingreso 08:05 y 08:06), (ii) trazado, y (iii) revisión de requisitos [7], [41].

6.2.3. Errores de ejecución

Los errores de ejecución ocurren cuando el programa encuentra una situación que impide continuar su ejecución normal. La ejecución del programa se ve interrumpida cuando enfrenta situaciones no previstas. Java ofrece mecanismos de excepciones para tratarlos de manera controlada [23], [24]. Ejemplos comunes incluyen divisiones por cero, acceso a posiciones inválidas de una estructura de datos o fallos en la lectura de archivos, datos inválidos, recursos externos no disponibles (archivo, red, BD), conversiones fallidas, fallas en la comunicación, etc.

En el desarrollo de aplicaciones para sistemas de información, estos errores pueden tener consecuencias significativas, ya que pueden afectar la integridad de los datos o interrumpir procesos críticos del negocio. Por esta razón, es fundamental incorporar mecanismos de validación y manejo de excepciones desde las primeras etapas del desarrollo [23], [24].

Ejemplo 1: conversión de entero inválida

El Listado de código 6.11 ilustra un caso típico de error de ejecución asociado a la conversión de tipos. En este ejemplo se intenta convertir una cadena de texto que contiene caracteres no numéricos ("ABC") en un valor entero mediante el método `Integer.parseInt()`. Debido a que el contenido de la cadena no representa un número válido, el entorno de ejecución de Java genera la excepción `NumberFormatException`. Este tipo de situación es frecuente cuando los datos provienen de entradas del usuario, formularios web o archivos de texto, donde el formato de la información no siempre es confiable. Por ello, en aplicaciones reales es recomendable validar previamente el contenido del texto o utilizar estructuras de manejo de excepciones para evitar que el programa termine abruptamente y poder informar al usuario sobre el error de manera adecuada [23], [87].

```
1      Integer id = Integer.parseInt("ABC"); // lanza NumberFormatException
```

Listing 6.11: Error de ejecución: `NumberFormatException`.

Ejemplo 2: lectura de archivo inexistente

El Listado de código 6.12 presenta un ejemplo de error de ejecución relacionado con el acceso a recursos externos. En este caso, el programa intenta leer el contenido de un archivo mediante el método `Files.readAllLines()`, especificando una ruta que no corresponde a un archivo existente. Cuando el sistema operativo no encuentra el recurso solicitado, la biblioteca de entrada/salida de Java puede generar una excepción del tipo `IOException`. Este tipo de error es común en aplicaciones que interactúan con archivos, bases de datos, servicios de red o dispositivos externos. La correcta gestión de estas excepciones resulta esencial para preservar la estabilidad del sistema, permitir la recuperación ante fallos y evitar la pérdida de información durante la ejecución del programa [24], [107].

```
1      Files.readAllLines(Paths.get("no_existe.txt")); // puede lanzar  
      ↪ IOException
```

Listing 6.12: Error de ejecución: fallo al leer archivo.

Ejemplo 3: base de datos inaccesible

En SI reales, la conexión depende de red, credenciales y disponibilidad del servidor; por ello se manejan excepciones como `SQLException`. Esta situación se relaciona con la naturaleza distribuida del acceso a datos [23], [24].

6.3. Entorno de desarrollo profesional

El desarrollo moderno de software se realiza dentro de entornos de desarrollo integrados, conocidos como IDE (*Integrated Development Environment*). Estos entornos proporcionan herramientas que facilitan la escritura, compilación, ejecución y depuración de programas.

Para los estudiantes de IS, familiarizarse con estos entornos desde las primeras etapas del aprendizaje es fundamental, ya que forman parte del flujo de trabajo profesional en el desarrollo de aplicaciones.

i

IDE...

Un IDE integra editor, compilación, ejecución, depuración y gestión de proyectos. Para un estudiante de IS, aprender un flujo de trabajo consistente tiene valor formativo: reduce fricción, acelera retroalimentación y promueve hábitos de ingeniería (pruebas, depuración y limpieza del código) [40], [51].

6.3.1. IDE y flujo de trabajo

Un IDE integra diversas herramientas necesarias para el desarrollo de software dentro de una única plataforma. Entre sus funciones más comunes se encuentran el editor de código, el compilador o intérprete del lenguaje, el depurador y herramientas de gestión de proyectos.

El uso adecuado de un IDE permite mejorar la productividad del desarrollador y facilita la detección temprana de errores.

En esta obra se usará como referencia IntelliJ IDEA (IntelliJ IDEA 2025.3 en el entorno del curso). Debe tenerse presente que los accesos directos pueden variar según el sistema operativo y el *keymap* configurado; por ello se recomienda verificar el mapa de teclas activo desde la configuración del IDE.

Creación de proyectos

En el desarrollo profesional de software, los programas no se escriben como archivos aislados, sino como parte de proyectos que contienen múltiples componentes relacionados. La creación de un proyecto establece la estructura organizativa del código, define dependencias y configura el entorno de ejecución.

Para crear un proyecto Java típico en IntelliJ IDEA:

1. Seleccionar **New Project**.
2. Elegir **Java**.
3. Configurar el JDK (por ejemplo, una versión LTS).
4. Definir nombre del proyecto y ubicación.
5. Crear la clase principal (**RegistroIngresoApp**) con el método **main**.

Este proceso formaliza la estructura del programa como *proyecto* (no como archivo suelto), práctica alineada con el desarrollo profesional de software y con la organización modular del código [23], [24].

Compilación y ejecución

La compilación es el proceso mediante el cual el código fuente escrito por el programador se transforma en un programa ejecutable que el computador puede comprender. En lenguajes interpretados, un proceso equivalente ocurre durante la ejecución mediante un intérprete [35], [68].

Comprender este proceso permite al estudiante entender cómo el código que escribe se convierte en instrucciones que la máquina puede ejecutar.

En Java, la compilación transforma código fuente en *bytecode*; la ejecución corre sobre la JVM. Comprender esta idea explica por qué Java tiene una fase de compilación previa y por qué los errores de sintaxis aparecen antes de ejecutar.

En IntelliJ IDEA, las acciones más comunes son:

- **Run (ejecutar):** ejecutar la configuración de ejecución actual. En el keymap por defecto, **Shift+F10** ejecuta la configuración de contexto [108].
- **Debug (depurar):** ejecutar en modo depuración. En el keymap por defecto, **Shift+F9** inicia depuración de la configuración de contexto [108], [109].

Además, IntelliJ permite ejecutar desde el menú contextual de la clase con **main** o desde el botón de ejecución. La recomendación para principiantes es mantener *una sola configuración* al inicio (un programa) y practicar el ciclo: **editar** → **ejecutar** → **observar salida** → **corregir** [7], [51].

Depuración

La depuración (*debugging*) es el proceso de identificar y corregir errores en un programa. Los IDE modernos incluyen herramientas que permiten ejecutar el programa paso a paso, inspeccionar variables y analizar el flujo de ejecución.

Estas herramientas resultan especialmente útiles para estudiantes que están aprendiendo a comprender el comportamiento interno de los programas.

i

Para analizar...

La depuración no es un acto de “adivinar”: es un proceso controlado para observar el programa mientras se ejecuta.

IntelliJ IDEA permite:

- **Colocar *breakpoints*:** detenerse en una línea para inspeccionar variables.
- **Step over / into / out:** avanzar instrucción por instrucción (sin entrar o entrando en métodos).
- **Inspección de variables:** ver el valor actual de `idEmpleado`, `horaIngreso`, `minutosTarde`, etc.

Ejemplo didáctico: colocar un breakpoint justo antes de calcular `minutosTarde` y ejecutar en modo Debug (**Shift+F9**). Luego observar cómo cambia `estado` cuando la entrada es 08:05 y luego 08:06. Esta práctica refuerza la comprensión de condiciones límite y reduce errores lógicos [7], [72], [109].

6.3.2. Estándares básicos de codificación

El código fuente no solo debe ser correcto desde el punto de vista funcional; también debe ser comprensible para otros desarrolladores. En proyectos organizacionales, donde múltiples personas colaboran en el desarrollo del software, mantener estándares de codificación consistentes es esencial para garantizar la mantenibilidad del sistema [33], [51].

Por lo tanto, el código del software debe ser correcto y también legible. En equipos de desarrollo y en sistemas que evolucionan por años, la legibilidad impacta el costo de mantenimiento y reduce defectos [33], [40], [51]. En Java, los estándares básicos abarcan indentación, nombres y comentarios útiles.

Indentación

La indentación consiste en organizar visualmente el código mediante espacios o tabulaciones que reflejan la estructura lógica del programa. Una indentación adecuada facilita la lectura del código y permite identificar rápidamente bloques de control como ciclos y condicionales.

En Java se utiliza de forma extendida indentación consistente (frecuentemente 4 espacios) para reflejar bloques (clases, métodos, condicionales, ciclos). En comparación:

- **Java:** la indentación mejora legibilidad pero el bloque lo definen llaves { }.
- **Python:** la indentación *define* el bloque; un error de espacios cambia el significado del programa.

- **Fortran (tradicional):** el formato del código ha tenido reglas históricas (columnas y estilo), y en versiones modernas la indentación sigue siendo una convención para claridad.

Para el principiante, la regla práctica es: *indentar siempre y de manera uniforme* para que la estructura del programa sea visible a simple vista, incluso antes de entender cada línea [33], [51].

Nombres significativos

El uso de nombres significativos para variables, funciones y clases mejora considerablemente la claridad del código. Un nombre bien elegido describe el propósito de la entidad que representa, lo que reduce la necesidad de comentarios adicionales.

Los nombres deben revelar intención. En Java se usan convenciones:

- **Clases:** UpperCamelCase (por ejemplo, `RegistroIngresoApp`).
- **Métodos y variables:** lowerCamelCase (horaIngreso, calcularMinutosTarde).
- **Constantes:** UPPER_SNAKE_CASE cuando son constantes globales; en el ejemplo se usan `final` locales, que también pueden seguir convención clara (toleranciaMinutos).

Ejemplos correctos en el ejercicio: `idEmpleado`, `horaInicioJornada`, `minutosTarde`, `estado`. Ejemplos a evitar: `x`, `a1`, `dato2` sin significado. La mejora no es estética: reduce ambigüedad, disminuye defectos y facilita revisión [33], [51].

Comentarios técnicos

Los comentarios técnicos permiten documentar aspectos importantes del programa, como decisiones de diseño, supuestos del algoritmo o limitaciones conocidas. Sin embargo, los comentarios deben utilizarse de manera equilibrada: el objetivo no es explicar cada línea de código, sino proporcionar contexto cuando sea necesario.

i

Muy importante...

Los comentarios deben aportar *contexto* y *justificación*, no describir lo obvio.

En Java, además, existe documentación estructurada con JavaDoc, útil para describir contratos (qué recibe, qué retorna, qué condiciones asume) [23], [51].

Ejemplo de JavaDoc en el ejercicio. El Listado de código 6.13 muestra un ejemplo de documentación utilizando el formato *JavaDoc*. Este tipo de comentario estructurado permite describir formalmente el propósito de un método, los parámetros que recibe y el valor que retorna. En entornos profesionales de desarrollo, la documentación JavaDoc resulta especialmente valiosa porque puede procesarse automáticamente para generar manuales técnicos o documentación de APIs. Además, este estilo de comentario facilita que otros desarrolladores comprendan el comportamiento esperado del método sin necesidad de analizar en detalle su implementación interna. La especificación clara de los parámetros (`@param`) y del valor de retorno (`@return`) ayuda a establecer contratos de uso del método y mejora la mantenibilidad del software [23], [51].

```

1  /**
2   * Calcula minutos de tardanza comparando la hora de ingreso con el
   ↪ inicio de jornada.
3   * @param inicio hora oficial de inicio (por ejemplo, 08:00).
4   * @param ingreso hora real de ingreso.
5   * @return minutos tarde; retorna 0 si el ingreso es igual o anterior al
   ↪ inicio.
```

```

6      */
7      private static int calcularMinutosTarde(LocalTime inicio, LocalTime
    ↪ ingreso) { ... }

```

Listing 6.13: Ejemplo de JavaDoc en el método de cálculo.

Ejemplo de comentario técnico (regla de negocio). El Listado de código 6.14 ilustra un comentario técnico que documenta una regla de negocio incorporada en el programa. En este caso, el comentario explica el significado de la constante `toleranciaMinutos`, indicando que una tardanza de hasta cinco minutos se considera aceptable dentro del sistema. Este tipo de comentario resulta útil cuando el código refleja decisiones organizacionales, políticas institucionales o restricciones específicas del dominio del problema. Al registrar estas reglas directamente en el código, se facilita la comprensión del sistema por parte de futuros desarrolladores y se reduce el riesgo de modificar accidentalmente valores que representan decisiones de negocio relevantes [23], [51].

```

1      // Regla de negocio: hasta 5 minutos se considera puntual.
2      final int toleranciaMinutos = 5;

```

Listing 6.14: Comentario técnico sobre regla de negocio.

6.4. Gestión de entrada y salida de datos

En aplicaciones organizacionales, la gestión de datos constituye una de las responsabilidades más importantes del software. Los programas deben ser capaces de recibir información, validarla, procesarla y presentar resultados de manera comprensible. Una gestión adecuada de entrada y salida contribuye a garantizar la confiabilidad del sistema y la calidad de la información generada.

En SI, la confiabilidad del resultado depende tanto del algoritmo como de la calidad de los datos. Por ello, la gestión de entrada/salida incluye validación, conversión, manejo de errores y presentación de resultados. Estas prácticas reducen fallas, mejoran trazabilidad y sostienen el valor de la información en el proceso organizacional [1], [2], [23].

6.4.1. Lectura y validación

La lectura de datos debe realizarse siempre considerando la posibilidad de que la información recibida no sea válida. Los usuarios pueden cometer errores al ingresar datos, o los datos provenientes de sistemas externos pueden contener inconsistencias.

Por esta razón, los programas deben incorporar mecanismos de validación que permitan detectar y manejar estas situaciones.

Validación de entrada

La validación de entrada consiste en verificar que los datos recibidos cumplan con las condiciones necesarias para ser procesados correctamente.

Buenas prácticas para entradas válidas en Java (especialmente para principiantes):

1. **Validar presencia:** no aceptar vacío ("") ni espacios.
2. **Validar formato:** hora HH:mm, enteros con `parseInt`.

3. **Validar dominio:** `idEmpleado > 0`; tolerancia no negativa; hora dentro de un rango razonable.
4. **Validar consistencia:** si el empleado no existe en el catálogo, rechazar o solicitar corrección.
5. **Responder con mensajes accionables:** indicar qué se espera y cómo corregir.

Estas prácticas se alinean con la idea de prevenir errores de ejecución y mejorar la interacción usuario-sistema en aplicaciones introductorias [7], [23], [59].

```
1    if (idEmpleado <= 0) {  
2        System.out.println("ERROR: el ID debe ser un entero positivo.");  
3        return;  
4    }
```

Listing 6.15: Validación simple de dominio para `idEmpleado`.

Conversión de tipos

Durante la ejecución de un programa es frecuente que los datos deban transformarse de un tipo a otro antes de poder utilizarse en cálculos, comparaciones o procesos de almacenamiento. Este proceso se denomina *conversión de tipos* (*type conversion*) y constituye una operación fundamental en la programación. Los datos pueden provenir de diferentes fuentes —como entradas de usuario, archivos, sensores, interfaces web o bases de datos— y no siempre se encuentran en el formato adecuado para ser procesados directamente. Por esta razón, los lenguajes de programación modernos incorporan mecanismos formales para transformar valores entre distintos tipos de datos de manera controlada y segura [87], [110].

La conversión de tipos desempeña un papel central en la correcta interpretación de los datos dentro de un sistema informático. Por ejemplo, un número leído desde un formulario suele recibirse inicialmente como texto, pero posteriormente debe convertirse en un valor numérico para poder realizar operaciones aritméticas. De forma análoga, un valor numérico calculado dentro del programa puede transformarse en texto para mostrarse en una interfaz gráfica o registrarse en un archivo de salida. En consecuencia, comprender cómo se realizan estas transformaciones permite evitar errores lógicos y garantizar la coherencia de los datos manipulados por el software [24], [107].

En Java, las conversiones de tipos pueden clasificarse en varias categorías. Algunas conversiones se realizan automáticamente cuando el lenguaje puede garantizar que no se perderá información. Otras requieren una indicación explícita del programador cuando existe riesgo de pérdida de precisión. Asimismo, es frecuente convertir datos entre representaciones textuales y tipos numéricos o temporales, especialmente cuando los valores se leen desde interfaces de usuario o archivos de texto [110].

1) Conversión automática (*widening*). La conversión automática ocurre cuando un valor de un tipo numérico con menor capacidad se asigna a una variable de un tipo con mayor capacidad de almacenamiento. En estos casos, el lenguaje puede realizar la conversión sin intervención del programador porque no existe riesgo de pérdida de información. Este proceso también se denomina *widening conversion* (conversión por ampliación) y está formalmente definido en la especificación del lenguaje Java [110].

El Listado de código 6.16 muestra ejemplos de este tipo de conversión.

```
1    // Variable entera  
2    int minutos = 12;  
3  
4    // Conversión automática a tipos de mayor capacidad  
5    long minutosLargos = minutos;
```

```
6    double minutosDouble = minutos;
7
8    // Mostrar resultados
9    System.out.println(minutosLargos);
10   System.out.println(minutosDouble);
```

Listing 6.16: Conversión automática: int a long y double.

En el ejemplo anterior, el valor almacenado en la variable `minutos` se asigna a variables de tipo `long` y `double`. Debido a que estos tipos poseen un rango de representación mayor que el tipo `int`, Java realiza la conversión de forma automática sin requerir instrucciones adicionales. Este comportamiento facilita la escritura de programas, ya que evita conversiones innecesarias cuando el resultado puede representarse sin pérdida de información.

Para comprender mejor este mecanismo, considérese el siguiente ejemplo adicional:

```
1    // Valor entero
2    int edad = 25;
3
4    // Asignación a un tipo de mayor capacidad
5    double edadDouble = edad;
6
7    // Resultado
8    System.out.println(edadDouble); // 25.0
```

Listing 6.17: Ejemplo adicional de conversión automática.

En este caso, el valor entero se transforma automáticamente en un número decimal equivalente.

2) Conversión explícita (*narrowing*). La conversión explícita se produce cuando un valor de un tipo con mayor capacidad se transforma en un tipo con menor capacidad. En estas situaciones existe la posibilidad de perder información, por lo que el programador debe indicar explícitamente la conversión mediante una operación denominada *cast*. Este proceso también se conoce como *narrowing conversion* (conversión por reducción) [110].

El Listado de código 6.18 ilustra un ejemplo de este tipo de conversión.

```
1    // Número con parte decimal
2    double tardanza = 12.9;
3
4    // Conversión explícita a entero
5    int tardanzaEntera = (int) tardanza;
6    // Se pierden los decimales
7
8    // Mostrar resultado
```

```
9      System.out.println(tardanzaEntera); // 12
```

Listing 6.18: Conversión explícita: double a int mediante cast.

En este ejemplo, el valor decimal 12.9 se convierte en un entero. Debido a que el tipo `int` no puede representar la parte fraccionaria, el resultado final es 12. Este comportamiento implica que la conversión puede producir pérdida de precisión, por lo que debe utilizarse con precaución.

Un ejemplo adicional que permite observar el mismo fenómeno en un contexto distinto se muestra en el Listado de código 6.19.

```
1      // Valor decimal
2      double promedio = 8.75;
3
4      // Conversión a entero
5      int promedioEntero = (int) promedio;
6
7      // Resultado
8      System.out.println(promedioEntero); // 8
```

Listing 6.19: Otro ejemplo de conversión con pérdida de precisión.

En este caso, la parte decimal del promedio se elimina durante la conversión.

3) Conversión desde texto (String) a valores numéricos. En numerosos programas, los datos ingresados por el usuario se reciben inicialmente como texto. Para poder realizar operaciones matemáticas con estos valores, es necesario convertir la cadena de caracteres en un tipo numérico. Java proporciona diversos métodos estáticos en clases como `Integer` y `Double` que permiten realizar estas conversiones [87].

El Listado de código 6.20 presenta ejemplos de conversión desde texto a valores numéricos.

```
1      // Texto que contiene un número
2      String idTexto = "25";
3
4      // Conversión a entero
5      int id = Integer.parseInt(idTexto.trim());
6
7      // Conversión a número decimal
8      double salario = Double.parseDouble("1200.50");
9
10     // Mostrar resultados
11     System.out.println(id);
12     System.out.println(salario);
```

Listing 6.20: Conversión desde String a valores numéricos.

Los métodos `parseInt()` y `parseDouble()` interpretan el contenido de la cadena como un número y devuelven el valor convertido. Si el texto no tiene un formato numérico válido, el programa generará una excepción. Por esta razón, en aplicaciones reales suele realizarse previamente una validación del contenido para garantizar que la información introducida por el usuario sea correcta.

El segmento de código del Listado 6.21 muestra una conversión adicional desde texto.

```
1      // Texto que representa un número decimal
2      String temperaturaTexto = "23.5";
3
4      // Conversión a número
5      double temperatura = Double.parseDouble(temperaturaTexto);
6
7      // Mostrar resultado
8      System.out.println(temperatura);
```

Listing 6.21: Ejemplo adicional de conversión desde String.

4) Conversión de valores numéricos a texto. En sentido inverso, también es frecuente transformar valores numéricos en texto, especialmente cuando se desea mostrar información al usuario o construir mensajes descriptivos. Esta conversión puede realizarse mediante concatenación de cadenas o utilizando métodos auxiliares como `String.valueOf()` [107].

El Listado de código 6.22 muestra dos formas comunes de realizar esta conversión.

```
1      // Variable numérica
2      int minutosTarde = 5;
3
4      // Conversión mediante concatenación
5      String mensaje1 = "Minutos tarde: " + minutosTarde;
6
7      // Conversión mediante método de la clase String
8      String mensaje2 = String.valueOf(minutosTarde);
9
10     // Mostrar resultados
11     System.out.println(mensaje1);
12     System.out.println(mensaje2);
13
```

Listing 6.22: Conversión de números a texto.

En ambos casos el resultado es una cadena de texto que puede utilizarse para generar mensajes, reportes o registros dentro del sistema.

5) Conversión de texto a tipos temporales. Los programas que manejan información relacionada con fechas u horas también requieren convertir valores textuales en objetos temporales. Java proporciona clases especializadas en el paquete `java.time` que permiten representar y manipular este tipo de información de forma segura y consistente [107].

El Listado de código 6.23 presenta un ejemplo de conversión desde texto a un objeto de tipo `LocalTime`.

```
1 // Conversión de una cadena a un objeto LocalTime
2 LocalTime hora = LocalTime.parse("08:12"); // Formato HH:mm
3
4 // Mostrar resultado
5 System.out.println(hora);
6
```

Listing 6.23: Conversión de texto a hora mediante `LocalTime`.

El método `parse()` interpreta la cadena de texto y crea un objeto que representa la hora indicada. Este tipo de conversión es común en aplicaciones que registran horarios, citas, eventos o actividades programadas.

Una recomendación importante consiste en evitar conversiones realizadas sin verificación previa. Antes de convertir datos proporcionados por el usuario o provenientes de fuentes externas, es conveniente validar el formato del texto para asegurar que la información es correcta. Si la conversión falla, el programa debe manejar la situación de forma controlada, mostrando un mensaje claro o solicitando nuevamente la entrada de datos. Esta práctica mejora la robustez del software y contribuye a prevenir errores durante la ejecución de aplicaciones reales [24], [87].

Manejo de datos inválidos

Cuando se detectan datos inválidos, el programa debe decidir cómo manejar la situación: solicitar nuevamente la entrada, mostrar un mensaje de error o registrar el evento para su revisión.

En Java hay dos enfoques didácticos principales:

1) Manejo por validación previa (sin excepciones). Una estrategia común para manejar conversiones consiste en verificar previamente si el contenido de un dato es válido antes de intentar convertirlo. Este enfoque se denomina *validación previa* y permite evitar que el programa genere errores durante la ejecución. En lugar de realizar directamente la conversión y depender del manejo de excepciones, el programa inspecciona primero el contenido del dato —generalmente una cadena de texto— y determina si cumple con el formato esperado.

En el lenguaje Java, esta práctica es especialmente útil cuando los datos provienen de entradas externas, como formularios, interfaces gráficas o lectura desde teclado mediante `Scanner`. En estos casos, los valores suelen recibirse inicialmente como texto (`String`). Antes de convertir ese texto a un tipo numérico mediante métodos como `Integer.parseInt()` o `Double.parseDouble()`, es recomendable comprobar que el texto realmente representa un número válido.

El Listado de código 6.24 muestra un ejemplo sencillo de validación previa antes de realizar la conversión.

```
1 // Texto ingresado por el usuario
```

```
2      String edadTexto = "25";
3
4      // Verificar si el texto contiene solo dígitos
5      if (edadTexto.matches("\\d+")) {
6
7          // Conversión segura a entero
8          int edad = Integer.parseInt(edadTexto);
9
10         System.out.println("Edad válida: " + edad);
11
12     } else {
13
14         // El texto no representa un número entero válido
15         System.out.println("Error: el valor ingresado no es un número.");
16     }
```

Listing 6.24: Validación previa antes de convertir texto a entero.

En este ejemplo, el método `matches()` se utiliza para comprobar si el texto contiene únicamente dígitos. Si la condición se cumple, el programa realiza la conversión mediante `Integer.parseInt()`. En caso contrario, se muestra un mensaje indicando que el valor ingresado no es válido.

Este enfoque resulta particularmente útil en programas interactivos, donde los datos ingresados por el usuario pueden contener errores tipográficos o formatos incorrectos. Al validar primero el contenido, el programa puede evitar fallos inesperados y proporcionar mensajes claros que orienten al usuario sobre cómo corregir la entrada. En sistemas reales, esta práctica forma parte de las estrategias de validación de datos y mejora la robustez del software al prevenir errores antes de que ocurran durante la ejecución [24], [107].

2) Manejo por excepciones (try-catch). Se intenta convertir y se captura el error. Este enfoque es necesario cuando se trabaja con recursos externos (archivos, red, bases de datos), porque los fallos son parte del entorno, no solo de la entrada [23], [24].

```
1      while (true) {
2          System.out.print("Digite el ID del empleado: ");
3          String t = sc.nextLine();
4          Integer id = parseEntero(t);
5          if (id != null && id > 0) { idEmpleado = id; break; }
6          System.out.println("Entrada inválida. Intente de nuevo.");
7      }
```

Listing 6.25: Manejo de entrada inválida solicitando reintento (esquema).

6.4.2. Presentación de resultados

La presentación de resultados es la etapa final del procesamiento de datos y tiene un impacto directo en la utilidad de la aplicación para el usuario.

La presentación de resultados incluye **clases y métodos** para salida, **formato**, y (en SI reales) **distribución** hacia usuarios correctos y canales correctos. En Java, la salida puede ir a consola (`System.out`), archivos (`Files`), logs (`java.util.logging` u otros), o interfaces gráficas/web/móvil en etapas posteriores [2], [23].

Mensajes informativos

Los mensajes informativos ayudan a los usuarios a comprender el estado del sistema y las acciones que deben realizar.

Buenas prácticas de retroalimentación al usuario:

- Indicar qué acción se espera (*"Digite..."*).
- Indicar formato (*"HH:mm"*).
- Reportar errores con causa y solución (*"ID debe ser entero positivo"*).
- Evitar mensajes genéricos (*"Error"*) sin guía.

En Java:

```
1      System.out.print("Digite la hora de ingreso (HH:mm): ");
2      ...
3      System.out.println("ERROR: formato inválido. Ejemplo válido: 08:12");
```

Listing 6.26: Mensajes informativos: solicitud y error claro.

Salidas principales y secundarias

En muchas aplicaciones, los resultados incluyen tanto salidas principales (resultados del cálculo) como salidas secundarias (información adicional o diagnóstica).

En el ejercicio, una salida principal es el **estado**. Salidas secundarias incluyen minutos de tardanza y registro para auditoría. Ejemplo:

```
1      // principal
2      System.out.println("[Empleado] Estado de ingreso: " + estado);
3      // secundaria
4      System.out.println("[Supervisor] Minutos tarde: " + minutosTarde);
5      // secundaria
6      System.out.println("[Auditoría] id=" + idEmpleado + ", hora=" +
    ↪ horaIngreso);
```

Listing 6.27: Salida principal y secundaria (por roles).

Esta diferenciación refleja una idea central en SI: no toda información se entrega igual; se contextualiza y se dirige [1], [2].

Formato de resultados

El formato de los resultados influye directamente en la claridad de la información presentada al usuario. Un formato adecuado facilita la interpretación de los datos y mejora la experiencia de uso del sistema.

El formato mejora comprensión y evita interpretaciones erróneas. Java ofrece algunas alternativas:

- **Concatenación simple:** adecuada en etapas iniciales.
- **`String.format` / `printf`:** útil para alinear, fijar decimales, y producir reportes básicos.

```
1      System.out.printf("Empleado: %d | Ingreso: %s | Estado: %s | Tarde: %d  
    ↪ min%n",  
2      idEmpleado, horaIngreso, estado, minutosTarde);
```

Listing 6.28: Formato con `printf` para reportes básicos.

Capítulo 7

Modelado de Datos y Expresiones en Aplicaciones

El desarrollo de software implica representar información del mundo real mediante estructuras comprensibles para una computadora. En un sistema informático, los datos constituyen la materia prima sobre la cual operan los programas: describen entidades, registran eventos, representan estados y permiten tomar decisiones automatizadas. Comprender cómo se modelan estos datos y cómo se combinan mediante expresiones constituye uno de los fundamentos más importantes en el aprendizaje de la programación.

Cuando una organización desarrolla un sistema informático —por ejemplo, un sistema de gestión académica, una aplicación bancaria o una plataforma de comercio electrónico— necesita representar diversos tipos de información: números, textos, fechas, estados lógicos o identificadores. Cada uno de estos elementos debe almacenarse y manipularse utilizando estructuras formales que permitan al programa procesarlos de forma precisa y eficiente.

En programación, estas representaciones se materializan mediante **tipos de datos**. Un tipo de dato define qué clase de valores pueden almacenarse, qué operaciones pueden realizarse sobre ellos y cómo se interpretan dentro de un programa. Por ejemplo, una edad puede representarse mediante un número entero, mientras que un nombre debe representarse mediante una cadena de caracteres.

Además de los tipos de datos, los programas utilizan **variables**, que actúan como contenedores donde se almacenan temporalmente los valores que utiliza el programa durante su ejecución. Las variables permiten que el software registre información, la modifique y la combine para generar nuevos resultados.

Otro concepto fundamental es el de **expresión**. Una expresión es una combinación de valores, variables y operadores que produce un resultado. Por ejemplo, una operación como calcular el promedio de dos notas o verificar si un usuario tiene edad suficiente para registrarse en un sistema constituye una expresión que el programa debe evaluar.

El propósito de este capítulo es introducir los principios fundamentales que permiten modelar datos y construir expresiones dentro de una aplicación informática. Primero se presentan los conceptos generales que aplican a cualquier lenguaje de programación y posteriormente se muestran ejemplos concretos utilizando el lenguaje Java, uno de los lenguajes más utilizados en el desarrollo de software moderno.

Desde la perspectiva de la ingeniería del software, el modelado adecuado de los datos constituye una de las primeras decisiones de diseño que determinan la calidad y robustez de un sistema. Una representación incorrecta de la información puede generar errores de cálculo, inconsistencias en los registros o dificultades para mantener y evolucionar el software. Por esta razón, los lenguajes de programación modernos incorporan mecanismos tipados que permiten restringir y controlar las operaciones que pueden realizarse sobre los datos, reduciendo así la probabilidad de errores durante el desarrollo [35], [73].

Diversos estudios empíricos en educación en informática han demostrado que la comprensión temprana de los tipos de datos y de las expresiones aritméticas y lógicas tiene un impacto significativo en la capacidad de los estudiantes para resolver problemas algorítmicos y construir programas correctos [75], [76]. En particular, investigaciones recientes indican que los errores conceptuales relacionados con la interpretación de variables, tipos de datos y operadores constituyen una de las principales dificultades para los estudiantes que se inician en la programación [74]. Por ello, una presentación clara y

progresiva de estos conceptos resulta esencial para desarrollar una comprensión sólida de los fundamentos de la programación.

Con el fin de maximizar la transferencia de aprendizaje, este capítulo se organiza de forma incremental: (i) se definen las categorías de datos más comunes en aplicaciones informacionales; (ii) se introduce la noción de estado del programa mediante variables y constantes; (iii) se formaliza la construcción de expresiones a través de operadores y reglas de evaluación; y (iv) se discuten mecanismos de conversión y validación de datos, enfatizando errores frecuentes en programadores novatos identificados en estudios empíricos [74], [75]. Esta organización busca reducir la carga cognitiva inicial al separar conceptos (qué es un dato) de procedimientos (cómo operar con datos), estrategia consistente con hallazgos de investigación educativa en programación [76].

7.1. Tipos de datos en aplicaciones informacionales

En cualquier sistema informático es necesario representar diferentes clases de información. Cada tipo de información requiere un formato específico que permita almacenarla y procesarla correctamente. Los tipos de datos constituyen el mecanismo mediante el cual los lenguajes de programación clasifican y gestionan estas representaciones.

Un tipo de dato se modela como una abstracción compuesta por: (a) un conjunto de valores posibles; (b) un conjunto de operaciones definidas sobre dichos valores; y (c) una representación interna en memoria [35], [73]. Esta formalización explica por qué no todas las operaciones son válidas para cualquier tipo: por ejemplo, la suma tiene sentido para números, mientras que para cadenas suele definirse la concatenación; de igual modo, comparar por *orden* dos textos requiere reglas diferentes a comparar dos valores numéricos.

Un tipo de dato define tres aspectos fundamentales:

- el conjunto de valores posibles que puede tomar un dato;
- las operaciones que pueden aplicarse sobre dichos valores;
- la forma en que estos valores se almacenan en la memoria del sistema.

Por ejemplo, un número entero puede participar en operaciones de suma o multiplicación, mientras que una cadena de texto puede concatenarse con otras cadenas o analizarse para buscar palabras específicas. Utilizar el tipo de dato adecuado permite evitar errores, mejorar la claridad del programa y optimizar el uso de los recursos computacionales.

En lenguajes de tipado estático como Java, el sistema de tipos ofrece además una ventaja práctica: parte de los errores se detecta durante la compilación, antes de ejecutar el programa. Esta detección temprana es particularmente relevante en proyectos grandes y mantenibles, y se vincula con la confiabilidad del software al impedir combinaciones inválidas de tipos en operaciones y asignaciones [73]. En términos pedagógicos, el sistema de tipos actúa como un mecanismo de retroalimentación inmediata para el estudiante, pues obliga a explicitar y revisar la intención semántica de cada dato [74].

En aplicaciones informacionales suelen distinguirse cuatro categorías básicas de datos: números enteros, números reales, valores lógicos y datos textuales. A continuación se describen estas categorías.

7.1.1. Datos numéricos y lógicos

Los datos numéricos permiten representar cantidades que pueden participar en cálculos matemáticos. En los sistemas informáticos se utilizan principalmente dos tipos: enteros y reales. Además, los programas emplean valores lógicos para representar condiciones o estados que solo pueden ser verdaderos o falsos.

En la práctica, la selección de un tipo numérico no es una decisión meramente sintáctica: afecta la precisión, el rango representable, el rendimiento y la posibilidad de errores como desbordamientos o redondeos. Por esta razón, los lenguajes y las plataformas se apoyan en representaciones estandarizadas: enteros en complemento a dos y números reales en IEEE 754 [11]. Este punto es esencial para comprender por qué una misma expresión puede producir resultados diferentes según el tipo elegido, un patrón de error frecuente en estudiantes novatos [74].

Enteros

Los lenguajes de programación proporcionan diversos tipos de datos enteros destinados a representar cantidades discretas, es decir, valores numéricos que no poseen componente fraccionaria. Este tipo de datos resulta fundamental para modelar conteos,

identificadores, índices y otros valores asociados a estructuras discretas dentro de un sistema computacional. Desde una perspectiva computacional, los enteros se almacenan mediante representaciones binarias que utilizan un número fijo de bits; por tanto, cada tipo entero define un rango finito de valores representables determinado por la cantidad de memoria asignada.

En el lenguaje Java, los tipos enteros primitivos se implementan mediante representación binaria en complemento a dos. Este esquema de codificación permite representar de forma eficiente tanto valores positivos como negativos dentro de un intervalo simétrico determinado por el número de bits disponibles. La representación en complemento a dos constituye el estándar predominante en arquitecturas de computadores modernas debido a su simplicidad aritmética y a la uniformidad que proporciona en la implementación de operaciones básicas como suma y resta [34], [110].

i

Aclaración conceptual

Los números enteros modelan cantidades discretas, es decir, magnitudes que se expresan como unidades completas sin parte fraccionaria. En programación, este tipo de dato se emplea frecuentemente para representar contadores, identificadores, posiciones dentro de estructuras de datos, cantidades de elementos o resultados de operaciones aritméticas que producen valores enteros.

En numerosos dominios de aplicación, los valores enteros representan magnitudes observables dentro del sistema. Por ejemplo, el número de estudiantes matriculados en una asignatura, la cantidad de productos disponibles en un inventario, el número de solicitudes procesadas por un servidor o el índice de un elemento dentro de un arreglo constituyen ejemplos típicos de información representada mediante variables enteras. Este tipo de datos permite modelar de forma natural situaciones en las que la cuantificación se realiza mediante unidades discretas.

El lenguaje Java ofrece varios tipos enteros que difieren principalmente en el tamaño de memoria utilizado para almacenar los valores y, en consecuencia, en el rango numérico que pueden representar. El tipo más utilizado en aplicaciones generales es `int`, el cual corresponde a un entero de 32 bits y ofrece un equilibrio adecuado entre capacidad de representación y eficiencia computacional. El Listado de código 7.1 presenta ejemplos de declaración e inicialización de variables de tipo entero.

```
1      // Declaración de variables enteras
2      int numeroEstudiantes = 35;
3      int edad = 20;
4      int cantidadProductos = 120;
5
6      // Uso de variables enteras en cálculos
7      int total = numeroEstudiantes + cantidadProductos;
8      int diferencia = cantidadProductos - 10;
9
10     // Mostrar resultados
11     System.out.println(total);
12     System.out.println(diferencia);
13
```

Listing 7.1: Datos enteros más comunes en Java.

En el ejemplo anterior se declaran varias variables enteras que representan cantidades discretas utilizadas por el programa. Posteriormente, estas variables se emplean en operaciones aritméticas básicas que generan nuevos valores enteros.

Los tipos de datos enteros disponibles en Java se diferencian principalmente por su tamaño en memoria y el rango de valores que pueden representar. La Tabla 7.1 resume estas características fundamentales.

Cuadro 7.1: Tipos de datos enteros en Java

Tipo	Bytes	Bits	Rango
<code>byte</code>	1	8	−128 a 127
<code>short</code>	2	16	−32 768 a 32 767
<code>int</code>	4	32	−2 147 483 648 a 2 147 483 647
<code>long</code>	8	64	−2 ⁶³ a 2 ⁶³ − 1

Cada uno de estos tipos resulta adecuado para distintos escenarios dependiendo del rango numérico requerido y de las restricciones de memoria del sistema. En aplicaciones de propósito general, `int` suele ser la elección predeterminada, mientras que `long` se utiliza cuando es necesario representar valores considerablemente mayores, como contadores de eventos, marcas temporales o identificadores globales.

Nota: El lenguaje Java no define tipos enteros sin signo (*unsigned*) como tipos primitivos. Sin embargo, a partir de Java 8 se introdujeron métodos utilitarios en la clase `Integer` y `Long` que permiten interpretar valores enteros con signo como si fueran enteros sin signo. Este mecanismo resulta particularmente útil en contextos como procesamiento de protocolos de red, manipulación de estructuras binarias o interoperabilidad con sistemas que utilizan representaciones sin signo.

Una propiedad importante de los tipos enteros de tamaño fijo es la posibilidad de que ocurra un fenómeno denominado *desbordamiento* (*overflow*). Este fenómeno se produce cuando el resultado de una operación aritmética excede el rango máximo o mínimo representable por el tipo de dato utilizado. En tales situaciones, el valor resultante se ajusta mediante aritmética modular conforme a la representación en complemento a dos, provocando que el resultado “envuelva” el rango permitido. Este comportamiento, aunque definido por la especificación del lenguaje, puede generar errores lógicos difíciles de detectar cuando no se considera explícitamente durante el diseño del software [34], [107].

En sistemas profesionales, la prevención de desbordamientos suele abordarse mediante diversas estrategias: selección cuidadosa del tipo de dato más adecuado para el dominio del problema, validaciones de rango antes de realizar cálculos críticos, o utilización de bibliotecas de precisión arbitraria. En Java, la clase `BigInteger` permite representar enteros de tamaño prácticamente ilimitado, lo cual resulta apropiado en aplicaciones criptográficas, cálculos científicos o manipulación de grandes cantidades numéricas.

i

Error conceptual frecuente

Un error común consiste en asumir que los números enteros en programación poseen capacidad ilimitada. En realidad, cada tipo entero tiene un rango finito determinado por el número de bits que utiliza para representar los valores. La selección adecuada del tipo entero y la verificación de límites constituyen prácticas esenciales para evitar comportamientos inesperados durante la ejecución de un programa [74], [87].

Reales

Además de los valores enteros, muchas aplicaciones requieren representar cantidades que contienen parte decimal, como mediciones, promedios, porcentajes o resultados de cálculos científicos. Para este propósito, los lenguajes de programación emplean tipos de datos de punto flotante, que permiten representar números reales mediante una aproximación binaria.

Los números reales permiten modelar magnitudes que incluyen una parte fraccionaria. Este tipo de datos resulta imprescindible cuando se realizan cálculos asociados a mediciones físicas, análisis estadísticos, estimaciones numéricas o representación de valores continuos.

Por ejemplo, el promedio de calificaciones de un estudiante puede incluir valores decimales como 8.75 o 9.50. De forma similar, el precio de un producto, la temperatura registrada por un sensor o el resultado de una medición experimental suelen representarse mediante números reales.

En Java, los números reales se representan mediante los tipos de datos de punto flotante `float` y `double`. El tipo `double` es el más utilizado en la práctica debido a su mayor precisión y rango de representación. El Listado de código 7.2 presenta ejemplos de declaración e inicialización de variables reales utilizando este tipo de dato.

```
1      // Declaración de variables de tipo double
2      double promedio = 8.75;
3      double temperatura = 23.4;
4      double precio = 19.99;
5
6      // Uso de valores reales en operaciones aritméticas
7      double suma = promedio + temperatura;
8      double producto = precio * 2;
9
10     // Mostrar resultados
11     System.out.println(suma);
12     System.out.println(producto);
13
```

Listing 7.2: Datos Reales de Mayor Precisión en Java.

Estos valores permiten que el programa realice cálculos matemáticos con mayor precisión que los números enteros, especialmente cuando intervienen fracciones o resultados que no pueden representarse mediante valores discretos.

i

Normalización científica

En Java, los tipos de punto flotante siguen el estándar IEEE 754 para la representación de números reales en sistemas digitales. Este estándar define la forma en que los valores se almacenan en memoria, cómo se realizan las operaciones aritméticas y cómo se gestionan situaciones especiales como el desbordamiento o la representación de infinitos y valores no numéricos (*NaN*) [11].

La Tabla 7.2 presenta los tipos de datos de punto flotante disponibles en Java, indicando su tamaño en memoria, el rango aproximado de valores que pueden representar y su nivel de precisión decimal.

Una propiedad fundamental de la aritmética de punto flotante es que muchas fracciones decimales no poseen una representación binaria exacta. En consecuencia, ciertas operaciones pueden producir pequeños errores de redondeo que se manifiestan al realizar cálculos sucesivos o iterativos. Este comportamiento ha sido ampliamente estudiado en la literatura científica sobre computación numérica y constituye una característica inherente a la representación de números reales en sistemas digitales [11], [12].

Cuadro 7.2: Tipos de datos de punto flotante en Java

Tipo	Tamaño (bytes)	Rango aproximado	Precisión decimal
<code>float</code>	4	$\pm 1.4 \times 10^{-45}$ a $\pm 3.4 \times 10^{38}$	~ 7 dígitos
<code>double</code>	8	$\pm 4.9 \times 10^{-324}$ a $\pm 1.8 \times 10^{308}$	~ 15–16 dígitos

En aplicaciones donde se requiere exactitud decimal estricta —como cálculos financieros, contabilidad o manejo de impuestos— suele preferirse el uso de la clase `BigDecimal`, la cual permite controlar explícitamente la precisión y las reglas de redondeo utilizadas durante las operaciones aritméticas [107].

i

Recomendación práctica

Para cálculos científicos, estadísticos o de ingeniería general, el tipo `double` suele ser la elección por defecto debido a su equilibrio entre precisión y rendimiento. En cambio, para operaciones monetarias o financieras, se recomienda utilizar `BigDecimal` junto con políticas explícitas de redondeo para evitar errores acumulativos en los cálculos [107].

Booleanos

Los valores booleanos se emplean para representar condiciones lógicas que únicamente pueden adoptar dos estados posibles: verdadero o falso. Este tipo de dato constituye uno de los fundamentos del razonamiento computacional, ya que permite que un programa evalúe situaciones y determine qué acciones ejecutar en función de determinadas condiciones.

En términos prácticos, muchas decisiones dentro de un programa dependen de evaluaciones booleanas. Por ejemplo, un sistema puede necesitar comprobar si un usuario ha iniciado sesión correctamente, si una transacción ha sido confirmada, si un archivo existe en el sistema o si un estudiante ha alcanzado la calificación mínima requerida para aprobar una asignatura. En todos estos casos, la respuesta a la evaluación se expresa mediante un valor lógico que puede ser verdadero o falso.

En Java, los valores booleanos se representan mediante el tipo primitivo `boolean`. Este tipo de dato permite almacenar únicamente dos posibles valores: `true` o `false`. El Listado de código 7.3 muestra ejemplos simples de declaración e inicialización de variables booleanas.

```

1          // Declaración de variables booleanas
2  boolean usuarioAutenticado = true;
3  boolean examenAprobado = false;
4
5  // Uso de valores booleanos en mensajes
6  System.out.println(usuarioAutenticado);
7  System.out.println(examenAprobado);

```

Listing 7.3: Datos Booleanos.

Las variables booleanas suelen emplearse para representar el resultado de una verificación o de una comparación. Estas verificaciones se expresan mediante *expresiones booleanas*, las cuales producen como resultado un valor lógico. Una expresión booleana puede construirse utilizando operadores relacionales que comparan valores numéricos o textuales.

El Listado de código 7.4 ilustra algunos ejemplos de expresiones booleanas que comparan valores.


```
1      // Comparaciones entre números
2      boolean mayor = 10 > 5;
3      boolean menor = 3 < 1;
4      boolean igual = 7 == 7;
5
6      // Mostrar resultados de las comparaciones
7      System.out.println(mayor);
8      System.out.println(menor);
9      System.out.println(igual);
```

Listing 7.4: Ejemplos de expresiones booleanas.

Además de los operadores relacionales, Java dispone de operadores lógicos que permiten combinar múltiples condiciones. Estos operadores se utilizan cuando una decisión depende de más de una evaluación.

Los operadores lógicos más utilizados son:

- **&&** (AND lógico): el resultado es verdadero solo si ambas condiciones son verdaderas.
- **||** (OR lógico): el resultado es verdadero si al menos una de las condiciones es verdadera.
- **!** (NOT lógico): invierte el valor lógico de una condición.

El Listado de código 7.5 presenta ejemplos del uso de estos operadores.

```
1      // Condiciones iniciales
2      boolean tieneCuenta = true;
3      boolean tieneSaldo = false;
4
5      // Uso del operador AND
6      boolean puedeComprar = tieneCuenta && tieneSaldo;
7
8      // Uso del operador OR
9      boolean accesoPermitido = tieneCuenta || tieneSaldo;
10
11     // Uso del operador NOT
12     boolean sinCuenta = !tieneCuenta;
13
14     // Mostrar resultados
15     System.out.println(puedeComprar);
16     System.out.println(accesoPermitido);
17     System.out.println(sinCuenta);
```

Listing 7.5: Uso de operadores lógicos.

Estos valores lógicos se utilizan posteriormente en estructuras de control que permiten al programa decidir qué acciones ejecutar. Por ejemplo, las sentencias condicionales (`if`) o las estructuras repetitivas (`while`, `for`) evalúan expresiones booleanas para determinar el flujo de ejecución del programa.

Desde una perspectiva pedagógica, las expresiones booleanas suelen generar dificultades en estudiantes que comienzan a programar. Las confusiones más comunes se relacionan con la diferencia entre operadores relacionales (`>`, `<`, `==`) y operadores lógicos (`&&`, `||`, `!`), así como con la precedencia de estos operadores dentro de una expresión. Una estrategia recomendable consiste en construir condiciones complejas de manera gradual, evaluando primero subexpresiones simples y utilizando paréntesis para hacer explícita la prioridad de las operaciones. Esta práctica favorece la legibilidad del código y reduce errores lógicos durante el desarrollo de programas [74], [87].

7.1.2. Texto y estructuras básicas

Además de números y valores lógicos, los programas deben manipular información textual. Los textos permiten representar nombres de personas, direcciones, descripciones, mensajes informativos, códigos de identificación y otros elementos que forman parte del intercambio de información entre los usuarios y los sistemas informáticos. En consecuencia, el tratamiento adecuado del texto constituye una capacidad esencial dentro de cualquier lenguaje de programación.

La manipulación de texto interviene en múltiples tareas computacionales: validación de formularios, generación de reportes, comunicación con el usuario, procesamiento de datos provenientes de archivos o redes, e incluso análisis de información estructurada o semiestructurada. Por esta razón, los lenguajes de programación modernos incorporan mecanismos específicos para representar y procesar información textual de forma eficiente y segura. El manejo correcto de cadenas y caracteres forma parte de las competencias fundamentales del desarrollo de software, ya que los sistemas informáticos operan constantemente sobre información expresada en forma textual [24], [87].

Caracteres

Un carácter representa la unidad mínima de información textual que puede manipular un programa. En términos generales, un carácter puede corresponder a una letra del alfabeto, un número, un signo de puntuación o cualquier símbolo utilizado en la escritura.

En Java, los caracteres se representan mediante el tipo primitivo `char`. Este tipo de dato permite almacenar un único símbolo y se utiliza cuando el programa necesita trabajar con elementos individuales de texto. El Listado de código 7.6 muestra ejemplos sencillos de declaración e inicialización de variables de tipo `char`.

```
1      // Declaración de caracteres individuales
2      char inicial = 'A';
3      char simbolo = '#';
4
5      // Los caracteres también pueden mostrarse en pantalla
6      System.out.println(inicial);
7      System.out.println(simbolo);
```

Listing 7.6: Datos Carácter (char).

Los caracteres se escriben entre comillas simples (' '). Esta convención permite diferenciar claramente un carácter individual de una cadena de texto, la cual se delimita mediante comillas dobles (" "). Comprender esta diferencia es fundamental para evitar errores sintácticos durante la programación.

El uso de caracteres resulta particularmente útil cuando se requiere analizar texto de forma individual. Por ejemplo, algunos algoritmos recorren palabra por palabra o letra por letra para verificar patrones, contar ocurrencias de determinados símbolos o validar formatos de datos.

El Listado de código 7.7 presenta un ejemplo donde se obtiene un carácter específico de una cadena utilizando su posición.

```
1      // Cadena de texto
2      String palabra = "Java";
3
4      // Obtención del primer caracter de la cadena
5      char primeraLetra = palabra.charAt(0);
6
7      // Mostrar el caracter obtenido
8      System.out.println(primeraletra);    // Resultado: J
```

Listing 7.7: Acceso a caracteres dentro de una cadena.

En el ejemplo anterior se observa cómo el método `charAt()` permite acceder a un carácter específico dentro de una cadena. Las posiciones se numeran a partir de cero, lo cual significa que el primer carácter se encuentra en la posición 0, el segundo en la posición 1, y así sucesivamente. Esta convención de indexación es común en la mayoría de lenguajes de programación y constituye un concepto fundamental al trabajar con estructuras de datos indexadas.

El tipo `char` en Java representa unidades de código del estándar Unicode utilizando codificación UTF-16. Este estándar permite representar una gran variedad de símbolos utilizados en diferentes idiomas y sistemas de escritura alrededor del mundo. Como consecuencia, Java puede manipular texto multilingüe de manera consistente y portable entre plataformas [110].

No obstante, algunos símbolos Unicode complejos requieren dos unidades de código denominadas *surrogate pairs*. Aunque este aspecto pertenece a niveles más avanzados del tratamiento de texto, es importante reconocer que la arquitectura del lenguaje fue diseñada para soportar la diversidad lingüística y cultural presente en los sistemas informáticos actuales. Para un nivel introductorio, basta comprender que el tipo `char` almacena un símbolo individual, mientras que las cadenas de texto (`String`) representan secuencias completas de caracteres que pueden manipularse mediante diversos métodos [107].

Cadenas

Las cadenas de caracteres permiten representar secuencias completas de texto. Este tipo de dato es uno de los más utilizados en las aplicaciones informáticas, ya que la mayoría de los sistemas interactúan con los usuarios mediante mensajes, nombres, direcciones, descripciones, códigos o resultados expresados en forma textual.

En Java, las cadenas se representan mediante la clase `String`. Una variable de este tipo almacena una secuencia de caracteres que puede mostrarse en pantalla, almacenarse en archivos o bases de datos, o manipularse mediante diferentes métodos definidos en dicha clase.

El Listado de código 7.8 muestra un ejemplo básico de declaración e inicialización de variables de tipo `String`. En este ejemplo se crean dos objetos que almacenan textos literales y posteriormente pueden utilizarse dentro del programa.

```
1      // Declaración e inicialización de cadenas
2      String nombre = "Carlos";
```

```
3      String mensaje = "Bienvenido al sistema";
4
5      // Las cadenas pueden mostrarse en pantalla
6      System.out.println(nombre);
7      System.out.println(mensaje);
```

Listing 7.8: Datos (Objetos) String.

Las cadenas permiten almacenar textos completos que posteriormente pueden mostrarse al usuario, almacenarse en bases de datos o procesarse mediante diversas operaciones como búsqueda, comparación, extracción de fragmentos o transformación de letras.

Un rasgo esencial de **String** en Java es su **inmutabilidad**. Esto significa que, una vez creada una cadena, su contenido no cambia. Cuando un método parece modificar una cadena, en realidad se crea un nuevo objeto con el resultado de la operación.

El Listado de código 7.9 ilustra esta característica. Aunque se invoca el método `toUpperCase()`, el valor almacenado en la variable no cambia porque el resultado no se asigna a ninguna variable.

```
1      // Se crea una cadena inicial
2      String nombre = "Carlos";
3
4      // Se invoca un método que genera una nueva cadena
5      nombre.toUpperCase();
6
7      // La cadena original no cambia
8      System.out.println(nombre);    // Resultado: Carlos
```

Listing 7.9: Ejemplo de inmutabilidad de String.

En el ejemplo anterior, el método `toUpperCase()` produce una nueva cadena, pero el resultado no se guarda en ninguna variable. Por ello, la variable `nombre` mantiene su valor original.

i

Implicación práctica

Si se desea conservar el resultado de una operación sobre una cadena, el valor devuelto debe asignarse a una variable.

El Listado de código 7.10 muestra la forma correcta de conservar el resultado de una operación sobre una cadena.

```
1      // Cadena original
2      String nombre = "Carlos";
3
```

```
4      // Conversión a mayúsculas
5      nombre = nombre.toUpperCase();
6
7      // Ahora la variable contiene el nuevo valor
8      System.out.println(nombre);    // Resultado: CARLOS
```

Listing 7.10: Guardar el resultado de una operación sobre String.

La sintaxis general para utilizar los métodos de la clase `String` sigue el siguiente patrón:

```
variableCadena.metodo(argumentos);
```

Esto significa que primero se escribe el nombre de la variable que contiene la cadena, seguido de un punto (.) y posteriormente el nombre del método que se desea ejecutar.

El Listado de código 7.11 presenta algunos ejemplos básicos de uso de métodos de la clase `String`.

```
1      // Cadena de ejemplo
2      String texto = "Programacion";
3
4      // Obtener longitud de la cadena
5      int longitud = texto.length();
6
7      // Obtener un caracter específico
8      char letra = texto.charAt(0);
9
10     // Convertir a mayúsculas
11     String mayusculas = texto.toUpperCase();
```

Listing 7.11: Uso básico de métodos de la clase String.

A partir del ejemplo mostrado en el Listado de código 7.11, resulta conveniente disponer de una referencia compacta que permita identificar rápidamente qué método aplicar según la tarea de procesamiento de texto requerida (por ejemplo, medir longitudes para validación, extraer subcadenas para segmentación, normalizar mayúsculas/minúsculas para comparación, o dividir entradas en campos). Con este objetivo, la Tabla 7.3 sintetiza un conjunto de métodos de uso frecuente de `String`, indicando su tipo de retorno, el efecto funcional esperado y un ejemplo mínimo de invocación; esta organización favorece la selección informada del método y reduce errores típicos en principiantes, como confundir comparación de contenido con identidad o asumir modificaciones “en el lugar” sobre cadenas inmutables [74], [107], [110].

Los métodos anteriores permiten realizar múltiples operaciones sobre cadenas. El Listado de código 7.12 muestra algunos ejemplos prácticos de estas operaciones.

```
1      // Cadena de ejemplo
2      String texto = "Aprender Java es interesante";
```

Cuadro 7.3: Métodos comunes de la clase `String` en Java

Método	Retorno	Función	Ejemplo
<code>length()</code>	<code>int</code>	Devuelve la longitud de la cadena.	<code>nombre.length()</code>
<code>charAt(int)</code>	<code>char</code>	Devuelve el carácter en la posición indicada.	<code>nombre.charAt(0)</code>
<code>substring(int)</code>	<code>String</code>	Devuelve una subcadena desde una posición hasta el final.	<code>texto.substring(3)</code>
<code>substring(int,int)</code>	<code>String</code>	Devuelve una subcadena entre dos posiciones.	<code>texto.substring(1,4)</code>
<code>toUpperCase()</code>	<code>String</code>	Convierte la cadena a mayúsculas.	<code>nombre.toUpperCase()</code>
<code>toLowerCase()</code>	<code>String</code>	Convierte la cadena a minúsculas.	<code>nombre.toLowerCase()</code>
<code>equals(String)</code>	<code>boolean</code>	Compara si dos cadenas tienen el mismo contenido.	<code>nombre.equals("Juan")</code>
<code>equalsIgnoreCase(String)</code>	<code>boolean</code>	Compara cadenas ignorando mayúsculas y minúsculas.	<code>nombre.equalsIgnoreCase("juan")</code>
<code>contains(CharSequence)</code>	<code>boolean</code>	Verifica si la cadena contiene otra cadena.	<code>texto.contains("Java")</code>
<code>replace(char,char)</code>	<code>String</code>	Reemplaza caracteres dentro de la cadena.	<code>texto.replace('a','o')</code>
<code>trim()</code>	<code>String</code>	Elimina espacios al inicio y al final de la cadena.	<code>texto.trim()</code>
<code>split(String)</code>	<code>String[]</code>	Divide la cadena en partes usando un separador.	<code>texto.split(",")</code>

```

3
4          // Obtener la longitud del texto
5          int longitud = texto.length();
6
7          // Extraer una parte del texto
8          String palabra = texto.substring(9, 13);
9

```

```
10          // Verificar si el texto contiene una palabra
11          boolean contiene = texto.contains("Java");
12
13          // Reemplazar palabras
14          String nuevoTexto = texto.replace("Java",
    ↪ "programacion");
15
16          // Convertir el texto a minúsculas
17          String minusculas = texto.toLowerCase();
18
```

Listing 7.12: Operaciones comunes con cadenas.

Otro uso frecuente consiste en dividir una cadena en varias partes mediante un separador. Esto resulta útil cuando se procesan listas de valores o datos separados por comas.

El Listado de código 7.13 ilustra cómo utilizar el método `split()` para separar una cadena en múltiples elementos.

```
1          // Cadena con valores separados por comas
2          String datos = "Ana,Carlos,Maria,Pedro";
3
4          // Dividir la cadena usando la coma como
    ↪ separador
5          String[] nombres = datos.split(",");
6
7          // Acceso a los elementos del arreglo resultante
8          System.out.println(nombres[0]);    // Ana
9          System.out.println(nombres[1]);    // Carlos
10         System.out.println(nombres[2]);    // Maria
11
```

Listing 7.13: División de cadenas con `split`.

En la comparación de cadenas es importante distinguir entre *igualdad de contenido* y *igualdad de referencia*. El método `equals()` compara si dos cadenas tienen exactamente el mismo contenido. En cambio, el operador `==` verifica si ambas variables apuntan al mismo objeto en memoria.

El Listado de código 7.14 muestra un ejemplo de comparación entre cadenas.

```
1          // Dos cadenas con el mismo texto
2          String a = "Java";
3          String b = "Java";
```

```
4
5                                     // Comparacion por contenido
6    boolean iguales = a.equals(b);    // true
7
8                                     // Comparacion por referencia
9    boolean mismaReferencia = (a == b);
10
```

Listing 7.14: Comparación correcta de cadenas.

Por esta razón, para comparar textos se recomienda utilizar siempre `equals()` o `equalsIgnoreCase()` cuando no se desea distinguir entre mayúsculas y minúsculas. Esta práctica evita errores comunes en los programas que procesan entradas de texto proporcionadas por el usuario [74], [107].

Operaciones frecuentes

En la mayoría de aplicaciones, las cadenas de texto constituyen un tipo de dato central para representar nombres, mensajes, códigos, rutas, descripciones y resultados de procesamiento. Por esta razón, es habitual realizar operaciones sobre textos tales como obtener su longitud, acceder a caracteres específicos, comparar contenidos, concatenar valores y transformar el formato de letras (por ejemplo, mayúsculas y minúsculas). Estas operaciones se emplean tanto en la validación de entradas del usuario como en la generación de salidas legibles y consistentes.

En Java, dichas tareas se implementan mediante métodos provistos por la clase `String`. Dado que `String` es un tipo inmutable, cada operación que aparenta modificar el contenido (como concatenar o transformar a mayúsculas) produce en realidad una nueva instancia. En consecuencia, el diseño del programa debe considerar el contexto: en operaciones puntuales, el uso de `String` es suficiente; en construcción incremental repetida, conviene usar `StringBuilder` [107].

```
1    // Cadena base (literal) asignada a una variable
2    String nombre = "Maria";
3
4    // 1) Longitud de la cadena: cantidad de caracteres
5    int longitud = nombre.length();
6    // Devuelve un int con el número de caracteres
7
8    // 2) Concatenación de cadenas
9    String saludo = "Hola " + nombre;
10   // Crea una nueva cadena combinando partes
11
12   // 3) Transformaciones de mayúsculas/minúsculas
13   String nombreMayus = nombre.toUpperCase();
14   // Devuelve una nueva cadena en mayúsculas
15   String nombreMinus = nombre.toLowerCase();
16   // Devuelve una nueva cadena en minúsculas
```



```
17
18 // 4) Eliminación de espacios en extremos (útil para entradas del
    ↪ usuario)
19 String nombreConEspacios = "  Maria  ";
20 String nombreLimpio = nombreConEspacios.trim();
21 // Quita espacios al inicio y al final
22
23 // 5) Acceso a un carácter por índice (índices desde 0)
24 char primeraLetra = nombre.charAt(0);
25 // Obtiene el primer carácter ('M')
26
27 // 6) Búsqueda dentro de la cadena
28 boolean contieneRi = nombre.contains("ri");
29 // Indica si existe la subcadena "ri"
30 int posA = nombre.indexOf("a");
31 // Devuelve la primera posición de "a" (o -1 si no existe)
32
33 // 7) Comparación de cadenas (contenido), no referencias
34 boolean mismoTexto = nombre.equals("Maria");
35 // true si el contenido es exactamente igual
36
37 // 8) Extracción de un fragmento (subcadena)
38 String sub = nombre.substring(0, 3);
39 // "Mar" (desde 0 inclusive hasta 3 exclusivo)
40
41 // 9) Reemplazo de partes del texto
42 String reemplazo = nombre.replace("a", "á");
43 // "Maríá" (ejemplo de reemplazo simple)
44
45 // 10) Verificación de prefijo/sufijo
46 boolean iniciaConMa = nombre.startsWith("Ma");
47 // true si comienza con "Ma"
48
49 boolean terminaConIa = nombre.endsWith("ia");
50 // true si termina con "ia"
51
```

Listing 7.15: Inicialización de variables en Java.

El Listado de código 7.15 integra operaciones representativas que aparecen de forma recurrente en ejercicios académicos y en aplicaciones reales. En primer lugar, `length()` retorna la cantidad de caracteres, lo cual resulta útil para validar longitudes mínimas o máximas (por ejemplo, contraseñas o códigos). La concatenación con el operador `+` permite construir mensajes de forma directa, como en `"Hola " + nombre`. En ambos casos, el resultado final es un nuevo valor almacenado en una variable: la cadena original permanece sin cambios debido a la inmutabilidad de `String`.

La sintaxis general para invocar un método de `String` sigue el patrón:

```
objetoString.metodo(argumentos)
```

i

Implicación práctica

Si se ejecuta `nombre.toUpperCase()`; sin asignar el resultado, la variable `nombre` mantiene su valor original. Para conservar el cambio se requiere `nombre = nombre.toUpperCase()`;

En términos prácticos, `nombre.length()` se lee como: “aplicar el método `length` al objeto `nombre`”. Si el método requiere parámetros, se colocan entre paréntesis, por ejemplo `nombre.contains("ri")` o `nombre.substring(0,3)`. El valor devuelto por el método puede asignarse a una variable del tipo correspondiente: `int` para `length()` o `indexOf()`, `boolean` para `contains()` o `startsWith()`, `char` para `charAt()`, y `String` para `toUpperCase()`, `trim()`, `substring()` o `replace()`.

i

Muy Importante...

Un aspecto crítico para el aprendizaje correcto es la comparación de cadenas. En Java, `equals()` compara el *contenido* textual, mientras que el operador `==` compara si dos referencias apuntan al mismo objeto. Por ello, para validar entradas (por ejemplo, un usuario o una opción de menú) debe emplearse `equals()` o, si se desea ignorar mayúsculas/minúsculas, `equalsIgnoreCase()`.

En escenarios donde se concatena repetidamente dentro de bucles (por ejemplo, al generar reportes, armar salidas tabulares o construir textos a partir de listas), la inmutabilidad de `String` puede degradar el rendimiento, ya que cada concatenación crea objetos intermedios y aumenta el trabajo del recolector de basura. Para construcción incremental, se recomienda `StringBuilder`, que permite añadir fragmentos con `append()` y obtener el resultado final con `toString()`, manteniendo eficiencia y claridad en el código [107].

7.2. Variables y gestión del estado

Las variables permiten almacenar información que puede cambiar durante la ejecución de un programa. Cada variable posee un nombre, un tipo de dato y un valor asociado.

El uso correcto de las variables permite que los programas registren información temporalmente y la utilicen para realizar cálculos o tomar decisiones.

En términos de ejecución, una variable puede entenderse como un identificador asociado a una ubicación de memoria cuyo contenido representa el estado parcial del programa. Dado que muchos errores en programación introductoria se originan por estados no esperados (variables no inicializadas, variables reutilizadas con significados distintos o valores que cambian en el orden equivocado), se recomienda practicar la trazabilidad del estado: observar cómo cambian las variables paso a paso y relacionar cada cambio con la instrucción que lo produce [74], [76].

7.2.1. Declaración e inicialización

Las variables constituyen uno de los mecanismos fundamentales mediante los cuales un programa puede almacenar y manipular información durante su ejecución. Para que una variable pueda utilizarse dentro de un programa, primero debe ser declarada. La declaración de una variable consiste en indicar al compilador el tipo de dato que la variable almacenará y el nombre que permitirá referenciarla dentro del programa. Este proceso establece la existencia de la variable dentro del ámbito correspondiente y define el tipo de operaciones que pueden realizarse con ella [35], [73].

En el lenguaje Java, la declaración de una variable se realiza especificando primero el tipo de dato y posteriormente el identificador que representará la variable dentro del código. El Listado de código 7.16 muestra algunos ejemplos simples de declaración de variables utilizando distintos tipos de datos.

```
1    int edad;  
2    double promedio;  
3    String nombre;  
4
```

Listing 7.16: Ejemplos de declaración de variables en Java.

En el Listado de código 7.16, cada instrucción indica al compilador que el programa utilizará una variable con un nombre determinado y un tipo específico de dato. Por ejemplo, la variable `edad` se declara como un valor entero, lo que significa que podrá almacenar únicamente números enteros. De manera similar, `promedio` se declara como un valor de tipo `double`, lo cual permite almacenar números reales con parte decimal, mientras que `nombre` se declara como un objeto de tipo `String`, utilizado para representar texto.

Una vez declarada la variable, el siguiente paso consiste en asignarle un valor inicial. Este proceso se denomina inicialización y consiste en asociar a la variable un dato concreto que podrá ser utilizado posteriormente en operaciones o cálculos. El Listado de código 7.17 presenta un ejemplo sencillo de inicialización de variables.

```
1    int edad = 18;  
2    double promedio = 9.5;  
3    String nombre = "Laura";  
4
```

Listing 7.17: Inicialización de variables en Java.

En el Listado de código 7.17, cada variable se declara y se inicializa en una misma instrucción. Este estilo de programación resulta común en programas reales porque permite garantizar que las variables comienzan su ciclo de vida con un valor definido. La inicialización temprana de variables contribuye a evitar errores durante la ejecución del programa, especialmente en situaciones donde una variable podría utilizarse antes de haber recibido un valor válido.

Desde una perspectiva de calidad del software, asignar valores iniciales adecuados a las variables contribuye a mejorar la confiabilidad del programa. Cuando las variables poseen valores definidos desde el inicio, resulta más sencillo comprender el comportamiento del algoritmo y reducir la probabilidad de errores asociados con estados indefinidos del sistema [23], [24].

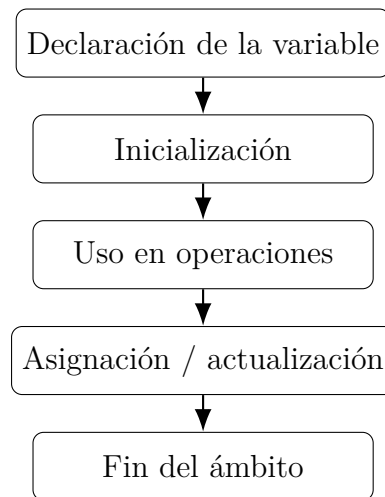
Para comprender mejor el proceso de manejo de variables, resulta útil distinguir tres operaciones fundamentales: declaración, inicialización y asignación (ver Tabla 7.4). Aunque estas operaciones suelen aparecer juntas en muchos programas, conceptualmente representan acciones diferentes dentro del ciclo de vida de una variable.

Cuadro 7.4: Declaración, inicialización y asignación de variables

Concepto	Descripción	Ejemplo en Java
Declaración	Se define el tipo de dato y el nombre de la variable que se utilizará en el programa.	<code>int edad;</code>
Inicialización	Se asigna el primer valor a la variable en el momento de su declaración o inmediatamente después.	<code>int edad = 18;</code>
Asignación	Se modifica el valor almacenado en la variable después de haber sido declarada.	<code>edad = 20;</code>

En Java, el proceso de inicialización también se relaciona con el concepto de *valor por defecto*. Los atributos de una clase reciben automáticamente un valor inicial cuando el objeto es creado, incluso si el programador no ha especificado explícitamente dicho valor. Por ejemplo, los atributos de tipo `int` se inicializan automáticamente con el valor 0, los atributos de tipo `boolean` con el valor `false`, y las referencias a objetos con el valor `null`. La Tabla

Es importante distinguir entre variables locales y atributos de clase al analizar el proceso de inicialización. Mientras que los atributos de clase reciben valores por defecto automáticamente, las variables locales deben inicializarse explícitamente antes de utilizarse. Si un programa intenta utilizar una variable local sin haberle asignado previamente un valor, el compilador generará un error que impedirá la ejecución del programa.

**Figura 7.1:** Ciclo de vida conceptual de una variable dentro de un programa.

Comprender este ciclo de vida (ver Figura 7.1) permite interpretar cómo evolucionan los valores de las variables durante la ejecución de un programa y cómo los distintos bloques de código determinan el momento en que una variable es creada, utilizada y finalmente descartada.

Generalmente, en las primeras etapas del aprendizaje de la programación es frecuente cometer ciertos errores al declarar o utilizar variables. Identificar estos errores permite mejorar la comprensión del funcionamiento del lenguaje y desarrollar programas más correctos. Algunos de estos errores se muestran en la Tabla 7.5

Cuadro 7.5: Errores comunes al declarar variables

Error	Ejemplo	Consecuencia
Usar una variable sin inicializar	<code>int x;</code> <code>System.out.println(x);</code>	Error de compilación en Java.
Utilizar nombres poco descriptivos	<code>int x = 20;</code>	Dificulta comprender el propósito de la variable.
Reutilizar una variable para distintos propósitos	<code>x = edad;</code> <code>x = precio;</code>	Puede generar confusión en la lógica del programa.
Declarar variables innecesarias	Variables que nunca se utilizan	Aumenta la complejidad del código.

Buenas prácticas

El manejo adecuado de variables constituye uno de los pilares fundamentales en la construcción de programas correctos, comprensibles y mantenibles. Aunque la sintaxis de los lenguajes de programación permite declarar variables de manera relativamente sencilla, la calidad de un programa depende en gran medida de las decisiones que el programador toma al nombrarlas, inicializarlas y utilizarlas dentro de los algoritmos. En la práctica profesional de la ingeniería de software, una parte significativa de la calidad del código está asociada con la claridad semántica de los identificadores y con la organización coherente de los datos que intervienen en los procesos computacionales [23], [24].

Desde una perspectiva formativa, desarrollar buenas prácticas en el uso de variables permite construir programas que no solo funcionan correctamente, sino que también pueden ser comprendidos, analizados y modificados con facilidad. Diversos estudios sobre aprendizaje de la programación muestran que muchos errores cometidos por estudiantes principiantes se relacionan con la gestión incorrecta de variables, como el uso de nombres ambiguos, la reutilización innecesaria de variables para diferentes propósitos o la falta de inicialización adecuada antes de su utilización [74], [75], [76]. Por esta razón, adoptar principios de organización y claridad desde las primeras etapas del aprendizaje contribuye a desarrollar habilidades de programación más sólidas.

Entre las prácticas más recomendadas para el manejo adecuado de variables se encuentran las siguientes:

- **Utilizar nombres descriptivos y semánticamente significativos.** Los nombres de las variables deben expresar claramente el significado de la información que almacenan. Un identificador bien elegido permite comprender el propósito de la variable sin necesidad de analizar el resto del algoritmo. Por ejemplo, un nombre como `promedioEstudiante` comunica de manera explícita el tipo de dato representado, mientras que identificadores genéricos como `x`, `valor1` o `dato` dificultan la interpretación del código.
- **Inicializar siempre las variables antes de utilizarlas.** Toda variable debe poseer un valor definido antes de participar en operaciones aritméticas, comparaciones o evaluaciones lógicas. Operar con variables no inicializadas puede producir resultados impredecibles o comportamientos incorrectos. En el lenguaje Java, el compilador exige que las variables locales sean inicializadas antes de su uso, lo cual contribuye a prevenir errores en tiempo de ejecución.
- **Asignar a cada variable una única responsabilidad dentro del algoritmo.** Una variable debe representar un único concepto dentro del programa. Reutilizar una misma variable para representar distintos significados en diferentes partes del algoritmo puede generar ambigüedad y dificultar la comprensión del código. Definir variables con responsabilidades claras mejora la estructura conceptual del programa.
- **Declarar las variables en el ámbito más reducido posible.** Limitar el alcance de una variable al bloque donde realmente se utiliza permite controlar con mayor precisión su ciclo de vida y reduce la probabilidad de modificaciones accidentales desde otras partes del programa. Esta práctica contribuye a mejorar la organización del código y facilita su análisis.
- **Evitar valores literales repetidos dentro del código.** Cuando un mismo valor aparece en múltiples partes del programa, es recomendable declararlo como una constante. Esta estrategia evita la presencia de valores dispersos y

facilita la modificación futura de reglas del sistema.

- **Escribir código orientado a la comprensión humana.** Un programa debe ser escrito no solo para que la computadora lo ejecute, sino también para que otros desarrolladores puedan comprender su funcionamiento. La claridad en la definición de variables facilita la lectura del código y reduce el esfuerzo necesario para analizar su lógica.

La aplicación sistemática de estas prácticas contribuye a producir código más claro, robusto y mantenible. En proyectos de software reales, donde múltiples desarrolladores trabajan de manera colaborativa sobre un mismo sistema, la claridad en la definición de variables se convierte en un factor esencial para comprender rápidamente la lógica del programa y evitar errores durante las etapas de mantenimiento o evolución del sistema [23], [24].

Además de estas recomendaciones generales, el lenguaje Java promueve ciertas convenciones de estilo que favorecen la uniformidad del código dentro de los proyectos de software. En particular, los nombres de variables se escriben habitualmente utilizando el estilo *lowerCamelCase*, en el cual la primera palabra comienza en minúscula y las siguientes palabras comienzan con mayúscula (por ejemplo, `totalVentas`, `numeroEstudiantes`). Por otro lado, las constantes suelen escribirse en *UPPER_SNAKE_CASE*, utilizando letras mayúsculas separadas por guiones bajos, como en `TASA_INTERES` o `MAX_USUARIOS`. Estas convenciones, aunque no afectan la ejecución del programa, contribuyen a mejorar la legibilidad del código y facilitan la comunicación dentro de equipos de desarrollo [35], [73].

El uso de nombres adecuados para las variables constituye uno de los aspectos más importantes para mejorar la claridad del código. En la práctica, un nombre apropiado permite comprender el significado de la variable incluso sin conocer todos los detalles del algoritmo en el que se utiliza. La Tabla 7.6 presenta algunos ejemplos comparativos que ilustran cómo pequeños cambios en la elección del nombre pueden mejorar significativamente la comprensión del programa.

Cuadro 7.6: Ejemplos de nombres adecuados e inadecuados de variables

Nombre poco descriptivo	Nombre mejorado	Motivo de la mejora
x	edadEstudiante	Indica claramente la información almacenada.
n	numeroEstudiantes	Describe la cantidad representada por la variable.
d	distanciaRecorrida	Permite comprender el significado del dato.
v1	velocidadPromedio	Evita identificadores ambiguos.
res	resultadoPromedio	Explica el propósito del valor calculado.

En muchos casos, mejorar la claridad de un programa implica simplemente reemplazar identificadores ambiguos por nombres más descriptivos. Este proceso se conoce comúnmente como refactorización del código y permite mejorar la comprensión del programa sin modificar su comportamiento. El Listado de código 7.18 muestra un ejemplo sencillo de este proceso.

```

1          /* Version inicial */
2          int x = 3;
3          int y = 5;
4          int z = x * y;
5
6          /* Version mejorada */

```

```

7          int cantidadProductos = 3;
8          int precioUnitario = 5;
9          int costoTotal = cantidadProductos *
           ↪ precioUnitario;
10

```

Listing 7.18: Refactorización de nombres de variables para mejorar la claridad del programa.

Finalmente, al declarar variables dentro de un programa, resulta útil realizar una breve revisión para verificar que se han seguido las buenas prácticas recomendadas. La Tabla 7.7 presenta una lista de verificación sencilla que puede utilizarse como guía durante el proceso de desarrollo de programas.

Cuadro 7.7: Lista de verificación para el uso adecuado de variables

Criterio de revisión	Sí	No	Par- cial	Observaciones
¿El nombre de la variable describe claramente la información que almacena?	☐	☐	☐	
¿La variable fue inicializada antes de utilizarse en una operación?	☐	☐	☐	
¿La variable representa un único concepto dentro del algoritmo?	☐	☐	☐	
¿El ámbito de la variable está limitado al bloque donde se utiliza?	☐	☐	☐	
¿Se evita repetir valores literales que deberían declararse como constantes?	☐	☐	☐	

Ámbito y ciclo de vida

El ámbito de una variable determina las regiones del programa en las que dicha variable puede ser utilizada. En otras palabras, el ámbito define la visibilidad de una variable dentro del código, estableciendo en qué partes del programa el compilador reconoce su existencia y permite acceder a su valor. Comprender este concepto resulta fundamental para estructurar correctamente los programas y evitar errores asociados al uso de variables fuera del contexto en el que fueron declaradas [35], [73].

La organización de un programa en distintos niveles estructurales determina el ámbito en el que las variables pueden utilizarse. En Java, las variables suelen declararse dentro de bloques de código que pertenecen a métodos, los cuales a su vez forman parte de clases dentro de un programa. La Figura 7.2 ilustra esta estructura jerárquica y muestra cómo cada nivel delimita la visibilidad y el ciclo de vida de las variables declaradas dentro del sistema [35], [73].

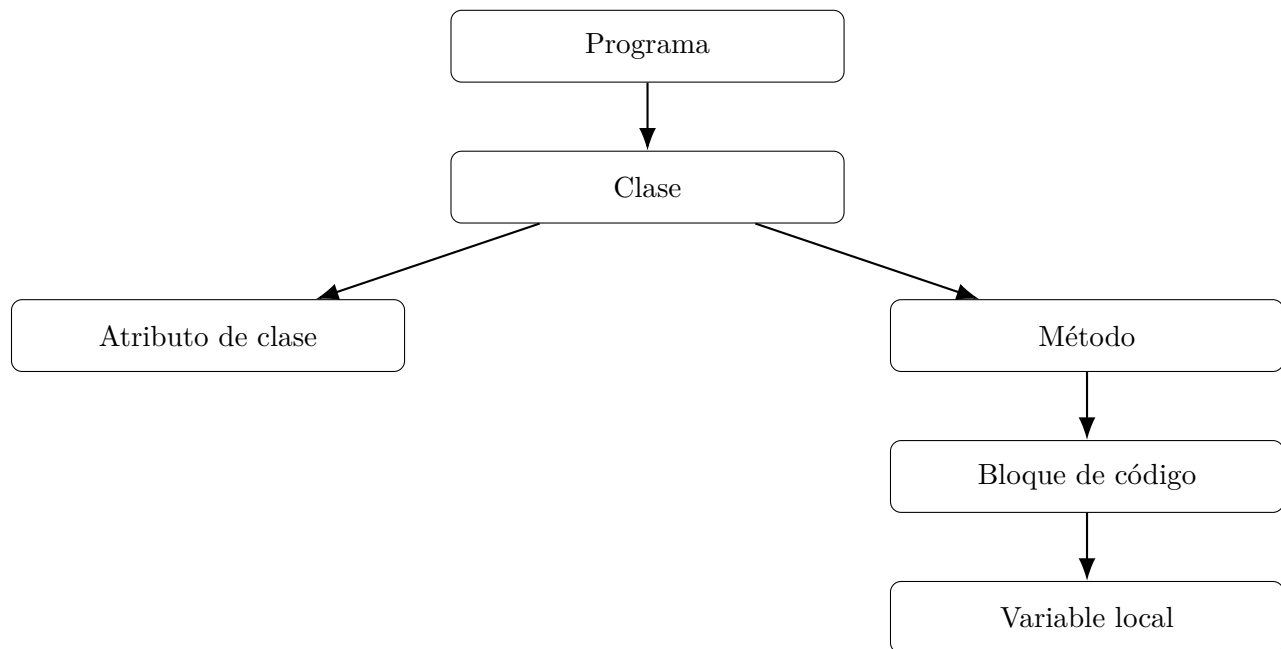


Figura 7.2: Estructura jerárquica del ámbito en un programa Java. Los atributos y los métodos pertenecen al mismo nivel dentro de una clase; las variables locales se declaran dentro de bloques asociados a métodos, por lo que su visibilidad y ciclo de vida quedan restringidos al bloque que las contiene.

En el nivel más interno de esta estructura —representado en la Figura 7.2 como un bloque de código— las variables declaradas únicamente existen dentro de ese bloque.

```
1  if (true) {  
2      int numero = 5;  
3      System.out.println(numero);  
4  }  
5
```

Listing 7.19: Variable con ámbito limitado a un bloque de código.

En el Listado de código 7.19, la variable `numero` se declara dentro del bloque delimitado por las llaves del `if`. Como consecuencia, dicha variable solo puede utilizarse dentro de ese bloque. Mientras la ejecución del programa se encuentra dentro del bloque, la variable existe y puede emplearse normalmente; sin embargo, una vez que el flujo de ejecución abandona el bloque, la variable deja de existir y cualquier intento de acceder a ella produciría un error de compilación.

Comprender el ámbito de las variables contribuye a escribir programas más claros y seguros. Declarar las variables lo más cerca posible del lugar donde serán utilizadas reduce la probabilidad de errores, facilita la comprensión del código y mejora su mantenibilidad. Además, este principio permite controlar con mayor precisión el ciclo de vida de los datos utilizados dentro de un algoritmo [23], [24].

Cuando en un programa existen variables con el mismo nombre declaradas en distintos ámbitos, el lenguaje Java aplica una regla de resolución conocida como *alcance léxico* (*lexical scope*). Según esta regla, el compilador busca primero la variable dentro del ámbito más cercano al punto donde se utiliza. Si encuentra una variable con ese nombre en el bloque actual,

utilizará dicha variable; de lo contrario, continuará buscando en los ámbitos externos hasta encontrar una definición válida. Este mecanismo permite que diferentes partes del programa utilicen nombres similares sin interferir entre sí, aunque también puede dar lugar a situaciones en las que una variable interna oculte temporalmente a otra declarada en un ámbito más externo [35], [73].

El Listado de código 7.20 muestra un ejemplo sencillo de este comportamiento.

```
1         int numero = 10;
2
3         if (true) {
4             int numero = 5;
5             System.out.println(numero); // imprime 5
6         }
7
8         System.out.println(numero);    // imprime 10
9
```

Listing 7.20: Ejemplo de shadowing en Java.

En el Listado de código 7.20, la variable `numero` declarada dentro del bloque del `if` oculta temporalmente a la variable externa con el mismo nombre. Mientras la ejecución se encuentra dentro del bloque, cualquier referencia a `numero` corresponde a la variable interna. Una vez que el programa abandona el bloque, la variable externa vuelve a ser visible.

Un fenómeno relacionado con el ámbito es el denominado *shadowing* o sombreado de variables. Este ocurre cuando una variable declarada en un ámbito interno utiliza el mismo nombre que otra variable declarada en un ámbito externo, ocultando temporalmente su valor dentro del bloque interno. Aunque el lenguaje permite este comportamiento, en etapas iniciales del aprendizaje se recomienda evitar nombres repetidos en ámbitos anidados, ya que pueden generar confusión al interpretar el comportamiento del programa y dificultar el proceso de depuración [74], [75], [76].

Constantes

En el desarrollo de programas es frecuente que ciertos valores deban permanecer invariables durante toda la ejecución del sistema. Estos valores reciben el nombre de *constantes* y representan información fija que el programa utiliza como referencia para realizar cálculos, validar condiciones o aplicar determinadas reglas de negocio. A diferencia de las variables, cuyo contenido puede modificarse a lo largo de la ejecución, las constantes mantienen siempre el mismo valor una vez que han sido definidas, lo que permite garantizar estabilidad y coherencia en el comportamiento del software [35], [73].

En el lenguaje Java, las constantes se declaran mediante el modificador `final`, el cual indica al compilador que el valor asignado a la variable no podrá modificarse posteriormente durante la ejecución del programa. Este mecanismo permite representar valores que deben permanecer invariables, como constantes matemáticas, parámetros de configuración del sistema o límites utilizados en procesos de validación. La utilización de constantes contribuye a mejorar la claridad del código y a prevenir modificaciones accidentales de datos críticos dentro del programa. El Listado de código ?? presenta un ejemplo sencillo de declaración de una constante en Java, mientras que los ejemplos posteriores muestran formas adicionales de definir constantes en distintos contextos de uso dentro de aplicaciones reales [23], [35], [73].

```
1         // Constante local dentro de un método
2         final int MAX_INTENTOS = 3;
3
```

```
4      // Constante de tipo decimal utilizada en cálculos financieros
5      final double TASA_INTERES = 0.05;
6
7      // Constante booleana utilizada como bandera lógica
8      final boolean SISTEMA_ACTIVO = true;
9
10     // Constante de tipo String para mensajes del sistema
11     final String MENSAJE_ERROR = "Acceso no autorizado";
12
13     // Constante de clase (recomendado en programas grandes)
14     public static final double PI = 3.141592653589793;
15
16     // Constante para límites del sistema
17     public static final int MAX_USUARIOS = 1000;
18
19     // Constante derivada de otra constante
20     final int MINUTOS_POR_HORA = 60;
21     final int SEGUNDOS_POR_MINUTO = 60;
22     final int SEGUNDOS_POR_HORA = MINUTOS_POR_HORA * SEGUNDOS_POR_MINUTO;
```

Listing 7.21: Ejemplos de declaración de constantes en Java.

En el Listado de código ??, la constante PI representa una aproximación del valor del número π . Al declararse mediante la palabra clave `final`, el lenguaje garantiza que dicho valor no podrá modificarse posteriormente en el programa. Este tipo de definiciones resulta especialmente útil cuando se trabaja con constantes matemáticas, límites de validación, tasas, porcentajes o cualquier otro dato que deba mantenerse estable durante la ejecución del sistema.

El uso de constantes contribuye a evitar modificaciones accidentales en valores críticos del programa, lo cual reduce la probabilidad de introducir errores durante el desarrollo o mantenimiento del software. Desde una perspectiva de ingeniería de software, la definición explícita de constantes permite controlar de manera más segura la manipulación de datos que representan parámetros esenciales del sistema [23], [24].

En proyectos de software de mayor escala, el empleo sistemático de constantes mejora significativamente la consistencia del código y facilita su evolución. Cuando una regla de negocio cambia —por ejemplo, el porcentaje de un impuesto, un límite de validación o una tasa aplicada en un cálculo— basta con modificar el valor declarado en la constante correspondiente, sin necesidad de revisar múltiples secciones del programa. Además, el uso de constantes favorece la documentación del propósito de los valores utilizados en el código, evitando la presencia de los llamados *números mágicos* (*magic numbers*), es decir, valores literales cuyo significado no resulta evidente para otros programadores [51].

En el desarrollo de software, las constantes permiten representar valores que deben permanecer invariables durante la ejecución del programa. Estos valores suelen corresponder a parámetros del sistema, límites de validación, constantes matemáticas o reglas de negocio utilizadas en múltiples partes del código. La Tabla 7.8 presenta ejemplos representativos de constantes utilizadas en aplicaciones desarrolladas en Java, mostrando su tipo de dato, un ejemplo de valor y el contexto en el que suelen emplearse.

Cuadro 7.8: Ejemplos de constantes utilizadas en aplicaciones Java

Tipo de dato	Constante	Ejemplo de valor	Uso típico	Observaciones
double	PI	3.141592653589793	Cálculos geométricos y trigonométricos	Frecuentemente se usa <code>Math.PI</code> en la biblioteca estándar de Java.
double	IVA	0.12	Cálculo de impuestos o porcentajes en aplicaciones financieras	Centralizar este valor facilita actualizarlo si la normativa cambia.
int	MAX_INTENTOS	3	Control de intentos de autenticación o validación de usuario	Evita repetir valores literales en diferentes partes del programa.
int	EDAD_MINIMA	18	Validación de acceso a servicios o sistemas	Representa reglas de negocio relacionadas con restricciones legales.
double	PORC_DESCUENTO	0.10	Aplicación de descuentos en sistemas comerciales	Permite modificar fácilmente políticas comerciales del sistema.
String	MENSAJE_ERROR	"Acceso denegado"	Mensajes del sistema utilizados en múltiples módulos	Evita duplicar textos literales y facilita la internacionalización.
int	DIAS_SEMANA	7	Cálculos de calendarios o programación de eventos	Ejemplo de constante basada en propiedades del dominio del problema.
long	MAX_TAM_ARCHIVO	1048576	Límites de tamaño en sistemas de almacenamiento o carga de archivos	Representa restricciones técnicas del sistema.

7.3. Operadores y construcción de expresiones

Los operadores permiten combinar valores y variables para producir nuevos resultados. Las expresiones resultantes pueden utilizarse para realizar cálculos o evaluar condiciones.

En términos formales, un operador define una función sobre uno o más operandos, y una expresión es un término que, al evaluarse, produce un valor. Comprender esta distinción es clave para interpretar correctamente el flujo de ejecución: la evaluación de expresiones produce resultados, mientras que las instrucciones (*statements*) producen efectos en el estado (por ejemplo, asignar un valor a una variable). La confusión entre ambos niveles aparece de manera recurrente en errores de principiantes, especialmente en expresiones con asignaciones o en condiciones complejas [74].

7.3.1. Operadores aritméticos, relacionales y lógicos

Los operadores constituyen uno de los mecanismos fundamentales mediante los cuales un programa puede procesar información y producir resultados. En términos generales, un operador define una acción que se aplica sobre uno o más operandos, generando como resultado un nuevo valor. En los lenguajes de programación imperativos modernos, como Java, los operadores permiten construir expresiones que representan cálculos numéricos, comparaciones entre valores y evaluaciones lógicas utilizadas para controlar el flujo de ejecución del programa [35], [73].

Dentro de las aplicaciones informacionales, estos operadores permiten transformar datos almacenados en variables en nuevos resultados que posteriormente pueden ser utilizados por otras partes del programa. Por esta razón, comprender el comportamiento de los operadores y su interacción dentro de una expresión constituye un aspecto esencial para escribir programas correctos y evitar errores de razonamiento en los cálculos o en las decisiones lógicas que realiza el software [23], [24].

Los operadores aritméticos permiten realizar operaciones matemáticas sobre valores numéricos. Entre los más utilizados se encuentran la suma, la resta, la multiplicación, la división y el operador módulo. Estos operadores se emplean frecuentemente en cálculos de totales, promedios, porcentajes y transformaciones de datos numéricos dentro de una aplicación.

Un ejemplo básico de uso de operadores aritméticos se presenta en el Listado de código 7.22.

```
1      int suma = 5 + 3;  
2      int producto = 4 * 2;
```

Listing 7.22: Uso de operadores aritméticos en Java.

En el Listado de código 7.22, la variable `suma` almacena el resultado de la operación `5 + 3`, mientras que la variable `producto` contiene el resultado de multiplicar 4 por 2. Aunque estos ejemplos son simples, ilustran el principio básico de construcción de expresiones aritméticas, que posteriormente pueden formar parte de cálculos más complejos dentro de un programa.

Por otra parte, los operadores relacionales permiten comparar dos valores y determinar la relación que existe entre ellos. El resultado de estas comparaciones siempre es un valor booleano, es decir, `true` o `false`. Este tipo de operadores resulta fundamental para evaluar condiciones dentro de estructuras de control como `if`, `while` o `for`, donde el programa debe decidir qué acciones ejecutar en función de determinados criterios [35].

Un ejemplo de comparación relacional se presenta en el Listado de código 7.23.

```
1      boolean resultado = 10 > 5;
```

Listing 7.23: Uso de operadores relacionales en Java.

En el Listado de código 7.23, la expresión `10 > 5` evalúa si el número diez es mayor que cinco. Dado que esta condición es verdadera, el valor almacenado en la variable `resultado` será `true`. Este tipo de comparaciones es ampliamente utilizado en aplicaciones informacionales, por ejemplo para verificar límites numéricos, validar rangos de valores o comprobar condiciones asociadas al estado de un sistema.

Los operadores lógicos permiten combinar múltiples condiciones booleanas para construir expresiones más complejas. Estos operadores resultan especialmente importantes cuando una decisión depende de varias condiciones simultáneamente. Entre los operadores lógicos más utilizados se encuentran la conjunción lógica (`&&`), la disyunción lógica (`||`) y la negación lógica (`!`), los cuales permiten expresar reglas de decisión dentro del programa [35], [73].

Un ejemplo de combinación de condiciones se presenta en el Listado de código 7.24.

```
1      boolean acceso = edad >= 18 && usuarioAutenticado;
```

Listing 7.24: Uso de operadores lógicos en Java.

En el Listado de código 7.24, la variable `acceso` será verdadera únicamente cuando ambas condiciones se cumplan: que el valor de `edad` sea mayor o igual que 18 y que la variable `usuarioAutenticado` sea verdadera. Este tipo de expresiones permite representar reglas de negocio o políticas de acceso dentro de los sistemas de información.

En Java, los operadores lógicos `&&` y `||` implementan un mecanismo denominado *evaluación de cortocircuito* (*short-circuit evaluation*). Esto significa que el lenguaje evalúa la segunda condición únicamente cuando el resultado de la expresión no puede determinarse a partir del primer operando. Por ejemplo, en una expresión con el operador `&&`, si la primera condición es falsa, el resultado completo de la expresión necesariamente será falso, por lo que el segundo operando no necesita evaluarse.

Este comportamiento resulta particularmente útil para prevenir errores durante la ejecución del programa. Un caso frecuente consiste en verificar primero si una referencia a objeto es distinta de `null` antes de evaluar una condición adicional sobre dicho objeto. De esta forma, el cortocircuito evita que el programa intente acceder a un objeto inexistente, lo que produciría una excepción en tiempo de ejecución. Además, este mecanismo contribuye a mejorar la eficiencia del programa al evitar evaluaciones innecesarias en condiciones complejas [23], [35].

Precedencia

La precedencia de operadores determina el orden en que se evalúan las operaciones dentro de una expresión cuando esta contiene varios operadores distintos. En ausencia de paréntesis que indiquen explícitamente el orden de evaluación, el lenguaje de programación aplica un conjunto de reglas predefinidas que establecen qué operadores deben ejecutarse primero. Estas reglas permiten que el compilador interprete de manera consistente las expresiones escritas por el programador y constituyen un elemento fundamental del sistema sintáctico de los lenguajes de programación modernos [35], [73].

La Tabla 7.9 presenta los operadores más utilizados en el lenguaje Java junto con el tipo de operación que realizan, su nivel de prioridad y algunos ejemplos representativos. En esta tabla, un número menor en la columna de prioridad indica que el operador se evalúa antes que aquellos con valores mayores. Conocer esta jerarquía permite comprender cómo el lenguaje interpreta expresiones complejas y ayuda a evitar resultados inesperados durante la ejecución de los programas.

Cuadro 7.9: Prioridad y características de los operadores en Java

Operador	Operación	Prioridad	Ejemplo	Observaciones
()	Agrupación / control de evaluación	1	(a + b) * c	Permite modificar explícitamente el orden de evaluación.
++, -	Incremento / decremento	2	i++, --j	Pueden ser prefijo o sufijo.
!, +, -, ++, -	Operadores unarios	3	!activo, -numero	Operan sobre un solo operando.
*, /, %	Multiplicación, división, módulo	4	a * b, n % 2	Usados en operaciones aritméticas.
+, -	Suma y resta	5	a + b, x - y	También se utiliza para concatenación de cadenas.
>, >=, <, <=	Comparación relacional	6	edad >= 18	Devuelven valores booleanos.
==, !=	Igualdad y desigualdad	7	a == b	Comparan valores de variables.
&&	Conjunción lógica (AND)	8	cond1 && cond2	Requiere que ambas condiciones sean verdaderas.
	Disyunción lógica (OR)	9	cond1 cond2	Basta con que una condición sea verdadera.
=, +=, -=, *=, /=	Asignación	10	x = 5, x += 2	Se evalúan al final de la expresión.

Un ejemplo sencillo de precedencia de operadores se presenta en el Listado de código 7.25.

```
1  int resultado = 5 + 3 * 2;
```

Listing 7.25: Evaluación de una expresión considerando la precedencia de operadores.

En el Listado de código 7.25, la expresión aritmética contiene dos operadores distintos: suma (+) y multiplicación (*). De acuerdo con las reglas de precedencia del lenguaje Java, el operador de multiplicación posee una prioridad mayor que el operador de suma. Por esta razón, el programa evalúa primero la operación `3 * 2`, cuyo resultado es 6. Posteriormente, el valor obtenido se utiliza en la operación de suma, evaluándose finalmente la expresión `5 + 6`, cuyo resultado es 11. Este comportamiento corresponde al mismo principio utilizado en la aritmética matemática tradicional, donde las multiplicaciones y divisiones se realizan antes que las sumas y restas [35].

Comprender la precedencia de operadores es esencial para interpretar correctamente el resultado de una expresión y evitar errores de razonamiento durante el desarrollo de programas. Cuando un programador desconoce estas reglas, puede asumir un orden de evaluación distinto al que realmente ejecuta el compilador, produciendo resultados inesperados en el programa. Por esta razón, la precedencia constituye uno de los aspectos fundamentales en la construcción de expresiones correctas dentro del software [23], [24].

Asociatividad

La asociatividad define el orden en que se evalúan los operadores cuando varios de ellos poseen el mismo nivel de precedencia dentro de una expresión. Mientras que la precedencia establece qué operadores deben evaluarse primero, la asociatividad determina cómo se agrupan las operaciones cuando no existen paréntesis que indiquen explícitamente el orden de evaluación. Este mecanismo forma parte de las reglas formales que los lenguajes de programación utilizan para interpretar expresiones aritméticas y lógicas de manera consistente [35], [73].

En muchos lenguajes de programación, incluidos Java y otros lenguajes imperativos modernos, la mayoría de los operadores aritméticos son asociativos por la izquierda. Esto significa que, cuando varios operadores del mismo tipo aparecen consecutivamente en una expresión, la evaluación se realiza de izquierda a derecha. Comprender esta regla resulta fundamental para interpretar correctamente el resultado de las operaciones y evitar errores de razonamiento durante el desarrollo de programas [35].

Un ejemplo ilustrativo se presenta en el Listado de código 7.26, donde se comparan dos expresiones aparentemente similares que producen resultados distintos debido a la forma en que se agrupan las operaciones.

```
1    int a = 10 - 5 - 2;    // equivale a (10 - 5) - 2
2    int b = 10 - (5 - 2);  // produce un resultado distinto
```

Listing 7.26: Evaluación de expresiones considerando la asociatividad de operadores.

En el Listado de código 7.26, la expresión asignada a la variable **a** se evalúa aplicando la asociatividad por la izquierda del operador de resta. Por lo tanto, la operación $10 - 5$ se realiza primero, produciendo el valor 5, y posteriormente se evalúa $5 - 2$, cuyo resultado final es 3. En cambio, en la expresión asignada a la variable **b**, los paréntesis modifican el orden natural de evaluación al obligar a calcular primero la subexpresión $5 - 2$. El resultado de esa operación es 3, y posteriormente se evalúa $10 - 3$, produciendo el valor final 7.

La diferencia entre los valores almacenados en **a** y **b**, mostrada en el Listado de código 7.26, evidencia que el orden de agrupación de los operadores puede modificar el resultado de una expresión. Por esta razón, comprender las reglas de asociatividad es esencial para interpretar correctamente el comportamiento de los programas.

Aunque los lenguajes de programación establecen reglas claras de precedencia y asociatividad, una práctica ampliamente recomendada consiste en utilizar paréntesis para expresar explícitamente el orden de evaluación cuando una expresión contiene múltiples operadores. Esta estrategia mejora la claridad del código, facilita su comprensión y reduce errores durante el desarrollo y mantenimiento del software [23], [24]. Además, diversos estudios sobre aprendizaje de programación han demostrado que el uso explícito de paréntesis ayuda a los programadores principiantes a comprender mejor el proceso de evaluación de expresiones y reduce errores conceptuales en etapas iniciales del aprendizaje [74], [75], [76].

La evaluación correcta de expresiones resulta esencial para garantizar que los cálculos realizados por un programa correspondan con la intención lógica del programador. En muchos casos, una misma expresión puede interpretarse de diferentes maneras dependiendo de la precedencia de los operadores, de la asociatividad o del uso explícito de paréntesis. La Tabla 7.10 presenta diversos ejemplos representativos que ilustran cómo estas reglas influyen en la evaluación de expresiones en Java. Los ejemplos incluyen situaciones habituales en el desarrollo de aplicaciones, como cálculos aritméticos, operaciones estadísticas simples, evaluaciones lógicas y concatenación de texto, mostrando cuándo los paréntesis son obligatorios para modificar el resultado y cuándo su uso es opcional pero recomendable para mejorar la claridad del código [23], [35], [73].

Cuadro 7.10: Ejemplos prácticos de asociatividad y uso de paréntesis en expresiones Java

Problema o contexto	Expresión sin paréntesis	Expresión con paréntesis	Interpretación
Cálculo aritmético simple	<code>10 - 5 - 2</code>	<code>10 - (5 - 2)</code>	La resta es asociativa por la izquierda: <code>(10-5)-2=3</code> . Con paréntesis cambia el resultado a 7.
División de valores	<code>20 / 5 / 2</code>	<code>20 / (5 / 2)</code>	Por asociatividad izquierda se evalúa <code>(20/5)/2=2</code> . Con paréntesis el resultado cambia a 8.
Cálculo de promedio académico	<code>nota1 + nota2 + nota3 / 3</code>	<code>(nota1 + nota2 + nota3) / 3</code>	Sin paréntesis solo la tercera nota se divide entre 3. Con paréntesis se calcula correctamente el promedio.
Cálculo estadístico de media ponderada	<code>examen * 0.6 + tareas * 0.4</code>	<code>(examen * 0.6) + (tareas * 0.4)</code>	Los paréntesis no son obligatorios, pero hacen explícita la estructura del cálculo.
Condición lógica en admisión	<code>edad >= 18 && promedio >= 7</code>	<code>(edad >= 18) && (promedio >= 7)</code>	Los paréntesis mejoran la legibilidad al separar claramente las subcondiciones.
Evaluación lógica combinada	<code>cond1 cond2 && cond3</code>	<code>(cond1 cond2) && cond3</code>	El operador <code>&&</code> tiene mayor precedencia que <code> </code> . Los paréntesis modifican la lógica de evaluación.
Concatenación de texto y suma	<code>"Resultado: " + 5 + 3</code>	<code>"Resultado: " + (5 + 3)</code>	Sin paréntesis produce <code>"Resultado: 53"</code> . Con paréntesis produce <code>"Resultado: 8"</code> .
Conversión y concatenación	<code>"Promedio: " + suma / 3</code>	<code>"Promedio: " + (suma / 3)</code>	Los paréntesis clarifican que primero se calcula el promedio y luego se convierte a texto.
Asignación con cálculo	<code>total = precio + impuesto * cantidad</code>	<code>total = (precio + impuesto) * cantidad</code>	Los paréntesis cambian el modelo de cálculo del costo total.
Negación lógica	<code>!aprobado && asistencia >= 80</code>	<code>!(aprobado && asistencia >= 80)</code>	Los paréntesis cambian completamente la interpretación lógica de la expresión.

Expresiones compuestas

Las expresiones compuestas se construyen combinando múltiples operadores y operandos con el propósito de evaluar condiciones más elaboradas o realizar cálculos que dependen de varios criterios simultáneamente. A diferencia de las expresiones simples, que evalúan una única operación o comparación, las expresiones compuestas integran varias subexpresiones dentro de una misma estructura lógica o aritmética. Esta capacidad permite representar decisiones más complejas dentro de un programa, ya que el resultado final depende de la evaluación conjunta de varias condiciones [35], [73].

Un ejemplo de expresión compuesta se presenta en el Listado de código 7.27, donde se evalúan simultáneamente dos condiciones para determinar el valor de una variable booleana.

```
1      boolean aprobado = promedio >= 7 && asistencia >= 80;
```

Listing 7.27: Evaluación de una expresión compuesta mediante operadores lógicos.

En el Listado de código 7.27, la variable `aprobado` toma el valor `true` únicamente cuando ambas condiciones son verdaderas. La primera subexpresión evalúa si el valor almacenado en la variable `promedio` es mayor o igual que siete, mientras que la segunda verifica si el valor de `asistencia` es mayor o igual que ochenta. Ambas subexpresiones se combinan mediante el operador lógico `&&`, el cual representa una conjunción lógica que solo produce un resultado verdadero cuando todas las condiciones involucradas son verdaderas. Este tipo de evaluación corresponde a la lógica booleana clásica utilizada en los lenguajes de programación modernos [35].

Las expresiones compuestas permiten representar reglas de decisión dentro de los programas. En muchos casos, la ejecución de una acción depende de verificar simultáneamente múltiples condiciones, como validar rangos de valores, comprobar requisitos o evaluar el estado de varias variables. Por esta razón, comprender cómo se construyen y cómo se evalúan estas expresiones es esencial para escribir programas correctos y evitar errores en la lógica del sistema [23], [24].

Cuando una expresión incluye varias condiciones, resulta recomendable analizar o construir la expresión de manera incremental. Esto significa evaluar inicialmente cada subcondición de forma independiente y posteriormente combinar los resultados mediante operadores lógicos. Por ejemplo, puede verificarse primero la condición `promedio >= 7` y luego la condición `asistencia >= 80`. Este enfoque facilita la comprensión del comportamiento de la expresión y permite identificar con mayor facilidad posibles errores durante el desarrollo del programa.

Otra práctica recomendable consiste en asignar cada subcondición a variables booleanas con nombres descriptivos, de manera que cada variable represente claramente una regla lógica específica. Esta estrategia se ilustra en el Listado de código 7.28.

```
1      boolean cumplePromedio = promedio >= 7;  
2      boolean cumpleAsistencia = asistencia >= 80;  
3      boolean aprobado = cumplePromedio && cumpleAsistencia;
```

Listing 7.28: Descomposición de una expresión compuesta en subcondiciones booleanas.

En el Listado de código 7.28, cada subcondición se evalúa y se almacena en una variable booleana independiente. Posteriormente, estas variables se combinan para determinar el valor final de la variable `aprobado`. Esta forma de estructurar las expresiones mejora significativamente la legibilidad del programa, ya que cada variable representa una condición lógica claramente identificable. Además, facilita el proceso de depuración, porque permite verificar cada condición por separado antes de evaluar la expresión completa.

Diversos estudios sobre el aprendizaje de la programación han señalado que la descomposición de expresiones complejas y el uso de nombres descriptivos para variables booleanas reducen errores lógicos y mejoran la comprensión del código, especialmente en programadores principiantes [74], [75], [76].

7.3.2. Conversión y validación de datos

En muchas ocasiones los programas necesitan convertir datos de un tipo a otro.

En aplicaciones informacionales, una fuente típica de conversión es la entrada del usuario o la lectura de datos desde archivos y redes, que a menudo se reciben como texto (**String**). La conversión, por tanto, no solo es un mecanismo técnico sino un paso de validación: el programa debe confirmar que los datos cumplen el formato esperado antes de operar con ellos. La omisión de validaciones es una causa frecuente de fallos en sistemas reales y de errores de ejecución en programas de aprendizaje.

Conversión implícita

La conversión implícita, también denominada *conversión automática* o *type promotion*, ocurre cuando el lenguaje de programación transforma un valor de un tipo de dato a otro sin requerir una instrucción explícita por parte del programador. Este tipo de conversión es posible únicamente cuando el nuevo tipo de dato puede representar completamente el valor original sin riesgo de pérdida de información. En lenguajes fuertemente tipados como Java, estas conversiones están cuidadosamente definidas por el sistema de tipos con el objetivo de preservar la consistencia de los datos y evitar errores durante la ejecución del programa [35], [73].

En términos generales, la conversión implícita se produce cuando un tipo de dato de menor capacidad o precisión se transforma en otro de mayor capacidad. Este proceso suele denominarse *ensanchamiento de tipo* (*widening conversion*), ya que el valor pasa a representarse en un dominio más amplio. Por ejemplo, un valor entero puede convertirse automáticamente en un número de punto flotante, ya que el formato de punto flotante dispone de un rango de representación mayor que permite almacenar el valor original sin modificaciones [107].

El ejemplo del Listado de código 7.29 muestra un caso típico de conversión implícita en Java:

```
1    int numero = 5;
2    double valor = numero;
```

Listing 7.29: Conversión Implícita de Datos.

En este fragmento de código, el valor entero almacenado en la variable **numero** se convierte automáticamente en un valor de tipo **double** cuando se asigna a la variable **valor**. El lenguaje realiza esta conversión de manera transparente para el programador, interpretando el entero como un número real equivalente.

Desde la perspectiva del desarrollo de Sistemas de Información, este mecanismo es especialmente útil cuando los programas combinan distintos tipos de datos en operaciones aritméticas o cálculos estadísticos, como promedios, tasas o indicadores derivados de conteos enteros. Sin embargo, es importante comprender que la conversión automática solo se produce cuando el lenguaje puede garantizar que el valor original se representará correctamente en el nuevo tipo. Cuando existe la posibilidad de pérdida de precisión o reducción del rango representable, el lenguaje requiere una conversión explícita indicada por el programador [23], [35].

Por esta razón, aunque la conversión implícita simplifica la escritura del código y mejora la legibilidad de los programas, el desarrollador debe comprender las reglas que gobiernan estas transformaciones para evitar interpretaciones incorrectas de los resultados y asegurar la consistencia de los datos dentro del sistema.

Conversión explícita

La conversión explícita requiere una instrucción directa del programador, ya que el lenguaje no puede realizar la transformación de tipos de manera automática sin riesgo de pérdida de información. Este proceso se denomina comúnmente *casting* y consiste en indicar de forma explícita el tipo de dato al que se desea convertir un valor o el resultado de una expresión.

La sintaxis general de esta operación se muestra en el Listado de código 7.30. En ella se coloca, entre paréntesis, el tipo de dato destino inmediatamente antes del valor o de la expresión que se desea convertir. De esta manera, el compilador interpreta que el resultado debe tratarse como perteneciente al nuevo tipo especificado, aun cuando el dato original pertenezca a un tipo diferente.

```
1    double valor = 9.7;
2    int numero = (int) valor;
```

Listing 7.30: Conversión Explícita de Datos.

En este caso, la parte fraccionaria se descarta. Comprender este comportamiento evita interpretaciones erróneas: el resultado no es un redondeo, sino un truncamiento hacia cero.

Este mecanismo es particularmente útil cuando se necesita transformar valores que provienen de cálculos o de otras variables, por ejemplo al convertir un número de punto flotante a un número entero o al adaptar datos para operaciones que requieren un tipo específico. Sin embargo, es importante tener en cuenta que algunas conversiones pueden implicar pérdida de precisión o truncamiento de información, por lo que el programador debe utilizarlas de manera consciente y comprender el efecto que tendrán sobre el resultado final del programa.

Errores por conversión

Si el programa intenta convertir un valor incompatible, puede producirse un error durante la ejecución del programa. Este tipo de situación ocurre cuando el contenido del dato no puede interpretarse correctamente dentro del tipo de destino solicitado. Un ejemplo típico aparece cuando se intenta convertir una cadena de texto que no contiene una representación numérica válida a un tipo entero. En estos casos, el entorno de ejecución genera una excepción que indica que la conversión no puede realizarse correctamente, como se ilustra en el Listado de código 7.31.

```
1    String texto = "abc";
2    int numero = Integer.parseInt(texto);
```

Listing 7.31: Conversión Errónea de Datos.

Este tipo de errores es frecuente en aplicaciones que procesan datos introducidos por los usuarios o provenientes de fuentes externas, como archivos o servicios web. Por esta razón, las buenas prácticas de programación recomiendan validar previamente el contenido de los datos y utilizar mecanismos de manejo de excepciones que permitan detectar y gestionar estas situaciones de forma controlada, evitando fallos inesperados en el sistema [35], [107]. Desde la perspectiva de la Ingeniería de Sistemas de Información, el tratamiento adecuado de estas conversiones resulta fundamental para garantizar la integridad de los datos y la confiabilidad de los procesos automatizados dentro de las aplicaciones [23], [24].

Este tipo de situaciones debe manejarse adecuadamente para evitar fallos en la aplicación. En Java, estas conversiones pueden lanzar excepciones (por ejemplo, `NumberFormatException`). Por ello, además de convertir, se recomienda validar y gestionar errores de entrada, especialmente en aplicaciones orientadas a usuarios.

A nivel introductorio, una práctica formativa consiste en envolver conversiones en bloques `try-catch` para preservar la estabilidad del programa y proporcionar mensajes claros, como se muestra en el Listado de código 7.32

```
1    try {
2        int numeroSeguro = Integer.parseInt(texto);
3    } catch (NumberFormatException e) {
4        System.out.println("Entrada invalida: se esperaba un numero
↪ entero.");
5    }
```

Listing 7.32: Excepciones Producidas en Conversión de Datos.

Esta estrategia conecta el modelado de datos con la robustez del software: un programa confiable no solo calcula, sino que también resiste entradas inesperadas y comunica errores de forma comprensible.

i

Cerrando el capítulo...

El modelado de datos y la construcción de expresiones constituyen el núcleo operacional de la programación. Los tipos de datos delimitan qué valores existen y cómo se interpretan; las variables permiten almacenar y evolucionar el estado; los operadores combinan valores bajo reglas de precedencia y asociatividad; y las conversiones conectan el mundo del texto (entrada/salida) con el mundo numérico y lógico del cómputo. Este conjunto de conceptos, además de ser esencial para escribir programas correctos, ha sido identificado por la literatura empírica como un determinante clave del éxito en el aprendizaje inicial de la programación [74], [75], [76].

Capítulo 8

Ejercicios de Aplicación

La resolución de ejercicios de aplicación representa el momento en que el razonamiento algorítmico abandona el plano puramente abstracto y se convierte en una solución computacional ejecutable. En esta transición, el estudiante deja de trabajar únicamente con descripciones lógicas expresadas en lenguaje natural, diagramas de flujo o pseudocódigo, y comienza a enfrentarse a las exigencias formales de un lenguaje de programación real. Este paso posee una importancia decisiva en la formación inicial, porque permite comprender que un algoritmo no alcanza su valor práctico mientras no pueda traducirse en instrucciones precisas, verificables y ejecutables por una computadora [19], [23], [24].

Desde la perspectiva de la ingeniería de software, la codificación no constituye un acto improvisado ni una tarea aislada, sino una actividad inserta dentro de un proceso disciplinado de desarrollo. Las prácticas contemporáneas de construcción de software establecen que antes de programar deben haberse realizado tareas previas de comprensión del problema, especificación de requisitos, estructuración lógica de la solución y diseño de los artefactos que orientarán la implementación. En este sentido, los algoritmos, los diagramas de flujo y los pseudocódigos no son accesorios pedagógicos, sino representaciones intermedias valiosas que permiten reducir ambigüedades, anticipar decisiones y conservar la trazabilidad entre lo que se necesita resolver y la forma concreta en que se implementará la solución [16], [40], [41], [42].

La norma ISO/IEC/IEEE 29148 subraya la importancia de formular los requisitos con claridad, consistencia y verificabilidad, mientras que ISO/IEC/IEEE 12207 organiza los procesos del ciclo de vida del software de manera que la implementación se entienda como una actividad dependiente del análisis, del diseño y de la validación posterior [41], [42]. En términos didácticos, esto significa que programar correctamente no consiste únicamente en escribir sentencias que “funcionen”, sino en construir una solución coherente con un problema previamente comprendido, con decisiones de diseño explícitas y con criterios de calidad que permitan sostener su evolución posterior. Por ello, este capítulo no debe leerse como una simple colección de programas terminados, sino como una etapa formativa orientada a mostrar cómo un diseño algorítmico puede materializarse de manera ordenada en código fuente.

Codificar un algoritmo implica, en primer lugar, preservar la lógica esencial de la solución. Si el análisis ha identificado entradas, procesos, decisiones, repeticiones y salidas, la implementación debe reflejar esa misma estructura con fidelidad. Sin embargo, al pasar del algoritmo al programa aparecen exigencias adicionales que el diseño abstracto suele omitir deliberadamente: validación de entradas, manejo de casos límite, prevención de errores de ejecución, elección de nombres comprensibles, estandarización del estilo y documentación del código. Estas decisiones no alteran la esencia del algoritmo, pero sí determinan su robustez, su legibilidad y su mantenibilidad, que son atributos centrales de la calidad del software [33], [80], [81], [111].

En efecto, las características de calidad definidas por ISO/IEC 25010 permiten entender que un programa correcto desde el punto de vista funcional puede seguir siendo deficiente si resulta difícil de entender, probar, corregir o extender [111], [112]. Un ejercicio de programación formativo debe enseñar, desde los primeros capítulos, que la calidad no comienza cuando el sistema alcanza gran tamaño, sino desde el momento en que se decide cómo nombrar una variable, cómo organizar un bloque condicional o cómo documentar la intención de una instrucción. Por esta razón, la construcción de programas a partir de algoritmos debe promover hábitos tempranos de claridad, consistencia y disciplina técnica.

Una decisión especialmente relevante en este proceso es la selección del lenguaje de programación. Escoger un lenguaje adecuado no es un asunto meramente instrumental, ya que influye en la forma en que los estudiantes modelan problemas,

expresan soluciones y comprenden conceptos fundamentales de la computación. Un lenguaje apropiado para etapas iniciales debe ofrecer una sintaxis suficientemente rigurosa para fomentar precisión, pero al mismo tiempo debe proporcionar una estructura clara que permita mantener una relación visible entre el algoritmo diseñado y el programa implementado. También resulta conveniente que disponga de herramientas maduras, documentación abundante, un compilador confiable y un ecosistema técnico que facilite su uso en contextos académicos y profesionales [35], [87], [110].

En este marco, Java constituye una elección particularmente sólida como primer lenguaje de programación. Una de sus principales ventajas es que obliga a escribir código con una sintaxis explícita, tipada y estructurada, lo cual favorece la disciplina formal del principiante. Su sistema de tipos, el control ejercido por el compilador y la necesidad de declarar con precisión variables, expresiones y estructuras de control contribuyen a que muchos errores se detecten tempranamente, antes de la ejecución del programa [87], [107], [110]. Esta característica posee un valor pedagógico notable, ya que ayuda a que el estudiante relacione la lógica del algoritmo con la semántica real del lenguaje y comprenda que una solución computacional debe ser correcta no solo en su idea general, sino también en su formulación concreta.

Otra ventaja importante de Java es su amplio uso en la industria, en la formación universitaria y en múltiples dominios de aplicación. Se trata de un lenguaje maduro, portable y respaldado por un ecosistema tecnológico consolidado, lo cual permite que los conocimientos adquiridos en ejercicios básicos no queden restringidos a un entorno puramente escolar, sino que sirvan de base para el desarrollo posterior de aplicaciones empresariales, sistemas web, aplicaciones móviles y software distribuido [15], [40], [87]. Esta proyección profesional resulta particularmente valiosa en carreras vinculadas con sistemas de información, ya que permite articular el aprendizaje inicial con competencias más avanzadas relacionadas con arquitectura, integración, persistencia, pruebas y mantenimiento.

Java también ofrece una vía natural para introducir progresivamente el paradigma de la orientación a objetos. Aunque en una primera etapa los programas puedan construirse con estructuras simples y con un énfasis concentrado en el método `main`, el lenguaje dispone de mecanismos que permiten evolucionar desde la codificación secuencial hacia una organización modular basada en clases y objetos. Esta transición es importante porque la orientación a objetos no debe entenderse solo como un conjunto de construcciones sintácticas, sino como una forma de modelar entidades, responsabilidades y relaciones dentro de un dominio de problema [15], [16], [20]. En este paradigma, conceptos como encapsulación, abstracción, herencia y polimorfismo aportan medios para construir software más reusable, extensible y comprensible [15], [71].

El enfoque orientado a objetos adquiere mayor relevancia cuando se considera que gran parte del software contemporáneo se desarrolla mediante componentes, clases, bibliotecas y patrones reutilizables. El currículo Leidig2021Competency reconoce explícitamente que la programación orientada a objetos exige competencias relacionadas con clases, instanciación, encapsulación, abstracción, herencia, polimorfismo, modelado y principios de diseño, precisamente porque estas capacidades constituyen una base para la construcción de sistemas mantenibles y escalables [15]. Por ello, incluso cuando en este capítulo se trabaje con programas pequeños y cercanos al algoritmo, conviene anticipar al lector que tales ejercicios no son un fin en sí mismos, sino el punto de partida para formas más avanzadas de estructuración del software.

Del mismo modo, la codificación de algoritmos debe vincularse con actividades de prueba, depuración y revisión. Un programa no se completa en el instante en que compila o produce una salida aparentemente correcta. Es necesario verificar su comportamiento con entradas representativas, examinar condiciones límite, revisar la claridad del código y corregir defectos lógicos o estructurales. La literatura reciente sobre refactorización y pruebas muestra que la mejora continua del código constituye una práctica esencial para sostener la calidad del software a lo largo del tiempo [33], [80], [81]. En términos pedagógicos, esto implica que el estudiante debe aprender desde temprano que programar incluye también leer críticamente el propio código, depurarlo y refinarlo.

A lo anterior se añade una dimensión cada vez más importante: la sostenibilidad del software. Las corrientes contemporáneas asociadas con Green Software y Green IT han puesto de relieve que las decisiones de diseño e implementación también poseen consecuencias energéticas, operativas y ambientales. Aunque en ejercicios introductorios el consumo computacional no sea el problema central, sí resulta formativo comprender que la elección de estructuras, algoritmos y prácticas de implementación influye en la eficiencia del sistema y, por extensión, en su impacto material [8], [15]. Desde esta perspectiva, escribir código claro, evitar operaciones innecesarias, reducir complejidad accidental y seleccionar soluciones proporcionadas al problema no solo mejora la calidad técnica, sino que también se alinea con una visión responsable del desarrollo de software.

En consecuencia, los ejercicios de este capítulo deben asumirse como experiencias de construcción progresiva en las que convergen razonamiento algorítmico, disciplina de implementación, calidad del código y responsabilidad profesional. Cada programa desarrollado a partir de un algoritmo constituye una oportunidad para ejercitar la relación entre análisis, diseño, codificación y verificación dentro de un proceso coherente con el ciclo de vida del software [23], [24], [41], [42]. Bajo esta lógica, la programación deja de ser una simple actividad mecánica de traducción de instrucciones y se convierte en una práctica

intelectual y técnica orientada a producir soluciones correctas, comprensibles, mantenibles y útiles.

Por ello, antes de abordar cada ejercicio, conviene que el lector adopte algunas directrices fundamentales: comprender con precisión el problema antes de programar; identificar entradas, salidas y restricciones; revisar el algoritmo y el diagrama de flujo antes de escribir código; escoger nombres significativos; mantener una estructura clara de secuencia, selección e iteración; documentar solo aquello que realmente añade comprensión; validar datos de entrada; probar casos normales y casos límite; y revisar críticamente la solución final en busca de simplificación y mejora [23], [24], [33], [40]. Estas orientaciones no son recomendaciones ornamentales, sino prácticas básicas de ingeniería que contribuyen a formar programadores capaces de construir software con rigor técnico desde las primeras etapas de aprendizaje.

8.1. Promedio aritmético de N valores

El objetivo de este ejercicio consiste en desarrollar un programa capaz de leer una cantidad determinada de valores numéricos y calcular su promedio aritmético. Para ello, el programa debe solicitar al usuario el número de datos que se desea procesar, capturar cada uno de los valores introducidos y acumularlos progresivamente hasta completar la cantidad especificada. Finalmente, el promedio se obtiene dividiendo la suma total de los valores ingresados entre el número de datos procesados. Desde el punto de vista algorítmico, este problema constituye un ejemplo clásico del esquema *contador-acumulador*, ampliamente utilizado en la resolución de problemas que requieren procesamiento iterativo de datos [19], [23].

El análisis del problema y el diseño de la solución fueron desarrollados previamente mediante el diagrama de flujo y el pseudocódigo correspondientes (Ver la Figura 4.24 y el Listado de código 4.39). En dichos artefactos se definieron explícitamente las entradas del programa, el proceso iterativo encargado de acumular los valores mediante un contador y un acumulador, y la salida final asociada al cálculo del promedio. La implementación presentada en esta sección conserva una correspondencia directa con ese diseño lógico, permitiendo apreciar con claridad cómo una solución algorítmica se traduce en instrucciones concretas dentro de un lenguaje de programación. Desde la perspectiva de la ingeniería de software, esta transición desde la formulación lógica hacia la codificación constituye una práctica coherente con el desarrollo disciplinado del software, en el que el análisis, el diseño y la implementación deben mantener consistencia y trazabilidad entre sí [23], [24].

El programa mostrado en el Listado 8.1 implementa en Java el algoritmo diseñado previamente para calcular el promedio aritmético de un conjunto de valores numéricos. En primer lugar, se solicita al usuario la cantidad de datos que serán procesados. Posteriormente, mediante una estructura iterativa controlada por un contador, el programa captura cada valor ingresado y lo acumula progresivamente en una variable destinada a almacenar la suma total. Este procedimiento responde al esquema algorítmico clásico de *contador-acumulador*, ampliamente utilizado en problemas de procesamiento secuencial de datos. Una vez completada la lectura de los valores, el programa calcula el promedio dividiendo la suma acumulada entre la cantidad de datos introducidos y presenta el resultado en pantalla. Asimismo, se incluye una verificación inicial que evita realizar el cálculo cuando la cantidad de valores indicada es menor o igual a cero, lo cual contribuye a mejorar la robustez básica de la implementación.

```
1 import java.util.Scanner;
2
3 public class PromedioNValores {
4     public static void main(String[] args) {
5         Scanner entrada = new Scanner(System.in);
6
7         int cantidad;
8         int contador;
9         double valor;
10        double suma;
11        double promedio;
```

```
12
13     System.out.print("Ingrese la cantidad de valores: ");
14     cantidad = entrada.nextInt();
15
16     if (cantidad <= 0) {
17         System.out.println("No se puede calcular el promedio.");
18     } else {
19         suma = 0;
20         contador = 0;
21
22         while (contador < cantidad) {
23             System.out.print("Ingrese un valor: ");
24             valor = entrada.nextDouble();
25             suma = suma + valor;
26             contador = contador + 1;
27         }
28
29         promedio = suma / cantidad;
30
31         System.out.println("El promedio es: " + promedio);
32     }
33
34     entrada.close();
35 }
36 }
```

Listing 8.1: Calculo del Promedio de N Valores

La Tabla 8.1 muestra la correspondencia entre las instrucciones del algoritmo *PromedioNValores* y su implementación en el programa desarrollado en Java. Este tipo de comparación resulta especialmente útil en el aprendizaje inicial de la programación, ya que permite observar cómo cada paso del razonamiento algorítmico se traduce en una o varias instrucciones concretas dentro del lenguaje de programación. En algunos casos la equivalencia es directa, como ocurre con las operaciones de lectura, asignación o actualización de variables; en otros, una misma idea algorítmica se expresa mediante estructuras propias del lenguaje, como el uso de objetos de entrada (*Scanner*) o la gestión de recursos mediante el cierre del flujo de datos. De esta manera, la tabla permite identificar con claridad la relación entre la solución conceptual expresada en el algoritmo y su materialización práctica en el programa ejecutable.

Cuadro 8.1: Equivalencia entre el algoritmo *PromedioNValores* y su implementación en Java

Instrucción del algoritmo	Código en Java	Explicación
1.1 cantidad: entero 1.2 valor: real 1.3 suma: real 1.4 contador: entero 1.5 promedio: real	int cantidad; int contador; double valor; double suma; double promedio;	La sección de variables del algoritmo se implementa en Java mediante declaraciones explícitas de tipo.
2. Escribir "Digite la cantidad de valores"	System.out.print("Ingrese la cantidad de valores: ");	La instrucción de salida del algoritmo se implementa mediante <code>System.out.print()</code> .
3. Leer cantidad	cantidad = entrada.nextInt();	La lectura de datos se realiza mediante el objeto <code>Scanner</code> .
4. suma <- 0	suma = 0;	Inicialización del acumulador que almacenará la suma de los valores ingresados.
5. contador <- 0	contador = 0;	Inicialización del contador que controla el número de iteraciones del ciclo.
6. Mientras contador < cantidad hacer	while (contador < cantidad) {	La estructura repetitiva del algoritmo se implementa mediante <code>while</code> .
6.1 Escribir "Digite un valor"	System.out.print("Ingrese un valor: ");	Mensaje mostrado dentro del ciclo para solicitar cada nuevo valor.
6.2 Leer valor	valor = entrada.nextDouble();	Lectura de un número real mediante <code>nextDouble()</code> .
6.3 suma <- suma + valor	suma = suma + valor;	Operación de acumulación de los valores ingresados.
6.4 contador <- contador + 1	contador = contador + 1;	Actualización del contador del ciclo.
7. FinMientras	}	Cierre del ciclo <code>while</code> .
8. Si cantidad <= 0 Entonces	if (cantidad <= 0) {	Condición para validar que existan datos suficientes.
8.1 Escribir "No se puede calcular el promedio. No hay datos."	System.out.println("No se puede calcular el promedio.");	Mensaje mostrado cuando la cantidad de datos es inválida.
8.2 SiNo	} else {	Bloque alternativo de la condición.
8.3 promedio <- suma / cantidad	promedio = suma / cantidad;	Cálculo del promedio aritmético.
8.4 Escribir "El promedio es: ", promedio	System.out.println("El promedio es: " + promedio);	Salida del resultado final.
FinAlgoritmo	entrada.close();	Cierre del flujo de entrada en Java.

8.2. Raíces de una ecuación cuadrática

Se requiere desarrollar un programa capaz de resolver una ecuación de segundo grado de la forma $ax^2 + bx + c = 0$. Para ello, es necesario verificar en primer lugar que el coeficiente a sea distinto de cero, ya que solo en ese caso la expresión corresponde a una ecuación cuadrática. Una vez validada esta condición, el procedimiento debe calcular el discriminante de la ecuación, definido como $\Delta = b^2 - 4ac$. El valor de esta expresión determina la naturaleza de las soluciones: si el discriminante es positivo, la ecuación posee dos raíces reales diferentes; si es igual a cero, existe una única raíz real doble; y si es negativo, la ecuación no presenta soluciones reales dentro del conjunto de los números reales.

El análisis del problema y el diseño de la solución se desarrollaron previamente mediante el diagrama de flujo y el pseudocódigo correspondientes. En dichos artefactos se define una secuencia lógica que inicia con la lectura de los coeficientes de la ecuación, continúa con la verificación del coeficiente principal y posteriormente calcula el discriminante para determinar el tipo de solución mediante una estructura condicional. Este proceso refleja el principio de refinamiento progresivo utilizado en el desarrollo de algoritmos, en el cual una solución conceptual se transforma gradualmente en una implementación concreta dentro de un lenguaje de programación. En el contexto de la ingeniería de software, esta transición desde el diseño algorítmico hacia la codificación se encuentra alineada con las prácticas del ciclo de vida del software descritas por la norma [42] y con la necesidad de mantener trazabilidad entre los artefactos de desarrollo, tal como recomienda la norma ISO/IEC/IEEE 15289 [113].

A partir del algoritmo previamente definido, la solución puede implementarse en Java utilizando estructuras condicionales que reflejan directamente las decisiones planteadas en el pseudocódigo. El programa solicita al usuario los coeficientes de la ecuación, verifica que el valor de a sea diferente de cero y calcula el discriminante para determinar el tipo de raíces que posee la ecuación. Dependiendo del resultado obtenido, el programa muestra las dos raíces reales distintas, una raíz real doble o informa que no existen soluciones reales. La implementación completa se presenta en el Listado 8.2.

```
1 import java.util.Scanner;
2
3 public class EcuacionCuadratica {
4     public static void main(String[] args) {
5         Scanner entrada = new Scanner(System.in);
6
7         double a;
8         double b;
9         double c;
10        double discriminante;
11        double x1;
12        double x2;
13
14        System.out.print("Ingrese el valor de a: ");
15        a = entrada.nextDouble();
16
17        System.out.print("Ingrese el valor de b: ");
18        b = entrada.nextDouble();
19
20        System.out.print("Ingrese el valor de c: ");
```

```
21     c = entrada.nextDouble();
22
23     if (a == 0) {
24         System.out.println("No es una ecuacion cuadratica.");
25     } else {
26         discriminante = (b * b) - (4 * a * c);
27
28         if (discriminante > 0) {
29             x1 = (-b + Math.sqrt(discriminante)) / (2 * a);
30             x2 = (-b - Math.sqrt(discriminante)) / (2 * a);
31
32             System.out.println("Tiene dos raices reales diferentes:");
33             System.out.println("x1 = " + x1);
34             System.out.println("x2 = " + x2);
35         } else {
36             if (discriminante == 0) {
37                 x1 = (-b) / (2 * a);
38
39                 System.out.println("Tiene una raiz real doble:");
40                 System.out.println("x = " + x1);
41             } else {
42                 System.out.println("Tiene dos raíces imaginarias.");
43             }
44         }
45     }
46
47     entrada.close();
48 }
49 }
```

Listing 8.2: Resolución de una ecuación cuadrática

Como complemento a la implementación anterior, puede considerarse una variante de la solución que incorpora algunos refinamientos orientados a mejorar la claridad del programa y ampliar la cobertura de casos posibles durante la ejecución. En particular, esta versión introduce una verificación explícita de que el coeficiente principal sea distinto de cero antes de continuar con el cálculo, reorganiza la estructura condicional mediante el uso de `else if` para expresar de forma más directa las tres situaciones derivadas del discriminante, y contempla además el caso en el que la ecuación posee raíces complejas conjugadas (ver Listado de código 8.3), escenario que con frecuencia no se incluye en los algoritmos introductorios centrados únicamente en soluciones reales. De esta manera, el programa no solo reproduce la lógica definida en el algoritmo original,

sino que también muestra cómo una implementación puede ampliarse para manejar situaciones adicionales y mejorar la robustez de la solución cuando se ejecuta en un entorno real de programación.

```
1 import java.util.Locale;
2 import java.util.Scanner;
3
4 public class EcuacionCuadratica {
5
6     public static void main(String[] args) {
7         Locale.setDefault(Locale.US);
8         Scanner sc = new Scanner(System.in);
9
10        System.out.print("Ingrese el valor de a: ");
11        double a = sc.nextDouble();
12        System.out.print("Ingrese el valor de b: ");
13        double b = sc.nextDouble();
14        System.out.print("Ingrese el valor de c: ");
15        double c = sc.nextDouble();
16
17        // Validación de "modelo cuadrático" (a == 0).
18        if (a == 0.0) {
19            System.out.println("No es una ecuación cuadrática.");
20            sc.close();
21            return;
22        }
23
24        double discriminante = (b * b) - (4.0 * a * c);
25
26        if (discriminante > 0.0) {
27            double x1 = (-b + Math.sqrt(discriminante)) / (2.0 * a);
28            double x2 = (-b - Math.sqrt(discriminante)) / (2.0 * a);
29            System.out.println("La ecuación tiene dos raíces reales
↪ diferentes:");
30            System.out.println("x1 = " + x1);
31            System.out.println("x2 = " + x2);
32        } else if (discriminante == 0.0) {
33            double x = (-b) / (2.0 * a);
```

```
34     System.out.println("La ecuación tiene una raíz real doble:");
35     System.out.println("x = " + x);
36 } else {
37     // Caso adicional frecuente en implementación: raíces complejas (no
    ↪ siempre tratado en pseudocódigo básico).
38     double parteReal = (-b) / (2.0 * a);
39     double parteImaginaria = Math.sqrt(-discriminante) / (2.0 * a);
40     System.out.println("La ecuación tiene dos raíces complejas
    ↪ conjugadas:");
41     System.out.println("x1 = " + parteReal + " + " + parteImaginaria +
    ↪ "i");
42     System.out.println("x2 = " + parteReal + " - " + parteImaginaria +
    ↪ "i");
43 }
44
45 sc.close();
46 }
47 }
```

Listing 8.3: Resolución de una ecuación cuadrática

La Tabla 8.2 presenta la correspondencia entre las instrucciones del algoritmo *EcuacionCuadratica* y los fragmentos de código que implementan dicha lógica en el programa desarrollado en Java. Esta comparación permite observar cómo cada operación definida en el algoritmo —como la declaración de variables, la lectura de datos, las evaluaciones condicionales y el cálculo del discriminante— se traduce en construcciones concretas del lenguaje de programación. Asimismo, la tabla pone de manifiesto que algunas expresiones algorítmicas deben adaptarse a las convenciones del lenguaje, como ocurre con el cálculo de la raíz cuadrada mediante el método `Math.sqrt()` o con el uso del objeto `Scanner` para la lectura de datos desde el teclado. De esta manera, la relación establecida entre el algoritmo y el programa facilita comprender el proceso de transformación de una solución conceptual en un programa ejecutable.

Cuadro 8.2: Equivalencia entre el algoritmo *EcuacionCuadratica* y su implementación en Java

Instrucción del algoritmo	Código en Java	Explicación
1.1 a, b, c: real	double a; double b; double c;	Las variables reales del algoritmo se implementan en Java utilizando el tipo <code>double</code> , que permite representar números reales de doble precisión.
1.2 discriminante: real	double discriminante;	Variable destinada a almacenar el valor del discriminante de la ecuación cuadrática.
1.3 x1, x2: real	double x1; double x2;	Variables que almacenarán las raíces de la ecuación cuando existan soluciones reales.
2 Escribir "Ingrese el valor de a:"	System.out.print("Ingrese el valor de a:");	La instrucción de salida del algoritmo se implementa con <code>System.out.print()</code> , que muestra el mensaje al usuario.
3 Leer a	a = entrada.nextDouble();	La lectura del valor se realiza mediante el objeto <code>Scanner</code> . El método <code>nextDouble()</code> permite capturar un número real desde el teclado.
4 Escribir "Ingrese el valor de b:"	System.out.print("Ingrese el valor de b:");	Mensaje que solicita el segundo coeficiente de la ecuación.
5 Leer b	b = entrada.nextDouble();	Lectura del valor del coeficiente <i>b</i> .
6 Escribir "Ingrese el valor de c:"	System.out.print("Ingrese el valor de c:");	Mensaje para solicitar el tercer coeficiente.
7 Leer c	c = entrada.nextDouble();	Lectura del valor del coeficiente <i>c</i> .
8 Si a = 0 Entonces	if (a == 0)	Condición que verifica si el coeficiente <i>a</i> es cero. Si esto ocurre, la expresión deja de ser una ecuación cuadrática.
8.1 Escribir "No es una ecuación cuadrática"	System.out.println("No es una ecuacion cuadratica.");	Mensaje mostrado cuando la ecuación no cumple la condición necesaria para ser cuadrática.
8.3 discriminante <- $b^2 - 4*a*c$	discriminante = (b * b) - (4 * a * c);	Cálculo del discriminante de la ecuación cuadrática. En Java se utiliza multiplicación explícita en lugar del operador de potencia.
8.4 Si discriminante > 0 Entonces	if (discriminante > 0)	Condición que determina si la ecuación tiene dos raíces reales diferentes.
8.4.1 $x1 \leftarrow \frac{-b + \text{raiz2}(\text{discriminante})}{2*a}$	x1 = (-b + Math.sqrt(discriminante)) / (2 * a);	El algoritmo utiliza la función matemática raíz cuadrada. En Java se implementa mediante <code>Math.sqrt()</code> .
8.4.2 $x2 \leftarrow \frac{-b - \text{raiz2}(\text{discriminante})}{2*a}$	x2 = (-b - Math.sqrt(discriminante)) / (2 * a);	Cálculo de la segunda raíz real de la ecuación.

Continúa en la siguiente página

Continuación de la Tabla 8.2

Instrucción del algoritmo	Código en Java	Explicación
8.4.3 Escribir "La ecuación tiene dos raíces reales diferentes"	<code>System.out.println("Tiene dos raices reales diferentes:");</code>	Mensaje informativo sobre el tipo de solución encontrada.
8.4.4 Escribir "x1 = ", x1	<code>System.out.println("x1 = " + x1);</code>	Salida del valor de la primera raíz real.
8.4.5 Escribir "x2 = ", x2	<code>System.out.println("x2 = " + x2);</code>	Salida del valor de la segunda raíz real.
8.6 Si discriminante = 0 Entonces	<code>if (discriminante == 0)</code>	Condición que identifica el caso de raíz real doble.
8.6.1 $x_1 \leftarrow -b / (2 \cdot a)$	<code>x1 = (-b) / (2 * a);</code>	Cálculo de la única raíz real de la ecuación cuando el discriminante es cero.
8.6.2 Escribir "La ecuación tiene una raíz real doble"	<code>System.out.println("Tiene una raiz real doble:");</code>	Mensaje que describe el tipo de solución encontrada.
8.6.3 Escribir "x = ", x1	<code>System.out.println("x = " + x1);</code>	Salida del valor de la raíz real doble.
8.7.1 Escribir "La ecuación tiene dos raíces imaginarias"	<code>System.out.println("Tiene dos raices imaginarias.");</code>	Mensaje mostrado cuando el discriminante es negativo, indicando la presencia de raíces complejas.
FinAlgoritmo	<code>entrada.close();</code>	Cierre del recurso de entrada. Esta instrucción pertenece a la implementación en Java y no forma parte del algoritmo conceptual.

8.3. Determinar si una palabra es palíndroma

La solución de este problema fue previamente estructurada mediante su representación gráfica y algorítmica. En la Figura 4.26 se presenta el diagrama de flujo que describe el procedimiento para determinar si una palabra se lee de la misma forma de izquierda a derecha y de derecha a izquierda, mientras que el Algoritmo 4.41 formaliza dicha secuencia lógica en pseudocódigo. En ambos casos se plantea un proceso que comienza con la lectura de la palabra ingresada por el usuario, continúa con la construcción de una versión invertida de la cadena mediante un recorrido secuencial de sus caracteres y finaliza con una comparación entre la palabra original y la cadena resultante.

Si ambas cadenas coinciden exactamente, la palabra analizada cumple la condición de ser palíndroma; en caso contrario, se concluye que no presenta dicha propiedad. Este procedimiento constituye un ejemplo sencillo de procesamiento de cadenas de caracteres y permite ilustrar cómo una operación aparentemente simple puede representarse mediante estructuras iterativas y comparaciones lógicas. La implementación en Java reproduce esta misma secuencia de pasos: el programa solicita una palabra, construye su versión invertida recorriendo los caracteres desde el final hacia el inicio y finalmente compara ambas cadenas para determinar el resultado. La implementación completa del programa se presenta en el Listado 8.4.

```
1  import java.util.Scanner;
2
3  public class Palindromo {
4
5      public static void main(String[] args) {
6          Scanner sc = new Scanner(System.in);
7
8          System.out.print("Ingrese una palabra: ");
9          String palabra = sc.nextLine();
10
11          // Construcción de la inversión (equivalente al for decreciente del
12          ↪ pseudocódigo).
13          String invertida = "";
14          for (int i = palabra.length() - 1; i >= 0; i--) {
15              invertida += palabra.charAt(i);
16          }
17
18          if (palabra.equals(invertida)) {
19              System.out.println(palabra + " es palabra palíndroma");
20          } else {
21              System.out.println(palabra + " NO es palabra palíndroma");
22          }
23
24          sc.close();
25      }
```

Listing 8.4: Resolución de una ecuación cuadrática

La correspondencia entre el algoritmo *Palindromo* y su implementación en Java puede comprenderse mejor mediante un análisis comparativo entre las instrucciones del procedimiento algorítmico y los fragmentos de código que materializan dicha lógica en el programa. En el algoritmo se describe de forma conceptual el proceso de lectura de la palabra, la determinación de su longitud, la construcción de una cadena invertida mediante un recorrido decreciente de sus caracteres y la comparación final entre la palabra original y la cadena resultante. En la implementación en Java, estas operaciones se concretan a través de elementos propios del lenguaje, tales como el uso del tipo **String** para representar cadenas de caracteres, el método **length()** para obtener su longitud, el método **charAt()** para acceder a cada carácter durante el recorrido inverso y el método **equals()** para comparar correctamente el contenido de dos cadenas.

Con el fin de visualizar esta relación de forma sistemática, en la Tabla 8.3 se presenta una correspondencia detallada entre las instrucciones del algoritmo y los fragmentos de código que implementan dichas operaciones en el programa desarrollado en Java. Este análisis comparativo permite identificar cómo cada acción expresada en el nivel algorítmico se transforma en construcciones específicas del lenguaje de programación, evidenciando tanto las equivalencias directas entre ambos niveles de representación como las adaptaciones necesarias derivadas de la sintaxis y de las convenciones propias del lenguaje.

Cuadro 8.3: Equivalencia entre el algoritmo *Palindromo* y su implementación en Java

Instrucción del algoritmo	Código en Java	Explicación
1.1 palabra, invertida: cadena	String palabra; String invertida = "";	Las variables de tipo <i>cadena</i> del algoritmo se implementan en Java mediante el tipo String , utilizado para almacenar secuencias de caracteres.
1.2 i, longPal: entero	for (int i = palabra.length() - 1; i >= 0; i--)	Las variables enteras utilizadas para controlar el recorrido de la cadena se implementan con el tipo int . En Java, la variable de control i se declara directamente dentro de la estructura for .
2 Escribir "Ingrese una palabra"	System.out.print("Ingrese una palabra:");	La instrucción de salida del algoritmo se implementa en Java mediante System.out.print() , que muestra el mensaje al usuario.
3 Leer palabra	String palabra = sc.nextLine();	La lectura de la cadena se realiza utilizando el objeto Scanner y el método nextLine() , que captura una línea completa de texto.
4 longPal <- longitud(palabra)	palabra.length()	La función algorítmica <i>longitud()</i> se implementa en Java mediante el método length() , que devuelve la cantidad de caracteres de la cadena. En el programa este valor se utiliza directamente en el ciclo.
5 invertida <- ""	String invertida = "";	Inicialización de la cadena que almacenará la palabra invertida.
6 Para i desde longPal hasta 1 con paso -1 hacer	for (int i = palabra.length() - 1; i >= 0; i--)	El ciclo <i>Para</i> del algoritmo se implementa en Java mediante la estructura for . El recorrido se realiza desde el último índice de la cadena hasta el primero, disminuyendo el valor del índice en cada iteración.
6.1 invertida <- invertida + subcadena(palabra, i, 1)	invertida += palabra.charAt(i);	La operación de concatenación del algoritmo se implementa mediante el operador += . El método charAt(i) permite acceder al carácter ubicado en la posición i de la cadena.
7 FinPara	}	Cierre del bloque del ciclo for .
8 Si palabra = invertida Entonces	if (palabra.equals(invertida))	La comparación entre cadenas se realiza en Java mediante el método equals() , ya que el operador == compara referencias y no el contenido de las cadenas.

Continúa en la siguiente página

Continuación de la Tabla 8.3

Instrucción del algoritmo	Código en Java	Explicación
8.1 Escribir palabra, " es palabra palíndroma"	System.out.println(palabra + " es palabra palíndroma");	Salida del mensaje cuando la palabra coincide con su versión invertida.
8.2 SiNo	} else {	La estructura alternativa del algoritmo se implementa mediante else .
8.3 Escribir palabra, " NO es palabra palíndroma"	System.out.println(palabra + " NO es palabra palíndroma");	Mensaje mostrado cuando la palabra no coincide con su versión invertida.
FinAlgoritmo	sc.close();	Cierre del flujo de entrada utilizado para leer datos desde el teclado. Esta instrucción pertenece a la implementación en Java y no forma parte del algoritmo conceptual.

8.4. Cálculo de la edad exacta

El problema de calcular la edad exacta de una persona en años, meses y días exige algo más que una resta directa entre fechas. La solución requiere distinguir correctamente las unidades temporales involucradas y aplicar ajustes cuando el día o el mes actuales son menores que los valores correspondientes de la fecha de nacimiento. Por esta razón, la propuesta desarrollada previamente en la Figura 4.27 y en el Algoritmo ?? organiza el procedimiento en cuatro momentos bien definidos: captura de la fecha de nacimiento, obtención de la fecha actual, cálculo inicial por diferencias y corrección de resultados negativos mediante préstamos. Esta estructura resulta especialmente valiosa en programación básica porque separa con claridad el cálculo principal de las correcciones necesarias para preservar la consistencia lógica del resultado, favoreciendo así la comprensión del problema y su posterior traducción a código [23], [24].

Uno de los aspectos didácticos más importantes de este algoritmo es que muestra cómo un problema cotidiano puede requerir reglas de tratamiento específicas cuando se implementa computacionalmente. En este caso, si la diferencia de días resulta negativa, se realiza un préstamo desde la unidad mes; de manera análoga, si la diferencia de meses es negativa, se toma un préstamo desde la unidad año. Esta estrategia evita resultados imposibles, como edades con meses o días negativos, y enseña al estudiante que no toda operación entre datos puede resolverse con una sola expresión aritmética. En términos formativos, este tipo de ejercicios fortalece la capacidad de descomponer el problema en pasos ordenados, identificar condiciones de corrección y construir soluciones que respondan no solo al caso normal, sino también a situaciones límite.

También conviene advertir que, en su forma algorítmica introductoria, esta solución suele emplear una simplificación útil para fines pedagógicos: el ajuste de días mediante un préstamo fijo de 30. Tal aproximación facilita el razonamiento inicial y mantiene una relación muy cercana entre el diagrama de flujo, el pseudocódigo y la implementación. Sin embargo, desde una perspectiva profesional, el tratamiento exacto del calendario requiere considerar la cantidad real de días de cada mes, así como los años bisiestos y otras validaciones asociadas a fechas válidas. Por ello, este ejercicio debe entenderse como una representación formativa orientada a consolidar estructuras de selección, asignación y validación, antes de introducir mecanismos más sofisticados de manejo de fechas. Precisamente allí radica su valor didáctico: permite al estudiante concentrarse primero en la lógica del procedimiento antes de pasar a soluciones apoyadas en bibliotecas especializadas.

Otro aspecto que debe subrayarse es la importancia de validar la coherencia temporal de los datos de entrada. Una fecha de nacimiento posterior a la fecha actual no pertenece al dominio válido del problema y, por tanto, debe detectarse antes de ejecutar el cálculo final. Esta verificación no constituye un detalle accesorio, sino una práctica elemental de robustez en el desarrollo de software, ya que impide producir resultados carentes de sentido y mejora la confiabilidad del programa. En términos de calidad, este tipo de control fortalece la correspondencia entre la intención del algoritmo y el comportamiento efectivo del sistema, promoviendo soluciones más seguras, comprensibles y mantenibles [23], [24].

Al trasladar esta solución a Java, conviene conservar una correspondencia visible con la lógica del algoritmo. Esto significa que el programa debe reflejar de manera explícita la lectura de datos, la obtención o definición de la fecha actual, el cálculo inicial de diferencias y los bloques condicionales que aplican los préstamos. Mantener esa equivalencia entre representación algorítmica y representación codificada resulta especialmente importante en un capítulo de programación básica, porque ayuda al estudiante a reconocer cómo las decisiones expresadas en pseudocódigo se convierten en instrucciones ejecutables. Asimismo, favorece la trazabilidad entre la solución diseñada previamente y su implementación concreta, reforzando la disciplina de construcción gradual propia de la programación formativa.

La implementación en Java que se presenta en el Listado de código 8.5, el mismo conserva esa relación directa con la solución estructurada diseñada previamente. El programa solicita los componentes de la fecha de nacimiento, obtiene o utiliza los valores correspondientes a la fecha actual y realiza el cálculo de la edad exacta mediante diferencias sucesivas y correcciones por préstamos cuando los resultados parciales son negativos. Además, incorpora una verificación previa para impedir que se procese una fecha de nacimiento posterior a la fecha actual. De este modo, el código no solo ejecuta el procedimiento previsto en la solución algorítmica, sino que también muestra con claridad cómo un razonamiento expresado en diagrama de flujo y pseudocódigo puede transformarse en un programa funcional dentro del lenguaje Java.

```
1 import java.util.Scanner;
2 import java.time.LocalDate;
3
4 public class EdadExacta {
```

```
5     public static void main(String[] args) {
6         Scanner entrada = new Scanner(System.in);
7
8         int diaNacimiento;
9         int mesNacimiento;
10        int anioNacimiento;
11
12        int diaActual;
13        int mesActual;
14        int anioActual;
15
16        int dias;
17        int meses;
18        int anios;
19
20        System.out.print("Ingrese el día de nacimiento: ");
21        diaNacimiento = entrada.nextInt();
22
23        System.out.print("Ingrese el mes de nacimiento: ");
24        mesNacimiento = entrada.nextInt();
25
26        System.out.print("Ingrese el año de nacimiento: ");
27        anioNacimiento = entrada.nextInt();
28
29        // Obtener la fecha actual del sistema
30        LocalDate fechaActual = LocalDate.now();
31
32        // Extraer día, mes y año
33        diaActual = fechaActual.getDayOfMonth();
34        mesActual = fechaActual.getMonthValue();
35        anioActual = fechaActual.getYear();
36
37        if (anioNac > anioHoy || (anioNac == anioHoy && mesNac > mesHoy) ||
    ↪ (anioNac == anioHoy && mesNac == mesHoy && diaNac > diaHoy)) {
38            System.out.println("La fecha de nacimiento es incorrecta.");
39        } else {
40            if (mesNacimiento > mesActual) {
```

```
41         años = añoActual - añoNacimiento - 1;
42         meses = mesActual + 12 - mesNacimiento;
43     } else {
44         años = añoActual - añoNacimiento;
45         meses = mesActual - mesNacimiento;
46     }
47
48     if (díaNacimiento > díaActual) {
49         meses = meses - 1;
50         días = díaActual + 30 - díaNacimiento;
51     } else {
52         días = díaActual - díaNacimiento;
53     }
54
55     System.out.println("Edad exacta:");
56     System.out.println("Años: " + años);
57     System.out.println("Meses: " + meses);
58     System.out.println("Días: " + días);
59 }
60
61 entrada.close();
62 }
63 }
```

Listing 8.5: Resolución de una ecuación cuadrática

La correspondencia entre la formulación algorítmica del procedimiento y su implementación concreta en el lenguaje Java puede apreciarse mediante una comparación sistemática de las instrucciones que componen ambas representaciones. Con el propósito de evidenciar esta relación, se presenta a continuación una síntesis comparativa donde se confrontan las operaciones definidas en el algoritmo con los fragmentos de código que materializan dichas operaciones en el programa.

Este análisis permite observar cómo las instrucciones abstractas del algoritmo —expresadas mediante operaciones de lectura, asignación, estructuras condicionales y cálculos aritméticos— se traducen en construcciones propias del lenguaje Java, tales como declaraciones de variables, uso de clases de biblioteca, instrucciones de entrada y salida, y estructuras de control. Asimismo, la comparación pone de manifiesto aquellos casos en los que la correspondencia no es estrictamente literal, debido a decisiones de implementación propias del lenguaje o a optimizaciones en la organización del cálculo. De esta manera, la comparación presentada en la Tabla 8.4 permite comprender con claridad la continuidad conceptual existente entre el diseño algorítmico previo y su realización programática.

Cuadro 8.4: Equivalencia entre el algoritmo *EdadExactaPrestamos* y su implementación en Java

Instrucción del algoritmo	Código en Java	Explicación
1.1 anioNac, mesNac, diaNac: entero	<pre>int diaNacimiento; int mesNacimiento; int anioNacimiento;</pre>	Las variables del algoritmo que representan la fecha de nacimiento se implementan en Java con el tipo <code>int</code> . Aunque los nombres no son idénticos, la función que cumplen dentro de la solución es equivalente.
1.2 anioHoy, mesHoy, diaHoy: entero	<pre>int diaActual; int mesActual; int anioActual;</pre>	Las variables asociadas con la fecha actual también se implementan con el tipo <code>int</code> . En Java se emplean nombres distintos, pero semánticamente corresponden a las mismas unidades temporales definidas en el algoritmo.
1.3 edadAnio, edadMes, edadDia: entero	<pre>int dias; int meses; int anios;</pre>	Las variables del algoritmo que almacenan el resultado final de la edad se representan en Java mediante variables enteras. En este caso, el programa utiliza nombres más breves, pero con el mismo propósito lógico.
2. anioHoy <- obtenerAnio(obtenerFecha())	<pre>LocalDate fechaActual = LocalDate.now(); anioActual = fechaActual.getYear();</pre>	La obtención del año actual no se implementa con funciones abstractas como en el algoritmo, sino mediante la clase <code>LocalDate</code> de Java. Primero se obtiene la fecha del sistema y luego se extrae el año.
3. mesHoy <- obtenerMes(obtenerFecha())	<pre>mesActual = fechaActual.getMonthValue();</pre>	La extracción del mes actual se implementa mediante el método <code>getMonthValue()</code> , que devuelve el número del mes contenido en la fecha actual del sistema.
4. diaHoy <- obtenerDia(obtenerFecha())	<pre>diaActual = fechaActual.getDayOfMonth();</pre>	La obtención del día actual se implementa con el método <code>getDayOfMonth()</code> , que devuelve el día del mes correspondiente a la fecha del sistema.
5. Escribir "Digite el anio de nacimiento"	<pre>System.out.print("Ingrese el anio de nacimiento: ");</pre>	La instrucción de salida del algoritmo encuentra su correspondencia en la instrucción <code>System.out.print()</code> del programa Java.
6. Leer anioNac	<pre>anioNacimiento = entrada.nextInt();</pre>	La lectura del año de nacimiento se implementa mediante el objeto <code>Scanner</code> y el método <code>nextInt()</code> .
7. Escribir "Digite el mes de nacimiento"	<pre>System.out.print("Ingrese el mes de nacimiento: ");</pre>	El algoritmo solicita el mes de nacimiento mediante una salida en pantalla; en Java esta acción se implementa mediante <code>System.out.print()</code> .
8. Leer mesNac	<pre>mesNacimiento = entrada.nextInt();</pre>	La lectura del mes de nacimiento se implementa directamente con <code>nextInt()</code> .
Continúa en la siguiente página		

Continuación de la Tabla 8.4

Instrucción del algoritmo	Código en Java	Explicación
9. Escribir "Digite el día de nacimiento"	<code>System.out.print("Ingrese el día de nacimiento: ");</code>	La solicitud del día de nacimiento se traduce en el programa mediante una instrucción de salida estándar.
10. Leer diaNac	<code>diaNacimiento = entrada.nextInt();</code>	La lectura del día de nacimiento también se implementa con el método <code>nextInt()</code> del objeto <code>Scanner</code> .
11. Si (anioNac > anioHoy OR ...) Entonces	<code>if (anioNac > anioHoy (anioNac == anioHoy && mesNac > mesHoy) (anioNac == anioHoy && mesNac == mesHoy && diaNac > diaHoy)) {</code>	La validación lógica de consistencia temporal es equivalente: en ambos casos se comprueba si la fecha de nacimiento es posterior a la fecha actual. No obstante, en el código Java existe una inconsistencia nominal importante, ya que se utilizan las variables <code>anioNac</code> , <code>mesNac</code> , <code>diaNac</code> , <code>anioHoy</code> , <code>mesHoy</code> y <code>diaHoy</code> , pero las variables declaradas previamente fueron <code>anioNacimiento</code> , <code>mesNacimiento</code> , <code>diaNacimiento</code> , <code>anioActual</code> , <code>mesActual</code> y <code>diaActual</code> . Tal como está escrito, ese fragmento produce error de compilación y debe corregirse para mantener coherencia con las variables declaradas.
11.1 Escribir "La fecha de nacimiento no puede ser mayor que la fecha actual."	<code>System.out.println("La fecha de nacimiento es incorrecta.");</code>	Ambas instrucciones cumplen la misma función: informar que la fecha de nacimiento ingresada no pertenece al dominio válido del problema. El mensaje del programa es más breve, pero semánticamente equivalente.
11.2 FinAlgoritmo	No existe una instrucción equivalente explícita.	En el algoritmo se indica explícitamente que la ejecución termina en este punto. En Java, aunque no aparece una sentencia <code>return</code> , el resto del código se encuentra en el bloque <code>else</code> , por lo que la ejecución práctica también se detiene respecto del cálculo de la edad. La equivalencia es funcional, aunque no textual.
13. edadAnio <- anioHoy - anioNac	<code>anios = anioActual - anioNacimiento;</code> o bien <code>anios = anioActual - anioNacimiento - 1;</code>	No existe una traducción literal inmediata, porque el programa Java reorganiza el cálculo de la edad en función de una condición previa sobre los meses. Mientras el algoritmo calcula primero la diferencia simple y corrige después mediante préstamos, el programa anticipa parte de esa corrección en la estructura condicional sobre los meses.

Continúa en la siguiente página

Continuación de la Tabla 8.4

Instrucción del algoritmo	Código en Java	Explicación
14. edadMes <- mesHoy - mesNac	meses = mesActual - mesNacimiento; o bien meses = mesActual + 12 - mesNacimiento;	El algoritmo realiza inicialmente una resta directa y luego corrige si el resultado es negativo. En cambio, el programa Java implementa una variante equivalente desde el punto de vista lógico: cuando el mes de nacimiento supera al mes actual, agrega doce meses y descuenta un año de manera anticipada.
15. edadDia <- diaHoy - diaNac	dias = diaActual - diaNacimiento; o bien dias = diaActual + 30 - diaNacimiento;	De forma análoga al caso anterior, el algoritmo plantea primero una resta simple y después corrige si el resultado es negativo. El programa Java adelanta la corrección cuando el día de nacimiento es mayor que el día actual.
16. Si edadDia < 0 Entonces	if (diaNacimiento > diaActual) {	No existe equivalencia literal, pero sí equivalencia lógica. La condición del algoritmo verifica si la diferencia de días fue negativa; el programa Java expresa esa misma situación comparando directamente el día de nacimiento con el día actual.
16.1 edadDia <- edadDia + 30	dias = diaActual + 30 - diaNacimiento;	La corrección por préstamo de días se implementa en el programa incorporando directamente el valor 30 al cálculo de dias . Aunque la forma sintáctica cambia, la operación matemática es equivalente.
16.2 edadMes <- edadMes - 1	meses = meses - 1;	La disminución de un mes, producto del préstamo realizado para ajustar los días, se conserva de manera directa en el programa Java.
17. FinSi	}	El cierre del bloque condicional se representa en Java mediante la llave de cierre correspondiente.
18. Si edadMes < 0 Entonces	if (mesNacimiento > mesActual) {	El algoritmo revisa si la diferencia de meses fue negativa; el programa Java detecta esa situación comparando directamente el mes de nacimiento con el mes actual. Ambas formulaciones son equivalentes desde el punto de vista lógico, aunque no desde la forma textual.
18.1 edadMes <- edadMes + 12	meses = mesActual + 12 - mesNacimiento;	La corrección de meses mediante préstamo de un año se implementa directamente dentro del cálculo de meses , sin pasar por un valor negativo intermedio.
Continúa en la siguiente página		

Continuación de la Tabla 8.4

Instrucción del algoritmo	Código en Java	Explicación
18.2 edadAnio <- edadAnio - 1	<code>anios = anioActual - anioNacimiento - 1;</code>	La disminución de un año como consecuencia del préstamo de doce meses se refleja de manera directa en la asignación utilizada por el programa Java.
19. FinSi	<code>}</code>	Corresponde al cierre del bloque condicional que ajusta la edad en años y meses.
20. Escribir "Su edad es: ", edadAnio, " anios ", edadMes, " meses ", edadDia, " dias"	<code>System.out.println("Edad exacta:"); System.out.println("Anios: " + anios); System.out.println("Meses: " + meses); System.out.println("Dias: " + dias);</code>	La salida final del algoritmo se implementa en Java mediante varias instrucciones <code>println()</code> . Aunque el algoritmo presenta toda la información en una sola instrucción conceptual, el programa distribuye la salida en varias líneas para mejorar la legibilidad del resultado.
FinAlgoritmo	<code>entrada.close();</code>	El cierre del recurso de entrada es una operación propia de la implementación en Java y no forma parte del algoritmo conceptual. Se incluye como buena práctica para liberar el recurso asociado al teclado.

La solución presentada en el Listado 8.6 muestra una variante de la implementación anterior basada en el uso de las clases especializadas de manejo de fechas incluidas en la biblioteca estándar de Java. A diferencia de la primera versión, en la cual el cálculo de la edad se realiza mediante diferencias aritméticas entre días, meses y años junto con ajustes por préstamos entre unidades temporales, esta implementación delega el cálculo en las abstracciones proporcionadas por la API `java.time`. En particular, el empleo de la clase `LocalDate` permite representar fechas reales del calendario gregoriano, mientras que la clase `Period` calcula automáticamente la diferencia exacta entre dos fechas mediante el método `Period.between`, considerando la duración real de los meses, los años bisiestos y las reglas del calendario.

Otra diferencia relevante que puede observarse en el Listado 8.6 es el fortalecimiento de la robustez del programa mediante el uso de mecanismos de validación proporcionados por la propia biblioteca. La construcción del objeto `LocalDate.of(anio, mes, dia)` permite detectar automáticamente combinaciones inválidas de día, mes y año, lo cual se gestiona mediante una estructura `try-catch`. Asimismo, se mantiene la verificación de consistencia temporal mediante el método `isAfter`, evitando que se procese una fecha de nacimiento posterior a la fecha actual.

```
1 import java.time.LocalDate;
2 import java.time.Period;
3 import java.util.Scanner;
4
5 public class EdadExacta {
6
7     public static void main(String[] args) {
8         Scanner sc = new Scanner(System.in);
9
10        // Entrada didáctica: año, mes, día.
11        System.out.print("Ingrese año de nacimiento (YYYY): ");
12        int anio = sc.nextInt();
13        System.out.print("Ingrese mes de nacimiento (1-12): ");
14        int mes = sc.nextInt();
15        System.out.print("Ingrese día de nacimiento (1-31): ");
16        int dia = sc.nextInt();
17
18        LocalDate nacimiento;
19        try {
20            nacimiento = LocalDate.of(anio, mes, dia);
21        } catch (Exception ex) {
22            System.out.println("Fecha inválida. Verifique año/mes/día.");
23            sc.close();
24            return;
25        }
26
27        LocalDate hoy = LocalDate.now();
```

```

28
29      // Refuerzo de robustez: consistencia temporal (nacimiento no
    ↪ posterior a hoy).
30      if (nacimiento.isAfter(hoy)) {
31          System.out.println("La fecha de nacimiento no puede ser
    ↪ posterior a la fecha actual.");
32          sc.close();
33          return;
34      }
35
36      Period edad = Period.between(nacimiento, hoy);
37
38      System.out.println("Edad exacta:");
39      System.out.println("Años : " + edad.getYears());
40      System.out.println("Meses: " + edad.getMonths());
41      System.out.println("Días : " + edad.getDays());
42
43      sc.close();
44  }
45 }

```

Listing 8.6: Resolución de una ecuación cuadrática

Esta versión ilustra cómo, en contextos de programación más avanzados, es posible reemplazar cálculos manuales por bibliotecas especializadas que encapsulan reglas complejas del dominio. En consecuencia, el código mostrado en el Listado 8.6 no solo simplifica la implementación del algoritmo, sino que también refleja una práctica habitual en el desarrollo profesional de software: aprovechar las bibliotecas del lenguaje para mejorar la precisión, claridad y mantenibilidad de los programas.

8.5. Cálculo del MCD mediante el Algoritmo de Euclides

El objetivo de este ejercicio consiste en desarrollar un programa capaz de determinar el máximo común divisor (MCD) de dos números enteros. Este problema posee una gran importancia matemática y computacional, ya que el cálculo del MCD interviene en procedimientos como la simplificación de fracciones, la teoría de números, la criptografía y diversos algoritmos de verificación de divisibilidad. Desde el punto de vista algorítmico, la solución se fundamenta en el algoritmo de Euclides, un método clásico que permite obtener el máximo común divisor de dos enteros mediante divisiones sucesivas con residuo. Su relevancia en la formación inicial radica no solo en su eficiencia, sino también en que constituye un ejemplo claro de refinamiento iterativo y de uso sistemático de estructuras condicionales dentro de un proceso repetitivo [19], [23], [24].

El análisis del problema y el diseño de la solución fueron desarrollados previamente mediante su representación gráfica y algorítmica (ver la Figura 4.29 y el Listado de código 4.44). En dichos artefactos se define una secuencia lógica que inicia con la lectura de dos números enteros, continúa con una verificación orientada a asegurar el orden de los operandos y posteriormente

desarrolla un proceso iterativo basado en la operación de residuo. En este procedimiento, el número mayor se divide entre el menor, el residuo obtenido reemplaza al segundo operando y el proceso se repite hasta que dicho residuo sea igual a cero. En ese momento, el valor restante corresponde al máximo común divisor de los números originales.

El diagrama de flujo permite observar además una decisión de diseño importante: antes de iniciar el ciclo principal se verifica si el primer número es menor que el segundo. Cuando esta condición se cumple, se realiza un intercambio de valores mediante una variable auxiliar. Esta operación no modifica el resultado matemático del procedimiento, pero facilita la ejecución del algoritmo al garantizar que el mayor valor se utilice como primer operando. Desde una perspectiva didáctica, este paso introduce al estudiante en el uso de variables auxiliares y en la lógica del intercambio de datos, una técnica frecuente en la construcción de algoritmos.

La fase central del procedimiento se desarrolla mediante un ciclo repetitivo sustentado en la operación MOD, la cual permite calcular el residuo de la división entera entre dos números. En cada iteración se actualizan los operandos de acuerdo con el principio fundamental del algoritmo de Euclides: el máximo común divisor de dos enteros permanece invariante si el mayor se reemplaza por el residuo de dividirlo entre el menor. El proceso continúa hasta que el residuo se vuelve cero, momento en el cual el valor almacenado en el primer operando corresponde al MCD buscado. Este mecanismo ilustra con claridad cómo una propiedad matemática puede transformarse en una estrategia computacional eficiente y bien estructurada.

El programa presentado en el Listado 8.7 implementa en Java el algoritmo de Euclides para calcular el máximo común divisor de dos números enteros. En primer lugar, el programa solicita al usuario los dos valores que serán utilizados en el cálculo. Posteriormente se verifica si el primer número es menor que el segundo; cuando esta condición se cumple, se realiza un intercambio de valores mediante una variable auxiliar con el propósito de mantener el mayor número en la primera posición. A continuación, el programa ejecuta un ciclo iterativo basado en la operación de residuo, en el cual se calcula sucesivamente el resto de la división entre ambos números y se actualizan los operandos conforme a la lógica del algoritmo de Euclides. Este proceso se repite hasta que el segundo número se convierte en cero, momento en el cual el primer valor contiene el máximo común divisor buscado. Finalmente, el programa muestra el resultado al usuario y libera el recurso asociado al flujo de entrada.

```
1  import java.util.Scanner;
2
3  public class MCD {
4      public static void main(String[] args) {
5          Scanner entrada = new Scanner(System.in);
6
7          int num1;
8          int num2;
9          int resto;
10         int aux;
11
12         System.out.print("Digite el primer numero: ");
13         num1 = entrada.nextInt();
14
15         System.out.print("Digite el segundo numero: ");
16         num2 = entrada.nextInt();
17
18         if (num1 < num2) {
19             aux = num1;
```

```
20         num1 = num2;
21         num2 = aux;
22     }
23
24     while (num2 != 0) {
25         resto = num1 % num2;
26         num1 = num2;
27         num2 = resto;
28     }
29
30     System.out.println("El MCD es: " + num1);
31
32     entrada.close();
33 }
34 }
```

Listing 8.7: Cálculo del máximo común divisor mediante el algoritmo de Euclides

Con el propósito de evidenciar la correspondencia entre la formulación algorítmica del procedimiento y su implementación en Java, se presenta a continuación una comparación estructurada entre las instrucciones del algoritmo y los fragmentos de código que las materializan en el programa. Este tipo de análisis permite identificar cómo cada paso del razonamiento algorítmico se traduce en construcciones concretas del lenguaje de programación, tales como declaraciones de variables, estructuras condicionales, ciclos iterativos y operaciones aritméticas. Asimismo, facilita comprender que algunas operaciones del algoritmo encuentran una equivalencia directa en el código, mientras que otras se implementan mediante estructuras propias del lenguaje que cumplen la misma función lógica dentro del programa. La comparación presentada en la Tabla 8.5 permite apreciar con claridad esta relación entre el diseño conceptual del algoritmo y su realización programática.

Cuadro 8.5: Equivalencia entre el algoritmo *MCDEuclides* y su implementación en Java

Instrucción del algoritmo	Código en Java	Explicación
1.1 num1, num2, resto, aux: entero	int num1; int num2; int resto; int aux;	Las variables declaradas en el algoritmo se implementan en Java mediante variables de tipo entero. Cada una cumple una función específica dentro del cálculo del máximo común divisor.
2. Escribir "Digite el primer numero"	System.out.print("Digite el primer numero: ");	La instrucción de salida del algoritmo se implementa mediante <code>System.out.print()</code> , que muestra el mensaje en pantalla para solicitar el primer valor.
3. Leer num1	num1 = entrada.nextInt();	La lectura del primer número se realiza utilizando el objeto <code>Scanner</code> y el método <code>nextInt()</code> .
4. Escribir "Digite el segundo numero"	System.out.print("Digite el segundo numero: ");	El programa solicita al usuario el segundo valor mediante otra instrucción de salida estándar.
5. Leer num2	num2 = entrada.nextInt();	La lectura del segundo número se realiza mediante el mismo mecanismo de entrada de datos.
6. Si num1 < num2 Entonces	if (num1 < num2) {	La estructura condicional del algoritmo se implementa mediante una sentencia <code>if</code> .
6.1 aux <- num1	aux = num1;	La variable auxiliar almacena temporalmente el valor del primer número para poder realizar el intercambio.
6.2 num1 <- num2	num1 = num2;	Se asigna el valor del segundo número al primero como parte del proceso de intercambio.
6.3 num2 <- aux	num2 = aux;	El valor original del primer número se recupera desde la variable auxiliar para completar el intercambio.
7. Mientras num2 <> 0 hacer	while (num2 != 0) {	La estructura iterativa del algoritmo se implementa mediante el ciclo <code>while</code> , el cual se ejecuta mientras el segundo número sea distinto de cero.
7.1 resto <- num1 MOD num2	resto = num1% num2;	La operación de residuo del algoritmo se implementa en Java mediante el operador <code>%</code> .
7.2 num1 <- num2	num1 = num2;	El segundo número pasa a ocupar la posición del primero como parte del proceso iterativo del algoritmo de Euclides.
7.3 num2 <- resto	num2 = resto;	El residuo calculado se convierte en el nuevo segundo operando para la siguiente iteración.
8. FinMientras	}	Cierre del ciclo iterativo que implementa el núcleo del algoritmo de Euclides.

Continúa en la siguiente página

Continuación de la Tabla 8.5

Instrucción del algoritmo	Código en Java	Explicación
9. Escribir "El MCD es", num1	<code>System.out.println("El MCD es: " + num1);</code>	Una vez finalizado el ciclo, el valor almacenado en num1 corresponde al máximo común divisor de los números originales y se muestra en pantalla.
FinAlgoritmo	<code>entrada.close();</code>	El cierre del flujo de entrada constituye una práctica recomendable en Java para liberar el recurso asociado al objeto <code>Scanner</code> .

8.6. Cálculo del mínimo común múltiplo

El objetivo de este ejercicio consiste en desarrollar un procedimiento capaz de determinar el mínimo común múltiplo (MCM) de dos números enteros. Este concepto posee una gran importancia en matemáticas discretas y en diversas aplicaciones computacionales, ya que permite identificar el menor número entero positivo que es divisible por ambos valores considerados. El cálculo del MCM se emplea con frecuencia en operaciones aritméticas como la suma y resta de fracciones, en problemas de sincronización de eventos periódicos y en diversos algoritmos utilizados en teoría de números y computación científica [19], [23], [24].

El análisis del problema y el diseño de la solución fueron elaborados previamente mediante el diagrama de flujo y el pseudocódigo correspondientes (ver la Figura 4.30 y el Listado de código 4.45). En estos artefactos se establece una secuencia lógica que comienza con la lectura de dos números enteros introducidos por el usuario. Posteriormente se realiza una verificación inicial destinada a identificar un caso particular importante: cuando alguno de los números es igual a cero. En tal situación, el mínimo común múltiplo se define como cero, ya que ningún número positivo distinto de cero puede ser múltiplo simultáneamente de un valor nulo y de otro número entero. Esta validación temprana permite evitar cálculos innecesarios y constituye una práctica habitual en el diseño de algoritmos robustos.

Cuando ambos números son distintos de cero, el procedimiento continúa con una etapa de normalización de los valores mediante la aplicación de la función valor absoluto. Esta operación asegura que el cálculo posterior se realice únicamente con magnitudes positivas, lo cual simplifica el razonamiento matemático del algoritmo y evita inconsistencias derivadas de la presencia de números negativos. A continuación, el algoritmo utiliza como paso intermedio el cálculo del máximo común divisor (MCD) entre los dos valores ingresados. Este resultado permite aplicar la conocida relación matemática que vincula el MCD y el MCM de dos números enteros.

La fase final del procedimiento consiste en aplicar la expresión

$$\text{mcm}(a, b) = \frac{|a \cdot b|}{\text{mcd}(a, b)},$$

la cual establece que el mínimo común múltiplo de dos números puede obtenerse dividiendo el producto de ambos entre su máximo común divisor. Esta relación constituye una propiedad fundamental de la aritmética de enteros y permite calcular el MCM de manera eficiente cuando ya se dispone de un algoritmo para determinar el MCD. Una vez obtenido el resultado, el algoritmo muestra en pantalla el valor del mínimo común múltiplo y concluye su ejecución.

Desde la perspectiva del diseño algorítmico, este ejercicio resulta especialmente ilustrativo porque integra distintos conceptos importantes en la construcción de programas: validación de datos de entrada, uso de funciones auxiliares, reutilización de algoritmos previamente desarrollados y aplicación de propiedades matemáticas para simplificar el cálculo. De esta manera, el problema permite observar cómo una relación matemática puede transformarse en un procedimiento computacional estructurado y eficiente, manteniendo una correspondencia clara entre el análisis conceptual, el diseño algorítmico y su posterior implementación en un lenguaje de programación.

El programa mostrado en el Listado 8.8 implementa en Java el algoritmo diseñado previamente para calcular el mínimo común múltiplo de dos números enteros. En primer lugar, el programa solicita al usuario los dos valores que serán utilizados en el cálculo. Posteriormente se verifica si alguno de los números es igual a cero; en tal caso, el mínimo común múltiplo se establece directamente en cero, evitando así realizar operaciones adicionales innecesarias. Cuando ambos valores son distintos de cero, el programa normaliza los operandos mediante la función de valor absoluto con el propósito de trabajar únicamente con magnitudes positivas. A continuación se calcula el máximo común divisor utilizando un procedimiento auxiliar basado en el algoritmo de Euclides. Finalmente, el mínimo común múltiplo se obtiene aplicando la relación matemática que vincula el MCM con el MCD, dividiendo el producto de los dos números entre su máximo común divisor. El resultado obtenido se muestra en pantalla como salida final del programa.

```
1      import java.util.Scanner;
2
3      public class MCM {
4
5          public static int calcularMCD(int a, int b) {
```

```
6         int resto;
7         while (b != 0) {
8             resto = a % b;
9             a = b;
10            b = resto;
11        }
12        return a;
13    }
14
15    public static void main(String[] args) {
16
17        Scanner entrada = new Scanner(System.in);
18
19        int a;
20        int b;
21        int mcm;
22        int divisor;
23
24        System.out.print("Digite el primer numero: ");
25        a = entrada.nextInt();
26
27        System.out.print("Digite el segundo numero: ");
28        b = entrada.nextInt();
29
30        if (a == 0 || b == 0) {
31            mcm = 0;
32        } else {
33
34            a = Math.abs(a);
35            b = Math.abs(b);
36
37            divisor = calcularMCD(a, b);
38
39            mcm = (a * b) / divisor;
40        }
41
42        System.out.println("El minimo comun multiplo
```

```
    ↪ es: " + mcm);  
43  
44         entrada.close();  
45     }  
46 }  
47
```

Listing 8.8: Cálculo del mínimo común múltiplo

Con el propósito de evidenciar la correspondencia entre la formulación algorítmica del procedimiento y su implementación en el programa desarrollado en Java, se presenta a continuación una comparación estructurada entre las instrucciones del algoritmo y los fragmentos de código que materializan dichas operaciones dentro del programa. Este tipo de análisis resulta particularmente útil en etapas iniciales del aprendizaje de la programación, ya que permite observar cómo cada paso del razonamiento algorítmico se traduce en una o varias instrucciones concretas del lenguaje de programación. En algunos casos la equivalencia es directa, como ocurre con las operaciones de lectura, asignación o impresión de resultados; en otros, una misma operación conceptual del algoritmo se implementa mediante construcciones propias del lenguaje, como el uso de métodos auxiliares o funciones matemáticas. De esta manera, la comparación presentada en la Tabla 8.6 permite identificar con claridad la relación existente entre el diseño conceptual del algoritmo y su materialización práctica en el programa ejecutable.

Cuadro 8.6: Equivalencia entre el algoritmo *MCM* y su implementación en Java

Instrucción del algoritmo	Código en Java	Explicación
1.1 a, b, mcm: entero	int a; int b; int mcm;	Las variables declaradas en el algoritmo se implementan en Java mediante variables de tipo entero que almacenan los valores de entrada y el resultado final del cálculo.
2. Escribir "Digite el primer numero"	System.out.print("Digite el primer numero: ");	La instrucción de salida del algoritmo se implementa mediante <code>System.out.print()</code> , que muestra el mensaje solicitando el primer número.
3. Leer a	a = entrada.nextInt();	La lectura del primer número se realiza mediante el objeto <code>Scanner</code> y el método <code>nextInt()</code> .
4. Escribir "Digite el segundo numero"	System.out.print("Digite el segundo numero: ");	El programa solicita el segundo número utilizando otra instrucción de salida estándar.
5. Leer b	b = entrada.nextInt();	La lectura del segundo número se realiza mediante el mismo mecanismo de entrada de datos.
6. Si a = 0 o b = 0 Entonces	if (a == 0 b == 0) {	La condición del algoritmo se implementa mediante una sentencia <code>if</code> que evalúa si alguno de los valores ingresados es cero.
6.1 mcm <- 0	mcm = 0;	Cuando uno de los números es cero, el mínimo común múltiplo se establece directamente en cero.
7. SiNo	} else {	Bloque alternativo que se ejecuta cuando ambos números son distintos de cero.
7.1 a <- abs(a)	a = Math.abs(a);	La función de valor absoluto se implementa mediante el método <code>Math.abs()</code> , garantizando que el cálculo posterior se realice con valores positivos.
7.2 b <- abs(b)	b = Math.abs(b);	Se aplica el mismo procedimiento al segundo operando para normalizar los valores de entrada.
7.3 divisor <- MCD(a,b)	divisor = calcularMCD(a, b);	El cálculo del máximo común divisor se realiza mediante un método auxiliar que implementa el algoritmo de Euclides.
7.4 mcm <- a*b/divisor	mcm = (a * b) / divisor;	Se aplica la relación matemática que vincula el mínimo común múltiplo con el máximo común divisor.
8. Escribir "El minimo comun multiplo es", mcm	System.out.println("El minimo comun multiplo es: " + mcm);	El programa muestra el resultado final del cálculo en la salida estándar.
FinAlgoritmo	entrada.close();	El cierre del objeto <code>Scanner</code> libera el recurso asociado al flujo de entrada del programa.

Parte III

Control de Flujo y Gestión de Datos

Capítulo 9

Estructuras de Control en Aplicaciones Informacionales

El desarrollo de aplicaciones informacionales requiere mecanismos que permitan alterar el orden natural de ejecución de las instrucciones para responder a condiciones cambiantes, flujos de entrada heterogéneos y necesidades operativas concretas. Sin estas construcciones, un programa se limitaría a ejecutar instrucciones de forma lineal, lo que resultaría insuficiente para representar validaciones, clasificación de datos, procesamiento de colecciones, navegación entre estados o tratamiento de eventos. En términos formativos, las estructuras de control constituyen uno de los primeros espacios donde el estudiante deja de escribir instrucciones aisladas y comienza a modelar comportamiento algorítmico con intención, criterio y secuencia lógica [15], [24], [75], [87].

En Java, el control del flujo se expresa mediante estructuras sintácticamente explícitas, semánticamente rigurosas y estrechamente ligadas al tipo booleano. Esta relación entre sintaxis, tipos y ejecución favorece un aprendizaje disciplinado, ya que obliga a formular condiciones correctas, delimitar bloques con precisión y explicitar las rutas posibles del algoritmo. Desde la perspectiva de la construcción de software, dicha disciplina beneficia la mantenibilidad y la lectura posterior del código, aspecto especialmente relevante cuando el programa evoluciona, se integra con otros módulos o debe ser comprendido por terceros [35], [51], [114].

En aplicaciones informacionales reales, las estructuras de control aparecen en casi todos los procesos significativos. Un sistema académico las utiliza para validar matrículas, calcular estados de aprobación y restringir accesos; un sistema comercial las emplea para aplicar descuentos, autorizar pagos y clasificar inventarios; un sistema de monitoreo las aprovecha para reaccionar ante umbrales, disparar alertas y filtrar eventos. Aprender a programarlas no debe reducirse a memorizar la forma de `if`, `switch`, `while` o `for`; exige comprender la lógica de decisión, la organización de los bloques y las consecuencias de cada diseño sobre la claridad, la corrección y la extensibilidad del sistema [15], [24], [115].

Además, el dominio de estas estructuras fortalece una habilidad intelectual más amplia: la capacidad de descomponer un problema en estados, decisiones y repeticiones observables. La literatura sobre enseñanza de la programación ha mostrado de manera consistente que las mayores dificultades del principiante no provienen solo de la sintaxis, sino de la comprensión del comportamiento dinámico del programa, del seguimiento del estado y de la relación entre la intención algorítmica y la ejecución efectiva [75], [116], [117]. Por ello, el tratamiento de este capítulo debe ser simultáneamente técnico, pedagógico y aplicado.

9.1. Estructuras de selección

Las estructuras de selección permiten que un programa elija entre dos o más rutas de ejecución a partir del resultado de una evaluación lógica. Su función central consiste en introducir variabilidad controlada en el comportamiento del software. En lugar de ejecutar siempre el mismo conjunto de instrucciones, el programa analiza el estado de sus datos y, con base en

ello, determina la acción apropiada. Este mecanismo es esencial en sistemas informacionales porque la mayor parte de sus operaciones depende de reglas, restricciones, excepciones o clasificaciones [15], [35], [87].

Desde un punto de vista operativo, seleccionar implica comparar, discriminar y decidir. Dicho de otra forma, la estructura no solo pregunta si algo es verdadero o falso, sino que transforma esa respuesta en una consecuencia computacional observable. Cuando una aplicación verifica si un usuario posee permisos suficientes, si un monto supera un límite, si un estudiante cumple un requisito o si un sensor rebasa un umbral, está ejecutando una forma de selección. La calidad de esa selección incide directamente en la confiabilidad funcional del programa [24], [51].

En Java, las estructuras de selección más relevantes en la etapa inicial son `if`, `if-else`, las cadenas `if-else if-else` y `switch`. Aunque todas persiguen el mismo propósito general, no son intercambiables en todos los contextos. `if` ofrece máxima flexibilidad para evaluar expresiones booleanas arbitrarias; `switch`, en cambio, resulta más apropiada cuando se desea seleccionar entre alternativas discretas basadas en el valor de una sola expresión. Saber distinguir cuándo conviene cada una forma parte de la competencia programadora y no constituye un detalle secundario de estilo [35], [71], [87].

La claridad de las decisiones también influye en la legibilidad del código. La investigación sobre comprensión y calidad de código ha mostrado que la organización local del texto fuente afecta la facilidad con que otros desarrolladores interpretan su intención, detectan errores y realizan modificaciones [114]. Por ello, una estructura de selección correctamente elegida no solo resuelve un problema; además comunica con nitidez la lógica del algoritmo.

9.1.1. Selección simple y doble

La selección simple representa la forma más elemental de control condicional. Su propósito consiste en ejecutar un bloque de instrucciones únicamente si una condición es verdadera. Si la condición es falsa, el programa no realiza ninguna acción asociada a ese bloque y continúa con las instrucciones siguientes. Esta forma resulta adecuada cuando el problema exige una reacción positiva ante cierta situación, pero no requiere una respuesta explícita en caso contrario. En validaciones preliminares, verificación de rangos y activación de mensajes informativos, esta estructura suele ser suficiente [35], [87].

La selección doble amplía este patrón incorporando una alternativa formal para el caso en que la condición no se cumpla. En vez de limitarse a decidir si se ejecuta un bloque, el algoritmo define dos caminos mutuamente excluyentes: uno para la verdad y otro para la falsedad. Esta estructura es más expresiva en situaciones donde ambas ramas son significativas, como la determinación del estado de aprobación, la autorización o rechazo de una operación, o la clasificación inicial de un dato en dos categorías opuestas [71], [87].

Didácticamente, la distinción entre ambas formas es decisiva. El principiante suele pensar en la selección como una pregunta simple, pero en la práctica profesional muchas decisiones requieren modelar también la consecuencia del caso negativo. La ausencia de una rama alternativa, cuando esta sí es necesaria, genera algoritmos incompletos, estados silenciosos o comportamientos ambiguos. Por ello, la enseñanza no debe presentar `if` y `if-else` como meras variantes sintácticas, sino como herramientas con intenciones algorítmicas distintas [75], [115].

En lenguajes como Python, la idea algorítmica es la misma, aunque el bloque se determina por indentación; en Java y C++, la delimitación con llaves hace visible la frontera estructural del cuerpo condicional. Esta diferencia sintáctica puede parecer menor, pero pedagógicamente es relevante: Java fuerza a percibir con claridad el bloque como unidad de ejecución, lo que favorece una comprensión más explícita de la agrupación de instrucciones [35].

```
1      int edad = 20;
2
3      if(edad >= 18){
4          System.out.println("El usuario es mayor de edad.");
5      }
```

Listing 9.1: Selección simple: validación de mayoría de edad

En este primer ejemplo, la intención del algoritmo es puntual: solo interesa reaccionar cuando la condición se cumple. No existe una consecuencia operativa definida para el caso contrario. Esta clase de diseño aparece, por ejemplo, cuando una

plataforma agrega una observación a un registro, activa un aviso o habilita una opción sin necesidad de manejar todavía el escenario opuesto.

```
1      int nota = 65;
2
3      if(nota >= 70){
4          System.out.println("Aprobado");
5      }else{
6          System.out.println("Reprobado");
7      }
```

Listing 9.2: Selección doble: evaluación de estado académico

En el segundo caso, ambas ramas tienen valor funcional. El algoritmo no queda completo si solo se define el resultado favorable. Obsérvese que la estructura doble no solo decide: también documenta la política académica que se está implementando. Esa capacidad para volver explícita una regla de negocio es una de las razones por las que las estructuras de selección ocupan un lugar central en el diseño de aplicaciones informacionales [15], [24].

Condición

Toda estructura de selección depende de una condición, es decir, de una expresión cuyo resultado final debe ser **true** o **false**. En Java, este requisito no es opcional: la condición debe ser booleana de manera explícita. Esta exigencia fortalece la seguridad semántica del lenguaje, porque impide ambigüedades comunes en otros entornos donde valores numéricos o referencias pueden interpretarse indirectamente como verdad o falsedad. Desde la formación inicial, esta característica obliga a pensar con rigor qué se está evaluando y por qué [35], [87].

Las condiciones simples se construyen a partir de una sola relación. Suelen apoyarse en operadores relacionales como **<**, **>**, **<=**, **>=**, **==** y **!=**. Aunque su forma es sencilla, su correcta interpretación es crucial. Una comparación mal planteada puede introducir errores funcionales difíciles de detectar si el resultado aparentemente luce razonable. Por esa razón, el aprendizaje de las condiciones no debe restringirse a la enumeración de operadores, sino extenderse a la lectura precisa de la lógica expresada [51], [71].

```
1      int numero = 10;
2
3      if(numero > 0){
4          System.out.println("El número es positivo");
5      }
```

Listing 9.3: Condición simple

Las condiciones compuestas aparecen cuando una sola comparación no basta para representar la regla del problema. En tales casos, se combinan varias proposiciones mediante operadores lógicos como **&&** (conjunción), **||** (disyunción) y **!** (negación). Estas composiciones permiten modelar restricciones simultáneas, alternativas válidas o exclusiones. En un sistema de información, es habitual exigir que un usuario cumpla varias condiciones a la vez, o que satisfaga al menos una de varias posibles credenciales [35], [87].

```
1      int edad = 25;
2      boolean licencia = true;
3
4      if(edad >= 18 && licencia){
5          System.out.println("Puede conducir.");
6      }
```

Listing 9.4: Condición compuesta

También existen evaluaciones directas, donde se consulta una variable booleana sin compararla de nuevo. Este patrón es especialmente importante porque enseña a distinguir entre preguntar por un hecho lógico ya calculado y recalcularlo innecesariamente una condición. Cuando el programa dispone de indicadores como `activo`, `autenticado`, `disponible` o `procesado`, la forma más clara y correcta suele ser evaluarlos directamente [51], [114].

```
1      boolean sistemaActivo = true;
2
3      if(sistemaActivo){
4          System.out.println("El sistema se encuentra operativo.");
5      }
```

Listing 9.5: Evaluación directa de una variable booleana

Desde una perspectiva aplicada, conviene que el estudiante advierta que la dificultad no reside en escribir la condición, sino en formular correctamente la regla que esa condición representa. Muchos errores del principiante no nacen de la sintaxis, sino de confundir la política del problema con una expresión booleana incorrecta o incompleta. De ahí que la práctica deliberada sobre condiciones deba integrar lectura, interpretación, reformulación y prueba con distintos datos de entrada [75], [115], [116].

Bloques alternativos

Los bloques alternativos permiten asociar consecuencias distintas a los resultados opuestos de una evaluación. En vez de dejar implícito lo que sucede cuando una condición falla, la estructura expresa con claridad qué ruta debe seguirse en cada caso. Esta explicitud es pedagógicamente valiosa porque fuerza a pensar el algoritmo como un espacio de decisiones completas y no solo como una secuencia de respuestas parciales [71], [87].

En Java, estos bloques se delimitan mediante llaves, lo que facilita identificar con precisión el alcance de cada rama. Aunque en ejemplos pequeños podría parecer suficiente colocar una sola instrucción sin llaves, en material formativo y en software mantenible resulta preferible conservar siempre la delimitación explícita. Esta decisión reduce errores por edición posterior, evita ambigüedades de lectura y mejora la uniformidad del estilo [51], [114].

```
1      double saldo = 100.0;
2      double retiro = 120.0;
3
4      if(retiro <= saldo){
5          saldo = saldo - retiro;
```

```
6      System.out.println("Retiro realizado.");
7  }else{
8      System.out.println("Fondos insuficientes.");
9  }
```

Listing 9.6: Bloques alternativos en una operación financiera simple

En este ejemplo, la rama verdadera no solo confirma una condición, sino que modifica el estado del sistema; la rama falsa, por su parte, preserva dicho estado e informa la razón de la no ejecución. Esta dualidad ilustra una idea importante en aplicaciones informacionales: las decisiones suelen afectar tanto los datos como la comunicación con el usuario. La selección no es simplemente una bifurcación lógica; es también una política de acción y de respuesta.

Cuando la lógica crece, los bloques alternativos pueden encadenarse mediante `else if`. Esta posibilidad conviene cuando se modelan varios escenarios mutuamente excluyentes ordenados por prioridad. Sin embargo, no debe utilizarse como sustituto automático de cualquier forma de selección múltiple. Si la lógica depende del valor discreto de una misma expresión, suele ser más expresiva una estructura `switch`; si depende de relaciones booleanas heterogéneas, el encadenamiento con `else if` puede ser la opción más adecuada [35], [71].

Anidamiento

El anidamiento aparece cuando una estructura de decisión contiene en su interior otra estructura condicional. Su utilidad radica en que permite representar decisiones jerárquicas: primero se resuelve una evaluación general y, dentro de la rama correspondiente, se plantean nuevas evaluaciones más específicas. Este patrón se ajusta bien a situaciones donde la validez de una segunda pregunta depende del resultado de la primera [35], [87].

En el aprendizaje inicial, el anidamiento cumple una función importante porque obliga a seguir el flujo del programa con atención. El estudiante debe comprender que no todas las condiciones están activas al mismo tiempo; algunas solo se evalúan si una ruta previa las habilita. Esta comprensión dinámica del control es fundamental para evitar interpretaciones erróneas del algoritmo y para desarrollar habilidad de trazado mental o manual [75], [116].

```
1      int edad = 22;
2      boolean estudiante = true;
3
4      if(edad < 30){
5          if(estudiante){
6              System.out.println("Accede al descuento estudiantil.");
7          }else{
8              System.out.println("No posee condición estudiantil.");
9          }
10     }else{
11         System.out.println("No aplica para esta promoción.");
12     }
```

Listing 9.7: Anidamiento de decisiones

El código anterior muestra una jerarquía lógica clara. La primera decisión determina si el solicitante entra al rango etario de análisis; solo dentro de esa rama tiene sentido preguntar por la condición estudiantil. Este orden no es accidental: responde a una economía lógica y funcional. Plantear ambas preguntas sin esa jerarquía haría menos comprensible el algoritmo y podría conducir a mensajes inconsistentes.

No obstante, un uso excesivo de anidamientos deteriora la legibilidad. Cuando las ramas se profundizan demasiado, el lector pierde facilidad para identificar el contexto de cada instrucción. En tales casos, conviene reconsiderar el diseño: a veces es preferible separar la lógica en métodos auxiliares, usar indicadores booleanos intermedios o reestructurar las decisiones para que cada bloque mantenga una complejidad manejable. Esta preocupación por la legibilidad no es estética; se relaciona directamente con la calidad y la mantenibilidad del software [51], [114].

9.1.2. Selección múltiple

Cuando la decisión del algoritmo depende del valor de una sola expresión que puede asumir varias alternativas discretas, la selección múltiple ofrece una forma más ordenada de expresar la lógica. En Java, esta necesidad se resuelve mediante **switch**, estructura que organiza cada alternativa en ramas etiquetadas con **case**. Su principal virtud no consiste solo en abreviar el código, sino en hacer visible que todas las opciones pertenecen a una misma dimensión de decisión [71], [87].

Desde el punto de vista de la lectura, **switch** comunica con rapidez que el programa está seleccionando entre categorías predefinidas. Esta claridad supera, en muchos casos, a una cadena extensa de **if-else if**, particularmente cuando todas las comparaciones se realizan sobre el mismo valor. Halterman señala expresamente que, cuando es aplicable, **switch** suele resultar más legible que un equivalente multinivel con **if/else** y puede incluso permitir al compilador generar código más eficiente [75].

En Java moderno, **switch** puede utilizarse con tipos enteros, **char**, **String** y enumeraciones. Esta amplitud de uso lo vuelve especialmente útil en menús, clasificación de códigos, interpretación de estados, traducción de abreviaturas y despacho de acciones. En aplicaciones informacionales, ello se traduce en decisiones como procesar el rol de un usuario, interpretar el estado de un trámite o clasificar el mes para calcular una regla asociada [15], [87].

```
1      int dia = 3;
2
3      switch(dia){
4          case 1:
5              System.out.println("Lunes");
6              break;
7          case 2:
8              System.out.println("Martes");
9              break;
10         case 3:
11             System.out.println("Miércoles");
12             break;
13         default:
14             System.out.println("Día no válido");
15     }
```

Listing 9.8: Selección múltiple básica

Un elemento didáctico decisivo en esta estructura es comprender que todos los **case** pertenecen al mismo bloque de selección y que el flujo puede continuar hacia casos posteriores si no se interrumpe explícitamente. Esta particularidad hace de **switch** una estructura potente, pero también propensa a errores si no se la entiende con precisión.

Casos

Cada **case** representa un valor específico que, al coincidir con la expresión evaluada, habilita la ejecución de un conjunto de instrucciones. En esencia, los casos son alternativas etiquetadas dentro de una misma estructura de decisión. Su orden puede favorecer la lectura humana, aunque la lógica de selección depende del valor y no de la posición textual. Aun así, organizar los casos de manera coherente mejora la mantenibilidad y la comprensión global del programa [51], [71].

Una técnica particularmente útil consiste en agrupar varios **case** consecutivos que comparten la misma consecuencia. Este patrón evita repetir instrucciones idénticas y expresa con claridad que varios valores pertenecen a una misma categoría lógica. En problemas de clasificación discreta, esta agrupación no es una mera conveniencia sintáctica, sino una forma de modelar taxonomías dentro del algoritmo.

```
1    int mes = 7;
2    int dias;
3
4    switch(mes){
5        case 1: case 3: case 5: case 7:
6        case 8: case 10: case 12:
7            dias = 31;
8            break;
9        case 4: case 6: case 9: case 11:
10           dias = 30;
11           break;
12        case 2:
13           dias = 28;
14           break;
15        default:
16           dias = -1;
17    }
```

Listing 9.9: Agrupación de casos

En términos aplicados, este diseño es valioso porque hace visible la relación entre el dato de entrada y la categoría operativa que le corresponde. El algoritmo no “aprende” meses; refleja una regla del dominio. Por ello, el estudiante debe advertir que escribir un **switch** correcto exige comprender la estructura conceptual del problema y no solo recordar palabras reservadas.

En Java, **switch** no está pensado para intervalos arbitrarios de valores numéricos, como “entre 0 y 10” o “mayor que 100”. Para esas situaciones, **if-else** sigue siendo la herramienta más apropiada. La fortaleza de **switch** reside en la selección por coincidencia de valores discretos. Confundir estos dominios de uso conduce a diseños forzados o poco naturales [35], [87].

Cobertura

La cobertura en una estructura de selección múltiple se refiere al grado en que el conjunto de casos definidos contempla todos los valores relevantes que pueden aparecer durante la ejecución. En programación formativa, este principio es crucial porque enseña a pensar en el algoritmo no solo desde los casos esperados, sino también desde los escenarios faltantes, atípicos o inválidos. Un `switch` aparentemente correcto puede ser funcionalmente incompleto si deja sin tratamiento valores que el sistema sí podría recibir [24], [51].

En este sentido, el bloque `default` desempeña una función de cierre lógico y de defensa operacional. Su presencia no sustituye el diseño cuidadoso de los casos válidos, pero sí ofrece una respuesta controlada cuando aparece una situación imprevista. En sistemas informacionales, esta rama puede utilizarse para informar un error, registrar una incidencia, asignar un valor centinela o evitar que el programa continúe en un estado incoherente.

La cobertura también tiene una dimensión de prueba y verificación. Diseñar bien una estructura de selección implica comprobar qué sucede con entradas típicas, entradas límite y entradas incorrectas. Esta práctica conecta directamente con la calidad del software y con la necesidad de reducir fallos provocados por rutas no consideradas. El currículo contemporáneo de sistemas y programación insiste en la importancia de construir competencias que articulen lógica, prueba y mantenimiento, no solo escritura inicial de código [114], [115].

Desde el punto de vista pedagógico, trabajar cobertura desarrolla una actitud de anticipación. El estudiante deja de pensar únicamente en “qué debería pasar” y empieza a preguntarse también “qué podría pasar y cómo respondería el programa”. Esa transición es una señal clara de madurez algorítmica.

Errores frecuentes

Uno de los errores más comunes en `switch` consiste en omitir `break`. Cuando esto sucede, el flujo continúa hacia el siguiente `case`, fenómeno conocido como *fall-through*. Aunque en algunos contextos esta continuidad puede aprovecharse deliberadamente, en la programación inicial suele ser fuente de resultados erróneos porque el estudiante espera que cada caso actúe como una rama independiente. Comprender este comportamiento es indispensable para leer correctamente la estructura [71].

Otro error habitual consiste en usar `switch` cuando la decisión depende de expresiones relacionales o rangos. Forzar esta estructura en problemas que naturalmente requieren `if-else` genera código artificial, difícil de mantener y más propenso a equivocaciones. La buena programación no consiste en usar la estructura más vistosa, sino la que expresa con mayor fidelidad la lógica del problema [35], [51].

También son frecuentes los errores de comparación en estructuras `if`. En Java, los tipos primitivos se comparan con `==`, pero los objetos, como las cadenas, deben compararse semánticamente mediante `equals`. Ignorar esta diferencia produce fallos funcionales sutiles que el principiante no siempre detecta de inmediato. Dado que gran parte de las decisiones en sistemas informacionales involucra texto, estados codificados o identificadores, esta distinción merece tratamiento explícito en la formación inicial.

Finalmente, las condiciones excesivamente largas y poco estructuradas reducen la comprensibilidad del algoritmo. Cuando una sola línea acumula muchas negaciones, conjunciones y disyunciones, el problema deja de ser sintáctico y pasa a ser cognitivo: el lector ya no puede verificar con facilidad qué regla se está implementando. La investigación en legibilidad del código advierte precisamente que la comprensión humana del programa constituye un componente central de su calidad [114]. En tales casos, conviene introducir variables booleanas intermedias o dividir la decisión en pasos más transparentes.

9.2. Estructuras de repetición

Las estructuras de repetición permiten ejecutar un conjunto de instrucciones varias veces bajo un criterio de continuidad. Su función es fundamental porque muchos problemas computacionales no consisten en decidir una sola vez, sino en procesar secuencias de datos, recorrer colecciones, acumular resultados, validar entradas reiteradamente o explorar combinaciones posibles. Sin repetición, el software tendría que expresar manualmente acciones idénticas una y otra vez, lo que volvería los programas más largos, frágiles y difíciles de modificar [35], [87].

En Java, las formas más representativas de repetición inicial son `while`, `do-while` y `for`. Aunque las tres permiten reiterar instrucciones, difieren en la manera en que ubican la condición de continuación y en el tipo de problema para el que resultan más expresivas. Elegir bien entre ellas no es una cuestión superficial de preferencia; es una decisión de diseño que influye en la legibilidad, la trazabilidad de la lógica y la facilidad de mantenimiento [71].

En aplicaciones informacionales, la repetición se manifiesta en tareas como leer lotes de registros, sumar transacciones, revisar resultados de evaluación, recorrer menús hasta que el usuario solicite salir o examinar estructuras de datos. La competencia para programar ciclos no solo requiere conocer su sintaxis, sino comprender el estado que cambia en cada iteración, la condición que sostiene el proceso y el mecanismo que eventualmente lo detiene. Una proporción considerable de los problemas de aprendizaje en programación inicial se relaciona precisamente con la dificultad para seguir el estado cambiante dentro de los ciclos [75], [116], [117].

9.2.1. Repetición condicional

La repetición condicional se caracteriza porque la continuación del ciclo depende de una expresión lógica evaluada durante la ejecución. A diferencia de los ciclos donde el número de repeticiones se conoce de antemano, aquí el algoritmo persiste mientras la condición siga siendo verdadera. Esta forma resulta apropiada cuando el final del proceso depende del estado del sistema, de la entrada del usuario o de la aparición de un evento particular [71], [87].

En Java, este enfoque se materializa principalmente mediante `while` y `do-while`. Ambas estructuras comparten el mismo principio: repetir un bloque según una condición. Sin embargo, difieren en un aspecto crucial para el diseño: el momento en que se evalúa dicha condición. Esa diferencia determina si el cuerpo puede no ejecutarse nunca o si, por el contrario, se ejecutará al menos una vez antes de verificar la continuidad.

Desde el aprendizaje, esta distinción ayuda a desarrollar sensibilidad algorítmica. No basta con saber que dos estructuras “repiten”; es necesario reconocer qué garantías ofrece cada una respecto de la primera ejecución y qué tipo de situación modela mejor. Ese razonamiento es parte esencial de la formación en programación y de la capacidad de transferir conceptos entre lenguajes como Java, C++, C# o incluso Python, donde los patrones lógicos son análogos aunque su sintaxis difiera [35], [115].

```
1    int contador = 1;
2
3    while(contador <= 5){
4        System.out.println(contador);
5        contador++;
6    }
```

Listing 9.10: Repetición condicional con while

Precondición

La estructura `while` es una repetición de precondición porque evalúa primero la expresión lógica y solo después, si esta resulta verdadera, ejecuta el bloque. La consecuencia inmediata de este diseño es que el cuerpo puede no ejecutarse ninguna vez. Esta propiedad lo vuelve especialmente apropiado cuando el algoritmo solo debe iniciar si existe un estado previo válido que lo justifique [35], [87].

En aplicaciones informacionales, este patrón aparece cuando se procesan datos mientras haya registros disponibles, cuando se recorre una colección mientras el índice permanezca dentro del rango o cuando se ejecuta una validación mientras aún no se cumpla un criterio aceptable. La condición no es un adorno sintáctico: expresa el contrato de entrada que autoriza cada nueva iteración. Si dicho contrato deja de cumplirse, el ciclo debe terminar.

```
1    int numero = 1;
2
3    while(numero <= 3){
```

```
4      System.out.println(numero);  
5      numero++;  
6  }
```

Listing 9.11: Precondición con while

Una ventaja pedagógica de **while** es que hace visible la relación entre estado y continuidad. El estudiante puede observar con nitidez que el ciclo depende de una variable cuyo valor cambia en cada vuelta. Sin embargo, justamente por ello, también es una fuente frecuente de errores: olvidar actualizar la variable de control o modificarla incorrectamente conduce a ciclos infinitos o a terminaciones prematuras [75], [116].

Postcondición

La estructura **do-while** desplaza la evaluación de la condición al final del bloque. Por esta razón, garantiza al menos una ejecución, aun cuando la condición termine siendo falsa inmediatamente después. Esta característica la vuelve idónea para modelar procesos donde la primera acción debe ocurrir antes de decidir si conviene repetirla, como la presentación inicial de un menú, la solicitud de una entrada o la ejecución de un intento de validación [71], [87].

Desde una perspectiva de diseño, **do-while** no es simplemente “otro ciclo”. Su semántica expresa un compromiso algorítmico diferente: primero actuar, luego verificar continuidad. En programación básica, conviene que el estudiante perciba que esta estructura no se elige por variedad, sino porque representa fielmente ciertos problemas. Si se utilizara **while** en los mismos contextos, sería necesario duplicar alguna acción antes del ciclo o reestructurar el algoritmo.

```
1      int opcion;  
2  
3      do{  
4          System.out.println("Ingrese una opción válida");  
5          opcion = 1;  
6      }while(opcion != 1);
```

Listing 9.12: Postcondición con do-while

Aplicativamente, esta estructura es muy útil en menús iterativos, autenticación con reintentos y captura de datos obligatoria. En esos escenarios, la primera interacción no depende de una evaluación previa, porque precisamente es esa interacción la que produce la información necesaria para decidir si el proceso debe continuar.

Condiciones de parada

Todo ciclo correcto necesita una condición de parada comprensible, alcanzable y verificable. La claridad de esa condición separa un algoritmo confiable de uno potencialmente defectuoso. En términos simples, repetir no basta; es indispensable saber qué hecho o qué cambio de estado hará que la repetición termine. Esta preocupación es central tanto en programación formativa como en ingeniería de software, porque muchos defectos graves se originan en ciclos mal controlados [24], [51].

Una condición de parada suele depender de una variable de control, de una señal de entrada, del agotamiento de una colección o de la detección de un criterio de suficiencia. Lo importante es que el algoritmo modifique el estado relevante de forma consistente para que la condición pueda cambiar. Si nada altera el estado que sostiene la repetición, el programa queda atrapado en un ciclo infinito. Halterman muestra que la actualización del control debe estar claramente localizada y no dispersa, pues la dispersión dificulta comprender cómo finaliza realmente el ciclo [51], [116].

Desde el punto de vista pedagógico, conviene enseñar a identificar tres preguntas en cada ciclo: qué condición permite seguir, qué variable o estado cambia en cada iteración, y qué hecho concreto provoca la salida. Si estas tres preguntas no pueden responderse con facilidad al leer el código, el diseño del ciclo probablemente necesita revisión. Este hábito de lectura fortalece la depuración y reduce errores típicos del principiante [75], [115].

9.2.2. Repetición por contador

La repetición por contador se utiliza cuando el número de iteraciones es conocido o puede determinarse de manera directa antes de iniciar el ciclo. En Java, esta forma se expresa habitualmente mediante `for`, estructura que concentra en un solo encabezado la inicialización del contador, la condición de continuidad y la actualización del control. Esta concentración hace explícita la mecánica del ciclo y mejora su legibilidad cuando la repetición sigue un patrón regular [71], [87].

Una ventaja importante de `for` es organizativa. En lugar de repartir la lógica de control entre varias líneas del cuerpo, el ciclo presenta en un solo punto la información esencial que determina su comportamiento. Halterman destaca precisamente que esta agrupación ayuda a evitar olvidos, como la omisión de la actualización del contador, y que cualquier `for` puede reescribirse como `while`, aunque `for` suele ofrecer una organización superior cuando el problema es naturalmente contable [24].

Esta estructura es particularmente adecuada para recorrer posiciones de arreglos, repetir un número fijo de veces una operación, construir tablas, procesar secuencias indexadas o generar acumulaciones controladas. En sistemas informacionales, ello incluye recorrer listas de productos, iterar sobre estudiantes matriculados o calcular resúmenes sobre un conjunto ya delimitado de registros.

```
1   for (int i = 1; i <= 5; i++) {  
2       System.out.println(i);  
3   }
```

Listing 9.13: Repetición por contador con `for`

A diferencia de la repetición condicional abierta, la repetición por contador tiende a ofrecer mayor previsibilidad. No obstante, ello no elimina la necesidad de análisis. Un límite mal formulado, una actualización incorrecta o un valor inicial inadecuado bastan para producir errores de una iteración de más, de menos o incluso accesos fuera de rango. La aparente simplicidad de `for` no debe hacer olvidar que su corrección también depende de una formulación lógica cuidadosa [35], [51].

Contadores

Un contador es una variable cuyo valor cambia de manera regular para representar cuántas veces se ha ejecutado una repetición o en qué posición del proceso se encuentra el algoritmo. En Java, normalmente es entero porque su función principal es enumerar iteraciones, índices o pasos discretos. Su uso va mucho más allá del operador `++`; constituye el eje temporal o posicional del ciclo [71], [87].

Las formas más habituales de actualización incluyen `c = c + 1`, `c++`, `c--`, `c += valor` y `c -= valor`. Elegir una u otra depende de la intención del algoritmo. Incrementar de uno en uno es apropiado cuando se recorre una secuencia completa; avanzar de dos en dos puede servir para procesar posiciones pares; decrementar puede utilizarse en cuentas regresivas o recorridos inversos. Lo pedagógicamente importante es que el estudiante comprenda que el contador no se “mueve” por sí solo: su actualización expresa la forma en que el programa recorre el espacio del problema.

```
1   for (int i = 0; i < 10; i++) {  
2       System.out.println(i);  
3   }  
4
```

```
5     for (int j = 10; j >= 0; j--) {  
6         System.out.println(j);  
7     }
```

Listing 9.14: Contador con distintos patrones de actualización

Una buena práctica consiste en que la variable de control refleje con claridad su propósito. Aunque *i* es tradicional y aceptable en recorridos breves o indexados, en muchos contextos nombres como *indice*, *fila*, *mes* o *posicion* mejoran la lectura del algoritmo. La legibilidad del código depende también de estas decisiones locales de denominación [51], [114].

Acumuladores

Un acumulador es una variable que conserva y actualiza progresivamente un resultado parcial dentro de un ciclo. A diferencia del contador, cuya misión principal es controlar la repetición, el acumulador representa el valor que el algoritmo va construyendo con cada iteración: una suma, un producto, un máximo, un mínimo, una concatenación o cualquier otro resultado incremental [35], [87].

En problemas informacionales, los acumuladores son imprescindibles. Permiten sumar ventas, totalizar créditos, contar aprobados mediante incrementos condicionados, calcular promedios cuando se combinan con contadores y construir indicadores derivados a partir de registros. La comprensión de esta idea transforma el modo en que el estudiante percibe los ciclos: ya no se repite solo para “hacer muchas veces lo mismo”, sino para construir progresivamente información útil.

```
1     int suma = 0;  
2  
3     for(int i=1;i<=5;i++){  
4         suma += i;  
5     }  
6  
7     System.out.println(suma);
```

Listing 9.15: Acumulador de suma

Didácticamente, uno de los puntos críticos consiste en elegir correctamente el valor inicial del acumulador. Una suma debe empezar en cero; un producto, en uno; la búsqueda de un máximo o mínimo puede requerir inicialización con el primer dato disponible. Una elección inicial incorrecta puede invalidar todo el resultado aunque el ciclo parezca funcionar. Por eso, el estudio de acumuladores debe incluir no solo la actualización, sino también la justificación de su inicialización [75], [116].

Iteraciones

Se denomina iteración a cada ejecución completa del cuerpo de un ciclo. Comprender este concepto es indispensable porque permite seguir el comportamiento dinámico del programa paso a paso. Cada iteración parte de un estado, ejecuta instrucciones, modifica variables y conduce a un nuevo estado desde el cual el ciclo puede continuar o finalizar. Esta visión secuencial resulta esencial para depurar, verificar y explicar algoritmos [35], [87].

En la práctica, pensar por iteraciones facilita analizar cómo evolucionan contadores, acumuladores y condiciones de parada. Cuando un estudiante logra describir qué ocurre en la primera, segunda y tercera iteración de un ciclo, suele estar en condiciones de entender el patrón general del algoritmo. Esta capacidad de trazado es uno de los pilares cognitivos del aprendizaje de la programación y uno de los ámbitos donde con mayor frecuencia se observan dificultades iniciales [75], [116], [117].

Aplicativamente, las iteraciones son la base del procesamiento masivo de datos. Cada fila de una tabla, cada elemento de una lista y cada registro de una colección puede asociarse con una iteración distinta. Por ello, dominar la noción de iteración no solo ayuda a aprender ciclos, sino que prepara al lector para estructuras de datos, procesamiento de archivos, consultas y análisis de información más avanzados. En esa progresión formativa, comprender el ciclo como una sucesión de estados controlados representa un paso decisivo hacia el dominio de la programación.

9.3. Ejercicios integradores de estructuras de control

9.3.1. Ejercicio integrador: control de calificaciones en una aplicación informacional

El problema que se propone consiste en desarrollar un programa que simule un sistema básico de gestión de calificaciones en un contexto académico. La aplicación debe permitir registrar un conjunto de notas, validarlas rigurosamente, clasificarlas según distintos criterios y generar indicadores relevantes como el promedio general, la cantidad de estudiantes aprobados, reprobados y aquellos con desempeño destacado. Además, el sistema debe ofrecer un menú interactivo que permita consultar los resultados procesados de manera estructurada. Este escenario no solo representa una situación común en aplicaciones educativas, sino que también refleja procesos típicos de los sistemas de información, donde los datos deben ser capturados, transformados y analizados para apoyar la toma de decisiones [54], [118]. En este sentido, el problema integra múltiples decisiones, validaciones y repeticiones, por lo que constituye un caso adecuado para aplicar de manera conjunta las estructuras de control estudiadas en este capítulo dentro de un contexto informacional realista y significativo.

El Listado de código 9.16 presenta una solución completa implementada en Java dentro de una única función `main`. El código ha sido cuidadosamente estructurado y documentado con comentarios que evidencian el uso explícito de estructuras de selección simple y múltiple, condiciones simples y compuestas, anidamiento de decisiones, ciclos de repetición (`while`, `do-while` y `for`), así como el empleo sistemático de contadores y acumuladores. Cada sección del programa responde a una intención algorítmica claramente identificable, lo que no solo permite ejecutar correctamente la solución, sino también comprender cómo se organizan y articulan las distintas estructuras de control en una aplicación concreta. Este enfoque resulta coherente con las prácticas recomendadas en ingeniería de software, donde la claridad estructural y la legibilidad del código son factores clave para su mantenimiento, evolución y refactorización futura [68], [80].

```
1      import java.util.Scanner;
2
3      public class ControlCalificacionesIntegrado {
4          public static void main(String[] args) {
5              // =====
6              // EJERCICIO INTEGRADOR DE ESTRUCTURAS DE CONTROL
7              // -----
8              // Este programa integra:
9              // 1. Selección simple
10             // 2. Selección doble
11             // 3. Condiciones simples y compuestas
12             // 4. Evaluación directa de variables booleanas
13             // 5. Anidamiento de decisiones
14             // 6. Selección múltiple con switch
```

```

15      // 7. Repetición condicional con while
16      // 8. Repetición con postcondición mediante do-while
17      // 9. Repetición por contador con for
18      // 10. Contadores y acumuladores
19      // 11. Menú de opciones en una aplicación informacional
20      // =====
21
22      Scanner entrada = new Scanner(System.in);
23
24      // -----
25      // DECLARACIÓN DE VARIABLES GENERALES DEL PROGRAMA
26      // -----
27      int cantidadEstudiantes;    // Cantidad total de estudiantes a
↪ procesar
28      int contador = 0;           // Contador principal del ciclo while
29      int aprobados = 0;          // Contador de estudiantes aprobados
30      int reprobados = 0;         // Contador de estudiantes reprobados
31      int excelentes = 0;         // Contador de estudiantes con nota
↪ excelente
32      int notasAltas = 0;         // Contador de notas altas (85 o más)
33      double sumaNotas = 0;       // Acumulador para sumar todas las
↪ notas
34      double promedio = 0;        // Promedio general del grupo
35      double nota;                // Nota individual ingresada
36      boolean datosProcesados = false; // Indica si ya se ingresaron
↪ y procesaron datos
37      boolean salir = false;      // Controla la salida del
↪ menú principal
38
39
40      ↪ System.out.println("=====");
      System.out.println(" SISTEMA BÁSICO DE CONTROL DE
↪ CALIFICACIONES ");
41
42      ↪ System.out.println("=====");
43      // -----

```

```
44         // DO-WHILE
45         // Se usa una repetición de postcondición porque se desea
46         // pedir al menos una vez la cantidad de estudiantes.
47         // Además, se valida que la cantidad sea mayor que cero.
48         // -----
49         do {
50             System.out.print("Ingrese la cantidad de estudiantes: ");
51             cantidadEstudiantes = entrada.nextInt();
52
53             // SELECCIÓN SIMPLE:
54             // Solo actúa si la cantidad es inválida.
55             if (cantidadEstudiantes <= 0) {
56                 System.out.println("Error: la cantidad debe ser mayor
↪ que cero.");
57             }
58
59         } while (cantidadEstudiantes <= 0);
60
61         // -----
62         // WHILE
63         // Repetición condicional para ingresar las notas de todos
64         // los estudiantes. Se repite mientras el contador sea menor
65         // que la cantidad total de estudiantes.
66         // -----
67         while (contador < cantidadEstudiantes) {
68             System.out.print("Ingrese la nota del estudiante " +
↪ (contador + 1) + " (0 a 100): ");
69             nota = entrada.nextDouble();
70
71             // CONDICIÓN COMPUESTA:
72             // Se valida que la nota esté en un rango permitido.
73             if (nota < 0 || nota > 100) {
74                 System.out.println("Nota inválida. Debe estar entre 0 y
↪ 100.");
75                 continue; // Vuelve a pedir la misma nota sin avanzar
↪ el contador
76             }
```

```
77
78         // ACUMULADOR:
79         // Se suma cada nota válida para luego calcular el promedio.
80         sumaNotas += nota;
81
82         // SELECCIÓN DOBLE:
83         // Clasifica la nota en aprobada o reprobada.
84         if (nota >= 70) {
85             aprobados++;
86         } else {
87             reprobados++;
88         }
89
90         // SELECCIÓN SIMPLE:
91         // Detecta el caso particular de nota perfecta.
92         if (nota == 100) {
93             System.out.println("Observación: calificación
↪ perfecta.");
94         }
95
96         // CONDICIÓN COMPUESTA:
97         // Cuenta cuántos estudiantes tienen excelencia.
98         if (nota >= 90 && nota <= 100) {
99             excelentes++;
100         }
101
102         // EVALUACIÓN DIRECTA DE BOOLEANO:
103         // La condición se calcula primero y luego se evalúa
↪ directamente.
104         boolean esNotaAlta = nota >= 85;
105         if (esNotaAlta) {
106             notasAltas++;
107         }
108
109         // ANIDAMIENTO:
110         // Se realiza una clasificación más detallada según la nota.
111         if (nota >= 70) {
```

```
112         if (nota >= 90) {
113             System.out.println("Clasificación: aprobado con
↪ excelencia.");
114         } else {
115             System.out.println("Clasificación: aprobado.");
116         }
117     } else {
118         if (nota >= 50) {
119             System.out.println("Clasificación: reprobado, pero
↪ recuperable.");
120         } else {
121             System.out.println("Clasificación: reprobado con
↪ bajo rendimiento.");
122         }
123     }
124
125     // CONTADOR:
126     // Avanza al siguiente estudiante.
127     contador++;
128 }
129
130 // -----
131 // CÁLCULO FINAL DEL PROMEDIO
132 // -----
133 promedio = sumaNotas / cantidadEstudiantes;
134 datosProcesados = true;
135
136 // -----
137 // DO-WHILE + SWITCH
138 // Menú principal de consulta. El menú debe mostrarse
139 // al menos una vez y repetirse hasta que el usuario
140 // seleccione la opción de salida.
141 // -----
142 do {
143     System.out.println("\n===== MENÚ DE CONSULTA
↪ =====");
144     System.out.println("1. Mostrar cantidad de aprobados");
```

```

145         System.out.println("2. Mostrar cantidad de reprobados");
146         System.out.println("3. Mostrar promedio general");
147         System.out.println("4. Mostrar cantidad de excelentes");
148         System.out.println("5. Mostrar cantidad de notas altas");
149         System.out.println("6. Mostrar resumen final");
150         System.out.println("7. Mostrar tabla de números del 1 al 5
↪ ");
151         System.out.println("8. Salir");
152         System.out.print("Seleccione una opción: ");
153
154         int opcion = entrada.nextInt();
155
156         // SWITCH:
157         // Selección múltiple para procesar la opción elegida.
158         switch (opcion) {
159             case 1:
160                 // EVALUACIÓN DIRECTA DE BOOLEANO:
161                 if (datosProcesados) {
162                     System.out.println("Aprobados: " + aprobados);
163                 }
164                 break;
165             case 2:
166                 if (datosProcesados) {
167                     System.out.println("Reprobados: " + reprobados);
168                 }
169                 break;
170             case 3:
171                 if (datosProcesados) {
172                     System.out.printf("Promedio general: %.2f%n",
↪ promedio);
173                 }
174                 break;
175             case 4:
176                 if (datosProcesados) {
177                     System.out.println("Excelentes (90 a 100): " +
↪ excelentes);
178                 }

```



```
179         break;
180     case 5:
181         if (datosProcesados) {
182             System.out.println("Notas altas (85 o más): " +
↪ notasAltas);
183         }
184         break;
185     case 6:
186         if (datosProcesados) {
187             System.out.println("\n===== RESUMEN FINAL
↪ =====");
188             System.out.println("Cantidad de estudiantes: "
↪ + cantidadEstudiantes);
189             System.out.println("Aprobados: " + aprobados);
190             System.out.println("Reprobados: " + reprobados);
191             System.out.println("Excelentes: " + excelentes);
192             System.out.println("Notas altas: " +
↪ notasAltas);
193             System.out.printf("Promedio general: %.2f%n",
↪ promedio);
194
195             // -----
196             // IF - ELSE IF - ELSE
197             // Clasifica el promedio general del grupo.
198             // -----
199             int categoriaPromedio;
200
201             if (promedio >= 90) {
202                 categoriaPromedio = 1;
203             } else if (promedio >= 80) {
204                 categoriaPromedio = 2;
205             } else if (promedio >= 70) {
206                 categoriaPromedio = 3;
207             } else if (promedio >= 60) {
208                 categoriaPromedio = 4;
209             } else {
210                 categoriaPromedio = 5;
```

```
211         }
212
213         // SWITCH adicional:
214         // Interpreta la categoría del promedio.
215         switch (categoriaPromedio) {
216             case 1:
217                 System.out.println("Interpretación del
↪ grupo: rendimiento sobresaliente.");
218                 break;
219             case 2:
220                 System.out.println("Interpretación del
↪ grupo: rendimiento muy bueno.");
221                 break;
222             case 3:
223                 System.out.println("Interpretación del
↪ grupo: rendimiento aceptable.");
224                 break;
225             case 4:
226                 System.out.println("Interpretación del
↪ grupo: rendimiento en riesgo.");
227                 break;
228             default:
229                 System.out.println("Interpretación del
↪ grupo: rendimiento deficiente.");
230         }
231     }
232     break;
233 case 7:
234     System.out.println("Tabla de números del 1 al 5:");
235     // FOR:
236     // Repetición por contador con número conocido de
↪ iteraciones.
237     for (int i = 1; i <= 5; i++) {
238         System.out.println("Iteración " + i + ": valor
↪ = " + i);
239     }
240     break;
```

```
241         case 8:
242             salir = true;
243             System.out.println("Fin del programa.");
244             break;
245         default:
246             System.out.println("Opción no válida.");
247     }
248
249     } while (!salir);
250
251     entrada.close();
252 }
253 }
```

Listing 9.16: Ejercicio integrador en una sola función `main`

La solución integra, dentro de una sola función `main`, los principales mecanismos de control estudiados en este capítulo. Su planteamiento se inspira en un escenario académico sencillo, pero suficientemente rico desde el punto de vista algorítmico: registrar calificaciones, validarlas, clasificarlas, acumular resultados y ofrecer un menú de consulta. De este modo, el problema no se limita a ilustrar la sintaxis de un lenguaje, sino que muestra cómo las estructuras de control cooperan para resolver una necesidad propia de una aplicación informacional, donde los datos son procesados de forma dinámica y contextualizada [118].

Desde la perspectiva didáctica, este ejercicio resulta especialmente útil porque reúne en un mismo programa la selección simple, la selección doble, las condiciones compuestas, la evaluación directa de variables booleanas, el anidamiento, la selección múltiple con `switch`, la repetición condicional con `while` y `do-while`, la repetición por contador con `for`, así como el uso de contadores, acumuladores e iteraciones. En consecuencia, permite que el lector observe que estas estructuras no deben aprenderse como piezas aisladas, sino como recursos complementarios que modelan decisiones, repeticiones y estados dentro de un sistema computacional más amplio, en línea con los principios fundamentales de diseño algorítmico [68].

También conviene destacar que el programa se mantiene deliberadamente en una sola función `main`. Esta decisión no responde a una recomendación de diseño para sistemas de gran escala, sino a una intención pedagógica: facilitar que el estudiante principiante pueda seguir la continuidad del flujo completo sin fragmentarlo todavía en métodos. Esta estrategia didáctica permite concentrar la atención en la lógica del control de flujo antes de introducir conceptos de modularización, los cuales pueden abordarse posteriormente mediante procesos de refactorización progresiva del código [80].

Este programa comienza utilizando una repetición de postcondición para garantizar que el usuario ingrese una cantidad válida de estudiantes. Esta primera fase resulta especialmente relevante, ya que muestra que, antes de procesar información, una aplicación debe verificar la validez de sus datos de entrada. En otras palabras, el programa no asume que el usuario proporcionará siempre valores correctos, sino que incorpora una validación explícita mediante estructuras de control, lo que contribuye a la robustez del sistema.

Posteriormente, se emplea una repetición condicional con `while` para capturar las calificaciones. En esta etapa aparecen los contadores y acumuladores como elementos fundamentales del procesamiento iterativo. Los contadores permiten registrar cuántos estudiantes pertenecen a determinadas categorías, mientras que el acumulador reúne la suma total de notas para calcular posteriormente el promedio. Esta combinación constituye una de las aplicaciones más frecuentes de los ciclos en sistemas informacionales: recorrer conjuntos de datos y construir resultados agregados a partir de cada iteración.

A lo largo del ingreso de notas, el programa integra diversas formas de selección. La validación del rango permitido utiliza una condición compuesta; la clasificación entre aprobado y reprobado emplea una selección doble; la identificación de excelencia y notas altas incorpora condiciones adicionales; y la clasificación detallada del rendimiento se desarrolla mediante anidamiento. Este conjunto de decisiones muestra que, en aplicaciones reales, la lógica de control rara vez se limita a una

única comparación, sino que suele involucrar múltiples niveles de análisis sobre un mismo dato.

Una vez procesadas las calificaciones, el programa activa un menú repetitivo controlado mediante **do-while** y gestionado con **switch**. Esta combinación resulta particularmente ilustrativa, ya que evidencia el uso de la selección múltiple en un contexto de interacción con el usuario. Cada opción del menú representa un caso discreto, lo que demuestra que **switch** es especialmente adecuado cuando una única variable determina una acción entre varias alternativas posibles.

El ejercicio también incorpora un segundo **switch** para interpretar la categoría del promedio general del grupo. Esta decisión refuerza la idea de que la selección múltiple no solo es útil para la navegación en menús, sino también para traducir valores discretos en interpretaciones semánticas dentro del dominio del problema. Finalmente, el uso del ciclo **for** en una de las opciones añade una demostración clara de repetición por contador, completando así la integración de los mecanismos fundamentales de control estudiados en el capítulo.

El valor principal de este ejercicio reside en que no presenta las estructuras de control como elementos aislados, sino como componentes interrelacionados dentro de un mismo flujo algorítmico, apoyando a los principiantes de la programación. Esta integración favorece una comprensión más profunda del comportamiento dinámico de los programas y prepara al estudiante para enfrentar problemas más complejos en el desarrollo de software.

9.3.2. Ejercicio integrador: análisis de rendimientos en una aplicación financiera

El problema que se propone consiste en desarrollar un programa que simule un sistema básico de análisis de rendimientos financieros en un contexto organizacional. La aplicación debe permitir registrar una serie de valores numéricos que representen rendimientos porcentuales de un conjunto de periodos, validarlos dentro de un rango razonable, clasificarlos según distintos criterios de desempeño y generar indicadores relevantes para la interpretación del comportamiento financiero. Entre dichos indicadores deben incluirse el promedio general de rendimiento, la cantidad de periodos con ganancia, con pérdida y con estabilidad, así como la identificación de rendimientos sobresalientes o críticos. Además, el sistema debe ofrecer un menú interactivo que permita consultar los resultados procesados de manera estructurada. Este escenario resulta especialmente pertinente porque refleja procesos habituales en aplicaciones financieras y estadísticas, donde los datos deben ser capturados, depurados, resumidos e interpretados para apoyar decisiones de control, seguimiento y evaluación [54], [118].

El Listado de código 9.17 presenta una solución completa implementada en Java dentro de una única función **main**. El código ha sido organizado y comentado para evidenciar el uso explícito de estructuras de selección simple y múltiple, condiciones simples y compuestas, anidamiento de decisiones, ciclos de repetición (**while**, **do-while** y **for**), así como el empleo sistemático de contadores, acumuladores y variables auxiliares para el cálculo estadístico. Cada bloque del programa responde a una intención algorítmica claramente identificable, lo que permite no solo ejecutar correctamente la solución, sino también comprender cómo se articulan las estructuras de control dentro de una aplicación financiera de complejidad inicial. Este enfoque resulta coherente con principios de diseño algorítmico y con prácticas de construcción de software donde la claridad estructural y la legibilidad del código constituyen factores esenciales para su comprensión y mantenimiento [68], [80].

```

1      import java.util.Scanner;
2
3      public class AnalisisRendimientosIntegrado {
4          public static void main(String[] args) {
5              // =====
6              // EJERCICIO INTEGRADOR DE ESTRUCTURAS DE CONTROL
7              // ORIENTADO A FINANZAS Y ESTADÍSTICA
8              // -----
9              // Este programa integra:
10             // 1. Selección simple

```

```

11      // 2. Selección doble
12      // 3. Condiciones simples y compuestas
13      // 4. Evaluación directa de variables booleanas
14      // 5. Anidamiento de decisiones
15      // 6. Selección múltiple con switch
16      // 7. Repetición condicional con while
17      // 8. Repetición con postcondición mediante do-while
18      // 9. Repetición por contador con for
19      // 10. Contadores, acumuladores, máximo y mínimo
20      // 11. Menú de consulta en una aplicación financiera
21      // =====
22
23      Scanner entrada = new Scanner(System.in);
24
25      // -----
26      // DECLARACIÓN DE VARIABLES GENERALES
27      // -----
28      int cantidadPeriodos;      // Total de periodos a analizar
29      int contador = 0;           // Control principal del ciclo while
30      int ganancias = 0;          // Cuenta periodos con rendimiento
    ↪ positivo
31      int perdidas = 0;           // Cuenta periodos con rendimiento
    ↪ negativo
32      int estables = 0;           // Cuenta periodos con rendimiento
    ↪ igual a cero
33      int sobresalientes = 0;     // Rendimientos muy altos
34      int criticos = 0;           // Rendimientos muy bajos
35      double rendimiento;         // Valor de rendimiento ingresado
36      double sumaRendimientos = 0; // Acumulador para el promedio
37      double promedio = 0;        // Promedio general de rendimientos
38      double maximo = 0;          // Mayor rendimiento registrado
39      double minimo = 0;          // Menor rendimiento registrado
40      boolean primerDato = true;  // Permite inicializar máximo y
    ↪ mínimo
41      boolean datosProcesados = false; // Indica si ya se procesaron
    ↪ datos
42      boolean salir = false;      // Controla la salida del menú

```

```

43
44
↳ System.out.println("=====");
45     System.out.println(" SISTEMA BÁSICO DE ANÁLISIS DE RENDIMIENTOS
↳ FINANCIEROS ");
46
↳ System.out.println("=====");
47
48     // -----
49     // DO-WHILE
50     // Solicita al menos una vez la cantidad de periodos.
51     // Se valida que sea un entero positivo.
52     // -----
53     do {
54         System.out.print("Ingrese la cantidad de periodos a
↳ analizar: ");
55         cantidadPeriodos = entrada.nextInt();
56
57         // SELECCIÓN SIMPLE:
58         // Solo reacciona cuando la cantidad es inválida.
59         if (cantidadPeriodos <= 0) {
60             System.out.println("Error: la cantidad debe ser mayor
↳ que cero.");
61         }
62
63     } while (cantidadPeriodos <= 0);
64
65     // -----
66     // WHILE
67     // Permite ingresar los rendimientos de todos los periodos.
68     // -----
69     while (contador < cantidadPeriodos) {
70         System.out.print("Ingrese el rendimiento del periodo " +
↳ (contador + 1) + " (entre -100 y 100): ");
71         rendimiento = entrada.nextDouble();
72
73         // CONDICIÓN COMPUESTA:

```

```
74          // Se valida que el rendimiento esté dentro de un rango
    ↪ razonable.
75          if (rendimiento < -100 || rendimiento > 100) {
76              System.out.println("Dato inválido. El rendimiento debe
    ↪ estar entre -100 y 100.");
77              continue; // Repite el mismo periodo sin aumentar el
    ↪ contador
78          }
79
80          // ACUMULADOR:
81          // Se suma el rendimiento válido para calcular el promedio.
82          sumaRendimientos += rendimiento;
83
84          // INICIALIZACIÓN DE MÁXIMO Y MÍNIMO CON EL PRIMER DATO
85          if (primerDato) {
86              maximo = rendimiento;
87              minimo = rendimiento;
88              primerDato = false;
89          }
90
91          // SELECCIÓN SIMPLE:
92          // Actualiza máximo y mínimo cuando corresponde.
93          if (rendimiento > maximo) {
94              maximo = rendimiento;
95          }
96          if (rendimiento < minimo) {
97              minimo = rendimiento;
98          }
99
100         // SELECCIÓN DOBLE + ANIDAMIENTO:
101         // Clasifica el rendimiento como ganancia o pérdida.
102         if (rendimiento > 0) {
103             ganancias++;
104
105             if (rendimiento >= 15) {
106                 sobresalientes++;
107                 System.out.println("Clasificación: ganancia
```

```

    ↪ sobresaliente.");
108         } else {
109             System.out.println("Clasificación: ganancia
    ↪ moderada.");
110         }
111
112     } else {
113         if (rendimiento < 0) {
114             perdidas++;
115
116             if (rendimiento <= -10) {
117                 criticos++;
118                 System.out.println("Clasificación: pérdida
    ↪ crítica.");
119             } else {
120                 System.out.println("Clasificación: pérdida
    ↪ moderada.");
121             }
122         } else {
123             estables++;
124             System.out.println("Clasificación: periodo
    ↪ estable.");
125         }
126     }
127
128     // EVALUACIÓN DIRECTA DE BOOLEANO:
129     // Determina si el rendimiento pertenece a una zona de bajo
    ↪ riesgo.
130     boolean bajoRiesgo = rendimiento >= -2 && rendimiento <= 5;
131     if (bajoRiesgo) {
132         System.out.println("Observación: rendimiento dentro de
    ↪ zona de bajo riesgo.");
133     }
134
135     // SELECCIÓN SIMPLE:
136     // Detecta un caso financiero especial.
137     if (rendimiento == 0) {

```



```
138         System.out.println("Observación: no hubo variación en
↳ el periodo.");
139     }
140
141     contador++;
142 }
143
144 // -----
145 // CÁLCULO FINAL DEL PROMEDIO
146 // -----
147 promedio = sumaRendimientos / cantidadPeriodos;
148 datosProcesados = true;
149
150 // -----
151 // DO-WHILE + SWITCH
152 // Menú principal de consulta de resultados.
153 // -----
154 do {
155     System.out.println("\n===== MENÚ DE CONSULTA
↳ =====");
156     System.out.println("1. Mostrar cantidad de periodos con
↳ ganancia");
157     System.out.println("2. Mostrar cantidad de periodos con
↳ pérdida");
158     System.out.println("3. Mostrar cantidad de periodos
↳ estables");
159     System.out.println("4. Mostrar promedio general de
↳ rendimiento");
160     System.out.println("5. Mostrar rendimiento máximo y
↳ mínimo");
161     System.out.println("6. Mostrar cantidad de rendimientos
↳ sobresalientes y críticos");
162     System.out.println("7. Mostrar resumen interpretativo");
163     System.out.println("8. Mostrar tabla de índices de
↳ periodos");
164     System.out.println("9. Salir");
165     System.out.print("Seleccione una opción: ");
```

```
166
167         int opcion = entrada.nextInt();
168
169         switch (opcion) {
170             case 1:
171                 if (datosProcesados) {
172                     System.out.println("Periodos con ganancia: " +
↪ ganancias);
173                 }
174                 break;
175             case 2:
176                 if (datosProcesados) {
177                     System.out.println("Periodos con pérdida: " +
↪ perdidas);
178                 }
179                 break;
180             case 3:
181                 if (datosProcesados) {
182                     System.out.println("Periodos estables: " +
↪ estables);
183                 }
184                 break;
185             case 4:
186                 if (datosProcesados) {
187                     System.out.printf("Promedio general de
↪ rendimiento: %.2f%%\n", promedio);
188                 }
189                 break;
190             case 5:
191                 if (datosProcesados) {
192                     System.out.printf("Rendimiento máximo:
↪ %.2f%%\n", maximo);
193                     System.out.printf("Rendimiento mínimo:
↪ %.2f%%\n", minimo);
194                 }
195                 break;
196             case 6:
```

```
197         if (datosProcesados) {
198             System.out.println("Rendimientos
↳ sobresalientes: " + sobresalientes);
199             System.out.println("Rendimientos críticos: " +
↳ criticos);
200         }
201         break;
202     case 7:
203         if (datosProcesados) {
204             System.out.println("\n===== RESUMEN
↳ INTERPRETATIVO =====");
205             System.out.println("Cantidad de periodos
↳ analizados: " + cantidadPeriodos);
206             System.out.println("Ganancias: " + ganancias);
207             System.out.println("Pérdidas: " + perdidas);
208             System.out.println("Estables: " + estables);
209             System.out.println("Sobresalientes: " +
↳ sobresalientes);
210             System.out.println("Críticos: " + criticos);
211             System.out.printf("Promedio: %.2f%%\n",
↳ promedio);
212             System.out.printf("Máximo: %.2f%%\n", maximo);
213             System.out.printf("Mínimo: %.2f%%\n", minimo);
214
215             // -----
216             // IF - ELSE IF - ELSE
217             // Clasifica el promedio financiero general.
218             // -----
219             int categoriaPromedio;
220
221             if (promedio >= 12) {
222                 categoriaPromedio = 1;
223             } else if (promedio >= 5) {
224                 categoriaPromedio = 2;
225             } else if (promedio >= 0) {
226                 categoriaPromedio = 3;
227             } else if (promedio >= -5) {
```

```
228         categoriaPromedio = 4;
229     } else {
230         categoriaPromedio = 5;
231     }
232
233     // SWITCH adicional:
234     // Interpreta la categoría del promedio.
235     System.out.print("Interpretación: ");
236     switch (categoriaPromedio) {
237         case 1:
238             System.out.println("rendimiento
↪ financiero sobresaliente.");
239             break;
240         case 2:
241             System.out.println("rendimiento
↪ financiero favorable.");
242             break;
243         case 3:
244             System.out.println("rendimiento
↪ financiero estable.");
245             break;
246         case 4:
247             System.out.println("rendimiento
↪ financiero en riesgo.");
248             break;
249         default:
250             System.out.println("rendimiento
↪ financiero desfavorable.");
251     }
252     }
253     break;
254     case 8:
255         System.out.println("Tabla de índices de periodos:");
256
257         // FOR:
258         // Repetición por contador con un número conocido
↪ de iteraciones.
```

```
259         for (int i = 1; i <= cantidadPeriodos; i++) {
260             System.out.println("Periodo " + i);
261         }
262         break;
263     case 9:
264         salir = true;
265         System.out.println("Fin del programa.");
266         break;
267     default:
268         System.out.println("Opción no válida.");
269     }
270
271     } while (!salir);
272     entrada.close();
273 }
274 }
```

Listing 9.17: Ejercicio integrador de análisis financiero-estadístico en una sola función `main`

La solución integra, dentro de una sola función `main`, los principales mecanismos de control estudiados en este capítulo. Su planteamiento se inspira en un escenario financiero y estadístico sencillo, pero suficientemente rico desde el punto de vista algorítmico: registrar rendimientos, validarlos, clasificarlos, acumular resultados y ofrecer un menú de consulta. De este modo, el problema no se limita a ilustrar la sintaxis de un lenguaje, sino que muestra cómo las estructuras de control cooperan para resolver una necesidad propia de una aplicación informacional, donde los datos son capturados, resumidos e interpretados para apoyar decisiones [54], [118].

Desde la perspectiva didáctica, este ejercicio resulta especialmente valioso porque reúne en un mismo programa la selección simple, la selección doble, las condiciones compuestas, la evaluación directa de variables booleanas, el anidamiento, la selección múltiple con `switch`, la repetición condicional con `while` y `do-while`, la repetición por contador con `for`, así como el uso de contadores, acumuladores y variables para el cálculo de extremos. En consecuencia, permite que el lector observe que estas estructuras no deben aprenderse como construcciones aisladas, sino como recursos complementarios que modelan decisiones, repeticiones y estados dentro de un proceso analítico más amplio [68].

También conviene destacar que el programa se mantiene deliberadamente en una sola función `main`. Esta decisión responde a una finalidad pedagógica concreta: facilitar que el estudiante principiante pueda seguir la continuidad del flujo completo sin fragmentarlo todavía en métodos auxiliares. De esta manera, la atención se concentra en la lógica de control, en el encadenamiento de decisiones y en la forma en que las iteraciones transforman progresivamente el estado del programa. Una vez comprendida esta versión integral, el mismo problema puede refactorizarse como ejercicio posterior para introducir modularización y diseño más estructurado [80].

El programa comienza con una repetición de postcondición para asegurar que la cantidad de periodos ingresada sea válida. Esta primera fase enseña que, en una aplicación financiera, la confiabilidad del análisis depende de la calidad de los datos desde el inicio. Posteriormente, mediante una repetición condicional con `while`, se capturan los rendimientos y se procesan de manera iterativa. En esta etapa intervienen contadores para clasificar los periodos y acumuladores para construir los indicadores globales, lo que permite apreciar cómo una estructura repetitiva puede servir al mismo tiempo para registrar información y para resumirla estadísticamente.

A lo largo del ingreso de rendimientos, el programa aplica decisiones sucesivas que distinguen entre ganancias, pérdidas, periodos estables, comportamientos sobresalientes, situaciones críticas y zonas de bajo riesgo. Esta combinación pone de

manifiesto que la selección, en contextos reales, no suele reducirse a una sola comparación aislada, sino que implica interpretar el mismo dato desde diferentes perspectivas funcionales. Después, una vez procesados todos los valores, el programa activa un menú repetitivo controlado con `do-while` y gestionado con `switch`, lo que evidencia cómo la selección múltiple puede utilizarse tanto para la navegación funcional dentro del sistema como para la interpretación semántica de resultados.

Finalmente, el ejercicio incorpora un segundo `switch` para traducir la categoría del promedio financiero en una valoración comprensible y añade un ciclo `for` para ilustrar una repetición por contador en un contexto de consulta. Desde una perspectiva formativa, el valor central de este problema reside en que no presenta las estructuras de control como elementos desconectados, sino como partes interdependientes de un mismo flujo algorítmico. Esta integración favorece una comprensión más profunda del comportamiento dinámico del programa y prepara al lector para problemas más complejos en finanzas computacionales, estadística aplicada y desarrollo de software orientado al análisis de datos.

Capítulo 10

Estructuración y Organización de Datos

El desarrollo de aplicaciones informacionales no solo requiere controlar el flujo de ejecución mediante estructuras de selección y repetición, sino también organizar adecuadamente los datos que serán procesados. En contextos reales, la información rara vez se presenta como valores aislados; por el contrario, se gestiona en conjuntos estructurados que representan entidades, registros o colecciones de elementos relacionados. Por esta razón, la estructuración de datos constituye un componente esencial en el diseño algorítmico y en la construcción de software [1], [2].

Desde una perspectiva computacional, organizar los datos implica definir cómo se almacenan, cómo se accede a ellos y cómo se recorren para aplicar operaciones de cálculo, clasificación o análisis. Este proceso no solo influye en la eficiencia del programa, sino también en su claridad, mantenibilidad y capacidad de adaptación. En consecuencia, el uso de estructuras como arreglos y registros permite modelar de manera más fiel los problemas del mundo real dentro de un entorno computacional [36], [53].

En este capítulo se abordan los fundamentos de los arreglos unidimensionales, los arreglos bidimensionales y los datos compuestos. Cada una de estas estructuras amplía las posibilidades de representación de la información, permitiendo pasar de variables individuales a colecciones organizadas. Asimismo, se presentan ejercicios integradores que retoman los algoritmos y diagramas de flujo desarrollados previamente, incorporando ahora el manejo estructurado de datos [7], [87].

10.1. Arreglos unidimensionales

Los arreglos unidimensionales constituyen una de las estructuras más fundamentales para la organización de datos en programación. Su principal objetivo es permitir almacenar múltiples valores del mismo tipo bajo un único identificador, evitando la necesidad de declarar múltiples variables independientes. Esta característica resulta especialmente útil cuando se requiere trabajar con conjuntos de datos homogéneos, como listas de calificaciones, registros de ventas o mediciones de sensores [7], [71].

Desde una perspectiva práctica, un arreglo permite modelar una colección de datos que serán procesados de manera repetitiva. Esto simplifica significativamente el diseño del programa, ya que permite aplicar operaciones sobre todos los elementos mediante estructuras de control, en lugar de repetir instrucciones manualmente [36], [53].

10.1.1. Concepto y uso

Un arreglo unidimensional puede entenderse como una colección ordenada de elementos almacenados en posiciones contiguas de memoria. Cada posición contiene un valor y puede ser accedida mediante un índice numérico. Esta estructura permite representar múltiples datos bajo una sola variable, facilitando su manipulación [87].

Declaración de un arreglo

La declaración de un arreglo define su tipo y su tamaño. El tamaño indica cuántos elementos puede almacenar.

El Listado de código 10.1 muestra la declaración básica de un arreglo en Java.

```
1 // Declaración de un arreglo de 5 elementos enteros
2 int[] numeros = new int[5];
```

Listing 10.1: Declaración de un arreglo unidimensional

En este caso, el arreglo puede almacenar cinco valores de tipo entero, accesibles mediante índices del 0 al 4.

Inicialización de un arreglo

La inicialización consiste en asignar valores a las posiciones del arreglo. Esto puede hacerse manualmente o en el momento de la declaración.

El Listado de código 10.2 muestra ambas formas.

```
1 // Inicialización manual
2 numeros[0] = 10;
3 numeros[1] = 20;
4
5 // Inicialización directa
6 int[] datos = {5, 15, 25, 35, 45};
```

Listing 10.2: Inicialización de un arreglo

Esta flexibilidad permite adaptar el uso del arreglo según el contexto del problema.

Acceso a los elementos

El acceso a los elementos se realiza mediante índices. Cada índice identifica una posición específica dentro del arreglo.

El Listado de código 10.3 muestra cómo acceder a un elemento.

```
1 // Acceso al tercer elemento (índice 2)
2 System.out.println(datos[2]); // Imprime 25
```

Listing 10.3: Acceso a elementos del arreglo

Índices

Los índices representan la posición de cada elemento dentro del arreglo. En Java, los índices comienzan en cero, lo que implica que el primer elemento está en la posición 0 y el último en $n - 1$, siendo n el tamaño del arreglo [87].

Comprender este aspecto es fundamental, ya que muchos errores en programación se originan por un uso incorrecto de los índices. Un acceso fuera del rango válido produce errores en tiempo de ejecución [71].

Recorridos

El recorrido de un arreglo consiste en acceder a todos sus elementos de manera secuencial. Esta operación se realiza mediante estructuras repetitivas, principalmente el ciclo `for`, ya que permite controlar fácilmente el índice.

El Listado de código 10.4 muestra un recorrido completo.

```
1   int[] valores = {10, 20, 30, 40, 50};
2
3   for (int i = 0; i < valores.length; i++) {
4       System.out.println("Elemento " + i + ": " + valores[i]);
5   }
```

Listing 10.4: Recorrido de un arreglo

Este tipo de recorrido permite aplicar operaciones sobre todos los elementos, como sumas, búsquedas o transformaciones [7].

Errores de rango

Los errores de rango ocurren cuando se intenta acceder a una posición que no existe dentro del arreglo. Este problema suele surgir cuando el índice supera los límites definidos.

El Listado de código 10.5 muestra un ejemplo de error.

```
1   int[] arreglo = {1, 2, 3};
2
3   // Error: índice fuera de rango
4   System.out.println(arreglo[3]); // Provoca excepción
```

Listing 10.5: Ejemplo de error de rango

Para evitar este tipo de errores, es necesario controlar los límites del arreglo antes de acceder a sus elementos. Esto puede lograrse utilizando condiciones o estructuras de control adecuadas [71].

Desde una perspectiva de calidad del software, la prevención de errores de rango contribuye a la robustez del sistema y evita fallos inesperados durante la ejecución [51].

El siguiente ejemplo integra los conceptos estudiados: declaración, inicialización, recorrido, acumulación y validación básica. Se plantea el procesamiento de calificaciones para calcular el promedio general.

El Listado de código 10.6 presenta una solución completa.

```
1   import java.util.Scanner;
2
3   public class Calificaciones {
```

```
4      public static void main(String[] args) {
5
6          Scanner sc = new Scanner(System.in);
7
8          System.out.print("Ingrese la cantidad de estudiantes: ");
9          int n = sc.nextInt();
10
11         double[] notas = new double[n];
12         double suma = 0;
13
14         // Ingreso de datos
15         for (int i = 0; i < n; i++) {
16             System.out.print("Nota " + (i + 1) + ": ");
17             notas[i] = sc.nextDouble();
18
19             // Validación básica
20             if (notas[i] < 0 || notas[i] > 100) {
21                 System.out.println("Nota inválida.");
22                 i--; // Repetir ingreso
23                 continue;
24             }
25
26             suma += notas[i];
27         }
28
29         // Cálculo del promedio
30         double promedio = suma / n;
31
32         System.out.println("Promedio: " + promedio);
33     }
34 }
```

Listing 10.6: Ejemplo completo con arreglos unidimensionales

Este ejemplo evidencia cómo los arreglos permiten gestionar múltiples datos de manera estructurada y eficiente. Además, muestra la integración de estructuras de control previamente estudiadas, lo que refuerza la idea de que los arreglos no son elementos aislados, sino componentes que trabajan en conjunto dentro de un sistema algorítmico más amplio [53].

10.2. Arreglos bidimensionales

Los arreglos bidimensionales constituyen una extensión natural de los arreglos unidimensionales y permiten representar información organizada en dos dimensiones. Mientras que un arreglo simple puede interpretarse como una lista, un arreglo bidimensional puede entenderse como una tabla, donde los datos se distribuyen en filas y columnas. Esta forma de organización resulta especialmente útil cuando los datos poseen una estructura tabular inherente, como ocurre en registros administrativos, reportes financieros o matrices de análisis [7], [87].

Trabajar con arreglos bidimensionales implica incorporar una segunda dimensión de acceso a los datos. Esto significa que ya no es suficiente un único índice para identificar un elemento, sino que se requieren dos: uno para ubicar la fila y otro para identificar la columna. Esta característica permite modelar relaciones más complejas entre los datos, facilitando su análisis y procesamiento dentro de sistemas de información [36], [53].

10.2.1. Matrices

Una matriz es una estructura de datos bidimensional que organiza elementos en una cuadrícula de filas y columnas. Cada elemento se accede mediante una pareja de índices, comúnmente expresada como `matriz[fila][columna]`. Esta representación permite ubicar cualquier dato de manera precisa dentro de la estructura.

En el ámbito de los sistemas de información, las matrices son ampliamente utilizadas para representar datos tabulares, tales como registros de inventario, reportes contables o indicadores de desempeño. Su uso facilita la organización sistemática de la información y permite aplicar operaciones analíticas sobre los datos almacenados [27], [28].

El Listado de código 10.7 muestra la declaración, inicialización y acceso básico a una matriz en Java.

```
1      public class MatrizBasica {
2          public static void main(String[] args) {
3
4              // Declaración de una matriz de 3 filas y 3 columnas
5              int[] [] matriz = new int[3][3];
6
7              // Asignación de valores a posiciones específicas
8              matriz[0][0] = 10;
9              matriz[1][2] = 25;
10             matriz[2][1] = 40;
11
12             // Acceso a un elemento de la matriz
13             System.out.println("Valor en [1][2]: " + matriz[1][2]);
14         }
15     }
```

Listing 10.7: Declaración y acceso básico a una matriz

Este ejemplo permite comprender que cada elemento está determinado por dos coordenadas, lo que diferencia claramente a las matrices de los arreglos unidimensionales.

Filas y columnas

En una matriz, las filas representan conjuntos horizontales de datos, mientras que las columnas representan conjuntos verticales. Esta organización no es arbitraria, sino que responde a la necesidad de estructurar la información de manera coherente con el dominio del problema.

Por ejemplo, en un sistema de gestión de clientes, cada fila puede representar un cliente distinto, mientras que cada columna corresponde a un atributo específico, como edad, saldo o nivel de servicio. Esta forma de organización permite interpretar los datos de manera más clara y facilita su procesamiento posterior [1], [28].

El Listado de código 10.8 ilustra cómo una matriz puede representar información estructurada en términos de filas y columnas.

```
1  public class MatrizClientes {
2      public static void main(String[] args) {
3
4          // Cada fila representa un cliente
5          // Cada columna representa un atributo: [edad, saldo]
6          int[][] datos = {
7              {25, 1500},
8              {30, 2300},
9              {28, 1800}
10         };
11
12         // Acceso: cliente 2 (fila 1), saldo (columna 1)
13         System.out.println("Saldo del cliente 2: " + datos[1][1]);
14     }
15 }
```

Listing 10.8: Representación conceptual de filas y columnas

Este enfoque permite interpretar la matriz como una tabla estructurada, donde cada posición tiene un significado definido.

Recorridos

El procesamiento de matrices requiere recorrer todos sus elementos, lo cual se logra mediante estructuras repetitivas anidadas. En este esquema, un ciclo externo se encarga de recorrer las filas, mientras que un ciclo interno recorre las columnas de cada fila.

Este tipo de recorrido es fundamental, ya que permite aplicar operaciones sobre todos los elementos de la matriz, como sumas, búsquedas o transformaciones. La correcta implementación de estos recorridos garantiza que el algoritmo sea eficiente y que todos los datos sean procesados adecuadamente [7], [53].

El Listado de código 10.9 presenta un ejemplo completo de recorrido de una matriz.

```
1  public class RecorridoMatriz {
2      public static void main(String[] args) {
3
```

```
4         int[][] matriz = {
5             {1, 2, 3},
6             {4, 5, 6},
7             {7, 8, 9}
8         };
9
10        // Recorrido de filas y columnas
11        for (int i = 0; i < matriz.length; i++) {           // Filas
12            for (int j = 0; j < matriz[i].length; j++) {    // Columnas
13                System.out.print(matriz[i][j] + " ");
14            }
15            System.out.println();
16        }
17    }
18 }
```

Listing 10.9: Recorrido de una matriz con ciclos anidados

Este patrón de recorrido constituye una base fundamental para el procesamiento de datos estructurados en múltiples dimensiones.

Aplicaciones organizacionales

Las matrices se utilizan ampliamente en sistemas de información debido a su capacidad para representar datos en formato tabular. Esta característica las convierte en una herramienta clave para el análisis y la toma de decisiones.

Entre sus aplicaciones más comunes se encuentran:

- Registros de ventas por período y producto
- Control de asistencia por día y entidad
- Análisis de indicadores financieros

Estas aplicaciones evidencian que las matrices permiten organizar grandes volúmenes de información de manera estructurada, facilitando su análisis y procesamiento. En consecuencia, su uso resulta fundamental en el diseño de sistemas que gestionan datos complejos y requieren operaciones analíticas eficientes [1], [28].

10.3. Registros y datos compuestos

En el desarrollo de aplicaciones informacionales, la representación adecuada de los datos constituye un aspecto fundamental para modelar correctamente las entidades del dominio del problema. Mientras que estructuras como los arreglos permiten almacenar colecciones homogéneas de datos, los registros —o estructuras compuestas— permiten agrupar información heterogénea bajo una misma unidad lógica. Esta capacidad resulta esencial cuando se requiere representar entidades complejas, como clientes, productos o transacciones, cuyos atributos poseen distintos tipos y significados [28], [87].

El uso de datos compuestos contribuye a mejorar la modularidad y la claridad del código, ya que permite encapsular información relacionada en estructuras coherentes, aspectos fundamentales de la ingeniería de software. Esta organización

favorece la comprensión del programa, reduce la probabilidad de errores y facilita la evolución del sistema a medida que se incorporan nuevos requerimientos [24], [51].

10.3.1. Estructuras simples

Las estructuras simples, como los registros o las clases en Java, constituyen el mecanismo básico para modelar entidades del mundo real mediante la agrupación de campos relacionados. En términos prácticos, una clase puede interpretarse como un molde que define la forma y el comportamiento de un conjunto de objetos, permitiendo instanciar múltiples entidades con la misma estructura [7], [87].

Este enfoque no solo mejora la organización del código, sino que también permite establecer una correspondencia directa entre los elementos del sistema y los conceptos del dominio. De este modo, la estructura del programa refleja de manera más fiel la realidad que se pretende modelar, lo cual es un principio fundamental en el diseño de sistemas de información [1].

El Listado de código 10.10 presenta un ejemplo básico de una estructura simple que representa un cliente dentro de un sistema informacional.

```
1      /**
2      * Clase que representa la entidad Cliente
3      * Integra múltiples atributos (datos heterogéneos)
4      */
5      class Cliente {
6          String nombre;      // Campo de tipo cadena
7          int edad;           // Campo de tipo entero
8          double saldo;       // Campo de tipo real
9          boolean activo;     // Campo booleano
10
11         // Método para mostrar información del cliente
12         void mostrarInformacion() {
13             System.out.println("Nombre: " + nombre);
14             System.out.println("Edad: " + edad);
15             System.out.println("Saldo: " + saldo);
16             System.out.println("Estado: " + (activo ? "Activo" :
↵ "Inactivo"));
17         }
18     }
```

Listing 10.10: Definición de una estructura simple para representar un cliente

Este ejemplo evidencia cómo una entidad del dominio puede representarse mediante un conjunto de atributos relacionados, permitiendo manipular la información de manera estructurada y coherente.

Campos

Los campos representan los atributos que describen una entidad dentro de una estructura compuesta. Cada campo almacena un tipo específico de dato, lo que permite modelar características diversas dentro de una misma estructura. Por ejemplo, un cliente puede tener un nombre (cadena), una edad (entero) y un saldo (valor numérico real).

La correcta definición de los campos es un aspecto crítico, ya que determina la capacidad del sistema para representar fielmente la información. En el contexto de bases de datos y sistemas de información, una definición inadecuada de los atributos puede conducir a inconsistencias, pérdida de información o dificultades en el procesamiento posterior [27], [28].

Acceso

El acceso a los campos de una estructura compuesta se realiza mediante operadores específicos, siendo el operador punto (.) el más común en lenguajes orientados a objetos como Java. Este mecanismo permite referenciar de manera directa cada atributo de la entidad, facilitando su lectura y modificación [87].

El Listado de código 10.11 ilustra cómo se accede y manipula la información contenida en una estructura compuesta.

```
1      public class SistemaClientes {
2          public static void main(String[] args) {
3
4              // Creación de un objeto de tipo Cliente
5              Cliente cliente1 = new Cliente();
6
7              // Asignación de valores a los campos
8              cliente1.nombre = "Ana Torres";
9              cliente1.edad = 30;
10             cliente1.saldo = 2450.75;
11             cliente1.activo = true;
12
13             // Acceso a los campos
14             System.out.println("Cliente: " + cliente1.nombre);
15             System.out.println("Saldo disponible: " + cliente1.saldo);
16
17             // Uso de un método de la clase
18             cliente1.mostrarInformacion();
19         }
20     }
```

Listing 10.11: Acceso y manipulación de campos en una estructura compuesta

Este tipo de acceso estructurado permite mantener la coherencia del sistema, ya que los datos se manipulan dentro de un contexto claramente definido.

Validación

La validación de los datos en estructuras compuestas constituye un mecanismo fundamental para garantizar la integridad de la información. En sistemas reales, los datos pueden provenir de múltiples fuentes —usuarios, sensores o sistemas externos—, por lo que es necesario verificar que cumplan con las reglas establecidas por el dominio del problema [24], [27].

La ausencia de validación puede provocar inconsistencias que afecten el funcionamiento del sistema, generando resultados incorrectos o comportamientos inesperados. Por esta razón, es recomendable incorporar mecanismos de control que verifiquen los valores antes de asignarlos a los campos de una estructura.

El Listado de código 10.12 presenta un ejemplo en el que se incorporan validaciones básicas para garantizar la consistencia de los datos.

```
1      class ClienteValidado {
2          String nombre;
3          int edad;
4          double saldo;
5
6          // Método para asignar edad con validación
7          void setEdad(int edad) {
8              if (edad >= 0 && edad <= 120) {
9                  this.edad = edad;
10             } else {
11                 System.out.println("Error: edad inválida.");
12             }
13         }
14
15         // Método para asignar saldo con validación
16         void setSaldo(double saldo) {
17             if (saldo >= 0) {
18                 this.saldo = saldo;
19             } else {
20                 System.out.println("Error: saldo no puede ser negativo.");
21             }
22         }
23     }
24
25     public class ValidacionCliente {
26         public static void main(String[] args) {
27
28             ClienteValidado c = new ClienteValidado();
29         }
```



```
30         c.nombre = "Carlos Pérez";
31
32         // Intento de asignación inválida
33         c.setEdad(-5);           // Genera error
34         c.setSaldo(1500.0);     // Correcto
35
36         System.out.println("Cliente: " + c.nombre);
37         System.out.println("Edad: " + c.edad);
38         System.out.println("Saldo: " + c.saldo);
39     }
40 }
```

Listing 10.12: Validación de datos en una estructura compuesta

Este ejemplo evidencia que la validación no solo protege la integridad de los datos, sino que también constituye una práctica esencial en el desarrollo de software confiable y mantenible [51].

En síntesis, los registros y datos compuestos permiten estructurar la información de manera coherente con el dominio del problema, facilitando su manipulación, validación y mantenimiento. Su uso adecuado representa un paso fundamental en la transición desde programas simples hacia sistemas de información más robustos y escalables.

10.3.2. Ejercicio 1: Análisis de ventas mensuales

Se requiere diseñar un programa que permita registrar las ventas de un producto durante un conjunto de meses y, a partir de esa información, calcular el total vendido, el promedio mensual y el mes en el que se alcanzó la mayor venta. El procedimiento debe solicitar primero la cantidad de meses que se desea analizar, validar que dicha cantidad sea correcta y, posteriormente, capturar el valor correspondiente a cada mes. Una vez almacenados los datos, el algoritmo debe recorrerlos para acumular el total de ventas, calcular el promedio y comparar cada valor con el mayor registrado hasta el momento, de manera que al finalizar sea posible identificar no solo el valor máximo, sino también la posición del mes en el que ocurrió [7], [53], [87].

Este ejercicio requiere el uso de un **arreglo** o **vector**, porque no se trabaja con una sola venta, sino con una colección ordenada de valores homogéneos, uno por cada mes. El arreglo permite conservar todos los datos introducidos y asociarlos con una posición específica, lo que resulta indispensable cuando se necesita recorrer el conjunto de ventas de manera organizada. Sin esta estructura, el problema obligaría a declarar múltiples variables individuales, lo que volvería la solución rígida, difícil de ampliar y poco coherente con el tratamiento sistemático de la información. El vector, en cambio, permite almacenar, recuperar y recorrer cada dato mediante índices, lo que lo convierte en la estructura natural para representar este tipo de problema [7], [35], [71].

La importancia del problema radica en que enlaza directamente el uso de arreglos con operaciones algorítmicas ya estudiadas en capítulos anteriores, como la suma acumulada, el cálculo de promedios y la comparación sucesiva de valores. A diferencia de ejercicios donde solo se procesa un dato en cada momento, aquí la solución debe coordinar almacenamiento, recorrido, acumulación y decisión sobre una colección completa. Esa articulación fortalece la comprensión de cómo una estructura de datos modifica la forma en que se resuelve un problema, ya que obliga a pasar de un razonamiento centrado en variables individuales a un razonamiento centrado en conjuntos organizados de información [74], [75], [76].

Pasos para Desarrollar la Solución

Los pasos para desarrollar la solución permiten transformar una necesidad de análisis comercial en una secuencia ordenada de operaciones que puedan ejecutarse de manera sistemática. En este ejercicio, la lógica del problema se apoya en tres ideas principales: almacenar cada venta, acumular el total y comparar los valores para localizar el mayor. La solución no se limita a

“leer y mostrar” datos, sino que exige coordinar varias operaciones sobre el mismo conjunto de elementos, lo que convierte al arreglo en el eje de la propuesta [35], [53].

1. Análisis del problema. Para comprender el problema conviene partir de pequeños conjuntos de ventas y analizar manualmente qué resultados deberían obtenerse. Si se registran varias ventas mensuales, el total se obtiene sumando todos los valores; el promedio, dividiendo dicha suma entre la cantidad de meses; y el mes con mayor venta, comparando cada dato con el mayor encontrado hasta el momento. La dificultad algorítmica no está en la fórmula del promedio, sino en la necesidad de coordinar la lectura ordenada de los datos, su almacenamiento en un arreglo y el seguimiento simultáneo de la suma total y del valor máximo. Por tanto, la solución debe estructurarse de manera que cada venta se procese una sola vez durante el recorrido principal, acumulando información parcial que luego permita obtener los resultados finales [7], [74], [87].

Ejercicios de análisis manual con valores representativos. Antes de escribir el algoritmo, resulta conveniente resolver manualmente algunos casos que permitan verificar el comportamiento esperado del programa.

■ **Caso 1: Ventas crecientes**

1. **Datos de entrada**

- $n = 4$
- $\text{ventas} = [100, 150, 200, 250]$

2. **Resolución manual**

$$\text{total} = 100 + 150 + 200 + 250 = 700$$

$$\text{promedio} = \frac{700}{4} = 175$$

La mayor venta es 250, correspondiente al mes 4.

3. **Resultado esperado**

$$\text{total} = 700, \quad \text{promedio} = 175, \quad \text{mes mayor} = 4$$

4. **Observación.** Este caso permite verificar el comportamiento más sencillo, donde el valor máximo aparece al final del arreglo.

■ **Caso 2: Mayor venta al inicio**

1. **Datos de entrada**

- $n = 5$
- $\text{ventas} = [500, 320, 280, 410, 390]$

2. **Resolución manual**

$$\text{total} = 500 + 320 + 280 + 410 + 390 = 1900$$

$$\text{promedio} = \frac{1900}{5} = 380$$

La mayor venta es 500, correspondiente al mes 1.

3. **Resultado esperado**

$$\text{total} = 1900, \quad \text{promedio} = 380, \quad \text{mes mayor} = 1$$

4. **Observación.** Este caso confirma que el algoritmo debe conservar correctamente el primer valor como máximo inicial cuando ningún valor posterior lo supera.

■ **Caso 3: Ventas repetidas con máximo compartido**

1. **Datos de entrada**

- $n = 4$
- $\text{ventas} = [300, 450, 450, 200]$

2. Resolución manual

$$\text{total} = 300 + 450 + 450 + 200 = 1400$$

$$\text{promedio} = \frac{1400}{4} = 350$$

La mayor venta es 450. Si el algoritmo actualiza solo cuando encuentra un valor estrictamente mayor, conservará el primer mes en que apareció el máximo, es decir, el mes 2.

3. Resultado esperado

$$\text{total} = 1400, \quad \text{promedio} = 350, \quad \text{mes mayor} = 2$$

4. **Observación.** Este caso permite razonar sobre el criterio de comparación y mostrar que, ante máximos repetidos, puede mantenerse la primera aparición.

■ Caso 4: Un solo mes

1. Datos de entrada

- $n = 1$
- $\text{ventas} = [875.50]$

2. Resolución manual

$$\text{total} = 875.50, \quad \text{promedio} = 875.50$$

La mayor venta coincide con el único dato y corresponde al mes 1.

3. Resultado esperado

$$\text{total} = 875.50, \quad \text{promedio} = 875.50, \quad \text{mes mayor} = 1$$

4. **Observación.** Este caso muestra el funcionamiento mínimo válido del problema.

■ Caso 5: Cantidad no válida

1. Datos de entrada

- $n = 0$

2. **Resolución manual** La cantidad de meses no puede ser cero ni negativa, porque no existirían datos que procesar ni sería posible calcular el promedio.

3. Resultado esperado

4. *“Error: la cantidad de meses debe ser mayor que cero. Intente de nuevo.”*

5. **Observación.** Este caso obliga a incorporar una validación inicial del tamaño del arreglo y evita divisiones inválidas o arreglos sin uso [23], [24].

2. Identificación de datos de entrada. Los datos de entrada son los valores que el algoritmo necesita para desarrollar el análisis de ventas.

- **n (entero):** cantidad de meses que serán procesados.
- **ventas[i] (real):** venta correspondiente a cada mes.

El valor **n** determina el tamaño del arreglo, mientras que cada **ventas[i]** representa una observación del conjunto. Esta relación entre cantidad y almacenamiento es esencial, porque la estructura del problema depende de saber cuántos datos se deben guardar antes de comenzar el recorrido [7], [87].

3. Identificación de datos de salida. Los datos de salida son los resultados que el algoritmo entrega al finalizar el procesamiento.

- **Total vendido (real):** suma de todas las ventas mensuales.

- **Promedio mensual (real):** cociente entre el total vendido y la cantidad de meses.
- **Mes con mayor venta (entero):** posición del mes donde se registró el valor máximo.

Estas salidas resumen tres perspectivas distintas del mismo conjunto de datos: acumulación total, tendencia central e identificación del valor máximo. Por ello, el ejercicio muestra cómo un único arreglo puede ser procesado para obtener varios resultados complementarios.

4. Identificación de cálculos intermedios (procesos intermedios). Los procesos intermedios son las operaciones necesarias antes de obtener las respuestas finales.

- **Validación inicial de n:** comprobar que la cantidad de meses sea mayor que cero.
- **Declaración del arreglo:** reservar espacio para almacenar una venta por cada mes.
- **Inicialización de variables auxiliares:** `suma <- 0`, `mayor <- venta inicial o valor mínimo posible` y `mesMayor <- 0`.
- **Recorrido del arreglo:** capturar cada venta, almacenarla, sumarla al total y compararla con el mayor valor registrado.
- **Cálculo del promedio:** `promedio <- suma / n`.

Estos cálculos muestran que la solución combina almacenamiento y procesamiento en una misma secuencia de trabajo. El arreglo conserva la información; el recorrido la analiza; y las variables auxiliares permiten construir los resultados finales [36], [53].

5. Identificación de cálculos finales (procesos finales). Los procesos finales son aquellos que convierten la información acumulada en resultados interpretables.

- **Obtención del total:** resultado del acumulador `suma`.
- **Obtención del promedio:** resultado de dividir `suma` entre `n`.
- **Determinación del mes con mayor venta:** resultado del seguimiento de `mesMayor`.
- **Presentación de resultados:** impresión del total, el promedio y el mes asociado al máximo.

Estas operaciones no añaden nuevos recorridos, sino que aprovechan la información reunida durante el procesamiento principal, lo que hace más eficiente la solución.

6. Identificación de condiciones (decisiones). El algoritmo necesita decisiones que controlen la validez de los datos y el seguimiento del valor máximo.

- **Validación del número de meses:** `n > 0`.
- **Comparación de máximo:** `ventas[i] > mayor`.

La primera condición evita comenzar el procedimiento con un problema mal planteado. La segunda permite actualizar el valor máximo y la posición asociada solo cuando aparece una venta mayor que la conocida hasta ese momento [35], [59].

7. Identificación de repeticiones o iteraciones. La solución contiene dos formas de repetición claramente distinguibles.

- **Repetición para validar n:** mientras la cantidad de meses no sea válida.
- **Recorrido del arreglo:** para capturar, acumular y comparar cada venta.

La repetición principal del problema se concentra en el recorrido del arreglo. Allí se observa con claridad cómo el índice avanza de manera ordenada y permite tratar a la colección como una secuencia de datos estructurados [7], [75].

8. Condiciones para finalizar el algoritmo. El algoritmo finaliza cuando:

- se ha ingresado una cantidad válida de meses;
- todas las ventas han sido capturadas y almacenadas;
- se ha completado el recorrido del arreglo;

- se han calculado y mostrado el total, el promedio y el mes con mayor venta.

La finalización depende, por tanto, del cumplimiento ordenado de cada fase del proceso: validación, captura, recorrido y presentación de resultados. Esta secuencia convierte al problema en un ejercicio apropiado para consolidar el uso de arreglos unidimensionales en combinación con acumuladores y comparaciones sucesivas.

Propuesta de Solución

La propuesta de solución organiza el problema en tres momentos claramente diferenciados: validación de la cantidad de meses, captura y procesamiento de las ventas, y presentación de resultados. Esta organización evita mezclar operaciones heterogéneas en una sola etapa y facilita seguir el comportamiento del algoritmo completo. La idea central consiste en utilizar un arreglo para almacenar una venta por cada mes y, durante el mismo recorrido de captura, aprovechar cada dato para construir el total y comparar si se ha encontrado una nueva venta máxima. De esta forma, la solución mantiene un flujo claro y económico, sin recorridos innecesarios [59], [72].

El Algoritmo 10.13 presenta una propuesta estructurada en pseudocódigo UPSAM. En él se distinguen explícitamente la validación inicial, la declaración del arreglo, la inicialización del acumulador y de la variable de comparación, el recorrido controlado por índice y el cálculo final del promedio. Esta separación permite apreciar que el problema no se resuelve mediante una fórmula aislada, sino mediante la coordinación de almacenamiento, recorrido, acumulación y decisión [53], [74].

```

1      Algoritmo AnalisisVentasMensuales
2      1.      Var:
3      1.1.      n, i, mesMayor: entero
4      1.2.      ventas[]: real
5      1.3.      suma, promedio, mayor: real
6
7      2.      Escribir "Ingrese la cantidad de meses:"
8      3.      Leer n
9      4.      Mientras n <= 0 Hacer
10     4.1.      Escribir "Error: la cantidad de meses debe ser mayor que
    ↪ cero. Intente de nuevo."
11     4.2.      Escribir "Ingrese la cantidad de meses:"
12     4.3.      Leer n
13     5.      FinMientras
14
15     6.      suma <- 0
16     7.      mayor <- -1
17     8.      mesMayor <- 0
18
19     9.      Para i <- 1 Hasta n Hacer
20     9.1.      Escribir "Ingrese la venta del mes ", i, ":"
21     9.2.      Leer ventas[i-1]
22     9.3.      suma <- suma + ventas[i-1]

```

```

23  9.4.      Si ventas[i-1] > mayor Entonces
24  9.4.1.      mayor <- ventas[i-1]
25  9.4.2.      mesMayor <- i-1
26  9.5.      FinSi
27  10.  FinPara
28
29  11.  promedio <- suma / n
30
31  12.  Escribir "Total vendido: ", suma
32  13.  Escribir "Promedio mensual: ", promedio
33  14.  Escribir "Mes con mayor venta: ", mesMayor + 1
34  FinAlgoritmo

```

Listing 10.13: Análisis de ventas mensuales con arreglo unidimensional

La solución en Java, presentada en el Listado de código 10.14, adopta estrategias propias del lenguaje que facilitan la implementación del algoritmo. La instrucción `double[] ventas = new double[n];` permite crear un arreglo cuyo tamaño se conoce en tiempo de ejecución, a partir del valor que se introduce por teclado. El recorrido se implementa con un ciclo `for`, adecuado cuando la cantidad de iteraciones es conocida, mientras que la lectura de datos utiliza la clase `Scanner`, habitual en programas introductorios escritos en Java. El control del valor máximo se apoya en una variable auxiliar y en una comparación directa dentro del recorrido, lo que evita una segunda pasada sobre el arreglo y mantiene la solución compacta y clara [51], [71], [87].

```

1  import java.util.Scanner;
2
3  public class VentasMensuales {
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6
7          // Lee la cantidad de meses que se procesarán
8          System.out.print("Ingrese número de meses: ");
9          int n = sc.nextInt();
10
11         // Valida que la cantidad de meses sea correcta
12         while (n <= 0) {
13             System.out.println("Error: la cantidad de meses debe ser
↪ mayor que cero.");
14             System.out.print("Ingrese número de meses: ");
15             n = sc.nextInt();

```

```
16         }
17
18         // Declara un arreglo dinámico con una posición por cada mes
19         double[] ventas = new double[n];
20
21         // Inicializa variables auxiliares
22         double suma = 0;
23         double mayor = Double.MIN_VALUE;
24         int mesMayor = 0;
25
26         // Recorre el arreglo para capturar, acumular y comparar
27         for (int i = 0; i < n; i++) {
28             System.out.print("Venta del mes " + (i + 1) + ": ");
29             ventas[i] = sc.nextDouble();
30
31             // Acumula el total vendido
32             suma += ventas[i];
33
34             // Actualiza el mayor valor y la posición del mes
35             if (ventas[i] > mayor) {
36                 mayor = ventas[i];
37                 mesMayor = i;
38             }
39         }
40
41         // Calcula el promedio mensual
42         double promedio = suma / n;
43
44         // Presenta los resultados finales
45         System.out.println("Total: " + suma);
46         System.out.println("Promedio: " + promedio);
47         System.out.println("Mayor venta en mes: " + (mesMayor + 1));
48
49         sc.close();
50     }
51 }
```

Listing 10.14: Análisis de ventas mensuales en Java

El algoritmo del Listado de código 10.13 y el programa del Listado de código 10.14 conservan la misma lógica, aunque expresada en dos niveles distintos de formalidad. En el pseudocódigo, el interés principal recae en la secuencia de pasos; en Java, esa secuencia se traduce a una sintaxis precisa mediante tipos, clases, operadores y estructuras de control concretas. Esta correspondencia resulta importante porque permite reconocer que el programa no surge de forma improvisada, sino como una traducción directa del razonamiento algorítmico previo [35], [59].

Conviene prestar atención al papel del índice dentro del recorrido. En Java, el arreglo comienza en la posición cero, de modo que el primer mes corresponde al índice 0, el segundo al índice 1 y así sucesivamente. Sin embargo, al presentar el resultado final, la numeración habitual de los meses se expresa desde uno. Por esa razón, el programa imprime `mesMayor + 1`. Este pequeño ajuste es fundamental para que la salida sea comprensible y esté alineada con la interpretación natural del problema [7], [87].

También debe notarse que el programa almacena las ventas en el arreglo aunque, en esta versión concreta, el total, el promedio y el máximo podrían calcularse durante la captura sin necesidad de reutilizar después todos los valores. Mantener el arreglo sigue siendo una decisión didácticamente valiosa, porque prepara el terreno para ejercicios posteriores donde sí será indispensable volver a recorrer los datos, por ejemplo para ordenarlos, calcular estadísticas adicionales, identificar meses por encima del promedio o comparar varios productos. En consecuencia, la solución no solo resuelve el problema planteado, sino que introduce una forma de organización de datos que será reutilizable en problemas de mayor complejidad [53], [75], [76].

En conjunto, este ejercicio amplía el trabajo realizado con acumuladores y comparaciones en capítulos previos y muestra cómo esas operaciones se integran con una estructura de datos específica. El resultado es una solución compacta, clara y escalable, adecuada para consolidar la comprensión de arreglos unidimensionales en un contexto de análisis comercial sencillo, pero suficientemente representativo de problemas reales en aplicaciones informacionales [23], [24].

10.3.3. Ejercicio 2: Matriz de ingresos por sucursal

Se requiere diseñar un programa que permita registrar los ingresos de varias sucursales durante diferentes periodos y, a partir de esa información, calcular los totales por sucursal, los totales por periodo y el total general acumulado. Cada sucursal aporta un conjunto de valores distribuidos a lo largo del tiempo, por lo que el problema no puede describirse adecuadamente con una sola lista lineal de datos. En este caso, la información posee dos dimensiones simultáneas: una asociada a las sucursales y otra a los periodos. La solución debe conservar esa estructura para que los datos mantengan su significado original, de modo que sea posible responder preguntas como cuánto produjo una sucursal en todos los periodos, cuánto se recaudó en un periodo específico entre todas las sucursales y cuál fue el ingreso global del conjunto analizado [7], [36], [87].

Este ejercicio exige el uso de una **matriz** o **arreglo bidimensional**, ya que no se trabaja únicamente con una colección simple de valores, sino con una tabla de datos donde cada elemento depende de dos coordenadas: la sucursal a la que pertenece y el periodo en el que fue registrado. Si se intentara resolver el problema con variables individuales o con un único arreglo unidimensional, se perdería la relación estructural entre filas y columnas, lo que dificultaría tanto la lectura de los datos como el cálculo de los totales por cada dimensión. La matriz se convierte, por tanto, en la estructura natural del problema, porque permite almacenar una tabla completa y recorrerla de manera ordenada según filas o columnas [28], [35], [87].

El valor formativo del ejercicio radica en que introduce un nivel adicional de organización de datos respecto de los arreglos unidimensionales. Ya no basta con recorrer una sola secuencia de elementos; ahora es necesario comprender que la información está distribuida en dos dimensiones y que los cálculos pueden realizarse en ambas direcciones. Esta exigencia fortalece el manejo de recorridos anidados, la interpretación de índices dobles y la construcción de acumuladores dependientes del contexto de recorrido. Además, el problema aproxima una situación muy frecuente en sistemas de información, donde los datos suelen organizarse en estructuras tabulares para apoyar reportes, análisis y toma de decisiones [1], [74], [75], [76].

Pasos para Desarrollar la Solución

Los pasos para desarrollar la solución algorítmica permiten transformar una tabla de ingresos en una secuencia precisa de operaciones que puedan ejecutarse de manera sistemática. En este ejercicio, la dificultad principal no reside en la suma misma, sino en mantener la relación correcta entre sucursal y periodo mientras se capturan los datos y se calculan los totales.

La organización de la solución debe reflejar que la información está dispuesta en filas y columnas, y que cada una de esas dimensiones puede ser recorrida con un propósito distinto.

1. Análisis del problema. Para comprender el problema conviene partir de tablas pequeñas y calcular manualmente sus totales. Si se dispone de una matriz donde cada fila representa una sucursal y cada columna representa un periodo, entonces el total por sucursal se obtiene sumando horizontalmente los valores de cada fila, mientras que el total por periodo se obtiene sumando verticalmente los valores de cada columna. Esta observación permite descomponer el problema en dos tareas de análisis diferentes aplicadas sobre la misma estructura. La propuesta algorítmica sigue una secuencia clara: primero se define el tamaño de la matriz, luego se capturan los ingresos en cada posición, después se recorren las filas para calcular sus totales y, finalmente, se recorren las columnas para calcular los totales por periodo. A ello se añade el cálculo del total general, que puede obtenerse durante cualquiera de esos recorridos siempre que la acumulación se haga de manera consistente [36], [53], [59].

Ejercicios de análisis manual con valores representativos. Antes de diseñar el algoritmo, resulta recomendable resolver manualmente algunos casos que permitan verificar el cálculo de totales por fila, por columna y el total general.

■ **Caso 1: Tabla rectangular simple**

1. **Datos de entrada**

$$\text{ingresos} = \begin{bmatrix} 100 & 200 & 150 \\ 120 & 180 & 210 \end{bmatrix}$$

2. **Resolución manual**

- Total sucursal 1: $100 + 200 + 150 = 450$
- Total sucursal 2: $120 + 180 + 210 = 510$
- Total periodo 1: $100 + 120 = 220$
- Total periodo 2: $200 + 180 = 380$
- Total periodo 3: $150 + 210 = 360$
- Total general: $450 + 510 = 960$

3. **Resultado esperado**

$$\text{totales por fila} = [450, 510], \quad \text{totales por columna} = [220, 380, 360], \quad \text{total general} = 960$$

4. **Observación.** Este caso muestra con claridad la diferencia entre recorrer horizontalmente y recorrer verticalmente la matriz.

■ **Caso 2: Una sola sucursal**

1. **Datos de entrada**

$$\text{ingresos} = \begin{bmatrix} 300 & 250 & 400 & 350 \end{bmatrix}$$

2. **Resolución manual**

- Total sucursal 1: $300 + 250 + 400 + 350 = 1300$
- Totales por periodo: $[300, 250, 400, 350]$
- Total general: 1300

3. **Resultado esperado**

$$\text{totales por fila} = [1300], \quad \text{totales por columna} = [300, 250, 400, 350], \quad \text{total general} = 1300$$

4. **Observación.** Este caso muestra que el algoritmo debe funcionar incluso cuando solo existe una fila.

■ **Caso 3: Un solo periodo**

1. Datos de entrada

$$\text{ingresos} = \begin{bmatrix} 500 \\ 700 \\ 650 \end{bmatrix}$$

2. Resolución manual

- Totales por sucursal: [500, 700, 650]
- Total periodo 1: $500 + 700 + 650 = 1850$
- Total general: 1850

3. Resultado esperado

totales por fila = [500, 700, 650], totales por columna = [1850], total general = 1850

4. **Observación.** Este caso confirma que el algoritmo debe operar correctamente aunque la matriz tenga una sola columna.

■ Caso 4: Tabla con valores cero

1. Datos de entrada

$$\text{ingresos} = \begin{bmatrix} 0 & 200 \\ 150 & 0 \end{bmatrix}$$

2. Resolución manual

- Total sucursal 1: $0 + 200 = 200$
- Total sucursal 2: $150 + 0 = 150$
- Total periodo 1: $0 + 150 = 150$
- Total periodo 2: $200 + 0 = 200$
- Total general: 350

3. Resultado esperado

totales por fila = [200, 150], totales por columna = [150, 200], total general = 350

4. **Observación.** Este caso permite verificar que la presencia de ceros no altera la lógica de acumulación.

■ Caso 5: Dimensiones no válidas

1. Datos de entrada

- `filas` = 0
- `columnas` = 3

2. **Resolución manual** No es posible construir una matriz si no existe al menos una fila y una columna válidas.

3. Resultado esperado

4. *“Error: la cantidad de sucursales y periodos debe ser mayor que cero. Intente de nuevo.”*

5. **Observación.** Este caso obliga a incorporar una validación inicial del tamaño de la matriz y evita que el algoritmo continúe con una estructura inválida [23], [24].

2. Identificación de datos de entrada. Los datos de entrada corresponden a la información necesaria para construir y procesar la matriz de ingresos.

- **filas (entero):** cantidad de sucursales.

- **columnas (entero):** cantidad de periodos.
- **ingresos[i][j] (real):** ingreso registrado para la sucursal i en el periodo j .

La lectura de estas entradas debe respetar el significado de cada dimensión, porque una inversión de filas y columnas alteraría la interpretación completa del problema. La matriz no es solo una estructura de almacenamiento; es también una representación lógica de la realidad que se desea analizar [1], [28].

3. Identificación de datos de salida. Los datos de salida son los resultados que el algoritmo entrega después del procesamiento.

- **Totales por sucursal (real):** suma horizontal de cada fila.
- **Totales por periodo (real):** suma vertical de cada columna.
- **Total general (real):** suma de todos los elementos de la matriz.

Estas salidas muestran tres niveles de resumen sobre la misma estructura de datos: uno por filas, otro por columnas y otro global. Esta triple lectura de la información refuerza la utilidad de las matrices en problemas tabulares.

4. Identificación de cálculos intermedios (procesos intermedios). Los procesos intermedios son las operaciones necesarias para obtener los resultados finales.

- **Validación inicial:** comprobar que la cantidad de sucursales y periodos sea mayor que cero.
- **Declaración de la matriz:** crear una estructura `double[filas][columnas]`.
- **Captura de datos:** leer cada ingreso y almacenarlo en su posición correspondiente.
- **Acumulación por fila:** sumar los ingresos de cada sucursal.
- **Acumulación por columna:** sumar los ingresos de cada periodo.
- **Acumulación general:** sumar todos los elementos de la matriz.

Estos procesos dejan ver que la matriz no se recorre de una sola manera. Según el objetivo del cálculo, puede recorrerse por filas o por columnas. Esa doble posibilidad constituye una de las ideas centrales del trabajo con arreglos bidimensionales [36], [53].

5. Identificación de cálculos finales (procesos finales). Los procesos finales convierten las acumulaciones realizadas en información interpretable.

- **Obtención del total de cada sucursal:** resultado del acumulador horizontal.
- **Obtención del total de cada periodo:** resultado del acumulador vertical.
- **Obtención del total general:** suma de todos los valores registrados.
- **Presentación de resultados:** impresión de los tres tipos de totales.

La presencia de varios resultados sobre el mismo conjunto de datos permite apreciar que una matriz puede ser utilizada para más de un propósito dentro de una misma solución algorítmica.

6. Identificación de condiciones (decisiones). Las decisiones que controlan el flujo del algoritmo son las siguientes:

- **Validación de dimensiones:** `filas > 0` y `columnas > 0`.
- **Condiciones de los recorridos:** control de los índices de fila y columna durante la captura y el procesamiento.

Estas decisiones aseguran que la matriz exista y que sus recorridos se mantengan dentro de sus límites válidos [35], [59].

7. Identificación de repeticiones o iteraciones. El algoritmo utiliza varias estructuras repetitivas claramente diferenciadas.

- **Repetición para validar dimensiones:** mientras filas o columnas no sean válidas.
- **Recorrido de captura:** ciclos anidados para leer cada ingreso de la matriz.
- **Recorrido por filas:** ciclos anidados para calcular totales por sucursal.

- **Recorrido por columnas:** ciclos anidados para calcular totales por periodo.

La utilización de más de un recorrido sobre la misma matriz enseña que una estructura bidimensional puede reutilizarse en distintas fases del problema, lo que fortalece la comprensión del procesamiento tabular [74], [75], [76].

8. Condiciones para finalizar el algoritmo. El algoritmo finaliza cuando:

- se han ingresado dimensiones válidas;
- todos los ingresos han sido leídos y almacenados;
- se han completado los recorridos por filas y por columnas;
- se han obtenido y mostrado los totales por sucursal, por periodo y el total general.

La finalización del proceso depende, por tanto, del cumplimiento ordenado de varias etapas sucesivas. Esto convierte al ejercicio en un ejemplo adecuado para mostrar cómo una misma matriz puede alimentar distintas fases de procesamiento dentro de una única solución algorítmica.

Diseño de la Propuesta de Solución

El diseño de la propuesta de solución organiza el problema en cuatro momentos: validación de dimensiones, captura de ingresos, cálculo de totales por sucursal y cálculo de totales por periodo. Esta organización hace visible que la misma matriz puede recorrerse con propósitos diferentes según la orientación del análisis. Cuando el recorrido se hace por filas, el interés se concentra en cada sucursal; cuando se hace por columnas, el foco se desplaza hacia cada periodo. Esta separación mejora la claridad del procedimiento y facilita la verificación del resultado [59], [72].

El resultado del diseño es el Algoritmo 10.15, que presenta una propuesta estructurada en pseudocódigo UPSAM. En él se distinguen explícitamente la validación inicial, la construcción de la matriz, el ingreso de datos y los dos recorridos principales para calcular totales. Esta organización evita confundir el proceso de captura con el proceso de análisis y muestra que una estructura bidimensional puede servir para producir varios resúmenes a partir del mismo conjunto de datos [53], [74].

```

1      Algoritmo IngresosPorSucursalMatriz
2      1.      Var:
3      1.1.      filas, columnas, i, j: entero
4      1.2.      ingresos[] []: real
5      1.3.      sumaFila, sumaColumna, totalGeneral: real
6
7      2.      Escribir "Digite la cantidad de sucursales:"
8      3.      Leer filas
9      4.      Escribir "Digite la cantidad de periodos:"
10     5.      Leer columnas
11     6.      Mientras filas <= 0 O columnas <= 0 Hacer
12     6.1.      Escribir "Error: la cantidad de sucursales y periodos debe
↪ ser mayor que cero."
13     6.2.      Escribir "Digite la cantidad de sucursales:"
14     6.3.      Leer filas
15     6.4.      Escribir "Digite la cantidad de periodos:"
16     6.5.      Leer columnas

```

```

17      7.      FinMientras
18
19      8.      totalGeneral <- 0
20
21      9.      Para i <- 1 Hasta filas Hacer
22      9.1.      Para j <- 1 Hasta columnas Hacer
23      9.1.1.      Escribir "Digite el ingreso de la sucursal ", i, " en el
↪ periodo ", j, ":"
24      9.1.2.      Leer ingresos[i-1][j-1]
25      9.1.3.      totalGeneral <- totalGeneral + ingresos[i-1][j-1]
26      9.2.      FinPara
27      10.     FinPara
28
29      11.     Para i <- 1 Hasta filas Hacer
30      11.1.     sumaFila <- 0
31      11.2.     Para j <- 1 Hasta columnas Hacer
32      11.2.1.     sumaFila <- sumaFila + ingresos[i-1][j-1]
33      11.3.     FinPara
34      11.4.     Escribir "Total sucursal ", i, ": ", sumaFila
35      12.     FinPara
36
37      13.     Para j <- 1 Hasta columnas Hacer
38      13.1.     sumaColumna <- 0
39      13.2.     Para i <- 1 Hasta filas Hacer
40      13.2.1.     sumaColumna <- sumaColumna + ingresos[i-1][j-1]
41      13.3.     FinPara
42      13.4.     Escribir "Total periodo ", j, ": ", sumaColumna
43      14.     FinPara
44
45      15.     Escribir "Total general: ", totalGeneral
46      FinAlgoritmo

```

Listing 10.15: Cálculo de ingresos por sucursal y por periodo mediante una matriz

La solución en Java, presentada en el Listado de código 10.16, utiliza estrategias propias del lenguaje que hacen natural la implementación de matrices. La declaración `double[][] ingresos = new double[filas][columnas]`; permite crear un arreglo bidimensional cuyo tamaño se determina en tiempo de ejecución. La propiedad `length` de Java facilita conocer el número de filas y, dentro de cada fila, el número de columnas, lo cual hace más seguro el recorrido de la estructura. La captura y el procesamiento se realizan mediante ciclos `for` anidados, una técnica característica cuando se trabaja con tablas

de datos. Además, la lectura por teclado se apoya en la clase `Scanner`, ampliamente utilizada en programas introductorios en Java [7], [71], [87].

```
1  import java.util.Scanner;
2
3  public class IngresosSucursal {
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6
7          // Lee la cantidad de sucursales y periodos
8          System.out.print("Ingrese la cantidad de sucursales: ");
9          int filas = sc.nextInt();
10
11         System.out.print("Ingrese la cantidad de periodos: ");
12         int columnas = sc.nextInt();
13
14         // Valida que ambas dimensiones sean correctas
15         while (filas <= 0 || columnas <= 0) {
16             System.out.println("Error: la cantidad de sucursales y
↪ periodos debe ser mayor que cero.");
17             System.out.print("Ingrese la cantidad de sucursales: ");
18             filas = sc.nextInt();
19             System.out.print("Ingrese la cantidad de periodos: ");
20             columnas = sc.nextInt();
21         }
22
23         // Declara la matriz de ingresos
24         double[][] ingresos = new double[filas][columnas];
25
26         double totalGeneral = 0;
27
28         // Captura de datos y acumulación del total general
29         for (int i = 0; i < filas; i++) {
30             for (int j = 0; j < columnas; j++) {
31                 System.out.print("Ingreso [" + i + "][" + j + "]: ");
32                 ingresos[i][j] = sc.nextDouble();
33                 totalGeneral += ingresos[i][j];
```

```
34         }
35     }
36
37     // Cálculo de totales por fila (sucursal)
38     for (int i = 0; i < filas; i++) {
39         double sumaFila = 0;
40         for (int j = 0; j < columnas; j++) {
41             sumaFila += ingresos[i][j];
42         }
43         System.out.println("Total sucursal " + (i + 1) + ": " +
↪ sumaFila);
44     }
45
46     // Cálculo de totales por columna (periodo)
47     for (int j = 0; j < columnas; j++) {
48         double sumaColumna = 0;
49         for (int i = 0; i < filas; i++) {
50             sumaColumna += ingresos[i][j];
51         }
52         System.out.println("Total periodo " + (j + 1) + ": " +
↪ sumaColumna);
53     }
54
55     // Presenta el total general
56     System.out.println("Total general: " + totalGeneral);
57
58     sc.close();
59 }
60 }
```

Listing 10.16: Matriz de ingresos por sucursal en Java

El algoritmo del Listado de código 10.15 y el programa del Listado de código 10.16 mantienen la misma lógica general, aunque se expresan en niveles distintos. El pseudocódigo resalta la secuencia conceptual del proceso, mientras que Java exige concretar tipos, sintaxis de arreglos, ciclos y objetos de entrada. Esta correspondencia resulta importante porque permite reconocer que la implementación no es una invención aislada, sino la traducción directa de una estructura algorítmica previamente definida [35], [59].

Un aspecto esencial de la solución es la interpretación correcta de los dos índices de la matriz. El primer índice representa la fila, asociada en este problema a la sucursal; el segundo índice representa la columna, asociada al periodo. Esta convención

debe mantenerse de manera consistente a lo largo de toda la solución. Si se invirtiera el sentido de los índices, la matriz seguiría existiendo como estructura, pero la interpretación del problema quedaría alterada. Por ello, el significado semántico de cada dimensión es tan importante como la sintaxis del acceso [1], [28].

También conviene advertir que la solución utiliza más de un recorrido sobre la misma matriz. Durante la captura se acumula el total general, pero los totales por fila y por columna se obtienen mediante recorridos adicionales. Esta decisión hace más visible la lógica del análisis y evita mezclar demasiadas responsabilidades en una sola etapa. En ejercicios iniciales, esta separación favorece la comprensión, porque permite observar con claridad qué recorre cada ciclo y qué resultado produce [36], [53].

La estructura de la solución prepara además el terreno para problemas posteriores de mayor complejidad. A partir de la misma matriz podrían incorporarse cálculos de promedios por sucursal, periodos con máximo ingreso, comparaciones entre sucursales o análisis de crecimiento temporal. En este sentido, el ejercicio no solo enseña a declarar y recorrer una matriz, sino que introduce una forma de pensamiento tabular indispensable en el trabajo con datos organizacionales, estadísticos y financieros [23], [24].

En conjunto, el problema muestra que los arreglos bidimensionales no son simplemente una ampliación técnica de los arreglos unidimensionales, sino una herramienta necesaria cuando los datos deben conservar relaciones en dos direcciones simultáneas. La combinación de matriz, recorridos anidados y acumuladores hace visible una de las ideas más importantes de este capítulo: la organización de los datos influye directamente en la forma en que puede resolverse un problema [75], [76], [87].

10.3.4. Ejercicio 3: Registro de clientes

Se requiere diseñar un programa que permita registrar información básica de varios clientes dentro de un sistema informacional. Cada cliente debe quedar representado mediante un conjunto de atributos relacionados, tales como identificación, nombre, edad, saldo disponible y estado de actividad. El programa debe permitir capturar esos datos, almacenarlos de forma organizada y, posteriormente, presentar un resumen estructurado de la información registrada. A diferencia de ejercicios centrados únicamente en datos numéricos homogéneos, este problema exige trabajar con información heterogénea, ya que los atributos de un cliente no pertenecen todos al mismo tipo: algunos son numéricos, otros textuales y otros lógicos. Esta característica convierte al problema en un caso apropiado para introducir estructuras compuestas, puesto que la entidad que se desea modelar no puede describirse de manera adecuada mediante una sola variable ni mediante un arreglo simple de valores del mismo tipo [1], [28], [87].

Este problema resulta especialmente importante porque aproxima una situación característica en el desarrollo de sistemas de información: la necesidad de representar entidades del mundo real mediante estructuras de datos con múltiples atributos. En un sistema comercial o administrativo, un cliente no se reduce a un valor aislado, sino que se describe mediante un conjunto coherente de campos que deben conservarse y manipularse de forma conjunta. La construcción de esa unidad lógica es una idea central en la modelación de información, ya que permite que el programa refleje con mayor fidelidad la estructura del dominio del problema. Por ello, este ejercicio no solo amplía el tratamiento de arreglos y recorridos estudiados previamente, sino que también introduce una forma diferente de organizar datos: agrupar varios atributos en una sola entidad estructurada [27], [28], [51].

La relevancia formativa del ejercicio se aprecia en que obliga a distinguir entre dos niveles de organización. Por un lado, cada cliente está compuesto por varios campos relacionados; por otro, puede existir un conjunto de clientes dentro del mismo programa. Esa doble organización prepara el tránsito desde estructuras simples hacia soluciones más cercanas a la realidad de los sistemas informacionales. El problema también refuerza la comprensión de captura de datos, validación elemental, acceso a campos y presentación de resultados, pero ahora bajo una lógica de entidades compuestas en lugar de listas simples de números o cadenas [74], [75], [76].

Pasos para Desarrollar la Solución

Los pasos para desarrollar la solución algorítmica permiten convertir una entidad organizacional real, como un cliente, en una estructura formal que pueda manipularse dentro de un programa. En este ejercicio, la dificultad principal no reside en una fórmula matemática compleja, sino en identificar con claridad qué atributos definen al cliente, cómo deben almacenarse y de qué manera pueden presentarse después de su captura. La solución debe organizar la información de forma que cada dato conserve su significado y permanezca asociado a la misma entidad durante todo el procesamiento.

1. Análisis del problema. Para comprender el problema conviene comenzar identificando qué significa “registrar un

cliente”. Registrar no consiste solo en leer datos, sino en construir una representación coherente de una entidad mediante campos relacionados. Si se capturan por separado un identificador, un nombre, una edad, un saldo y un estado, esos valores deben permanecer unidos para que el programa sepa que todos pertenecen al mismo cliente. Por ello, el problema exige una estructura compuesta que agrupe atributos heterogéneos bajo una sola unidad lógica. El procedimiento algorítmico sigue una secuencia clara: primero se determina cuántos clientes se van a registrar, luego se capturan los datos de cada uno, después se construye la representación estructurada del cliente y finalmente se presentan los resultados organizados. Esta descomposición facilita el aprendizaje porque separa con nitidez la etapa de definición de la entidad, la etapa de captura y la etapa de presentación [1], [28], [59].

Ejercicios de análisis manual con valores representativos. Antes de diseñar el algoritmo, resulta recomendable resolver manualmente algunos casos que permitan verificar la validez de la estructura propuesta y la forma en que se deben interpretar los datos registrados.

■ **Caso 1: Registro normal de un cliente**

1. **Datos de entrada**

- `id = 101`
- `nombre = “Ana Torres”`
- `edad = 30`
- `saldo = 2450.75`
- `activo = verdadero`

2. **Resolución manual** Se construye una entidad cliente que agrupa los cinco atributos bajo una misma unidad.

3. **Resultado esperado** El sistema debe mostrar un registro coherente con todos los campos asociados al mismo cliente.

4. **Observación.** Este caso representa el escenario normal y permite comprobar la agrupación correcta de atributos heterogéneos.

■ **Caso 2: Cliente con saldo igual a cero**

1. **Datos de entrada**

- `id = 102`
- `nombre = “Luis Pérez”`
- `edad = 41`
- `saldo = 0.0`
- `activo = verdadero`

2. **Resolución manual** El cliente sigue siendo válido, aunque su saldo sea nulo.

3. **Resultado esperado** La información debe mostrarse correctamente, conservando el valor cero del saldo.

4. **Observación.** Este caso permite comprobar que un saldo cero no debe confundirse con un dato faltante o inválido.

■ **Caso 3: Cliente inactivo**

1. **Datos de entrada**

- `id = 103`
- `nombre = “María Gómez”`
- `edad = 27`
- `saldo = 890.40`
- `activo = falso`

2. **Resolución manual** La entidad cliente sigue siendo válida, pero el estado lógico indica que no se encuentra activa.

3. **Resultado esperado** El sistema debe mostrar el cliente con estado inactivo.

4. **Observación.** Este caso confirma la importancia de los campos booleanos para representar condiciones del dominio.

■ **Caso 4: Edad inválida**

1. **Datos de entrada**

- `id = 104`
- `nombre = "Carlos Ruiz"`
- `edad = -5`
- `saldo = 1500.00`
- `activo = verdadero`

2. **Resolución manual** La edad no puede ser negativa, por lo que el dato debe rechazarse o solicitarse nuevamente.

3. **Resultado esperado** *"Error: la edad debe ser mayor o igual que cero."*

4. **Observación.** Este caso obliga a incorporar validación sobre los campos y muestra que una estructura compuesta también debe proteger la integridad de sus datos [23], [24].

■ **Caso 5: Cantidad no válida de clientes**

1. **Datos de entrada**

- `n = 0`

2. **Resolución manual** No es posible registrar clientes si la cantidad indicada es cero o negativa.

3. **Resultado esperado** *"Error: la cantidad de clientes debe ser mayor que cero. Intente de nuevo."*

4. **Observación.** Este caso obliga a validar la cantidad de registros antes de iniciar la captura de datos.

2. Identificación de datos de entrada. Los datos de entrada corresponden a la información necesaria para construir cada registro de cliente.

- **n (entero):** cantidad de clientes que serán registrados.
- **id (entero):** identificador único del cliente.
- **nombre (cadena):** nombre del cliente.
- **edad (entero):** edad del cliente.
- **saldo (real):** saldo disponible.
- **activo (lógico):** estado del cliente.

La variedad de tipos de entrada es precisamente lo que justifica el uso de una estructura compuesta. A diferencia de un arreglo homogéneo, aquí los datos no son todos del mismo tipo ni cumplen la misma función dentro del problema [27], [28].

3. Identificación de datos de salida. Los datos de salida son los resultados que el programa debe mostrar una vez concluido el registro.

- **Registro de cada cliente:** conjunto de atributos mostrados de forma estructurada.
- **Resumen de clientes activos o inactivos:** interpretación del campo lógico.
- **Listado completo:** presentación ordenada de todos los clientes registrados.

La salida no consiste en un único valor numérico, sino en la presentación organizada de entidades completas, lo que distingue este ejercicio de los problemas centrados exclusivamente en cálculos.

4. Identificación de cálculos intermedios (procesos intermedios). Los procesos intermedios son las operaciones necesarias para construir los registros y garantizar la consistencia de sus datos.

- **Validación inicial de n:** comprobar que la cantidad de clientes sea mayor que cero.
- **Definición del tipo compuesto:** establecer los campos que representarán al cliente.
- **Captura de datos:** leer los atributos de cada cliente.
- **Validación de campos:** verificar edad, saldo u otras restricciones básicas.
- **Construcción del registro:** agrupar los datos dentro de la misma entidad.

Estos procesos muestran que el problema no se resuelve solo leyendo valores, sino organizándolos bajo una estructura con sentido semántico dentro del dominio del problema [1], [51].

5. Identificación de cálculos finales (procesos finales). Los procesos finales son aquellos que producen la representación visible del resultado.

- **Construcción definitiva del cliente:** una vez validados los datos.
- **Presentación del registro:** impresión de cada campo del cliente.
- **Presentación del conjunto de clientes:** recorrido y visualización del total registrado.

El objetivo final es conservar la relación entre los campos de cada cliente y mostrar esa relación de forma ordenada y comprensible.

6. Identificación de condiciones (decisiones). Las decisiones que controlan el flujo del algoritmo son las siguientes:

- **Validación de cantidad de clientes:** $n > 0$.
- **Validación de edad:** $\text{edad} \geq 0$.
- **Validación de saldo:** $\text{saldo} \geq 0$.

Estas decisiones permiten separar la fase de captura de la fase de aceptación definitiva del registro y refuerzan la idea de integridad de datos [35], [59].

7. Identificación de repeticiones o iteraciones. El algoritmo contiene varias repeticiones claramente distinguibles.

- **Repetición para validar n:** mientras no sea mayor que cero.
- **Recorrido de captura:** para registrar cada cliente.
- **Recorrido de presentación:** para mostrar todos los clientes almacenados.

Estas iteraciones muestran que los registros no reemplazan a los arreglos, sino que pueden complementarse con ellos cuando existe un conjunto de entidades del mismo tipo [74], [75].

8. Condiciones para finalizar el algoritmo. El algoritmo finaliza cuando:

- se ha ingresado una cantidad válida de clientes;
- todos los registros han sido capturados y validados;
- se ha presentado la información completa almacenada;
- el recorrido del conjunto de clientes ha concluido.

La finalización depende, por tanto, de la culminación ordenada de la definición, captura, validación y presentación de entidades compuestas.

Propuesta de Solución

La propuesta de solución organiza el problema en tres momentos claramente diferenciados: definición del tipo de dato que representará al cliente, captura y validación de la información, y presentación del conjunto registrado. Esta organización hace visible que el objetivo no es únicamente almacenar datos, sino construir una representación coherente de una entidad real. El proceso separa con claridad la estructura del cliente de la colección de clientes, lo que permite comprender que una cosa es la definición de la entidad y otra distinta el conjunto de instancias registradas [59], [72].

El Algoritmo 10.17 presenta una propuesta estructurada en pseudocódigo UPSAM. En él se distinguen explícitamente la validación inicial de la cantidad de clientes, la captura de los atributos de cada uno, la verificación de campos básicos y el almacenamiento de la información dentro de una estructura compuesta. Esta organización facilita el razonamiento, porque evita confundir el registro de una entidad con el recorrido del conjunto total de entidades [53], [74].

```

1      Algoritmo RegistroClientes
2      1.      Var:
3      1.1.      n, i: entero
4      1.2.      clientes[]: registro
5      1.3.      id, edad: entero
6      1.4.      nombre: cadena
7      1.5.      saldo: real
8      1.6.      activo: logico
9
10     2.      Escribir "Ingrese la cantidad de clientes:"
11     3.      Leer n
12     4.      Mientras n <= 0 Hacer
13     4.1.      Escribir "Error: la cantidad de clientes debe ser mayor que
↪ cero."
14     4.2.      Escribir "Ingrese la cantidad de clientes:"
15     4.3.      Leer n
16     5.      FinMientras
17
18     6.      Para i <- 1 Hasta n Hacer
19     6.1.      Escribir "Ingrese el id del cliente ", i, ":"
20     6.2.      Leer id
21     6.3.      Escribir "Ingrese el nombre del cliente ", i, ":"
22     6.4.      Leer nombre
23     6.5.      Escribir "Ingrese la edad del cliente ", i, ":"
24     6.6.      Leer edad
25     6.7.      Mientras edad < 0 Hacer
26     6.7.1.      Escribir "Error: la edad debe ser mayor o igual que
↪ cero."

```

```

27     6.7.2.      Escribir "Ingrese la edad del cliente ", i, ":"
28     6.7.3.      Leer edad
29     6.8.        FinMientras
30     6.9.        Escribir "Ingrese el saldo del cliente ", i, ":"
31     6.10.       Leer saldo
32     6.11.       Mientras saldo < 0 Hacer
33     6.11.1.     Escribir "Error: el saldo debe ser mayor o igual que
↪     cero."
34     6.11.2.     Escribir "Ingrese el saldo del cliente ", i, ":"
35     6.11.3.     Leer saldo
36     6.12.       FinMientras
37     6.13.       Escribir "Ingrese el estado activo (verdadero/falso):"
38     6.14.       Leer activo
39     6.15.       clientes[i-1] <- registro(id, nombre, edad, saldo, activo)
40     7.          FinPara
41
42     8.          Para i <- 1 Hasta n Hacer
43     8.1.        Escribir "Cliente ", i, ": ", clientes[i-1]
44     9.          FinPara
45     FinAlgoritmo

```

Listing 10.17: Registro de clientes mediante una estructura compuesta

La solución en Java, presentada en el Listado de código 10.18, se formula mediante **record**, una construcción del lenguaje diseñada precisamente para representar datos inmutables compuestos por varios componentes relacionados. El uso de **record** resulta especialmente adecuado en este ejercicio porque el interés principal recae en la representación de la entidad y no en una lógica compleja de comportamiento. Java genera automáticamente el constructor, los métodos de acceso y una representación textual básica, lo que simplifica notablemente el código y permite concentrar la atención en la estructura del dato y en su uso dentro del programa [35], [87].

```

1     import java.util.Scanner;
2
3     public class SistemaClientes {
4
5         // Definición del registro (record) que representa a un cliente
6         record Cliente(int id, String nombre, int edad, double saldo,
↪         boolean activo) {}
7
8         public static void main(String[] args) {

```

```
9      Scanner sc = new Scanner(System.in);
10
11      // Lee la cantidad de clientes que se registrarán
12      System.out.print("Ingrese la cantidad de clientes: ");
13      int n = sc.nextInt();
14      sc.nextLine(); // Limpia el salto de línea pendiente
15
16      // Valida que la cantidad de clientes sea correcta
17      while (n <= 0) {
18          System.out.println("Error: la cantidad de clientes debe ser
↪ mayor que cero.");
19          System.out.print("Ingrese la cantidad de clientes: ");
20          n = sc.nextInt();
21          sc.nextLine();
22      }
23
24      // Declara un arreglo de registros Cliente
25      Cliente[] clientes = new Cliente[n];
26
27      // Captura de datos de cada cliente
28      for (int i = 0; i < n; i++) {
29          System.out.println("\nCliente " + (i + 1));
30
31          System.out.print("Id: ");
32          int id = sc.nextInt();
33          sc.nextLine();
34
35          System.out.print("Nombre: ");
36          String nombre = sc.nextLine();
37
38          System.out.print("Edad: ");
39          int edad = sc.nextInt();
40          while (edad < 0) {
41              System.out.println("Error: la edad debe ser mayor o
↪ igual que cero.");
42              System.out.print("Edad: ");
43              edad = sc.nextInt();
```

```
44         }
45
46         System.out.print("Saldo: ");
47         double saldo = sc.nextDouble();
48         while (saldo < 0) {
49             System.out.println("Error: el saldo debe ser mayor o
↳ igual que cero.");
50             System.out.print("Saldo: ");
51             saldo = sc.nextDouble();
52         }
53
54         System.out.print("Activo (true/false): ");
55         boolean activo = sc.nextBoolean();
56         sc.nextLine();
57
58         // Construye el registro y lo almacena en el arreglo
59         clientes[i] = new Cliente(id, nombre, edad, saldo, activo);
60     }
61
62     // Presenta los clientes registrados
63     System.out.println("\n=== CLIENTES REGISTRADOS ===");
64     for (int i = 0; i < clientes.length; i++) {
65         System.out.println("Cliente " + (i + 1));
66         System.out.println("Id: " + clientes[i].id());
67         System.out.println("Nombre: " + clientes[i].nombre());
68         System.out.println("Edad: " + clientes[i].edad());
69         System.out.println("Saldo: " + clientes[i].saldo());
70         System.out.println("Estado: " + (clientes[i].activo() ?
↳ "Activo" : "Inactivo"));
71         System.out.println();
72     }
73
74     sc.close();
75 }
76 }
```

Listing 10.18: Registro de clientes en Java usando record

El algoritmo del Listado de código 10.17 y el programa del Listado de código 10.18 conservan la misma lógica general, aunque se expresan en niveles de formalización distintos. El pseudocódigo resalta la secuencia conceptual del proceso: definir, capturar, validar y presentar. Java exige traducir esa secuencia a una sintaxis precisa mediante tipos, estructuras de datos, lectura desde teclado y objetos inmutables contruidos con **record**. La correspondencia entre ambas formulaciones permite apreciar que la implementación no es un añadido arbitrario, sino la materialización directa del razonamiento algorítmico previo [35], [59].

Conviene observar que la solución utiliza un arreglo de registros y no solo un único registro. Esta decisión es importante porque el problema no consiste en representar un solo cliente, sino un conjunto de clientes de la misma clase lógica. Así, el **record** resuelve la necesidad de agrupar los atributos de una entidad, mientras que el arreglo resuelve la necesidad de conservar varias entidades del mismo tipo. La combinación de ambas estructuras muestra con claridad cómo los datos compuestos y las colecciones pueden trabajar de forma complementaria dentro de una misma solución [7], [87].

También es importante notar que el acceso a los componentes de un **record** en Java no se realiza escribiendo campos públicos de la forma clásica, sino mediante métodos de acceso generados automáticamente, como `id()`, `nombre()` o `saldo()`. Esta característica refuerza la consistencia del lenguaje y reduce el código repetitivo, pues ya no es necesario escribir manualmente constructores, métodos **getter** o implementaciones básicas de representación. En ejercicios introductorios, esta simplificación ayuda a concentrar el esfuerzo cognitivo en la lógica del problema y en la estructura de la información, sin distraer la atención en detalles ceremoniales del lenguaje [51], [87].

La validación aplicada a la edad y al saldo muestra que una estructura compuesta no solo organiza información, sino que también debe preservar su coherencia. Un cliente con edad negativa o saldo inconsistente no representa un estado válido del dominio. Por esa razón, aun cuando el ejercicio se mantenga en un nivel inicial, ya introduce una idea central en el desarrollo de software: la organización de datos y la validación de datos deben ir de la mano. Un registro correctamente diseñado, pero alimentado con valores inválidos, no cumple adecuadamente su función dentro del sistema [23], [24], [27].

Registros (**record**) en Java

En Java, los **record** constituyen una forma especializada de declarar tipos de datos orientados a representar entidades compuestas de manera clara, compacta y consistente. Su propósito principal es encapsular un conjunto de atributos relacionados dentro de una unidad lógica, reduciendo la necesidad de escribir código repetitivo. Esta construcción resulta especialmente útil cuando el problema se centra en la organización de datos y no en la definición de comportamientos complejos, como ocurre en el registro de clientes [35], [87].

A diferencia de una clase tradicional, un **record** permite definir todos los componentes de una entidad en su encabezado, a partir del cual Java genera automáticamente el constructor, los métodos de acceso (**getters**), y métodos auxiliares como `toString()`, `equals()` y `hashCode()`. Esta automatización mejora la legibilidad del código y permite concentrar la atención en la estructura semántica del problema en lugar de en detalles sintácticos repetitivos [51], [87].

En el contexto de este ejercicio, el uso de **record** es adecuado porque el cliente se define fundamentalmente por sus atributos: identificador, nombre, edad, saldo y estado. No se requiere aún una lógica compleja asociada a la entidad, sino una representación fiel y estructurada de sus datos. Por ello, los registros permiten modelar directamente la entidad cliente de manera precisa y expresiva [1], [28].

Para una correcta organización del programa, es recomendable separar la definición del **record** en un archivo independiente, lo cual refleja una práctica habitual en el desarrollo de software: definir primero las entidades del dominio y luego utilizarlas en otras clases del sistema.

El Listado de código 10.19 muestra el archivo que contiene exclusivamente la definición del **record** `Cliente`.

```
1 // Archivo: Cliente.java
2
3 public record Cliente(
4     int id,
5     String nombre,
6     int edad,
7     double saldo,
```



```
8         boolean activo
9     ) {}
```

Listing 10.19: Archivo `Cliente.java` con la definición del `record`

En este archivo se define la estructura de la entidad cliente. Cada componente del `record` representa un atributo del cliente, y Java genera automáticamente los métodos de acceso correspondientes (`id()`, `nombre()`, etc.). Esta definición constituye la base sobre la cual se construirá el resto del programa.

El Listado de código 10.20 presenta el archivo principal que implementa la solución completa utilizando el `record` definido previamente.

```
1      // Archivo: SistemaClientes.java
2
3      import java.util.Scanner;
4
5      public class SistemaClientes {
6
7          public static void main(String[] args) {
8              Scanner sc = new Scanner(System.in);
9
10             // Lee la cantidad de clientes
11             System.out.print("Ingrese la cantidad de clientes: ");
12             int n = sc.nextInt();
13             sc.nextLine();
14
15             // Valida que la cantidad sea correcta
16             while (n <= 0) {
17                 System.out.println("Error: la cantidad de clientes debe
↪ ser mayor que cero.");
18                 System.out.print("Ingrese la cantidad de clientes: ");
19                 n = sc.nextInt();
20                 sc.nextLine();
21             }
22
23             // Declara un arreglo de registros Cliente
24             Cliente[] clientes = new Cliente[n];
25
26             // Captura de datos
27             for (int i = 0; i < n; i++) {
```

```

28         System.out.println("\nCliente " + (i + 1));
29
30         System.out.print("Id: ");
31         int id = sc.nextInt();
32         sc.nextLine();
33
34         System.out.print("Nombre: ");
35         String nombre = sc.nextLine();
36
37         System.out.print("Edad: ");
38         int edad = sc.nextInt();
39         while (edad < 0) {
40             System.out.println("Error: la edad debe ser mayor o
↪ igual que cero.");
41             System.out.print("Edad: ");
42             edad = sc.nextInt();
43         }
44
45         System.out.print("Saldo: ");
46         double saldo = sc.nextDouble();
47         while (saldo < 0) {
48             System.out.println("Error: el saldo debe ser mayor
↪ o igual que cero.");
49             System.out.print("Saldo: ");
50             saldo = sc.nextDouble();
51         }
52
53         System.out.print("Activo (true/false): ");
54         boolean activo = sc.nextBoolean();
55         sc.nextLine();
56
57         // Construye el registro
58         clientes[i] = new Cliente(id, nombre, edad, saldo,
↪ activo);
59     }
60
61     // Presentación de resultados

```

```

62         System.out.println("\n=== CLIENTES REGISTRADOS ===");
63         for (int i = 0; i < clientes.length; i++) {
64             System.out.println("Cliente " + (i + 1));
65             System.out.println("Id: " + clientes[i].id());
66             System.out.println("Nombre: " + clientes[i].nombre());
67             System.out.println("Edad: " + clientes[i].edad());
68             System.out.println("Saldo: " + clientes[i].saldo());
69             System.out.println("Estado: " + (clientes[i].activo() ?
↪ "Activo" : "Inactivo"));
70             System.out.println();
71         }
72
73         sc.close();
74     }
75 }

```

Listing 10.20: Archivo `SistemaClientes.java` que utiliza el record `Cliente`

La separación en dos archivos permite distinguir claramente entre la definición del modelo de datos y su utilización dentro de la lógica del programa. Esta práctica mejora la organización del código, facilita su mantenimiento y refleja una estructura común en aplicaciones reales, donde las entidades del dominio se definen de manera independiente de los procesos que las manipulan [23], [24].

El uso conjunto de `record` y arreglos muestra cómo pueden combinarse estructuras compuestas y colecciones homogéneas. Cada elemento del arreglo es una instancia del `record` `Cliente`, lo que permite manejar múltiples entidades sin perder la coherencia interna de sus atributos. Esta combinación constituye un paso fundamental en la evolución desde estructuras simples hacia modelos de datos más cercanos a los sistemas de información reales [1], [28].

En conjunto, los registros en Java proporcionan una herramienta poderosa y concisa para modelar entidades, mientras que su integración con estructuras de control y arreglos permite construir soluciones completas que mantienen tanto la claridad conceptual como la organización estructural del problema.

10.3.5. Ejercicio 4: Implementación en Java del Algoritmo 4.9.9

Este ejercicio consiste en desarrollar un programa que permita procesar un conjunto de valores numéricos reales ingresados por el usuario para calcular tres medidas estadísticas fundamentales: el promedio, la varianza y la desviación estándar. El procedimiento requiere primero determinar cuántos datos serán analizados, validar que exista una cantidad suficiente de observaciones y luego capturar cada uno de los valores para almacenarlos. A partir de esta información, el programa debe calcular la media aritmética y, posteriormente, evaluar la dispersión de los datos respecto de dicha media, lo que conduce al cálculo de la varianza y la desviación estándar [35], [53].

El ejercicio sobre el cálculo del promedio, la varianza y la desviación estándar requiere una solución que conserve todos los datos ingresados para reutilizarlos en más de una etapa del procesamiento. Esa necesidad hace que un arreglo sea indispensable, porque la media se obtiene a partir de la suma total de los valores, mientras que la varianza exige volver a recorrer exactamente el mismo conjunto para calcular las desviaciones cuadráticas respecto del promedio. En consecuencia, la estrategia no puede limitarse a una lectura directa y a una fórmula inmediata, sino que debe organizar el procedimiento en fases sucesivas: validación de la cantidad de datos, carga del arreglo, primer recorrido para la suma y el promedio, segundo recorrido para la dispersión y cálculo final de la desviación estándar [7], [35], [53].

La solución en Java, presentada en el Listado de código 10.21, traduce esa estrategia a una secuencia concreta de instrucciones apoyadas en recursos propios del lenguaje. Se utiliza `Scanner` para la lectura de datos desde teclado, un arreglo de tipo `double[]` para conservar las observaciones, variables acumuladoras para la suma total y la suma de cuadrados, y el método `Math.sqrt()` para obtener la raíz cuadrada de la varianza. La estructura general del programa refleja con claridad la dependencia lógica entre los resultados: primero debe existir el promedio para que luego puedan calcularse las desviaciones cuadráticas, y solo después de ello es posible obtener la varianza y la desviación estándar [71], [87].

```

1      import java.util.Scanner;
2
3      public class EstadisticasBasicasVector {
4          public static void main(String[] args) {
5              Scanner sc = new Scanner(System.in);
6
7              int n;
8
9              //
10             ↪ -----
11                 // Validación de la cantidad de datos
12                 // Se exige al menos 2 valores para que el
13                 ↪ análisis tenga sentido
14                 //
15                 ↪ -----
16                 System.out.print("Ingrese la cantidad de
17                 ↪ datos (n): ");
18                 n = sc.nextInt();
19
20                 while (n < 2) {
21                     System.out.println("Error: la cantidad
22                     ↪ de valores debe ser mayor o igual que 2. Intente de nuevo.");
23                     System.out.print("Ingrese la cantidad
24                     ↪ de datos (n): ");
25                     n = sc.nextInt();
26                 }
27
28                 //
29                 ↪ -----
30                 // Declaración del arreglo y de las
31                 ↪ variables acumuladoras

```

```

24                                     //
    ↪ -----
25                                     double[] datos = new double[n];
26                                     double suma = 0;
27                                     double sumaCuadrados = 0;
28                                     double promedio;
29                                     double varianza;
30                                     double desviacion;
31
32                                     //
    ↪ -----
33                                     // Primer recorrido:
34                                     // lectura de los datos y acumulación de la
    ↪ suma total
35                                     //
    ↪ -----
36                                     for (int i = 0; i < n; i++) {
37                                         System.out.print("Ingrese el dato " +
    ↪ (i + 1) + ": ");
38                                         datos[i] = sc.nextDouble();
39                                         suma += datos[i];
40                                     }
41
42                                     //
    ↪ -----
43                                     // Cálculo del promedio
44                                     //
    ↪ -----
45                                     promedio = suma / n;
46
47                                     //
    ↪ -----
48                                     // Segundo recorrido:
49                                     // acumulación de las diferencias
    ↪ cuadráticas respecto del promedio
50                                     //
    ↪ -----

```

```

51         for (int i = 0; i < n; i++) {
52             sumaCuadrados += Math.pow(datos[i] -
↪ promedio, 2);
53         }
54
55         //
↪ -----
56         // Cálculo de la varianza poblacional y la
↪ desviación estándar
57         //
↪ -----
58         varianza = sumaCuadrados / n;
59         desviacion = Math.sqrt(varianza);
60
61         //
↪ -----
62         // Presentación de resultados
63         //
↪ -----
64         System.out.println("\n== RESULTADOS
↪ ESTADÍSTICOS ==");
65         System.out.println("Promedio: " + promedio);
66         System.out.println("Varianza: " + varianza);
67         System.out.println("Desviación estándar: "
↪ + desviacion);
68
69         sc.close();
70     }
71 }
72

```

Listing 10.21: Cálculo del promedio, la varianza y la desviación estándar en Java

La estrategia de solución utilizada en el Listado de código 10.21 se apoya en la idea de **procesamiento por etapas**. Esta organización evita mezclar en una sola fase operaciones que dependen unas de otras. El primer recorrido se dedica exclusivamente a capturar los datos y construir la suma total; con ese resultado se calcula el promedio. Luego, el segundo recorrido reutiliza el arreglo ya cargado para medir cuánto se aparta cada dato de esa media. Esta separación es importante porque hace explícita la dependencia entre las medidas: la varianza no puede calcularse antes del promedio, y la desviación estándar no puede obtenerse antes de la varianza [53], [74].

Uno de los aspectos más importantes de Java en esta solución es el uso del arreglo `double[]`. Esta estructura permite conservar los valores en memoria y acceder a ellos mediante índices, lo que hace posible recorrerlos varias veces sin volver a pedirlos al usuario. Precisamente aquí se aprecia por qué las variables individuales no serían adecuadas para este problema: una vez calculada la suma, sería imposible volver a consultar ordenadamente cada observación para calcular las diferencias respecto del promedio. El arreglo resuelve ese problema de almacenamiento y recuperación de manera directa y eficiente [7], [35], [87]. También merece atención la decisión de utilizar `double` y no `int`. Las medidas estadísticas pueden involucrar números decimales incluso cuando los datos de entrada son enteros. El promedio rara vez coincide exactamente con uno de los datos observados, y la varianza suele producir valores no enteros. Por ello, el uso de `double` resulta necesario para preservar precisión suficiente en los cálculos. La llamada a `Math.pow(datos[i] - promedio, 2)` permite expresar con claridad la elevación al cuadrado de cada desviación, mientras que `Math.sqrt(varianza)` implementa de forma directa el cálculo de la desviación estándar [71], [87].

La validación inicial de `n` también forma parte de la estrategia y no debe considerarse un detalle secundario. Un algoritmo correcto no solo calcula cuando los datos son válidos, sino que impide avanzar cuando la entrada no satisface las condiciones mínimas del problema. En este caso, exigir al menos dos valores fortalece la coherencia del procedimiento y evita trabajar con conjuntos insuficientes para el análisis planteado. Esta práctica está alineada con principios básicos de calidad del software, en los que la validación de datos de entrada constituye un mecanismo esencial para reducir errores y mejorar la confiabilidad del programa [23], [24].

Otra decisión relevante es el uso de dos acumuladores diferentes: `suma` y `sumaCuadrados`. El primero resume la información necesaria para la media; el segundo resume la dispersión respecto de esa media. Esta distinción hace que el código sea más legible, porque cada variable cumple una función precisa y no se reutiliza para propósitos distintos. En programas introductorios, mantener esta separación contribuye a que el procedimiento pueda seguirse mentalmente con mayor facilidad, lo que favorece la comprensión del algoritmo y su depuración posterior [51], [75].

En conjunto, la solución en Java del Listado de código 10.21 no solo resuelve correctamente el cálculo de las tres medidas estadísticas solicitadas, sino que también muestra una estrategia de diseño ordenada, progresiva y coherente con la estructura del problema. La combinación de validación, almacenamiento en arreglo, recorridos secuenciales, acumuladores y funciones matemáticas de la biblioteca estándar convierte a este ejercicio en una excelente aplicación de arreglos unidimensionales y en una base sólida para problemas posteriores de análisis numérico y procesamiento de datos [19], [59], [76].

Parte IV

Modularidad, Calidad e Integración en Sistemas de Información

Capítulo 11

Modularidad y Funciones en Aplicaciones

11.1. Abstracción y descomposición funcional

11.1.1. Diseño modular

Responsabilidad única

Cohesión y acoplamiento

Reutilización

11.2. Funciones y procedimientos

11.2.1. Definición e invocación

Firma

Paso de parámetros

Ámbito

11.3. Pruebas y calidad básica

11.3.1. Estrategia de pruebas

Casos representativos

Casos límite

Regresión

Parte V

Material Complementario

Capítulo 12

Proyecto Integrador en Sistemas de Información

12.1. Planteamiento organizacional

12.1.1. Contexto y alcance

Objetivo

Requisitos

Restricciones

12.2. Diseño e implementación

12.2.1. Estructura del sistema

Módulos

Flujo principal

Casos alternativos

Capítulo 13

Gestión básica de versiones en entornos colaborativos

13.1. Fundamentos

13.1.1. Repositorio y control de cambios

Repositorio

Commit

Ramas

Capítulo 14

Glosario y Recursos Académicos

14.1. Términos fundamentales

14.2. Bibliografía y fuentes de consulta

Apéndice A

Operaciones bit a bit, máscaras y estados

Las operaciones bit a bit permiten manipular directamente los bits que componen un valor almacenado en memoria. En lenguajes como C y C++, estas operaciones actúan sobre representaciones binarias completas (palabras de 8, 16, 32 o 64 bits), mientras que los operadores lógicos trabajan sobre valores booleanos derivados de expresiones. Distinguir entre ambos niveles resulta fundamental en programación de SI cuando se diseñan mecanismos de control de estados, permisos y banderas codificadas dentro de variables enteras.

A.1. Operador OR bit a bit |

El operador | realiza una operación OR entre cada par de bits correspondientes de dos operandos. Actúa a nivel de bit dentro de una palabra completa.

A.1.1. Ejemplo 1: OR a nivel de bit (8 bits)

$$A = 01011010_2$$

$$B = 00110101_2$$

Aplicando OR bit a bit:

$$\begin{array}{r} 01011010 \\ 00110101 \\ \hline 01111111 \end{array}$$

$$A | B = 01111111_2$$

Cada posición binaria se evalúa de manera independiente. El bit resultante es 1 cuando al menos uno de los bits comparados es 1.

A.1.2. Ejemplo 2: OR a nivel de palabra (32 bits en C)

```

1  #include <stdio.h>
2
3  int main() {
4      unsigned int permisos = 0x00000004; // 00000000 00000000 00000000
      ↪ 00000100
5      unsigned int escritura = 0x00000002; // 00000000 00000000 00000000
      ↪ 00000010
6
7      permisos = permisos | escritura;
8
9      printf("%u\n", permisos);
10     return 0;
11 }

```

Listing A.1: Uso de OR bit a bit en C

Resultado binario:

00000000 00000000 00000000 00000110

Aquí se activan simultáneamente dos banderas dentro de una palabra de 32 bits.

A.2. Operador OR lógico ||

El operador || es un operador lógico que evalúa expresiones completas y produce un valor booleano (0 o 1 en C/C++). No actúa bit a bit.

A.2.1. Ejemplo 1: Evaluación lógica con cortocircuito

```

1  #include <stdio.h>
2
3  int main() {
4      int a = 5;
5      int b = 0;
6
7      if (a || b) {
8          printf("Verdadero\n");
9      }
10 }

```

```

11     return 0;
12 }

```

Listing A.2: Uso de OR lógico en C

Como **a** es distinto de 0, la expresión es verdadera y **b** no necesita evaluarse (cortocircuito).

A.2.2. Ejemplo 2: Diferencia práctica entre `|` y `||`

```

1      #include <stdio.h>
2
3      int main() {
4          int x = 2;    // 00000010
5          int y = 1;    // 00000001
6
7          int r1 = x | y;    // OR bit a bit
8          int r2 = x || y;   // OR lógico
9
10         printf("r1 = %d\n", r1);
11         printf("r2 = %d\n", r2);
12
13         return 0;
14     }

```

Listing A.3: Comparación entre OR bit a bit y OR lógico

$$x|y = 00000011_2 = 3$$

$$x||y = 1$$

`|` conserva estructura binaria. `||` produce únicamente 0 (falso) o 1 (verdadero).

A.3. Operador AND &

El operador `&` realiza una operación AND bit a bit. El bit resultante es 1 únicamente cuando ambos bits son 1.

```

11001010
10101100
-----
10001000

```

Uso típico: verificación de bandera.

```

1  if (permisos & 0x04) {
2      // permiso activo
3  }
```

Listing A.4: Verificación de una bandera con AND

A.4. Operador XOR ^

El operador ^ produce 1 cuando los bits comparados son diferentes.

1100
1010
0110

Uso típico: alternar un estado.

```

1  bandera = bandera ^ 0x01;
```

Listing A.5: Alternar un bit con XOR

A.5. Desplazamientos

A.5.1. Desplazamiento a la izquierda <<

$$00000101 \ll 1 = 00001010$$

Equivale a multiplicar por 2 cuando no hay desbordamiento.

```

1  unsigned int x = 5; // 00000101
2  x = x << 1;        // 00001010
```

Listing A.6: Desplazamiento a la izquierda

A.5.2. Desplazamiento a la derecha >>

$$00001010 \gg 1 = 00000101$$

Equivale a dividir entre 2 para enteros sin signo.


```
1 unsigned int y = 10; // 00001010
2 y = y >> 1;          // 00000101
```

Listing A.7: Desplazamiento a la derecha

A.6. Resumen comparativo

Operador	Nivel de operación
	Bit a bit (palabra completa)
&	Bit a bit
^	Bit a bit
<< >>	Bit a bit (desplazamiento)
	Lógico (booleano)
&&	Lógico (booleano)

i

Importante

Las operaciones bit a bit modifican directamente la representación binaria almacenada en memoria. Los operadores lógicos evalúan expresiones completas y producen valores booleanos. Esta distinción permite diseñar estructuras compactas de control y validación dentro de sistemas computacionales.

Apéndice B

Codificación de caracteres: de ASCII a Unicode

La *codificación de caracteres* establece una correspondencia entre símbolos (letras, dígitos, signos, controles) y valores numéricos que un sistema digital puede almacenar, transmitir y procesar. En programación, este mapeo condiciona tareas tan diversas como validación de entradas, serialización de datos, interoperabilidad entre sistemas, normalización de texto y depuración de errores por interpretación de bytes. Por tal motivo, un aprendizaje riguroso de programación se beneficia cuando el estudiante distingue con precisión entre: (i) *repertorio* de caracteres (qué símbolos existen), (ii) *codificación* (cómo se traducen a números) y (iii) *codificación de bytes* (cómo esos números se representan en memoria y en archivos) [119], [120], [121].

Dentro de este panorama, ASCII se consolidó como base histórica de múltiples convenciones de software al definir un repertorio mínimo de 128 caracteres sobre 7 bits, suficiente para el alfabeto inglés y señales de control heredadas de teletipos. No obstante, la evolución hacia lenguas con acentos, símbolos monetarios y tipografías diversas motivó extensiones de 8 bits (*code pages*) y, posteriormente, la adopción de Unicode como estándar global. Esta progresión explica por qué, en sistemas actuales, UTF-8 domina como codificación de intercambio: mantiene compatibilidad hacia atrás con ASCII en el rango 0–127 y habilita la representación de prácticamente todos los sistemas de escritura [82], [119], [121].

B.0.1. ASCII (7 bits): repertorio base 0–127

ASCII define 128 posiciones numeradas de 0 a 127. Las posiciones 0–31 y 127 se reservan para caracteres de control (no imprimibles), mientras que 32–126 representan caracteres imprimibles (espacio, signos, dígitos y letras). La tabla siguiente presenta el mapeo completo, expresado en decimal y hexadecimal, destacando el nombre convencional de los controles [120], [122].

Dec	Hex	Car.	Nombre	Dec	Hex	Car.	Nombre
0	00		NUL	64	40	@	Car. Arroba
1	01		SOH	65	41	A	A
2	02		STX	66	42	B	B
3	03		ETX	67	43	C	C
4	04		EOT	68	44	D	D
5	05		ENQ	69	45	E	E

Dec	Hex	Car.	Nombre	Dec	Hex	Car.	Nombre
6	06		ACK	70	46	F	F
7	07		BEL	71	47	G	G
8	08		BS	72	48	H	H
9	09		HT	73	49	I	I
10	0A		LF	74	4A	J	J
11	0B		VT	75	4B	K	K
12	0C		FF	76	4C	L	L
13	0D		CR	77	4D	M	M
14	0E		SO	78	4E	N	N
15	0F		SI	79	4F	O	O
16	10		DLE	80	50	P	P
17	11		DC1	81	51	Q	Q
18	12		DC2	82	52	R	R
19	13		DC3	83	53	S	S
20	14		DC4	84	54	T	T
21	15		NAK	85	55	U	U
22	16		SYN	86	56	V	V
23	17		ETB	87	57	W	W
24	18		CAN	88	58	X	X
25	19		EM	89	59	Y	Y
26	1A		SUB	90	5A	Z	Z
27	1B		ESC	91	5B	[	Corchete izquierdo
28	1C		FS	92	5C	\	Barra invertida
29	1D		GS	93	5D	]	Corchete derecho
30	1E		RS	94	5E	^	Acento circunflejo
31	1F		US	95	5F	_	Guion bajo
32	20		Espacio	96	60	‘	Acento grave
33	21	!	Signo de exclamación	97	61	a	a
34	22	”	Comillas dobles	98	62	b	b
35	23	#	Signo numeral	99	63	c	c
36	24	\$	Signo dólar	100	64	d	d
37	25	%	Signo de porcentaje	101	65	e	e

Dec	Hex	Car.	Nombre	Dec	Hex	Car.	Nombre
38	26	&	Ampersand	102	66	f	f
39	27	'	Apóstrofo	103	67	g	g
40	28	(	Paréntesis izquierdo	104	68	h	h
41	29	)	Paréntesis derecho	105	69	i	i
42	2A	*	Asterisco	106	6A	j	j
43	2B	+	Signo más	107	6B	k	k
44	2C	,	Coma	108	6C	l	l
45	2D	-	Guion	109	6D	m	m
46	2E	.	Punto	110	6E	n	n
47	2F	/	Barra diagonal	111	6F	o	o
48	30	0	0	112	70	p	p
49	31	1	1	113	71	q	q
50	32	2	2	114	72	r	r
51	33	3	3	115	73	s	s
52	34	4	4	116	74	t	t
53	35	5	5	117	75	u	u
54	36	6	6	118	76	v	v
55	37	7	7	119	77	w	w
56	38	8	8	120	78	x	x
57	39	9	9	121	79	y	y
58	3A	:	Dos puntos	122	7A	z	z
59	3B	;	Punto y coma	123	7B	{	Llave izquierda
60	3C	<	Signo menor que	124	7C		Barra vertical
61	3D	=	Signo igual	125	7D	}	Llave derecha
62	3E	>	Signo mayor que	126	7E	~	Virgulilla
63	3F	?	Signo de interrogación	127	7F		Suprimir (DEL)

La tabla ASCII resulta indispensable para lectura de archivos, protocolos y depuración, debido a que múltiples formatos interpretan bytes 0–127 como ASCII. Paralelamente, el estudiante adquiere un criterio operativo: una cadena es, en última instancia, una secuencia de códigos, y el comportamiento de comparaciones, ordenamiento y validaciones depende de esa codificación [119], [122].

Extensiones de 8 bits: por qué “ASCII extendido” no es una tabla única

En la práctica se usa la expresión “ASCII extendido” para referirse a codificaciones de 8 bits que preservan ASCII en 0–127 y asignan caracteres a 128–255. No obstante, ese bloque superior depende de una tabla específica. Dos referencias ampliamente citadas son: (i) ISO/IEC 8859-1 (Latin-1), utilizada históricamente en sistemas UNIX y protocolos antiguos; y (ii) Windows-1252, extensión dominante en entornos Windows que asigna símbolos tipográficos a 0x80–0x9F (posición que en ISO/IEC 8859-1 se reserva para controles) [82], [119], [123].

Tabla completa ISO/IEC 8859-1 (Latin-1) para 128–255

En ISO/IEC 8859-1, el rango 0x80–0x9F corresponde a controles C1. El rango 0xA0–0xFF contiene símbolos y letras latinas acentuadas.

Cuadro B.2: Códigos ASCII extendidos (128–255) según ISO-8859-1

Dec	Hex	Car.	Nombre	Dec	Hex	Car.	Nombre
128	80		Control	192	C0	À	A grave
129	81		Control	193	C1	Á	A aguda
130	82		Control	194	C2	Â	A circunfleja
131	83		Control	195	C3	Ã	A tilde
132	84		Control	196	C4	Ä	A diéresis
133	85		Control	197	C5	Å	A anillo
134	86		Control	198	C6	Æ	AE
135	87		Control	199	C7	Ç	C cedilla
136	88		Control	200	C8	È	E grave
137	89		Control	201	C9	É	E aguda
138	8A		Control	202	CA	Ê	E circunfleja
139	8B		Control	203	CB	Ë	E diéresis
140	8C		Control	204	CC	Ì	I grave
141	8D		Control	205	CD	Í	I aguda
142	8E		Control	206	CE	Î	I circunfleja
143	8F		Control	207	CF	Ï	I diéresis
144	90		Control	208	D0	Ð	Eth mayúscula
145	91		Control	209	D1	Ñ	N con tilde
146	92		Control	210	D2	Ò	O grave

Continúa en la siguiente página

Cuadro B.2 (continuación)

Dec	Hex	Car.	Nombre	Dec	Hex	Car.	Nombre
147	93		Control	211	D3	Ó	O aguda
148	94		Control	212	D4	Ô	O circunfleja
149	95		Control	213	D5	Õ	O tilde
150	96		Control	214	D6	Ö	O diéresis
151	97		Control	215	D7	×	Multiplicación
152	98		Control	216	D8	Ø	O tachada
153	99		Control	217	D9	Û	U grave
154	9A		Control	218	DA	Ú	U aguda
155	9B		Control	219	DB	Û	U circunfleja
156	9C		Control	220	DC	Ü	U diéresis
157	9D		Control	221	DD	Ý	Y aguda
158	9E		Control	222	DE	Þ	Thorn mayúscula
159	9F		Control	223	DF	ß	Eszett
160	A0		Espacio no separable	224	E0	à	a grave
161	A1	¡	Exclamación invertida	225	E1	á	a aguda
162	A2	¢	Centavo	226	E2	â	a circunfleja
163	A3	£	Libra esterlina	227	E3	ã	a tilde
164	A4	¤	Moneda	228	E4	ä	a diéresis
165	A5	¥	Yen	229	E5	å	a anillo
166	A6		Barra partida	230	E6	æ	ae
167	A7	§	Sección	231	E7	ç	c cedilla
168	A8	¨	Diéresis	232	E8	è	e grave
169	A9	©	Copyright	233	E9	é	e aguda
170	AA	ª	Ordinal femenino	234	EA	ê	e circunfleja
171	AB	«	Comillas angulares izq.	235	EB	ë	e diéresis
172	AC	¬	Negación	236	EC	ì	i grave
173	AD		Guion suave	237	ED	í	i aguda
174	AE	®	Marca registrada	238	EE	î	i circunfleja
175	AF	ˉ	Macrón	239	EF	ï	i diéresis
176	B0	°	Grado	240	F0	ð	eth minúscula
177	B1	±	Más/menos	241	F1	ñ	n con tilde

Continúa en la siguiente página

Cuadro B.2 (continuación)							
Dec	Hex	Car.	Nombre	Dec	Hex	Car.	Nombre
178	B2	²	Superíndice 2	242	F2	ò	o grave
179	B3	³	Superíndice 3	243	F3	ó	o aguda
180	B4	´	Acento agudo	244	F4	ô	o circunfleja
181	B5	µ	Micro	245	F5	õ	o tilde
182	B6	¶	Párrafo	246	F6	ö	o diéresis
183	B7	·	Punto medio	247	F7	÷	División
184	B8	¸	Cedilla	248	F8	ø	o tachada
185	B9	¹	Superíndice 1	249	F9	ù	u grave
186	BA	º	Ordinal masculino	250	FA	ú	u aguda
187	BB	»	Comillas angulares der.	251	FB	û	u circunfleja
188	BC	¼	Un cuarto	252	FC	ü	u diéresis
189	BD	½	Un medio	253	FD	ý	y aguda
190	BE	¾	Tres cuartos	254	FE	þ	thorn minúscula
191	BF	¿	Interrogación invertida	255	FF	ÿ	y diéresis

Tabla completa Windows-1252 para 128–255

Windows-1252 mantiene ASCII en 0–127, pero asigna caracteres tipográficos a 0x80–0x9F. Esta distinción resulta indispensable en programación al interpretar archivos de texto “ANSI15” de Windows que contienen comillas curvas, guiones largos o el símbolo del euro [119], [123].

Cuadro B.3: Códigos 128–255 según Windows-1252 (CP1252)

Dec	Hex	Car.	Nombre	Dec	Hex	Car.	Nombre
128	80	€	Euro	192	C0	À	A grave
129	81		No asignado	193	C1	Á	A aguda
130	82	,	Comilla baja simple	194	C2	Â	A circunfleja
131	83	f	Florín	195	C3	Ã	A tilde
132	84	„	Comillas bajas dobles	196	C4	Ä	A diéresis
133	85	...	Puntos suspensivos	197	C5	Å	A anillo
134	86	†	Daga	198	C6	Æ	AE
135	87	‡	Daga doble	199	C7	Ç	C cedilla

Continúa en la siguiente página

Cuadro B.3 (continuación)

Dec	Hex	Car.	Nombre	Dec	Hex	Car.	Nombre
136	88	^	Acento circunflejo (diacrítico)	200	C8	È	E grave
137	89	‰	Por mil	201	C9	É	E aguda
138	8A	Š	S carón (mayúscula)	202	CA	Ê	E circunfleja
139	8B	◁	Comilla angular izq.	203	CB	Ë	E diéresis
140	8C	Œ	OE (mayúscula)	204	CC	Ì	I grave
141	8D		No asignado	205	CD	Í	I aguda
142	8E	Ž	Z carón (mayúscula)	206	CE	Î	I circunfleja
143	8F		No asignado	207	CF	Ï	I diéresis
144	90		No asignado	208	D0	Ð	Eth (mayúscula)
145	91	‘	Comilla simple izquierda	209	D1	Ñ	N con tilde
146	92	’	Comilla simple derecha	210	D2	Ò	O grave
147	93	“	Comilla doble izquierda	211	D3	Ó	O aguda
148	94	”	Comilla doble derecha	212	D4	Ô	O circunfleja
149	95	•	Viñeta	213	D5	Õ	O tilde
150	96	–	Raya corta (en dash)	214	D6	Ö	O diéresis
151	97	—	Raya larga (em dash)	215	D7	×	Multiplicación
152	98	~	Tilde (diacrítico)	216	D8	Ø	O tachada
153	99	™	Marca registrada (TM)	217	D9	Ù	U grave
154	9A	š	s carón (minúscula)	218	DA	Ú	U aguda
155	9B	›	Comilla angular der.	219	DB	Û	U circunfleja
156	9C	œ	oe (minúscula)	220	DC	Ü	U diéresis
157	9D		No asignado	221	DD	Ý	Y aguda
158	9E	ž	z carón (minúscula)	222	DE	Þ	Thorn (mayúscula)
159	9F	ÿ	Y diéresis	223	DF	ß	Eszett
160	A0		Espacio no separable	224	E0	à	a grave
161	A1	¡	Exclamación invertida	225	E1	á	a aguda
162	A2	¢	Centavo	226	E2	â	a circunfleja
163	A3	£	Libra esterlina	227	E3	ã	a tilde
164	A4	¤	Signo monetario	228	E4	ä	a diéresis
165	A5	¥	Yen	229	E5	å	a anillo
166	A6		Barra partida	230	E6	æ	ae

Continúa en la siguiente página

Cuadro B.3 (continuación)

Dec	Hex	Car.	Nombre	Dec	Hex	Car.	Nombre
167	A7	§	Sección	231	E7	ç	c cedilla
168	A8	¨	Diéresis	232	E8	è	e grave
169	A9	©	Copyright	233	E9	é	e aguda
170	AA	ª	Ordinal femenino	234	EA	ê	e circunfleja
171	AB	«	Comillas angulares izq.	235	EB	ë	e diéresis
172	AC	¬	Negación	236	EC	ì	i grave
173	AD		Guion suave	237	ED	í	i aguda
174	AE	®	Marca registrada	238	EE	î	i circunfleja
175	AF	ˉ	Macrón	239	EF	ï	i diéresis
176	B0	°	Grado	240	F0	ð	eth (minúscula)
177	B1	±	Más/menos	241	F1	ñ	n con tilde
178	B2	²	Superíndice 2	242	F2	ò	o grave
179	B3	³	Superíndice 3	243	F3	ó	o aguda
180	B4	´	Acento agudo	244	F4	ô	o circunfleja
181	B5	µ	Micro	245	F5	õ	o tilde
182	B6	¶	Párrafo	246	F6	ö	o diéresis
183	B7	·	Punto medio	247	F7	÷	División
184	B8	¸	Cedilla	248	F8	ø	o tachada
185	B9	¹	Superíndice 1	249	F9	ù	u grave
186	BA	º	Ordinal masculino	250	FA	ú	u aguda
187	BB	»	Comillas angulares der.	251	FB	û	u circunfleja
188	BC	¼	Un cuarto	252	FC	ü	u diéresis
189	BD	½	Un medio	253	FD	ý	y aguda
190	BE	¾	Tres cuartos	254	FE	þ	thorn (minúscula)
191	BF	¿	Interrogación invertida	255	FF	ÿ	y diéresis

En síntesis, cuando un programa recibe un byte con valor 0x93, el símbolo resultante depende de la tabla: en Windows-1252 corresponde a comillas tipográficas (“), mientras que en ISO/IEC 8859-1 ese valor se reserva para control C1. Esta diferencia explica fallas de interoperabilidad, aparición de caracteres “extraños” y necesidad de conversiones explícitas en lectura de archivos [119], [123].

Unicode y UTF-8: estándar global para texto

Unicode define un repertorio amplio de caracteres (códigos U+XXXX) y asigna puntos de código a letras, símbolos, emoji y escrituras completas. En la práctica, UTF-8 codifica esos puntos de código en secuencias de 1 a 4 bytes: para U+0000 a U+007F coincide con ASCII, lo cual facilita compatibilidad con software heredado. Este diseño sugiere una práctica de programación recomendada: conservar UTF-8 como codificación por defecto en archivos y comunicaciones, y convertir a tablas de 8 bits únicamente cuando el contexto lo exija (por ejemplo, archivos legacy) [119], [121].

De manera complementaria, UTF-16 y UTF-32 existen para necesidades específicas: UTF-16 usa unidades de 16 bits (con pares sustitutos en rangos superiores) y UTF-32 usa 32 bits fijos. En términos pedagógicos, la distinción más útil al inicio es que Unicode separa “carácter” (punto de código) de “cómo se almacena” (UTF-8/UTF-16/UTF-32). Esa idea permite al estudiante razonar con rigor sobre longitudes de cadenas, índices, conteos de bytes y validaciones de entrada [119], [124].

Codificación Unicode: UTF-8, UTF-16 y UTF-32

Unicode define un repertorio universal de caracteres asignando a cada uno un **punto de código** único en el rango U+0000 hasta U+10FFFF. Esta asignación está normada por el estándar ISO/IEC 10646 [125] y especificada operativamente en distintos esquemas de codificación, entre ellos UTF-8, UTF-16 y UTF-32 [119], [121].

1. Codificación UTF-8. Es una codificación de longitud variable (1 a 4 bytes) compatible hacia atrás con ASCII.

Estructura general

Rango Unicode	Patrón binario UTF-8
U+0000–007F	0xxxxxxx
U+0080–07FF	110xxxxx 10xxxxxx
U+0800–FFFF	1110xxxx 10xxxxxx 10xxxxxx
U+10000–10FFFF	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

Diagrama visual de bits. Ejemplo: carácter “ñ” (U+00F1)

$$U + 00F1 = 00000000 \ 11110001_2$$

Distribución en UTF-8 (2 bytes):

$$\begin{array}{cc} \underbrace{11000011}_{\text{Byte 1}} & \underbrace{10110001}_{\text{Byte 2}} \end{array}$$

En hexadecimal:

$$C3 \ B1$$

Ejemplo con 4 bytes. Carácter: ☹️ (U+1F600)

$$1F600_{16} = 0001\ 1111\ 0110\ 0000\ 0000_2$$

Distribución en patrón UTF-8:

$$\begin{array}{cccc} \underbrace{11110000}_{F0} & \underbrace{10011111}_{9F} & \underbrace{10011000}_{98} & \underbrace{10000000}_{80} \end{array}$$

Resultado:

$$F0\ 9F\ 98\ 80$$

2. Codificación UTF-16. Este tipo de codificación utiliza unidades de 16 bits.

Caso simple. Para puntos U+0000--U+FFFF:

$$\text{"ñ"} = 00F1_{16}$$

En memoria:

Big-endian:

$$00\ F1$$

Little-endian:

$$F1\ 00$$

Pares sustitutos (Surrogates). Para puntos mayores que U+FFFF:

Carácter ☹️ (U+1F600)

1. Restar 0x10000:

$$1F600 - 10000 = F600$$

2. Dividir en dos bloques de 10 bits.

3. Calcular:

High surrogate = $D800 + \text{parte alta}$

Low surrogate = $DC00 + \text{parte baja}$

4. Resultado:

$D83D DE00$

En memoria:

Big-endian:

$D8\ 3D\ DE\ 00$

Little-endian:

$3D\ D8\ 00\ DE$

3. Codificación UTF-32. Almacena directamente el punto de código en 32 bits.

Ejemplo:

$\text{``ñ''} = 000000F1_{16}$

En memoria:

Big-endian:

$00\ 00\ 00\ F1$

Little-endian:

$F1\ 00\ 00\ 00$

No requiere transformación ni pares sustitutos.

4. Diagramas comparativos de almacenamiento

Carácter	UTF-8	UTF-16
ñ	C3 B1	00 F1
☺	F0 9F 98 80	D8 3D DE 00

5. Problemas de codificación: Mojibake. El fenómeno denominado mojibake ocurre cuando una secuencia de bytes se interpreta con una codificación distinta a la original.

Ejemplo clásico:

La palabra ``Señor'' en UTF-8:

$53\ 65\ C3\ B1\ 6F\ 72$

Si esos bytes se interpretan como ISO-8859-1, se obtiene:

Se±or

Esto ocurre porque:

- C3 B1 es correcto en UTF-8 para ``ñ".
- ISO-8859-1 interpreta C3 y B1 como caracteres independientes.

El resultado es una corrupción visual del texto.

6. Consideraciones normativas. UTF-8 está definido formalmente en RFC 3629 [121]. Unicode y su mapeo con ISO/IEC 10646 están regulados por ISO/IEC 10646 [125].

Las diferencias fundamentales son:

- UTF-8: eficiente y dominante en Internet.
- UTF-16: usado históricamente en Windows y Java.
- UTF-32: representación directa, mayor consumo de memoria.

Todas representan exactamente el mismo conjunto Unicode.

Bibliografía

- [1] K. E. Kendall y J. E. Kendall, *Systems Analysis and Design*, 11.^a ed. Pearson, 2024, ISBN: 9780137947850.
- [2] K. C. Laudon y J. P. Laudon, *Management Information Systems: Managing the Digital Firm*, 16.^a ed. Pearson, 2020, ISBN: 978-0135191816.
- [3] F. M. González-Longatt, «Introducción a los Sistemas de Información: Fundamentos,» *Manuscrito (Universidad Experimental Politécnica de la Fuerza Armada, Venezuela)*, pág. 7, 2007, pdf disponible en línea. dirección: <https://www.uv.mx/personal/artulopez/files/2012/08/fundamentossistemasinformacion.pdf>
- [4] C. Ghezzi, M. Jazayeri y D. Mandrioli, *Fundamentals of Software Engineering*, 2.^a ed. Prentice Hall, 2003, ISBN: 978-0133056990.
- [5] J. R. Hanly y E. B. Koffman, *Problem Solving and Program Design in C*, 8.^a ed. Boston, MA: Pearson, 2015, ISBN: 978-1292098814.
- [6] ISO/IEC, *Programming languages — Avoiding vulnerabilities in programming languages Part 1: Language-independent catalogue of vulnerabilities*, Geneva, Switzerland: ISO/IEC, 2024. dirección: <https://www.iso.org/standard/83629.html>
- [7] T. Gaddis, *Starting Out with Programming Logic and Design*, 6th. Boston, MA: Pearson, 2022, ISBN: 978-0137602148.
- [8] S. Calzati, «An Ecosystemic View on Information, Data, and Knowledge: Insights on Agential AI and Relational Ethics,» *AI and Ethics*, vol. 5, págs. 3763-3776, 2025. DOI: 10.1007/s43681-025-00665-0
- [9] W. Stallings, *Computer Organization and Architecture: Designing for Performance*, 11.^a ed. London: Pearson, 2021, pág. 896, ISBN: 978-1292420103.
- [10] A. Silberschatz, P. B. Galvin y G. Gagne, *Operating System Concepts*, 10.^a ed. Hoboken, NJ: Wiley, 2018, pág. 1040, ISBN: 978-1119320913.

- [11] IEEE, «IEEE Standard for Floating-Point Arithmetic,» *IEEE Std 754-2019 (Revision of IEEE 754-2008)*, págs. 1-84, 2019. DOI: 10.1109/IEEESTD.2019.8766229
- [12] D. Goldberg, «What Every Computer Scientist Should Know About Floating-Point Arithmetic,» *ACM Computing Surveys*, vol. 23, n.º 1, págs. 5-48, 1991. DOI: 10.1145/103162.103163
- [13] N. Wirth, *Algorithms + Data Structures = Programs*, 1.^a ed. Prentice Hall, 1976, pág. 366, Última Actualización Moscow, 2 de febrero 2012-02-22 por Fyodor Tkachov, ISBN: 978-0130224187.
- [14] P. Leidig, G. Anderson, R. Sooriamurthi y J. Babb, «IS2020: Updating the Information Systems Model Curriculum,» en *Proceedings of the 51st ACM Technical Symposium on Computer Science Education*, ép. SIGCSE '20, Portland, OR, USA: Association for Computing Machinery, 2020, págs. 803-804, ISBN: 9781450367936. DOI: 10.1145/3328778.3366999
- [15] P. Leidig y H. Salmela, «IS2020: Competency Model for Undergraduate Programs in Information Systems: The Joint ACM/AIS IS2020 Task Force,» 2021. DOI: 10.1145/3460863
- [16] D. P. Tegarden, B. Samuel, R. Lukyanenko, A. Dennis y B. H. Wixom, *Systems Analysis and Design: An Object-Oriented Approach with UML*, 7.^a ed. John Wiley & Sons, 2025, ISBN: 978-1394331765.
- [17] J. M. Wing, «Computational Thinking,» *Communications of the ACM*, vol. 49, n.º 3, págs. 33-35, 2006. DOI: 10.1145/1118178.1118215
- [18] L. Perković, A. Settle, S. Hwang y J. Jones, «A framework for computational thinking across the curriculum,» en *Proceedings of the Fifteenth Annual Conference on Innovation and Technology in Computer Science Education*, ép. ITiCSE '10, Bilkent, Ankara, Turkey: Association for Computing Machinery, 2010, págs. 123-127, ISBN: 978-1605588209. DOI: 10.1145/1822090.1822126
- [19] G. Futschek, «Algorithmic Thinking: The Key for Understanding Computer Science,» en *Informatics Education – The Bridge between Using and Understanding Computers*, ép. Lecture Notes in Computer Science, vol. 4226, Springer, 2006, págs. 159-168. DOI: 10.1007/11915355_15
- [20] G. Booch, R. A. Maksimchuk, M. W. Engle, B. J. Young, J. Conallen y K. A. Houston, *Object-Oriented Analysis and Design with Applications*, 3.^a ed. Addison-Wesley, 2007, pág. 720, ISBN: 978-0201895513.

- [21] R. Fitzpatrick, «Software quality revisited,» English, en *Proceedings of the Software Measurement European Forum (SMEF)*, ép. Proceedings of the Software Measurement European Forum (SMEF), Milan, Italy: Instituto di Ricerca Internazionale, 2004, págs. 305-315. DOI: 10.21427/e2gf-0035
- [22] J. H. Bernstein, «The Data–Information–Knowledge–Wisdom Hierarchy and Its Antithesis,» *Journal of Management and Information Systems*, vol. 26, n.º 2, págs. 187-220, 2009, análisis crítico del modelo tradicional DIKW, discutiendo sus límites y proporcionando una visión más realista sobre cómo la información puede (o no) transformarse en conocimiento. DOI: 10.2753/MIS0742-1222260206
- [23] R. S. Pressman y B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9.ª ed. New York: McGraw-Hill Education, 2019, pág. 704, ISBN: 978-1260548006.
- [24] I. Sommerville, *Software Engineering*, 10.ª ed. Harlow, England: Pearson, 2015, pág. 816, obra clásica que define el rol integral del ingeniero de software, desde requisitos hasta mantenimiento, ISBN: 978-0133943030.
- [25] K. D. Cato, K. McGrow y S. C. Rossetti, «Transforming Clinical Data Into Wisdom: Artificial Intelligence Implications for Nurse Leaders,» *Nursing Management (Springhouse)*, vol. 51, n.º 11, págs. 24-30, 2020. DOI: 10.1097/01.NUMA.0000719396.83518.d6
- [26] A. Dix, J. Finlay, G. Abowd y R. Beale, *Human-Computer Interaction*, 3.ª ed. Pearson/Prentice-Hall, 2004, pág. 834, ISBN: 978-0130461094.
- [27] C. J. Date, *An Introduction to Database Systems*, 8.ª ed. Addison-Wesley, 2004, Edición clásica aún ampliamente citada; reimpressiones posteriores, ISBN: 978-0321197849.
- [28] R. Elmasri y S. B. Navathe, *Fundamentals of Database Systems*, 7th. Pearson, 2016, ISBN: 978-0133970777.
- [29] A. Silberschatz, H. F. Korth y S. Sudarshan, *Database System Concepts*, 7th. McGraw-Hill Education, 2020, ISBN: 978-0078022159.
- [30] A. Martín-Navarro, M. P. Lechuga Sancho y J. A. Medina-Garrido, «BPMS for management: a systematic literature review,» *Revista Espanola de Documentacion Cientifica*, vol. 41, n.º 3, 2018, n. art. e213, revisión sistemática reciente sobre los sistemas de gestión de procesos de negocio (BPMS) como tecnología que automatiza procesos organizacionales. DOI: 10.3989/redc.2018.3.1532

- [31] M. J. Medina, A. D. Santo, P. Oswald y M. Sokhn, «Digital Business Transformation for SMEs: Maturity Model for Systematic Roadmap,» en *Information Systems and Technologies*, ép. Lecture Notes in Networks and Systems, Á. Rocha, H. Adeli, L. P. Reis y S. Costanzo, eds., vol. 801, Springer, Cham, 2024, págs. 229-240. DOI: 10.1007/978-3-031-45648-0_23
- [32] A.-A. Vărzaru y otros, «Digital Transformation and Innovation: The Influence of Digital Technologies on Turnover from Innovation Activities and Types of Innovation,» *Systems*, vol. 12, n.º 9, 2024, n. art. 359, analiza cómo las tecnologías digitales, habilitadas por software, impulsan innovación y ventajas competitivas. DOI: 10.3390/systems12090359
- [33] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*, 1.^a ed. Upper Saddle River, NJ: Prentice Hall, 2008, ISBN: 9780132350884.
- [34] A. S. Tanenbaum y T. Austin, *Structured Computer Organization*, 6.^a ed. Boston, MA: Pearson, 2021, pág. 777, ISBN: 978-0137618446.
- [35] R. W. Sebesta, *Concepts of Programming Languages*, 12.^a ed. Pearson, 2018, Update 2018, ISBN: 978-0134997186.
- [36] R. Sedgewick y K. Wayne, *Algorithms*, 4.^a ed. Addison-Wesley, 2011, pág. 976, ISBN: 978-0321573513.
- [37] J. L. Manzano, *Programación: Algoritmos y estructuras de datos*. Paraninfo, 2017.
- [38] S. C. Chapra y R. P. Canale, *Numerical Methods for Engineers*, 7.^a ed. New York, NY: McGraw-Hill Education, 2014, ISBN: 978-0073397924.
- [39] W. Stallings, *Data and Computer Communications*, 10.^a ed. Pearson, 2015, ISBN: 978-1292014395.
- [40] Hironori Washizaki, ed., *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide)*. Los Alamitos, CA, USA: IEEE Computer Society, 2025, IEEE Computer Society. visitado 3 de mar. de 2026. dirección: <https://ieeecs-media.computer.org/media/education/swebok/swebok-v4.pdf>
- [41] ISO/IEC/IEEE, *ISO/IEC/IEEE International Standard - Systems and software engineering – Life cycle processes – Requirements engineering*, International Standard, ISO/IEC/IEEE, 2018. DOI: 10.1109/IEEESTD.2018.8559686

- [42] ISO/IEC/IEEE, *ISO/IEC/IEEE International Standard - Systems and software engineering – Software life cycle processes*, 2017. DOI: 10.1109/ieeestd.2017.8100771
- [43] R. T. Fielding, «Architectural Styles and the Design of Network-based Software Architectures,» Tesis doct., University of California, Irvine, 2000. visitado 5 de mar. de 2026. dirección: <https://roy.gbiv.com/pubs/dissertation/top.htm>
- [44] L. Floridi, «From Data to Semantic Information,» *Entropy*, vol. 5, n.º 2, págs. 125-140, 2003, presenta un marco filosófico-científico clásico sobre cómo los datos se transforman en información mediante semántica, contexto y procesamiento. DOI: 10.3390/e5020125
- [45] C. Zins, «Conceptual approaches for defining data, information, and knowledge,» *Journal of the American Society for Information Science and Technology*, vol. 58, n.º 4, págs. 479-493, 2007, revisión clásica que documenta decenas de definiciones disciplinares de datos, información y conocimiento, útil para fundamentar los conceptos en el campo de la ciencia de la información. DOI: 10.1002/asi.20508
- [46] R. Gellert, «Comparing Definitions of Data and Information in Information Science,» *Journal of the Association for Information Science and Technology*, vol. 73, n.º 5, págs. 657-672, 2022, análisis empírico de cómo distintas disciplinas definen “dato” e “información”, útil para fundamentar conceptualmente la distinción data–information. DOI: 10.1002/asi.24724
- [47] E. Horvitz y T. Mitchell, «From Data to Knowledge to Action: A Global Enabler for the 21st Century,» *arXiv preprint arXiv:2008.00045*, 2020, documenta cómo los datos masivos, correctamente procesados y analizados, permiten generar información útil para la toma de decisiones en diferentes dominios, desde la ciencia hasta políticas públicas.
- [48] P. C. Verhoef et al., «Digital transformation: A multidisciplinary reflection and research agenda,» *Journal of Business Research*, vol. 122, págs. 889-901, 2021, revisión conceptual que distingue claramente los tres niveles: digitization, digitalization y digital transformation, y sus implicaciones organizacionales. DOI: 10.1016/j.jbusres.2019.09.022
- [49] A. Bitkowska, B. Detyna y J. Detyna, «Importance of IT systems in integration of knowledge and business process management,» *Information Systems International Journal*, vol. 23, n.º 1, págs. 117-130, 2022, Estudio empírico que muestra relación

- positiva entre uso de sistemas de TI, BPM y calidad/efectividad de procesos organizacionales. DOI: 10.48009/1_iis_2022_109
- [50] A. A. Atieh, A. A. Hussein, S. Al-Jaghoub, A. F. Alheet y M. Attiany, «The Impact of Digital Technology, Automation, and Data Integration on Supply Chain Performance: The Moderating Role of Digital Transformation,» *Logistics*, vol. 9, n.º 1, 2025, n. art. 11, estudio empírico que evidencia que la automatización y la integración de datos mejoran el rendimiento organizacional cuando van acompañadas de iniciativas de transformación digital. DOI: 10.3390/logistics9010011
 - [51] S. McConnell, *Code Complete*, 2.^a ed. Microsoft Press, 2004, ISBN: 978-0735619678.
 - [52] D. E. Knuth, *The Art of Computer Programming, Volume 1: Fundamental Algorithms*, 3.^a ed. Addison-Wesley, 1997, pág. 650, ISBN: 978-0201896831.
 - [53] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 4.^a ed. Cambridge, MA: MIT Press, 2022, ISBN: 978-0262046305.
 - [54] A. Anand, S. Fosso Wamba y D. Gnanzou, «A Literature Review on Business Process Management, Business Process Reengineering and Business Process Innovation,» en *9th International Workshop on Enterprise & Organizational Modeling and Simulation (EOMAS 2013)*, revisión sobre BPM, reingeniería de procesos y su relación con sistemas de información, Valencia, Spain, 2013, págs. 1-16.
 - [55] G. Marchionini, «Information and data sciences: Context, units of analysis, meaning, and human impact,» *Data and Information Management*, vol. 7, n.º 1, 2023, n. art. 100031, ISSN: 2543-9251. DOI: 10.1016/j.dim.2023.100031
 - [56] J. M. Rodríguez Rodríguez y M. J. Daureo Campillo, *Sistemas de Información: Aspectos Técnicos y Legales*. Almería, España: Universidad de Almería, 2003. dirección: <http://biblioteca.udgvirtual.udg.mx/jspui/handle/123456789/973>
 - [57] S. Baškarada y A. Koronios, «Data, Information, Knowledge, Wisdom (DIKW): A Semiotic Theoretical and Empirical Exploration of the Hierarchy and Its Quality Dimension,» *Australasian Journal of Information Systems*, vol. 18, n.º 1, pág. 5, 2013, artículo influyente que analiza definiciones de dato, información y conocimiento, y examina cómo los datos se transforman en información significativa. DOI: 10.3127/ajis.v18i1.748

- [58] M. Frické, «The Knowledge Pyramid: the DIKW Hierarchy,» *Knowledge Organization*, vol. 46, n.º 1, págs. 33-47, 2019, revisión crítica del modelo DIKW y de la distinción entre datos e información en ciencias de la información. DOI: 10.5771/0943-7444-2019-1-33
- [59] J. Farrell, *Programming Logic and Design*, 9th. Boston, MA: Cengage Learning, 2018, ISBN: 9781337102070.
- [60] B. Kernighan y D. Ritchie, *The C Programming Language*, 2.^a ed. Pearson/Prentice Hall, 1988, pág. 272, ISBN: 978-0131103627.
- [61] L. J. Aguilar, *Fundamentos de Programación: Algoritmos, Estructuras de Datos y Objetos*, 3.^a ed. Madrid: McGraw-Hill, 2020, pág. 744, ISBN: 978-6071514684.
- [62] P. Wentworth, J. Elkner, A. B. Downey y C. Meyers, *How to Think Like a Computer Scientist: Learning with Python 3*, 3rd. OpenBookProject.net / Green Tea Press, 2011, Version for Python 3 (RLE), freely available online under the GNU Free Documentation License. dirección: <https://openbookproject.net/thinkcs/python/english3e/>
- [63] B. N. Miller y D. L. Ranum, *Problem Solving with Algorithms and Data Structures Using Python*, 2.^a ed. Wilsonville, OR: Franklin, Beedle & Associates, 2011, ISBN: 978-1590282571.
- [64] Y. N. Patt y S. J. Patel, *Introduction to Computing Systems*, 2.^a ed. New York, NY: McGraw-Hill Education, 2019, ISBN: 978-0072467505.
- [65] J. Farrell, *Programming Logic and Design*, 8.^a ed. Cengage Learning, 2015, pág. 704, ISBN: 978-1285776712.
- [66] M. Li, W. Fang, Q. Zhang y Z. Xie, «SpecLLM: Exploring Generation and Review of VLSI Design Specification with Large Language Model,» en *2025 International Symposium of Electronics Design Automation (ISED)*, 2025, págs. 749-755. DOI: 10.1109/iseda65950.2025.11100410
- [67] D. Duckworth y J. Fraillon, «Computational Thinking Framework,» *Educational Research Review*, vol. 22, J. Fraillon y M. Rožman, eds., págs. 35-43, 2025. DOI: 10.1007/978-3-031-61194-0_3
- [68] A. V. Aho, M. S. Lam, R. Sethi y J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2 update. Addison-Wesley, 2012, ISBN: 978-0133002140.

- [69] SmartDraw. «Símbolos de diagramas de flujo.» Consultado en diciembre de 2025. dirección: <https://www.smartdraw.com/flowchart/simbolos-de-diagramas-de-flujo.htm>
- [70] W. P. Stevens, G. J. Myers y L. L. Constantine, «Structured Design,» vol. 13, n.º 2, págs. 115-139, 1974. DOI: 10.1147/sj.132.0115
- [71] R. L. Halterman, *Fundamentals of Programming: An Introduction to Computer Programming Using C++*. Business & Educational, 1994, ISBN: 978-0697251107.
- [72] D. Hooshyar, R. B. Ahmad, M. H. N. Md Nasir, S. Shamshirband y S.-J. Horng, «Flowchart-based programming environments for improving comprehension and problem-solving skill of novice programmers: a survey,» *International Journal of Advanced Intelligence Paradigms*, vol. 7, n.º 1, págs. 24-56, 2015. DOI: 10.1504/IJAIP.2015.070343
- [73] B. C. Pierce, *Types and Programming Languages*, ilustrada. MIT Press, 2002, ISBN: 978-0262162098.
- [74] Y. Qian y J. Lehman, «Students' Misconceptions and Other Difficulties in Introductory Programming: A Literature Review,» *ACM Trans. Comput. Educ.*, vol. 18, n.º 1, oct. de 2018. DOI: 10.1145/3077618
- [75] A. Robins, J. Rountree y N. Rountree, «Learning and Teaching Programming: A Review and Discussion,» *Computer Science Education*, vol. 13, págs. 137-172, 2003, Published online: 09 Aug 2010. DOI: 10.1076/csed.13.2.137.14200
- [76] R. Lister et al., «A multi-national study of reading and tracing skills in novice programmers,» en *Working Group Reports from ITiCSE on Innovation and Technology in Computer Science Education*, ép. ITiCSE-WGR '04, Leeds, United Kingdom: Association for Computing Machinery, 2004, págs. 119-150, ISBN: 9781450377942. DOI: 10.1145/1044550.1041673
- [77] V. Baladi y B. Vallée, «Euclidean algorithms are Gaussian,» *Journal of Number Theory*, vol. 110, n.º 2, págs. 331-386, 2005. DOI: 10.1016/j.jnt.2004.08.008
- [78] S. J., «Two Fast GCD Algorithms,» *Journal of Algorithms*, vol. 16, n.º 1, págs. 110-144, 1994. DOI: 10.1006/jagm.1994.1006
- [79] R. C. Seacord, *Secure Coding in C and C++*, 2.^a ed. Addison-Wesley Professional, 2013, ISBN: 978-0321822130.

- [80] C. Abid, V. Alizadeh, M. Kessentini, T. d. N. Ferreira y D. Dig, «30 Years of Software Refactoring Research: A Systematic Literature Review,» *arXiv preprint arXiv:2007.02194*, 2020, n. art. arXiv:2007.02194v1, revisión sistemática sobre refactorización de código — práctica esencial para mantener calidad, legibilidad y robustez en sistemas de software complejos. DOI: 10.48550/arXiv.2007.02194 dirección: <https://arxiv.org/abs/2007.02194>
- [81] D. L. Lima, R. de Souza Santos, G. Pires Garcia, S. S. da Silva, C. Franca y L. F. Capretz, «Software Testing and Code Refactoring: A Survey with Practitioners,» *arXiv preprint arXiv:2310.01719*, 2023, informe reciente sobre la práctica de pruebas automáticas y refactorización en entornos reales, destaca su importancia en mantenimiento y calidad del software. DOI: 10.48550/arXiv.2310.01719
- [82] *Information technology — 8-bit single-byte coded graphic character sets — Part 1: Latin alphabet No. 1*, Latin-1 mapping, 0–255 with C0/C1 controls, International Organization for Standardization, 1998. dirección: <https://www.iso.org/obp/ui/en/#iso:std:iso-iec:8859:-1:ed-1:v1:en>
- [83] M. Scott y J. Aldrich, *Programming Language Pragmatics*, 5.^a ed. Morgan Kaufmann, mayo de 2025, ISBN: 978-0323999663. dirección: <https://shop.elsevier.com/books/programming-language-pragmatics/scott/978-0-323-99966-3>
- [84] P. Van Roy y S. Haridi, *Concepts, Techniques, and Models of Computer Programming*. Cambridge, MA: MIT Press, feb. de 2004, ISBN: 978-0262220699. dirección: <https://mitpress.mit.edu/9780262220699/concepts-techniques-and-models-of-computer-programming/>
- [85] S. W. Dietrich, «Teaching Introduction to Programming Languages with a Database Twist,» *Journal of Computing Sciences in Colleges*, vol. 36, n.º 3, págs. 43-52, 2020, ACM Digital Library. dirección: <https://dl.acm.org/doi/10.5555/3447080.3447086>
- [86] P. J. Deitel y H. M. Deitel, *C How to Program, with Case Studies and Applications*, 9.^a ed. Pearson, 2022, ISBN: 978-1292437071.
- [87] P. Deitel y H. Deitel, *Java How to Program: An Objects-Natural Approach*, 12.^a ed. Boston: Pearson, 2025, ISBN: 978-0137598359.
- [88] D. Srivastava, *Ultimate Python Programming: Learn Python with 650+ Programs, 900+ Practice Questions, and 5 Projects*. India: BPB Publications, 2024, English Edition, ISBN: 978-9355516558.

- [89] J. Skeet, *C# in Depth*, 4.^a ed. Shelter Island, NY: Manning Publications, 2019, Fourth Edition, ISBN: 978-1617294532.
- [90] L. L. Svekis, M. van Putten y R. Percival, *JavaScript from Beginner to Professional: Learn JavaScript Quickly by Building Fun, Interactive, and Dynamic Web Apps, Games, and Pages*, 1.^a ed. Birmingham, UK: Packt Publishing, 2021, Paperback, English Edition, ISBN: 978-1800562523.
- [91] R. Elizarov, S. Isakova, S. Aigner y D. Jemerov, *Kotlin in Action*, 2.^a ed. Shelter Island, NY: Manning Publications, 2024, pág. 560, ISBN: 978-1617299605.
- [92] J. Hoffman, *Mastering Swift 6: Modern Programming Techniques for High-Performance Apps in Swift 6.2*, 7.^a ed. Birmingham, UK: Packt Publishing, 2025, Seventh Edition, ISBN: 978-1836203698.
- [93] N. Metzler, *Ruby Programming for Beginners: An Introduction to Ruby Programming and a Hands-On Guide*. Independently Published, 2020, Kindle Edition, ISBN: 979-8648692756.
- [94] M. Summerfield, *Programming in Go: Creating Applications for the 21st Century*. Upper Saddle River, NJ: Addison-Wesley, 2012, Developer's Library, ISBN: 978-0132764094.
- [95] Python Software Foundation. «The Python Tutorial.» ver. 3.14.3, Python Software Foundation, visitado 20 de mar. de 2026. dirección: <https://docs.python.org/3/tutorial/>
- [96] A. M. Kuchling. «Functional Programming HOWTO.» ver. 3.14.3. Last updated on Mar 20, 2026, Python Software Foundation, visitado 20 de mar. de 2026. dirección: <https://docs.python.org/3/howto/functional.html>
- [97] G. Van Rossum y F. L. Drake, *Python 3 Reference Manual*. Scotts Valley, CA: CreateSpace, 2009, pág. 242, Python Documentation Manual Part 2, ISBN: 978-1441412690.
- [98] Mozilla Developer Network (MDN). «JavaScript Guide,» Mozilla Foundation, visitado 20 de mar. de 2026. dirección: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide>

- [99] N. Mehlhorn y S. Hanenberg, «Imperative versus Declarative Collection Processing: An RCT on the Understandability of Traditional Loops versus the Stream API in Java,» en *Proceedings of the 44th International Conference on Software Engineering*, ép. ICSE '22, Association for Computing Machinery, 2022, págs. 1157-1168. DOI: 10.1145/3510003.3519016
- [100] J. Sorva, «Notional Machines and Introductory Programming Education,» *ACM Transactions on Computing Education*, vol. 13, n.º 2, págs. 1-31, 2013. DOI: 10.1145/2483710.2483713
- [101] Oracle. «Objects.» The Java Tutorials, Learning the Java Language, Classes and Objects, Oracle, visitado 20 de mar. de 2026. dirección: <https://docs.oracle.com/javase/tutorial/java/java00/objects.html>
- [102] Oracle. «Classes.» The Java Tutorials, Learning the Java Language, Classes and Objects, Oracle, visitado 20 de mar. de 2026. dirección: <https://docs.oracle.com/javase/tutorial/java/java00/classes.html>
- [103] Oracle. «Summary of Creating and Using Classes and Objects.» The Java Tutorials, Learning the Java Language, Classes and Objects, Oracle, visitado 20 de mar. de 2026. dirección: <https://docs.oracle.com/javase/tutorial/java/java00/summaryclasses.html>
- [104] Haskell.org. «Haskell Language.» Official language site, visitado 20 de mar. de 2026. dirección: <https://www.haskell.org/>
- [105] S. Peyton Jones, ed., *Haskell 98 Language and Libraries: The Revised Report*, Cambridge University Press, 2003. visitado 20 de mar. de 2026. dirección: <https://www.haskell.org/definition/haskell98-report.pdf>
- [106] *Proceedings of the 6th ACM SIGPLAN International Conference on Principles and Practice of Declarative Programming*, Verona, Italy: Association for Computing Machinery, 2004, ISBN: 978-1581138191. DOI: 10.1145/1013963
- [107] J. Bloch, *Effective Java*, 3.^a ed. Addison-Wesley, 2018, ISBN: 978-0134685991.
- [108] JetBrains. «Predefined Windows keymap,» JetBrains, visitado 5 de mar. de 2026. dirección: <https://www.jetbrains.com/help/idea/reference-keymap-win-default.html>

- [109] Android Developers. «Keyboard shortcuts,» Android Developers, visitado 5 de mar. de 2026. dirección: <https://developer.android.com/studio/intro/keyboard-shortcuts>
- [110] J. Gosling, B. Joy, G. L. S. Jr., G. Bracha y A. Buckley, *The Java Language Specification*, Java SE 8 Edition. Redwood City, CA, USA: Oracle America, Inc., feb. de 2015. dirección: <https://docs.oracle.com/javase/specs/jls/se8/html/index.html>
- [111] *ISO/IEC 25002:2024 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Quality model overview and usage*, Geneva, Switzerland: International Organization for Standardization (ISO) e International Electrotechnical Commission (IEC), 2024. dirección: <https://www.iso.org/standard/78175.html>
- [112] A. Mishra, «Review: Software Quality Assurance—From Theory to Implementation,» *The Computer Journal*, vol. 47, n.º 6, págs. 728-728, 2004. DOI: 10.1093/comjnl/47.6.728
- [113] ISO/IEC/IEEE, *ISO/IEC/IEEE 15289:2019 Systems and software engineering — Content of life-cycle information items*, Geneva, Switzerland: ISO, IEC y IEEE, 2019.
- [114] R. P. L. Buse y W. R. Weimer, «Learning a Metric for Code Readability,» *IEEE Transactions on Software Engineering*, vol. 36, n.º 4, págs. 546-558, 2010. DOI: 10.1109/TSE.2009.70
- [115] S. Sentance, J. Waite y M. Kallia, «Teaching Computer Programming with PRIMM: A Sociocultural Perspective,» *Computer Science Education*, vol. 29, n.º 2–3, págs. 136-176, 2019. DOI: 10.1080/08993408.2019.1608781
- [116] E. Lahtinen, K. Ala-Mutka y H.-M. Järvinen, «A study of the difficulties of novice programmers,» *SIGCSE Bull.*, vol. 37, n.º 3, págs. 14-18, jun. de 2005, ISSN: 0097-8418. DOI: 10.1145/1151954.1067453
- [117] J. Bennedsen y M. E. Caspersen, «Failure rates in introductory programming,» *SIGCSE Bull.*, vol. 39, n.º 2, págs. 32-36, jun. de 2007, ISSN: 0097-8418. DOI: 10.1145/1272848.1272879

- [118] M. Alenezi, «Software and Security Engineering in Digital Transformation,» *arXiv preprint arXiv:2201.01359*, 2021, discusión sobre el rol crítico de la ingeniería de software en la transformación digital organizacional. DOI: 10.48550/arXiv.2201.01359 dirección: <https://arxiv.org/abs/2201.01359>
- [119] The Unicode Consortium, *The Unicode® Standard: Core Specification*, ver. 17, South San Francisco, CA, 2025. visitado 27 de ene. de 2026. dirección: <https://www.unicode.org/versions/Unicode17.0.0/core-spec/>
- [120] ISO/IEC. «ISO/IEC 646:1991: Information technology — ISO 7-bit coded character set for information interchange.» Confirmed and reviewed in 2020, visitado 25 de feb. de 2026. dirección: <https://www.iso.org/standard/4777.html>
- [121] F. Yergeau, «RFC 3629: UTF-8, a transformation format of ISO 10646,» RFC Editor, Internet Standard 3629, nov. de 2003. DOI: 10.17487/RFC3629 visitado 25 de feb. de 2026. dirección: <https://www.rfc-editor.org/rfc/rfc3629>
- [122] V. Cerf, *ASCII format for Network Interchange*, RFC - Internet Standard, ver. RFC 20, Superseded by INCITS 4-1986 (R2012), American National Standards Institute, 1969. dirección: <https://datatracker.ietf.org/doc/html/rfc20>
- [123] Microsoft. «Code Page Identifiers,» visitado 25 de feb. de 2026. dirección: <https://learn.microsoft.com/en-us/windows/win32/intl/code-page-identifiers>
- [124] P. Hoffman y F. Yergeau, «RFC 2781: UTF-16, an encoding of ISO 10646,» RFC Editor, Informational 2781, feb. de 2000. DOI: 10.17487/RFC2781 visitado 25 de feb. de 2026.
- [125] ISO/IEC, *Information technology — Universal coded character set (UCS)*, ISO/IEC, 2020. visitado 27 de ene. de 2026. dirección: <https://www.iso.org/standard/76835.html>